



Thursday, 30 April 2026

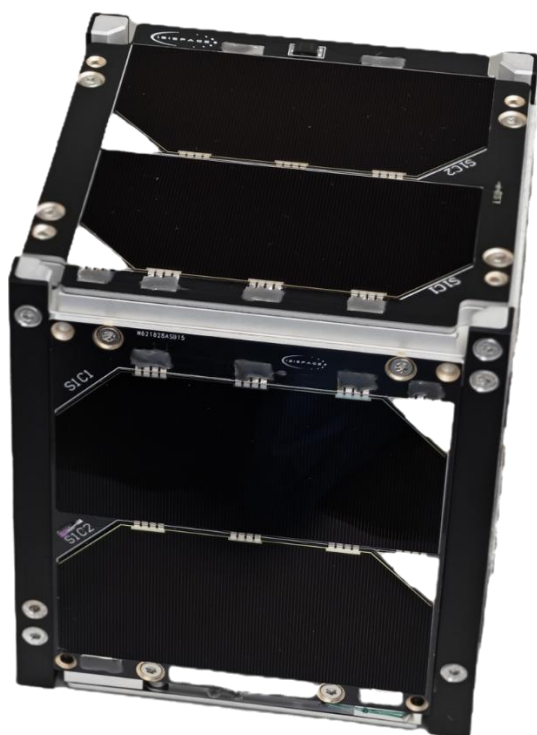
תשפ"ו

פרויקט גמר בסייבר ומערכות הפעלה

מגישה מעיין כרמי

ת"ז: 331826966

תיכון הנדסאים הרצליה



פניקס



HoopoeWorld

תוכן עניינים

4	פתיחה
5	תוכנה מוטסת ללוויין מסמך אפיון
6	ראשי תיבות
7	מבוא
7	תיאור כללי של הפרויקט
7	קהל היעד
7	סיבות לבחירה בפרויקט
7	מטרות הפרויקט
7	ייחודיות וחדשנות
8	מגבלות ופיתוח עתידי
8	היבטים טכניים
9	בדיקות המערכת
10	גנט
11	פירוט יכולות התוכנה
13	ארכיטקטורת הפרויקט
13	פירוט החומרה
14	תיאור טכנולוגי
14	תיאור זרימת המידע
16	תיאור אלגוריתמים
17	בעיית אלגוריתם
18	תיאור סביבת הפיתוח
19	פרוטוקול התקשורת
28	תיאור מבני נתונים
28	סקירת חולשות
29	HoopoeWorld מסמך אפיון
30	מבוא
30	תיאור כללי של הפרויקט
30	קהל היעד

30	סיבות לבחירה בפרויקט
30	מטרות הפרויקט
31	ייחודיות וחדשנות
31	מגבלות ופיתוח עתידי
31	היבטים טכניים
31	בדיקות המערכת
32	גנט
33	פירוט יכולות התוכנה
33	צד לקוח
34	צד שרת
35	ארכיטקטורת הפרויקט
35	פירוט חומרה
35	תיאור טכנולוגי
35	תיאור זרימת המידע
36	תיאור אלגוריתמים
37	תיאור סביבת הפיתוח
38	פרוטוקול תקשורת
42	תיאור מסכי המערכת
43	תיאור מבני הנתונים
46	סקירת חולשות ואיומים
47	תוכנה מוטסת ללוויין מסמך תוכנה
48	דרייברים מיובאים
50	"מחלקות" – קבצים בתוכנה המוטסת
50	src תחת תיקיית
57	src\SubSystemModules תחת
57	PowerManagement תחת
60	Maintenance תחת
63	Housekeeping תחת
65	Communication תחת
72	SubsystemCommands תחת

82	הבעיה האלגוריתמית
87	מסמך בדיקות
91	HoopoeWorld מסמך תוכנה
92	מודלים חיצוניים
94	מחלקות ופונקציות בתוכנה
94	תחת תיקיית webpage
97	קובץ createSQLTable.py
98	קובץ DecodeDataInPacket.py
103	קובץ server.py
104	קובץ ServerSetup.py
105	מסמך בדיקות
107	אלגוריתמים שאני מוצאת מעניינים
112	תוכנה מוטסת ללוויין עץ קבצים
115	HoopoeWorld עץ קבצים
117	HoopoeWorld מדריך משתמש
118	התקנה, הפעלה ותחזוקת השרת
118	הכנת סביבת העבודה
118	התקנה והרצה
118	הנחיות תחזוקה
120	שימוש בצד לקוח
122	רפלקציה
123	תודות
124	תמונות ממהלך הפיתוח של הלוויין
130	ביבליוגרפיה ונספחים

פתיחה

במסמך הזה יש 2 פרויקטים.

1. קוד תוכנה מוטסת של לווין

2. תוכנה לתחנת קרקע שנקראת HoopoeWorld

חילקתי את זה כך שיש מסמך אפיון לכל אחד אחד אחרי השני. המלצה שלי לקרוא קודם את ההסבר על התוכנה המוטסת מכיוון שאז ההסבר על התוכנה שמשתמשת בנתונים שנקלטים מהלווין תשמע יותר הגיוני.

תוכנה מוטסת ללווין מסמך אפיון



ראשי תיבות

DPS	– Down power safe mode
UC	– Up to cruise
DC	– Down to cruise
UO	– Up to operational
VHF	– Very high frequency (130 – 170 MHz)
UHF	– Ultra high frequency (400 – 450 MHz)
TRXVU	– Transceiver, UHF Transmitter and VHF Receiver
EPS	– Electric power system
OBC	– On board computer
I ² C	– Inter integrated circuit communication bus.
ANTS	– Antennas System
AX.25	– Amateur X.25
FS	– File system
RTC	– Real time clock
RFCB	– Radio frequency communication branch
SP	– Solar Panels
WOD	– Whole Orbit Data
SPL	– Satellite Packet Link

מבוא

תיאור כללי של הפרויקט

הפרויקט הוא תוכנה מוטסט ללוויין. הלוויין היינו לוויין מסוג ננו לוויין בגודל 1CubeSat שזו יחידת מידה של $10.8\text{cm} \times 10\text{cm} \times 10\text{cm}$. התוכנה כתובה בשפת C ויועדה לשימוש של המרכז המדעים שלנו לניהול ולחקר.

קהל היעד

קהל היעד העיקרי הוא:

1. **האנשים בתוכנית שלי** (וגם אלו שיצטרפו אליה) – שיוכלו להשתמש בלוויין שכתבתי לו את הקוד, וגם בקוד הלוויין שהוספתי לו הערות כדי שיהיה אפשר לשפר אותו ולהוסיף לו דברים אחרי בלי הצורך להתחיל את הכל מחדש כמו שקרה עד כה.
2. **חובבי רדיו מרחבי העולם** – שיוכלו להשתמש במשיב שעל הלוויין, שהוא אמצעי שמעביר מידע דרך הלוויין לנקודות שונות ברחבי העולם, ובכך יוכלו לתקשר דרכו.

סיבות לבחירה בפרויקט

אני חושבת שיותר נכון לשאול למה הצטרפתי לתוכנית מכיוון שכבר שהצטרפתי ידעתי שאני ככל הנראה יהיה מפתחת של לוויין. במהלך התוכנית גם ראיתי את האהבה שלי לקוד של מערכת זאת אומרת לא במדויק ללוויין אבל בכללי לתכנת מוצר מאשר תוכנה.

עוד אחת הסיבות שהיא תהיה נכונה גם תוכנית השנייה היא שאני אוהבת ליצור דברים שבאמת ישתמשו בהם ויש בהם צורך. ואני יודעת שבלוויין ישתמשו גם אם לא אנחנו בתוכנית יהיה מקום שישתמש בו.

מטרות הפרויקט

- חקר החלל והקרינה.
- מתן שירות לחובבי הרדיו ברחבי העולם.
- חקר רכיבי הלוויין, התוכנה ותפקודם בפועל.

ייחודיות וחדשנות

בעוד שתוכנת לוויין כבר קיימת הייתי צריכה ליצור חדשה עבור הלוויין שלנו. במהלך התקופה לא הייתה תוכנה ברורה שמתאימה בדיוק ללויינים שלנו אז יצרתי אחת עבורם. התוכנה הזאת נמצאת על 4 מתוך 9 לויינים שכרגע נמצאים בחלל.

לקחתי השראה מלוגיקה של לוויינים קודמים של התוכנית ומהקוד שסופק עם הדרייברים (שמסביר איך להשתמש בהם) אבל חלק מהלוגיקה גם ראיתי דרך לשפר אותה אז שיניתי אותה שתהיה למשהו יותר מתאים ויעיל.

מגבלות ופיתוח עתידי

תוכנת הלוויין מאוד מוגבלת לחומרה הספציפית שלו וכאשר נעבור חומרה נצטרך לשנות חלק מהקוד בין אם ה-drivers משתמשים בהם או חלק מהלוגיקה כמו משימת הלוויין שמשתנה. אבל רוב הלוגיקה תישאר אותו הדבר רק צריך להבין איך לשנות את התוכנה כדי להתאים.

זוהי תוכנה שהיא embedded. צריך שתהיה אפשרות לצרוב אותה כדי שתוכל לרוץ כל הזמן וגם היא בסופו של דבר מנהלת את הלוויין.

השינויים בקוד לא חייבים להיות דרסטיים וכתבתי מספיק הערות כדי שאנשים בהמשך ידעו מאיפה להמשיך את מה שעשיתי ויוכלו להבין מה עשיתי. ובו בזמן אני כבר מלמדת את הדור המשך של התוכנית את התוכנה כך שהם ידעו איך להמשיך אותה גם אם לא היו הערות (אבל תמיד טוב שיהיו).

היבטים טכניים

הפרויקט של הלוויין:

- פיתוח חלק ממערכת הפעלה בזמן אמת (Real-Time) בשפת C עבור הלוויין (התוכנה צרובה עליו).
- פיתוח מערכת הקבצים של הלוויין (וגם כל שאר המערכות).
- כתיבה ויישום של פרוטוקול תקשורת ייעודי.

לא כולל (אך נכלל בתוכנה השנייה):

- עבודה עם מבני נתונים
- שימוש בפרוטוקול תקשורת מוצפן – התקשורת של הלוויין לא יכולה להיות מוצפנת מכיוון שמתוך רישיון חובבות הרדיו אסור לנו לשדר בתדרים שלהם באופן מוצפן.
- מערכת שרת ולקוח – הלוויין הוא יותר גוף שעובד עצמאית אבל הוא גם סוג של שרת שתחנות קרקע ברחבי העולם יוכלו לקבל ממנו נתונים אבל רק מי שאם הפיענוח הנכון יוכל לדעת מה כתוב ומה לשלוח מכיוון שהתקשורת ב-Hexa.

בדיקות המערכת

מערכת הלוויין כבר מזמן יצאה מאזור שאפשר לבדוק ולתקן את הלוויין אבל אפשר לתקן את התוכנה לפעם הבאה וזה כבר בתהליך או נעשה.

הבדיקות לפני השיגור כללו:

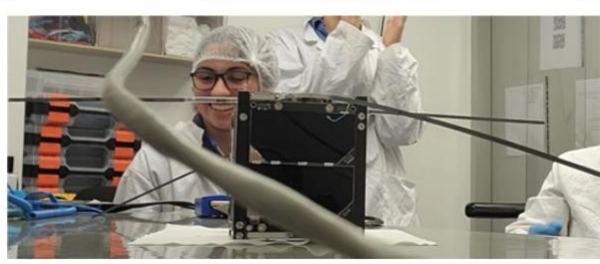
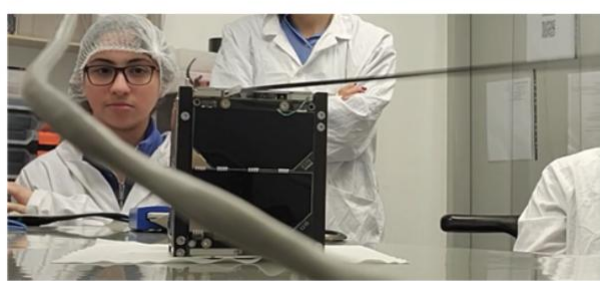
- הרצה של כל הפקודות של הלוויין.
- הרצת בדיקות קצה של פקודות לוויין.
- בדיקה של ריצה של הלוויין.
- בדיקת ניהול הקבצים וניהול מחיקתם.
- פריסת אנטנות – אחד מהחשובים ביותר. אם לא עובד הלוויין לא יוכל לשדר.
- לתת לו לרוץ כמות רבה של זמן בלי הפרעות
- בדיקת התממשקות עם התוכנה שלנו בתחנת הקרקע.
- בדיקת מעבר דרייברים. כל הקוד נבדק על הדרייברים של הלוויין הפרוס אך כאשר הגיע הלוויין הייתי צריכה לשנות את הקוד ולוודא שהכל עדיין עובד.

היו עוד בדיקות שהייתי רוצה לעשות אבל תאריך השיגור כבר הגיע אז הייתי צריכה לשלוח אותו אם רק בדיקות מינימום אפילו שהייתי רוצה יותר זמן. עבור בדיקות נוספות.

אלו שתי תמונות מסרטון שצילמנו בעת פריסת האנטנות.

כמו שאפשר לראות זאת הייתה הקלה מאוד גדולה כאשר זה עבד ועבר חלק מכיוון שזהו החלק שהכי מפחיד למתכנת כאשר הוא מתכנת לוויין.

האנטנות מאוד עדינות ואין לנו ספייר, גם ממה ששומר עליהם היה מספיק רק ל-2 פריסות – אחת במעבדה ואחת בחלל. אז אם זה לא היה עובד זאת הייתה יכולה להיות קטסטרופה.



גנט

עבר זמן מאז שעשיתי את פרויקט זה אז אני כותבת את ההליך אליו. עד קבלת הלוויין בערך עבדתי עם דרייברים ישנים שעובדים על הלוויין שיש לנו במעבדה אבל הוא לא הלוויין שעלה לחלל. רק אחרי קבלת הלוויין (או בערך שבועיים לפני קבלתו) יכולתי להחליף את הדרייברים.

הגנט מורכב מבערך מה שהיה בפועל, אבל בעיקרון תמיד היו בדיקות לקוד וכשנמצאו שגיאות תיקנתי אותם. התוכנה לא הייתה גמורה עד שהטסנו את הלוויין לארה"ב מכיוון שהיה מידע חדש שקיבלנו על תקלה ברכיב אז ניסיתי לשנות את הקוד כדי שיתאים לבעיה. וכמובן עד הרגע האחרון (לפני הטענת הבטריות) בדקתי את קוד הלוויין.

שלב	תאריך – סיום
לוגיקת לוויין (הבנה וכתיבה)	דצמבר 2023
הבנת כתיבת תוכנת לוויין (גם C נכלל פה אפילו שנלמד גם תוך כדי)	פברואר 2024
כתיבת demo – קוד בדיקות ללוויין שעוזר להבין איך לכתוב לוויין	יוני 2024
כתיבת תוכנת ה-Init System (פעולות האתחול וקוד הפריסה של האנטנות – בלי לבדוק את הפריסה) TRXVU (מערכת התקשורת) עם הדרייברים הישנים (כולל פקודות ה-TRXVU)	אמצע ספטמבר 2024
Management – קוד של ניהול כללי של הלוויין	אמצע אוקטובר 2024
EPS – מערכת החשמל	תחילת נובמבר 2024
קבלת הלוויין והתחלת החלפת דרייברים	תחילת דצמבר 2024
Payload-ו File System	סוף דצמבר 2024
תיקונים אחרונים ופריסת אנטנות	תחילת ינואר 2025

פירוט יכולות התוכנה

הלוויין הוא "שרת" שניתן "להתחבר אליו" רק במובן הבא:

תחנת הקרקע משדרת פקודה, ואם הלוויין נמצא בקשר) מיקום נכון, אנטנה מופנית- (הוא קולט את האות, מבצע או דוחה פקודה, ומשדר בחזרה תגובה.

כל המערכות הפנימיות להלן כתובות בשפת C, לרוב תוך שימוש בדרייברים ייעודיים, enums, מבני נתונים

קוד הלוויין הוא רק צד שרת שאפשר "להתחבר" אליו באמצעות תחנות קרקע. למה להתחבר בגרשיים? מכיוון שזה יותר ששולחים מתחנת הקרקע ומקווים שהלוויין יקלוט (זאת אומרת שאנחנו עוקבים אחריו נכון) ונקבל תגובה שאנחנו הצלחנו לקלוט.

1. Beacon

a. שידור בפרק זמן ידוע אות חיים המכיל מידע קריטי על הלוויין:

- i. מצב מערכת החשמל: מתח סוללה (mV), זרם, טמפרטורת תאים
 - ii. מידע כללי: הזמן בלוויין, מספר אתחולים יזומים, מספר אתחולים כוללים
 - iii. נתוני payload
 - iv. מידע תפעולי נוסף הנאסף ממודולים פנימיים
- b. נדרש:
- i. פונקציה שבודקת מעבר זמן
 - ii. איסוף נתונים מהדרייברים
 - iii. בניית Packet בהתאם לפרוטוקול שנקבע
 - iv. שידור דרך מודול התקשורת
- c. נחוץ:

- i. מבנה נתונים union שבתוכו מערך מסוג struct-י char שמכיל את תוכן ה-beacon.
- ii. דרייברים של קריאה מהמערכות ושליחה ב-TRXVU.
- iii. הכרת המערכת (זה בהכול אחרת לא היה אפשרי לעשות את הדברים)

2. Transponder

a. מצב שבו הלוויין הופך ל"מראת רדיו": תחנת אחרת → לוויין → תחנה רחוקה. הלוויין מפעיל/מכבה את המשיב לפי פקודות שמתקבלות מהקרקע (פקודת הפעלה כוללת כמות זמן).

b. נדרש:

- i. קבלת פקודה "הפעל משיב" או "כבה משיב"
- ii. במצב הפעלה: שמירת זמן פעולה (Duration)
- iii. מנגנון Timeout שמכבה אוטומטית

c. נחוץ:

- i. enum לכדי להחליף בין מצב משדר לבין מצב משיב.
- ii. דרייברים ל-FRAM, ול-TRXVU
- iii. תחנת קרקע פעילה

3. Payload

- a. חלקי מדידות שהלווין אוסף לפי תזמונים שונים (כל 10 שניות אחד ו-15 דקות אחר) ונשמרים ונשלחים לניתוח מדעי. המדידות הן בקשר לקרינה.
- b. נדרש:
 - i. יצירת קבצי payload במערכת הקבצים.
 - ii. קריאה תקופתית של חיישנים
 - iii. הפעלת דרייבר ייעודי שפותח עבור המשימה (TAU)
- c. נחוץ:
 - i. union שבו אנחנו שומרים את המידע
 - ii. Timestamp – שנדע מתי נשמר
 - iii. דרייבר של payload.

4. פקודות לווין

- a. רשימת פקודות קבועה מראש שהלווין יודע לפרש. לכל מערכת בלווין ישנה מספר פקודות שאפשר לבצע עבורה. חלק מהפקודות הן ניהול/בקרה, חלק קבלת מידע נוסף, חלק שינוי ערכי משתנים. לדוגמה ביצוע dump ששייך למערכת הקבצים, reset ששייך למערכת הניהול ועוד.
- b. נדרש:
 - i. דרך לפרש את הפקודות
 - ii. מזהה פקודה (יש מספר enums לכך)
 - iii. switch case עבור כל פקודה
 - iv. בקרת שגיאת opcode ומערכת
- c. נחוץ:
 - i. פונקציות ייעודיות
 - ii. תיעוד עבור משתמשי התחנה כדי שיוכלו להשתמש בלווין.

5. מערכת הקבצים

- a. מערכת קבצים פנימית המבוססת FAT32 - דרייבר שמטפל בגישה לקבצים, יצירה, כתיבה ושמירה. התוכנה המוטסת של הלווין בונה אוטומטית את שמות הקבצים בהתאם ללוגיקה המוגדרת ב-TLM_management וגם שומרת, משנה ומוחקת את הקבצים בהתאם.
- b. נדרש:
 - i. דרייבר FAT32
 - ii. פרוטוקול יצירת שמות קבצים וסיימות.
 - iii. ניהול טלמטריה
- c. נחוץ:
 - i. מבני נתונים
 - ii. פונקציות מדרייבר FAT32
 - iii. תיעוד טוב

6. הפעלה עצמית

- a. מנגנון פנימי שמנתר מצב הלווין (לדוגמה - חוסר תקשורת לאורך זמן) ומבצע אתחול אוטומטי לשחזור פעיל. אבל גם הפעלה כללית עצמאית של המערכות כך שהכל פועל גם אם לא מתקשרים לאורך זמן.

ארכיטקטורת הפרויקט

פירוט החומרה

כל הרכיבים שאכתוב פה הם של חברה בשם ISISPACE חוץ מה-payload שהוא של שורק

OBC – המחשב המוטס.

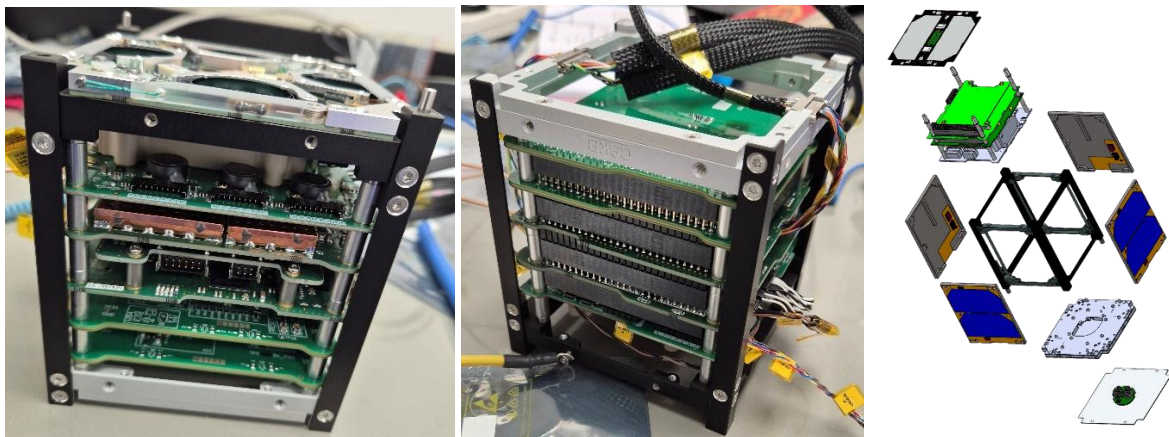
- מחשב בעל ליבה 1 במערכת 32bit.
- המערכת הפועלת עליה היא FreeRTOS והתוכנה שאני כותבת.
- יש עליו גם 2 כרטיסי SD של 2GB שעובד במערכת FAT32.
- חוץ מזה יש שם RAM, FRAM, NORFLASH, supervisor, RTC...
- חיבור שנקרא PC-104 שבאמצעותו מתחברים לרכיבים האחרים.
- לכאן אני כותבת את כל הקוד והוא מתחבר לרכיבים האחרים באחת מדרכי התקשורת (הרוב 1°C) וזה מפעיל את הפקודה שאני כותבת מהפונקציות שנתונות.

EPS – מערכת החשמל

- המערכת כוללת בתוכה גם את בטריות
- חיבור לפאנלים הסולאריים (אני לרוב מחשיבה גם את הפאנלים כחלק מהמערכת אבל זה רכיב אחר)
- מנהלת את יציאת החשמל לרכיבים השונים.

TRXVU – מערכת התקשורת

- כוללת מקלט, משדר ומשיב (שהוא אותו הרכיב כמו המשדר).
- אני מחשיבה גם את האנטנות כחלק מהמערכת שכמו הפאנלים הם רכיבים שונים.
- האנטנות מחוברות ל-TRXVU בכבלים קואקסיאליים.



הקרדיט לשרטוט הולך בכולו לרותם אור שהיה עד שנה שעברה חלק מצוות מכניקה של התוכנית. (היינו כ-5 אנשים שהרכיבו את הלוויין שאחת מהן הייתה אני ומתכנתת אחת שהיא גם כן אני)

תיאור טכנולוגי

- שפת תכנות הלוויין היא שפת C כמו שנאמר, אבל את הפקודות שלחתי ללוויין באמצעות Hexa.
- ה-OS כפי שכבר אמרתי הוא FreeRTOS אבל הקוד שכתבתי משתלב שם כך שהוא בעצם נחשב כחלק ממערכת ההפעלה.
- פרוטוקול התקשורת בין הרכיבים הוא I²C או SPI ופרוטוקול התקשורת עם הקרקע הוא פרוטוקול Ax25. פרוטוקול Ax25 הוא פרוטוקול של שכבת הרשת כך שאין ממש מה להגיד על השכבות מעל.

תיאור זרימת המידע

נתחיל עם מבנה ה-packet, מדובר כאן על המבנה בתוך חלק ה-data של ה-Ax25. אז בעיקרון זה פרוטוקול השליחה של ה-packet:

ID 4 bytes	type byte	subtype byte	data Length 2 byte	data 0 - 240 byte
----------------------	---------------------	------------------------	------------------------------	-----------------------------

ID – מייצג גם איזה לוויין זה מתוך כל הלוויינים שאנחנו יכולים לקלוט במבנה הזה וגם את מספר הפקודה אחרי ההפעלה.

type – מייצג את סוג המערכת. FS, EPS, TRXVU וכדומה.

subtype – זה בעצם מספר הפקודה בתוך הקטגוריה. למשל יכול להיות פעמיים 00 subtype אבל אחד מ type 1 והשני מ type 4.

data length – אורך ה-data שרוצים לשלוח בלי ה-headers שהוספתי. מכיוון שגם אם יש באפר אז הפאקטה תמיד בגודל קבוע אנחנו רוצים לדעת עד איפה אנחנו צריכים לקרוא את המידע.

Data – המידע. יהיה כאן את הפרמטרים של קבלת הפקודה ללוויין או שליחת הפקודה מהלוויין.

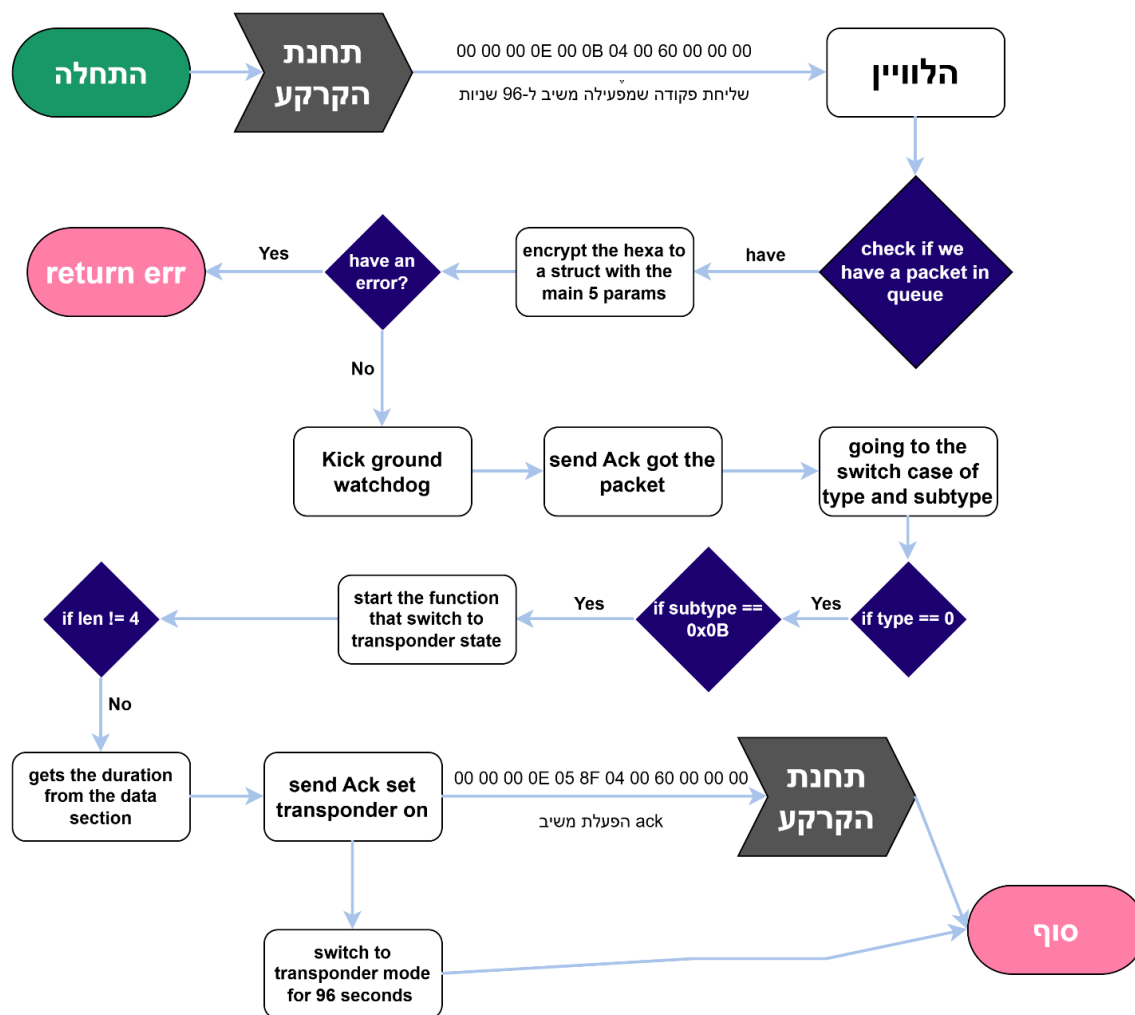
אני אציג את זרימת המידע עם דוגמא של פקודה שאפשר לשלוח ללוויין. וכדי להכניס 2 יכולות ביחד אני אציג את האחת של ה-Transponder. אני גם מתארת אז זה עבור לוויין מספר 14.

השליחה מהקרקע ללוויין וגם הפוך נעשית ב-little endian כל פרמטר לא כל ה-packet ביחד. כך שלמשל int יתהפך כדי שיהיה אפשר לקרוא אותו אבל אם יש אחריו מחרוזת היא לא תתהפך מכיוון שזה בעצם מערך של chars והם מתהפכים אבל בגלל שהם עם 1 byte זה נראה כאילו לא.

אני לא מכניסה את כל הלוגיקה מכיוון שזה יהיה יותר מדי ארוך אז אני רק כתבתי מה קורה אם הכל עובד כמו שצריך אפילו שיש בדיקות קצה בכל מקום.

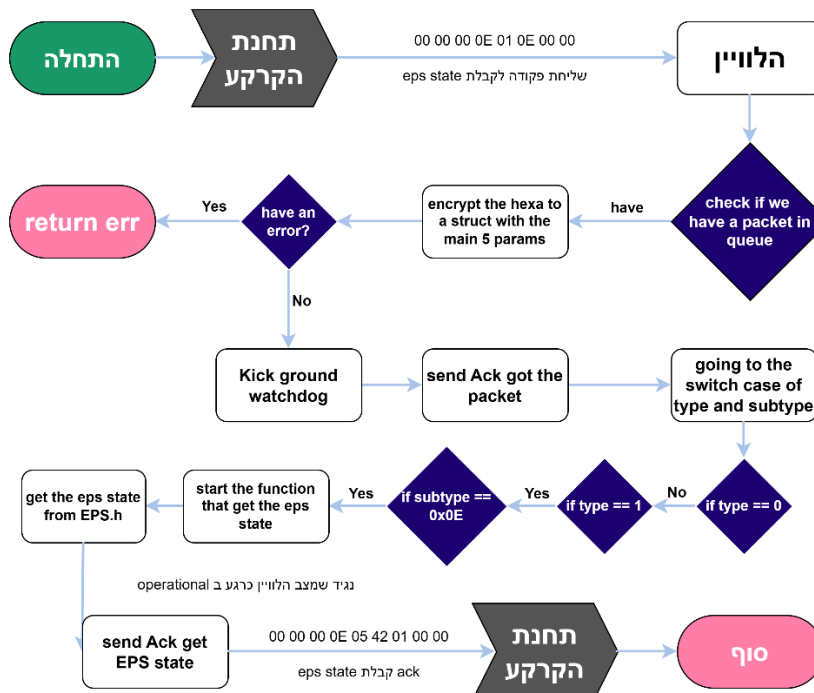
האלגוריתם של היכולת נמצא בשקופית הבאה.

לא כתוב כפונקציות מכיוון שזה יהיה פחות מובן אז הכל במילים. האלגוריתם מייצג שליחה של בקשה להפעלת משיב ל-96 שניות.



החצי הראשון של האלגוריתם נכון בעיקרון לכל פקודה שנשלח. מה שמשתנה הוא אחרי בדיקת איזה type ו-subtype הפקודה.

דוגמא נוספת: נשלחת פקודת קבלת eps status.



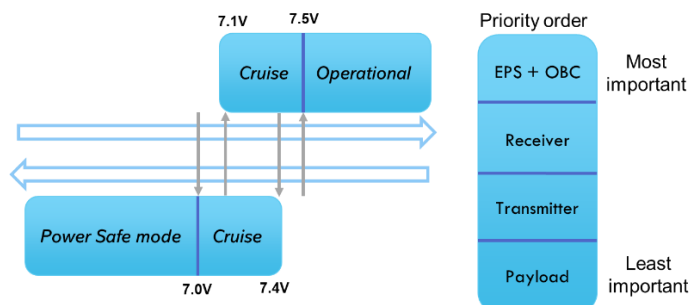
תיאור אלגוריתמים

זהו האלגוריתם הראשי של התוכנית שנמצא בתוך קובץ main.c.

1. Bootup – מדבר על הפעלה ראשונית של המערכת או על אתחול. אחרי אתחול וגם בהפעלה התוכנה ישירות מתחילה לרוץ.

2. Init system – זוהי פונקציה שנמצאת בקובץ אחר שנקרא InitSystem.c. פקודה זו אחראית על לקשר את כל הרכיבים של המערכת (בין עם על ה-OBC או רכיב נפרד) כך שהתוכנה תוכל להשתמש בהם במהלך הקוד. בפקודה זאת יש גם את ההפעלה הראשונית. לפני השיגור לחלל מאתחלים מספר פרמטרים ואחד מהם הוא הפרמטר שזו הפעלה ראשונית ונצטרך לפרוס אנטנות. כאשר הלוויין נמצא בתהליך פריסת אנטנות הוא גם שומר טלמטריה, סופר זמן עד הפריסה ולפני הכל קודם כל מאפס לערכי ה-default את כל הערכים הדינאמיים.

3. EPS Conditioning – לוגיקת ה-EPS קובעת את מצב הלוויין לפי כמות הסוללה (יהיה טבלה למטה של הספים). יש 3 מצבים ו-4 ספים. יש בדיקות למעברים כדי שלא יהיה מצב שאנחנו כל הזמן זזים בין 2 מצבים אלה יש שינוי.

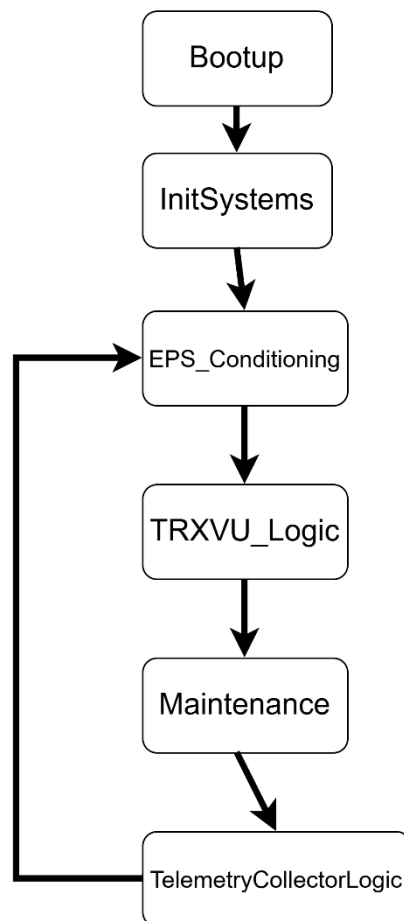


4. TRXVU Logic – כאן מתרחשת כל לוגיקת התקשורת. בין אם שליחת beacon, קבלת פקודות, תשובות לפקודות וכדומה. לחלק הזה צריך את כל הקבצים שקיימים בתיקייה של ה-TRXVU אבל הפקודה עצמה נמצאת ב-TRXVU.c. הוא בודק אם צריך לשדר ביקון ואם כן משדר, בודק האם קיבלנו פקודה ואם כן מטפל בה וכדומה.

5. Maintenance – פונקציה זו שנמצאת בפונקציית Maintenance.c עושה מספר דברים.

- בודקת מהו זמן הלוויין הכי מעודכן ושומרת אותו על ה-FRAM. למקרה של אתחול שיהיה לנו זמן מעודכן כמה שאפשר.

- בודקת אם יש צורך למחוק קבצים בגלל חוסר מקום. אם יש צורך היא מוחקת את 5% הישנים יותר. (המערכת שומרת תמיד 20% מקום פנוי)
- לאחר מכן מכיוון שה-payload הוא רכיב שלא נוסה לפני בחלל יש מנגנון fail check שאם הלוויין חווה reset פעמיים ברציפות במהלך פחות מדקה ה-payload נכבה לאלתר עד הפעלה מחודשת דרך פקודה מתחנת הקרקע.

main algorithm

- לבסוף נבדק האם אנחנו צריכים לאתחל את הלווין בגלל חוסר תקשורת ממושך עם תחנת הקרקע. אם כן מבצעים מספר פקודות ואז עושים hard reset ללווין.

6. TelemetryCollectorLogic – לוגיקת שמירת מידע במערכת הקבצים. הפונקציה נמצאת ב-TelemetryCollector.c והיא אחראית על שמירת ה-telemetry של כל המערכות. היא בודקת קודם כל האם עבר מספיק זמן מהשמירה האחרונה. אם כן היא לוקחת את ה-data ומכניסה אותו למבנה הנתונים המתאים. המבנה נשלח לפונקציה שכותבת לכרטיס ושם נעשה:

- בודקים מה הזמן הנוכחי.
- משנים את הדרך שהזמן מוצג. (מייצוג של שניות מ-1970 לייצוג רגיל של תאריך ושעה ב-struct)
- נכנסים לפונקציה עם סוג הטלמטריה, ויוצאים עם הסוג שלה, גודל הטלמטריה שרוצים לשמור.
- אחר כך יוצרים את שם הקובץ באמצעות התאריך והסיומת
- מנסים לפתוח את הקובץ ולכתוב לו. אם הוא לא קיים הוא נוצר ואם פשוט אי אפשר הטלמטריה לא נשמרת ויוצאים מהפונקציה.
- לבסוף כותבים לקובץ את המידע יחד עם הזמן שהמידע נלקח בו (כמובן בסוף הכל נשלח ב-hex)
- וסוגרים את הקובץ.

אחרי כל אלו אנחנו חוזרים להתחלה ומבצעים את כל הלולאה שוב. במקרה של reset חוזרים ממש להתחלה ל-Bootup.

בעיית אלגוריתם

בפונקציית ה-Maintenance כמו שאמרתי יש אלגוריתם למחיקת הקבצים. במקור חשבתי לעשות את האלגוריתם הזה כמו בלוויינים הקודמים, אך כאשר ראיתי כמה מקום בממוצע נתפס ביום, אחרי ביצוע חישוב מקורב באמצעות גודל המידע, כל כמה זמן הוא נשאר וניחוש ל-log, רציתי לשנות אותו.

חלק מהלוגיקה הכללית מהלוויינים הקודמים של התוכנית נשמרה כך שאולי לא היה קוד לקחת ממנו השראה אבל כן ללוגיקה.

בעבר הלוגיקה של אותה הפונקציה הייתה מחיקה של מסמכים שעברו 30 יום. זאת אומרת שאחרי כחודש הטלמטריה תעלה אם לא הורדנו אותה לתחנה.

בעוד שבלוגיקה עכשיו הלווין לא מוחק את הטלמטריה שהוא שומר כשנה, מכיוון שאחרי החישוב יצא שהלווין יכול לשמור בכרטיס SD 1 סביבות השנה וחצי של מידע ואני עשיתי כך שבאודאט לא נגיע ל-0 מקום פנוי אז אחרי שהגענו ל-80% תפוס מפנים את המסמכים הישנים ביותר כך שרק 75% יהיה תפוס.

הפונקציה הזאת נקראת DeleteOldFiles והיא נמצאת במיקום הבא בתוך הפרויקט.

src\SubSystemModles\Maintenance\Maintenance.c

יש לזה גם את קובץ ה-h. באותו המקום.

תיאור סביבת הפיתוח

פיתוח קוד הלוויין מתבצע בסביבת עבודה ייעודית המבוססת על **Eclipse Kepler (2014)**. גרסה זו מותאמת למערכות Embedded ולדרייברים של הלוויין, ולכן מאפשרת שימוש מלא בכל הכלים והיכולות הנדרשים עבור הפיתוח.

בדיקות הלוויין מתבצעות על גבי מספר פלטפורמות שונות — חלקן תוכנות תקשורת, חלקן כלים לצריבה וחלקן מערכות ייעודיות שנבנו עבור התוכנית. להלן פירוט כלי העבודה העיקריים:

1. PuTTY – תקשורת טורית ובדיקות בסיסיות

בזמן שהלוויין נמצא אצלנו בקרקע, חלק גדול מהבדיקות מתבצע באמצעות חיבור ישיר (Serial) למחשב:

- ניתן לבצע בדיקות שגרתיות, להדפיס הודעות Debug ולשלוח פקודות.
- הפקודות ללוויין נשלחות מתוכנת בדיקה אחרת, אך התוצאות מופיעות גם ב-PuTTY. בעוד שבתוכנת הבדיקות הכל (חוץ מבדיקת שידור) נעשה ב-PuTTY.
- החיבור מתבצע דרך קטגוריית Serial, כאשר מהירות התקשורת משתנה בהתאם לסוג התוכנה שמורצת בלוויין:
 - i. Speed 2000000 – לתוכנת לוויין מוטסת
 - ii. Speed 115200 – לתוכנת ה-demo בה יש את הבדיקות השגרתיות.

2. RFCB ISISPACE – מערכת לבדיקת תקשורת רדיו

זהו רכיב חומרה חיצוני המתחבר למחשב ומשמש בעיקר לבדיקת מערכת התקשורת של הלוויין:

- מאפשר לשלוח פקודות ללוויין ב-Hexa בתדר הנכון (ב-VHF).
- מאפשר לקבל מהלוויין את ההודעות שהוא משדר בתדר החוזר (ב-UHF).
- משמש לבדיקה ראשונית של תקינות מערך התקשורת.
- בשלב זה אין ממשק משתמש גרפי נוח, ולכן נדרש פיענוח ידני של הודעות Hexa לפי פרוטוקול התקשורת, שיצרת. כך שאחרי מספיק פעמים ידעתי כבר לפענח את ה-Hexa ללא מפענח או הפרוטוקול.

3. SAM-BA – כלי הצריבה הרשמי

Samba (SAM-BA) משמש לצריבת הקוד למעבד של הלוויין:

- לאחר קומפילציה, הקוד נכתב לתוך זיכרון ה-NOR Flash.
- לצורך בדיקות שאמורות לשרוד Reset, חייבים לצרוב ולא רק לטעון ל-RAM.
- במהלך הפיתוח בוצעו מספר צריבות, אך האחרונה שבהן היא הצריבה הסופית ללוויין, יום לפני שליחתו לארה"ב.

4. HoopoeNest – ממשק משתמש פנימי לשליחת פקודות

זהו אחד הכלים החשובים ביותר כיום לצוות תקשורת והוא נוצר על ידי מספר תלמידים בתוכנית עבודה.

- זהו ה-GUI בתחנת הקרקע.
- מאפשר לשלוח פקודות ולפרש תשובות ללא צורך בכתיבה ידנית ב-Hexa או קריאת ה-Hexa.
- משמש גם בבדיקות קרקע של הלוויין כדי לוודא שניתן לקלוט נתונים דרך תחנת הקרקע שלנו.
- מפשט מאוד את תהליך התפעול והבדיקה.

5. ציוד תקשורת ותוכנות עזר

בנוסף, נדרש ציוד תקשורת ותוכנות תקשורת חיצוניות המשמשות להאזנה, ניתוח, מעקב ושידור. דוגמאות למספר מתוכנות אלו:

- SatPC32 — מעקב אחרי מסלול הלוויין ואנטנות
- HDSDR — תוכנת SDR לקבלת אותות מהתדר, מהלוויין גם כן אבל הסינון הוא בתוכנה של soundmodem היא לא מראה שום דבר חוץ מאת מה שאמרנו שאפשר.

- Soundmodem — מודם תוכנה בו קולטים את ה-packets מהלוויינים.

6. הלוויין עצמו – סביבת הבדיקה החשובה ביותר (כי בלעדיו אין לוויין)

למרות שניתן לכתוב ולקמפל, אין אפשרות לבצע בדיקה עמוקה ואמיתית ללא הרצה על גבי הלוויין עצמו.

במסגרת העבודה על תוכנת הלוויין אני השתמשתי ב-Flatsat – לוויין פרוס שנמצא בחדר הנקי ומשמש כסביבת הבדיקות המרכזית לפני הגעת הלוויין שישוגר לחלל. ה-Flatsat כולל את רוב רכיבי החומרה, המודולים והחיבורים, כך שאפשר להריץ עליו כמעט את כל המערכות כפי שיפעלו בפועל. (מכיוון שהוא לוויין שפשוט לא עמד בקריטריון של שיגור לחלל אבל אלו רכיבי לוויין אמיתיים).

עם זאת, החומרה בלוויין הסופי שונה, ולכן בעת המעבר ללוויין האמיתי נדרשים עדכונים בדרייברים והתאמות נוספות — אך **מרבית הקוד כבר כתוב ומוכן**.

7. העבודה בחדר נקי נדרשת כדי לספק סביבת עבודה סטרילית ובטוחה ללוויין ולמנוע זיהומים, פגיעות סטטיות ונזקים אחרים.

לסיכום: הפיתוח מתבצע ב-Eclipse Kepler, אך הבדיקות הן אלו שמאפשרות לאמת את התנהגות הלוויין במציאות – נעשות רק על גבי ה-Flatsat ועל הלוויין עצמו כאשר הוא מגיע והסביבות תומכות בכך (רואות את התוצאות/שולחות לו וכדומה).

פרוטוקול התקשורת

פרוטוקול התקשורת שהלוויין משדר בו הוא Ax25. שזהו פרוטוקול תקשורת של חובבי רדיו בתחום של חובבי רדיו.

אך הפרוטוקול שכתבנו בתוכנית הוא באמת מה שמשתמשים בתוכנה מכיוון שכאשר אני מפענחת את הקוד בלוויין שאני מקבלת מה-driver אני מקבלת רק את החלק של ה-data בפרוטוקול שם אנחנו מכניסים את כל הלוגיקה שלנו כמו שהוצג לעיל, (אך אני אכניס את המבנה גם כאן).

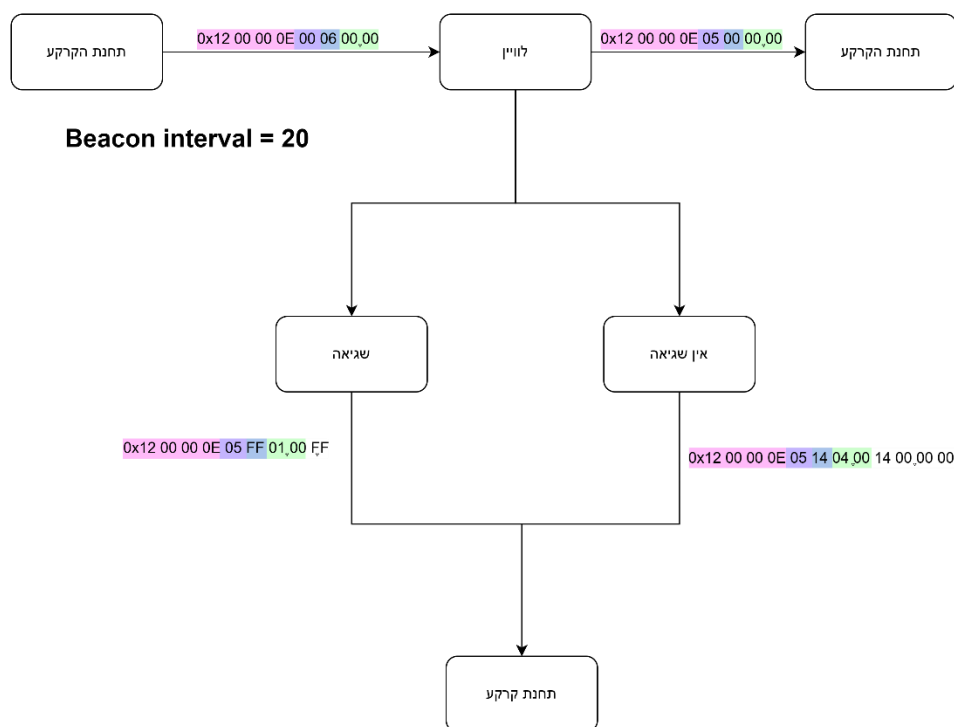
ID 4 bytes	type byte	subtype byte	data Length 2 byte	data 0 - 240 byte
----------------------	---------------------	------------------------	------------------------------	-----------------------------

כרגע אני אפרט על 3 פקודות, 1 שנשלחת מהלוויין לקרקע ו-2 מהקרקע ללוויין ופקודת החזרה שלהן. הפירוש של כל פקודה יהיה למטה בפרוטוקול, אך גם שם יהיו דברים בנספחים.

בנוסף ה-ID של כל לוויין שונה כך שאני כרגע מציגה ללוויין 14, אבל הקוד פועל גם על 11, 16, 19.

1. שליחת ack אחרי reset. (מהלוויין לקרקע)
 0x00 00 00 0E 05 7F 04 00 69 29 D1 4D
 a. ID – 00 00 00 0E. שזה מפורש הפוך אבל זה בעצם מראה שזו הפקודה הראשונה שנשלחה מהלוויין לקרקע וגם את זה שאנחנו על לוויין שהמזהה שלו הוא 14 (0E)
 b. Type – 05. זהו סוג של ack, כל פעם שנשלח ack מהלוויין זה בפורמט הזה. (רק הלוויין יכול לשלוח סוג כזה).
 c. Subtype – 7F. מייצג בפרוטוקול ack מסוג של הדלקה אחרי אתחול.
 d. Length – 04 00. נשלח ב-little endian וצריך לתרגם אותו ככזה. אורך ה-data שיבוא אחרי.
 e. Data – 4D D1 29 69. שוב כמו הכל הוא ב-little endian (ב-big: 69 29 D1 4D) מייצג את המספר 1764348237 שזה הזמן שעבר מאז 00:00:00 1.1.1970 בשניות (unix time). מכיוון שהפקודה מחזירה את הזמן על הלוויין לקרקע.

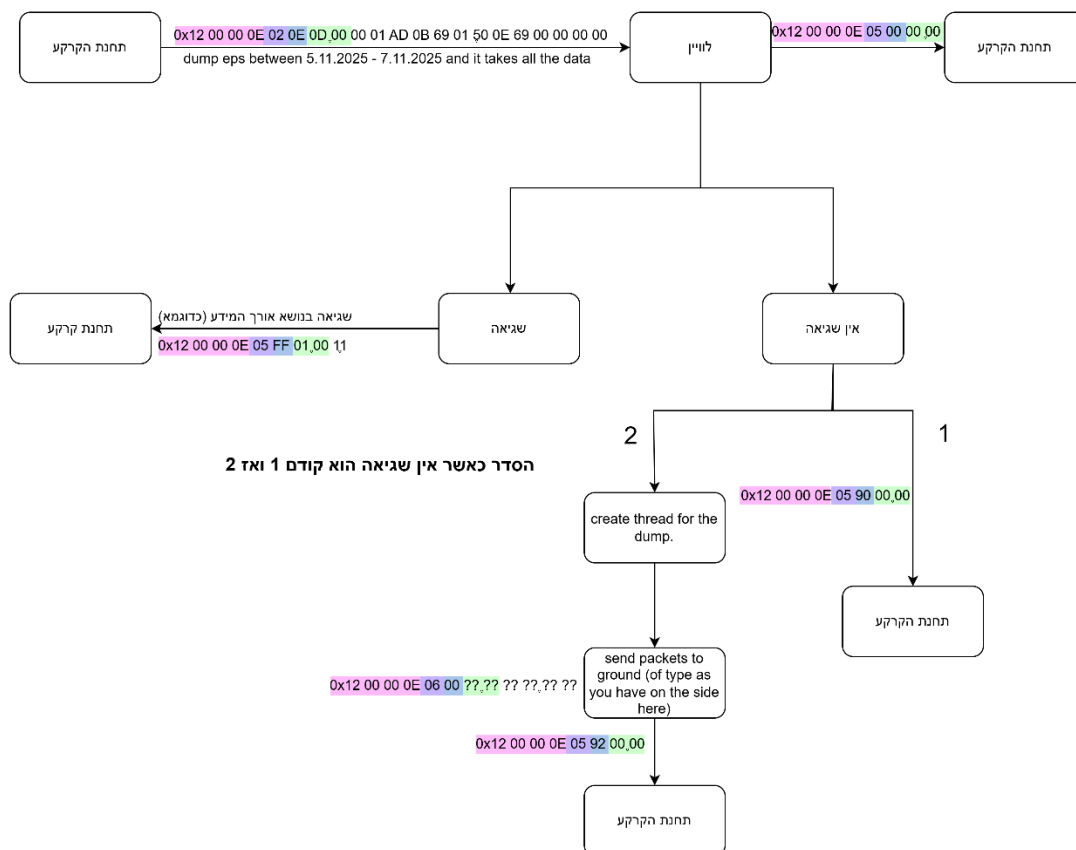
2. שליחת פקודה לקבלת ערך הזמן בין שני beacons (interval)



כמה הערות חשובות.

- a. הלוויין קודם שולח לקרקע הודעה (הודעת קבלת הפאקטה שכתובה בסוף המשפט) ורק אז מבצע את הלוגיקה. (0x12 00 00 0E 05 00 00 00)
- b. התשובה כאשר אין בעיה מחזירה לנו את המספר 20 כמו שכתוב בצד שזהו ה-interval.
- c. הפעם שלחתי שגיאה כללית שלא מייצגת מה היא אבל יש הרבה סוגי שגיאות שאפשר לראות אך זאת הלוגיקה הבסיסית.

3. שליחת פקודה לקבלת dump (בכלליות מאד)



הסבר קצר.

ה-dump היא פונקציה של הורדת הרבה מידע מהלוויין בפעם אחת. אז זה למה יש סימני שאלה מכיוון שאני לא יודעת מה המידע (אני כן יודעת את הגודל אבל הוא משתנה מ-dump ל-dump. הפונקציה מחזירה מספר רב של מידע.

הפקודה הראשונה שמבצעים לאחר מציאת בחוסר שגיאה היא פקודת "התחלתי dump" הפקודה שחוזרת היא פקודת "סיימתי dump". הפקודה ששולחים בקרקע מכילה: שם מערכת, זמן התחלה, זמן סיום, תדירות ה-data. במהלך ה-dump יש גם אפשרות לשלוח הודעה שעוצרת אותו למקרה שהוא נמשך יותר זמן ממה שרצית.

גם כאן הסדר הוא קודם כל שליחת – קבלתי את הפאקטה, אחר כך ack של התחלת הפעולה ובגלל שזהו dump יש גם את כל הפעולה באמצע ואז שליחת סיום dump.

הפרוטוקול המלא:

יש אותו יותר ברור ב-excel חיצוני.

פרוטוקול תחנת קרקע -> לוויין						
מערכת	שם הפקודה	מספר מערכת	מספר פקודה	אורך [bytes]	פרמטרים	תיאור
TRXVU	Mute	0x00	0x02	4	duration [sec]	מפעיל את דגל ההשתקה (max: 7 days)
	Unmute		0x03	0	-	מכבה את דגל ההשתקה
	Set beacon interval		0x04	4	cycleTime [sec]	קובע קצב חדש בשביל שליחת הפאקטה. (min: 5 sec, max: 60 sec)
	Restore default beacon interval		0x05	0	-	מחזיר את הקצב המקורי. Interval = 20 [sec]
	Get beacon interval		0x06	0	-	מבקש את beacon interval
	Get Tx uptime		0x07	0	-	מבקש כמה זמן הרכיב פועל (Tx)
	Get Rx uptime		0x08	0	-	מבקש כמה זמן הרכיב פועל (Rx)
	Turn on idle		0x09	4	duration [sec]	מפעיל מצב idle בלוויין (max: 40 min)
	Turn off idle		0x0A	0	-	מכבה מצב idle בלוויין
	Set transponder on		0x0B	4	duration [sec]	מפעיל מצב משיב בלוויין (max: 14 days)
	Set transponder off		0x0C	0	-	מכבה מצב משיב
	Set RSSI transponder		0x0D	2	rssiVarue [short]	קובע גודל rssi (בעיקרון כמה קולטים). גודל: 0-4095
	Set RSSI default		0x0E	0	-	מחזיר לברירת מחדל: rssi = 10
	Get Ant uptime		0x10	1	antSide [0/1]	מבקש כמה זמן צד אנטנות היה פועל
	Ping		0x13	0	-	מחזיק "pong", רק כדי להראות תקשורת
	Get RSSI		0x14	0	-	מחזיר את ערך RSSI
	Off ants deploy		0x15	0	-	עוצר את ניסיונות פריסת האנטנות
EPS	Reset EPS WDT	0x01	0x01	0	-	
	Update alpha		0x02	4	newAlpha [float]	מספר בין 0-1 smoothing factor
	Restore default alpha		0x03	0	-	ברירת מחדל 0.3
	Get alpha		0x04	0	-	מבקש לדעת מה הערך של alpha
	Update thresholds		0x05	16	DPS, DC, UC, UO *	שולחים בקשה לעדכון ספי מתח. יש טווחים אפשריים. [mV]

DPS = 6900, DC = 7300, UC = 7000, UO = 7400 [mV]	-	0	0x06	0x01	Restore default thresholds	EPS
מבקש קבלת ספי המתח. [mV]	-	0	0x07		Get thresholds	
מבקש את הספים להפעלת מחממים	-	0	0x0C		Get heaters value	
מספר עשרוני. מתי מפעילים מחממים. נוצר מתוך דרישה לא מומלץ להשתמש.	lowThreshold, highThreshold	8	0x0D		Set heaters value	
מבקש לדעת באיזה מצב הלוויין	-	0	0x0E		Get EPS status	
מוחק את כל הקבצים השמורים על הלוויין.	-	0	0x01	0x02	Delete all files	File System
מוחק לפי סוג ותאריך. (סוג - byte)	type, start, end	9	0xA1		Delete TLM	
מספר בין 0-10. כמו הסוג של dump	type	1	0xB1		Delete TLM by type	
מבקש את השגיאה האחרונה ב-FS	-	0	0xBB		Get last FS error	
סוג מ-0-6. תדירות של שמירה.	type [byte], newCycle [int]	5	0xCC		Set TLM cycle	
מחזיר את הכל ל-default. ***	-	0	0xCE		Restore default TLM cycle	
מקבל את הרשימה של כל כמה זמן נשמר data מכל סוג	-	0	0xCD		Get TLM cycle	
שולח בקשה להוריד מידע שנשמר. סוג אפשר לראות בפרוטוקול לוויין-> תחנת קרקע ב-dump לפי מספר פקודה	type [byte], startDate [int], endDate [int], resolution [int]	13	0x0E		Dump	
כמה מקום פנוי יש בלוויין (ב-bytes)	-	0	0x0B		Get amount of free space	
מספר בין 0-1. שולח בקשה להחליף	SD_toSwitch	1	0x0A		Switch SD card	
כאשר רוצים לעצור dump באמצע	-	0	0x1E		Abort dump	
מכבה payload עד להודעה חדשה	-	0	0x11	0x04	Turn off payload	Payload
מדליק payload ומוריד את הדגל	-	0	0x20		Turn on payload	
מבקש את הזמן שבו חוסר תקשורת תגרום ל-reset ללוויין	-	0	0x01	0x03	Get ground WDT duration	Maintenance
שינוי אותו הזמן לזמן אחר. (days min: 3)	newDuration [sec]	4	0x02		Set ground WDT duration	
פקודה חשובה אך קטלנית. מינימום לתחילת שנת 2000. משנה את זמן הלוויין.	newTime [sec]	4	0xDD		Update sat time	

שולח פקודת .reset ***	type	1	0xDE	0x03	Reset component	Maintenance
מבקש את הזמן שבו הלוויין היה דלוק בשניות	-	0	0xE3		Get sat uptime	
מבקש את זמן הלוויין ב-unix	-	0	0xE4		Get sat time	

פרוטוקול לוויין -> תחנת קרקע							
מערכת	שם הפקודה	מספר מערכת	מספר פקודה	אורך [bytes]	פרמטרים	תיאור	מערכת שאליה זה קשור
TRXVU	Beacon	0x00	0x01	141	יהיה בנספחים json שיסביר פרמטרים חשובים של שניות אות חיים בעל interval שולח לקרקע כל	כללי	
Maintenance	FS reset	0x05	0x70	0	-	נשלח לפני שהולכים לאתחל את אחת המערכות. לכל מערכת יש את המספר שלה ששולחים מה כל מספר:	ACK
	Payload soft reset		0x71	0	-	0. Soft	
	Payload hard reset		0x72	0	-	1. Hard	
	TX hard reset		0x73	0	-	2. TX	
	RX hard reset		0x74	0	-	3. RX	
	ANTS reset		0x75	0	-	4. Ants	
	EPS reset		0x80	0	-	5. Soft payload	
	Soft reset		0x81	0	-	6. Hard payload	
	Hard reset		0x82	0	-	7. FRAM	
	FRAM reset		0x83	0	-	8. File system	
	Get ground WDT duration		0x48	4	groundWDT		
	Set ground WDT duration		0xA5	0	-	שולח ACK שהפקודה בוצעה	
	Update sat time		0x12	4	newSatTime	שולח ACK שהפקודה בוצעה (וגם למה השתנה) [sec - unix]	
	Get sat uptime		0x47	4	satUptime	שולח את התשובה לבקשה [sec]	
	Get sat time		0x46	4	satTime	שולח את התשובה לבקשה [unix]	
Payload	Kill payload		0x50	0	-	שולח אישור לבקשת הכיבוי עד להודעה חדשה	
	Turn on payload		0x51	0	-	שולח אישור על הדלקה (גם מוריד את הדגל של עד להודעה חדשה)	
File System	Delete TLM		0xC0	0	-	בכל סוג של מחיקה זה מה שנשלח	
	FS last error		0xC1	4	FS_lastError	מחזיר את השגיאה האחרונה ב-FS	
	Set TLM cycle		0x85	0	-	שולח שהפקודה התבצעה	

File System	שולח את כל התדירויות של שמירה למערכת הקבצים של data [sec]	eps, TRxVU, ants, SP, wod, radfet, sel_seu	28	0x45	0x05	Get TLM cycle	ACK
	שולח מקום פנוי על הכרטיס הנוכחי	freeSpace	4	0x44		Get free space	
	נשלח בתחילת dump	-	0	0x90		Dump start	
	נשלח בסיום dump	-	0	0x92		Dump finished	
	בכל סוג של מחיקה זה מה שנשלח	-	0	0xC0		switch SD card	
EPS	מצביע על כך שהפקודה התבצעה	-	0	0x15	0x05	Reset EPS WDT	ACK
	שולח שבוצעה הפקודה	-	0	0x17		Update alpha	
	מחזיר את ערך ה-alpha	alpha	4	0x40		Get alpha	
	שולח שבוצעה הפקודה	-	0	0x16		Update thresholds	
	מחזיר את ספי המתחים של הלוויין	DPS, DC, UC, UO **	16	0x41		Get thresholds	
	מחזיר את ערכי הסף של הדלקה וכיבוי מחממים	lowThreshold, highThreshold	8	0x43		Get heaters value	
	מודיע ששונה	-	0	0x55		Set heaters value	
	מספר בין 0-2 מציג את מצב הלוויין	state	1	0x42		Get EPS status	
TRXVU	ack הופעל, לכמה זמן הופעל	duration [sec]	4	0x8D	0x05	Mute	ACK
	פקודה בוצעה	-	0	0x8E		Unmute	
	הצליח, המחזור של שליחה החדש.	newCycle [uint]	4	0x14		Set beacon interval	
	מחזיר את המחזור של שליחה.	interval [uint]	4	0x4A		Get beacon interval	
	כמה זמן ה-Tx פועל מאז reset	TxUptime [sec]	4	0x4B		Get Tx uptime	
	כמה זמן ה-Rx פועל מאז reset	RxUptime [sec]	4	0x4C		Get Rx uptime	
	ack הופעל, לכמה זמן הופעל	duration [sec]	4	0xA1		Turn on Idle	
	פקודה בוצעה	-	0	0xA2		Turn off Idle	
	ack הופעל, לכמה זמן הופעל	duration [sec]	4	0x8F		Set transponder on	
	פקודה בוצעה	-	0	0x9F		Set transponder off	
	הצליח, מחזיר את ערך ה-rssi בלוויין	newRssi	2	0x18		Set RSSI	
	בשניות. כמה זמן פועל	AntUptime	4	0x4D		Get Ant uptime	
	שולח pong אחרי שקיבל ping	"pong"	4	0xAA		ping	

TRXVU	מחזיר את ערך ה-rssi בלויין	rssi	2	0x49	ACK	Get RSSI	ACK
כללי	כל פעם שהתקבלה פקודה בלויין	-	0	0x00		Receive com	
	כאשר אין ack מובנה לכך	-	0	0x01		Commend exec	
	שולח שבוע אתחול ואת זמן הלוויין	satTime	4	0x7F		reset wakeup	
	נשלח כשהסתיימה ההפעלה ה-1	-	0	0x35		Finish first active	
	נשלח כאשר מכינים את הדגלים.	-	0	0xFB		Ready for takeoff	
	למקרה של מספר פקודה לא ידוע	-	0	0xBB		Unknown subtype	
	למקרה של מספר מערכת לא ידוע	-	0	0xCC		Unknown type	
	שולח מספר שגיאה, פירוט בהמשך.	errorType	1	0xFF		Error msg	
EPS	dump הוא מספר שליחות. כאן יש שליחה אחת של כל מערכת. כל אחת מהמערכות כאשר עושים לה dump שולחת מספר עצום (לרוב) של packets כאלו. המספר של פרמטרים הוא לרוב גדול לכאן הוא בנספחים. הסבר גם על המשתנה יהיה שם.	****	120	0x00	0x06	EPS TLM	Dump
TRXVU		****	22	0x01		TX TLM	
TRXVU		****	13	0x02		Ants 0 TLM	
EPS		****	28	0x03		Solar Panels TLM	
TRXVU		****	145	0x04		WOD TLM	
Payload		****	20	0x05		Radfet TLM	
Payload		****	16	0x06		SEL TLM	
Payload		****	8	0x07		SEU TLM	
TRXVU		****	26	0x08		RX TLM	
TRXVU		****	13	0x09		Ants 1 TLM	
כללי		****	משתנה	0x0A		Log TLM	

*ראשי תיבות בתחילת המסמך

** אפשר לראות בטבלה dump type את סוגי הקבצים

***יהיה רשום בהמשך. בטבלה של reset type.

****תיאור לערכים ב-dump יהיו בנספחים

טבלאות עזר לפרוטוקול:

default	type	number
10 [sec]	EPS	0
10 [sec]	TRXVU	1
10 [sec]	ANTS	2
10 [sec]	Solar panels	3
10 [sec]	WOD	4
15 [min]	Radfet	5

טבלה: Type TLM period

type	number
soft	0
hard	1
Tx	2
Rx	3
ants	4
payload	5
payload hard	6
FS	7
FRAM	8

טבלה: Reset type

type	number
EPS	0
TX	1
Ant 0	2
Solar Panels	3
WOD	4
Radfet	5
SEL	6
SEU	7
Rx	8
Ant 1	9
log	10

טבלה: Dump type

Error msg	Error number
בעיה בקריאה מה-FRAM	1
בעיה בכתיבה ל-FRAM	2
נכתב ב-FRAM שגוי	3
שגיאה בכתיבה ל-I2C	10
לא מצליח לקבל זמן	15
שליחת גודל לא נכון לפקודה	17
לא מצליח לקבוע זמן סיום	20
לא צד אנטנות קיים	23
לא מצליח לקבל uptime	24
אין אפשרות לאתחל WDT	26
בעיה בלקבל משתנים מה-struct	27
ערך alpha בלתי אפשרי	28
ערך threshold לא אפשרי	29
אי אפשר לעשות את מה שרוצים	30
לא מצליח לעשות אתחול	40
לא מצליח לעדכן זמן לוויין	50
לא הצליח למחוק את כל הקבצים	60
לא קיים כרטיס SD כזה	69
אותו כרטיס שכבר משומש	70
שגיאה לא ידועה	255

טבלה: error type

תיאור מבני נתונים

בלווין אין שימוש במבני נתונים כפי שאנחנו מכירים (חוץ ממבני נתונים של התוכנה כמו struct, union וכדומה), אך קיימים קבצים שנוצרים לשם אחסון נתונים. הקבצים הללו מגיעים עם סיומת המותאמת למערכת. לדוגמה, מידע על מערכת ה-EPS נשמר בקובץ עם הסיומת "eps". זה מתאים משום שהנתונים שמתקבלים מהלווין מגיעים בקטע Hexadecimal, ולכן נשמרים באופן זהה, והסיומת מאפשר להבחין איזו מערכת היא זאת שאנחנו קוראים.

בנוסף, קיים גם קובץ log, אשר נועד לעקוב אחרי שגיאות שמתרחשות בלווין, כך שנוכל לדעת בדיוק מה היו הבעיות ומתי.

מבנה הקבצים ומידע בתוכם:

- **שם קובץ:** {date}.{subsystem}
- **תאריך יצירה:** כל יום נוצר קובץ חדש שמכיל את המידע של אותה מערכת, עבור כל אותו היום.
- **מבנה שורה בקובץ:** {time}{telemetry at that time}

סקירת חולשות

בלווין אין אפשרות להפעיל הצפנה, שכן אנחנו עובדים בתדרי חובבים, ובאופן כללי אסור לבצע תקשורת מוצפנת בתדרים אלו. זהו אחד התנאים שנדרשים כאשר מקבלים רישיון חובבות וכדי להשתמש בתדרים אלו.

המערכת בלווין מוגנת בצורה מיטבית עד שלב של שליחת פקודה חוקית. ברגע ששולחים פקודה חוקית, אין למעשה שום אמצעי למנוע את ביצועה. מעבר לכך, אין חשש שנוכל לשלוח פקודות שגויות או בטעות לגרום ללווין לבצע פעולה לא נכונה, שוב מכיוון שהוא יודע להבדיל וגם עליו יש הגבלות אבל יכולים לשנות לנו דברים בלי שנדע אם למישהו יש את הפקודות. כל פקודה חייבת להיות מדויקת ומובנת לחלוטין.

המערכת של הלווין עצמאית ואינה דורשת מעקב צמוד. כל פקודה שיש לשלוח אליה חייבת להיות מאוד מדויקת, אך מעבר לכך לא נדרשות התערבויות נוספות.

HoopoeWorld מסמך אפיון



מבוא

תיאור כללי של הפרויקט

הפרויקט הינו תוכנה בשם HoopoeWorld שיכולה לרוץ על כל מחשב (בהעדפה על שרת חזק עם הרבה מקום אחסון) שתשמש כשרת לאתר <https>.

התוכנה תשמש כשרת וכלקוח שיכול לגשת ל-Beacon, אות חיים מהלוויין שמכיל את כל המידע החשוב ביותר, שנקלט מתחנות קרקע ברחבי העולם.

באמצעותה ניתן יהיה לצפות בפיענוח הפאקטות ולקבל תמונה כוללת של מצב הלוויין.

קהל היעד

קהל היעד העיקרי הוא **האנשים בתוכנית שלי** (וגם אלו שיצטרפו אליה) – שיוכלו להשתמש ב"אפליקציה", בשרת ובאתר כדי לקבל מידע על הלוויינים מכל רחבי העולם ולא רק מתחנת הקרקע שלנו. וגם יוכלו להשתמש בלוויין שכתבתי לו את הקוד. כאן ספציפית זה בעיקר תלמידי צוות תקשורת.

- הם יוכלו להוסיף לתוכנה לוויינים חדשים לבדוק.
- ארצה שגם יהיה מספיק תיעוד כדי שיוכלו להמשיך ולשפר את הפרויקט אחרי.

סיבות לבחירה בפרויקט

בחרתי בפרויקט הזה כי רציתי לעסוק בתחום שמעניין אותי ואני מרגישה שיש צורך במוצר. אני אוהבת ליצור דברים שיש בהם צורך אמיתי, ובמקרה הזה אני מביאה תוכנה חדשה שתשמש את התוכנית שלי.

כבר זמן רב חשבנו על רעיון של מערכת שתאסוף את כל המידע ותמייין ותציג אותו בצורה ברורה ונוחה, וזאת מטרת התוכנה. בעוד שבלוויין נכנסתי לתוכנית כי רציתי לעשות את זה ובסוף באמת עשיתי.

מטרות הפרויקט

- הצגת מידע שנאסף אך כיום לא נעשה בו שימוש.
- ניתוח המידע כדי לאתר שגיאות ותקלות בזמן אמת באופן יותר יעיל ומהיר.
- מעקב צמוד יותר אחרי הלוויינים.
- אפשרות להפיק גרפים ונתונים ויזואליים נוספים לניתוח.

ייחודיות וחדשנות

נכון לעכשיו, אין תוכנה אחרת שעושה בדיוק את מה שאני מתכננת (התוכנה של ה-HoopoeWorld).

התוכנה הקיימת בתחנת הקרקע שלנו מסוגלת רק לקלוט ולפענח מידע שנקלט אצלנו ולשמור אותו, אך לא לאסוף מידע מכל העולם. ומצד שני האתר שממנו אני רוצה לקחת נתונים מציג מידע בפורמט גולמי (Hexa), ללא מיון לפי לוויין או הצגה קריאה.

אני מתכננת ליצור תוכנה שתדע לאסוף את המידע מהאתר, למיין אותו, להציג את המידע בצורה נוחה וברורה, במספר מחשבים דרך שרת עם אפשרות להרחבה בעתיד.

מגבלות ופיתוח עתידי

בעיקרון ה-HoopoeWorld הוא בלי הגבלות רק צריך מחשב מספיק חזק כדי שיוכל לשמש כשרת. בהגבלות לתוכנה יכול לקרות מצב בו האתר שהתוכנה מנסה לקרוא ממנו יקרוס ואז לא נקבל מידע חדש עד שהאתר יעלה מחדש ואם הלוויין לא נמצא באתר לא נוכל לקבל אליו מידע נוסף. התוכנה נועדה להיבנות כך שיהיה ניתן להוסיף בקלות לוויינים חדשים וסוגים שונים של Beacons, ולהתאים אותה בהמשך – בניגוד לתוכנה הנוכחית שלנו בתחנת הקרקע שמקבלת מידע רק מהתחנה, שאף אחד לא יודע לערוך אותה ולא מנסים כדי לא להרוס. היא גם נבנתה כבסיס מספיק ברור שיהיה אפשרות להגדיל אותה וכאשר יוצרים אצלנו תוכנת קרקע חדשה יוכלו לחבר אותה לאתר שלי.

היבטים טכניים

הפרויקט כולל בתוכו:

- עבודה עם מבני נתונים.
- פיתוח מערכת שרת ומספר לקוחות.
- שימוש בפרוטוקול תקשורת מוצפן (HTTPS).
- כתיבה ויישום של פרוטוקול תקשורת.
- הורדת מסמכים משרת ללקוח.
- שרת דינאמי הפועל לפי json.

בדיקות המערכת

- לראות שאין כפילויות ב-database בגלל שזה אומר שיש לי בעיה בקוד. תתבצע על ידי הכנסת הנתונים כאשר אני יודעת שיש כפילויות ואז לשלוח שאילת שסופרת את הכל ואחת שסופרת רק את המיוחדים. כאשר אני רואה שהמספרים שווים אני יודעת שזה עובד כראוי.
- שהלקוח באמת מקבל כל פעם שהוא גולל למטה את המידע הנוסף. יתבצע על ידי כניסה למערכת ולראות שבאמת מקבלים עוד דברים.

- שכאשר הוא למעלה הוא מתעדכן על המידע החדש כאשר עוברות שעותיים וגם בשרת התוסף מידע.
- לראות שאם אני משנה את המקום ממנו אני לוקחת הכל עדיין עובד ורץ. יעשה באמצעות פנייה לתוכנה ובקשת לווין אחר ולקחת מעט ואז אני אראה שהכל טוב (וגם אראה שזה מתרגם נכון לפי פרוטוקול Beacon שונה).
- כאשר לוחצים על הלוויינים השונים באמת משתנה המידע ללוויין הנכון.

גבט

שלב	תאריך – סיום
אפיון פרויקט	30.11.25
להכין את אופי ה-json ואת ה-config	10.12.25
להכין את התוכנה שיוצרת את הטבלאות	20.12.25
להכין קובצי אתר שיראו ערכים מפוברקים (מבחינת html, css, js)	1.1.26
להתחיל להכין את השרת – פיענוח הפאקטות לפי ה-json וה-SQL.	25.1.26
לשלב בין קבצים של האתר לפיענוח של השרת (שיהיה בפורמט שנוכל להכניס את זה כמידע לאתר)	30.1.26
להכין את ה-class לקוח בצד שרת	10.2.26
יצירת האפשרות לחיבור באמצעות HTTPS	1.3.26
הורדת המידע מהלוויין ב-excel למחשב הלקוח לפי כמות, זמן, או הכל.	10.3.26
סיום כתיבת ספר פרויקט.	1.4.26

פירוט יכולות התוכנה

צד לקוח

הלקוח הוא אתר אינטרנט המשמש ממשק להצגת נתונים ולביצוע בקשות לשרת.

- בחירת לווין.
 - המשתמש יכול לבחור מתוך רשימת לוויינים קיימים במערכת על מנת להציג את הנתונים שלהם.
 - נדרש:
 - כפתורי בחירה או רשימת Select ב-HTML/JS
 - שליחת בקשה מסוג GET לשרת לקבל את הדפים
 - שליחת GET נוסף לקבלת נתוני לווין שנבחר
 - נחוץ שיהיה בצד השרת את ה-JSON.
- הצגת ביקון
 - הצגת המידע שנאסף מה-Beacon של הלווין באופן קריא. במקום Hexa גולמי, מוצג מידע מפוענח לפי הפורמט שהוגדר בשרת וגם מקבל את המידע שהוא כבר לא גולמי מהשרת (מקבל מחדש את העמוד).
 - נדרש:
 - שליחת בקשת GET לשרת לקבלת נתוני ה-Beacon העדכניים
 - הצגת הנתונים בדפדפן בפורמט נוח לקריאה
 - יכולת לקבל נתונים חדשים בזמן שהמשתמש נמצא באתר (עדכון תקופתי וכדומה)
 - נחוץ:
 - פונקציית JS להצגת וקבלת הנתונים
 - אזור תצוגה ב-HTML (טבלה / כרטיסיית נתונים)
- מנגנון בקשת נתונים אוטומטית
 - על ידי גלילה של העכבר / כמות זמן שעבר הלקוח מקבל עוד נתונים על הלווין. גלילה מטה, מקבל עוד למעלה כל שעתיים, אך רק עם יש עוד
 - נדרש:
 - פונקציות JS המטפלות במנגנון
 - פרמטרים שנשמרים ב-JS
 - החזרה מסודרת של הפרמטרים
- בקשת הורדת excel
 - המשתמש יכול להוריד קובץ Excel של נתוני הלווין — את כל העבר, לפי זמנים או רק מספר פאקטות אחרונות.
 - נדרש:
 - כפתור הורדה ב-HTML
 - שליחת GET לשרת עם פרמטרים
 - ניהול הורדת קובץ ידידותית למשתמש

צד שרת

צד השרת בתוכנה הזאת הרבה יותר מורכב מכיוון שהוא עושה הרבה פעולות ובסוף שולח לדפדפן את האתר.

- מבני נתונים (SQL, JSON)
 - שמירת נתונים של כל לווין בטבלת SQL. קיימת טבלה שממפה בין לווין לבין הטבלה המתאימה (יכול להיות שיהיה ב-config של ה-json אבל במהלך הזמן אדע)
 - נדרש:
 - יצירת בסיס נתונים SQL
 - יצירת טבלאות דינמיות בהתאם לנתוני כל לווין
 - מנגנון הכנסת נתונים ל-SQL על בסיס קובצי JSON המגדירים פורמט
 - קבצי JSON של מבנה הנתונים
- Config
 - קבצי JSON המגדירים את פורמט ה-Beacon עבור כל לווין, וכן את כתובתו. בקובץ ה-config יהיה את מזההו של הלווין, כתובת ה-json, וקישור לטבלת הנתונים שלו שבאתר (ואולי גם בשרת עצמו).
 - נדרש:
 - שמירת קבצי JSON נפרדים לכל לווין
 - הגדרת פרוטוקול כתיבה אחיד (schema) שמאפשר להוסיף או להוריד לווים בקלות
 - תוכנה שתפרש שתבין את ה-JSON ותתרגם Hexa לנתונים.
- בניית אתר ושליחתו ללקוח
 - השרת יוצר את דפי ה-HTML/JS/CSS באופן דינמי בהתאם לנתונים הקיימים בבסיס הנתונים, ומגיש אותם בפרוטוקול HTTPS.
 - נדרש:
 - תבניות HTML (או יצירה דינמית)
 - הפעלת שרת HTTPS
 - קריאת הנתונים מבסיס הנתונים והזרמתם לדפדפן
 - מנגנון גלילה דינמית / טעינה מתמשכת אם נדרש
- שליחת כל המידע או חלק ב-excel
 - השרת אוסף את כל נתוני הלווין מה-SQL (או רק מספר השורות שהלקוח ביקש), מייצר מהם קובץ Excel ושולח ללקוח. * מביא את החדשים ביותר.
 - נדרש:
 - שאילתת SQL מבוקרת
 - יצירת Excel באמצעות ספרייה מתאימה
 - שליחת הקובץ בתגובה ל-GET
 - למידה של משהו חדש. (אני לא יודעת איך אפשר לעשות את זה אז אני לא יודעת מה זה מכיל. כאשר אגיע לשם אראה ואוסיף)

ארכיטקטורת הפרויקט

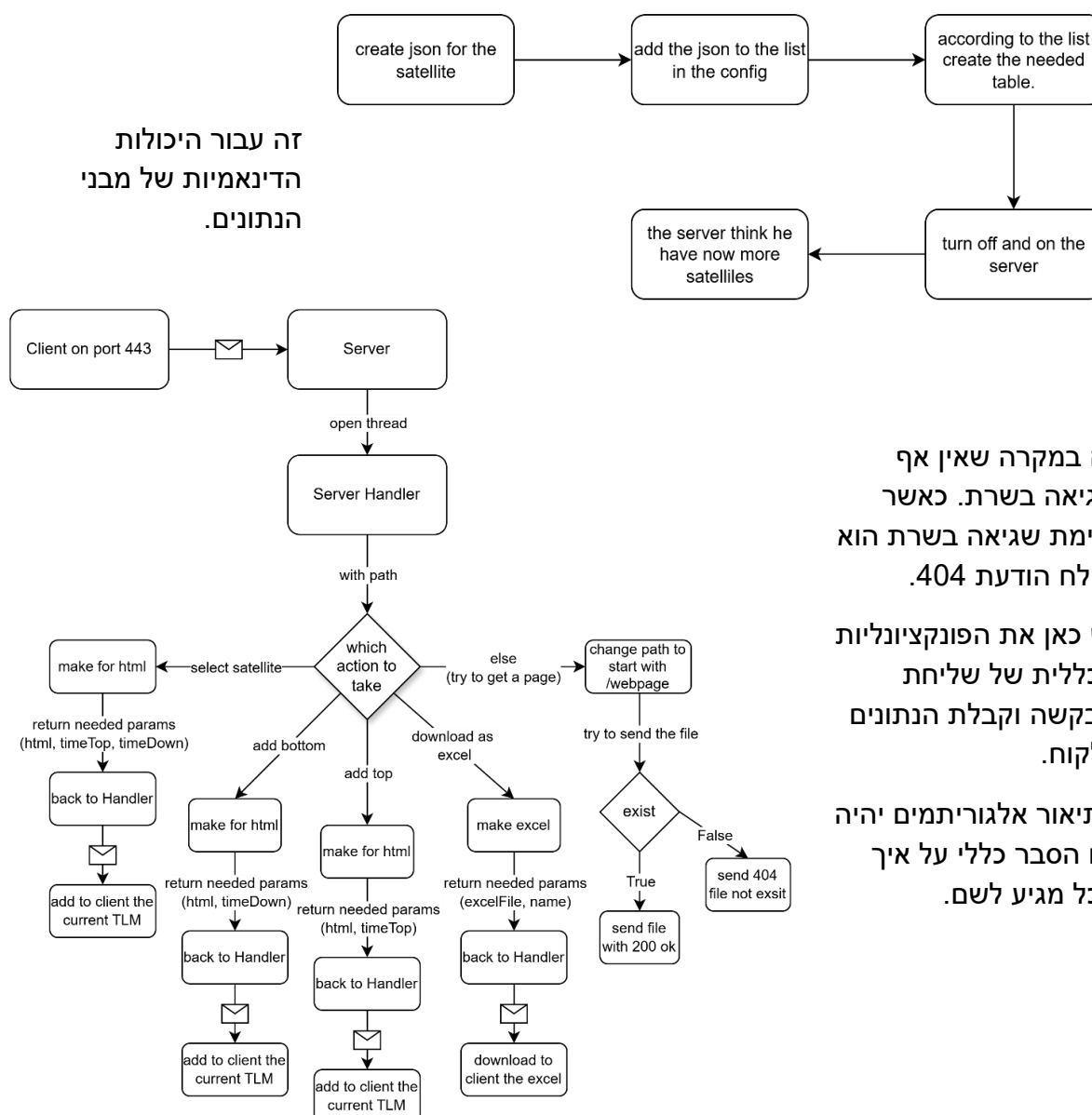
פירוט חומרה

- מחשב חזק עם הרבה מקום לשמש כשרת.
- לוויין (שיהיה מה לקלוט)
- מחשבים להתחבר דרכם.

תיאור טכנולוגי

- בתוכנה הזאת יש מספר שפות תוכנה. שפת השרת תהיה python אך כדי ליצור את האתר
- אצטרך שפות אתרים כגון: html, css, js.
- ה-OS עליו התוכנה פועלת יהיה windows.
- פרוטוקול התקשורת בין הרכיבים הוא HTTPS.

תיאור זרימת המידע



תיאור אלגוריתמים

האלגוריתמים של הפרויקט מתחלקים ב-3 קבצים. לכל קובץ מטרה משלו.

בתיאור הנוכחי אני אסביר בכלליות על האלגוריתם ומה קורה אחרי מה בלי להיכנס ממש לעומק. הסבר על כל פונקציה וכל מחלקה יהיו בחלק של מסמך התוכנה.

יש 2 דרכים לעשות את זה אך הדרך העיקרית תהיה כתובה כאן. חוץ מזה יש באלגוריתם זה התעלמות מכך שיכול להיות שהוסיפו לווין חדש שהוא כבר פועל שנים. התוכנה מיועדת בעיקר ללוויינים שהתחילו את פעולתם או שכבר הכניסו עליהם מידע ואפשר משם להמשיך עם מידע יותר חדש שיתעדכן כל הזמן.

זה תיאור אלגוריתם השרת.

האלגוריתם מתחיל בליצור את המילונים בהם התוכנה תשתמש לכל אורך התוכנית. הוא לוקח את המידע מה-json כך שלא תהיה בעיה של שינוי תוך כדי וגם כדי שהתוכנה תהיה יותר מהירה. בתוך ה-JSONS האלו נכללים: ההמרה ל-HTML, ההמרה ל-SQL ואחד שמחבר בין לווין להמרה.

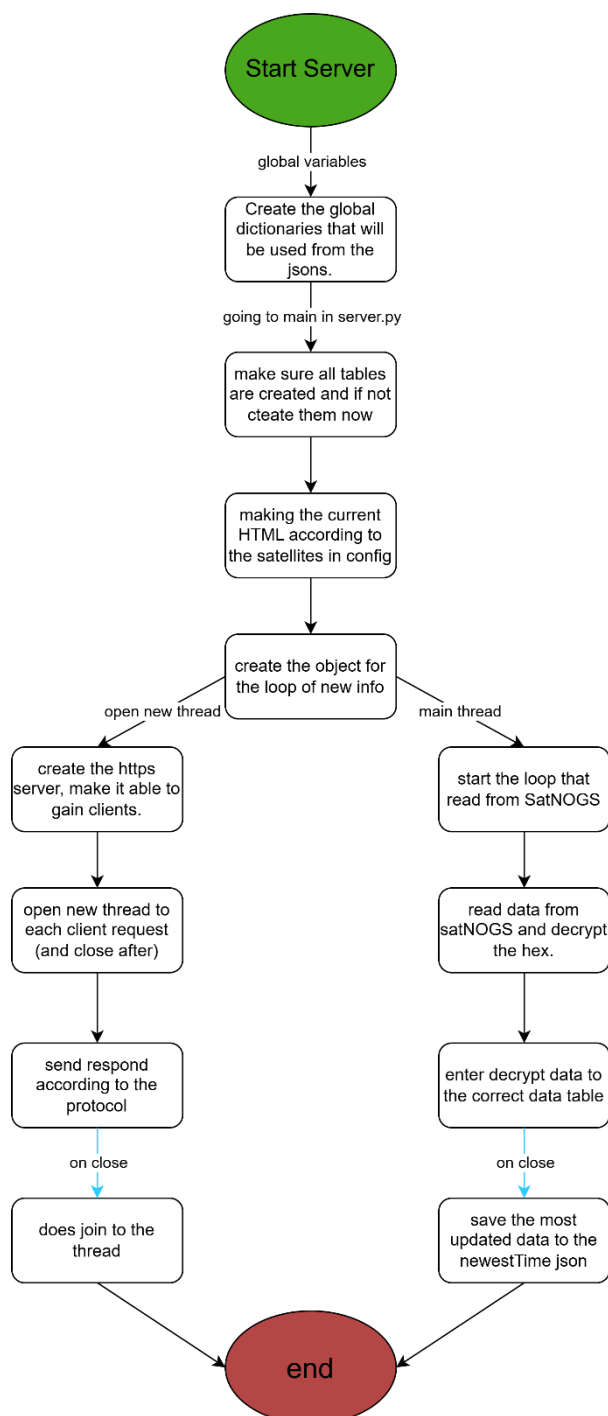
לאחר מכן יוצרים את הטבלאות עם לא קיימות (מצופה שיתבצע לפני אבל גם בתחילת הריצה קורה ליתר ביטחון).

משנה את ה-template שיצרתי ללוויינים שכרגע ב-config.

יוצר את האובייקט שאיתו אנחנו קוראים מה-SatNOGS. ההרצה של הלולאה האינסופית קוראת רק אחרי שאנחנו פותחים thread עבור תקשורת עם הלקוח (https server).

עכשיו הלקוח יכול להתחבר ולבקש נתונים ובאותו הזמן נקראים נתונים חדשים, מפוענחים ונכנסים לטבלת מידע המתאימה.

כאשר יש בעיה או שסוגרים את התוכנית מה שכתוב "on close" מתרחש. לחלוטין סוגרים את הכל וגם שומרים מה היה הזמן הכי עדכני שנקרא עבור כל אחד מהלוויינים כך שבפעם הבאה נתחיל מהכי חדש ונגיע רק עד איפה שהיינו ולא נמשיך יותר.



תיאור סביבת הפיתוח

במהלך הפרויקט נעשה שימוש במספר סביבות וכלים משלימים, שלכל אחד מהם תפקיד ייעודי בתהליך הפיתוח:

1. **PyCharm** – שימש לפיתוח ולתכנות צד שרת בשפת Python.
2. **Visual Studio** – שימש לפיתוח קבצי האתר. כולל CSS, HTML, JavaScript.
3. **Visual Studio Code** – שימש ליצירה ולעריכה נוחה של קובצי JSON.
4. **SQLiteStudio** – שימש לבדיקת בסיס הנתונים ולאימות שהמידע נוצר ונשמר כראוי.
5. **Google Developer Tools** – שימש לבדיקת קוד JavaScript ולתצוגה ועיצוב בזמן אמת של רכיבי האתר.
6. **SatNOGS** – משמש כמקור הנתונים המרכזי של המערכת, ממנו מורד וממין המידע. כלי זה אינו סביבת פיתוח כשלעצמו, אך מהווה רכיב מהותי בפעולת התוכנה.
7. **CMD** – בו נוצר ה-self-signed certificate עבור תקשורת HTTPS.

פרוטוקול תקשורת

פרוטוקול ב-HTTPS וכל הפקודות הן פקודות GET.

הפירוט שתראו יהיה בקשר לתוכן של פקודת ה-GET.

לקוח -> שרת

התקשורת בין הלקוח לשרת מתבצעת באמצעות בקשות HTTPS.

כל הנתונים נשלחים דרך ה-URL.

הכל בסוף מפוענח כמילון בשרת.

בחירת לווין:

Endpoint: /chooseSatellite/?satName={ }&width={ }

בעת בחירת לווין בפעם הראשונה, נשלחת בקשה זו לשרת.

גודל החלון בעת פתיחתו עבור התממשקות עם גדלים שונים - width

השרת מחזיר ללקוח את 25 ה-packets האחרונות של הלווין שנבחר בצורה הנראית ידידותית למשתמש.

קבלת נתונים חדשים (למעלה):

Endpoint: /addTop/?satName={ }&mostResent={ }&width={ }

כל שעייתם התוכנה בודקת מול השרת האם קיים מידע עדכני יותר מהמידע שנמצא אצל הלקוח. אם קיים מידע חדש – הוא נשלח ללקוח.

הקבלה מתבצעת על פי:

- שם הלווין – satName
- זמן הלווין העדכני ביותר הקיים אצל הלקוח – mostResent
- גודל החלון בעת פתיחתו עבור התממשקות עם גדלים שונים - width

קבלת נתונים חדשים (למטה):

Endpoint: /addBottom/?satName={ }&leastResent={ }

כאשר המשתמש גולל כלפי מטה והגיע כמעט לסוף הנתונים המוצגים, התוכנה מבקשת מהשרת פאקטות נוספות. גם כאן 25 מהן.

הקבלה מתבצעת על פי:

- שם הלווין – satName
- זמן הלווין הכי פחות עדכני הקיים אצל הלקוח – leastResent
- גודל החלון בעת פתיחתו עבור התממשקות עם גדלים שונים - width

הורדת נתונים כ-excel:

Endpoint: /downloadData/?type={typeOfDownload}&satName={}

זהו Endpoint כללי לכל סוגי ההורדות.

בהתאם לסוג ההורדה, מצורפים פרמטרים נוספים:

- &limit={} – חיפוש לפי כמות מקסימלית של נתונים.
- &end={} &start={} – חיפוש בטווח זמן מוגדר
- &start={} – חיפוש מזמן מסוים ועד הנתון העדכני ביותר
- ללא פרמטרים – קבלת כל הנתונים על הלוויין הספציפי.

סוגי ההורדה השונים:

פרמטרים להוספה	שם סוג
אין	All
limit	Limit
start	StartTime
start, end	StartToEndTime

שרת -> לקוח

בכל בקשה השרת פועל לפי הסדר הבא:

1. בדיקה האם מדובר בפעולה.
2. אם לא – ניסיון לשלוח קובץ.
3. במקרה של כישלון, מוחזרת שגיאת 404.

בחירת לווין:

בעת בחירת לווין, השרת מחזיר ללקוח:

- קטע HTML המכיל את ה-packets בפורמט הצגה (25 packets)
- כלל הנתונים נשלחים בפורמט JSON, כולל גם את חותמות הזמנים.

שדות ב-JSON:

1. **mostRecent** – זמן הלוויין העדכני ביותר שנשלח ללקוח.
2. **leastResent** – זמן הלוויין הכי פחות עדכני שנשלח ללקוח.
3. **data** – קטע ה-HTML המכיל את הנתונים להצגה.

קבלת נתונים חדשים (למעלה):

כאשר מתקבלת פקודה לכך השרת מחזיר

- קטע HTML המכיל את ה-packets בפורמט הצגה (כל אלו שיותר גדולים מהזמן שבלקוח)
- כלל הנתונים נשלחים בפורמט JSON, כולל גם את חותמת הזמן החדשה.

שדות ב-JSON:

1. **mostRecent** – זמן הלוויין העדכני ביותר שנשלח ללקוח.
2. **data** – קטע ה-HTML המכיל את הנתונים להצגה

קבלת נתונים חדשים (למטה):

כאשר מתקבלת פקודה לכך השרת מחזיר

- קטע HTML המכיל את ה-packets בפורמט הצגה (25 packets)
- כלל הנתונים נשלחים בפורמט JSON, כולל גם את חותמת הזמן המאוחרת ביותר החדשה.

שדות ב-JSON:

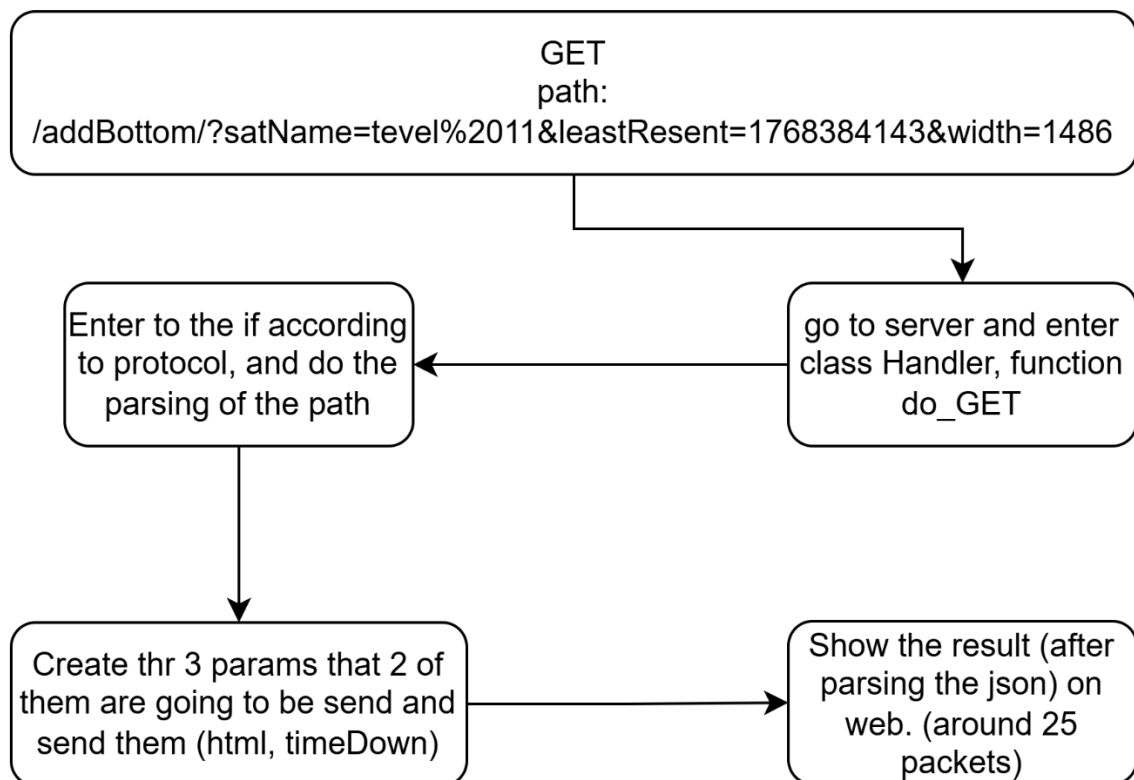
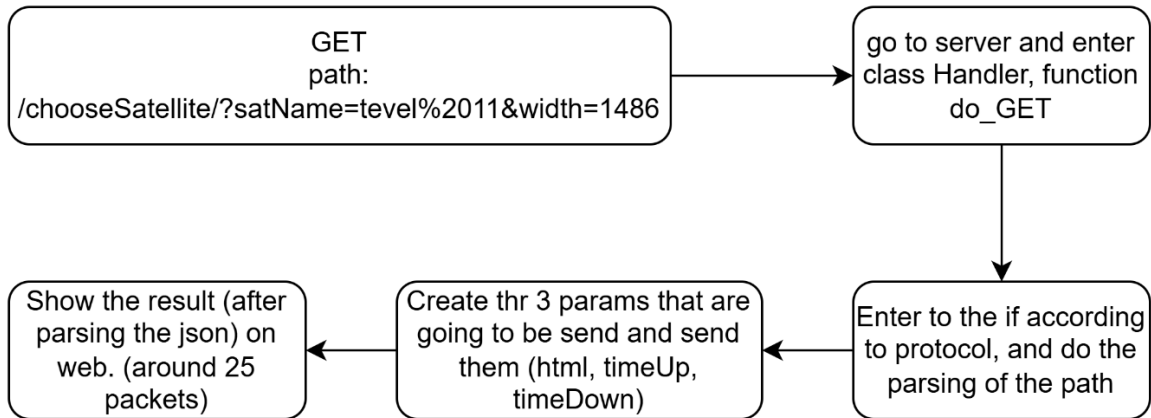
1. **leastResent** – זמן הלוויין הכי פחות עדכני שנשלח ללקוח.
2. **data** – קטע ה-HTML המכיל את הנתונים להצגה

הורדת נתונים כ-excel:

בעת בקשה להורדת נתונים, השרת מחזיר קובץ Excel בפורמט .xlsx, המכיל את כל הנתונים בהתאם לפרמטרי החיפוש שנשלחו בבקשה.

הקובץ נשלח ללקוח בקידוד Hex.

דוגמאות פשוטות להעברה:



תיאור מסכי המערכת

יש מסך אחד שהוא המסך הראשי אבל אפשר לראות עליו דברים שונים באמצעות הכפתורים.

כך נראה המסך ההתחלתי. ב- select של בחירת לוויין נמצאים כל הלוויינים שכתובים ב-config ואפשר לבחור ביניהם.

הבחירה של לוויין תוסיף בחלק התחתון את ה-data ואת האפשרות להוריד מידע כ-excel.

בצד יש זמן ב-UTC מכיוון שלרוב מכוונים את הלוויינים לזמן הזה אז שתהיה נקודת זמן.



Hex raw:	EPS	OBC	Payload	Solar Panels	TRXVU
00 00 00 00 00 01 8D 00 1E 42 00 FA 9A 41 00 C8 00 00 02 00 00 76 00 00 00 00 00 00 B8 EF A2	voltBattery: 7913 volt_5V: 4986 volt_3V3: 3322 charging_power: 350 consumed_power: 790 electric_current: 40 current_3V3: 74 current_5V: 0 bat_temp: 12 eps_state: operational	sat_time: 07.01.2026 11:38:23 free_memory: 1086464000 corrupt_bytes: 0 total_memory: 2012323840 used_bytes: 925859840 lastFS_error: 0 number_of_resets: 20 sat_uptime: 49509 num_of_cmd_resets: 2	sel_counter: 118 seu_counter: 0 sensor_one_radfet: 583656867 sensor_two_radfet: 579187025 radfet_temp: 20.72851562500002 last_radfet_read_time: 07.01.2026 11:26:05 power_payload: 200 payload_kill_flag: 0 change_in_mode: 13	solar_panels0: 31.5478515625 solar_panels1: 9.5234375 solar_panels2: 15.7705078125 solar_panels3: 39.5615234375 solar_panels4: 19.320703125 solar_panels5: 29.34765625	temp_pa: 20.367050170898438 temp_board: 20.21367073059082 reflected_power: 0.2119320034980774 forward_power: 0.47684699296951294 doppler: 457.763671875 rssi: -118.0

המסך הזה שנמצא למעלה מראה מה קורה במקרה שנבחר לוויין Tevel 11 ומה יוצג. יעלו ה-packets הכי חדשות שנמצאות ב-database שפוענחו לפי ה-json. כל שעתיים יש ניסיון לעלות עוד פאקטות, שוב את הכי חדשות. כאשר גוללים מספיק למטה הלקוח מבקש אוטומטית עוד פאקטות להצגה והם נוספות בסוף הגלילה.

בצג גם התווסף הכפתור שיראה את האפשרות להוריד ל-excel.

The screenshot shows the HoopoeWorld website interface. At the top, there's a header with the site name and a date/time stamp. Below the header, there's a section for 'Hex raw' data. A central modal window is open, titled 'Download data for level 11', with options to 'Choose type of download' and 'Start to End Time'. Below the modal, there are several panels displaying different types of data: EPS (voltage, current, power), OBC (satellite time, memory, errors), Payload (sensor data, temperature), Solar Panels (power output), and TRXVU (temperature, power, doppler). Each panel contains a list of numerical values and some text labels.

המסך הזה קופץ כאשר לוחצים על הכפתור. אפשר לבחור בו בין אפשרויות שונות של הורדות ללוויין ההוא. האפשרויות הן: start-end, start, limit, all. כל אפשרות תיצור בסוף שאילת שונה עבור השרת לשלוח ממנה.

הבקשה בסופו של דבר תוריד קובץ excel למחשב לפי המיון מה-SQL שנשמר בשרת.

תיאור מבני הנתונים

בתוכנה יהיו מספר מבני נתונים אך הם דינאמיים אבל בכללי:

- config – Json – מחבר בין ה-SQL, הכתובת ממנה לוקחים את הדברים ללוויין הספציפי, שם הלוויין, כתובת ה-json של הפיענוח.
- Json – ספציפי לכל פיענוח לווין – מחזיק את הפיענוח של ה-Beacon. מה אמור להיות בו, את מפתח ה-SQL, גודל ה-header, מיקום ה-opcode ב-header והאם big/little endian.
- SQL – אחד לכל לווין – כל המידע המופענח נשמר שם, נוצר על פי ה-JSON הספציפי. כאשר קוראים ממיון לפי זמן לווין.

ל-JSON יש פורמטים נחוצים. הפורמט לכל json יהיה בעמוד הבא.

newestTime:

(not a must at start but it's recommended)

```
{
  {satNogs_name}: "most current time got from there"
}
```

הקובץ ייווצר בכל מקרה אם הוא לא קיים אך העדפה שיהיה התחלה ולא מ-0, או שיהיה ישירות אחרי שיגור הלוויינים מכיוון שיכול להיות שהחיבור שלנו לאתר יקרוס ולא נוכל לקבל עוד לתקופה.

אם רוצים מההתחלה ממליצה להוריד ב-csv, לקחת ממנו את התאריך הכי מאוחר ולהשתמש בפונקציה שנמצאת בהערה כדי להוסיף את זה למקום הרצוי.

Config:

```
{
  "{satName}": {
    "callsign": "{sat_callsign}",
    "tableName": "{what the SQL table will be called}",
    "satnogs_name": "{how the sat is called in satnogs}",
    "beacon_json": "{the json of the data/beacon format}"
  },
  "{satName2}": {
    "callsign": "{sat_callsign}",
    "tableName": "{what the SQL table will be called}",
    "satnogs_name": "{how the sat is called in satnogs}",
    "beacon_json": "{the json of the data/beacon format}"
  }
}
```

אפשר להוסיף כמה לוויינים שרוצים, זה דוגמא עם 2, אך כל הדברים כאן נחוצים שיהיו בשניהם. (ה-beacon_json יכול להיות אותו אחד).

Beacon_json:

```
{
  "setting": {
    "isDefaultBigEndian": false,
    "opcode": "opcode in hex (type + subtype)",
    "sizeof_header": "full size of header it's an int",
    "place_start_opcode": "counter start at 1. It's an int",
    "prime_key": "which param is the primary key"
  },
  "subType": {
    "name": "name of function (most likely beacon)",
    "params": [
      {
        "name": "{name of param}",
        "type": "{type of param}",
        "subSystem": "{if we have a system it is about} - from here included not a must",
        "enum": "{if the type is byte and want to display enum}",
        "format": "{in case we have changes for the param (x). can do +, -, *, /, ** (for power)}",
        "isBigEndian": false
      },
      {
        "name": "second param, going according to the order in packet",
        "type": "{param type}"
      }
    ]
  }
}
```

יכולים להיות כמה פרמטרים שרוצים (יותר נכון כמה שיש בפאקטה), בפעם השנייה מוצגים כל הדברים שחובה שיהיו בכתיבה של פרמטר.

סקירת חולשות ואיומים

1. תלות במערכת SatNOGS

במקרה של קריסה באתר SatNOGS, לא ניתן יהיה לקבל נתונים חדשים. מאחר שמדובר בגורם חיצוני, אין אפשרות לטפל בתקלה זו באופן ישיר, מלבד טיפול בשגיאות בקוד (כגון שימוש במנגנוני try-catch) על מנת למנוע קריסת המערכת המקומית.

2. טעויות אנוש בקבצי ה-JSON

המערכת אינה תמיד מסוגלת לזהות טעויות הנובעות מהזנה ידנית של נתונים. כתוצאה מכך, ייתכן שקובצי ה-JSON הכוללים מבנה שגוי או נתונים לא תקינים יתקבלו על ידי המערכת ויגרמו לבעיות בפעולתה.

3. שימוש בתעודת HTTPS עצמית (self-signed)

מאחר והמערכת משתמשת בתעודת HTTPS מסוג self-signed, נדרש חידוש ידני של מפתח ההצפנה אחת לשנה. תהליך זה מהווה נקודת חולשה פוטנציאלית, הן מבחינת אבטחה והן מבחינת תחזוקה.

4. קבלת נתונים שגויים מצד גורמים חיצוניים

במידה וגורם חיצוני מבין את פרוטוקול התקשורת, קיימת אפשרות לשליחת נתונים שגויים למערכת. עם זאת, איום זה צפוי להשפיע בעיקר על חוויית אותו הגורם והצגת המידע אצלו אך לא אצל כלל המשתמשים, ולא לגרום לקריסת השרת, מאחר ואין אפשרות להעלות מידע מהלקוח לשרת.

התוכנה משתמשת בפרוטוקול מוצפן HTTPS, כך ששכבת התעבורה היא TCP.

תוכנה מוטסת ללוויין מסמך תוכנה



דרייברים מיובאים

בקוד הלוויין יש שימוש בדרייברים חיצוניים שקיבלנו עם החומרה. חוץ מזה גם דרייברים של ה-OS. אז במקום תיאור מחלקות זהו תיאור של קבצי ה-h שקיבלתי עם הלוויין. (לאחד מהם יש לי גם את קובץ ה-C וערכתי אותו מאט באישור הכותבת שהיא המדריכה שלי)

- תחת תיקיית satellite-subsystems
 - o isis_vu_e.h
 - דרייבר מערכת התקשורת. יש כאן את דרך קבלת הפקודות, אתחול, קבלת מידע וכדומה.
 - o isis_ants.h
 - דרייבר האנטנות. גם כן אפשרות לקבלת טלמטריה, פריסה של האנטנות ו-init.
 - o isismepsv2_ivid7_piu.h
 - דרייבר למערכת החשמל. קבלת טלמטריה, הכי חשוב קבלת מתח בטרזיה ועוד...
 - o IsisSolarPanelv2.h
 - דרייבר התקשורת עם הפאנלים. הפעלה, שינה, קבלת טמפרטורה וכיבוי זה עיקר הדברים.
- תחת תיקיית hcc
 - o api_fat.h
 - דרייבר התקשורת עם ה-SDs. פונקציות כמו read, write, switch_sd וכדומה.
- תחת תיקיית hal
 - o Timing
 - Time.h
 - ממשק דרייבר הזמן שמספק פונקציות רבות המשומשות ביותר הן מעקב אחר שעון המערכת, ושינוי השעון.
 - WatchDogTimer.h
 - מאפשר הפעלה, איפוס וניהול של ה-Watchdog באופן בטוח ומופשט מהחומרה כדי להגן על המערכת מפני תקיעות.
 - o supervisor.h
 - שייך לרכיב על ה-OBC. דרכו אני יכולה לקבל כל מיני דברים כמו uptime של הלוויין וכדומה.
 - o boolean.h
 - TRUE, FALSE כ-int וכ-char
 - o Storage
 - FRAM.h
 - מאפשר קריאה, כתיבה וניהול הנתונים בזיכרון ה-FRAM (זיכרון בלתי נדיף מהיר) באופן בטוח ומופשט מהחומרה עבור התוכנה.
 - o Drivers
 - I2C.h
 - אפשר לאתחל ולשלוט בממשק ה-I²C של המיקרו-בקר, ומספק דרך בטוחה ומופשטת מהחומרה עבור התוכנה לבצע פעולות קריאה וכתיבה ב-I²C.

▪ SPI.h

הדרייבר מאתחל ושולט בממשק ה-SPI של המיקרו-בקר, ומספק דרך בטוחה ומופשטת מהחומרה עבור התוכנה לבצע פעולות קריאה וכתובה ב-SPI. די דומה ל-C² אבל פשוט סוג שונה של חיבור בין רכיבים.

- תחת תיקיית freertos

◦ semphr.h

מספק semaphores, mutexes לניהול סנכרון בטוח בין threads.

◦ task.h

ממשק ניהול המשימות ב-FreeRTOS, כולל יצירה, מחיקה, תזמון והמתנה של משימות.

◦ FreeRTOS.h

מגדיר את ליבת מערכת ההפעלה FreeRTOS, כולל קונפיגורציה, טיפוסים בסיסיים ופונקציות ניהול כללי של RTOS עבור התוכנה.

- SubSystemModules/Payload/payload_drivers.h

דרייברים של ה-payload. לא במדויק מיובאים כי ראיתי את כתיבתם אבל אני לא כתבתי אותם בעצמי אפילו אם עשיתי בהם שינויים. כל התקשורת עם המערכת של ה-payload שזה בעצם הדלקות, כיבוי וקריאת טלמטריה.

- String.h

משמש בעיקר ל-memcpy. אין כל כך שימוש רב עם strings וגם כשיש זה מערך של chars ומשמש כך.

- stdio.h

כמובן שזה יהיה משמש זה הסטנדרט input-output בלי זה אין פקודות בסיסיות כמו printf.

- stdlib.h

בשונה ממה שלרוב משתמשים בספרייה זאת (בעיקר ל-malloc) אין כאן שימוש ב-malloc אלא ב-exit למקרה קצה שהייתה בעיה עם כניסה למערכת ה-freeRTOS.

- time.h

לרוב לא משתמשים בספרייה הזאת, אך משמש לפחות פעם אחת כאשר רוצים לקבל את הזמן בפורמט הזה כדי שנוכל לשנות לפורמט שלנו. (מקבל לפי unix_time הפונקציה שבה אני משתמש משם).

- at91/peripherals/cp15/cp15.h

משמש להעלאת ביצועי הלוויין שיהיו יותר מהירים.

[*ראשי תיבות כמו במסמך איפיון](#)

"מחלקות" – קבצים בתוכנה המוטסת

התכנות הוא בשפת C כך שאין דבר כזה מחלקות אך רוב הקבצים סוג של מתפקדים כמחלקה. לכל אחת יש את הפונקציות שלה והן מחוברות בסוף ב-main.

אני מסבירה רק על הפונקציות שנמצאות ב-headers (פונקציות חיצוניות) שזה כמעט הכל אבל קצת פחות. אך גם יהיו את המשתנים הגלובליים של כל קובץ (לא ה-struct וכדומה שהם חיצוניים).

תחת תיקיית src**- main.c**

יוצר את החיבור למערכת ההפעלה, מה היא מריצה ומריצים שם את הלולאה האין סופית של הלוויין - שפועלת כל הזמן עד אתחול ואז אחרי init חוזרת לפעול. פונקציית ה-init תהיה מוסברת בקובץ של הפונקציה.

- InitSystem.h

אחריות הקובץ: בתחילת הרצה עושה init לכל המערכות ומבצע הפעלה ראשונית כאשר פעם ראשונה בחלל.

○ **שם הפעולה: StartFRAM**

- **טענת כניסה:** אין
- **טענת יציאה:**

- מחזירה ערך מסוג int – מצב הפעלת ה-FRAM:

○ 0 – הפעלה הצליחה

○ ערך שלילי – אירעה שגיאה בהפעלה

- הפעולה מאתחלת את ה-FRAM באמצעות הדרייברים ובודקת אם קיימות שגיאות.

○ **שם פעולה: WriteDefaultValuesToFRAM**

- **טענת כניסה:** אין
- **טענת יציאה:**

- מחזירה ערך מסוג int:

○ 0 – הפעלה הצליחה

○ ערך שלילי – אירעה שגיאה. מספר שלילי לפי כמות הבעיות.

- הפעולה כותבת את ברירות המחדל לכתובות המתאימות ב-FRAM.

○ **שם פעולה: StartI2C**

- **טענת כניסה:** אין
- **טענת יציאה:**

- מחזירה ערך מסוג int – מצב הפעלת ה-I2C:

○ 0 – הפעלה הצליחה

○ ערך שלילי – אירעה שגיאה בהפעלה

- הפעולה מאתחלת את ממשק ה-I2C באמצעות הדרייברים ובודקת שגיאות.

○ **שם פעולה: StartSPI**

- **טענת כניסה:** אין
- **טענת יציאה:**

- מחזירה ערך מסוג int – מצב הפעלת ה-SPI:

○ 0 – הפעלה הצליחה

- ערך שלילי – אירעה שגיאה בהפעלה
- הפעולה מאתחלת את ממשק ה-SPI באמצעות הדרייברים ובודקת שגיאות.
- **שם פעולה:** StartTime
- **טענת כניסה:** אין
- **טענת יציאה:**
- מחזירה ערך מסוג int – מצב הפעלת מודול הזמן:
 - 0 – הפעלה הצליחה
 - ערך שלילי – אירעה שגיאה בהפעלה
- הפעולה מאתחלת את מודול הזמן של המערכת באמצעות הדרייברים ובודקת אם קיימות שגיאות.
- **שם פעולה:** UpdateTime
- **טענת כניסה:** אין
- **טענת יציאה:**
- מחזירה ערך מסוג int – מצב העדכון בהתאם ל-
 - Time_setUnixEpoch
 - 0 – הצלחה
 - ערך שלילי – אירעה שגיאה
- הפעולה מעדכנת את זמן הלוויין בהתאם לזמן העדכני ביותר הזמין במערכת.
- **שם פעולה:** InitSupervisor
- **טענת כניסה:** אין
- **טענת יציאה:**
- מחזירה ערך מסוג int – מצב אתחול מודול המפקח:
 - 0 – אתחול הצליח
 - ערך שלילי – אירעה שגיאה
- הפעולה מאתחלת את מודול Supervisor של המערכת באמצעות הדרייברים ובודקת אם קיימות שגיאות.
- **שם פעולה:** InitSubsystems
- **טענת כניסה:** אין
- **טענת יציאה:** תמיד מחזירה 0
- הפעולה מבצעת את כל אתחולי המערכות ותתי מערכות הנדרשות ובודקת אם קיימות שגיאות. בנוסף מפעילה הפעלה ראשונית בפעם הראשונה שבחלל.
- **שם פעולה:** AntArm
- **טענת כניסה:**
- side – uint8_t – הצד של האנטנות (0 או 1)
- **טענת יציאה:** 0 – הצלחה, 1 – שגיאה
- הפעולה מכינה את האנטנות לפי הצד שנבחר לפריסתן.
- **שם פעולה:** AntDeployment
- **טענת כניסה:**
- side – uint8_t – הצד של האנטנות (0 או 1)
- **טענת יציאה:** 0 – הצלחה, 1 – שגיאה

- הפעולה פורסת את האנטנות לפי הצד שנבחר.
- **שם פעולה:** FirstActivation
 - **טענת כניסה:** אין
 - **טענת יציאה:** 0 – הצלחה, 1 – שגיאה
 - הפעולה מבצעת את הליך הפריסה הראשוני של האנטנות והמערכות הנלוות, ומוודאת שהפריסה הושלמה בהצלחה.
 - הפונקציה בודקת שעבר לפחות 40 דקות מאז שהלוויין נדלק לראשונה לפני שהמערכת מנסה לפרוס אנטנות לראשונה.
 - באותו הזמן גם שומרת נתונים של המערכת (telemetry). ושולחת בסוף ACK כאשר הסתיים.
- **TLM_management.h**
 - אחריות הקובץ: פעולות עם מערכת הקבצים. כאשר משתמשים בה נכנסים לכאן.
 - **משתנים גלובליים (הם בקובץ c):**
 - **שם:** stopDump
 - **סוג:** Boolean
 - **מטרה:** במהלך dump יש אפשרות לעצור אותו. כאשר אומרים לעצור המשתנה הופך להחזיק TRUE.
 - **שם:** buffer
 - **סוג:** char[200*1000] – מערך גדול של תווים.
 - **מטרה:** כאשר קוראים מידע מה-SD אלו הקפיצות. זה לייעל ולא נקרא מלא מה-SD.
 - **שם פעולה:** Write2File
 - **טענות כניסה:**
 - data – מצביע כללי (*void) לנתונים שאותם יש לשמור בקובץ, ללא תלות בסוג הנתונים.
 - tlmType – משתנה מסוג tlm_type_t המגדיר את סוג הטלמטריה או מבנה הנתונים הנכתב לקובץ.
 - **טענות יציאה:**
 - הפעולה מחזירה ערך מסוג int בהתאם לתוצאת הביצוע:
 - 1- – שגיאה בפתיחת הקובץ
 - 2- – שגיאה בקבלת זמן הלוויין בפורמט שניות
 - 3- – שגיאה בקבלת זמן הלוויין כמבנה Time
 - 0 – הכתיבה לקובץ בוצעה בהצלחה
 - הפעולה כותבת נתונים לקובץ במערכת הקבצים, ויוצרת את הקובץ במידת הצורך אם אינו קיים. הנתונים נכתבים בהתאם לסוג הטלמטריה שהוגדר, וכוללים חותמת זמן של המערכת לצורכי תיעוד וניהול מידע.
 - **שם הפעולה:** DeleteTLMFiles
 - **טענות כניסה:**
 - tlmType – משתנה מסוג tlm_type_t המגדיר את סוג הטלמטריה שממנה יש למחוק קבצים.

- date – משתנה מסוג Time המייצג את זמן ההתחלה שממנו תתבצע מחיקת הקבצים.
- numOfDay – משתנה מסוג int המגדיר את מספר הימים למחיקה החל מתאריך ההתחלה.
- **טענת יציאה:** הפעולה מחזירה ערך מסוג int, המייצג את מספר הימים שנמחקו בהצלחה.
- הפעולה מוחקת קובצי טלמטריה מסוג מוגדר ממערכת הקבצים, החל מתאריך נתון ולמשך מספר ימים שנקבע, לצורך ניהול זיכרון ותחזוקת נתוני הטלמטריה.
- **שם הפעולה:** DeleteTLMFile
- **טענת כניסה:**
 - tlmType – משתנה מסוג tlm_type_t המגדיר מאיזה תת-מערכת או סוג טלמטריה יש למחוק את הקובץ
 - date – משתנה מסוג Time המציין את הזמן שממנו יש למחוק את הקובץ
 - days2Add – משתנה מסוג int המשמש כמונה ימים להוספה למבנה האחרון לצורך חישוב טווח המחיקה
 - **טענת יציאה:** הפעולה מחזירה ערך מסוג int, המייצג קוד שגיאה בהתאם להנחיות במסמך התיעוד של המערכת (לפי עמוד 142 בקובץ [FAT](#)).
 - הפעולה מוחקת קובץ טלמטריה מסוג מוגדר ממערכת הקבצים, בהתאם לזמן שנקבע ולמספר הימים שנקבעו להוספה, ומחזירה קוד שגיאה אם נתקלת בבעיה במהלך המחיקה.
 - **שם הפעולה:** GetTlmTypeInfo
 - **טענות כניסה:**
 - tlmType – משתנה מסוג tlm_type_t המגדיר את סוג הטלמטריה הרצויה לצורך פעולות שמירה, קריאה או מחיקה.
 - **טענות יציאה:**
 - endFileName – מצביע למחרוזת (*char) שבה יישמר סיומת הקובץ (.fileEnd). המתאימה לסוג הטלמטריה
 - structSize – מצביע למשתנה מסוג int שבו יישמר גודל המבנה המתאים בסיביות (Bytes) עבור הנתונים שיישמרו
 - הפעולה מספקת מידע חשוב אודות סוג הטלמטריה, כולל סיומת הקובץ וגודל המבנה המתאים, כדי לאפשר פעולות שמירה, קריאה ומחיקה של הנתונים במערכת הקבצים בצורה מדויקת ובטוחה.
 - **שם הפעולה:** CalculateFileName
 - **טענות כניסה:**
 - date – משתנה מסוג Time המייצג את הזמן של הנתונים שאותם רוצים לשמור, לקרוא או למחוק.
 - endFileName – מצביע למחרוזת (*char) המייצגת את סיומת הקובץ (.fileEnd). המתאימה עבור סוג הנתונים הרצוי.
 - days2Add – משתנה מסוג int המשמש להוספת מספר ימים לחישוב השם הסופי של הקובץ (למשל כאשר מורידים מספר ימים).

■ טענת יציאה:

- file_name – מצביע למחרוזת (*char) שבה יישמר השם הסופי של הקובץ לאחר החישוב
- הפעולה מחשבת את השם הסופי של הקובץ לפי הזמן הנתון, סיומת הקובץ וספרת הימים להוספה, ומחזירה את השם הסופי במשתנה file_name.
- פעולה זו משמשת לצורך שמירה, קריאה או מחיקה של הנתונים במערכת הקבצים בצורה עקבית ובטוחה.
- שם הפעולה: ReadTLMFile

■ טענות כניסה:

- tlmType – משתנה מסוג tlm_type_t המייצג את סוג הטלמטריה שברצוננו לקרוא ולשלוח
- date – משתנה מסוג Time המייצג את זמן ההתחלה לקריאה/dump של הנתונים
- days2Add – משתנה מסוג int המייצג מספר ימים שיש להוסיף ל-date כדי לקרוא נתונים מהימים הבאים
- cmd_id – מזהה פקודה (int) המייצג את סוג הפקודה או ההקשר של השידור
- resolution – משתנה מסוג int המציין את רזולוציית השידור, כלומר כמה אלמנטים לדלג לפני שליחת אלמנט הבא
- טענת יציאה: הפעולה מחזירה ערך מסוג int, המייצג את מספר האלמנטים שנשלחו בהצלחה לבסיס הקרקע.
- הפעולה קוראת קובץ טלמטריה מסוג מוגדר ומעבירה את הנתונים לתחנת הקרקע באמצעות תקשורת RF, בהתאם לזמן ההתחלה, מספר הימים, מזהה הפקודה ורזולוציית השידור, ומחזירה את מספר האלמנטים שנשלחו.
- שם הפעולה: ReadTLMFiles

■ טענות כניסה:

- tlmType – משתנה מסוג tlm_type_t המייצג את סוג הטלמטריה שיש לקרוא
- startDate – משתנה מסוג Time המייצג את זמן ההתחלה לקריאה
- numOfDay – משתנה מסוג int המייצג את מספר הימים לקריאה, כלומר כמה קבצים מאותו סוג יש לקרוא מהתאריך הנתון
- cmd_id – מזהה פקודה (int) המייצג את סוג הפקודה או ההקשר של הקריאה
- resolution – משתנה מסוג int המייצג את הרזולוציה (בשניות) של הטלמטריה באותו יום לקריאה
- טענת יציאה: הפעולה מחזירה ערך מסוג int, המייצג את מספר האלמנטים שנקראו בהצלחה.
- הפעולה קוראת מספר קבצי טלמטריה מאותו סוג, החל מתאריך ההתחלה שנבחר, לפי מספר הימים והרזולוציה הנתונה, ומחזירה את מספר האלמנטים שנקראו. פעולה זו מאפשרת גישה מסודרת לקריאת הטלמטריה לפי טווחי זמן מוגדרים.
- שם הפעולה: GetPeriodTimeAccordingToTlmType

- **טענות כניסה:**
 - type – משתנה מסוג tlm_type_t המייצג את סוג הטלמטריה שאותה רוצים לשמור או לשלוח
- **טענת יציאה:**
 - הפעולה מחזירה ערך מסוג int המייצג את זמן המחזור (Period Time) המתאים לסוג הטלמטריה.
 - מחזירה 0 עבור סוגי לוגים או סוגים לא מזהים
- הפעולה מחזירה את זמן המחזור המתאים לשמירה או לשליחה של טלמטריה לפי סוג הנתונים, וכך מאפשרת תזמון אוטומטי וניהול יעיל של שמירת ושליחת נתוני הטלמטריה במערכת.
 - **שם הפעולה:** Delete_allTMFilesFromSD
- **טענת כניסה:** אין
- **טענת יציאה:**
 - הפעולה מחזירה ערך מסוג int:
 - 2- אם יש בעיה בקריאה מהכונן (f_getdrive)
 - 1- אם אירעה שגיאה במהלך פעולת הפורמט
 - ערך אחר – פעולת הפורמט הושלמה בהצלחה
- הפעולה מבצעת פורמט לכרטיס ה-SD שבו המערכת פועלת ולאחר מכן קוראת לפונקציית אתחול (init) של מערכת הקבצים. משמשת למחיקה כוללת של כל קבצי הטלמטריה ולניקוי הכונן לצורך שימוש מחדש.
 - **שם הפעולה:** InitializeFS
- **טענת כניסה:** אין
- **טענת יציאה:**
 - הפעולה מחזירה ערך מסוג FileSystemResult:
 - FS_FAIL – אם נכשלה אתחול מערכת הקבצים
 - FS_ALLOCATION_ERROR – אם קרתה שגיאה במהלך הקצאת זיכרון (malloc)
 - FS_SUCCESS – אם אתחול מערכת הקבצים הושלם בהצלחה
- הפעולה מבצעת אתחול מלא של מערכת הקבצים. פעולה נועדה להיקרא פעם אחת בעת אתחול המערכת או לאחר קריאה ל-DeInitializeFS (פעם אחת לאתחול).
 - **שם הפעולה:** DeInitializeFS
- **טענת כניסה:** אין
- **טענת יציאה:** אין
- מבצע ביטול אתחול מערכת הקבצים. פעולה זו נועדה להחזיר את מערכת הקבצים למצב לא פעיל.

- utils.h

○ שם הפעולה: timeU2time

▪ טענות כניסה:

- utime – משתנה מסוג time_unix המייצג זמן בפורמט Unix המיועד להמרה שזהו בעיקרון unsigned int.
- time – מצביע למבנה מסוג Time שאליו יישמר הזמן לאחר ההמרה.
- טענת יציאה: אין, לא מחזירה את הערך אך מעדכנת את המבנה.
- הפעולה ממירה זמן בפורמט Unix למבנה Time ומאחסנת את תוצאת ההמרה במבנה היעד.

○ שם פעולה: logError

▪ טענות כניסה:

- error – משתנה מסוג int המייצג את קוד השגיאה של התקלה שאירעה
- msg – מצביע למחרוזת (*char) המכילה הודעת שגיאה המתארת באיזו פונקציה ובאיזה שלב התרחשה התקלה.
(אורך מקסימלי: 66 תווים)

▪ טענות יציאה:

- הפעולה מחזירה ערך מסוג int, המייצג את קוד השגיאה שנרשם בלוג. מכיוון שאנחנו מחזירים את השגיאה שקיבלנו.
- הפעולה שומרת לוג שגיאה הכולל קוד שגיאה והודעת שגיאה במערכת הקבצים (File System), לצורך תיעוד, ניטור ואבחון תקלות במערכת.

- FRAM_FlightParameters.h

קובץ זה מכיל את רשימת הכתובות והגדלים של כל הנתונים הנשמרים ב-FRAM. הקובץ מבוסס על defines, ולכל פרמטר הנשמר ב-FRAM מוגדרת בו הכתובת המתאימה.

- GlobalStandards.h

קובץ זה מסופק על-ידי ספק החומרה ומכיל סטנדרטים כלליים למערכת. הקובץ נכתב במקור על-ידי הספק, אך קיימת אפשרות לבצע בו שינויים או להוסיף הגדרות חדשות בהתאם לצרכים שלנו.

- SysI2CAddr.h

קובץ זה מכיל את כתובות ה-I2C של רכיבי המערכת, כפי שהתקבלו מספק החומרה. הכתובות הוגדרו כ-defines לצורך גישה נוחה, קריאה ותחזוקה של הקוד.

תחת src\SubSystemModules

תחת PowerManagement

- EPS.h

אחריות הקובץ: הפעלת לוגיקת מערכת החשמל.

○ משתנים גלובליים (הם בקובץ c):

▪ שם: lastVoltage

• סוג: short - voltage_t

• מטרה: שמירת הערך הקודם עבור smoothing.

▪ שם: threshold_volts

• סוג: union – EpsThreshVolt_t של הספים

• מטרה: להחזיק את ספי המתחים אחרי שקראנו אותם לראשונה

מה-FRAM.

▪ שם: Alpha

• סוג: float

• מטרה: ערך ההחלקה. גם כן נקרא לראשונה מה-FRAM.

○ שם הפעולה: EPS_And_SP_Init

▪ טענת כניסה: אין

▪ טענת יציאה:

• 0 – על הצלחה

• -1 – כישלון ביצוע init

▪ הפעולה מבצעת init של מערכת החשמל והפאנלים הסולאריים.

○ שם הפעולה: EPS_Conditioning

▪ טענת כניסה: אין

▪ טענת יציאה:

• 0 – הצלחה

• -1 – כישלון

▪ הפעולה מממשת את לוגיקת ה-EPS ושולטת במכונת המצבים של תתי המערכות (הפעלה/כיבוי), על בסיס מתח הסוללה בלבד (שנקבע בשילוב ערך alpha).

○ שם הפעולה: GetBatteryVoltage

▪ טענת כניסה:

• vbat – מצביע מסוג voltage_t* אליו יישמר מתח הסוללה הנוכחי.

(short voltage_t הוא מסוג)

▪ טענת יציאה:

• 0 – הצלחה

• קוד שגיאה בהתאם ל-satellites-subsystem/common_types.h

▪ הפעולה קוראת את מתח הסוללה הנוכחי ומחזירה אותו דרך הפרמטר

vbat.

○ שם הפעולה: UpdateThresholdVoltages

▪ טענת כניסה:

- thresh_volts – union מסוג EpsThreshVolt_t המכיל ערכי סף חדשים ללוגיקת ה-EPS.
- **טענת יציאה:**
 - 0 – הצלחה
 - -1 – כשל בעדכון ערכי הסף
 - -2 – ערכי סף חדשי לא תקינים
 - קוד שגיאה בהתאם ל-hal/errors.
- הפעולה מעדכנת את ערכי מתחי הסף של לוגיקת ה-EPS בזיכרון ה-FRAM, ובקוד כרגע.
 - **שם הפעולה:** GetThresholdVoltages
 - **טענת כניסה:**
 - thresh_volts – מצביע למבנה שיקבל את ערכי מתחי הסף.
 - **טענת יציאה:**
 - 0 – הצלחה
 - -1 – מצביע קלט ריק
 - -2 – שגיאה בקריאה מה-FRAM
 - הפעולה קוראת את ערכי מתחי הסף של לוגיקת ה-EPS מזיכרון ה-FRAM.
 - **שם הפעולה:** GetAlpha
 - **טענת כניסה:**
 - alpha – מצביע למשתנה שיקבל את ערך מקדם ההחלקה
 - **טענת יציאה:**
 - 0 – הצלחה
 - -1 – מצביע קלט ריק
 - -2 – שגיאה בקריאה מ-FRAM
 - הפעולה מחזירה את ערך מקדם ההחלקה (alpha) המשמש לסינון מתח הסוללה.
 - **שם הפעולה:** UpdateAlpha
 - **טענת כניסה:**
 - alpha – ערך חדש למקדם ההחלקה (בטווח 0–1)
 - **טענת יציאה:**
 - 0 – הצלחה
 - -1 – כשל בעדכון הערך
 - -2 – ערך alpha לא חוקי
 - -4 – שגיאה בקריאה מ-FRAM
 - -5 – נתונים שנכתבו אינם תואמים
 - הפעולה מעדכנת את מקדם ההחלקה (alpha) בזיכרון ה-FRAM וגם בקוד, המשמש במסנן מעבר נמוך (Low Pass Filter) לצורך ייצוב מדידת מתח הסוללה.

- EPSEOperationModes.h

אחריות הקובץ: אחראי על פונקציות המצבים של הלוויין.

○ משתנים גלובליים (הם בקובץ c):

▪ שם: txOff

• סוג: Boolean

• **מטרה:** דגל למערכת השידור. האם יכולה לפעול או לא.

▪ שם: payloadOff

• סוג: Boolean

• **מטרה:** דגל ל-payload האם פועל או כבוי בגלל סף מתח.

▪ שם: satState

• סוג: enum – EpsState_t של מצבי הלוויין

• **מטרה:** להחזיק את מצב הלוויין כדי שלא נצטרך לעשות את כל

הפעולות אם אנחנו יודעים שאנחנו כבר מראש במצב זה או כדי

לקבל אותו אחר כך.

○ שם הפעולה: EnterOperationalMode

▪ טענת כניסה: אין

▪ טענת יציאה: מחזיר 0

▪ מעביר את הלוויין למצב Operational, או משאיר אותו במצב זה. זה המצב

בו הכל דולק. מגיע מעל סף מתח מסוים.

○ שם הפעולה: BetweenOperationalToCruise

▪ טענת כניסה: אין

▪ טענת יציאה: מחזיר 0

▪ בודק באיזה מצב הלוויין ולפי כך מחליט עם הוא צריך להיות ב-Operational

או ב-Cruise. (בודק באיזה מצב הוא כרגע ונשאר שם)

○ שם הפעולה: EnterCruiseMode

▪ טענת כניסה: אין

▪ טענת יציאה: מחזיר 0

▪ מעביר את הלוויין למצב Cruise, או משאיר אותו במצב זה. זה המצב בו

הכל חוץ מה-Payload דולק. מגיע מעל סף מתח מסוים ומתחת לאחר.

○ שם הפעולה: BetweenCruiseToPowerSafeMode

▪ טענת כניסה: אין

▪ טענת יציאה: מחזיר 0

▪ בודק באיזה מצב הלוויין ולפי כך מחליט עם הוא צריך להיות ב-Power Safe

או ב-Cruise. (בודק באיזה מצב הוא כרגע ונשאר שם)

○ שם הפעולה: EnterPowerSafeMode

▪ טענת כניסה: אין

▪ טענת יציאה: מחזיר 0

▪ מעביר את הלוויין למצב Power Safe Mode, או משאיר אותו במצב זה. זה

המצב בו הכל חוץ מה-Payload וה-Tx דולק. מגיע מתחת סף מתח מסוים.

○ שם הפעולה: GetSystemState

▪ טענת כניסה: אין

- **טענת יציאה:** מחזיר 0/1/2 לפי ה-enum של מצב הלוויין הנוכחי.
- הפעולה מחזירה את מצב המערכת הנוכחי של EPS.
- **שם הפעולה:** GetTxFlag
 - **טענת כניסה:** אין
 - **טענת יציאה:**
 - TRUE – הדגל דלוק (אסור לשדר הרבה)
 - FALSE – הדגל כבוי (מותר לשדר)
- הפעולה מחזירה את מצב דגל השידור, לשימוש בקוד TRXVU לבדיקת הרשות לשליחה.
- **שם הפעולה:** GetPayloadFlag
 - **טענת כניסה:** אין
 - **טענת יציאה:**
 - TRUE – הדגל דלוק (payload כבוי)
 - FALSE – הדגל כבוי (payload דלוק)
- הפעולה מחזירה את מצב דגל המטען, לצורך בדיקות מערכת ומצבי עבודה שונים.

תחת Maintenance

- Maintenance.h

- אחריות הקובץ: מערכת תפעול כולל. לא ממש קשור למערכת ספציפית אלא יותר לניהול הלוויין.
- משתנים גלובליים (הם בקובץ c.):
 - **שם:** flag
 - **סוג:** Boolean
 - **מטרה:** אחרי שעברנו את בדיקת התקינות כדי שלא נצטרך לעשות אותה עוד מפועל ה-flag.
 - **שם הפעולה:** DeleteOldFiles
 - **טענת כניסה:** אין
 - **טענת יציאה:** אין
 - הפעולה מוחקת קבצי טלמטריה ישנים מכרטיס ה-SD בהתאם לנפח האחסון הפנוי. כאשר נפח האחסון יורד מתחת לסף המינימלי (20%), מתחילה מחיקה עד להגעה ל-25% מקום פנוי לפחות. (הסבר מלא בבעיה האלגוריתמית)
 - **שם הפעולה:** CheckExecutionTime
 - **טענות כניסה:**
 - prev_time – זמן הביצוע האחרון (בפורמט unix)
 - period – פרק הזמן המינימלי בין ביצועים
 - **טענת יציאה:**
 - TRUE – אם הזמן שחלף גדול מה-period
 - FALSE – אחרת
 - הפעולה בודקת האם חלף פרק הזמן הנדרש מאז זמן הביצוע האחרון של פעולה מחזורית, או בפעולה שצריכה להיות לכמות זמן מסוימת.

- שם הפעולה: IsFS_Corrupted
 - טענת כניסה: אין
 - טענת יציאה:
 - TRUE – קיימת פגיעה או שחיתות במערכת הקבצים
 - FALSE – מערכת הקבצים תקינה
 - הפעולה בודקת את תקינות מערכת הקבצים ומזהה אם יש פגיעה בזיכרון.
- שם הפעולה: KickGroundCommWDT
 - טענת כניסה: אין
 - טענת יציאה: אין
 - הפעולה מאפסת את ה-Watchdog של התקשורת עם תחנת הקרקע בעקבות תקשורת שהתקבלה.
- שם הפעולה: IsGroundCommunicationWDTReset
 - טענת כניסה: אין
 - טענת יציאה:
 - TRUE – נדרש איפוס תקשורת עקב חוסר תקשורת ממושך
 - FALSE – אין צורך באיפוס
 - הפעולה בודקת האם זמן התקשורת האחרון עם תחנת הקרקע חרג מהזמן המקסימלי המותר.
- שם הפעולה: WakeupFromResetCMD
 - טענת כניסה: אין
 - טענת יציאה:
 - 0 – הצלחה
 - קוד שגיאה בהתאם ל-hal/errors.h
 - הפעולה מטפלת בהתעוררות המערכת לאחר פקודת איפוס, מנקה את דגל האיפוס, מעלה את ה-counters ושולחת אישור (ACK) עם זמן ה-unix הנוכחי. מטפלת גם ברגיל אך פשוט לא נוגעת בדברים חוץ מלעלות את ה-counter.
- שם הפעולה: Maintenance
 - טענת כניסה: אין
 - טענת יציאה: אין
 - הפעולה מפעילה סדרת פונקציות תחזוקה של המערכת בסדר קבוע, כולל בדיקות תקינות, תקשורת וניהול משאבים.
- שם הפעולה: MostCurrentTimeToFRAM
 - טענת כניסה: אין
 - טענת יציאה: אין
 - הפעולה שומרת את זמן הלוויין העדכני ביותר בזיכרון ה-FRAM, לצורך שחזור זמן לאחר אתחול.
- שם הפעולה: NeedToKillPayload
 - טענת כניסה: אין
 - טענת יציאה: אין

- הפעולה בודקת האם יש צורך לכבות לצמיתות את המטען (Payload) בעקבות תנאי מערכת חריגים. זהו רכיב שלא נוסה בעבר בחלל אז יש לנקוט באמצעי זהירות כדי שלא יהרוס את כל הלוויין.

תחת Housekeeping

- TelemetryFiles.h

קובץ זה מגדיר את שמות הסימונים של קבצי הטלמטריה השונים במערכת, וכן את סוגי הטלמטריה באמצעות enum לשימוש עקבי וברור בקוד. ההגדרות בקובץ מאפשרות ניהול אחיד, קריא ותחזוקתי של קבצי הטלמטריה והרכיבים השונים.

- TelemetryCollector.h

אחריות הקובץ: קובץ זה מכיל את ההגדרות והממשקים לפונקציות ניהול טלמטריה במערכת, כולל איסוף נתוני טלמטריה, ארגון, ושמירתם לקבצים המתאימים. הפונקציות בקובץ אחראיות על טיפול בנתוני טלמטריה מתתי המערכות והמטענים השונים, וכן על שליפת נתונים מה-FRAM וניהול לוגיקת האיסוף.

○ משתנים גלובליים (הם בקובץ c):

▪ שם: lastTimeSave

• סוג: time_unix[9]

• מטרה: מתי הייתה הפעם האחרונה ששמרנו מידע בכל מערכת כדי שהלוויין יוכל לבדוק עם הוא צריך לשמור שוב.

○ שם הפעולה: InitSavePeriodTimes

▪ טענת כניסה: אין

▪ טענת יציאה: אין

▪ הפעולה קוראת את זמני המחזור לשמירת טלמטריה מכל הסוגים מזיכרון ה-FRAM ומאתחלת את ערכי התזמון במערכת.

○ שם הפעולה: TelemetryCollectorLogic

▪ טענת כניסה: אין

▪ טענת יציאה: אין

▪ הפעולה מנהלת את לוגיקת איסוף הטלמטריה של המערכת ושומרת את כל סוגי הטלמטריה לקבצים המתאימים בהתאם לפרקי הזמן שהוגדרו.

○ שם הפעולה: TelemetrySaveEPS

▪ טענת כניסה: אין

▪ טענת יציאה: אין

▪ הפעולה שומרת את נתוני הטלמטריה הנוכחיים של מערכת ה-EPS לקובץ ייעודי.

○ שם הפעולה: TelemetrySaveTx

▪ טענת כניסה: אין

▪ טענת יציאה: אין

▪ הפעולה שומרת את נתוני הטלמטריה הנוכחיים של מערכת ה-Tx לקובץ ייעודי.

○ שם הפעולה: TelemetrySaveRx

▪ טענת כניסה: אין

▪ טענת יציאה: אין

▪ הפעולה שומרת את נתוני הטלמטריה הנוכחיים של מערכת ה-Rx לקובץ ייעודי.

○ שם הפעולה: TelemetrySaveAnt0

▪ טענת כניסה: אין

- **טענת יציאה:** אין
- הפעולה שומרת את נתוני הטלמטריה הנוכחיים של מערך אנטנות מספר 0 לקובץ ייעודי.
- **שם הפעולה:** TelemetrySaveAnt1
- **טענת כניסה:** אין
- **טענת יציאה:** אין
- הפעולה שומרת את נתוני הטלמטריה הנוכחיים של מערך אנטנות מספר 1 לקובץ ייעודי.
- **שם הפעולה:** TelemetrySaveSolarPanels
- **טענת כניסה:** אין
- **טענת יציאה:** אין
- הפעולה שומרת את נתוני הטלמטריה הנוכחיים של הפאנלים הסולאריים (טמפרטורת כל פאנל) לקובץ ייעודי.
- **שם הפעולה:** TelemetrySavePayloadRADFET
- **טענת כניסה:** אין
- **טענת יציאה:** אין
- הפעולה שומרת את נתוני הטלמטריה הנוכחיים של ה-RADFET, אחד ממטעני הלוויין, לקובץ ייעודי.
- **שם הפעולה:** TelemetrySavePayloadEvents
- **טענת כניסה:** אין
- **טענת יציאה:** אין
- הפעולה שומרת את נתוני הטלמטריה של אירועי המטען (Payload Events) לקובץ ייעודי קוראת גם לפונקציית SEL.
- **שם הפעולה:** TelemetrySaveWOD
- **טענת כניסה:** אין
- **טענת יציאה:** אין
- הפעולה שומרת את נתוני ה-WOD נוכחיים לקובץ ייעודי.
- **שם הפעולה:** GetCurrentWODTelemetry
- **טענת כניסה:**
 - *wod – מצביע למבנה מסוג WOD_Telemetry_t שאליו יישמרו נתוני ה-WOD (במקרה של שגיאה, שדות הנתונים ימולאו בערך 1-)
- **טענת יציאה:** מחזיר 0
- הפעולה אוספת את כל נתוני הטלמטריה הדרושים ומארגנת אותם במבנה WOD לצורך שמירה או שידור.
- **שם הפעולה:** GetSEL_telemetry
- **טענות כניסה:**
 - eventsData – נתוני קריאה ממטען האירועים (SEL/SEU)
 - selData – מצביע למבנה שיקבל את נתוני ה-SEL
- **טענת יציאה:** אין
- הפעולה מעבדת את נתוני האירועים של המטען וממירה אותם למבנה SEL לשמירה.
- **שם הפעולה:** TelemetrySavePayloadSEL

- **טענת כניסה:**
 - eventData – נתוני קריאה מטע"ד האירועים (SEL/SEU)
- **טענת יציאה:** אין
- הפעולה שומרת את נתוני הטלמטריה של ה-SEL לקובץ ייעודי, לאחר קריאה לפונקציית עיבוד הנתונים.
 - **שם הפעולה:** IsThePayloadOn
- **טענת כניסה:** אין
- **טענת יציאה:**
 - TRUE – המטען דולק
 - FALSE – המטען כבוי או שגיה
- הפעולה בודקת את מצב ערוץ ה-EPS וקובעת האם ה-payload דלוק.

תחת Communication

- SatCommandHandler.h
 - אחריות הקובץ: הגדרת מבני הנתונים, הקבועים והממשקים לניהול פקודות הלוויין, כולל פירוק והרכבה של פקודות, אחסון וניהול פקודות מושהות, וביצוע פקודות בהתאם לסוגן. הקובץ משמש כשכבת התווך בין נתוני התקשורת הנכנסים לבין לוגיקת ביצוע הפקודות במערכת.
 - **שם הפעולה:** ParseDataToCommand
 - **טענת כניסה:**
 - data – מצביע מסוג unsigned char* לחבילת הנתונים שהתקבלה לאחר קריאה מה-frame, ללא כותרות פרוטוקול AX.25.
 - *cmd – מצביע למבנה sat_packet_t שאליו יוזנו כל שדות הפקודה המפוענחת. (שדות הפרוטוקול)
 - **טענת יציאה:**
 - מחזירה ערך מסוג CMD_ERR – קוד שגיה בהתאם להגדרות המערכת.
 - הפעולה ממירה חבילת נתונים גולמית שהתקבלה מהתקשורת למבנה פקודה פנימי (sat_packet_t) הכולל את כל שדות הפקודה הדרושים לביצוע.
 - **שם הפעולה:** AssembleCommand
 - **טענת כניסה:**
 - data – מצביע מסוג unsigned char* לנתוני הפקודה
 - data_length – משתנה מסוג unsigned short המייצג את אורך שדה הנתונים בבתים
 - type – משתנה מסוג char המגדיר את סוג הפקודה (תת המערכת השולחת)
 - subtype – משתנה מסוג char המגדיר את תת סוג הפקודה בתוך תת המערכת
 - id – משתנה מסוג unsigned int המייצג את מזהה הפקודה, כולל מזהה הלוויין ושדות נוספים
 - *cmd – מצביע למבנה sat_packet_t שיכיל את הפקודה המורכבת

- **טענת יציאה:**
 - מחזירה ערך מסוג CMD_ERR – קוד שגיאה בהתאם להגדרות הפקודות במערכת.
 - הפעולה יוצרת מבנה פקודה מסוג sat_packet_t מתוך הפרמטרים שהתקבלו, כך שניתן יהיה לשלוח את הפקודה או לעבד אותה במערכת.
 - **שם הפעולה:** ActUponCommand
 - **טענת כניסה:**
 - cmd – מצביע למבנה sat_packet_t המכיל את הפקודה לביצוע
 - **טענת יציאה:** מחזירה ערך מסוג int – קוד שגיאה בהתאם ל-hal/errors.h.
 - הפעולה שולחת לביצוע הפקודה בהתאם לסוגה ותת-סוגה. אם סוג הפקודה אינו מוכר, נשלח אישור שגיאה מתאים (ACK).
- TRXVU.h**
- אחריות הקובץ: הגדרת הממשקים, הקבועים ומבני הנתונים של מערכת התקשורת (TRXVU) והאנטנות בלוויין. הקובץ אחראי על ניהול לוגיקת השידור והקליטה, טיפול במצבי עבודה כגון Beacon, Idle, Mute ו-Transponder, אינטראקציה עם ה-FRAM, ושליטה בתהליכי תקשורת, פקודות ו-dumps בהתאם למצב המערכת ותנאי ההפעלה.
- **משתנים גלובליים (הם בקובץ c):**
 - **שם:** continueDeploy
 - **סוג:** Boolean
 - **מטרה:** דגל האם אנחנו עדיין צריכים להמשיך לפרוס אנטנות.
 - **שם:** lastTimeSendingBeacon
 - **סוג:** time_unix
 - **מטרה:** כדי לבדוק אם עבר מספיק זמן בין 2 שידורים
 - **שם:** period
 - **סוג:** unsigned int
 - **מטרה:** כל כמה זמן צריך לשדר Beacon. נלקח מה-FRAM.
 - **שם הפעולה:** getIdleEndTime
 - **טענת כניסה:** אין
 - **טענת יציאה:**
 - מחזירה ערך מסוג time_unix:
 - 0 – כשל בקריאה
 - ערך זמן – זמן סיום מצב Idle
 - הפעולה קוראת מזיכרון ה-FRAM את זמן הסיום של מצב Idle במערכת.
 - **שם הפעולה:** setMuteEndTime
 - **טענת כניסה:**
 - endTime – זמן סיום מצב Mute בפורמט unix
 - **טענת יציאה:**
 - מחזירה int:
 - 0 – הצלחה
 - -1 – שגיאה
 - -2 – נכתב ערך שגוי

- הפעולה מגדירה ושומרת ב-FRAM את זמן סיום מצב ה-Mute ומוודאת שהערך נכתב כראוי.
- שם הפעולה: `getMuteEndTime`
 - טענת כניסה: אין
 - טענת יציאה:
 - מחזירה ערך מסוג `time_unix`:
 - 0 – כשל בקריאה
 - ערך זמן – זמן סיום מצב Mute
- הפעולה קוראת מזיכרון ה-FRAM את זמן סיום מצב ה-Mute.
- שם הפעולה: `setNewBeaconIntervalToPeriod`
 - טענת כניסה: אין
 - טענת יציאה: מחזירה `int` – קוד שגיאה בהתאם ל-`hal/errors.h`
 - הפעולה מעדכנת את מחזור שידור ה-Beacon בהתאם להגדרות העדכניות.
- שם הפעולה: `getTransponderEndTime`
 - טענת כניסה: אין
 - טענת יציאה:
 - מחזירה ערך מסוג `time_unix`:
 - 0 – כשל בקריאה
 - ערך זמן – זמן סיום מצב Transponder
- הפעולה קוראת מזיכרון ה-FRAM את זמן סיום פעולת ה-Transponder.
- שם הפעולה: `setTransponderRSSIinFRAM`
 - טענת כניסה:
 - `val` – ערך RSSI לשמירה
 - טענת יציאה:
 - 0 – הצלחה
 - -1 – שגיאה
 - -2 – ערך שגוי
 - -3 – שגיאת כתיבה ב-I2C
- הפעולה שומרת את ערך ה-RSSI של ה-Transponder בזיכרון ה-FRAM.
- שם הפעולה: `getTransponderRSSIfromFRAM`
 - טענת כניסה: אין
 - טענת יציאה:
 - מחזירה `short`:
 - ערך RSSI
 - -1 – שגיאה
- הפעולה קוראת מזיכרון ה-FRAM את ערך ה-RSSI של המשיב.
- שם הפעולה: `InitTxModule`
 - טענת כניסה: אין
 - טענת יציאה: אין
 - הפעולה מאתחלת הפרמטרים הנחוצים אחרי אתחול, כולל סמפורים ופרמטרים הנקראים מה-FRAM.

- **שם הפעולה:** SetNeedToStopAntDeploy
 - **טענת כניסה:** אין
 - **טענת יציאה:** אין
 - הפעולה מגדירה ב-FRAM שיש להפסיק ניסיונות פריסת אנטנות.
- **שם הפעולה:** InitTrxvuAndAnts
 - **טענת כניסה:** אין
 - **טענת יציאה:** מחזירה int – קוד שגיאה בהתאם ל-hal/errors.h
 - הפעולה מאתחלת את יחידת התקשורת TRXVU ואת מנגנון האנטנות.
- **שם הפעולה:** TRX_Logic
 - **טענת כניסה:** אין
 - **טענת יציאה:** מחזירה מספר שגיאה אם יש, 0 במקרה של הצלחה.
 - הפעולה מממשת את לוגיקת המערכת תקשורת: שידור Beacon, בדיקת חבילות נכנסות, קריאת פקודות ועוד. הלוגיקה המרכזית.
- **שם הפעולה:** turnOffTransponder
 - **טענת כניסה:** אין
 - **טענת יציאה:**
 - מחזירה int:
 - 0 – הצלחה
 - -1 – זמן הסיום טרם עבר
 - קוד שגיאה אחר
 - הפעולה בודקת האם זמן פעולת המשיב הסתיים, ואם כן – מכבה אותו.
- **שם הפעולה:** turnOffIdle
 - **טענת כניסה:** אין
 - **טענת יציאה:**
 - מחזירה int:
 - 0 – הצלחה
 - -1 – שגיאה בקריאת זמן
 - קוד שגיאה אחר
 - הפעולה בודקת האם הסתיים זמן מצב Idle ומוציאה את המערכת ממצב זה.
- **שם הפעולה:** setTransponderOn
 - **טענת כניסה:** אין
 - **טענת יציאה:** מחזירה int – קוד שגיאה בהתאם ל-I2C.
 - הפעולה מדליקה את מצב המשיב דרך ה-I2C.
- **שם הפעולה:** setTransponderOff
 - **טענת כניסה:** אין
 - **טענת יציאה:** מחזירה int – קוד שגיאה בהתאם ל-I2C.
 - הפעולה מכבה את מצב המשיב דרך ה-I2C.
- **שם הפעולה:** setTransponderEndTime
 - **טענת כניסה:**
 - transponderEndTime – זמן סיום בפורמט unix.
 - **טענת יציאה:**

- 0 – הצלחה
- קוד שגיאה אחר.
- הפעולה שומרת ב-FRAM את זמן סיום פעולת המשיב.
- שם הפעולה: CheckDumpAbort
 - טענת כניסה: אין
 - טענת יציאה:
 - TRUE – יש לבטל Dump
 - FALSE – אין צורך
 - הפעולה בודקת מהתור האם יש צורך להפסיק תהליך Dump פעיל.
- שם הפעולה: CheckTransmitionAllowed
 - טענת כניסה: אין
 - טענת יציאה:
 - TRUE – מותר לשדר
 - FALSE – אסור לשדר
 - הפעולה בודקת האם מותר לבצע שידור, בהתאם למצב Mute.
- שם הפעולה: TransmitSplPacket
 - טענות כניסה:
 - packet – חבילת SPL לשידור
 - avalFrames – מצביע למספר frames זמינים
 - טענת יציאה:
 - 1- – פקודה ריקה
 - 2- – אסור לשדר
 - קוד שגיאה אחר
 - הפעולה משדרת חבילת SPL בפרוטוקול AX.25.
- שם הפעולה: BeaconLogic
 - טענת כניסה: אין
 - טענת יציאה:
 - 0 – הצלחה
 - 1- – לא אמור להישלח Beacon
 - קוד שגיאה אחר
 - הפעולה מנהלת את לוגיקת שידור ה-Beacon של הלוויין.
- שם הפעולה: SetIdleState
 - טענת כניסה:
 - state – לפי enum (isis_vu_e__onoff_t)
 - duration – משך זמן מצב Idle
 - טענת יציאה:
 - 0 – הצלחה
 - 1- – בשגיאה כללית
 - 2- – בעיה בקריאת FRAM
 - 3- – בעיה בכתיבה ל-FRAM
 - 4- – הוכנס זמן לא נכון

- 5- – שגיאה בקבלת זמן לוויין
 - 6- – state לא לפי ה-enum.
 - הפעולה מגדירה או מבטלת את מצב Idle של ה-TRXVU ושומרת את הנתונים ב-FRAM.
 - **שם הפעולה:** GetNumberOfFramesInBuffer
 - **טענת כניסה:** אין
 - **טענת יציאה:**
 - מספר פריימים במאגר
 - 1- – שגיאה
 - הפעולה מחזירה את מספר החבילות הממתינות במאגר הקליטה.
 - **שם הפעולה:** GetOnlineCommand
 - **טענת כניסה:**
 - cmd – מצביע למבנה פקודה
 - **טענת יציאה:** מחזיר לפי CMD_ERR
 - הפעולה שולפת פקודה מהמאגר וממלאת את מבנה sat_packet_t.
 - **שם הפעולה:** TransmitDataAsSPL_Packet
 - **טענות כניסה:**
 - cmd – פקודת בסיס
 - data – נתונים לשידור
 - length – אורך הנתונים
 - **טענת יציאה:** מחזירה קוד שגיאה מסוג int.
 - הפעולה אורזת נתונים כחבילת SPL ומשדרת אותם.
 - **שם הפעולה:** setBeaconIntervalNOT_FROM_FRAM
 - **טענת כניסה:**
 - interval – מרווח זמן לשידור Beacon
 - **טענת יציאה:** אין
 - הפעולה מגדירה מרווח שידור Beacon באופן זמני, ללא שימוש בערך השמור ב-FRAM.
- AckHandler.h** -
- אחריות קובץ: ניהול ושליחת חבילות אישור (ACK) בלוויין באמצעות פרוטוקול SPL. הקובץ מספק פונקציות לשליחת ACK עבור פקודות קיימות או ללא פקודה וללא נתונים.
- **שם הפעולה:** SendAckPacket
 - **טענות כניסה:**
 - acksubtype – סוג האישור שיש לשלוח (ack_subtype_t)
 - cmd – מצביע לפקודה שאליה התשובה היא ה-ACK
 - (sat_packet_t*) . ניתן לשלוח NULL אם אין פקודה קשורה
 - data – מצביע לנתונים שיש לשלוח עם ACK. ניתן לשלוח NULL
 - length – אורך הנתונים בשילוב עם ה-ACK, ב-bytes (unsigned short)
 - **טענת יציאה:** מחזירה int – קוד שגיאה בהתאם ל-<hal/errors.h>

- הפונקציה יוצרת ושולחת חבילת ACK בפרוטוקול SPL. ניתן לכלול נתונים נוספים בחבילה, וניתן לשלוח ACK גם ללא פקודה קשורה (cmd = NULL). החבילה נבנית בהתאם לסוג ה-ACK שנבחר (acksubtype).

- AckErrors.h

קובץ זה מגדיר קודי שגיאה המשמשים לשליחת חבילות אישור (ACK) במערכת. כל קוד מייצג סוג שגיאה שונה, כגון בעיות קריאה/כתיבה ב-FRAM, שגיאות ב-I2C, בעיות זמן או בעיות במבני נתונים, ומאפשר טיפול אחיד ומובנה בשגיאות בעת שליחת ACK או במהלך פעולות אחרות במערכת.

- SPL.h

קובץ זה מגדיר את פרוטוקול SPL המשמש לשליחת פקודות וחבילות אישור (ACK) בלוויין. הקובץ מכיל את סוגי הפקודות (spl_command_type_t) ותתי הסוגים שלהן עבור כל תת-מערכת – כגון EPS, TRXVU, קבצים, ניהול ומטענים – וכן סוגי acks שונים. הגדרות אלו מאפשרות תקשורת מסודרת, עקבית ומובנית בין תתי-המערכות והכנסת הפקודות והתגובות בצורה מובנת למערכת.

- CommandDictionary.h

אחריות הקובץ: מספק את הפונקציות להפניית פקודות שנשלחות אל הלוויין לתתי-המערכות המתאימות לפי תת-סוג הפקודה. כל פונקציה מקבלת פקודת sat_packet_t ומעבירה אותה ללוגיקה הרלוונטית של תת-המערכת, תוך טיפול בשגיאות ושליחת ACK מתאים במקרה של תת-סוג לא מוכר.

○ שם הפעולה: trxvu_command_router

▪ טענת כניסה:

- cmd – פקודה מהמערכת TRXVU ליישום (*sat_packet_t)

▪ טענת יציאה:

- מחזירה int:

○ קוד שגיאה בהתאם ל-<hal/errors.h>

○ מחזירה 1- אם cmd == NULL

- הפונקציה מנתבת את הפקודה שהתקבלה לפונקציה המתאימה במערכת התקשורת לפי סוג המשנה (subtype). אם סוג המשנה לא קיים, נשלח ACK של סוג "unknown subtype".

○ שם הפעולה: eps_command_router

▪ טענת כניסה:

- cmd – פקודה מהמערכת EPS ליישום (*sat_packet_t)

▪ טענת יציאה:

- מחזירה int:

○ קוד שגיאה בהתאם ל-<hal/errors.h>

○ מחזירה 1- אם cmd == NULL

- הפונקציה מנתבת את הפקודה שהתקבלה לפונקציה המתאימה במערכת החשמל לפי סוג המשנה המצוין בתוך cmd.

○ שם הפעולה: filesystem_command_router

▪ טענת כניסה:

- cmd – פקודה מהמערכת FS ליישום (*sat_packet_t)

▪ **טענת יציאה:**

- מחזירה int:

○ קוד שגיאה בהתאם ל-<hal/errors.h>

○ מחזירה 1- אם cmd == NULL

- הפונקציה מנתבת את הפקודה שהתקבלה לפונקציה המתאימה במערכת

ניהול הקבצים לפי סוג המשנה המצוין בתוך cmd.

○ **שם הפעולה:** management_command_router

▪ **טענת כניסה:**

- cmd – פקודה מהמערכת ה-OBC ליישום (*sat_packet_t)

▪ **טענת יציאה:**

- מחזירה int:

○ קוד שגיאה בהתאם ל-<hal/errors.h>

○ מחזירה 1- אם cmd == NULL

- הפונקציה מנתבת את הפקודה שהתקבלה לפונקציה המתאימה במערכת

הניהול לפי סוג המשנה המצוין בתוך cmd.

○ **שם הפעולה:** payload_command_router

▪ **טענת כניסה:**

- cmd – פקודה מהמערכת Payload ליישום (*sat_packet_t)

▪ **טענת יציאה:**

- מחזירה int:

○ קוד שגיאה בהתאם ל-<hal/errors.h>

○ מחזירה 1- אם cmd == NULL

- הפונקציה מנתבת את הפקודה שהתקבלה לפונקציה המתאימה במערכת

Payload לפי סוג המשנה המצוין בתוך cmd.

SubsystemCommands תחת

פקודות אלו תמיד נשלח ACK לקרקע, בין אם הצלחה, מה שמחפשים או כישלון (כישלון לפי

AckErrors.h). גם תמיד מוחזר 0 בהצלחה.

- TRXVU Commands.h

אחריות הקובץ: מכיל את מימוש הפקודות שאפשר לקבל מהקרקע עבור מערכת התקשורת.

○ **שם הפעולה:** CMD_SetOn_IdleState

▪ **טענת כניסה:**

- cmd – מצביע למבנה הפקודה. מכילה את כל המידע הנדרש

להפעלת מצב Idle, כולל משך הזמן והכותרות שנוספו ל-packet.

▪ **טענת יציאה:**

- -6 – הפרמטר cmd הוא NULL

- ערכים נוספים – בהתאם לרשימת השגיאות של SetIdleState

- ערך החזרה של SendAckPacket במקרה של הגעה עד שם.

- מוחזר 0 בהצלחה.

- הפעולה מעבירה את יחידת ה-TRXVU למצב Idle למשך זמן המוגדר

בנתונים שבפקודה.

- **שם הפעולה:** CMD_SetOff_IdleState
 - **טענת כניסה:**
 - cmd - מצביע למבנה הפקודה. מכילה את כל ה-headers הנחוצים.
 - **טענת יציאה:**
 - ערכים נוספים – בהתאם לרשימת השגיאות של SetIdleState
 - ערך החזרה של SendAckPacket במקרה של הגעה עד שם.
 - מוחזר 0 בהצלחה.
 - הפעולה מבטלת את הפעולה של מצב ה-idle.
- **שם הפעולה:** CMD_SetOn_Transponder
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה.
 - **טענת יציאה:**
 - 1- – כאשר cmd == NULL
 - 2- – נכתב ערך שגוי ל-FRAM
 - 3- – אורך נתונים שגוי
 - 4- – כשל בקריאה מה-FRAM
 - 5- – כשל בכתיבה ל-FRAM
 - ערכי שגיאה נוספים בהתאם ל-I2C_write
 - הפעולה מעבירה את המשדר למצב Transponder למשך הזמן שהוגדר בפקודה.
- **שם הפעולה:** CMD_SetOff_Transponder
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה.
 - **טענת יציאה:**
 - 0 בהצלחה
 - כל דבר אחר שגיאה.
 - הפעולה מכבה את מצב המשיב במשדר.
- **שם הפעולה:** CMD_SetRSSI_Transponder
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה.
 - **טענת יציאה:**
 - 1- – כאשר cmd == NULL
 - 2- – אורך הנתונים שגוי
 - ערכי שגיאה נוספים מ-setTransponderRSSlinFRAM
 - הפעולה משנה את ערך ה-RSSI של ה-Transponder ב-FRAM, בודקת שהערך עודכן בהצלחה, ומוודאת שהוא בתחום 0–4095.
- **שם הפעולה:** CMD_SetRSSI_Transponder_DEFAULT
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה
 - **טענת יציאה:** לפי CMD_SetRSSI_Transponder
 - הפעולה מאפסת את ערך ה-RSSI של המשיב לערך ברירת המחדל ושומרת אותו ב-FRAM. קוראת לפונקציה מקודם עם ערך ברירת המחדל.
- **שם הפעולה:** CMD_GetRSSI_Transponder
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה
 - **טענת יציאה:**

- 0 – הצלחה
- 1 – כשל קריאה מה-FRAM
- הפעולה קוראת את ערך ה-RSSI מה-FRAM ושולחת אותו חזרה למערכת הקרקע.
- **שם הפעולה:** CMD_MuteTRXVU
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה
 - **טענת יציאה:**
 - 1 – כאשר cmd == NULL
 - 3 – אורך נתונים שגוי
 - ערכי שגיאה נוספים מ-setMuteEndTime
 - הפעולה משתיקה את ה-TRXVU למשך זמן המוגדר בפקודה.
- **שם הפעולה:** CMD_UnMuteTRXVU
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה
 - **טענת יציאה:** מחזירה int שונה מ-0 במקרה של שגיאה.
 - הפעולה מבטלת את מצב ה-Mute של המשדר על ידי שינוי זמן סיום ההשתקה.
- **שם הפעולה:** CMD_SetBeacon_Interval
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה
 - **טענת יציאה:**
 - 1 – כאשר cmd == NULL
 - ערכי שגיאה נוספים בהתאם לבדיקות הטווח והכתיבה ל-FRAM.
 - הפעולה משנה את מרווח שידור ה-Beacon, מוודאת שהערך בתחום המותר ושומרת אותו ב-FRAM, ושולח אותו גם חזרה לקרקע.
- **שם הפעולה:** CMD_SetBeacon_Interval_DEFAULT
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה
 - **טענת יציאה:**
 - לפי CMD_SetBeacon_Interval
 - הפעולה משנה את מרווח שידור ה-Beacon לברירת מחדל.
- **שם הפעולה:** CMD_GetTxUptime
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה
 - **טענת יציאה:** קוד שגיאה בהתאם ל-<hal/errors.h>
 - הפעולה קוראת את זמן הפעולה המצטבר של המשדר ושולחת אותו לקרקע.
- **שם הפעולה:** CMD_GetRxUptime
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה
 - **טענת יציאה:** קוד שגיאה בהתאם ל-<hal/errors.h>
 - הפעולה קוראת את זמן הפעולה המצטבר של המקלט ושולחת אותו לקרקע.
- **שם הפעולה:** GetAntSide
 - **טענת כניסה:**
 - cmd - מצביע למבנה הפקודה
 - side – מצביע ל-char שיהיה בו את צד האנטנות.
 - **טענת יציאה:** 0 בהצלחה כל דבר אחר בשגיאה.

- פונקציית עזר לחילוף צד האנטנה מתוך חבילת הפקודה.
 - שם הפעולה: CMD_AntGetUptime
 - טענת כניסה: cmd - מצביע למבנה הפקודה
 - טענת יציאה:
 - 1 - כאשר cmd == NULL
 - 2 - אורך נתונים שגוי
 - 3 - צד אנטנה לא חוקי
 - ערכים נוספים מ- <hal/errors.h>
 - הפעולה מחזירה את זמן הפעולה של האנטנה בהתאם לצד המבוקש.
 - שם הפעולה: CMD_Ping
 - טענת כניסה: cmd - מצביע למבנה הפקודה
 - טענת יציאה: קוד שגיאה מ- <hal/errors.h>
 - הפעולה שולחת הודעת Ack (המכילה את ההודעה "pong") לבדיקת תקשורת.
 - שם הפעולה: CMD_OffAntsDeploy
 - טענת כניסה: cmd - מצביע למבנה הפקודה
 - טענת יציאה: קוד שגיאה מ- <hal/errors.h>
 - הפעולה המעלה את הדגל שמונע מהמשך ניסיון פריסה של האנטנות.
- EPS Commands.h
 - אחריות הקובץ: מכיל את מימוש הפקודות שאפשר לקבל מהקרקע עבור מערכת החשמל.
 - שם הפעולה: CMD_UpdateThresholdVoltages
 - טענת כניסה:
 - cmd - מצביע לפקודת לוויין (*sat_packet_t) הכוללת את כותרות הפקודה ואת ערכי ספי המתח החדשים.
 - טענת יציאה:
 - הפעולה מחזירה int:
 - 1 - כאשר cmd == NULL
 - ערכי שגיאה בהתאם ל-AckErrors.h (נשלח ACK בהתאם)
 - הפונקציה מעדכנת את ערכי ספי המתח של מערכת EPS בזיכרון ה-FRAM על פי הערכים שהתקבלו בפקודה.
 - שם הפעולה: CMD_GetThresholdVoltages
 - טענת כניסה: cmd - מצביע למבנה הפקודה
 - טענת יציאה:
 - 1 - שגיאה בקריאה מ-FRAM
 - שגיאות בהתאם ל-SendAckPacket
 - הפונקציה קוראת את ערכי ספי המתח של מערכת EPS מזיכרון ה-FRAM ומשדרת אותם לתחנת הקרקע.
 - שם הפעולה: CMD_RestoreDefaultThresholdVoltages
 - טענת כניסה: cmd - מצביע למבנה הפקודה
 - טענת יציאה: לפי CMD_UpdateThresholdVoltages

- הפונקציה משחזרת את ערכי ברירת המחדל של ספי המתח של מערכת EPS ושומרת אותם בזיכרון ה-FRAM.
- שם הפעולה: CMD_UpdateAlpha
 - טענת כניסה: cmd - מצביע למבנה הפקודה
 - טענת יציאה:
 - הפעולה מחזירה int:
 - -1 – כאשר cmd == NULL
 - ערכי שגיאה בהתאם ל-AckErrors.h (נשלח ACK בהתאם)
 - הפונקציה מעדכנת את מקדם ההחלקה (α) של סינון המתח במערכת EPS ושומרת אותו בזיכרון ה-FRAM.
 - שם הפעולה: CMD_RestoreDefaultAlpha
 - טענת כניסה: cmd - מצביע למבנה הפקודה
 - טענת יציאה: לפי CMD_UpdateAlpha
 - הפונקציה משחזרת את ערך ברירת המחדל של מקדם ההחלקה (α) במערכת EPS.
 - שם הפעולה: CMD_GetAlpha
 - טענת כניסה: cmd - מצביע למבנה הפקודה
 - טענת יציאה:
 - 1 – שגיאה בקריאה מ-FRAM
 - שגיאות בהתאם ל-SendAckPacket.
 - הפונקציה קוראת את ערך האלפא הנוכחי מזיכרון ה-FRAM ומשדרת אותו לתחנת הקרקע.
 - שם הפעולה: CMD_GetState
 - טענת כניסה: cmd - מצביע למבנה הפקודה
 - טענת יציאה: קוד שגיאה בהתאם ל-SendAckPacket.
 - הפונקציה מחזירה את מצב מערכת EPS הנוכחי של הלוויין לתחנת הקרקע.
 - שם הפעולה: CMD_EPS_ResetWDT
 - טענת כניסה: cmd - מצביע למבנה הפקודה
 - טענת יציאה:
 - 26 – לא ניתן לאפס את Watchdog של EPS
 - שגיאות בהתאם ל-TransmitDataAsSPL_Packet
 - הפונקציה מאפסת את Watchdog של מערכת EPS.
 - שם הפעולה: CMD_GetHeaterVal
 - טענת כניסה: cmd - מצביע למבנה הפקודה
 - טענת יציאה:
 - שגיאות בהתאם
 - isismeps2_ivid7_piu__getconfigurationparameter
 - שגיאות בהתאם ל-SendAckPacket
 - הפונקציה קוראת את ערכי המחממים מפרמטרי מערכת ה-EPS ומחזירה אותם.
 - שם הפעולה: CMD_SetHeaterVal

- **טענת כניסה:** cmd - מצביע למבנה הפקודה
- **טענת יציאה:**
 - שגיאות בהתאם
- isismepsv2_ivid7_piu__getconfigurationparameter
- שגיאות בהתאם ל-TransmitDataAsSPL_Packet
- הפונקציה מגדירה ערכי חימום חדשים במערכת EPS. הפונקציה לא מומלצת לשימוש.
- **FS Commands.h**
 - אחריות הקובץ: מכיל את מימוש הפקודות שאפשר לקבל מהקרקע עבור מערכת הקבצים.
 - **שם הפעולה:** CMD_DeleteTLM
 - **טענת כניסה:**
 - cmd – מצביע לפקודת לוויין (*sat_packet_t) הכוללת את סוג הטלמטריה, תאריך התחלה ותאריך סיום למחיקה.
 - **טענת יציאה:**
 - מחזירה int:
 - -1 – cmd == NULL
 - 17 – אורך נתונים שגוי
 - 60 – לא כל הקבצים נמחקו בהצלחה
 - שגיאות בהתאם ל-SendAckPacket
 - 0 – הצלחה
 - הפונקציה מוחקת קבצי טלמטריה מכרטיס ה-SD לפי סוג טלמטריה וטווח תאריכים שנשלחו בפקודה.
 - **שם הפעולה:** CMD_DeleteFilesOfType
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה
 - **טענת יציאה:**
 - מחזירה int:
 - -1 – cmd == NULL
 - 17 – אורך נתונים שגוי
 - 30 – סוג קובץ לא קיים
 - שגיאות בהתאם ל-SendAckPacket
 - 0 – הצלחה
 - הפונקציה מוחקת את כל קבצי הטלמטריה מכרטיס ה-SD לפי סוג טלמטריה נתון.
 - **שם הפעולה:** CMD_DeleteAllFiles
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה
 - **טענת יציאה:** קוד שגיאה בהתאם ל-SendAckPacket ול-Hard_ComponenetReset
 - הפונקציה מבצעת פירמוט מלא של כרטיס ה-SD ומוחקת את כל הקבצים הקיימים, בנוסף עושה אתחול ללוויין.
 - **שם הפעולה:** CMD_GetLastFS_Error
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה
 - **טענת יציאה:** קוד שגיאה בהתאם ל-SendAckPacket

- הפונקציה מחזירה לתחנת הקרקע את קוד השגיאה האחרון שנרשם במערכת הקבצים.
- **שם הפעולה:** CMD_SwitchSD_card
 - **טענת כניסה:**
 - cmd – מצביע לפקודת לוויין (*sat_packet_t) הכוללת את מספר כרטיס ה-SD
 - **טענת יציאה:**
 - -1 – cmd הוא NULL
 - 27 – אורך נתונים שגוי
 - -4 – כרטיס SD לא חוקי
 - -2 / -3 – שגיאת קריאה/כתיבה ל-FRAM
 - שגיאות בהתאם ל-Hard_ComponenetReset
- הפונקציה מחליפה בין כרטיסי SD פעילים במערכת, מעדכנת את המידע ב-FRAM ועושה אתחול למערכת.
- **שם הפעולה:** CMD_FreeSpace
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה
 - **טענת יציאה:**
 - 27 – שגיאה בשליפת נתונים מהמבנה
 - שגיאות בהתאם ל-SendAckPacket.
- הפונקציה מחזירה את נפח האחסון הפנוי בכרטיס ה-SD (בבתים), ברמת דיוק מוגבלת.
- **שם הפעולה:** CMD_StartDump
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה
 - **טענת יציאה:**
 - הפעולה מחזירה int:
 - -1 – cmd הוא NULL
 - 17 – אורך נתונים שגוי
 - 30 – תהליך Dump כבר פעיל
 - 0 – הצלחה
- הפונקציה מתחילה תהליך Dump של טלמטריה ויוצרת משימה ייעודית לביצוע הקריאה והשידור.
- **שם הפעולה:** CMD_SendDumpAbortRequest
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה
 - **טענת יציאה:** קוד שגיאה בהתאם ל-xQueueSend
 - הפונקציה שולחת בקשה לעצירת תהליך Dump פעיל.
- **שם הפעולה:** TrackDump
 - **טענת כניסה:**
 - dump – מצביע כללי (*void) למבנה הכולל את פרמטרי ה-Dump.
 - **טענת יציאה:** אין
 - מבצעת בפועל את לוגיקת ה-Dump של הטלמטריה בהתאם לפרמטרים שהועברו. ב-thread נפרד.

- **שם הפעולה:** CMD_SetTLMPeriodTimes
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה
 - **טענת יציאה:**
 - -1 – cmd הוא NULL
 - 17 – אורך נתונים שגוי
 - 30 – סוג טלמטריה לא חוקי
 - שגיאות בהתאם ל-SaveAndCheck
 - הפונקציה מעדכנת את תקופת שמירת הטלמטריה עבור סוג טלמטריה מסוים.
- **שם הפעולה:** CMD_SetTLMPeriodTimes_default
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה
 - **טענת יציאה:** קוד שגיאה בהתאם ל-FRAM_writeAndVerify
 - הפונקציה משחזרת את תקופות ברירת המחדל לשמירת טלמטריה.
- **שם הפעולה:** CMD_GetTLMPeriodTimes
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה
 - **טענת יציאה:**
 - קוד שגיאה בהתאם ל-FRAM_read
 - קוד שגיאה בהתאם ל-SendAckPacket
 - הפונקציה קוראת את תקופות שמירת הטלמטריה מזיכרון ה-FRAM ומשדרת אותן לתחנת הקרקע.
- Maintenance_Commands.h
 - אחריות הקובץ: מכיל את מימוש הפקודות שאפשר לקבל מהקרקע עבור ניהול הלוויין.
- **שם הפעולה:** CMD_UpdateSatTime
 - **טענת כניסה:**
 - cmd – מצביע לפקודת לוויין (*sat_packet_t) הכוללת את כותרות הפקודה ואת הזמן החדש להגדרה.
 - **טענת יציאה:**
 - מחזיר int:
 - -1 – cmd הוא NULL.
 - -3 – אורך נתונים שגוי
 - 1 – לא ניתן להגדיר זמן מחדש.
 - שגיאות בהתאם ל-<hal/errors.h>
 - 0 – הצלחה
 - הפונקציה מגדירה זמן חדש למחשב הלוויין (OBC) בהתאם לערך שהתקבל מהפקודה, מומלץ לא לעשות את זה הרבה (ובמיוחד לא לעבר) יכול לבלבל את הלוויין.
- **שם הפעולה:** CMD_GetSatTime
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה
 - **טענת יציאה:** קוד שגיאה, 0 במקרה של הצלחה.
 - הפונקציה קוראת את הזמן הנוכחי השמור בלוויין ומשדרת אותו לתחנת הקרקע.

- **שם הפעולה:** CMD_GetSatUptime
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה
 - **טענת יציאה:** קוד שגיאה בהתאם ל-SendAckPacket ול-<hal/errors.h>
 - הפונקציה מחזירה את זמן הפעולה המצטבר של ה-OBC מאז האתחול האחרון.
- **פעולות עזר לאתחול מערכת:**
 - **שם הפעולה:** HardPayload_ComponenetReset
 - **מטרה:** מבצעת איפוס חומרתי (Hard Reset) לרכיב ה-Payload בהתאם לפקודה שהתקבלה.
 - **שם הפעולה:** Payload_ComponenetReset
 - **מטרה:** מבצעת soft reset לרכיב ה-payload.
 - **שם הפעולה:** HardTX_ComponenetReset
 - **מטרה:** מבצעת איפוס חומרתי למשדר (TX).
 - **שם הפעולה:** HardRX_ComponenetReset
 - **מטרה:** מבצעת איפוס חומרתי למקלט (RX).
 - **שם הפעולה:** Soft_ComponenetReset
 - **מטרה:** מבצעת איפוס תוכנתי (Soft Reset) למחשב.
 - **שם הפעולה:** Hard_ComponenetReset
 - **מטרה:** מבצעת איפוס חומרתי כללי לכל המערכת. (אתחול למערכת החשמל כך שאף ממערכת אחרת לא מקבלת חשמל).
 - **שם הפעולה:** Ants_ComponenetReset
 - **מטרה:** מבצע איפוס למערכת האנטנות.
 - **שם הפעולה:** FRAM_ComponenetReset
 - **מטרה:** מבצע אתחול לרכיב ה-FRAM.
 - **שם הפעולה:** FS_ComponenetReset
 - **מטרה:** מבצעת אתחול למערכת הקבצים.
- **שם הפעולה:** CMD_ResetComponent
 - **טענת כניסה:**
 - cmd – מצביע לפקודת לוויין (*sat_packet_t) הכוללת את סוג האיפוס.
 - **טענת יציאה:**
 - שגיאות בהתאם ל-<hal/errors.h>
 - 0 – הצלחה
 - הפונקציה מפעילה תהליך איפוס לרכיב בהתאם לסוג האיפוס המבוקש. לפני ביצוע האיפוס, מורם דגל Reset ונשלח ACK לתחנת הקרקע.
- **שם הפעולה:** CMD_SetGsWdtKickTime
 - **טענת כניסה:** cmd - מצביע למבנה הפקודה
 - **טענת יציאה:**
 - 1- – כאשר cmd == NULL
 - 2- – אורך נתונים שגוי.
 - 3- – לא ניתן לבצע

- 4- – שגיאה בכתיבה ל-FRAM.
- שגיאות בהתאם ל-SendAckPacket
- הפונקציה מגדירה זמן חדש ל-Watchdog לאור חוסר של תקשורת עם תחנת הקרקע.
- שם הפעולה: CMD_GetGsWdtKickTime
- טענת כניסה: cmd - מצביע למבנה הפקודה
- טענת יציאה:
- 1- – שגיאת קריאה מה-FRAM
- שגיאות בהתאם ל-SendAckPacket
- הפונקציה מחזירה את משך זמן ה-Watchdog של תחנת הקרקע.
- שם הפעולה: CMD_SetFirstActiveFlags
- טענת כניסה: cmd - מצביע למבנה הפקודה
- טענת יציאה: סכום השגיאות (בערכים שליליים) במקרה של כשל
- הפונקציה מגדירה דגלי אתחול ראשוניים של המערכת לציון הפעלה ראשונה.
- משומש רק פעם אחת (אולי יותר עבור בדיקות) ולא פונים לפקודה הזאת שוב. לפני ששולחים את הלוויין למקום השיגור פונים לפקודה זאת שתכין את הלוויין עם פרמטרי שיגור.
- Payload_Commands.h
- אחריות הקובץ: מכיל את מימוש הפקודות שאפשר לקבל מהקרקע עבור ה-Payload.
- שם הפעולה: CMD_TurnOnKillPayloadFlag
- טענת כניסה:
- cmd – מצביע לפקודת לוויין (*sat_packet_t) המכילה את המידע הדרוש לביצוע הפעולה, כולל כותרות הפקודה.
- טענת יציאה:
- הפעולה מחזירה int:
- 2- – שגיאה בכתיבה ל-FRAM
- ערכי שגיאה נוספים בהתאם לפונקציה SendAckPacket
- הפונקציה מדליקה את דגל Kill Payload בזיכרון ה-FRAM ומכבה את מערכת המטען (Payload). לאחר ביצוע הפעולה נשלח ACK לתחנת הקרקע בהתאם לתוצאת הביצוע.
- שם הפעולה: CMD_TurnOffKillPayloadFlag
- טענת כניסה:
- cmd – מצביע לפקודת לוויין
- טענת יציאה:
- הפעולה מחזירה int:
- 2- – שגיאה בכתיבה ל-FRAM
- ערכי שגיאה נוספים בהתאם לפונקציה SendAckPacket
- הפונקציה מכבה את דגל Kill Payload בזיכרון ה-FRAM ומדליקה מחדש את מערכת המטען (Payload).

הבעיה האלגוריתמית

```

121/*
122 * Delete old files according to how much space we have left. (20% for start delete until we have 25% free)
123 * MIN_FREE_SPACE_PERCENTAGE - the minimum free space in bytes we want to keep in the SD in all times.
124 * If free_space < minFreeSpace we start deleting old TLM files
125 */
126void DeleteOldFiles()
127 {
128     F_SPACE space;
129     int sd = f_getdrive();
130     if(sd != 0 && sd != 1)
131     {
132         logError(sd, "Maintenance - f_getdrive");
133         return;
134     }
135     if(logError(f_getfreespace(sd, &space), "Maintenance - f_getfreespace")) return; //gets the variables to the struct
136     if(space.free >= ((space.total * MIN_FREE_SPACE_PERCENTAGE) / 100)) return; //if we only left 20% free
137     F_FIND find; // we delete until we have 25% free
138     f_findfirst("*.*", &find); // get the oldest file.
139     do
140     {
141         f_delete(find.filename); //delete
142         if(logError(f_getfreespace(sd, &space), "Maintenance - f_getfreespace")) return; //gets the variables to the struct
143     }
144     while((space.free < (space.total * MIN_FREE_SPACE_PERCENTAGE_TO_DELETE) / 100) && !f_findnext(&find));
145     // the while check if we still have more then 25% space used and we have more files to delete we get the next one after the oldest.
146
147
148 }

```

הקוד שלהלן מתייחס לפתרון אלגוריתמי לניהול מחיקת קבצים מכרטיס ה-SD במצב של מחסור במקום פנוי. בניגוד לגישה הקודמת, שבה קבצים נמחקו לאחר שחלף פרק זמן קבוע מראש (כחודש), הפתרון המוצא מתבסס על מצב הזיכרון בפועל. לאחר חישוב מוערך של כמות המידע הנשמרת מדי יום, נמצא כי ניתן לשמור נתונים לתקופה של כשנה וחצי על אותו כרטיס SD ועדיין להותיר מקום פנוי. ממצא זה הוביל לפיתוח גישה חדשה, שמטרתה לאפשר שמירת מידע לאורך זמן ארוך יותר, תוך שמירה על מנגנון מחיקה מבוקר ויעיל.

גישה זו משפרת את יעילות פעולת הלוויין ומאפשרת ניצול טוב יותר של משאבי האחסון הזמינים. בנוסף, היא מאפשרת לצוות התקשורת גישה למידע היסטורי רחוק יותר בעת הצורך. יתרה מזאת, במקרים שבהם גודל הקבצים גדול יותר, הגישה הישנה אינה בהכרח מתאימה, בעוד שהאלגוריתם הנוכחי מספק פתרון גמיש ועמיד יותר.

מהלך האלגוריתם הוא כדלקמן:

1. תחילה מתבצעת בדיקה לוודא שקיים חיבור תקין לכרטיס ה-SD.
2. לאחר מכן נבדק אחוז המקום הפנוי בכרטיס:
 - a. אם אחוז המקום הפנוי גדול מ-20% – התהליך מסתיים ללא פעולה נוספת.
 - b. אם אחוז המקום הפנוי הוא 20% או פחות – ממשיכים לשלב הבא.
3. האלגוריתם מאתר את הקובץ הראשון במערכת ומתחיל למחוק קבצים בסדר כרונולוגי. לאחר כל מחיקה מתבצעת בדיקה מחודשת של אחוז המקום הפנוי. כאשר אחוז המקום הפנוי חוצה את רף ה-25%, הלולאה נעצרת, והתוכנה מפסיקה למחוק קבצים עד להפעלה הבאה של מנגנון התחזוקה.

הקוד ממומש בקובץ: src/SubSystemModules/Maintenance/Maintenance.c

```

19-/*
20- * Change the packet we get to sat_packet_t struct
21- *
22- * @param[in] name= data; type= unsigned char *; The packet we get after reading from the frame (without the headers of the Ax25 protocol).
23- * @param[out] name= cmd; type= sat_packet_t *; The struct we put all the info we get from the packet.
24- * @return error according to CMD_ERR
25- *
26- */
27-CMD_ERR ParseDataToCommand(unsigned char * data, sat_packet_t *cmd)
28-{
29-    if(data == NULL || cmd == NULL) // check none of the pointers is null
30-        return null_pointer_error;
31-
32-    int plusPlace = 0; // get a counter for the start place of the data we read
33-    if(memcpy(&cmd->ID, data, sizeof(cmd->ID)) == NULL)
34-        // check the memcpy worked and put in the cmd->ID the id of the sat and the other stuff there
35-        return null_pointer_error;
36-    plusPlace += sizeof(cmd->ID); // add the size of the ID to the variable
37-
38-    if(cmd->ID>>24 != CUBE_SAT_ID && cmd->ID>>24 != ALL_SAT_ID) // check we really are the sat it was send to.
39-        return invalid_sat_id;
40-
41-    if(memcpy(&cmd->cmd_type, data + plusPlace, sizeof(cmd->cmd_type)) == NULL) // same just the cmd_type.
42-        return null_pointer_error;
43-    plusPlace += sizeof(cmd->cmd_type);
44-
45-    if(memcpy(&cmd->cmd_subtype, data + plusPlace, sizeof(cmd->cmd_subtype)) == NULL) // same just the cmd_subtype.
46-        return null_pointer_error;
47-    plusPlace += sizeof(cmd->cmd_subtype);
48-
49-    if(memcpy(&cmd->length, data + plusPlace, sizeof(cmd->length)) == NULL) // same just the length.
50-        return null_pointer_error;
51-    plusPlace += sizeof(cmd->length);
52-
53-    if(cmd->length > MAX_COMMAND_DATA_LENGTH)
54-        return index_out_of_bound;
55-
56-    if(cmd->length != 0){
57-        if(memcpy(&cmd->data, data + plusPlace, cmd->length) == NULL)
58-            // same just to the data which is the length that equal to the value of the variable length.
59-            return null_pointer_error;
60-    }
61-
62-    return command_success;
63-}

65-/*
66- * Create a sat_packet_t struct that in the end the sat send to us
67- *
68- * @param[in] name= data; type= unsigned char *; The data part in the struct
69- * @param[in] name= data_length; type= unsigned short; The length of the data that going to in the struct. Also it will be in the length part in the struct
70- * @param[in] name= type; type= char; The cmd_type part in the struct. Says which subsystem its from.
71- * @param[in] name= subtype; type= char; The cmd_subtype part in the struct. Says which command of the subsystem its from.
72- * @param[in] name= id; type= unsigned int; The ID part in the struct. Have the id of the sat and some other things.
73- * @param[out] name= cmd; type= sat_packet_t *; The struct we put all the info we get from the params.
74- * @return error according to CMD_ERR
75- *
76- */
77-CMD_ERR AssembleCommand(unsigned char *data, unsigned short data_length, char type, char subtype, unsigned int id, sat_packet_t *cmd)
78-{
79-    if(cmd == NULL)
80-        return null_pointer_error;
81-
82-    cmd->ID = id;
83-    cmd->cmd_type = type;
84-    cmd->cmd_subtype = subtype;
85-    cmd->length = data_length;
86-    if(data_length != 0)
87-    {
88-        if(data == NULL)
89-            return null_pointer_error;
90-        else
91-        {
92-            if(data_length > MAX_COMMAND_DATA_LENGTH) // Check the data_length is not larger then the max length available
93-                return index_out_of_bound;
94-
95-            memcpy(&cmd->data, data, data_length);
96-        }
97-    }
98-
99-    return command_success;
100-}

```

שתי הפונקציות המוצגות אחראיות על פירסור והרכבה של חבילות התקשורת (packets) בלוויין, בהתאם לפרוטוקול התקשורת הפנימי (SPL), לאחר הסרת מעטפת הפרוטוקול החיצוני (AX.25).

הפונקציה הראשונה – ParseDataToCommand

פונקציה זו אחראית על פיענוח חבילת נתונים גולמיים (raw data) שהתקבלה מהתקשורת והמרתה למבנה sat_packet_t.

הפונקציה קוראת את הנתונים מה-buffer לפי סדר השדות המוגדר בפרוטוקול SPL, ללא התחשבות במשמעות הלוגית של הפרמטרים, אלא רק במבנה הבינארי שלהם (רק ל-bytes לרוב מומר מ-char לסוג משתנה אחר).

הפיענוח מתבצע באמצעות memcpy, שבאמצעותו מועתקים מקטעי זיכרון באורכים מדויקים אל השדות המתאימים במבנה. במהלך התהליך הפונקציה:

- בודקת תקינות מצביעים (שאינם NULL).
- מאמתת כי מזהה הלוויין (ID) מתאים ללוויין הנוכחי או לשידור כללי.
- בודקת כי אורך הנתונים (length) אינו חורג מהאורך המקסימלי המותר.
- מעתיקה את שדה הנתונים (data) רק אם אורכו גדול מאפס.

בסיום, המבנה המפוענח מאוחסן ב-sat_packet_t, אשר הוא struct, ומאפשר גישה הן לשדות המובנים והן לייצוג הגולמי של החבילה.

הפונקציה השנייה – AssembleCommand

פונקציה זו אחראית על בניית חבילת תקשורת (sat_packet_t) לצורך שליחה.

היא מקבלת כקלט את כל רכיבי החבילה: מזהה הלוויין, סוג הפקודה, תת-סוג הפקודה, אורך הנתונים והנתונים עצמם, ומרכיבה מהם מבנה מלא.

הפונקציה:

- מאתחלת את שדות המבנה בהתאם לפרמטרים שהתקבלו.
- בודקת תקינות מצביעים ואורך הנתונים.
- מוודאת שאורך הנתונים אינו חורג מהמגבלה שהוגדרה.
- מעתיקה את הנתונים אל שדה data במבנה באמצעות memcpy.

מאחר שהמבנה sat_packet_t הינו struct, ניתן לשלוח את החבילה כב्लוק רציף של בתים כאשר מתבצעת המרת pointers ל-char*. בצד הקרקע מתבצע פיענוח של המידע הגולמי חזרה לשדות המבנה. (אפשר לראות זאת בפונקציה הבאה)

```

394/*
395 * Send Ax25 packet
396 * @param name=cmd; type=sat_packet_t*; The packet the sat got and use to find all the required information (like the headers we add)
397 * @param name=avalFrames; type=int*; availed frames
398 * @return type=int; return -1 on cmd NULL
399 * -2 if we can't transmit
400 * Error code according to <hal/errors.h>
401 */
402 int TransmitSplPacket(sat_packet_t *packet, int *avalFrames)
403 {
404     if(!CheckTransmissionAllowed())
405         return -2;
406     unsigned char avail;
407     if(packet == NULL)
408         return -1;
409     size_t place = sizeof(packet->ID) + sizeof(packet->cmd_subtype) + sizeof(packet->cmd_type) + sizeof(packet->length) + packet->length;
410     // Get the length of the data of the packet (including the headers we add)
411     int error = logError(isis_vu_e__send_frame(0, (unsigned char *)packet, place, &avail), "TransmitSplPacket - isis_vu_e__send_frame"); // Transmit packet
412     *avalFrames = (int)avail; // Get avail Frames
413     return error;
414 }

```

פונקציה זו אחראית על שידור חבילת תקשורת אחת בפורמט SPL דרך שכבת ה-Ax.25 לאחר שהחבילה כבר הורכבה במלואה. היא גם אחראית על כמעט כל השידורים של הלוויין לקרקע.

בשלב הראשון הפונקציה בודקת האם שידור מותר במצב הנוכחי של המערכת. בדיקה זו כוללת, את מצב ה-mute. אם שידור אינו מותר, הפונקציה מפסיקה את פעולתה ומחזירה קוד שגיאה המעיד שלא ניתן לשדר.

לאחר מכן מתבצעת בדיקת תקינות למצביע לחבילת התקשורת. אם המצביע הינו NULL, הפונקציה מחזירה קוד שגיאה מתאים.

בשלב הבא מחושב האורך הכולל של החבילה לשידור. אורך זה כולל את כל שדות הכותרת של פרוטוקול SPL (מזהה הלוויין, סוג הפקודה, תת-סוג הפקודה ושדה האורך), וכן את שדה הנתונים עצמו בהתאם לערך length.

לאחר חישוב האורך, החבילה משודרת באמצעות קריאה לפונקציית הדרייבר isis_vu_e_send_frame. תוצאת פעולת השידור נבדקת ונרשמת, אם יש שגיאה, באמצעות מנגנון רישום שגיאות.

```

182-/*!
183 * @brief deployment procedure
184 * @return 0 successful deployment
185 *        -1 failed to deploy
186 */
187 int FirstActivation()
188 {
189     Boolean false = FALSE;
190     int firstActiveFlag;
191     FRAM_read((unsigned char*)&firstActiveFlag, FIRST_ACTIVATION_FLAG_ADDR, FIRST_ACTIVATION_FLAG_SIZE);
192     if(!firstActiveFlag)
193         return 0;
194     int error = 0;
195 #ifdef FIRST_ACTIVE
196     int max;
197     int time = 0;
198     FRAM_read((unsigned char*)&max, DEPLOYMENT_TIME_ADDR, DEPLOYMENT_TIME_SIZE);
199     do
200     {
201         FRAM_read((unsigned char*)&time, SECONDS_SINCE_DEPLOY_ADDR, SECONDS_SINCE_DEPLOY_SIZE);
202         vTaskDelay(5000 / portTICK_RATE_MS); // wait 5 seconds.
203         time += 5;
204         TelemetryCollectorLogic();
205         if(logError(FRAM_writeAndVerify((unsigned char*)&time, SECONDS_SINCE_DEPLOY_ADDR, SECONDS_SINCE_DEPLOY_SIZE), "FirstActivation - FRAM_writeAndVerify"))
206             error = -1;
207         printf("%d\r\n", time);
208     }
209     while(max > time);
210     for(int i = 0; i < 2; i++)
211     {
212         AntArm(0);
213         AntDeployment(0);
214         AntArm(1);
215         AntDeployment(1);
216     }
217     time_unix timeUnix;
218     logError(Time_getUnixEpoch((unsigned int*)&timeUnix), "FirstActivation - Time_getUnixEpoch");
219     if(logError(FRAM_writeAndVerify((unsigned char*)&timeUnix, LAST_TRY_TO_DEPLOY_TIME_ADDR, LAST_TRY_TO_DEPLOY_TIME_SIZE), "FirstActivation - FRAM_writeAndVerify"))
220         error = -1;
221 #endif
222     if(logError(FRAM_writeAndVerify((unsigned char*)&>false, FIRST_ACTIVATION_FLAG_ADDR, FIRST_ACTIVATION_FLAG_SIZE), "FirstActivation - FRAM_writeAndVerify - flag = 0"))
223         error = -1;
224     SendAckPacket(ACK_FINISH_FIRST_ACTIVE, NULL, NULL, 0);
225     return error;
226 }
227 }
```

כאשר הלוויין נמצא בקרקע הוא נמצא לתכנות, בנייה ובדיקות. עם זאת, המטרה הסופית היא להפעיל אותו בחלל. לפיכך, ההרצה הראשונית של הלוויין מתייחסת למעשה להפעלתו הראשונה בחלל, שבה מתבצעים סדר פעולות חיוני לצורך הפעלתו הראשונית. פעולות אלו כוללות, בין השאר, אתחול ערכים בזיכרון ופריסת האנטנות.

הפונקציה המוצגת למעלה אחראית על פריסת האנטנות של הלוויין לאחר זמן מסוים מאז שהלוויין דולק (כ-40 דקות מאז ששוחרר מהטיל).

הפונקציה מתחילה בבדיקה של האם הלוויין נמצא במצב של הפעלה ראשונית. אם כן, היא ממשיכה לפעול. אחרת, היא יוצאת מהפונקציה.

לאחר מכן, הקוד קורא את הזמן שבו אנחנו רוצים להתחיל את פריסת האנטנות (כעבור כ-40 דקות מאז שהלוויין דולק).

במהלך הלולאה מתבצעות מספר פעולות:

1. **המתנה של 5 שניות** – זוהי המתנה שנועדה עבור הספירה. אין בעיה אם ידרשו יותר מ-5 שניות (נספר 5 אך יש עוד פעולות אחר כך), אך חשוב שהפריסה לא תתרחש לפני הזמן.
2. **הוספת 5 למונה** – המונה נקרא מה-FRAM. המונה מושווה מול הזמן שבו יש לבצע את הפריסה.
3. **שמירה של נתוני טלמטריה** – ה-WDT של ה-EPS היה מאתחל את הלוויין תוך כ-3 דקות אם לא היינו קוראים למערכת החשמל באיזושהי דרך.
4. **שמירת המונה ב-FRAM** – פעולה זו מבטיחה שבמקרה של אתחול, המונה לא יחזור ל-0.
5. **הדפסת הזמן למעקב בקרקע** – ההדפסה מספקת בזמן פריסת הניסיון את הזמן, וגם במקרה שבו הלוויין נדלק בטעות בעודו על הקרקע.
6. **הלולאה נמשכת עד שהזמן עובר או שווה לזמן הפריסה הרצוי**.

כאשר הזמן לפריסה הגיע, הפונקציה מנסה לפרוס את שני מערכות האנטנות (בסך הכל יש 4 אנטנות) פעמיים, למקרה שהפעם הראשונה לא הצליחה. אם הפנייה מצליחה, האנטנות נפרסות בהצלחה.

בסיום תהליך זה, נשמר הזמן האחרון שנעשה בו ניסיון לפרוס, כך שבמקרה ששתי ניסיונות הפריסה לא הצליחו, הלוויין ימשיך לנסות כל חצי שעה עד שיקבל פקודת עצירה מהקרקע.

הפונקציה מסיימת את ההפעלה הראשונית על ידי שמירת ה-flag, שקובע שאנחנו בהפעלה ראשונית כ-false, ושליחת ACK שמודיע על סיום.

בנוסף, קיים בקטע הקוד הוראת `ifdef`, שכן מדובר בקוד מאוד רגיש שברצוננו להריץ רק פעמיים: פעם אחת במהלך הבדיקה בקרקע ופעם נוספת במהלך הפעלה הראשונית האמיתית בחלל. לכן, כאשר מתכנתים את הקוד על הקרקע, כל עוד איננו במצב בדיקת המערכת, יש לוודא כי הקוד לא ירוץ בטעות.

זהו ה-struct שממולא עבור יצירת ב-WOD (שזהו המידע שנשלח ב-Beacon¹).

ה-struct הזה מכיל כל מיני ערכים חשובים עבור הלוויין שאנחנו רוצים לראות לעיתים קרובות. לדוגמא: מתח בבטרייה, זמן הלוויין, האם ה-payload דולק, מספר האתחולים וכדומה.

```

35 typedef struct __attribute__((__packed__)) WOD_Telemetry_t
36 {
37     voltage_t voltBattery;           ///< the current voltage on the battery [mV]
38     voltage_t volt_5V;              ///< the current voltage on the 5V bus [mV]
39     voltage_t volt_3V3;             ///< the current voltage on the 3V3 bus [mV]
40     power_t charging_power;         ///< the current charging power [mW]
41     power_t consumed_power;         ///< the power consumed by the satellite [mW]
42     current_t electric_current;      ///< the up-to-date electric current of the battery [mA]
43     current_t current_3V3;          ///< the up-to-date 3.3 Volt bus current of the battery [mA]
44     current_t current_5V;           ///< the up-to-date 5 Volt bus current of the battery [mA]
45     temp_t bat_temp;                /*!< 2 cell battery pack: [degC] */
46     float solar_panels[NUMBER_OF_SOLAR_PANELS]; // temp of each solar panel
47     time_unix sat_time;             ///< current Unix time of the satellites clock [sec]
48     unsigned int free_memory;        ///< number of bytes free in the satellites SD [byte]
49     unsigned int corrupt_bytes;      ///< number of corrupted bytes in the memory [bytes]
50     unsigned int total_memory;       ///< number of bytes in the memory [bytes]
51     unsigned int used_bytes;         ///< number of used bytes in the memory [bytes]
52     int lastFS_error;               ///< last error of the file system
53     unsigned int number_of_resets;    ///< counts the number of resets the satellite has gone through [#]
54     time_unix sat_uptime;            ///< Sat uptime
55     unsigned int num_of_cmd_resets;   ///< number of resets due to ground station command [#]
56     int sel_counter;
57     int seu_counter;
58     int sensor_one_radfet;
59     int sensor_two_radfet;
60     double radfet_temp;
61     time_unix last_radfet_read_time;
62     power_t power_payload;
63     Boolean payload_flag;
64     EpsState_t eps_state;
65     int change_in_mode;
66     float temp_pa;
67     float temp_board;
68     float reflected_power; //in mW
69     float forward_power; //in mW
70     float doppler;
71     float rssi;
72 } WOD_Telemetry_t;

```

¹ הפונקציה שיוצרת את המידע ל-Beacon נמצאת בנספחים במסמך TelemetryCollector.c.

מסמך בדיקות

לאורך תקופת הלוויין נעשו בדיקות מרובות גם על התוכנה וגם על החומרה. במסמך בדיקות זה יוסבר על בדיקות התוכנה וגם על בדיקות המשולבות בין חומרה לתוכנה.

מסמך זה מייצג את כל הבדיקות כבר אחרי סיום הפרויקט, כך שהתוצאות הצפויות הן לבסוף גם התוצאות בפועל. במהלך הפיתוח לא כל הבדיקות עברו בהצלחה והיה צריך לשנות דברים אך לבסוף הכל עבר.

סוג בדיקה	מערכת	הסבר בדיקה	תוצאה	הערות נוספות
פונקציונלית	EPS	בדיקת פעולה של כל הפקודות	כל הפונקציות פועלות כדלקמן ושולחות הודעת הצלחה.	ללא בדיקות קצה
		בדיקת ערכי קצה בכל הפונקציות	הפונקציה אמורה לפעול ולהצליח.	בדיקות קצה
		הכנסת ערך Alpha לא תקין.	קבלת ack שגיאה בערך.	
		בכל אחת מהפונקציות הכנסת אורך לא תקין	קבלת ack שגיאת אורך.	
		הכנסת ערך threshold גדול מהערך המקסימלי	קבלת ack שגיאה בערך.	
אוטומציה		הכנסת ערך threshold קטן מהערך המינימלי.	קבלת ack שגיאה בערך.	היו מספר בעיות בבדיקה זאת, אך הבעיה התבהרה ותוקנה כך שעובד. רואים גם את השינוי בכך שדברים מפסיקים לעבוד.
		שינוי מתח ספק הכוח לוודא עבירה נכונה בין המצבי הלוויין לפי סף המתח.	עבירה חלקה בין מצבי הלוויין. כאשר עוברים לפי threshold הלוגיקה עובר מצב.	
פונקציונלית	Maintenance	בדיקת פעולה של כל הפקודות.	כל הפונקציות פועלות כדלקמן ושולחות הודעת הצלחה.	ללא בדיקות קצה
		בכל אחת מהפונקציות הכנסת אורך לא תקין	קבלת ack שגיאת אורך.	בדיקות קצה
		בדיקת ערכי קצה בכל הפונקציות	הפונקציה אמורה לפעול ולהצליח.	
		קבלת ערך שלא קשור למערכת באתחול	קבלת ack שגיאה של אי-אפשרות ביצוע.	
		הכנסת בשינוי הזמן זמן לפני שנת 2000.	קבלת ack שגיאה של אי-אפשרות ביצוע.	
		בדיקת הכנסת זמן ישן יותר מזמן לווין	מצליח אך מוחק את כל הקבצים הנוכחיים מה-FS.	
	Maintenance	הכנסת זמן ל-WDT של חוסר התקשורת הקטן משלושה ימים	קבלת הודעת שגיאה של אי-אפשרות ביצוע.	נעשה על ידי הקטנת מינימום זיכרון פנוי. מ-20%
אוטומציה		בדיקת תקינות מנגנון מחיקת הקבצים. המשך מעמוד קודם	כאשר הגענו לסף התחלת מחיקה מתחיל	

ל-99% שיהיה אפשרות לבדוק	למחוק עד שיש יותר מקום.			
בוצעו גם התחלה של הלוויין וגם כפקודה. עבור זה היה צריך לצרוב את הקוד.	קבלת ack עם הזמן הנוכחי והסתכלות שהמונים עלו.	הסתכלות שמנגנון אחרי אתחול עובד		
מצב זה חשוד להיגרם בין אם מבעיה ב-payload או במערכת התקשורת אך יכול להיות גם משהו אחר אז מאפסים את כל הלוויין.	מכבה עד להודעה חדשה את ה-payload ומאתחל את מערכת התקשורת ואז את הלוויין.	כאשר עברנו את זמן ה-WDT של התקשורת עם הקרקע המנגנון פועל.		
זהו רכיב שלא נוסה בעבר בחלל אז היה צריך לקחת אמצעי זהירות. גם עבור הבדיקה הזאת הקוד כבר היה צריך להיות צרוב על הלוויין.	ה-flag עולה אחרי שינוי הדגל ב-FRAM וכיבוי המט"ע.	בדיקת מערכת כיבוי ה-payload. נעשו 2 resets בדקה.	Maintenance	אוטומציה
	ה-flag עולה ולא צריך לכבות את המט"ע עד בהדלקה הנוכחית	בדיקת מערכת כיבוי ה-payload. לא נעשו 2 resets בדקה.		
פונקציית CheckExecutionTime	כל 10 שניות מודפס למסך "hello"	בדיקת הזמן 5 פעמים ברציפות כדי לראות שהכול עובד		אוטומציה בדיקת ערכי קצה
	מחזיר TRUE.	הכנסת תקופת זמן 0.		
IsFS_Corrupted	בתקווה יחזיר FALSE אבל אם יש יחזיר TRUE	בודק את הפונקציה שמחזירה אם יש מקום לא טוב בזיכרון		
	לא תהיה שגיאה של חוסר חיבור	כל מערכות ה-inits פועלות.		
	לא מאותחלים הערכים והזמן מתעדכן ליותר מאוחר.	בדיקה שבהפעלה שלא אמורה להיות ראשונית הפונקציה לא רצה.		
המטען הייעודי אף פעם לא נוסה בחלל אז נוקטים באמצעי זהירות מרבים.	רואה את הדגל דלוק ולא מפעיל את ה-payload	בדיקה שכאשר ה-payload אמור להיות מכובה לאלתר הוא באמת נשאר כבוי	InitSystem	אוטומציה
	רואה ש-2 הדגלים לא דלוקים ומפעיל את ה-payload	בדיקה שכאשר ה-payload אמור להיות דלוק הוא באמת נדלק		
	כל הערכים ב-FRAM שונו לערכי ברירת המחדל	הפעלת הדגל של הפעלה ראשונית ובדיקת אתחול ערכים		
	הדפסה של הזמן עולה וקבלת ack בסוף שסיים.	בדיקת מנגנון הספירת הזמן של פריסת האנטנות (ללא פריסה)		הפעלה ראשונית
	כמו הקודם + האנטנות נפרסות.	פריסת אנטנות (המנגנון המלא)		
ללא בדיקות קצה.	כל הפונקציות פועלות כדלקמן ושולחות הודעת הצלחה.	בדיקת פעולה של כל הפקודות.	File System	פונקציונלית
בדיקות קצה	הפונקציה אמורה לפעול ולהצליח.	הצבת ערכי קצה לפונקציות.		

בדיקות קצה	שולח שגיאת SD לא קיים.	הכנסה בפונקציית שינוי SD מספר שונה מ-0/1	File system	פונקציונלית
	שולח שגיאת אי-אפשרות ביצוע.	הכנסת type לא קיים בפונקציות שמצריכות.		
בדיקות dump, כל פעם ש-dump יכול לעבוד הוא פותח thread כך שעדיין עשויים להישלח beacons	שולח ack התחלה, מידע הנשמר במרחק של 20 שניות, ack סיום	הכנסת פרמטרים ל-dump של שבוע. עם רזולוציה של 20 שניות		
	שולח ack התחלה, כל מידע הנשמר באותה תקופת זמן, ack סיום	הכנסת פרמטרים עם רזולוציה 0		
	שולח ack התחלה, כל מידע הנשמר באותה תקופת זמן, לפני שהיה אמור לסיים מפסיק ושולח ack סיום	שליחת dump ארוך ובמהלך ה-dump מתבצעת עזיבה של ה-dump		
	לניסיון של לפתוח את ה-dump השני שולח הודעת שגיאה אי-אפשרות ביצוע.	ניסיון לעשות 2 dumps במקביל		
	נשמרים במערכת קבצים / מידע כל כמות זמן. (בכל יום נוצר קובץ חדש לכל מערכת)	בדיקה שכל פעם שעובר מחזור של מערכת המידע שלה נשמר		
נעשה באמצעות debug	הקובץ נמחק מה-SD ומתפנה מקום.	בדיקה יחידנית של יכולת למחוק קובץ.		אוטומציה
נעשה באמצעות מחיקה של כל הקבצים מסוג ו-dump כדי לראות שלא נשאר כלום.	אי התרחשות של שגיאה במהלך ההרצה.	בדיקה כללית של המערכת		
כולל תהליך יצירת קובץ / שם קובץ	כל הפונקציות פועלות כדלקמן ושולחות הודעת הצלחה.	בדיקת פעולה של כל הפקודות		
ללא בדיקות קצה	הפונקציה אמורה לפעול ולהצליח.	בדיקת ערכי קצה בכל הפונקציות	TRXVU	פונקציונלית
	שליחת ack סוג לא קיים	שליחת type לא קיים בפקודה		
	שליחת ack תת-סוג לא קיים	שליחת sub-type לא קיים		
	שליחת הודעת שגיאה צד לא קיים.	קבלה לפקודה של אנטנות צד שהוא אינו 0/1		
	קבלת ack שגיאת אורך.	בכל אחת מהפונקציות הכנסת אורך לא תקין		
	הפקודה מתבצעת אך הזמן הינו הזמן המקסימלי של הפקודה.	כאשר נשלח זמן גדול מהזמן המקסימלי בפקודות הדורשות זאת		
בדיקות קצה				

פונקציונלית		כאשר נשלח זמן קטן מהזמן המינימלי בפקודות הדורשות זאת	הפקודה מתבצעת אך הזמן הינו הזמן המינימלי של הפקודה.	בדיקות קצה
אוטומציה	TRXVU	בדיקת שליחת ה- Beacon	כל 20 שניות (ברירת מחדל) נשלח beacon במערכת התקשורת.	התבצע באמצעות טיימר ו-RFCB
		בדיקת פרסור ההודעה	ההודעה פוענחה כמצופה בלוויין	test של הלוויין, מערך בנוי ל-test
		בדיקת פרסור הודעה הנשלחת מהקרקע	ההודעה פוענחה כמצופה בלוויין	התגלה באמצעות debug
		בדיקת פרסור עם ID לא נכון	הפונקציה מחזירה שגיאת ID	test של הלוויין, מערך בנוי ל-test
		בדיקת פרסור עם אורך שווה 0	ההודעה פוענחה כמצופה בלוויין	test של הלוויין, מערך בנוי ל-test
		יצירת הודעה בלוויין, ללא בעיה בפרמטרים.	ההודעה נבנתה כראוי.	גם כאשר שולחים הודעה ומקבלים תשובה אך לפני שמערכת התקשורת הייתה מוכנה התבצעו tests על הלוויין.
		יצירת הודעה עם מצביע (cmd) NULL.	מוחזרת שגיאה null_pointer	
		יצירת הודעה עם אורך גדול מ-0 אך בלי data	מוחזרת שגיאה null_pointer	
		יצירת הודעה עם אורך גדול מאורך מקסימלי	מוחזרת שגיאה index_out_of_bound	
		יצירת הודעה בלוויין עם אורך 0 ופרמטרים תקינים	ההודעה נבנתה כראוי.	
		נשלחים acks	אחרי פקודה שהלוויין קיבל נשלח ack.	
		בדיקת פעולת משיב	המשיב דולק כשהודלק עד שעבר הזמן ונכבה	
		הפעלת פונקציית הלוגיקה של מערכת התקשורת	הלוגיקה פועלת כמצופה, מתקבלות חבילות ופועלים לפי זה.	
פונקציונלית	payload	בדיקת פעולה של כל הפקודות	כל הפונקציות פועלות כדלקמן ושולחות הודעת הצלחה.	ללא בדיקות קצה, זה הדבר היחיד ב-payload שלא נכלל כבר במקום אחר.
פונקציונלית	כללי	התממשקות עם תחנת הקרקע	הפיענוח והשליחה מתבצעות כראוי.	
אוטומציה		הרצת לילה של כל התוכנה	הלוויין מבצע את הכל ואין שגיאות משמעותיות.	אחרי כתיבה של מירב הקוד בסוף כתיבת הקוד בדיקות אחרונות.
		הרצת התוכנה למספר ימים	לא הייתה בעיה בפונקציות (אחרי תיקונים בגלל שינויים בפונקציות)	מכיוון שהלוויין לא היה עבדתי עם דרייברים לא מתאימים אז כאשר הוא הגיע היה צריך לשנות
		בדיקת מעבר דרייברים	הקליטה והשידור מצליחים לרוב.	גם אם נמצאה בעיה כבר אי אפשר לתקן.
אחרי שיגור		בדיקת שידור וקליטה בין תחנת הקרקע ללוויין שכבר בחלל		

HoopoeWorld מסמך תוכנה



מודלים חיצוניים

- http.server

- ספרייה מובנית ליצירת שרת HTTP.
- שימשה כבסיס לשרת המספק קבצי HTML ומטפל בבקשות GET מהלקוח. כל בקשות ה-GET שאינן קבצים התנהלו באמצעות לוגיקה במחלקה.

- socketserver

- ספרייה לניהול שרתים מבוססי חיבור (socket).
- נעשה בה שימוש לצורך תמיכה בעיבוד בקשות במקביל באמצעות שילוב ThreadingMixIn.

- ssl

- ספרייה להצפנת תקשורת רשת.
- שימשה להפעלת שרת HTTPS באמצעות תעודות SSL שנוצרו עבור המערכת.

- requests

- ספרייה לביצוע בקשות HTTP חיצוניות.
- שימשה לשליפת נתוני טלמטריה ומידע על לוינים מ-API של SatNOGS תוך שימוש בטוקן אימות.

- threading

- ספרייה לניהול ושימוש ב-threads כדי להריץ את התוכנית במקביל.
- אפשרה הרצת שרת ה-HTTPS במקביל ללולאת קריאת הנתונים מ-SatNOGS וכתיבתם למסד הנתונים.

- json

- ספרייה לעבודה עם נתונים בפורמט JSON.
- שימשה לקריאת קובצי קונפיגורציה ופורמט טלמטריה, ולשמירה וטעינה של נתוני זמן אחרון שנקראו ממערכת SatNOGS.

- sqlite3

- ממשק למסד נתונים מסוג SQLite.
- שימש ליצירת טבלאות, אחסון נתוני טלמטריה מפוענחים, ושליפת נתונים לצורך הצגה באתר וייצוא לקובצי Excel.

- pandas

- ספרייה לעיבוד וניתוח נתונים.
- שימשה לקריאת נתונים ממסד הנתונים ולהמרתם לצורך ייצוא לקובצי Excel.

- io

- ספרייה לקלט/פלט בזיכרון.
- שימשה ליצירת קובצי Excel בזיכרון לפני שליחתם ללקוח להורדה בלי לשמור את הקבצים בשרת.

- struct

- ספרייה לפענוח ואריזה של נתונים בינאריים.
- שימשה לפענוח נתוני טלמטריה גולמיים (bytes) בהתאם לטיפוס הנתון והאם מתקבל מהלויין ב-Big Endian או Little Endian.

- datetime

- ספרייה לעבודה עם תאריכים וזמנים.

- שימשה להמרת זמני Unix לפורמט קריא ולהשוואת חותמות זמן של חבילות טלמטריה.

- time

- ספרייה לעבודה עם זמן והשעיות.
- שימשה לביצוע השעיות יזומות בין בקשות ל-SatNOGS, כדי למנוע חריגה ממגבלות השימוש של השרת.

- math

- ספרייה לפונקציות מתמטיות.
- שימשה לחישוב פריסת נתונים לצורך הצגה ויזואלית בדף האינטרנט (שהגודל ישתנה בהתאם לגודל המסך).

- os

- ספרייה לעבודה עם מערכת ההפעלה.
- שימשה לניהול נתיבים, בדיקת קיום קבצים, וניווט בין תיקיות עבור ה-setup של השרת.

- subprocess

- ספרייה להרצת פקודות מערכת.
- שימשה להרצת OpenSSL ליצירת מפתחות ותעודות SSL ולהתקנת ספריות נדרשות ב-setup של השרת.

- csv

- ספרייה לעבודה עם קובצי CSV.
- שימשה כחלופה לקריאת נתוני טלמטריה מקובץ מקומי לצורכי בדיקות והפעלה ראשונית של לוויין בעל היסטוריה.

מחלקות חיצוניות המשומשות בתוכנית:

- socketserver.ThreadingMixIn

- מוסיפה תמיכה בעיבוד בקשות במקביל באמצעות threads.
- בפרויקט נעשה בה שימוש כדי לאפשר לשרת לטפל במספר בקשות HTTP בו-זמנית

- http.server.HTTPServer

- מחלקה המממשת שרת HTTP בסיסי.
- שימשה כבסיס לשרת היישום, האחראי להאזנה לבקשות נכנסות ולניהול חיבורי HTTP מהלקוחות.

- http.server.SimpleHTTPRequestHandler

- מחלקה המטפלת בבקשות HTTP ומספקת מימוש ברירת-מחדל לבקשות GET.
- בפרויקט הורחבה מחלקה זו לצורך מימוש API מותאם אישית לטיפול בפרוטוקולי בקשה ייעודיים והשתמש במחלקת האב עבור שליחת קבצים סטטים.

מחלקות ופונקציות בתוכנה

תחת תיקיית webpage

תחת תיקיית webpage נמצאים כל הקבצים האחראים על הנראות והעיצוב של האתר. תיקייה זו כוללת קבצי CSS, HTML, JavaScript, ובנוסף קבצי מידה כגון תמונות וה-icon. בהסבר זה תינתן סקירה קצרה על קבצי ה-HHTML ו-CSS, כאשר הדגש העיקרי יהיה על קבצי ה-JavaScript, שבהם ממומשות הפונקציות והלוגיקה של צד הלקוח.

- templateWebsite.html

- קובץ זה מהווה את תבנית הבסיס (Template) של קוד ה-HTML של האתר. בשלב זה של התוכנית עדיין לא ידוע אילו לווים זמינים, או באיזה לווין נרצה לצפות. עם תחילת הרצת התוכנית, קובץ זה נפתח ומשמש ליצירת קובץ חדש בשם HoopoeWorld.html, שבו מופיעים הלווינים המוגדרים בקובץ ה-config ומאפשרים למשתמש לבחור ביניהם.

- HoopoeWorldStyle.css

- קובץ זה אחראי על עיצובו הוויזואלי של האתר ועל נראותו הכללית. כל ההגדרות הקשורות לעיצוב מתבצעות בקובץ זה, והוא קובע כיצד רכיבי האתר יוצגו למשתמש. בין היתר, הקובץ כולל הגדרות של צבעים, צורות, גופנים, גדלים, מרווחים, פריסת רכיבים ואלמנטים גרפיים נוספים. קובץ זה מאפשר הפרדה ברורה בין הלוגיקה והמבנה של האתר לבין העיצוב החזותי שלו.

- clock.js

- קובץ זה מכיל פונקציה אחת בלבד, האחראית על הצגת השעון באתר (ושינוי הגודל שלו). השעון המוצג הוא שעון מסוג UTC, המשמש כזמן הסטנדרטי והקונבנציונלי בעבודה עם לווים. שימוש בזמן UTC מספק נקודת ייחוס אחידה וברורה, המאפשרת להבין במדויק מתי נאסף המידע המוצג באתר.

- dynamincFuntions.js

אחראיות הקובץ: לוגיקת האתר והקישור בין השרת ללקוח.

○ משתנים גלובליים

▪ שם: satName

• סוג: מחרוזת (string)

• מטרה: מחזיק את שם הלוויין הנבחר לשימוש בהמשך.

▪ שם: newestTime

• סוג: מספר שלם (int)

• מטרה: מחזיק את הזמן המאוחר ביותר אותו האתר קיבל.

▪ שם: oldestTime

• סוג: מספר שלם (int)

• מטרה: מחזיק את הזמן הישן ביותר אותו האתר קיבל

▪ שם: isLoading

• סוג: Boolean

- **מטרה:** דגל כדי שלא נשלח 2 בקשות קבלה של מידע חדש ביחד (שלא יעלה אחד על השני).
- **שם:** TWO_HOURS
- **סוג:** מספר שלם (int)
- **מטרה:** פרמטר שאינו משתנה, מחזיק זמן בשניות של שתיים.
- **שם:** WINDOW_WIDTH
- **סוג:** מספר שלם (int)
- **מטרה:** פרמטר שאינו משתנה ב-session, מחזיק את רוחב החלון בעת פתיחה כדי שתהיה אפשרות לקבוע את גודל הדברים, וכמות הדברים המוצגים בשורה, על פי כך.
- **שם הפעולה:** ChooseSat
- **טענת כניסה:**
 - choice – מחרוזת (string) המייצגת את הלוויין שנבחר או ערך ברירת מחדל.
- **טענת יציאה:** אין ערך החזרה (אך כן משתנה ה-HTML)
- הפונקציה מופעלת בעת שינוי בחירת הלוויין. היא מבקשת מהשרת את המידע העדכני ביותר עבור הלוויין הנבחר, מעדכנת משתנים גלובליים בהתאם, ומנהלת את פתיחת וסגירת האפשרות להורדת הנתונים כקובץ Excel.
- **שם הפעולה:** DownloadType
- **טענת כניסה:**
 - choice – מחרוזת (string) המציינת את סוג מסנן ההורדה שנבחר.
- **טענת יציאה:** אין ערך החזרה (אך כן משתנה ה-HTML)
- הפונקציה מוסיפה את מסנן ההורדה המתאים בהתאם לבחירת המשתמש.
- **שם הפעולה:** CheckNull
- **טענת כניסה:**
 - element – מחרוזת (string) המייצגת קלט מהמשתמש.
- **טענת יציאה:**
 - מחזירה true אם הקלט ריק
 - false אם הקלט מכיל ערך.
- הפונקציה בודקת האם שדה קלט ריק, על מנת לוודא שלא הוזנו ערכים חסרים.
- **שם הפעולה:** OpenPopup
- **טענת כניסה:** אין
- **טענת יציאה:** אין ערך מוחזר
- הפונקציה פותחת או סוגרת חלון קופץ (Popup), ומעודכנת גם בכפתור עצמו. הסגירה מאפשרת למשתמש להישאר במיקומו הנוכחי בדף מבלי להזיז את העכבר למעלה.
- **שם הפעולה:** ClosePopup
- **טענת כניסה:** אין
- **טענת יציאה:** אין ערך מוחזר
- הפונקציה סוגרת את החלון הקופץ, ומיועדת לשימוש בכפתור הסגירה (x).

○ שם הפעולה: CheckForm

- טענת כניסה: אין
- טענת יציאה:
- מחזירה false במקרה של שגיאה
- true בהצלחה.
- הפונקציה בודקת את תקינות הטופס, ובמקרה של שגיאה מציגה הודעת שגיאה מתאימה למשתמש.

○ שם הפעולה: SendDownloadRequest

- טענת כניסה: אין
- טענת יציאה: אין ערך מוחזר (אך כן מוריד את הקובץ אם אפשרי)
- הפונקציה בודקת את תקינות הבקשה (לפי הפונקציות הקודמות), שולחת בקשה לשרת לצורך הורדת נתונים בפורמט Excel (xlsx), ומטפלת בקבלת התשובה.

○ שם הפעולה: ScrollDown

- טענת כניסה: אין
- טענת יציאה: אין ערך מוחזר (אך כן משתנה ה-HTML)
- הפונקציה מדמה גלילה אינסופית כלפי מטה באתר, ושולחת בקשה לטעינת נתונים נוספים בהתאם למיקום הגלילה.

○ שם הפעולה: ScrollUp

- טענת כניסה: אין
- טענת יציאה: אין ערך מוחזר
- הפונקציה מופעלת אחת לשעתיים לצורך וידוא שהנתונים המוצגים באתר מעודכנים.

○ שם הפעולה: scroll event listener

- רכיב זה מאזין לאירועי גלילה בדף. כאשר המשתמש מגיע לתחתית הדף (עם סטייה קטנה), מופעלת הפונקציה ScrollDown לצורך מימוש גלילה אינסופית וטעינת נתונים נוספים.

קובץ createSQLTable.py

קובץ זה אחראי על יצירת טבלאות ה-SQL הנדרשות למערכת. הוא קורא נתונים מקובץ JSON, ובאמצעותם יוצר טבלאות עבור לוויינים שטרם קיימת עבורם טבלה במסד הנתונים.

- משתנים גלובליים

- **שם:** connection_sql
 - **סוג:** אובייקט חיבור למסד נתונים מסוג SQL.
 - **מטרה:** ניהול התקשורת עם מסד הנתונים וביצוע פעולות יצירה, קריאה וכתיבה של טבלאות ונתונים.
- **שם:** cursor_obj
 - **סוג:** אובייקט מצביע (Cursor) של מסד נתונים מסוג SQL.
 - **מטרה:** ביצוע שאילתות SQL, והרצת פקודות מול מסד הנתונים דרך החיבור הפעיל.
- **שם:** types
 - **סוג:** מילון (dict)
 - **מטרה:** ביצוע המרה בין סוגי הנתונים המוגדרים בקובץ ה-JSON לבין סוגי הנתונים המתאימים ליצירת ושמירת הטבלאות במסד הנתונים.
- **שם הפעולה:** type_of_param
 - **טענת כניסה:**
 - param – מילון (dict) שמתקבל מקובץ ה-JSON, המתאר פרמטר של טלמטריה.
 - **טענת יציאה:**
 - מחזירה מחרוזת (string) המייצגת את סוג הפרמטר כפי שיישמר במסד הנתונים (SQL).
 - הפונקציה בודקת ומחזירה את סוג הפרמטר שנשמר ב-SQL, לפי המשתנה הגלובלי. במקרה שמדובר ב-enum, הפונקציה מחזירה סוג TEXT במקום תו יחיד, על מנת לשמור נתונים קריאים במסד הנתונים.
- **שם הפעולה:** make_params_for_table
 - **טענת כניסה:**
 - format_file – קובץ JSON המתקבל מקובץ ההגדרות (config), המכיל את תיאור הפרמטרים של הלוויין.
 - **טענת יציאה:**
 - מחזירה מחרוזת (string) הכוללת את כל הפרמטרים בפורמט המתאים ליצירת טבלת SQL.
 - הפונקציה קוראת את כל הפרמטרים מקובץ ה-JSON, ובהתחשב בכך שהפונקציה עובדת על לוויין אחד בלבד כל פעם, מעבדת את כלל הפרמטרים כך שיתאימו ליצירת טבלת SQL.
- **שם הפעולה:** create_tables
 - **טענת כניסה:** אין
 - **טענת יציאה:** אין
 - הפונקציה עוברת על כל הלוויינים המוגדרים בקובץ ההגדרות, ויוצרת טבלאות SQL עבור לוויינים שטרם קיימת עבורם טבלה במסד הנתונים.

קובץ DecodeDataInPacket.py

קובץ זה אחראי על כל ההתנהלות של התוכנה מול מערכת ה-SatNOGS, וכן על העבודה מול טבלאות ה-SQL, למעט שלב יצירת הטבלאות. הקובץ מבצע ניתוח של המידע המתקבל, מעבד אותו, מכניס אותו לטבלאות, לוקח מהן ומארגן אותו בפורמט נוח וברור לצורך הצגה למשתמש.

קובץ זה מחולק ל-2 ההתממשקות עם הלקוח אך גם אסיפת המידע.

- משתנים גלובליים

- **שם:** connection_sql
 - **סוג:** אובייקט חיבור למסד נתונים (sqlite3.Connection)
 - **מטרה:** חיבור למסד הנתונים המרכזי של המערכת, ביצוע שאילתות קריאה וכתובה, ושמירת נתוני טלמטריה.
- **שם:** SATELLITES
 - **סוג:** מילון
 - **מטרה:** שמירת מידע מרוכז עבור כל לוויין: שם טבלה, callsign, קובץ פורמט JSON, שם SatNOGS, Cursor ייעודי ונעילת Thread.
- **שם:** JSONS
 - **סוג:** מילון (מילון בעל מילונים)
 - **מטרה:** טעינת כל קובצי הפורמט (JSON) לזיכרון פעם אחת בלבד, לצורך ייעול ביצועים.
- **שם:** JSONS_FOR_HTML
 - **סוג:** מילון
 - **מטרה:** מבנה JSON מעובד לצורך הצגת נתונים באתר, כולל חלוקה לתת-מערכות וזיהוי פרמטרי זמן.
- **שם:** SatIds
 - **סוג:** רשימת מילונים
 - **מטרה:** שמירת מיפוי בין שם הלוויין במערכת לבין מזהה ה-NORAD שלו, כפי שמופיע במאגר SatNOGS.
- **שם:** SIZE_IN_BYTES
 - **סוג:** מילון
 - **מטרה:** מיפוי בין סוגי נתונים לוגיים (int, float, string וכו') לגודל שלהם ב-bytes לצורך פענוח חבילות.
- **שם הפעולה:** check_make_formula
 - **טענת כניסה:**
 - formula - מחרוזת המייצגת נוסחת חישוב לפרמטר טלמטריה
 - **טענת יציאה:** מחרוזת נוסחה תקינה או חריגת TypeError
 - הפונקציה בודקת תקינות של נוסחה מתמטית (כולל סוגריים, אופרטורים ומשתנה x), מתקנת פורמט במידת האפשר, ומונעת הכנסת תווים או מבנים לא חוקיים.
- **שם הפעולה:** get_norad_id

- **טענת כניסה:**
 - satellites_name – רשימת שמות לוויינים כפי שמוגדרים ב־SatNOGS
- **טענת יציאה:**
 - noradId ו-satName מילונים המכילים
- שליופת מזהי NORAD מאתר SatNOGS לצורך בקשות טלמטריה עתידיות.
- **שם הפעולה:** make_dicts_according_to_config
 - **טענת כניסה:** אין
 - **טענת יציאה:**
 - סוג: tuple
 - פרמטרים:
 - SATELLITES
 - JSONS
 - satIds
 - טעינת קובץ config.json, בדיקת תקינות ההגדרות, טעינת קובצי הפורמט, יצירת מבני נתונים ללוויינים, ואיסוף מזהי SatNOGS.

התממשקות עם הלקוח

- **שם הפעולה:** jsons_for_html
 - **טענת כניסה:**
 - json_format – מבנה הפורמט המקורי
 - **טענת יציאה:**
 - מבנה JSON מותאם ל-HTML.
 - חלוקת פרמטרים לתתי-מערכות, הגבלת מספר פרמטרים בכל קבוצה, וזיהוי פרמטרי זמן לצורך המרה מתאימה.
- **שם הפעולה:** create_options
 - **טענת כניסה:** אין
 - **טענת יציאה:** מחרוזת HTML
 - יצירת תגיות <option> לבחירת לוויין בדף ה-Web.
- **שם הפעולה:** get_raw
 - **טענת כניסה:**
 - raw – מחרוזת תווי Hex.
 - count – מספר תאים בשורה.
 - **טענת יציאה:** מחרוזת מותאמת לאתר שמחזיקה את המידע הגולמי
 - עיבוד נתוני RAW לתצוגה ויזואלית בסגנון HexDump.
- **שם הפעולה:** make_for_html
 - **טענת כניסה:**
 - sat_name - שם הלוויין כפי שהוא מופיע בטבלת SATELLITES.
 - last_date – תאריך בפורמט זמן יוניקס (Unix time). מציין מאיזה זמן להתחיל למשוך נתונים. הערך מתקבל כמחרוזת (string) אך מייצג מספר שלם.

- top – פרמטר שמציין האם הנתונים מיועדים לחלק העליון או התחתון של הדף. לפי הערך הזה הפונקציה יודעת האם לעבוד עם התאריך המינימלי או המקסימלי.
- limit – מספר החבילות (packets) שיש למשוך. אם הערך הוא 0, הפונקציה תיקח את כל החבילות.
- width – רוחב חלון התצוגה כאשר ה-HTML נפתח.
- **טענת יציאה:**
 - **סוג:** tuple
 - **פרמטרים:**
 - נתונים מוכנים להצגה ב-HTML
 - התאריך המינימלי (min_date) שנלקח
 - התאריך המקסימלי (max_date) שנלקח
- שליפת נתונים ממסד הנתונים, עיבודם, המרת זמני UNIX לתאריך קריא, ובניית HTML להצגה בדפדפן.
- **שם הפעולה:** make_excel
 - **טענת כניסה:**
 - params – מילון שמכיל את הפרמטרים לבניית קובץ ה-Excel:
 - type – סוג הבקשה. קובע כיצד לפרש את שאר הפרמטרים ואיך לשלוף את הנתונים מ-SQL.
 - satName – שם הלוויין שעליו מתבצעת הבקשה.
 - פרמטרים נוספים – משתנים בהתאם לערך של type, ומשמשים לסינון, שליפה ועיבוד הנתונים.
 - **טענת יציאה:**
 - **סוג:** tuple
 - **פרמטרים:**
 - excelTable – טבלת נתונים מוכנה בפורמט Excel.
 - file_name – שם קובץ ה-Excel שנוצר.
 - בונה קובץ Excel מנתונים שנשלפים מ-SQL לפי הפרמטרים שהתקבלו.

אסיפת נתונים – הכנסת נתונים למאגר הנתונים

- **שם הפעולה:** doing_format
 - **טענת כניסה:**
 - data – הנתון שעליו מתבצע תהליך העיצוב או ההמרה.
 - data_type – סוג הנתון, לפיו נקבע איזה עיבוד או פורמט יש להחיל.
 - format_data – מחרוזת או מילון (dict) שמכילים את הגדרת הפורמט או ה-enum שיש להחיל על הנתון.
 - **טענת יציאה:** הנתון לאחר החלת הפורמט או ההמרה.
 - ממירה נתונים לפורמט המתאים בהתאם לסוג הנתון והגדרת הפורמט.
- **שם הפעולה:** decoded_param
 - **טענת כניסה:**
 - data_bytes – הנתון בפורמט Hex שמיועד לפענוח.

- data_type – סוג הנתון, הקובע כיצד יש לפענח את המידע.
- is_big – מציין האם הנתון מקודד ב-Big Endian.
- טענת יציאה: הנתון לאחר פענוח מ-Hex לפורמט קריא.
- מפענחת נתונים מ-Hex לערך קריא בהתאם לסוג הנתון וסדר הבייטים.
- שם הפעולה: decode_data_for_sql
- טענת כניסה:
- data – נתון בפורמט Hex שמכיל רק את חלק ה-data מתוך מסגרת AX25.
- json_params – הגדרת הפרמטרים, שלפיה נלמדים שמות הפרמטרים והסוג (type) של כל אחד מהם.
- setting – אובייקט הגדרות שמכיל את תצורת ה-JSON הנדרשת לפענוח הנתונים, בנוסף ברירות המחדל שם.
- טענת יציאה: ערכים מוכנים לשאילתת INSERT למסד הנתונים SQL.
- מפענחת נתוני Hex והופכת אותם לערכים מתאימים להכנסה ל-SQL.
- שם הפעולה: check_respond
- טענת כניסה:
- response – התגובה שהתקבלה מהשרת (satNogs).
- טענת יציאה:
- False – אם יש שגיאה (כגון 429 או טוקן לא חוקי).
- response – אם התגובה תקינה.
- בדיקת שגיאות תקשורת מול SatNOGS (401, 404, 429) וניהול השהיות בהתאם.
- מחלקה SatNogsToSQL
- מטרה: ניהול מלא של קליטת טלמטריה מ-SatNOGS, סינון חבילות, פענוח, והכנסה למסד הנתונים.
- משתני המופע (נוצרים ב-init לאחר מכן):
- שם: self.newest_dates
 - סוג: מילון
 - מטרה: שמירת זמן הטלמטריה האחרון שנקלט עבור כל לווין, למניעת העמסה על השרת שממנו נלקחים.
- שם: self.__headers
 - סוג: מילון
 - מטרה: שמירת כותרות HTTP עם טוקן ההזדהות הדרוש לביצוע בקשות ל-SatNOGS API.
- שם: self.run
 - סוג: בוליאני
 - מטרה: שליטה על המשך או עצירת לולאת הקליטה האינסופית.
- שם: self.__write_cursor
 - סוג: sqlite3.Cursor
 - מטרה: ביצוע פעולות INSERT למסד הנתונים בצורה מרוכזת ויעילה.

- **שם הפעולה: __init__**
 - **טענת כניסה:**
 - newest_dates – מילון שמכיל את זמני החבילות האחרונות לכל לוויין. אם לא נשלח נוצר ברירת המחדל לכל לוויין (עדיף לא להשתמש בברירת מחדל אלא אם כן הלוויין חדש).
 - **טענת יציאה:** משתני המופע
 - מאתחלת את המחלקה עם משתני המופע, כולל זמני לוויינים וטוקן ל-API.
- **שם הפעולה: check_packet**
 - **טענת כניסה:**
 - packet – מסגרת נתונים מלאה מהלוויין (frame).
 - timestamp – תאריך ושעה של החבילה בפורמט ISO.
 - **טענת יציאה:**
 - (True, [sat_name, frame]) – חבילה תקינה
 - (False, ["", ""]) – חבילה לא רלוונטית (שגויה מבחינתנו)
 - (False, ["time", ""]) – חבילה ישנה מידי (כבר עברנו עליהם)
 - בודקת אם החבילה שייכת ללוויין הנכון, אם היא חדשה דיו ומכילה את ה-opcode הנכון לפי הגדרת ה-JSON.
 - רק חבילות שעוברות את כל הבדיקות מוחזרות לפענוח והכנסה למסד הנתונים.
- **שם הפעולה: enter_packets**
 - **טענת כניסה:**
 - packets – אובייקט JSON שמכיל מספר חבילות מ-satNOGS.
 - **טענת יציאה:**
 - חותמת זמן עדכנית
 - מחרוזת סטטוס – time אם הגיע לזמן האחרון, fine אחרת
 - עוברת על כל החבילות (מדף אחד של SatNOGS), בודקת תקינותן, מפענחת ערכים ל-SQL ומכניסה אותם למסד הנתונים.
 - במקרה שנתקל בחבילה ישנה מדי – התהליך נעצר לצורך שמירה על רצף זמנים.
- **infinite_loop**
 - **טענת כניסה:** אין
 - **טענת יציאה:** אין
 - לולאת העבודה הראשית של המערכת:
 - מעבר על כל הלוויינים המוגדרים
 - שליפת נתוני טלמטריה מ-SatNOGS לפי NORAD ID
 - ניהול דפדוף בין עמודים (next)
 - השהיות יזומות למניעת חסימה מה-API
 - עדכון זמני הקליטה האחרונים
 - שמירת הנתונים למסד הנתונים
 - בסיום הריצה או במקרה חריג, נשמר קובץ newestTime.json עם זמני הקליטה העדכניים ביותר.

קובץ server.py

זהו הקובץ הראשי של המערכת, וממנו מופעלים כל שאר הקבצים. בקובץ זה נוצר Thread האחראי על תקשורת ה-HTTPS, וכן מופעלת הלולאה המרכזית של התוכנית, המוגדרת בקובץ DecodeDataInPacket.py.

- משתנה גלובלי
 - שם: website_folder
 - סוג: מחרוזת
 - מטרה: מחזיקה את המיקום בדיסק של קבצי האתר.
- מחלקה ThreadedHTTPServer
 - יורשת מ: socketserver.ThreadingMixIn, http.server.HTTPServer
 - מטרה: ניהול שרת HTTPS התומך בטיפול בבקשות מקבילות, כאשר כל בקשה מטופלת ב-Thread נפרד.
 - מאפיינים: daemon_threads = True המאפשר סגירה אוטומטית של ה-Threads עם סיום הליך התקשורת עם השרת.
- מחלקה Handler
 - יורשת מ: http.server.SimpleHTTPRequestHandler כך שאת תכונות המחלקה לא שיניתי.
 - מטרת המחלקה: טיפול בכל בקשות ה-GET המתקבלות מהלקוח, ניתוח הפרוטוקול הפנימי של המערכת, יצירת תוכן HTML דינמי, והחזרת נתונים בפורמט JSON או קבצים להורדה.
 - שם הפעולה: create_send
 - טענת כניסה:
 - top – ערך בוליאני המציין האם מדובר בבקשה להוספת נתונים לראש הרשימה
 - טענת יציאה:
 - סוג: tuple
 - פרמטרים:
 - מחרוזת HTML להצגה בדפדפן
 - הערך הישן ביותר שהתקבל
 - הערך החדש ביותר שהתקבל
 - הפונקציה מנתחת את פרמטרי הבקשה מתוך כתובת ה-URL, יוצרת את הפרמטרים הדרושים לשליפת הנתונים, פונה לפונקציה ייעודית ליצירת HTML, ושולחת כותרות HTTP מתאימות ללקוח. פועל עבור ההכנסות שאינם ההכנסה הראשונה.
 - שם הפעולה: do_GET
 - טענת כניסה: אין
 - טענת יציאה: אין
 - הפונקציה מטפלת בכל בקשות ה-GET לשרת, ומחליפה את הפעולה המקורית של המחלקה האב. הפונקציה מזהה את סוג הבקשה בהתאם לנתיב ה-URL ופועלת לפי הפרוטוקול.

במקרה של שגיאה, מוחזרת תשובת שגיאה מתאימה ללקוח.
 בבקשות שאינן דינמיות, הפונקציה מגישה קבצי Web סטטיים מתוך תיקיית
 האתר, זה נלקח מההורשה.

- שם הפעולה: https_server

- טענת כניסה: אין
- טענת יציאה: אין
- הפונקציה יוצרת שרת HTTPS המאזין לפורט 443, מגדירה הצפנת SSL באמצעות תעודה דיגיטלית, ומפעילה את השרת במצב ריצה רציף.

- שם הפעולה: main

- טענת כניסה: אין
- טענת יציאה: אין
- הפונקציה הראשית של המערכת.
 - יצירת טבלאות SQL נדרשות במידת הצורך
 - יצירת דף Web דינמי מתוך תבנית HTML
 - אתחול מנגנון קליטת נתוני טלמטריה
 - הפעלת שרת ה-HTTPS ב-Thread נפרד
 - ניהול לולאת העבודה הראשית של המערכת
 - בסיומה, הפונקציה מבצעת סגירה מסודרת של Threads, חיבורים למסד הנתונים ותהליכים פעילים.

קובץ ServerSetup.py

הקובץ אחראי על יצירת תעודות SSL עבור השרת באמצעות OpenSSL, בנוסף על התקנת המודולים הנחוצים עבור python.

1. בדיקת תעודות קיימות:

אם לא קיימות תעודות עבור ה-rootCA ייוצרו תעודות חדשות.

2. יצירת מפתח ושרת SSL:

נוצר מפתח פרטי לשרת (server.key) ובקשה להנפקת תעודה. לאחר מכן, התעודה נחתמת על ידי ה-CA ומונפקת לשרת.

3. שליחת בקשות pip:

קודם כל בודק איפה pip אם לא נמצא תהיה שגיאה צריך להתקין pip. לאחר שנראה שהותקן pip מתקין את החבילות שאינן מגיעות עם python.

המטרה היא לאפשר יצירת תעודות SSL מאובטחות לשרת ומתקין את המודולים הנחוצים (כמו requests, pandas). דבר זה מאפשר עבודה יותר קלה כאשר רוצים להתקין את השרת על מחשב אחר.

הקוד מתנהל באמצעות subprocess.

מסמך בדיקות

לאורך תקופת תכנון, תכנות וגם בסיום נעשו בדיקות מרובות על התוכנה.

מסמך זה מייצג את כל הבדיקות לאחר סיום הפרויקט, כך שהתוצאות הצפויות הן לבסוף גם התוצאות בפועל.

במהלך הפיתוח לא כל הבדיקות עברו בהצלחה והיה צריך לשנות דברים אך לבסוף הכל עבר, תהיה דוגמא לכך בחלק של **אלגוריתמים שמצאתי מעניינים**.

סוג בדיקה	תת-סוג	תיאור הבדיקה	תוצאה בפועל	הערות
פונקציונלית	יצירת טבלאות	יצירת טבלאות חדשות	הטבלאות נוצרו כנדרש	במקרה של שגיאה - הצגה ותיקון לפני המשך ריצה
		יצירת טבלאות עם שגיאות	התגלתה שגיאה והריצה הופסקה (התנהגות רצויה)	
		ניסיון יצירת טבלאות קיימות	הטבלאות הקיימות לא נמחקו	עבר בהצלחה
	קבלת מידע	ניסיון קבלת מידע מ-SatNOGS	התקבל העמוד הראשון	המשך מעבר בין עמודים
		ניסיון קבלת כמות גדולה של מידע	מתקבלת שגיאה מ-SatNOGS	המתנה של יום לפני ניסיון נוסף
		עבודה עם לוויין ישן	שימוש בקובץ CSV	מומלץ כדי להימנע מהחלפת token
אוטומציה	פענוח מידע	פענוח פאקטה אחת לפי JSON	הפאקטה פוענחה בהצלחה	
		פענוח פאקטה מלוויין אחר	הפאקטה פוענחה לפי ה-JSON המתאים	
	סינון	בדיקת דף SatNOGS מלא	המידע סונן ללוויין הנכון	אימות לפי RAW
		הכנסת שאילתה ל-DB	נוצרה שאילתת INSERT והוזנה לטבלה	
אינטגרציה	לולאה אוטומטית	חיבור כל הרכיבים	הלולאה רצה ומכניסה פאקטות ל-DB	עבודה רציפה
	עיצוב	יצירת עיצוב מפוברק	הושג מראה רצוי	
אתר	HTTPS	בדיקת חיבור מאובטח	החיבור עבד	תוקן מצב "Not Secure"
		בקשת 25 פאקטות עדכניות	התקבלו 25 פאקטות מותאמות לרזולוציה	
	פרוטוקול	גלילה מטה	נשלחה בקשה לפאקטות נוספות	עובד כמצופה
		המתנה שעתיים	בדיקה למידע חדש	המידע מתעדכן
		בקשת כל הפאקטות	התקבל קובץ Excel מלא	
		בקשת 50 פאקטות	התקבל Excel עם 50 פאקטות	אומת ע"י פתיחה

אתר	פרוטוקול	בקשה מזמן מסוים	התקבל Excel מהזמן המבוקש והלאה
		בקשה בין זמנים	התקבל Excel בין הזמנים
אינטגרציה	ריבוי לולאות	בדיקות קצה טווח זמנים בלי מידע עבודה במקביל של 2 לולאות	מוחזר Excel ריק התנהגות צפויה
	HTTPS	חיבור ממחשב נוסף	החיבור עבר ללא בעיות
חיבור חיצוני	רזולוציה	בדיקה ברזולוציות שונות	הפורמט משתנה לפי רוחב המסך
	הרצה ארוכה	הרצת המערכת למשך 24-72 שעות ברצף	המערכת ממשיכה לפעול ללא קריסות
		קבלת שגיאות מ-SatNOGS במהלך ריצה	המערכת מזהה שגיאה וממשיכה לאחר זמן המתנה (אם השגיאה מסוג שמאפשר זאת)
	מסד נתונים	הכנסת נתונים לאורך זמן	הנתונים נכתבים ל-DB באופן עקבי
ביצועים	עומס נתונים	הכנסת כמות גדולה של פאקטות בפרק זמן קצר	המערכת עומדת בעומס
	שאליות	שליפת מידע ממסד נתונים גדול	זמני השליפה נשארים סבירים
	כפילויות	ניסיון הכנסת פאקטות כפולות	לא נוצרות כפילויות ב-DB
	עקביות	השוואת מידע מפוענח ל-RAW	המידע תואם למידע המקורי
שלמות מידע	שינוי במבנה	שינוי במבנה JSON חיצוני	מתקבלת שגיאה מבוקרת (אם יש אפשרות)
	אתחול מערכת	עצירה והפעלה מחדש של המערכת	המערכת עולה בהצלחה
תפעול	המשכיות	הפעלה מחדש לאחר עצירה	המשך עבודה בצורה מבוקרת
	התאוששות	אתחול בזמן פעילות לולאות	נסגר בצורה מבוקרת והלולאות מתחדשות בצורה תקינה

אלגוריתמים שאני מוצאת מעניינים

האלגוריתם הראשון שאציג אינו נראה זהה למימושו בקוד, מאחר שבחרתי לארגן אותו כך שניתן יהיה להציגו במלואו בצורה קריאה בפורמט A4 בספר. באלגוריתם זה אני אראה דוגמא לשינוי שנעשה כתוצאה מהבדיקות.

```
def make_for_html(sat_name, last_date, top, limit=0, width):
    """
    take from SQL and put it in the format for the html. it's added to what I already have.
    :param width: the size of the windows when opened
    :param sat_name: name of sat as written in SATELLITES
    :param last_date: in time unix. say from where I need to take. (as time come) - type(str) but have int.
    :param top: is for the top of the page or for the bottom. according to that I know if I gave the min I have or the max
    :param limit: how many packet I take. if 0 take all.
    :return: build for html, min_date, max_date <- know what to take according to top or bottom.
    """
    # how many in a row.
    count_raw = (
        math.ceil(width / 300)
        if width / 300 - math.floor(width) >= 0.8
        else math.floor(width / 300)
    ) # get how to separate the raw

    limit = f"LIMIT {limit}" if limit != 0 else ""

    sat: dict = SATELLITES[sat_name] # get dict of the sat.

    sql_query = (
        f"SELECT * FROM {sat['table_name']} "
        f"WHERE {JSONS_FOR_HTML[sat['json']]['primaryKey']} "
        f"{'>' if top else '<'} {last_date} "
        f"ORDER BY {JSONS_FOR_HTML[sat['json']]['primaryKey']} DESC {limit}"
    )

    with sat["threadLock"]: # when I can execute the query.
        sat["cursor"].execute(sql_query)
        data = sat["cursor"].fetchall()

    html_code = ""

    json_sorted = dict(
        sorted(JSONS_FOR_HTML[sat['json']].items())
    ) # sort according to dictionary place

    time_params = json_sorted["time_param"] # what are the ones who need change to date.
    primary = json_sorted["primaryKey"] # who is the primaryKey (need only at the end)

    del json_sorted["primaryKey"], json_sorted["time_param"]

    count = (
        count_raw
        if count_raw < len(json_sorted.keys())
        else len(json_sorted.keys())
    ) # get how to separate the subtypes

    width = 100 // count - 3 # count of % of subtype size

    for row in data: # go over each row that I got from the SQL (have 25 each time)
        # the start of the full part and the raw container
        html_code += (
            '<div class="containerPacket">\n'
            '<div class="divRaw">\n'
            '<div style="text-align: center"><u><b>Hex raw:</b></u></div>\n'
            f'<div class="divHexRaw">{get_raw(row["raw"], upper(), count_raw)}</div>'
            '</div>\n'
        )

        already_did = 0

        for i in range(1, len(json_sorted) // count + 1):
            html_code += '<div class="container">\n'

            for k in range(count):
                html_code += (
                    f'<div class="containerSub" style="width:{width}%>\n'
                    f'<div class="titleSub">{list(json_sorted.keys())[already_did]}</div>\n'
                    '<hr />\n'
                )

            for param_name in json_sorted[list(json_sorted.keys())[already_did]]:
                html_code += (
                    f'<u>{param_name}</u> '
                    f'{row[param_name] if param_name not in time_params else datetime.fromtimestamp(row[param_name]).strftime("%d.%m.%Y %H:%M:%S")}'
                    '<br />\n'
                )

            html_code += '</div></div>\n'
            already_did += 1

        html_code += '</div>\n'
```



```
# if len is not divided by count, going at the end for the part that is left
if len(json_sorted) % count != 0:
    html_code += '<div class="container">\n'

    for k in range(len(json_sorted) % count):
        html_code += (
            f'<div class="containerSub" style="width:{width}%">\n'
            f'<div class="titleSub">{list(json_sorted.keys())[k + already_done]}</div>\n'
            f'<hr />\n'
        )

    for param_name in json_sorted[list(json_sorted.keys())[k + already_done]]:
        html_code += (
            f'<u>{param_name}</u>: '
            f'[{row[param_name] if param_name not in time_params else datetime.fromtimestamp(row[param_name]).strftime("%d.%m.%Y %H:%M:%S")}]'
            f'<br />\n'
        )

    html_code += '</div></div>\n'

html_code += '</div>\n'

html_code += '</div>\n'

if not data:
    return "", last_date, last_date # if got nothing
return html_code, data[-1][primary], data[0][primary] # else return
```

אני מתנצלת על כך שהתרשים עדיין קטן, אך זהו הגודל המרבי שניתן היה להציג ושהכול ייכנס.

אלגוריתם זה נמצא ב-DecodeDataInPacket.py. הוא אחראי על לקיחת המידע מהמאגר נתונים ועיצובו לפורמט HTML.

התוצאה של קטע קוד זה הוא רשימת החבילות המוצגות. לדוגמא:

Hex raw:				
00 00 00 00 00 01 8D 00	E9 1E 79 13 FB 0C B8 01	0C 03 39 00 45 00 00 00	0B 00 00 1A 81 41 00 D4	FF 41 00 4C 8C 41 00 8E
FA 41 00 BC 24 42 00 E8	3B 41 74 76 7C 69 00 10	BA 3C 00 00 00 00 00 A0	F1 77 00 90 37 3B 00 00	00 00 14 00 00 00 52 0E
1F 00 02 00 00 00 76 00	00 00 00 00 00 00 71 DF	CA 22 E9 EB 87 22 F8 FF	FF FF 6F BB 36 40 F7 75	7C 69 D2 00 00 00 00 00
00 0D 00 00 00 8F 47 97	41 8F 47 97 41 B4 04 59	3E 4A 25 F4 3E 63 6A 14	C6 00 00 D7 C2	

EPS	OBC	Payload	Solar Panels	TRXVU
voltBattery: 7913 volt_5V: 4985 volt_3V3: 3323 charging_power: 440 consumed_power: 780 electric_current: 57 current_3V3: 69 current_5V: 0 bat_temp: 11 eps_state: operational	sat_time: 30.01.2026 11:14:28 free_memory: 1018826752 corrupt_bytes: 0 total_memory: 2012323840 used_bytes: 993497088 lastFS_error: 0 number_of_resets: 20 sat_uptime: 2035282 num_of_cmd_resets: 2	sel_counter: 118 sen_counter: 0 sensor_one_radfet: 583720817 sensor_two_radfet: 579333097 radfet_temp: 22.73217773437497 last_radfet_read_time: 30.01.2026 11:12:23 power_payload: 210 payload_kill_flag: 0 change_in_mode: 13	solar_panels0: 16.1376953125 solar_panels1: 31.978515625 solar_panels2: 17.537109375 solar_panels3: 31.3193359375 solar_panels4: 41.18359375 solar_panels5: 11.744140625	temp_pa: 18.909940719604492 temp_board: 18.909940719604492 reflected_power: 0.2119320034980774 forward_power: 0.47684699296951294 doppler: -9498.5966796875 rssi: -107.5

הפונקציה פועלת באופן הבא:

1. תחילה היא בודקת כמה חבילות יש לשלף ממסד הנתונים, וכן את מספר הבלוקים שיוצג בכל שורה.
2. לאחר מכן נוצרת השאילתה, והיא מורצת כאשר ה-Lock פנוי.
3. הפרמטרים שאינם קשורים לפיענוח החבילות לפי בלוקים מופרדים ומסירים אותם מהמילון.
4. פועלת לולאה שחוזרת על עצמה ככמות החבילות שנלקחו מהמסד נתונים.
 - a. נוצרת מסגרת הפתיחה של תצוגת החבילה.
 - b. נוצר אזור אחסון לנתוני ה-RAW, והמידע מוכנס אליו בפורמט HexDump.
 - c. מאופסת הספירה של מספר הקוביות שכבר נוצרו
 - d. מופעלת לולאה נוספת באורך מספר השורות הנדרש (בהתאם לערך count ולכמות הפרמטרים במילון):
 - i. נכתב שם הקובייה. לדוגמא: EPS.
 - ii. מתחילה לולאה בגודל count:
1. נוצר אזור אחסון פנימי (הקוביות הלבנות).
2. מתבצעת לולאה על כל פרמטר ברשימה, כאשר כל פרמטר מוכנס לקובייה המתאימה. פרמטרים מסוג זמן עוברים המרה לפורמט זמן.

3. אזור האחסון הפנימי נסגר, ומעודכנת הספירה של הקוביות שכבר נוצרו.

iii. במקרה שמספר הפריטים אינו מתחלק ללא שארית בגודל הלולאה, מופעלת שוב לולאה דומה, אך בגודל מספר הפריטים שנותרו (אורך רשימת הקוביות % הכמות).

e. תצוגת החבילה נסגרת, על מנת לאפשר הוספת חבילה נוספת.

במהלך הבדיקות עלתה בעיה הקשורה לגדלי מסך שונים, אשר הפונקציה בתחילה לא טיפלה בה. כתוצאה מכך, ברזולוציות קטנות או גדולות מהרגיל, התצוגה השתבשה.

לשם כך נוסף פרמטר המתאר את רוחב המסך. אמנם שינוי גודל המסך במהלך הריצה אינו נתמך, אך לאחר ריענון העמוד התצוגה מותאמת כראוי ומסודרת בהתאם לגודל המסך.

ההסתמכות על גודל המסך ההתחלתי נובעת מהאופן שבו המידע מפורק ומוצג. שינוי שיטת הפירוק בזמן ריצה היה מחייב מעבר חוזר על כלל החבילות, דבר שאינו יעיל ואינו נדרש. בנוסף, ההעדפה היא שהאתר יפעל במסך מלא.

להלן דוגמה להשוואה בין המצב הקודם לבין המצב הנוכחי, הן במסכים קטנים והן במסכים גדולים.

Hex raw:	
00 00 00 00 01 8D 00	E9 1E 79 13 FB 0C B8 01
0C 03 39 00 45 00 00 00	0B 00 00 1A 81 41 00 D4
FF 41 00 4C 8C 41 00 0E	FA 41 00 BC 24 42 00 E8
3B 41 74 76 7C 69 00 10	BA 3C 00 00 00 00 00 A0
F1 77 00 90 37 3B 00 00	00 00 14 00 00 00 52 0E
1F 00 02 00 00 00 76 00	00 00 00 00 00 00 71 DF
CA 22 E9 EB 87 22 F8 FF	FF FF 6F BB 36 40 F7 75
7C 69 D2 00 00 00 00 00	00 00 00 00 8F 47 97
41 8F 47 97 41 B4 04 59	3E 4A 25 F4 3E 63 6A 14
C6 00 00 D7 C2	

EPS	OBC
voltBattery: 7913	sat_time: 30.01.2026 11:14:28
volt_5V: 4985	free_memory: 1018826752
volt_3V3: 3323	corrupt_bytes: 0
charging_power: 440	total_memory: 2012323840
consumed_power: 780	used_bytes: 993497088
electric_current: 57	lastES_error: 0
current_3V3: 69	number_of_resets: 20
current_5V: 0	sat_uptime: 2035282
bat_temp: 11	num_of_cmd_resets: 2
eps_state: operational	

Payload	Solar Panels
sel_counter: 118	solar_panels0: 16.1376953125
sen_counter: 0	solar_panels1: 31.978515625
sensor_one_radfet: 583720817	solar_panels2: 17.537109375
sensor_two_radfet: 579333097	solar_panels3: 31.3193359375
radfet_temp: 22.73217773437497	solar_panels4: 41.18359375
last_radfet_read_time: 30.01.2026	

אחרי

Hex raw:	
00 00 00 00 01 8D 00	E9 1E 79 13 FB 0C B8 01
45 00 00 00 00 00 00 1A	81 41 00 D4 FF 41 00 4C
FA 41 00 BC 24 42 00 E8	3B 41 74 76 7C 69 00 10
00 00 00 A0 F1 77 00 90	37 3B 00 00 00 00 14 00
1F 00 02 00 00 00 76 00	00 00 00 00 00 00 71 DF
87 22 F8 FF FF FF 6F BB	36 40 F7 75 7C 69 D2 00
00 00 00 00 8F 47 97	41 8F 47 97 41 B4 04 59
3E 63 6A 14 C6 00 00 D7	C2

EPS	OBC	Payload	Solar Panels	TRXVU
voltBattery: 7913	sat_time: 30.01.2026 11:14:28	sel_counter: 118	solar_panels0: 16.1376953125	temp_pa: 18.90994071966
volt_5V: 4985	free_memory: 1018826752	sen_counter: 0	solar_panels1: 31.978515625	temp_board: 18.90994071966
volt_3V3: 3323	corrupt_bytes: 0	sensor_one_radfet: 583720817	solar_panels2: 17.537109375	reflected_power: 0.211932003498
charging_power: 440	total_memory: 2012323840	sensor_two_radfet: 579333097	solar_panels3: 31.3193359375	forward_power: 0.476846992966
consumed_power: 780	used_bytes: 993497088	radfet_temp: 22.732177734374	solar_panels4: 41.18359375	doppler: -9498.59667968
electric_current: 57	number_of_resets: 20	last_radfet_read: 30.01.2026 11:12:23	solar_panels5: 11.744140625	rssr: -107.5
current_3V3: 69	sat_uptime: 2035282	lastES_error: 0		
current_5V: 0	num_of_cmd_resets: 2	power_payload: 210		
bat_temp: 11		payload_kill_flag: 0		
eps_state: operational		change_in_mode: 13		

ישן

tevel 11

HoopoeWorld

Fri, 30 Jan 2026
18:12:38 UTC

Hex raw:

00 00 00 00 00 01 8D 00 E9 1E 79 13 FB 0C B8 01 0C 03 39 00 45 00 00 00 00 00 0A 1A 81 41 00 D4 FF 41 00 4C 8C 41 00 8E
FA 41 00 BC 24 42 00 E8 3B 41 74 76 7C 69 00 10 BA 3C 00 00 00 00 00 A0 F1 77 00 90 37 3B 00 00 00 00 14 00 00 00 52 0E
1F 00 02 00 00 00 76 00 00 00 00 00 00 71 DF CA 22 E9 EB 87 22 F8 FF FF FF 6F BB 36 40 F7 75 7C 69 D2 00 00 00 00 00
00 00 00 00 00 8F 47 97 41 8F 47 97 41 84 04 59 3E 4A 25 F4 3E 63 6A 14 C6 00 00 D7 C2

EPS

voltBattery: 7913
volt_5V: 4985
volt_3V3: 3323
charging_power: 440
consumed_power: 780
electric_current: 57
current_3V3: 69
current_5V: 0
bat_temp: 11
eps_state: operational

OBC

sat_time: 30.01.2026 11:14:28
free_memory: 1018826752
corrupt_bytes: 0
total_memory: 2012323840
used_bytes: 993497088
lastFS_error: 0
number_of_resets: 20
sat_uptime: 2035282
num_of_cmd_resets: 2

Payload

sel_counter: 118
sea_counter: 0
sensor_one_radlet: 583720817
sensor_two_radlet: 579333097
radlet_temp: 22.73217773437497
last_radlet_read_time: 30.01.2026 11:12:23
power_payload: 210
payload_kill_flag: 0
change_in_mode: 13

Solar Panels

solar_panel0: 16.1376953125
solar_panel1: 31.978515625
solar_panel2: 17.537109375
solar_panel3: 31.3193359375
solar_panel4: 41.18359375
solar_panel5: 11.744140625

TRXVU

temp_pa: 18.909940719604492
temp_board: 18.909940719604492
reflected_power: 0.2119320034980774
forward_power: 0.47684699296951294
doppler: -9498.5966796875
rsst: -107.5

Hex raw:

00 00 00 00 00 01 8D 00 EA 1E 7B 13 FC 0C B8 01 0C 03 40 00 4C 00 00 00 00 00 00 74 75 41 00 DA 02 42 00 20 8C 41 00 11
00 42 00 89 21 42 00 AC 3D 41 68 76 7C 69 00 10 BA 3C 00 00 00 00 00 A0 F1 77 00 90 37 3B 00 00 00 00 14 00 00 00 46 0E
1F 00 02 00 00 00 76 00 00 00 00 00 00 71 DF CA 22 E9 EB 87 22 F8 FF FF FF 6F BB 36 40 F7 75 7C 69 C8 00 00 00 00 00
00 00 00 00 00 7F AA 96 41 7F AA 96 41 84 04 59 3E 4A 25 F4 3E EA FD 17 C6 00 00 CF C2

EPS

voltBattery: 7914
volt_5V: 4987
volt_3V3: 3324
charging_power: 440
consumed_power: 780
electric_current: 64
current_3V3: 76
current_5V: 0
bat_temp: 11
eps_state: operational

OBC

sat_time: 30.01.2026 11:14:16
free_memory: 1018826752
corrupt_bytes: 0
total_memory: 2012323840
used_bytes: 993497088
lastFS_error: 0
number_of_resets: 20
sat_uptime: 2035270
num_of_cmd_resets: 2

Payload

sel_counter: 118
sea_counter: 0
sensor_one_radlet: 583720817
sensor_two_radlet: 579333097
radlet_temp: 22.73217773437497
last_radlet_read_time: 30.01.2026 11:12:23
power_payload: 200
payload_kill_flag: 0
change_in_mode: 13

Solar Panels

solar_panel0: 15.3408203125
solar_panel1: 32.712890625
solar_panel2: 17.515625
solar_panel3: 32.0166015625
solar_panel4: 40.4306640625
solar_panel5: 11.8544921875

TRXVU

temp_pa: 18.833250045776367
temp_board: 18.833250045776367
reflected_power: 0.2119320034980774
forward_power: 0.47684699296951294
doppler: -9727.478515625
rsst: -103.5

ישן

tevel 11

HoopoeWorld

Fri, 30 Jan 2026
18:13:07 UTC

Hex raw:

00 00 00 00 00 01 8D 00 E9 1E 79 13 FB 0C B8 01 0C 03 39 00 45 00 00 00 00 00 0A 1A 81 41 00 D4 FF 41 00 4C 8C 41 00 8E
3B 41 74 76 7C 69 00 10 BA 3C 00 00 00 00 00 A0 F1 77 00 90 37 3B 00 00 00 00 14 00 00 00 52 0E 1F 00 02 00 00 00 76 00
CA 22 E9 EB 87 22 F8 FF FF FF 6F BB 36 40 F7 75 7C 69 D2 00 00 00 00 00 00 00 00 00 8F 47 97 41 8F 47 97 41 84 04 59
C6 00 00 D7 C2

EPS

voltBattery: 7913
volt_5V: 4985
volt_3V3: 3323
charging_power: 440
consumed_power: 780
electric_current: 57
current_3V3: 69
current_5V: 0
bat_temp: 11
eps_state: operational

OBC

sat_time: 30.01.2026 11:14:28
free_memory: 1018826752
corrupt_bytes: 0
total_memory: 2012323840
used_bytes: 993497088
lastFS_error: 0
number_of_resets: 20
sat_uptime: 2035282
num_of_cmd_resets: 2

Payload

sel_counter: 118
sea_counter: 0
sensor_one_radlet: 583720817
sensor_two_radlet: 579333097
radlet_temp: 22.73217773437497
last_radlet_read_time: 30.01.2026 11:12:23
power_payload: 210
payload_kill_flag: 0
change_in_mode: 13

Solar Panels

solar_panel0: 16.1376953125
solar_panel1: 31.978515625
solar_panel2: 17.537109375
solar_panel3: 31.3193359375
solar_panel4: 41.18359375
solar_panel5: 11.744140625

TRXVU

temp_pa: 18.909940719604492
temp_board: 18.909940719604492
reflected_power: 0.2119320034980774
forward_power: 0.47684699296951294
doppler: -9498.5966796875
rsst: -107.5

Hex raw:

00 00 00 00 00 01 8D 00 EA 1E 7B 13 FC 0C B8 01 0C 03 40 00 4C 00 00 00 00 00 00 74 75 41 00 DA 02 42 00 20 8C 41 00 11
3D 41 68 76 7C 69 00 10 BA 3C 00 00 00 00 00 A0 F1 77 00 90 37 3B 00 00 00 00 14 00 00 00 46 0E 1F 00 02 00 00 00 76 00
CA 22 E9 EB 87 22 F8 FF FF FF 6F BB 36 40 F7 75 7C 69 C8 00 00 00 00 00 00 00 00 00 7F AA 96 41 7F AA 96 41 84 04 59
C6 00 00 CF C2

EPS

voltBattery: 7914
volt_5V: 4987
volt_3V3: 3324
charging_power: 440
consumed_power: 780
electric_current: 64
current_3V3: 76
current_5V: 0
bat_temp: 11
eps_state: operational

OBC

sat_time: 30.01.2026 11:14:16
free_memory: 1018826752
corrupt_bytes: 0
total_memory: 2012323840
used_bytes: 993497088
lastFS_error: 0
number_of_resets: 20
sat_uptime: 2035270
num_of_cmd_resets: 2

Payload

sel_counter: 118
sea_counter: 0
sensor_one_radlet: 583720817
sensor_two_radlet: 579333097
radlet_temp: 22.73217773437497
last_radlet_read_time: 30.01.2026 11:12:23
power_payload: 200
payload_kill_flag: 0
change_in_mode: 13

Solar Panels

solar_panel0: 15.3408203125
solar_panel1: 32.712890625
solar_panel2: 17.515625
solar_panel3: 32.0166015625
solar_panel4: 40.4306640625
solar_panel5: 11.8544921875

TRXVU

temp_pa: 18.833250045776367
temp_board: 18.833250045776367
reflected_power: 0.2119320034980774
forward_power: 0.47684699296951294
doppler: -9727.478515625
rsst: -103.5

אחרי

```
def decode_data_for_sql(data, json_params, setting):
    """
    getting parameters and by them making the params that needed to create the INSERT to the SQL.
    :param data: Hex that is only from the data section of Ax25
    :param json_params: according to what we will learn getting the params and the type of each.
    :param setting: have the setting of the json
    :return: values for SQL INSERT
    """
    i = setting["sizeof_header"] #where in the packet we start. (where the data is)
    decode_to_sql = []
    for param in json_params:
        if param["type"] != "string": #if not string, I know the size os take the data there
            bytes_from_data = data[i: i + SIZE_IN_BYTES[param["type"]]]
            i += SIZE_IN_BYTES[param["type"]]
        else: #try to find the first "\0" after to know where it ends.
            bytes_from_data = data[i: i + data[i:].index(0x00)]
            i += data[i:].index(0x00)
        if "isBigEndian" in param.keys():
            to_append = decoded_param(bytes_from_data, param["type"], param["isBigEndian"]) #if big send True or False
        else:
            to_append = decoded_param(bytes_from_data, param["type"], setting["isDefaultBigEndian"]) #send default
        if param["type"] == "byte" and "enum" in param.keys():
            to_append = doing_format(to_append, "byte", param["enum"]) #make the format of enum in byte, if we have
        elif "format" in param.keys():
            to_append = doing_format(to_append, param["type"], param["format"]) #format to all the other
        decode_to_sql.append(to_append) #add to the list of variables
    decode_to_sql.append(data.hex()) #also add raw
    return decode_to_sql
```

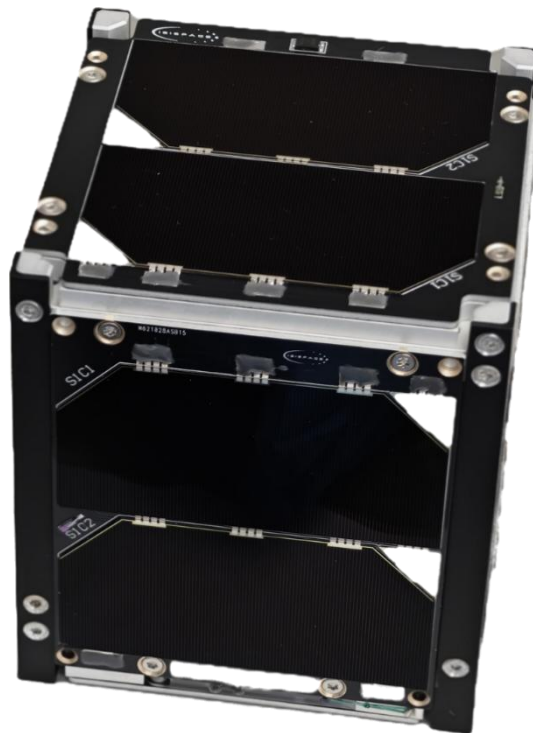
הקוד המוצג לעיל אחראי על יצירת הפרמטרים עבור בקשת ה-INSERT. הנתונים מפוענחים מפורמט Hex בהתאם למבנה ה-JSON.

החבילה מגיעה לשלב זה לאחר שעברה את שלב הסינון, וכאשר פורמט החבילה כבר ידוע ומוגדר מראש.

מהלך הפונקציה (בנוסף למה שסופר כבר קודם):

1. נשמר המיקום ההתחלתי של שדה ה-Data, בהתייחס לפרוטוקול SPL בלבד וללא Ax.25.
2. נוצרת רשימה ריקה המיועדת לאחסן את כלל הפרמטרים המפוענחים, לפי סדר הופעתם.
3. מתבצעת לולאה העוברת על כל פרמטר בנפרד:
 - a. תחילה נבדק האם סוג הפרמטר הוא מחרוזת:
 - i. אם לא – נשלפים התווים בהתאם לגודל המוגדר של סוג הפרמטר.
 - ii. אם כן – נשלפים כל התווים עד להופעה הראשונה של התו "\0", החל מהמיקום הנבדק.
 - b. נבדק האם לפרמטר מוגדר מפתח isBigEndian:
 - i. אם כן – מתבצע פיענוח הפרמטר בהתאם לכך, תוך שימוש במידע שנאסף עד כה.
 - ii. אחרת – מתבצע הפיענוח בהתאם להגדרות ברירת המחדל ולמידע שנאסף.
 - c. נבדק האם סוג הפרמטר הוא byte והאם מוגדר עבורו enum. במקרה כזה, הפיענוח מתבצע בהתאם לפורמט ה-enum.
 - d. בנוסף, אם סוג הפרמטר אינו byte, נבדק האם קיימת קטגוריית format. במידה וקיימת, הפיענוח מתבצע בהתאם לפורמט המוגדר.
 - e. בסיום הטיפול בפרמטר, הערך המפוענח מוכנס לרשימה.
4. לאחר פיענוח כלל הפרמטרים, הנתונים הגולמיים (RAW) נוספים גם הם לסוף הרשימה.
5. הפונקציה מחזירה את רשימת הערכים המפוענחים.

תוכנה מוטסת ללוויין עץ קבצים



רק תיקיות הקוד יוצגו כאן ולא תיקיות הכוללות drivers וכדומה שגם כן נמצאים בתיקייה (רק תיקיית ה-src).

```
GlobusSatProject/src/
├── FRAM_FlightParameters.h
├── GlobalStandards.h
├── InitSystem.c
├── InitSystem.h
├── main_test_code
├── main.c
├── main.h
├── SubSystemModules/
│   ├── Communication/
│   │   ├── AckErrors.h
│   │   ├── AckHandler.c
│   │   ├── AckHandler.h
│   │   ├── CommandDictionary.c
│   │   ├── CommandDictionary.h
│   │   ├── SatCommandHandler.c
│   │   ├── SatCommandHandler.h
│   │   ├── SPL.h
│   │   └── SubsystemCommands/
│   │       ├── EPS_Commands.c
│   │       ├── EPS_Commands.h
│   │       ├── FS_Commands.c
│   │       ├── FS_Commands.h
│   │       ├── Maintenance_Commands.c
│   │       ├── Maintenance_Commands.h
│   │       ├── Payload_Commands.c
│   │       ├── Payload_Commands.h
│   │       ├── TRXVU_Commands.c
│   │       └── TRXVU_Commands.h
│   ├── TRXVU.c
│   └── TRXVU.h
```

- Housekeeping/
 - TelemetryCollector.c
 - TelemetryCollector.h
 - TelemetryFiles.h
- Maintenance/
 - Maintenance.c
 - Maintenance.h
- PowerManagment/
 - EPS.c
 - EPS.h
 - EPSOperationModes.c
 - EPSOperationModes.h
- SysI2CAddr.h
- TestingConfigurations.h
- TestingDemos/
 - MaintenanceTestingDemo.c
 - MaintenanceTestingDemo.h
 - MainTest.c
 - MainTest.h
 - TrxvuTestingDemo.c
 - TrxvuTestingDemo.h
- TLM_management.c
- TLM_management.h
- utils.c
- utils.h

HoopoeWorld עץ קבצים



יהיו את כל הקבצים כולל את חלק מקבצי הבדיקה והתכנון.

```
HoopoeWorld/
├── code/
│   ├── Certificate/
│   │   ├── .rnd
│   │   ├── rootCA.crt
│   │   ├── rootCA.key
│   │   ├── rootCA.srl
│   │   ├── server.cnf
│   │   ├── server.crt
│   │   ├── server.csr
│   │   └── server.key
│   ├── webpage/
│   │   ├── clock.js
│   │   ├── dynamicFunctions.js
│   │   ├── hoopoeWorld.html
│   │   ├── HoopoeWorldStyle.css
│   │   ├── Images/
│   │   │   ├── 5.png
│   │   │   ├── downloadImg.svg
│   │   │   ├── favicon.png
│   │   │   ├── HoopoeWorld.ico
│   │   │   └── logo2v.png
│   │   ├── templateWebsite.html
│   │   └── webPageFake.html
│   ├── ServerSetup.py
│   ├── createSQLTable.py
│   ├── DecodeDataInPacket.py
│   └── server.py
├── data/
│   └── SatDatabase.db
├── doc/
│   ├── config_format.json
│   ├── data_format.json
│   ├── protocol ✱.xlsx
│   └── ספר פרויקט.docx
├── jsons/
│   ├── config.json
│   ├── newestTime.json
│   └── tevels_beacon.json
├── README.md
└── startServer.bat
```

HoopoeWorld מדריך משתמש



התקנה, הפעלה ותחזוקת השרת

הכנת סביבת העבודה

1. **OpenSSL**: במידה ומערכת ההפעלה אינה Windows 11, יש להתקין OpenSSL לפני הכל.
2. **Python**: ודאו שמותקן Python (מומלץ גרסה 3.12).
3. **PATH**: ודאו שגם python וגם pip מוגדרים במשתני הסביבה (PATH) של המחשב.
4. אם יש לכם תעודת חתימה (Certificate) לשרת, rootCA:
 - a. העבירו את התעודה ואת המפתח (Key) לתיקייה הפנימית בשם Certificate.

File Explorer: This PC > Windows 10 (C:) > HoopoeWorld > code > Certificate

Name	Date modified	Type	Size
.rnd	31/01/2026 22:21	RND File	1 KB
rootCA	31/01/2026 22:21	Security Certificate	2 KB
rootCA.key	31/01/2026 22:21	KEY File	4 KB
rootCA.srl	31/01/2026 22:21	SRL File	1 KB
server.cnf	31/01/2026 22:21	CNF File	1 KB
server	31/01/2026 22:21	Security Certificate	3 KB
server.csr	31/01/2026 22:21	CSR File	2 KB
server.key	31/01/2026 22:21	KEY File	4 KB

- b. ודאו שקובץ server.cnf נמצא באותה תיקייה.
- c. עדכנו בתוך הקובץ את כתובת ה-IP לכתובת הנוכחית של המחשב שלכם.

התקנה והרצה

1. התקנת מערכות חיצוניות ו-certificate לשרת: הריצו את הקובץ ServerSetup.py. הוא יתקין אוטומטית את כל הרכיבים הדרושים לפעולת השרת.
2. בדיקה סופית: ודאו שקיימים קובצי ה-JSON של הפיענוח וקובץ ה-config.

File Explorer: This PC > Windows 10 (C:) > HoopoeWorld > jsons

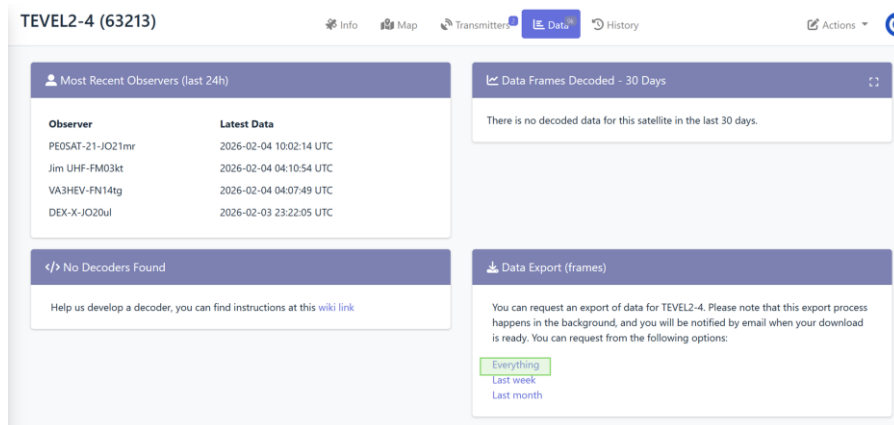
Name	Date modified	Type	Size
config	31/01/2026 22:21	JSON File	1 KB
newestTime	31/01/2026 22:21	JSON File	1 KB
tevels_beacon	31/01/2026 22:21	JSON File	6 KB

- לפחות קיימים קובץ config וקובץ פיענוח, במקרה זה tevels_beacon.
3. הרצה: כעת ניתן להפעיל את השרת, או דרך PyCharm או דרך קובץ bat.

הנחיות תחזוקה

- הוספת לוויין חדש

- אם לוויין לא קיים במאגר שלנו:
 - יש להוסיף את הלוויין ואת כל פרטיו לקובץ jsons/config.json
 - יש להוסיף את הגדרות הפיענוח המתאימות עבורו בקובץ הנפרד.
 - כדי למנוע תקלות מול שרת SatNOGS בעבודה על לוויין עם מידע ישן:
 - הורידו את המידע הקיים כקובץ CSV. קיימת האפשרות כאשר נרשמים ל-SatNOGS

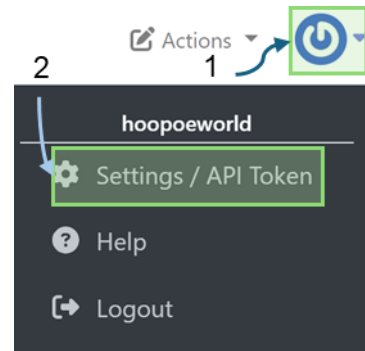


- תריצו בקובץ DecodeDataInPacket את ה-main המכיל הרצה ל-setup
- הכניסו את המיקום של הקובץ ואת שם הלוויין לפי ה-SatNOGS (בדוגמה הספציפית TEVEL2-4)
- אפשר לחזור להריץ את השרת כרגיל

- טיפול בתקלות Token

במקרה של שגיאת Token

- הכנסו לאתר של SatNOGS התחברו למשתמש.
- הנפיקו Token חדש.



- החליפו את ה-Token הישן בקוד לחדש. בקובץ DecodeDataInPacket תחת תכונות מחלקה SatNogsToSQL.

```
class SatNogsToSQL:
    # IMPORTANT: satNogs doesn't like when we get too much in ;
    # have how below. (that mean also change the newestDates)
    def __init__(self, newest_dates=None):
        if not newest_dates: self.newest_dates = {SATELLITES[sat]:
        else:
            self.newest_dates = newest_dates
        for sat in SATELLITES:
            if SATELLITES[sat]["satnogs"] not in newest_dates:
                self.newest_dates[SATELLITES[sat]["satnogs"]
                token = '935188971e64257d0736b4f89f575791312226fb'
                self.__headers = {'Authorization': f'Token {token}'}
```

שימוש בצד לקוח

מכיוון שהשרת חתום עצמית ולא על ידי גורם חיצוני מוכר, עליך להגדיר את הלקוח לכך ש"יבטח" בשרת שלך באופן ידני:

1. התקנת תעודת ה-Root CA:

יש לוודא שקובץ ה-rootCA.crt (הקיים בשרת) נמצא על מחשב הלקוח.

- פתח את ה-PowerShell כמנהל מערכת (Administrator).
- הרץ את הפקודה הבאה כדי להוסיף את התעודה למאגר התעודות המהימנות של המחשב:

```
Import-Certificate -FilePath "C:\path\to\rootCA.crt" -CertStoreLocation  
Cert:\LocalMachine\Root
```

(החלף את הנתיב במיקום האמיתי של הקובץ).

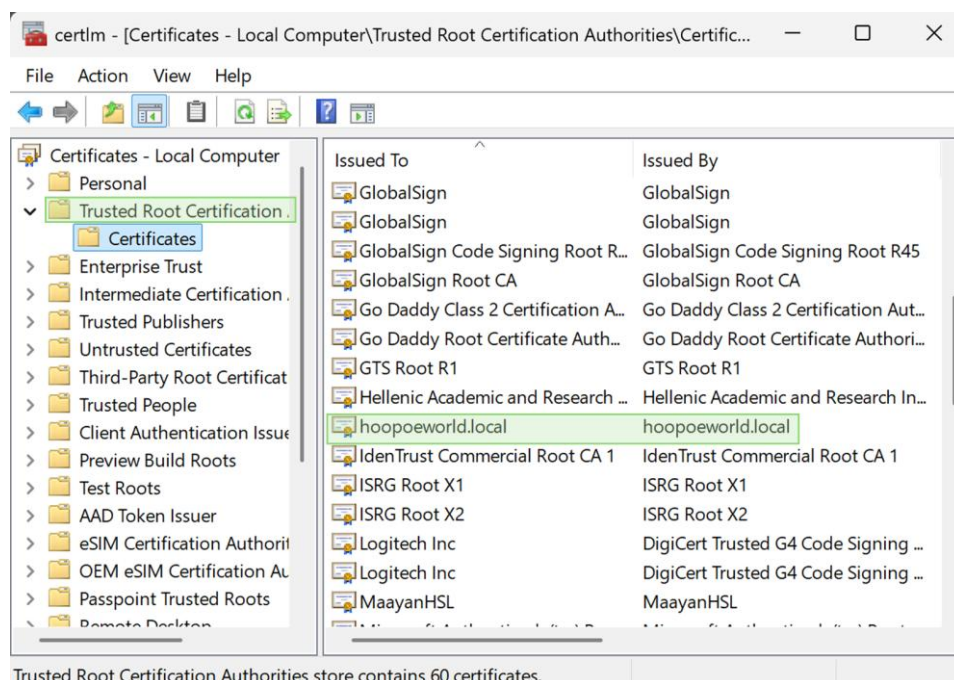
דוגמא:

```
PS C:\WINDOWS\system32> Import-Certificate -FilePath "C:\HoopoeWorld\code\rootCA.crt"  
>> -CertStoreLocation Cert:\LocalMachine\Root
```

2. אימות ההתקנה:

כדי לוודא שהתעודה אכן הותקנה:

- הקש Win + R, הקלד certlm.msc ולחץ Enter.
- נווט לתיקייה: Trusted Root Certification Authorities > Certificates.
- חפש את התעודה המתאימה ברשימה (HoopoeWorld).



3. הגדרת קובץ ה-Hosts (ניתוב DNS):

כדי שהמחשב יזהה את כתובת ה-DNS של השרת:

a. פתח את הקובץ `C:\Windows\System32\drivers\etc\hosts` (באמצעות פנקס רשימות כמנהל).

b. הוסף שורה חדשה עם כתובת ה-IP ושם ה-DNS של השרת.

לדוגמא:

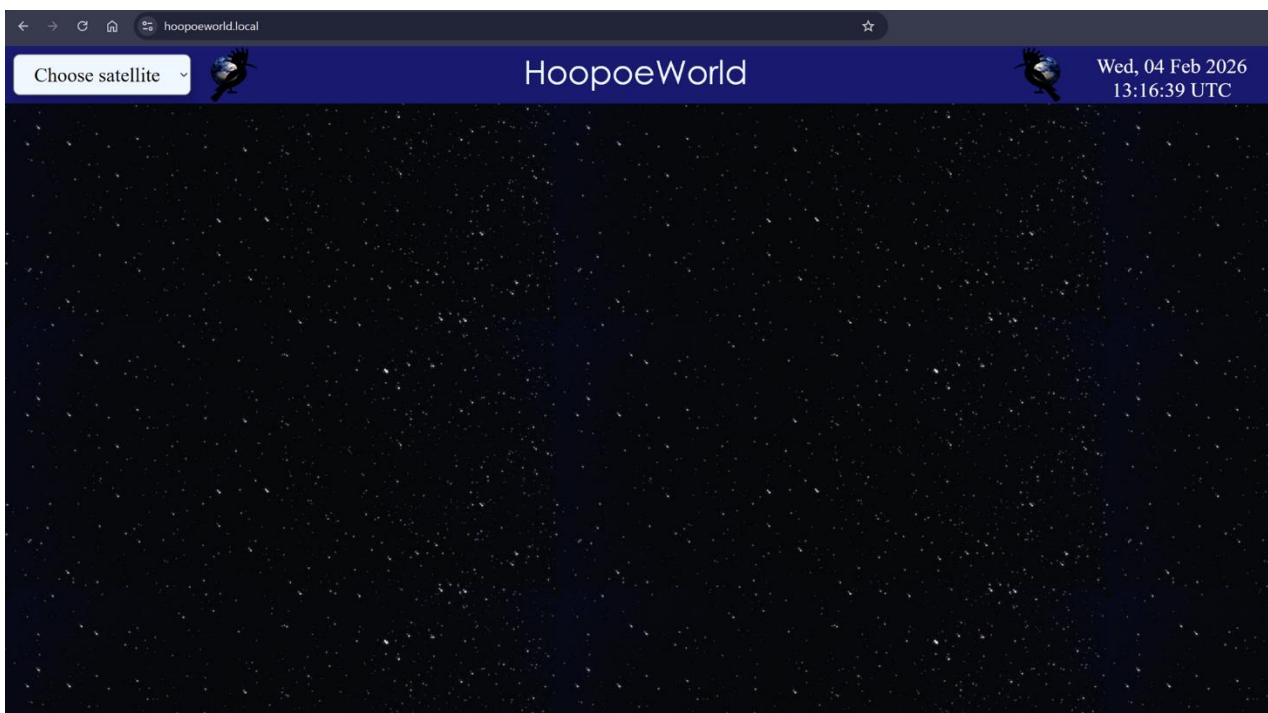
```
hosts
File Edit View H1  B I  ☰  🌐

# Copyright (c) 1993-2009 Microsoft Corp.
#
# This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
#
# This file contains the mappings of IP addresses to host names. Each
# entry should be kept on an individual line. The IP address should
# be placed in the first column followed by the corresponding host name.
# The IP address and the host name should be separated by at least one
# space.
#
# Additionally, comments (such as these) may be inserted on individual
# lines or following the machine name denoted by a '#' symbol.
#
# For example:
#
#       102.54.94.97     rhino.acme.com   # source server
#       38.25.63.10     x.acme.com      # x client host

# localhost name resolution is handled within DNS itself.
#       127.0.0.1       localhost
#       ::1             localhost

172.16.12.119 hoopoe.world.local
```

4. פתח את הדפדפן והקלד את כתובת ה-DNS שהגדרת (`hoopoe.world.local`). האתר אמור לעלות כעת ללא התרעת אבטחה חמורה.



רפלקציה

במהלך הפרויקט היו כמה דברים שעבדו טוב כבר מההתחלה, והרבה דברים שלא – לפחות לא בפעם הראשונה.

בקוד של הלוויין, לדוגמה, הייתה בעיה בקטע קוד של פריסת האנטנות. חיכינו כ-40 דקות בחדר הנקי והאנטנות לא נפרסו. ניסיתי להבין למה זה קורה, והבנתי שהבעיה נבעה מכמות ההגנות ששמתי על מנגנון הפריסה. הורדתי את רובן, ניסיתי לפרוס שוב, הפעם עם זמן המתנה קצר יותר, והכול עבד כמו שצריך.

בעיה נוספת בלוויין התגלתה בבדיקת המעבר בין מצבי מתח. הייתה בעיה בתחום שבין המצבים, מה שגרם לבעיה כללית במערכת מעבר המצבים. לאחר שהסתכלתי על ההתנהגות בפועל וכתבתי את מכונת המצבים הרצויה לעומת מה שקורה בפועל בפונקציה, הבנתי את הבעיה ואת איך צריך לשנות את הפונקציה כך שמעברי המצבים יעבדו בצורה חלקה. לאחר השינוי, בבדיקות, המערכת עבדה כמו שצריך.

בנוסף, בקוד של HoopoeWorld היו לי מדי פעם בעיות קטנות עם הפורמט. מכיוון שרוב ה-HTML נכתב בתוך Python, היה קצת יותר קשה לבדוק דברים במהירות. אפילו שהתחלתי את הכתיבה ישירות ב-HTML ואז להעביר ל-Python, ההמרה לא הייתה לגמרי חלקה, ונדרשו כמה ניסיונות עד שהכול עבד כמו שצריך.

גם בפעולת קבלת החבילות מ-SatNOGS נתקלתי בקשיים – נאלצתי להחליף את ה-Token כמה וכמה פעמים עד שהבנתי מה הבעיה ותיקנתי אותה.

בסופו של דבר אני לא זוכרת את כל הפרטים הקטנים, אבל כמו בהרבה פרויקטים בתוכנה, כל התהליך היה מלא בניסוי ותהייה, וחשיבה על פתרונות יצירתיים לבעיה. גם בתכנות הלוויין וגם בפיתוח HoopoeWorld תמיד צצו בעיות, והיה צריך לפתור אותן – וזה אחד הדברים שאני הכי אוהבת בעולם התוכנה ולמה אני כל כך אוהבת את המקצוע.

במהלך הפרויקט למדתי הרבה דברים: איך לעבוד בצורה עצמאית על פרויקט גדול, איך להתמודד עם לחץ של זמנים, כתיבה בשפת C, עבודה עם מערכות חשמליות, ניהול בין צוותים, וגם נושאים שקשורים לחשמל ותקשורת.

אני חושבת שבסופו של דבר כל דבר שלומדים הוא משהו שלוקחים איתנו הלאה. בין אם זה ניהול זמנים ועבודה תחת לחץ, ובין אם זה למידה תוך כדי תנועה ושיפור היכולות בשפת התוכנה.

בכל אחד מהפרויקטים היו דברים שהייתי עושה אחרת או מוסיפה אם היו לי יותר משאבים:

בלוויין:

- זמן – עם יותר זמן היה אפשר לבצע יותר בדיקות, לתקן או למצוא פתרונות לבעיות נוספות שצצות, ואפילו להחזיר את הלוויין לתיקון של רכיבים מסוימים.

:HoopoeWorld

- כסף – כדי לקנות דומיין ולהפוך את המערכת לשרת HTTPS מלא.
- זמן – מעל שנה וחצי כדי להחליף מערכת קיימת אצלנו ולהוסיף פקודות ופיצ'רים נוספים.

תודות

אני רוצה להודות למספר אנשים בפרויקט הלווין בעיקר.

קודם כל לעדי רבינוביץ', מדריכת התוכנה שלי בלווינים, שליוותה אותי בכל תהליך הלווין, בין אם בהתכוננות להצגות מול גורמים חשובים, בחשיבה משותפת, ובהישארות איתי במרכז בשעות לא שעות ואפילו בשינה במרכז. לא הייתי מצליחה את זה בחיים (לפחות לא במהלך תקופת הזמן שהייתה לי) בלעדיה. אז תודה עצומה 🍀

אני רוצה גם להודות לחברים שלי, בעיקר מהלווינים אך לא רק, שתמכו והיו שם איתי, אפילו שעסקנו בפרויקטים שונים או בחלקים שונים בפרויקט: חומרה, תקשורת ואני בעיקר תוכנה. החוויה הייתה שונה לחלוטין בלעדיהם (אפשר לראות מהתמונות למטה).

בנוסף ארצה להודות לעדן הכט, דוד גרינברג ורועי אורבך, המדריכים האחרים בתוכנית, שעזרו וליוו אותי בתחומים נוספים בתוכנית בין אם בבנייה, בתקשורת, בהתייעצות, או בעזרה נוספת בדיבור מול קהל ובניהול.

לבסוף, ארצה להודות גם לבית הספר ולהורים שלי. לבית הספר קודם כל שאפשר את הפרויקט ואת העבודה המרובה עליו, אפילו אם מדי פעם זה דחה עבודות של בית הספר. וכמובן להורים שלי, שזרמו איתי לאורך כל הדרך בין אם בזמנים שבהם הייתי בבית הספר עד 12 בלילה ועדיין אספו אותי, או כשישנתי שם לילה וסמכו עליי שהכול יהיה בסדר.

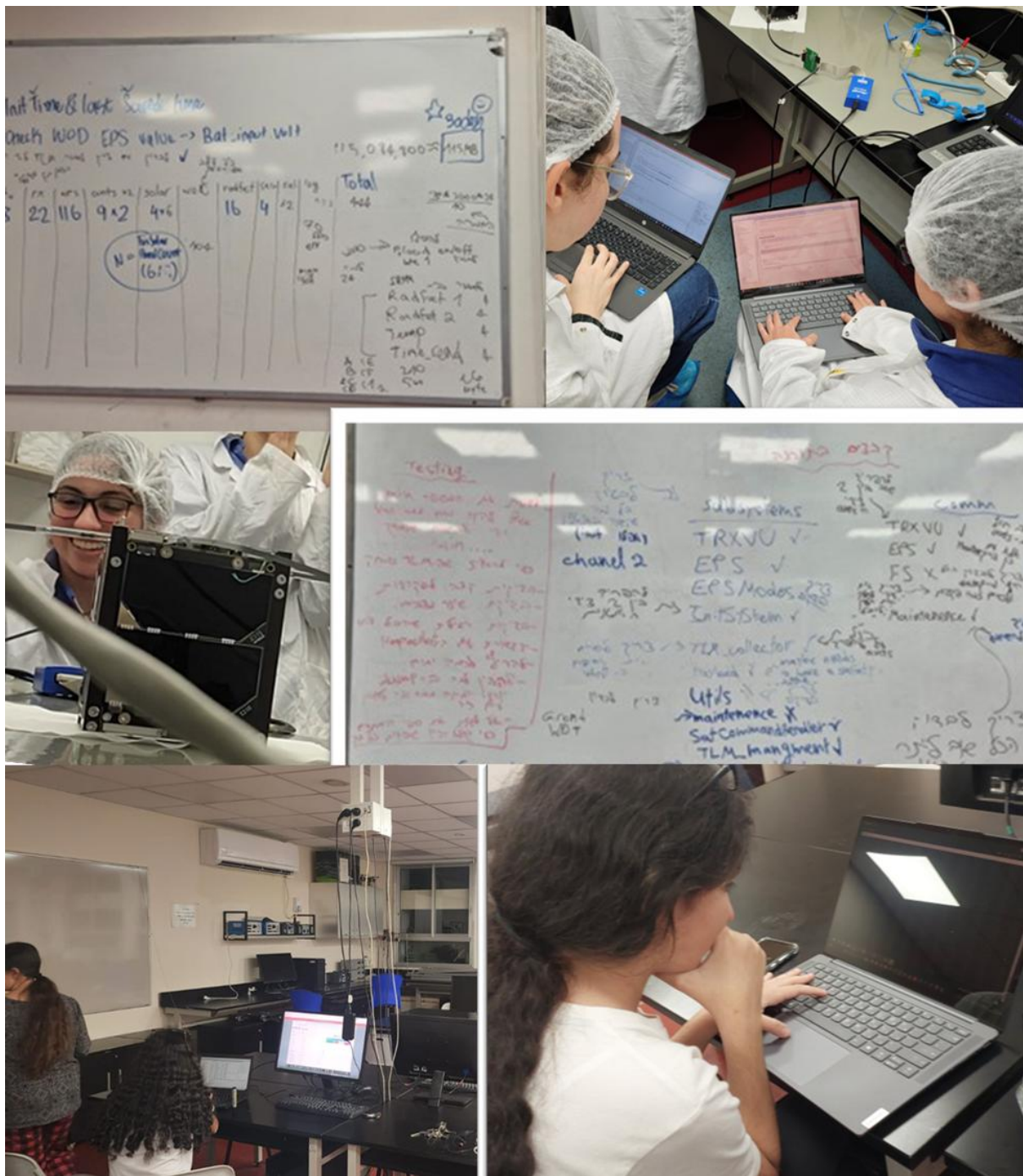
מעריכה את כולכם, מעין כרמי



תמונות ממהלך הפיתוח של הלוויין

במהלך העבודה על הפרויקט הזה חוויתי הרבה חוויות בנוסף לפיתוח הלוויין. בין אם פעילויות בתוכנית, צילומים למקומות, להרצות במקומות שונים, לבלות עם חברים וגם הפיתוח עצמו. אלו היו חוויות שאני לא אשכח ואת חלקם אני רוצה לשתף פה. בין אם תמונות ממהלך תכנות הלוויין, לבין הבניה וגם חוויות לא קשורות ישירות ללוויין.

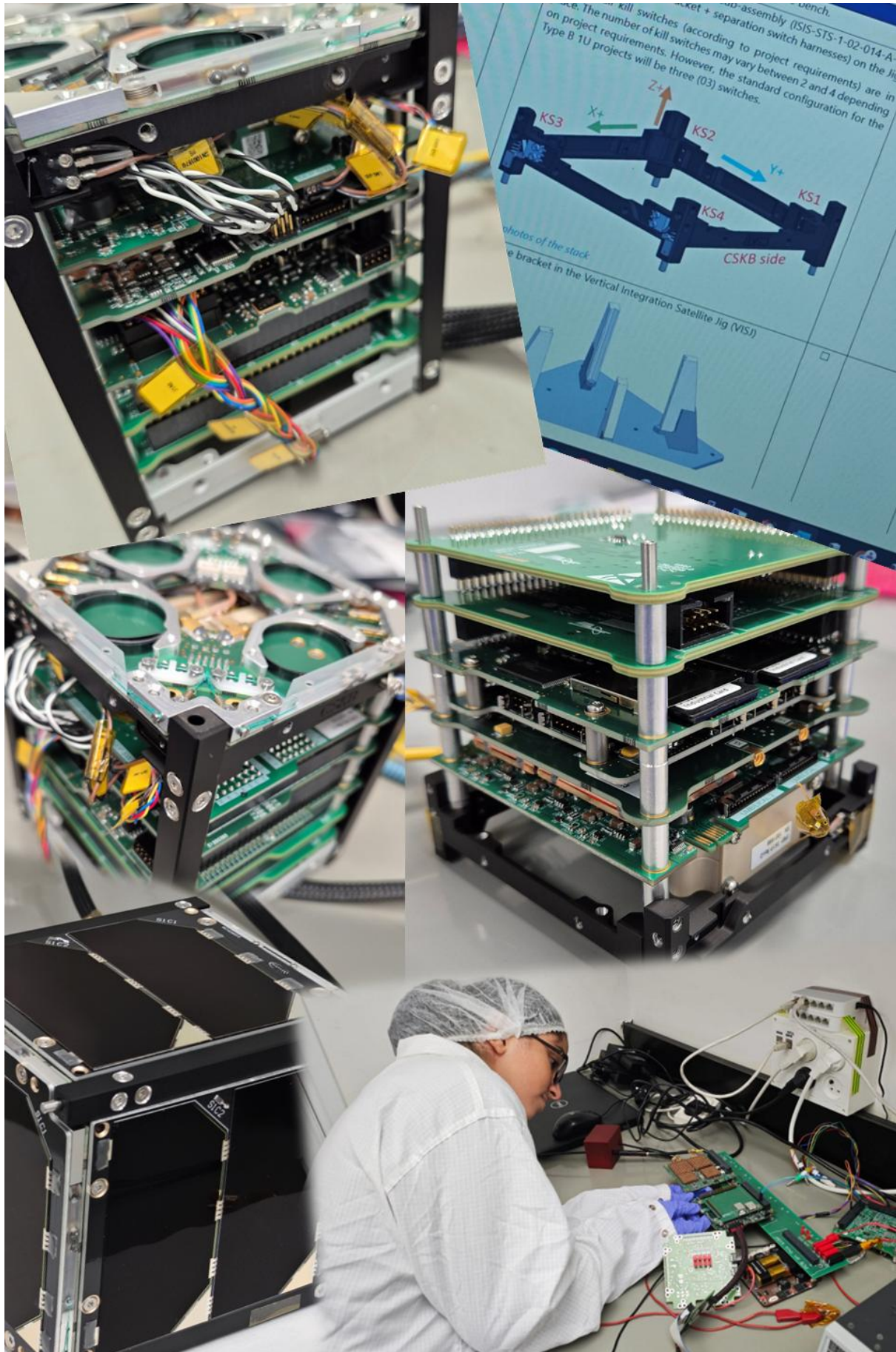
התמונות שיהיו פה מסכמות שנתיים של לוויינים ושל פיתוח הלוויין. הייתי בתוכנית גם לפני וגם המשכתי אחרי אך זאת הייתה חווייה מטורפת שכללה שינה בבית הספר, עבודה כל הזמן, הרבה הצגות אך גם הרבה צחוק וכיף עם חברים.













ביבליוגרפיה ונספחים



לדרייב המכיל את כל הקבצים

נספחים

Satellite

FlightSoftware


```

1 #ifndef MAIN_H_
2 #define MAIN_H_
3
4 #include "hal/boolean.h"
5
6 /*!
7 * @brief checks if communication with the ground station was made in the last
'NO_COMMUNICATION_WDT_KICK_TIME' seconds
8 * @return  TRUE if WDT was kicked and restart is needed
9 *          FALSE if WDT was NOT kicked and everything is fine. no restart needed
10 */
11 Boolean GroundCommunicationWDTKick();
12
13 /*!
14 * @brief The main operation function. This function implements the 'Loop'
15 * side of the logic and handles all of the satellite macro-logics(TRX,EPS etc')
16 * @note the function must use an infinite loop at its base;
17 * according to the logic and FreeRTOS thread requirements
18 */
19 void taskmain();
20
21 /*!
22 * This is the main function of the satellite operation
23 * The main function contains a 'Boot' call which is operated once,
24 * and the taskMain thread which is activated repeatedly.
25 */
26 int main();
27
28 #endif /* MAIN_H_ */
29
30

```

```

1 /*
2  * main.c
3  *
4  * Created on: 5 6 2024
5  * Author: maayan
6  */
7
8 #include "main.h"
9 #include "InitSystem.h"
10
11 #include <stdio.h>
12
13 #include <freertos/FreeRTOS.h>
14 #include <freertos/semphr.h>
15 #include <freertos/task.h>
16
17 #include <hal/Timing/WatchDogTimer.h>
18
19 #include <at91/peripherals/cp15/cp15.h>
20
21 #include "GlobalStandards.h"
22 #include "SubSystemModules/Communication/TRXVU.h"
23
24 #include "SubSystemModules/Housekeeping/TelemetryCollector.h"
25
26 #include "SubSystemModules/Maintenance/Maintenance.h"
27
28 #include "main.h"
29 #include <stdlib.h>
30 #include "TestingDemos/MainTest.h"
31
32 //define Testing_TRXVU 1
33 //define Testing_Maintenance 1
34
35 void taskMain()
36 {
37     WDT_startWatchdogKickTask(10 / portTICK_RATE_MS, FALSE);
38     InitSubsystems();
39
40     while(TRUE)
41     {
42         EPS_Conditioning();
43         TRX_Logic();
44         Maintenance();
45         TelemetryCollectorLogic();
46     }
47 }

```

```

44 }
45
46 // main operation function. will be called upon software boot.

47 int main()
48 {
49     xTaskHandle taskMainHandle;
50     /* xTaskHandle taskTestHandle;*/
51
52     TRACE_CONFIGURE_ISP(DBGU_STANDARD, 2000000, BOARD_MCK);
53     // Enable the Instruction cache of the ARM9 core. Keep the MMU and Data Cache
    disabled.
54     CP15_Enable_I_Cache();
55
56     // The actual watchdog is already started, this only initializes the watchdog-kick
    interface.
57     WDT_start();
58
59     // create the main operation task of the satellite

60     #ifdef TESTING
61         /* xTaskGenericCreate(taskTesting, (const signed char*) "taskTesting", 4096,
62            NULL,
63            configMAX_PRIORITIES - 2, &taskTestHandle, NULL, NULL);*/

64     #else
65         xTaskGenericCreate(taskMain, (const signed char*) "taskMain", 4096, NULL,
66            configMAX_PRIORITIES - 2, &taskMainHandle, NULL, NULL);
67     #endif
68
69
70
71     vTaskStartScheduler();
72     exit(0);
73 }
74

```

```
1 /*
2  * @file SysI2CAddr.h
3  * @brief system I2C addresses
4  */
5
6 #ifndef SYSI2CADDR_H_
7 #define SYSI2CADDR_H_
8
9
10 // Subsystem I2C addresses
11 #define ISIS_TRXVU_I2C_BUS_INDEX 0    //!< index of the trxvu I2C
12
13 #define I2C_TRXVU_TC_ADDR 0x61        //!< I2C address of the transceiver
14
15 #define I2C_TRXVU_RC_ADDR 0x60        //!< I2C address of the receiver
16
17 #define EPS_I2C_BUS_INDEX 0           //!< index of the EPS I2C
18
19 #define EPS_I2C_ADDR 0x20             //!< I2C address of the EPS
20
21 #define ANTS_SIDE_A_I2C_ADDR 0x31
22 #define ANTS_SIDE_B_I2C_ADDR 0x32
23
24 #endif /* SYSI2CADDR_H_ */
```

```

1 /*
2  * utils.h
3  *
4  * Created on: Sep 19, 2019
5  * Author: pc
6  */
7
8 #ifndef UTILS_H_
9 #define UTILS_H_
10
11 #include "GlobalStandards.h"
12 #include <hal/Timing/Time.h>
13
14
15 #define E_CANT_TRANSMIT -200
16 #define E_TOO_EARLY_4_BEACON -201
17 #define E_INVALID_PARAMETERS -204
18 #define TRXVU_MUTE_TOO_LONG -202
19
20 #define TRXVU_IDLE_TOO_LONG -203
21
22 #define TRXVU_TRANSPONDER_TOO_LONG -199
23
24 #define TRXVU_IDLE_WHILE_TRANSPONDER -205
25
26 #define TRXVU_TRANSPONDER_WHILE_MUTE -211
27
28 #define TRXVU_IDEL_WHILE_MUTE -212
29
30 #define BEACON_INTRAVL_TOO_SMALL -206
31
32 #define SPL_DATA_TOO_BIG -207
33 #define INVALID_TLM_TYPE -208
34 #define INVALID_IMG_TYPE -209
35 #define INFO_MSG -210
36 #define SAT_RESET 150
37
38
39 #define MAX_ERRORS 10 // max errors we want to log from the
40 same type toghether
41 #define MAX_TIME_BETWEEN_ERRORS 60 // max seconds we allow
42 between errors
43 #define MAX_LOG_STR 66
44
45 typedef struct data

```

```

38 {
39     int error;
40     char msg[MAX_LOG_STR];
41 } logData_t;
42
43
44     /*
45  * convert unix time to Time struct
46  * @param[in] name=utime; type=time_unix; unix time we want to convert.
47  * @param[out] name=time; type=Time*; where we save the convert time.
48  */
49     void timeU2time(time_unix utime, Time *time);
50
51     /*
52  * save log (error + error msg) -> in FS
53  * @param[in] name=error; type=int; enter the error code for the problem
54  * @param[in] name=msg; type=char*; have the msg of which function had error and
55  * @return type=int; return the error we got.
56  */
57     int logError(int error ,char* msg);
58
59
60
61     #endif /* UTILS_H_ */
62
63

```

```

1 /*
2  * utils.c
3  *
4  * Created on: 5 6 2024
5  * Author: maayan
6  */
7
8 #include <stdio.h>
9 #include <time.h>
10 #include "GlobalStandards.h"
11 #include <hal/Timing/Time.h>
12 #include "utils.h"
13 #include "TLM_management.h"
14 #include <string.h>
15
16 /*
17  * save log (error + error msg) -> in FS
18  * @param[in] name=error; type=int; enter the error code for the problem
19  * @param[in] name=msg; type=char*; have the msg of which function had error and
20  * where. (max length 66)
21  * @return type=int; return the error we got.
22  */
23 int logError(int error ,char* msg)
24 {
25     if(error == 0 || error == 6) return 0; //6 in not a real error
26     logData_t data;
27     data.error = error;
28     memset(data.msg, 0, MAX_LOG_STR);
29     sprintf(data.msg, "%s", msg);
30     printf("%d ERROR - %s\r\n", error, data.msg);
31
32     Write2File(&data, tlm_log);
33     return error;
34 }
35
36 /*
37  * convert unix time to Time struct
38  * @param[in] name=utime; type=time_unix; unix time we want to convert.
39  * @param[out] name=time; type=Time*; where we save the convert time.
40  */
41 void timeU2time(time_unix utime, Time *time)
42 {
43     struct tm *t = localtime((time_t*)&utime);
44     time->year = t->tm_year - 100;
45     time->month = t->tm_mon + 1;

```

```
46  time->date = t->tm_mday;
47  time->hours = t->tm_hour;
48  time->minutes = t->tm_min;
49  time->seconds = t->tm_sec;
50  time->day = t->tm_wday;
51 }
52
```



```

1 /*
2 *
3 * @file   FRAM_FlightParameters.h
4 * @brief  ordering all flight parameters(saved on the FRAM) addresses and sizes
5 * @note   sizes are measured in chars = 1 byte. e.g size = 4, meaning 4 bytes(=int)
6 */
7
8 #ifndef FRAM_FLIGHTPARAMETERS_H_
9 #define FRAM_FLIGHTPARAMETERS_H_
10
11 #include <hal/Storage/FRAM.h>
12
13
14 // <Satellite Management>
15 #define DEPLOYMENT_TIME_ADDR      0x05      //!< time at which the
satellites starts deployment
16 #define DEPLOYMENT_TIME_SIZE      4          //!< size of parameter in
bytes
17
18 #define SECONDS_SINCE_DEPLOY_ADDR  0x09      //!< counts how
many seconds has past since wakeup for use in deployment.
19
20 #define SECONDS_SINCE_DEPLOY_SIZE  4          //!< size of the
parameter in bytes
21
22 #define SD_CARD_USED_ADDR          0x15      //!< save which SD we
are using (0/1)
23 #define SD_CARD_USED_SIZE          1          //!< size of the parameter in
bytes
24
25 #define DEFAULT_NO_COMM_WDT_KICK_TIME (7*24*60*60)  //!< number
of seconds in 7 days
26 #define NO_COMM_WDT_KICK_TIME_ADDR 0x24      //!< number of
seconds of no communications before GS WDT kick
27
28 #define NO_COMM_WDT_KICK_TIME_SIZE 4          //!< size of the
parameter in bytes
29
30 #define STOP_REDEPLOY_FLAG_ADDR    0x30      //!< stop calling ant
deploy every 30 min flag
31 #define STOP_REDEPLOY_FLAG_SIZE    4          //!< length in bytes of
32
33 #define MUTE_END_TIME_ADDR          0x36      //!< mute end time
34
35 #define MUTE_END_TIME_SIZE          4          //!< length in bytes of

```

```

33
34  #define FIRST_ACTIVATION_FLAG_ADDR    0x42    //!< is this the first
activation after launch flag
35  #define FIRST_ACTIVATION_FLAG_SIZE    4        //!< length in bytes of
FIRST_ACTIVATION_FLAG
36
37  #define TRANSPONDER_END_TIME_ADDR      0x48    //!< mute end time

38  #define TRANSPONDER_END_TIME_SIZE      4        //!< length in bytes of

39
40  #define TRANSPONDER_RSSI_ADDR          0x52    //!< mute end time

41  #define TRANSPONDER_RSSI_SIZE          2        //!< length in bytes of

42
43  #define MOST_UPDATED_SAT_TIME_ADDR      0x54    //!< this
parameters saves the sat time to be read after resets

44  #define MOST_UPDATED_SAT_TIME_SIZE      4        //!< size of the
parameter in bytes
45
46  #define NUMBER_OF_RESETS_ADDR          0x60    //!< counts how many
restarts did the satellite endure
47  #define NUMBER_OF_RESETS_SIZE          4        //!< size of the
parameter in bytes
48
49  #define IDLE_END_TIME_ADDR             0x70    //!< mute end time

50  #define IDLE_END_TIME_SIZE             4        //!< length in bytes of

51
52  #define RESET_CMD_FLAG_ADDR            0x105    //!< the flag is raised
whenever a restart is commissioned
53  #define RESET_CMD_FLAG_SIZE            4        //!< size of the parameter
in bytes
54
55  #define NUM_OF_CHANGES_IN_MODE_ADDR    0x110    //!< changes
between cruise to operational
56  #define NUM_OF_CHANGES_IN_MODE_SIZE    4        //!< size of the
parameter in bytes
57
58  #define LAST_TRY_TO_DEPLOY_TIME_ADDR    0x115    //!< save
time_unix of the last time we tried to deploy ants

```

```

59  #define LAST_TRY_TO_DEPLOY_TIME_SIZE  4          //!< size of the
parameter in bytes
60
61  #define TLM_SAVE_PERIOD_START_ADDR      0x130      //!< start of the
save periods in the FRAM
62
63  #define DEFAULT_EPS_SAVE_TLM_TIME       10          //!< save EPS TLM
every 5 seconds
64  #define EPS_SAVE_TLM_PERIOD_ADDR        0x130      //!< address where
the save tlm period will be
65
66  #define DEFAULT_TRXVU_SAVE_TLM_TIME     10          //!< save TRXVU
TLM every 5 seconds
67  #define TRXVU_SAVE_TLM_PERIOD_ADDR      0x134      //!< address
where the save tlm period will be
68
69  #define DEFAULT_ANT_SAVE_TLM_TIME       10          //!< save antenna
TLM every 5 seconds
70  #define ANT_SAVE_TLM_PERIOD_ADDR        0x138      //!< address where
the save tlm period will be
71
72  #define DEFAULT_SOLAR_SAVE_TLM_TIME     10          //!< save solar
panel TLM every 5 seconds
73  #define SOLAR_SAVE_TLM_PERIOD_ADDR      0x13C      //!< address
where the save tlm period will be
74
75  #define DEFAULT_WOD_SAVE_TLM_TIME       10          //!< save WOD TLM
every 5 seconds
76  #define WOD_SAVE_TLM_PERIOD_ADDR        0x140      //!< address where
the save tlm period will be
77
78  #define DEFAULT_RADFET_SAVE_TLM_TIME    15*60      //!< save RADFET
TLM every 15 minutes
79  #define RADFET_SAVE_TLM_PERIOD_ADDR     0x144      //!< address
where the save tlm period will be
80
81  #define DEFAULT_SEU_SEL_SAVE_TLM_TIME   20          //!< save sel and
seu TLM every 20 seconds
82  #define SEU_SEL_SAVE_TLM_PERIOD_ADDR    0x148      //!< address
where the save tlm period will be
83
84
85  #define NUMBER_OF_CMD_RESETS_ADDR       0x160      //!< counts how
many restarts did the satellite endure from ground command
86
87  #define NUMBER_OF_CMD_RESETS_SIZE       4          //!< size of the

```

parameter in bytes

```
87
88     #define DELAYED_CMD_FRAME_COUNT_ADDR    0x200    //!< Address
where amount of frames in delayed buffer is held

89     #define DELAYED_CMD_FRAME_COUNT_SIZE    1        //!< number of
bytes describing the frame count of delayed buffer

90
91     #define LAST_RADFET_READ_START          0x210

92
93     #define LAST_RADFET_ONE_VALUE_ADDR      0x210    //!< the first value
of the fram need to keep after get telemetry.
94     #define LAST_RADFET_ONE_VALUE_SIZE     4        //!< size of the
parameter in bytes
95
96     #define LAST_RADFET_TWO_VALUE_ADDR      0x214    //!< the second
value of the fram need to keep after get telemetry.

97     #define LAST_RADFET_TWO_VALUE_SIZE     4        //!< size of the
parameter in bytes
98
99     #define LAST_RADFET_TEMP_ADDR           0x218    //!< last temp of
RADFET need to keep after get telemetry.
100    #define LAST_RADFET_TEMP_SIZE           8        //!< size of the
parameter in bytes
101
102    #define TIME_LAST_RADFET_READ_ADDR       0x220    //!< last time
save tlm of RADFET
103    #define TIME_LAST_RADFET_READ_SIZE      4        //!< size of the
parameter in bytes
104
105    #define HAD_RESET_IN_A_MINUTE_ADDR       0x230    //!< the flag that
check if we had a reset in less then a minute

106    #define HAD_RESET_IN_A_MINUTE_SIZE      4        //!< size of the
parameter in bytes (Boolean)
107
108    #define PAYLOAD_IS_DEAD_ADDR             0x240    //!< the flag that tales
us if we need to kill the payload or if he is already dead

109    #define PAYLOAD_IS_DEAD_SIZE            4        //!< size of the parameter
in bytes (Boolean)
110
111    #define TRY_TO_DEPLOY_ADDR               0x250    //!
```

to deploy ants

```
112     #define TRY_TO_DEPLOY_SIZE          4          //!< size of the parameter  
in bytes
```

```
113
```

```
114     #define TRANS_ABORT_FLAG_ADDR        0x500      //!< transmission  
abort request flag
```

```
115     #define TRANS_ABORT_FLAG_SIZE        1          //!< size of mute flag in  
bytes
```

```
116
```

```
117     #define EPS_ALPHA_FILTER_VALUE_ADDR  0x550      //!< filtering  
value in the LPF formula
```

```
118     #define EPS_ALPHA_FILTER_VALUE_SIZE  sizeof(float) //!< size of  
double (alpha)
```

```
119
```

```
120     #define EPS_THRESH_VOLTAGES_ADDR     0x666      //!< starting  
address for eps threshold voltages array
```

```
121     #define EPS_THRESH_VOLTAGES_SIZE  
(NUMBER_OF_THRESHOLD_VOLTAGES * sizeof(voltage_t)) //!< number of bytes in  
eps threshold voltages array
```

```
122
```

```
123     #define BEACON_INTERVAL_TIME_ADDR    0x4590     //!< address of  
value of the delay between 2 beacons
```

```
124     #define BEACON_INTERVAL_TIME_SIZE    4          //!< size of parameter  
in bytes
```

```
125
```

```
126     #define LAST_COMM_TIME_ADDR          0x9485     //!< saves the last  
unix time at which communication has occurred
```

```
127     #define LAST_COMM_TIME_SIZE          4          //!< size of last  
communication time in bytes
```

```
128
```

```
129     #define DEFAULT_BEACON_INTERVAL_TIME 20         //!< how many  
seconds between two beacons [sec]
```

```
130     #define MAX_BEACON_INTERVAL          60         //!< beacon every 1  
minute
```

```
131     #define MIN_BEACON_INTERVAL          5          //!< beacon every 5  
seconds
```

```
132
```

```
133     #endif /* FRAM_FLIGHTPARAMETERS_H_ */
```

```
134
```

```

1
2  #ifndef GLOBALSTANDARDS_H_
3  #define GLOBALSTANDARDS_H_
4
5  #include <hal/boolean.h>
6  #include <at91/utility/exithandler.h>
7  #include <hal/errors.h>
8
9  #include "FRAM_FlightParameters.h"
10 #include "SysI2CAddr.h"
11 #include "TestingConfigurations.h"
12 #ifndef NULL
13 #define NULL ((void *)0)
14 #endif
15
16 #ifndef TRUE
17 #define TRUE (0xFF)      //!< define logic 'true' value
18 #endif
19
20 #ifndef FALSE
21 #define FALSE (0x00)    //!< define logic 'false' value
22 #endif
23
24
25
26 #define TELEMETRY_SAVE_TIME_PERIOD   (10)                //!< save
telemetry every 10 seconds
27
28 #define MINUTES_TO_MILLISECONDS(minute) ((minute)*60*1000)
29
30 #define SECONDS_TO_TICKS(secs) (portTickType)((((secs) * 1000) /
portTICK_RATE_MS)
31 #define TICKS_TO_SECONDS(ticks) (unsigned int)((((ticks) / 1000) *
portTICK_RATE_MS)
32 #define MINUTES_TO_TICKS(minutes)  (portTickType)
(SECONDS_TO_TICKS(60*(minutes)))
33 #define MINUTES_TO_SECONDS(minutes) (portTickType)(60*(minutes))
34
35
36 #define MAX_SEMAPHORE_WAIT (SECONDS_TO_TICKS(10))      //!<
wait for 10 seconds to acquire semaphore
37
38 #define DELAYED_CMD_TIME_BOUND   60                    //!< if
requested execution time has passed by this measure the command will not be
executed and deleted from buffer
39

```

```

38  #define UNIX_DATE_JAN_D1_Y2000 {\
39  .seconds = 0,      \
40  .minutes = 0,      \
41  .hours = 0,        \
42  .day = 1,          \
43  .date = 1,         \
44  .month = 1,        \
45  .year = 0,         \
46  .secondsOfYear = 0}           ///< Time structure of the date 1/1/2000 at 00:00

47
48  #define UNIX_SECS_FROM_Y1970_TO_Y2000  946681200           ///<
number of seconds from 1 JAN 1970 to 1 JAN 2000

49
50  typedef unsigned long int time_unix;    ///< in seconds
51  typedef unsigned char byte;
52
53  typedef short voltage_t;    ///< in mV
54  typedef short current_t;    ///< in mA
55  typedef short power_t;      ///< in mW
56  typedef short temp_t;        ///< in Celsius
57  typedef unsigned char channel_t;  ///< each bit represents a specific system
state. 1 = ON; 0 = OFF
58
59
60  #endif /* GLOBALSTANDARDS_H_ */
61

```

```

1 /*
2 *
3 * @file   InitSystem.h
4 * @brief   All crucial initialization functions in one place
5 * @note    Order of function calls is important. Read system documents for further
analysis
6 */
7
8 #ifndef INITSYSTEM_H_
9 #define INITSYSTEM_H_
10
11 #include <hal/Storage/FRAM.h>
12 #include <hal/Drivers/I2C.h>
13 #include <hal/Drivers/SPI.h>
14 #include <hal/Timing/Time.h>
15 #include <hal/supervisor.h>
16
17 #include <hcc/api_fat.h>
18
19 #include "SubSystemModules/Communication/TRXVU.h"
20
21 #include "SubSystemModules/PowerManagment/EPS.h"
22
23 #include "SubSystemModules/Maintenance/Maintenance.h"
24
25 #include "TLM_management.h"
26
27 #include "GlobalStandards.h"
28
29 #define MIN_2_WAIT_BEFORE_DEPLOY 40 // how many minutes to wait
before we open the Ants
30 #define RESTART_TIME 3 // how much time does it take to restart the SAT
31
32 /*!
33 * @brief   Starts the FRAM using drivers, and checks for errors.
34 * @see FRAM.h
35 */
36 int StartFRAM();
37
38 /*!
39 * @brief   writes the default flight parameters to the corresponding FRAM
addresses
40 * @see FRAM_FlightParameters.h
41 */

```



```

40     int WriteDefaultValuesToFRAM();
41
42     /*!
43  * @brief   Starts the I2C using drivers, and checks for errors.
44  * @see    I2C.h
45  */
46     int StartI2C();
47
48     /*!
49  * @brief   Starts the SPI using drivers, and checks for errors
50  * @see    SPI.h
51  */
52     int StartSPI();
53
54
55     /*!
56  * @brief   Starts the Time module using drivers, and checks for errors.
57  * @see    Time.h
58  */
59     int StartTIME();
60
61     /*
62  * update sat time according to most update sat time
63  * @return type=int; according to Time_setUnixEpoch errors
64  */
65     int UpdateTime();
66
67     /*
68  * @brief   Starts the supervisor module using drivers, and checks for errors.
69  * @see    supervisor.h
70  */
71     int InitSupervisor();
72
73     /*!
74  * @brief   executes all required initializations of systems, including sub-systems,
and checks for errors
75  * @return   0
76  */
77     int InitSubsystems();
78
79     /*
80  * arm ants according to which side (0/1)
81  * @param[in] name=side; type=uint8_t; side of the ants (0/1)
82  * @return type=int; 0 on success -1 on error
83  */
84     int AntArm(uint8_t side);

```

```
85
86  /*
87  * deploy ants according to which side (0/1)
88  * @param[in] name=side; type=uint8_t; side of the ants (0/1)
89  * @return type=int; 0 on success -1 on error
90  */
91  int AntDeployment(uint8_t side);
92
93  /*!
94  * @brief  deployment procedure
95  * @return  0 successful deployment
96  *          -1 failed to deploy
97  */
98  int FirstActivation();
99
100  #endif /* INITSYSTEM_H_ */
101
```

```

1 /*
2  * InitSystem.c
3  *
4  * Created on: 5 6 2024
5  *   Author: maayan
6  */
7
8  #include "InitSystem.h"
9  #include "utils.h"
10
11  #define FIRST_ACTIVE
12  #define FIRST_ACTIVE_DEPLOY
13
14  #define I2CBusSpeed_Hz 100000
15  #define I2CTransferTimeout 10
16  #define TIME_SYNCINTERVAL 60
17
18  /*!
19  * @brief   Starts the FRAM using drivers, and checks for errors.
20  * @see FRAM.h
21  */
22  int StartFRAM(){
23  return FRAM_start();
24  }
25
26  /*!
27  * @brief   Starts the I2C using drivers, and checks for errors.
28  * @see I2C.h
29  */
30  int StartI2C(){
31  return I2C_start(I2CBusSpeed_Hz, I2CTransferTimeout);
32  }
33
34  /*!
35  * @brief   Starts the SPI using drivers, and checks for errors
36  * @see SPI.h
37  */
38  int StartSPI(){
39  return SPI_start(bus1_spi, slave1_spi);
40  }
41
42  /*!
43  * @brief   Starts the Time module using drivers, and checks for errors.
44  * @see Time.h
45  */
46  int StartTIME(){

```

```

47  const Time time = UNIX_DATE_JAN_D1_Y2000;
48  return Time_start(&time, TIME_SYNCINTERVAL);
49 }
50
51  /*
52  * update sat time according to most update sat time
53  * @return type=int; according to Time_setUnixEpoch errors
54  */
55  int UpdateTime()
56 {
57  time_unix mostUpdated;
58  if(FRAM_read((unsigned char*)&mostUpdated,
MOST_UPDATED_SAT_TIME_ADDR, MOST_UPDATED_SAT_TIME_SIZE)) return -1;
59  return Time_setUnixEpoch((unsigned int)mostUpdated);
60 }
61
62  /*
63  * @brief Starts the supervisor module using drivers, and checks for errors.
64  * @see supervisor.h
65  */
66  int InitSupervisor()
67 {
68  uint8_t po = SUPERVISOR_SPI_INDEX;
69  int error = Supervisor_start(&po, 0);
70  return logError(error, "InitSupervisor - Supervisor_start");
71 }
72
73  /*!
74  * @brief writes the default flight parameters to the corresponding FRAM
addresses
75  * @see FRAM_FlightParameters.h
76  */
77  int WriteDefaultValuesToFRAM()
78 {
79  int zero = 0;
80  time_unix param = 0;
81  int error = 0;
82
83  //we change the time on all of those to 0.
84  if(FRAM_writeAndVerify((unsigned char*)&param,
TRANSPONDER_END_TIME_ADDR, TRANSPONDER_END_TIME_SIZE)) error += -1
;
85  if(FRAM_writeAndVerify((unsigned char*)&param, MUTE_END_TIME_ADDR,
MUTE_END_TIME_SIZE)) error += -1;
86  if(FRAM_writeAndVerify((unsigned char*)&param, IDLE_END_TIME_ADDR,
IDLE_END_TIME_SIZE)) error += -1;

```

```

87
88  int beacon_interval = DEFAULT_BEACON_INTERVAL_TIME;
89  if(FRAM_writeAndVerify((unsigned char*)&beacon_interval,
BEACON_INTERVAL_TIME_ADDR, BEACON_INTERVAL_TIME_SIZE)) error += -1;
90
91  int rssi = DEFAULT_RSSI_VALUE;
92  if(FRAM_writeAndVerify((unsigned char*)&rssi, TRANSPONDER_RSSI_ADDR,
TRANSPONDER_RSSI_SIZE)) error += -1;
93
94  float alpha = DEFAULT_ALPHA_VALUE;
95  if(FRAM_writeAndVerify((unsigned char*)&alpha,
EPS_ALPHA_FILTER_VALUE_ADDR, EPS_ALPHA_FILTER_VALUE_SIZE)) error +=
-1;
96
97  int timeDeploy = MIN_2_WAIT_BEFORE_DEPLOY*60;
98
99  if(FRAM_writeAndVerify((unsigned char*)&timeDeploy,
DEPLOYMENT_TIME_ADDR, DEPLOYMENT_TIME_SIZE)) error += -1;
100
101  time_unix mostUpdated = UNIX_SECS_FROM_Y1970_TO_Y2000;
102  if(FRAM_writeAndVerify((unsigned char*)&mostUpdated,
MOST_UPDATED_SAT_TIME_ADDR, MOST_UPDATED_SAT_TIME_SIZE)) error +=
-1;
103
104  Boolean false = FALSE;
105  if(FRAM_writeAndVerify((unsigned char*)&>false, RESET_CMD_FLAG_ADDR,
RESET_CMD_FLAG_SIZE)) error += -1;
106
107  int arrPeriod[7] = {DEFAULT_EPS_SAVE_TLM_TIME,
DEFAULT_TRXVU_SAVE_TLM_TIME, DEFAULT_ANT_SAVE_TLM_TIME,
DEFAULT_SOLAR_SAVE_TLM_TIME, DEFAULT_WOD_SAVE_TLM_TIME,
DEFAULT_RADFET_SAVE_TLM_TIME, DEFAULT_SEU_SEL_SAVE_TLM_TIME};
108  if(FRAM_writeAndVerify((unsigned char*)&arrPeriod,
TLM_SAVE_PERIOD_START_ADDR, sizeof(arrPeriod))) error += -1;
109
110  if(FRAM_writeAndVerify((unsigned char*)&zero, TRANS_ABORT_FLAG_ADDR,
TRANS_ABORT_FLAG_SIZE)) error += -1;
111
112  if(FRAM_writeAndVerify((unsigned char*)&zero,
NUM_OF_CHANGES_IN_MODE_ADDR, NUM_OF_CHANGES_IN_MODE_SIZE))
error += -1;
113
114  if(FRAM_writeAndVerify((unsigned char*)&zero, SD_CARD_USED_ADDR,
SD_CARD_USED_SIZE)) error += -1;
115
116  voltage_t defaultThershold[NUMBER_OF_THRESHOLD_VOLTAGES] =

```

```

DEFAULT_EPS_THRESHOLD_VOLTAGES;
117  EpsThreshVolt_t thresh;
118  for(int i = 0; i < NUMBER_OF_THRESHOLD_VOLTAGES; i++)
119      thresh.raw[i] = defaultThershold[i];
120  if(FRAM_writeAndVerify((unsigned char*)&thresh,
EPS_THRESH_VOLTAGES_ADDR, EPS_THRESH_VOLTAGES_SIZE)) error += -1;
121
122  int lastRadfet[5] = {0};
123  if(FRAM_writeAndVerify((unsigned char*)&lastRadfet,
LAST_RADFET_READ_START, sizeof(lastRadfet))) error += -1;
124
125  if(FRAM_writeAndVerify((unsigned char*)&mostUpdated,
LAST_COMM_TIME_ADDR, LAST_COMM_TIME_SIZE)) error += -1;
126
127  unsigned int defaultTime = DEFAULT_NO_COMM_WDT_KICK_TIME;
128  if(FRAM_writeAndVerify((unsigned char*)&defaultTime,
NO_COMM_WDT_KICK_TIME_ADDR, NO_COMM_WDT_KICK_TIME_SIZE)) error
+= -1;
129
130  Boolean true = TRUE;
131  if(FRAM_writeAndVerify((unsigned char*)&>true, TRY_TO_DEPLOY_ADDR,
TRY_TO_DEPLOY_SIZE)) error += -1;
132
133  if(FRAM_writeAndVerify((unsigned char*)&mostUpdated,
LAST_TRY_TO_DEPLOY_TIME_ADDR, LAST_TRY_TO_DEPLOY_TIME_SIZE)) error
+= -1;
134
135  return error;
136 }
137
138  /*
139  * arm ants according to which side (0/1)
140  * @param[in] name=side; type=uint8_t; side of the ants (0/1)
141  * @return type=int; 0 on success -1 on error
142  */
143  int AntArm(uint8_t side)
144 {
145     #ifdef FIRST_ACTIVE_DEPLOY
146     int rv = logError(isis_ants__arm(side), "AntArm - isis_ants__arm");
147     if(rv)
148     {
149         printf("Ants not armed\r\n");
150         return -1;
151     }
152     printf("Ant %d arm\r\n", side);
153     #else

```

```

154     printf("Armed (test)\r\n");
155     #endif
156     return 0;
157 }
158
159     /*
160  * deploy ants according to which side (0/1)
161  * @param[in] name=side; type=uint8_t; side of the ants (0/1)
162  * @return type=int; 0 on success -1 on error
163  */
164     int AntDeployment(uint8_t side)
165 {
166
167     #ifdef FIRST_ACTIVE_DEPLOY
168     int rv = logError(isis_ants__start_auto_deploy(side, 10), "AntDeployment -
isis_ants__start_auto_deploy");
169     if(rv)
170     {
171         printf("Ants not deployed\r\n");
172         return -1;
173     }
174     printf("Ant %d deploy\r\n", side);
175     #else
176     printf("Deploy (test)\r\n");
177     #endif
178
179     return 0;
180 }
181
182     /*!
183  * @brief   deployment procedure
184  * @return   0 successful deployment
185  *          -1 failed to deploy
186  */
187     int FirstActivation()
188 {
189     Boolean false = FALSE;
190     int firstActiveFlag;
191     FRAM_read((unsigned char*)&firstActiveFlag,
FIRST_ACTIVATION_FLAG_ADDR, FIRST_ACTIVATION_FLAG_SIZE);
192     if(!firstActiveFlag)
193         return 0;
194     int error = 0;
195     #ifdef FIRST_ACTIVE
196     int max;
197     int time = 0;

```

```

198  FRAM_read((unsigned char*)&max, DEPLOYMENT_TIME_ADDR,
DEPLOYMENT_TIME_SIZE);
199  do
200  {
201      FRAM_read((unsigned char*)&time, SECONDS_SINCE_DEPLOY_ADDR,
SECONDS_SINCE_DEPLOY_SIZE);
202      vTaskDelay(5000 / portTICK_RATE_MS); // wait 5 seconds.
203      time += 5;
204      TelemetryCollectorLogic();
205      if(logError(FRAM_writeAndVerify((unsigned char*)&time,
SECONDS_SINCE_DEPLOY_ADDR, SECONDS_SINCE_DEPLOY_SIZE),
"FirstActivation - FRAM_writeAndVerify")) error = -1;
206      printf("%d\r\n", time);
207  }
208  while(max > time);
209  for(int i = 0; i < 2; i++)
210  {
211      AntArm(0);
212      AntDeployment(0);
213      AntArm(1);
214      AntDeployment(1);
215  }
216
217  time_unix timeUnix;
218  logError(Time_getUnixEpoch((unsigned int*)&timeUnix), "FirstActivation -
Time_getUnixEpoch");
219  if(logError(FRAM_writeAndVerify((unsigned char*)&timeUnix,
LAST_TRY_TO_DEPLOY_TIME_ADDR, LAST_TRY_TO_DEPLOY_TIME_SIZE),
"FirstActivation - FRAM_writeAndVerify")) error = -1;
220  #endif
221  if(logError(FRAM_writeAndVerify((unsigned char*)&>false,
FIRST_ACTIVATION_FLAG_ADDR, FIRST_ACTIVATION_FLAG_SIZE), "FirstActivation
- FRAM_writeAndVerify - flag = 0")) error = -1;
222  SendAckPacket(ACK_FINISH_FIRST_ACTIVE, NULL, NULL, 0);
223  return error;
224 }
225
226  /*
227  * This function reads from FRAM if the payload needs to init or if we have the
payload kill flag on.
228  * It also checks we didn't have the reset under a minute flag on. If so, it change the
payload flag kill to true and does not activate the payload.
229  */
230  void payloadKillOrInit()
231 {
232  Boolean checkPayloadFlag;

```



```

233  logError(FRAM_read((unsigned char*)&checkPayloadFlag,
PAYLOAD_IS_DEAD_ADDR, PAYLOAD_IS_DEAD_SIZE), "payloadKillOrInit -
FRAM_read");
234  if(checkPayloadFlag) return;
235  Boolean true = TRUE;
236  Boolean flagReset;
237  logError(FRAM_read((unsigned char*)&flagReset,
HAD_RESET_IN_A_MINUTE_ADDR, HAD_RESET_IN_A_MINUTE_SIZE),
"payloadKillOrInit - FRAM_read");
238  if(!flagReset)
239  {
240      logError(FRAM_writeAndVerify((unsigned char*)&true,
HAD_RESET_IN_A_MINUTE_ADDR, HAD_RESET_IN_A_MINUTE_SIZE),
"payloadKillOrInit - FRAM_writeAndVerify");
241      payloadInit();
242  }
243  else
244      logError(FRAM_writeAndVerify((unsigned char*)&true,
PAYLOAD_IS_DEAD_ADDR, PAYLOAD_IS_DEAD_SIZE), "payloadKillOrInit -
FRAM_writeAndVerify");
245
246 }
247
248  /*!
249  * @brief  executes all required initializations of systems, including sub-systems,
and checks for errors
250  * @return  0
251  */
252  int InitSubsystems()
253 {
254     StartI2C();
255
256     StartSPI();
257
258     StartFRAM();
259
260     StartTIME();
261
262     // Reads if the first active flag is on, if so write defaults to FRAM. Else update the
time.
263     int firstActiveFlag = 0;
264     FRAM_read((unsigned char*)&firstActiveFlag,
FIRST_ACTIVATION_FLAG_ADDR, FIRST_ACTIVATION_FLAG_SIZE);
265     if(firstActiveFlag)
266         WriteDefaultValuesToFRAM();
267     else

```

```
268     UpdateTime();
269
270     InitializeFS();
271
272     //Also with the first active flag. It's here just because we first needed to init the
FS.
273     if(firstActiveFlag) Delete_allTMFilesFromSD();
274
275     InitSavePeriodTimes();
276
277     InitSupervisor();
278
279     EPS_And_SP_Init();
280
281     InitTrxvuAndAnts();
282
283     WakeupFromResetCMD();
284
285     payloadKillOrInit();
286
287     FirstActivation();
288
289     logError(SAT_RESET, "InitSubsystems - reset");
290     printf("Did init\r\n");
291     return 0;
292 }
293
```

```

1 /*
2  * TM_managment.h
3  *
4  * Created on: Apr 8, 2019
5  * Author: Hoopoe3n
6  */
7
8 #ifndef TM_MANAGMENT_H_
9 #define TM_MANAGMENT_H_
10
11 #include <hal/Boolean.h>
12 #include <GlobalStandards.h>
13 #include "SubSystemModules/Housekeeping/TelemetryFiles.h"
14
15 #include "SubSystemModules/Housekeeping/TelemetryCollector.h"
16
17 #include <hal/Timing/Time.h>
18 #include <time.h>
19 #include "SubSystemModules/Communication/SatCommandHandler.h"
20
21
22 #define MAX_FILE_NAME_SIZE 11
23 #define NUM_ELEMENTS_READ_AT_ONCE 1000
24
25 #define FS_FILE_ENDING "TLM"
26 #define FS_FILE_ENDING_SIZE 3
27
28 #ifndef FSFRAM
29 #define FSFRAM 0x20000
30 #endif
31
32 typedef enum {
33     FS_SUCCESS,
34     FS_DUPLICATED,
35     FS_LOCKED,
36     FS_TOO_LONG_NAME,
37     FS_BUFFER_OVERFLOW,
38     FS_NOT_EXIST,
39     FS_ALLOCATION_ERROR,
40     FS_FRAM_FAIL,
41     FS_FAT_API_FAIL,
42     FS_FAIL
43 } FileSystemResult;

```

```

43     /*
44  * write to a file and create a file if not exist the file we need to write to
45  * @param[in] name=data; type=void*; which data we want to save (can be any
type).
46  * @param[in] name=days2Add; type=int; get size of the struct we want to keep in
bytes
47  * @return type=int;    -2 on get sat time problem (as seconds)
48  *                      -3 on get sat time as Time struct
49  *                      -1 on problem with opening the file
50  */
51     int Write2File(void* data, tlm_type_t tlmType);
52
53     /*
54  * Delete files from specific start time and type
55  * @param[in] name=tlmType; type=tlm_type_t; which type of telemetry we want to
delete.
56  * @param[in] name=date; type=Time; which time we want to start delete from
57  * @param[in] name=numOfDays; type=int; how many days we want to delete from
the date start
58  * @return type=int; how many days were successfully deleted
59  */
60     int DeleteTLMFiles(tlm_type_t tlmType, Time date, int numOfDays);
61
62     /*
63  * Delete a file from specific time and type
64  * @param[in] name=tlmType; type=tlm_type_t; which subsystem we are want to
delete a file from
65  * @param[in] name=date; type=Time; which time we want to delete
66  * @param[in] name=days2Add; type=int; for the counter of days to add a number of
days for the last struct
67  * @return type=int; return error according to "https://drive.google.com/file/
d/0B7WEDvdtTWV6NXImOWpPV20yelk/view?usp=sharing&resourcekey=0-
ltj73kO4lv1PaHq2sZopEw" page 142
68  */
69     int DeleteTLMFile(tlm_type_t tlmType, Time date, int days2Add);
70
71     /*
72  * Get important info about the file (for save, read and delete).
73  * @param[in] name=tlmType; type=tlm_type_t; which type of telemetry we want.
74  * @param[out] name=endFileName; type=char*; get the ".something" to the
specific type
75  * @param[out] name=structSize; type=int*; get size of the struct we want to keep in
bytes
76  */
77     void GetTlmTypeInfo(tlm_type_t tlmType, char* endFileName, int* structSize);
78

```

```

79     /*
80  * Get the final file name.
81  * @param[in] name=date; type=Time; which time we want to save/delete/read the
data.
82  * @param[out] name=file_name; type=char*; final file name will be written to here
83  * @param[in] name=endFileName; type=char*; the ".something" to the specific
type we want to find the name
84  * @param[in] name=days2Add; type=int; get size of the struct we want to keep in
bytes
85  */
86     void CalculateFileName(Time date, char* file_name, char* endFileName, int
days2Add);
87
88     /*
89  * Read telemetry file and send it to ground station over RF
90  * @param[in] name=tlmType; type=tlm_type_t; type of data we save.
91  * @param[in] name=startDate; type=Time; time to start from the reading/the
dump.
92  * @param[in] name=days2Add; type=int; day to add for startDate to read the next
days from there.
93  * @param[in] name=resolution; type=int; resolution is how many times we need
to not send until we send
94  * @return type=int; num of elements sent
95  */
96     int ReadTLMFile(tlm_type_t tlmType, Time date, int days2Add, int cmd_id, int
resolution);
97
98     /*
99  * call function read according to name of dates and start and resolution
100  * @param[in] name=tlmType; type=tlm_type_t; type of data we save.
101  * @param[in] name=startDate; type=Time; time to start from the reading.
102  * @param[in] name=numOfDays; type=int; how many days we want to read =>
how many files form the same type from start time
103  * @param[in] name=resolution; type=int; resolution in seconds of the tlm of the
same day we want to read
104  * @return type=int; num of elements read
105  */
106     int ReadTLMFiles(tlm_type_t tlmType, Time startDate, int numOfDays, int
cmd_id, int resolution);
107
108     /*
109  * Get save tlm period time according to type to save.
110  * @param[in] name=type; type=tlm_type_t; have which tlm we try to save/send
111  * @return type=int; period time for type and 0 for log and unknown type
112  */
113     int GetPeriodTimeAccordingToTlmType(tlm_type_t type);

```

```

114
115
116     /*
117  * Formating the SD we are on and then call to init
118  * return type=int; return -2 if we are having an error in f_getdrive and -1 on
119  * */
120     int Delete_allTMFilesFromSD();
121
122     /*!
123  * Initializes the file system.
124  * @note call once for boot and after DeInitializeFS.
125  * @return FS_FAIL if Initializing the FS failed,
126  * FS_ALLOCATION_ERROR on malloc error,
127  * FS_SUCCSESS on success.
128  */
129 FileSystemResult InitializeFS();
130
131     /*!
132  * DeInitializes the file system.
133  */
134     void DeInitializeFS();
135
136     #endif /* TM_MANAGMENT_H_ */
137
138

```

```

1 /*
2  * TLM_management.c
3  *
4  * Created on: 5 6 2024
5  * Author: maayan
6  */
7
8  #include "TLM_management.h"
9
10  #include <hcc/api_fat.h>
11  #include <hcc/api_mdriver_atmel_mcpdc.h>
12  #include <hcc/api_mdriver.h>
13  #include <hcc/api_hcc_mem.h>
14  #include "SubSystemModules/Housekeeping/TelemetryCollector.h"
15
16  #include "SubSystemModules/Communication/TRXVU.h"
17
18  #include "SubSystemModules/Payload/payload_drivers.h"
19
20  #include <string.h>
21
22  #include "utils.h"
23
24  static char buffer[MAX_COMMAND_DATA_LENGTH *
25  NUM_ELEMENTS_READ_AT_ONCE]; // buffer for data coming from SD (time+size of
26  data struct)
27
28  Boolean stopDump = FALSE;
29
30
31  FileSystemResult InitializeFS(){
32  uint8_t sd;
33  if(FRAM_read((unsigned char*)&sd, SD_CARD_USED_ADDR,
34  SD_CARD_USED_SIZE)) return FS_FAIL;
35  int flag = 0;
36  // in FS init we don't want to use a log file !
37
38  // Initialize the memory for the FS
39  int err = hcc_mem_init();
40  if (err != E_NO_SS_ERR){
41  printf("FS - hcc_mem_init Error - %d ", err);
42  flag = 1;
43  }
44  // Initialize the FS
45  err = fs_init();
46  if (err != E_NO_SS_ERR){

```

```

40     printf("FS - fs_init Error - %d ", err);
41     flag = 1;
42 }
43
44 fs_start();
45
46 // Tell the OS (freeRTOS) about our FS
47 err = f_enterFS();
48 if (err != E_NO_SS_ERR){
49     printf("FS - f_enterFS Error - %d ", err);
50     flag = 1;
51 }
52
53 // Initialize the volume of SD card 0 (A)
54 err = f_initvolume( 0, atmel_mcipdc_initfunc, sd );
55 if(flag)
56     return FS_FAIL;
57
58 return FS_SUCCESS;
59 }
60
61 /*
62 * Get the final file name.
63 * @param[in] name=date; type=Time; which time we want to save/delete/read the
data.
64 * @param[out] name=file_name; type=char*; final file name will be written to here
65 * @param[in] name=endFileName; type=char*; the ".something" to the specific
type we want to find the name
66 * @param[in] name=days2Add; type=int; get size of the struct we want to keep in
bytes
67 */
68 void CalculateFileName(Time date, char* file_name, char* endFileName, int
days2Add)
69 {
70     /* initialize */
71     struct tm t = { .tm_year = date.year + 100, .tm_mon = date.month - 1, .tm_mday =
date.date }; //need to read how mktime work and get and Time to understand
72     /* modify */
73     t.tm_mday += days2Add;
74     mktime(&t);
75     char file_buff[7];
76
77
78     strftime(file_buff, sizeof file_buff, "%y%0m%0d", &t);
79

```



```

80  sprintf(file_name, "%s.%s", file_buff, endFileName);
81 }
82
83  /*
84  * Get important info about the file (for save, read and delete).
85  * @param[in] name=tlmType; type=tlm_type_t; which type of telemetry we want.
86  * @param[out] name=endFileName; type=char*; get the ".something" to the
specific type
87  * @param[out] name=structSize; type=int*; get size of the struct we want to keep in
bytes
88  */
89  void GetTlmTypeInfo(tlm_type_t tlmType, char* endFileName, int* structSize)

90 {
91  switch(tlmType)
92  {
93      case tlm_tx:
94      {
95          memcpy(endFileName, END_FILE_NAME_TX, sizeof
(END_FILE_NAME_TX));
96          *structSize = sizeof(isis_vu_e__get_tx_telemetry__from_t);
97          break;
98      }
99      case tlm_rx:
100     {
101         memcpy(endFileName, END_FILE_NAME_RX, sizeof
(END_FILE_NAME_RX));
102         *structSize = sizeof(isis_vu_e__get_rx_telemetry__from_t);
103         break;
104     }
105     case tlm_ants0:
106     {
107         memcpy(endFileName, END_FILE_NAME_ANTENNA_SIDE_ZERO, sizeof
(END_FILE_NAME_ANTENNA_SIDE_ZERO));
108         *structSize = sizeof(isis_ants__get_all_telemetry__from_t);
109         break;
110     }
111     case tlm_ants1:
112     {
113         memcpy(endFileName, END_FILE_NAME_ANTENNA_SIDE_ONE, sizeof
(END_FILE_NAME_ANTENNA_SIDE_ONE));
114         *structSize = sizeof(isis_ants__get_all_telemetry__from_t);
115         break;
116     }
117     case tlm_eps:
118     {

```

```

119     memcpy(endFileName,END_FILENAME_EPS_TLM,sizeof
(END_FILENAME_EPS_TLM));
120     *structSize = sizeof(isismepsv2_ivid7_piu__gethousekeeping__from_t);
121     break;
122 }
123 case tlm_wod:
124 {
125     memcpy(endFileName,END_FILENAME_WOD_TLM,sizeof
(END_FILENAME_WOD_TLM));
126     *structSize = sizeof(WOD_Telemetry_t);
127     break;
128 }
129 case tlm_solar:
130 {
131     memcpy(endFileName,END_FILENAME_SOLAR_PANELS_TLM,sizeof
(END_FILENAME_SOLAR_PANELS_TLM));
132     *structSize = sizeof(solar_tlm_t);
133     break;
134 }
135 case tlm_log:
136 {
137     memcpy(endFileName,END_FILENAME_LOGS,sizeof
(END_FILENAME_LOGS));
138     *structSize = sizeof(logData_t);
139     break;
140 }
141 case tlm_radfet:
142 {
143     memcpy(endFileName,END_FILENAME_RADFET_TLM,sizeof
(END_FILENAME_RADFET_TLM));
144     *structSize = sizeof(PayloadEnvironmentData);
145     break;
146 }
147 case tlm_sel:
148 {
149     memcpy(endFileName,END_FILENAME_SEL_TLM,sizeof
(END_FILENAME_SEL_TLM));
150     *structSize = sizeof(payloadSEL_data);
151     break;
152 }
153 case tlm_seu:
154 {
155     memcpy(endFileName,END_FILENAME_SEU_TLM,sizeof
(END_FILENAME_SEU_TLM));
156     *structSize = sizeof(int);
157     break;

```

```

158     }
159     default:
160         break;
161 }
162 }
163
164 /*
165 * write to a file and create a file if not exist the file we need to write to
166 * @param[in] name=data; type=void*; which data we want to save (can be any
167 * @param[in] name=days2Add; type=int; get size of the struct we want to keep in
168 * @return type=int;    -2 on get sat time problem (as seconds)
169 *                      -3 on get sat time as Time struct
170 *                      -1 on problem with opening the file
171 */
172 int Write2File(void* data, tlm_type_t tlmType)
173 {
174     time_unix timeNow;
175     if(Time_getUnixEpoch((unsigned int*)&timeNow)) return -2;
176
177     Time currDate;
178     if(Time_get(&currDate)) return -3;
179     char endFile[3];
180     int structSize;
181     GetTlmTypeInfo(tlmType, endFile, &structSize);
182
183     char fileName[MAX_FILE_NAME_SIZE] = {0};
184     CalculateFileName(currDate, fileName, endFile, 0);
185     F_FILE *fp = f_open(fileName, "a");
186     if(!fp)
187     {
188         printf("Hi, we have an error in opening the file");
189         return -1;
190     }
191     f_write(&timeNow, sizeof(timeNow), 1, fp);
192     f_write(data, structSize, 1, fp);
193     /* close the file*/
194     f_flush(fp);
195     f_close (fp);
196     return 0;
197 }
198
199 /*
200 * delete files from specific start time and type
201 * @param[in] name=tlmType; type=tlm_type_t; which type of telemetry we want to

```

```

delete.
202 * @param[in] name=date; type=Time; which time we want to start delete from
203 * @param[in] name=numOfDays; type=int; how many days we want to delete from
the date start
204 * @return type=int; how many days were successfully deleted
205 */
206 int DeleteTLMFiles(tlm_type_t tlmType, Time date, int numOfDays){
207     int deletedFiles = 0;
208     for(int i = 0; i <= numOfDays; i++){
209         if (DeleteTLMFile(tlmType, date, i) == F_NO_ERROR){
210             deletedFiles++;
211         }
212     }
213     return deletedFiles;
214 }
215
216 /*
217 * delete a file from specific time and type
218 * @param[in] name=tlmType; type=tlm_type_t; which subsystem we are want to
delete a file from
219 * @param[in] name=date; type=Time; which time we want to delete
220 * @param[in] name=days2Add; type=int; for the counter of days to add a number
of days for the last struct
221 * @return type=int; return error according to "https://drive.google.com/file/
d/0B7WEDvdtTWV6NXImOWpPV20yelk/view?usp=sharing&resourcekey=0-
ltj73kO4lv1PaHq2sZopEw" page 142
222 */
223 int DeleteTLMFile(tlm_type_t tlmType, Time date, int days2Add){
224
225     char endFileName[3] = {0};
226     int size;
227     GetTlmTypeInfo(tlmType, endFileName, &size);
228     char fileName[MAX_FILE_NAME_SIZE] = {0};
229     CalculateFileName(date, fileName, endFileName, days2Add);
230
231     return f_delete(fileName);
232 }
233
234 /*
235 * Formating the SD we are on and then call to init
236 * return type=int; return -2 if we are having an error in f_getdrive and -1 on
formating error
237 */
238 int Delete_allTMFilesFromSD()
239 {
240     int error = 0;

```

```

241  int sd = f_getdrive(); //get in which sd we are using now

242  if(sd == 0 || sd == 1)
243  {
244      printf("SD: %d\r\n", sd);
245      if(logError(f_format(sd, F_FAT32_MEDIA), "Delete_allTMFilesFromSD -
f_format")) error = -1;
246  }
247  else
248      if(logError(sd, "Delete_allTMFilesFromSD - f_getdrive")) error = -2;
249  return error;
250 }
251
252  /*
253  * Get save tlm period time according to type to save.
254  * @param[in] name=type; type=tlm_type_t; have which tlm we try to save/send
255  * @return type=int; period time for type and 0 for log and unknown type
256  */
257  int GetPeriodTimeAccordingToTlmType(tlm_type_t type)
258  {
259      switch(type)
260      {
261          case tlm_eps:
262              return periods.fields.eps;
263          case tlm_tx:
264              return periods.fields.trxvu;
265          case tlm_rx:
266              return periods.fields.trxvu;
267          case tlm_ants1:
268              return periods.fields.ants;
269          case tlm_ants0:
270              return periods.fields.ants;
271          case tlm_radfet:
272              return periods.fields.radfet;
273          case tlm_wod:
274              return periods.fields.wod;
275          case tlm_sel:
276              return periods.fields.seu_sel;
277          case tlm_seu:
278              return periods.fields.seu_sel;
279          case tlm_solar:
280              return periods.fields.solar_panels;
281          default:
282              return 0;
283      }
284 }

```

```

285
286  /*
287  * Read telemetry file and send it to ground station over RF
288  * @param[in] name=tlmType; type=tlm_type_t; type of data we save.
289  * @param[in] name=startDate; type=Time; time to start from the reading/the
dump.
290  * @param[in] name=days2Add; type=int; day to add for startDate to read the
next days from there.
291  * @param[in] name=resolution; type=int; resolution is how many times we need
to not send until we send
292  * @return type=int; num of elements sent
293  */
294  int ReadTLMFile(tlm_type_t tlmType, Time date, int days2Add, int cmd_id , int
resolution)
295  {
296      unsigned int offset = 0;
297
298      int size=0;
299      char file_name[MAX_FILE_NAME_SIZE] = {0};
300      char endFileName[3] = {0};
301
302      GetTlmTypeInfo(tlmType, endFileName, &size);
303      CalculateFileName(date, file_name, endFileName , days2Add);
304      F_FILE *fp = f_open(file_name, "r");
305
306      if (!fp) return -1; //unable to open file
307
308      char element[(sizeof(int)+size)]; // buffer for a single element that we will send
309
310      int numElementsSent=0;
311      time_unix currTime = 0;
312      int count = resolution;
313      while(TRUE)
314      {
315          int readElements = f_read(buffer , sizeof(int)+size ,
NUM_ELEMENTS_READ_AT_ONCE, fp );
316          offset = 0;
317          if(!readElements) break;
318          for (;readElements > 0; readElements--){
319              memcpy( &element, buffer + offset, sizeof(int) + size); // copy time+data
320
321              offset += size + sizeof(int);
322
323              memcpy(&currTime, &element, sizeof(int));
324              /*      PrintTLM(&element,tlmType); //check only for some of the TRXVU*/

```

```

324     if(count != resolution)
325     {
326         count++;
327         continue;
328     }
329     sat_packet_t dump_tlm = { 0 };
330
331     AssembleCommand((unsigned char*)element, sizeof(int)+size, dump_type,
332 tlmType, cmd_id, &dump_tlm);
333     logError(TransmitSplPacket(&dump_tlm, NULL), "ReadTLMFile -
334 TransmitSplPacket");
335     numElementsSent++;
336     if(CheckDumpAbort()){
337         stopDump = TRUE;
338         break;
339     }
340     count = 0;
341 }
342 if(stopDump){
343     break;
344 }
345 }
346 /* close the file*/
347 f_close(fp);
348 return numElementsSent;
349 }
350
351 /*
352 * call function read according to name of dates and start and resolution
353 * @param[in] name=tlmType; type=tlm_type_t; type of data we save.
354 * @param[in] name=startDate; type=Time; time to start from the reading.
355 * @param[in] name=numOfDays; type=int; how many days we want to read =>
356 * how many files form the same type from start time
357 * @param[in] name=resolution; type=int; resolution in seconds of the tlm of the
358 * same day we want to read
359 * @return type=int; num of elements read
360 */
361 int ReadTLMFiles(tlm_type_t tlmType, Time startDate, int numOfDays, int
362 cmd_id, int resolution)
363 {
364     int periodSaveTime = GetPeriodTimeAccordingToTlmType(tlmType);
365     int newResolution = 0;
366     if(periodSaveTime != 0)

```

```

364     newResolution = resolution / periodSaveTime;
365     stopDump = FALSE;
366     int totalReads=0;
367     int elementsRead=0;
368     for(int i = 0; i <= numOfDays; i++){
369         elementsRead = ReadTLMFile(tlmType, startDate, i,cmd_id, newResolution);
370         totalReads+= (elementsRead>0) ? elementsRead : 0;
371         if(stopDump) break;
372         vTaskDelay(100);
373     }
374
375     return totalReads;
376 }
377
378     void DelInitializeFS()
379 {
380     uint8_t sd;
381     if(FRAM_read((unsigned char*)&sd, SD_CARD_USED_ADDR,
SD_CARD_USED_SIZE)) return;
382     f_delvolume(sd); /* delete the volid */
383
384     f_releaseFS();
385
386     fs_delete(); /* delete the filesystem */
387
388     hcc_mem_delete(); /* free the memory used by the filesystem */
389
390
391
392 }
393

```


**SubSystemModules/
PowerManagment**

```

1 #ifndef EPSOPERATIONMODES_H_
2 #define EPSOPERATIONMODES_H_
3
4
5 #include "GlobalStandards.h"
6 #include "SubSystemModules/Payload/payload_drivers.h"
7
7 #include <stdio.h>
8
9
10 #define CHANNELS_OFF 0x00    //!< channel state when all systems are off
11
11 #define CHNNELS_ON    0xFF    //!< channel
12 #define SYSTEM0      0x01    //!< channel state when 'SYSTEM0' is on
13
13 #define SYSTEM1      0x02    //!< channel state when 'SYSTEM1' is on
14
14 #define SYSTEM2      0x04    //!< channel state when 'SYSTEM2' is on
15
15 #define SYSTEM3      0x08    //!< channel state when 'SYSTEM3' is on
16
16 #define SYSTEM4      0x10    //!< channel state when 'SYSTEM4' is on
17
17 #define SYSTEM5      0x20    //!< channel state when 'SYSTEM5' is on
18
18 #define SYSTEM6      0x40    //!< channel state when 'SYSTEM6' is on
19
19 #define SYSTEM7      0x80    //!< channel state when 'SYSTEM7' is on
20
21 typedef enum{
22     OperationalMode,
23     CruiseMode,
24     PowerSafeMode
25 }EpsState_t;
26
27 /*!
28 * @brief Executes the necessary procedure in order to initiate the system into
Operational mode
29 * @return  0
30 */
31 int EnterOperationalMode();
32
33 /*!
34 * @brief Check if we need to be in Operational or Cruise
35 * @return  0

```

```

36 */
37 int BetweenOperationalToCruise();
38
39 /*!
40 * @brief Executes the necessary procedure in order to initiate the system into
41 * Cruise mode
42 * @return 0
43 */
44 int EnterCruiseMode();
45
46 /*!
47 * @brief Check if we need to be in Cruise or Power safe mode
48 * @return 0
49 */
50 int BetweenCruiseToPowerSafeMode();
51
52 /*!
53 * @brief Executes the necessary procedure in order to initiate the system into
54 * Power Safe mode
55 * @return 0
56 */
57 int EnterPowerSafeMode();
58
59 /*!
60 * returns the current system state according to the EpsState_t enumeration
61 * @return system state according to EpsState_t
62 */
63 EpsState_t GetSystemState();
64
65 /*
66 * Get if the Tx flag is on or off for us to put in the CheckAllowed in the TRXVU.c
67 * code.
68 */
69 Boolean GetTxFlag();
70
71 /*
72 * Get if the Payload flag is on or off.
73 */
74 Boolean GetPayloadFlag();
75
76 #endif /* EPSOPERATIONMODES_H_ */

```

```

1 /*
2  * EPSOperationModes.c
3  *
4  * Created on: 21 Nov 2024
5  * Author: maayan
6  */
7
8  #include "EPSOperationModes.h"
9  #include "utils.h"
10 #include "hal/Drivers/I2C.h"
11 #include "SubSystemModules/Communication/TRXVU.h"
12
13 Boolean txOff = FALSE;
14 Boolean payloadOff = FALSE;
15 EpsState_t satState = OperationalMode;
16
17  /*!
18  * @brief Executes the necessary procedure in order to initiate the system into
19  Operational mode
20  * @return 0
21  */
22 int EnterOperationalMode()
23 {
24     printf("entered Operational\r\n");
25     if(satState == OperationalMode) return 0;
26     logError(payloadTurnOn(), "EnterOperationalMode - payloadTurnOn");
27     int countChange = 0;
28     logError(FRAM_read((unsigned char*)&countChange,
29 NUM_OF_CHANGES_IN_MODE_ADDR, NUM_OF_CHANGES_IN_MODE_SIZE),
30 "EnterOperationalMode - FRAM_read");
31     countChange += 1;
32     logError(FRAM_writeAndVerify((unsigned char*)&countChange,
33 NUM_OF_CHANGES_IN_MODE_ADDR, NUM_OF_CHANGES_IN_MODE_SIZE),
34 "EnterOperationalMode - FRAM_writeAndVerify");
35     txOff = FALSE;
36     payloadOff = FALSE;
37     satState = OperationalMode;
38     return 0;
39 }
40
41  /*!
42  * @brief Check if we need to be in Operational or Cruise
43  * @return 0
44  */

```

```

41     int BetweenOperationalToCruise()
42 {
43     if(satState == OperationalMode) return EnterOperationalMode();
44     return EnterCruiseMode();
45 }
46
47     /*!
48 * @brief Executes the necessary procedure in order to initiate the system into
Cruise mode
49 * @return 0
50 */
51     int EnterCruiseMode()
52 {
53     printf("entered Cruise\r\n");
54     if(satState == CruiseMode) return 0;
55     logError(payloadTurnOff(), "EnterCruiseMode - payloadTurnOff");
56     vTaskDelay(100);
57     setNewBeaconIntervalToPeriod();
58     txOff = FALSE;
59     payloadOff = TRUE;
60     satState = CruiseMode;
61     return 0;
62 }
63
64     /*!
65 * @brief Check if we need to be in Cruise or Power safe mode
66 * @return 0
67 */
68     int BetweenCruiseToPowerSafeMode()
69 {
70     if(satState == CruiseMode) return EnterCruiseMode();
71     return EnterPowerSafeMode();
72 }
73
74     /*!
75 * @brief Executes the necessary procedure in order to initiate the system into
Power Safe mode
76 * @return 0
77 */
78     int EnterPowerSafeMode()
79 {
80     printf("entered power safe mode\r\n");
81     if(satState == PowerSafeMode) return 0;
82     logError(payloadTurnOff(), "EnterCruiseMode - payloadTurnOff");
83     vTaskDelay(100);
84     setBeaconIntervalNOT_FROM_FRAM(MAX_BEACON_INTERVAL);

```

```
85     txOff = TRUE;
86     payloadOff = TRUE;
87     satState = PowerSafeMode;
88     return 0;
89 }
90
91     /*
92  * Get if the Tx flag is on or off for us to put in the CheckAllowed in the TRXVU.c
code.
93  */
94 Boolean GetTxFlag()
95 {
96     return txOff;
97 }
98
99     /*
100  * Get if the Payload flag is on or off.
101  */
102 Boolean GetPayloadFlag()
103 {
104     return payloadOff;
105 }
106
107     /*!
108  * returns the current system state according to the EpsState_t enumeration
109  * @return system state according to EpsState_t
110  */
111 EpsState_t GetSystemState()
112 {
113     return satState;
114 }
115
116
```

```

1 /*
2 * @file   EPS.h
3 * @brief   EPS- Energy Powering System.This system is incharge of the energy
consumption
4 *         the satellite and switching on and off power switches(5V, 3V3)
5 * @see     inspect logic flowcharts thoroughly in order to write the code in a clean
manner.
6 */
7 #ifndef EPS_H_
8 #define EPS_H_
9
10 #include "GlobalStandards.h"
11 #include "EPSOperationModes.h"
12 #include "FRAM_FlightParameters.h"
13 #include "satellite-subsystems/isismepsv2_ivid7_piu.h"
14
15 #include "satellite-subsystems/IsisSolarPanelv2.h"
16
17 #include "utils.h"
18 #include "SysI2CAddr.h"
19
20 #include "SubSystemModules/Payload/payload_drivers.h"
21
22 #include "SubSystemModules/Communication/SatCommandHandler.h"
23
24 #include <stdint.h>
25 #include <hal/Drivers/SPI.h>
26 /*
27
28
29
30
31
32
33
34
35
36
37 */
38 #define DEFAULT_ALPHA_VALUE 0.3
39
40 #define NUMBER_OF_SOLAR_PANELS 6

```

```

41  #define NUMBER_OF_THRESHOLD_VOLTAGES    4    ///< first 2 are
discharging voltages, last 2 are charging voltages

42  #define DEFAULT_EPS_THRESHOLD_VOLTAGES    {(voltage_t)6900,
(voltage_t)7300, \
43      (voltage_t)7000, (voltage_t)7400}

44
45  typedef enum __attribute__((__packed__)) {
46      INDEX_DOWN_CRUISE,
47      INDEX_DOWN_OPERATIONAL,
48      INDEX_UP_CRUISE,
49      INDEX_UP_OPERATIONAL
50 }EpsThresholdsIndex;
51
52  typedef union __attribute__((__packed__)) {
53      voltage_t raw[NUMBER_OF_THRESHOLD_VOLTAGES];
54      struct {
55          voltage_t Vdown_cruise;
56          voltage_t Vdown_operational;
57          voltage_t Vup_cruise;
58          voltage_t Vup_operational;
59      }fields;
60 }EpsThreshVolt_t;
61
62  typedef union __attribute__((__packed__)) {
63      struct {
64          int16_t H1_MIN;
65          int16_t H1_MAX;
66          int16_t H2_MIN;
67          int16_t H2_MAX;
68          int16_t H3_MIN;
69          int16_t H3_MAX;
70      }value;
71 }HeaterValues;
72  /*!
73  * @brief initializes the EPS subsystem and the solar panels.
74  * @return    0 on success
75  *           -1 on failure of init
76  */
77  int EPS_And_SP_Init();
78
79  /*!
80  * @brief EPS logic. controls the state machine of which subsystem
81  * is on or off, as a function of only the battery voltage

```



```

82 * @return 0 on success
83 *        -1 on failure setting state of channels
84 */
85 int EPS_Conditioning();
86
87 /*!
88 * @brief returns the current voltage on the battery
89 * @param[out] vbat the current battery voltage
90 * @return 0 on success
91 *        Error code according to <hal/errors.h>
92 */
93 int GetBatteryVoltage(voltage_t *vbat);
94
95 /*!
96 * @brief setting the new EPS logic threshold voltages on the FRAM.
97 * @param[in] thresh_volts an array holding the new threshold values
98 * @return 0 on success
99 *        -1 on failure setting new threshold voltages
100 *        -2 on invalid thresholds
101 *        ERR according to <hal/errors.h>
102 */
103 int UpdateThresholdVoltages(EpsThreshVolt_t thresh_volts);
104
105 /*!
106 * @brief getting the EPS logic threshold voltages on the FRAM.
107 * @param[out] thresh_volts a buffer to hold the threshold values
108 * @return 0 on success
109 *        -1 on NULL input array
110 *        -2 on FRAM read errors
111 */
112 int GetThresholdVoltages(EpsThreshVolt_t *thresh_volts);
113
114 /*!
115 * @brief getting the smoothing factor (alpha) from the FRAM.
116 * @param[out] alpha a buffer to hold the smoothing factor
117 * @return 0 on success
118 *        -1 on NULL input array
119 *        -2 on FRAM read errors
120 */
121 int GetAlpha(float *alpha);
122
123 /*!
124 * @brief setting the new voltage smoothing factor (alpha) on the FRAM.
125 * @param[in] new_alpha new value for the smoothing factor alpha
126 * @note new_alpha is a value in the range - (0,1)
127 * @return 0 on success

```

```
128 *      -1 on failure setting new smoothing factor
129 *      -2 on invalid alpha
130 *      -4 FRAM_read problem
131 *      -5 written wrong data
132 * @see LPF- Low Pass Filter at wikipedia: https://en.wikipedia.org/wiki/Low-pass\_filter#Discrete-time\_realization
133 */
134     int UpdateAlpha(float alpha);
135
136     #endif
137
```

```

1 /*
2  * EPS.c
3  *
4  * Created on: 5 6 2024
5  * Author: maayan
6  */
7
8  #include "EPS.h"
9
10
11
12  #define SMOOTHING(currentVolt, alpha) (lastVoltage + (alpha * (currentVolt -
lastVoltage))) //What we enter and what we return. It's a macro function.
13
14  #define MAX_VOLTAGE_TO_STATES {7100, 7500, 7200, 7600}
15
16  #define MIN_VOLTAGE_TO_STATES {6000, 6500, 6100, 6600}
17
18
19  voltage_t lastVoltage;
20 EpsThreshVolt_t threshold_volts;
21 float Alpha;
22
23  /*!
24  * @brief initializes the EPS subsystem and the solar panels.
25  * @return 0 on success
26  *         -1 on failure of init
27  */
28  int EPS_And_SP_Init(){
29  int errorEPS = 0;
30  int errorSP = 0;
31
32  ISISMEPSV2_IVID7_PIU_t eps_init_stract;
33  eps_init_stract.i2cAddr = EPS_I2C_ADDR;
34  errorEPS = logError(ISISMEPSV2_IVID7_PIU_Init(&eps_init_stract, 1),
"EPS_And_SP_Init - ISISMEPSV2_IVID7_PIU_Init");
35  if(!errorEPS)
36  {
37      GetThresholdVoltages(&threshold_volts);
38      GetBatteryVoltage(&lastVoltage);
39      GetAlpha(&Alpha);
40  }
41  Pin solarpanelv2_pins[2] = {PIN_RESET, PIN_INT};

```

```

42  errorSP = logError(IsisSolarPanelv2_initialize(slave0_spi, &solarpanelv2_pins[0],
&solarpanelv2_pins[1]), "EPS_And_SP_Init - IsisSolarPanelv2_initialize");
43  if(errorSP == 0)
44      errorSP = logError(IsisSolarPanelv2_sleep(), "EPS_And_SP_Init -
IsisSolarPanelv2_sleep");
45  if(errorSP || errorEPS)
46      return -1;
47      return 0;
48 }
49
50  /*!
51  * @brief returns the current voltage on the battery
52  * @param[out] vbat the current battery voltage
53  * @return 0 on success
54  *         Error code according to <hal/errors.h>
55  */
56  int GetBatteryVoltage(voltage_t *vbat)
57  {
58      isimepsv2_ivid7_piu__gethousekeepingeng__from_t responseEPS; //Create a
variable that is the struct we need from EPS_isis

59      int error_eps = logError(isimepsv2_ivid7_piu__gethousekeepingeng(0
,&responseEPS), "GetBatteryVoltage - isimepsv2_ivid7_piu__gethousekeepingeng"); //
Get struct and get kind of error
60      if(error_eps) return error_eps;
61      *vbat = (voltage_t)responseEPS.fields.batt_input.fields.volt;
62      return 0;
63 }
64
65  /*!
66  * @brief getting the smoothing factor (alpha) from the FRAM.
67  * @param[out] alpha a buffer to hold the smoothing factor
68  * @return 0 on success
69  *         -1 on NULL input array
70  *         -2 on FRAM read errors
71  */
72  int GetAlpha(float *alpha)
73  {
74      if(alpha == NULL) return -1;
75      float alphaVal;
76      if(logError(FRAM_read((unsigned char*)&alphaVal,
EPS_ALPHA_FILTER_VALUE_ADDR, EPS_ALPHA_FILTER_VALUE_SIZE),
"GetAlpha - FRAM_read"))
77          return -2;
78      *alpha = alphaVal;
79      return 0;

```

```

80
81 }
82
83  /*!
84  * @brief setting the new voltage smoothing factor (alpha) on the FRAM.
85  * @param[in] new_alpha new value for the smoothing factor alpha
86  * @note new_alpha is a value in the range - (0,1)
87  * @return  0 on success
88  *          -1 on failure setting new smoothing factor
89  *          -2 on invalid alpha
90  *          -4 FRAM_read problem
91  *          -5 written wrong data
92  * @see LPF- Low Pass Filter at wikipedia: https://en.wikipedia.org/wiki/Low-pass\_filter#Discrete-time\_realization
93  */
94  int UpdateAlpha(float alpha)
95  {
96      if(alpha < 0 || alpha > 1) return -2;
97      if(logError(FRAM_write((unsigned char*)&alpha,
EPS_ALPHA_FILTER_VALUE_ADDR, EPS_ALPHA_FILTER_VALUE_SIZE),
"UpdateAlpha - FRAM_write")) return -1;
98      Alpha = alpha;
99      float writtenAlpha;
100     if(logError(GetAlpha(&writtenAlpha), "UpdateAlpha - GetAlpha")) return -4;
101     if(writtenAlpha != alpha) return -5;
102     return 0;
103 }
104
105  /*!
106  * @brief setting the new EPS logic threshold voltages on the FRAM.
107  * @param[in] thresh_volts an array holding the new threshold values
108  * @return  0 on success
109  *          -1 on failure setting new threshold voltages
110  *          -2 on invalid thresholds
111  *          -3 on failure to read
112  *          -4 written wrong
113  *          ERR according to <hal/errors.h>
114  */
115  int UpdateThresholdVoltages(EpsThreshVolt_t thresh_volts)

116  {
117      voltage_t maxes[NUMBER_OF_THRESHOLD_VOLTAGES] =
MAX_VOLTAGE_TO_STATES;
118      voltage_t mins[NUMBER_OF_THRESHOLD_VOLTAGES] =
MIN_VOLTAGE_TO_STATES;
119      for(int i = 0; i < NUMBER_OF_THRESHOLD_VOLTAGES; i++)

```

```

120     if(thresh_volts.raw[i] < mins[i] || thresh_volts.raw[i] > maxes[i]) return -2;
121     if(logError(FRAM_write((unsigned char*)&thresh_volts,
EPS_THRESH_VOLTAGES_ADDR, EPS_THRESH_VOLTAGES_SIZE),
"UpdateThresholdVoltages - FRAM_write")) return -1;
122
123     if(!GetThresholdVoltages(&threshold_volts)) return -3;
124     for(int i = 0; i < NUMBER_OF_THRESHOLD_VOLTAGES; i++)
125         if(thresh_volts.raw[i] != threshold_volts.raw[i]) return -4;
126     return 0;
127 }
128
129     /*!
130 * @brief getting the EPS logic threshold voltages on the FRAM.
131 * @param[out] thresh_volts a buffer to hold the threshold values
132 * @return 0 on success
133 *         -1 on NULL input array
134 *         -2 on FRAM read errors
135 */
136     int GetThresholdVoltages(EpsThreshVolt_t *thresh_volts)

137 {
138     if(thresh_volts == NULL) return -1;
139     if(logError(FRAM_read((unsigned char*)thresh_volts,
EPS_THRESH_VOLTAGES_ADDR, EPS_THRESH_VOLTAGES_SIZE),
"GetThresholdVoltages - FRAM_read"))
140         return -2;
141     return 0;
142 }
143
144     /*!
145 * @brief EPS logic. controls the state machine of which subsystem
146 * is on or off, as a function of only the battery voltage
147 * @return 0 on success
148 *         -1 on failure setting state of channels
149 */
150     int EPS_Conditioning()
151 {
152     voltage_t currentVoltage;
153     GetBatteryVoltage(&currentVoltage);
154     double smoothing = SMOOTHING(currentVoltage, Alpha);
155     printf("%d - ", (int)smoothing);
156     if(smoothing >= threshold_volts.fields.Vup_operational)
157         EnterOperationalMode();
158     else if(threshold_volts.fields.Vdown_operational >= smoothing && smoothing >=
threshold_volts.fields.Vup_cruise)
159         EnterCruiseMode();

```

```
160  else if(smoothing >= threshold_volts.fields.Vdown_operational)
161      BetweenOperationalToCruise();
162  else if(smoothing <= threshold_volts.fields.Vdown_cruise)
163      EnterPowerSafeMode();
164  else
165      BetweenCruiseToPowerSafeMode();
166
167  lastVoltage = smoothing;
168  return 0;
169 }
170
```

**SubSystemModules/
Payload**


```

1 #ifndef PAYLOAD_DRIVERS_H_
2 #define PAYLOAD_DRIVERS_H_
3
4 #include <hal/boolean.h>
5
6 /**
7  * @file payload_drivers.h
8  * @brief Header file for payload driver functions and structures.
9  *
10 * This file contains function declarations and data structures used
11 * to interface with the payload, including environment and event data
12 * management, as well as control operations.
13 */
14
15 /**
16 * @enum SoreqResult
17 * @brief Result codes for payload operations.
18 */
19 typedef enum {
20     PAYLOAD_SUCCESS,      /**< Operation completed successfully */
21
22     EPS_ERROR,            /**< General error in the EPS system */
23
24     PAYLOAD_I2C_WRITE_ERROR, /**< Error during I2C write operation */
25
26     PAYLOAD_I2C_READ_ERROR, /**< Error during I2C read operation */
27
28     PAYLOAD_TIMEOUT,      /**< Operation timed out */
29     FLAG_ON
30 } SoreqResult;
31
32 /**
33 * @struct PayloadEnvironmentData
34 * @brief Structure to hold RADFET and temperature readings.
35 */
36 typedef union __attribute__((packed)) _PayloadEnvironmentData
37 {
38     unsigned char raw[16];
39     struct __attribute__((packed))
40     {
41         int adc_conversion_radfet1; /**< ADC conversion result for RADFET 1 */
42
43         int adc_conversion_radfet2; /**< ADC conversion result for RADFET 2 */
44
45         double temperature;         /**< Temperature measurement in degrees Celsius

```

```

*/
40     } fields;
41 } PayloadEnvironmentData;
42
43     /**
44  * @struct PayloadEventData
45  * @brief Structure to hold SEU and SEL event readings.
46  */
47     typedef struct {
48         int seu_count; /**< Count of Single Event Upsets (SEUs) */
49
50         int sel_count; /**< Count of Single Event Latchups (SELs) */
51
52     } PayloadEventData;
53
54     /**
55  * @brief Initializes the payload system.
56  * This function performs all necessary setup for the payload, including
57  * configuring interfaces and performing initial checks.
58  * @return SoreqResult indicating success or failure of the initialization.
59  */
60     SoreqResult payloadInit();
61
62     /**
63  * @brief Reads RADFET voltages and temperature measurements.
64  * This function retrieves RADFET ADC conversion results and the current
65  * temperature measurement.
66  * @param[out] environment_data Pointer to a PayloadEnvironmentData structure
67  * where the readings will be stored.
68  * @return SoreqResult indicating success or failure of the read operation.
69  */
70     SoreqResult payloadReadEnvironment(PayloadEnvironmentData
71     *environment_data);
72
73     /**
74  * @brief Reads SEU and SEL event counts.
75  * This function retrieves the count of Single Event Upsets (SEUs) and
76  * Single Event Latchups (SELs) recorded by the payload.
77  * @param[out] event_data Pointer to a PayloadEventData structure where the

```

```

82 *          event counts will be stored.
83 *
84 * @return SoreqResult indicating success or failure of the read operation.
85 */
86 SoreqResult payloadReadEvents(PayloadEventData *event_data);
87
88 /**
89 * @brief Performs a soft reset of the payload system.
90 *
91 * This function resets the payload without affecting its power state, allowing
92 * recovery from transient errors or inconsistent states.
93 *
94 * @return SoreqResult indicating success or failure of the reset operation.
95 */
96 SoreqResult payloadSoftReset();
97
98 /**
99 * @brief Turns off the payload.
100 *
101 * This function powers down the payload system.
102 *
103 * @return SoreqResult indicating success or failure of the operation.
104 */
105 SoreqResult payloadTurnOff();
106
107 /**
108 * @brief Turns on the payload.
109 *
110 * This function powers up the payload system.
111 *
112 * @return SoreqResult indicating success or failure of the operation.
113 */
114 SoreqResult payloadTurnOn();
115
116 #endif /* PAYLOAD_DRIVERS_H_ */
117

```

```

1
2
3  #include "payload_drivers.h"
4  #include "hal/Drivers/I2C.h"
5  #include <string.h>
6  #include <hal/Timing/Time.h>
7  #include <satellite-subsystems/isismepsv2_ivid7_piu.h>

8  #include "FRAM_FlightParameters.h"
9
10 #define PAYLOAD_I2C_ADDRESS 0x55
11 #define CLEAR_WDT 0x3F
12 #define SOFT_RESET 0xF8
13 #define READ_RADFET_VOLTAGES 0x33
14 #define MEASURE_TEMPERATURE 0x77
15 #define READ_PIC32_SEU 0x47
16 #define READ_PIC32_SEL 0x66
17 #define TIMEOUT 4000
18 #define READ_DELAY 200
19
20 #define EPS_INDEX 0
21 #define PAYLOAD_BUS_CHANNEL 4
22
23 // Macro to convert endianness
24 #define CHANGE_ENDIAN(x) ((x) = ((x) >> 24 & 0xff) | ((x) << 8 & 0xff0000) |
((x) >> 8 & 0xff00) | ((x) << 24 & 0xff000000))
25
26
27 SoreqResult payloadInit() {
28     return payloadTurnOn();
29 }
30
31 SoreqResult payloadRead(int size, unsigned char *buffer) {
32     unsigned char wtd_and_read[] = {CLEAR_WDT};
33     int i;
34     for (i = 0; i < TIMEOUT / READ_DELAY; ++i) {
35         if (I2C_write(PAYLOAD_I2C_ADDRESS, wtd_and_read, 1) != 0) return
PAYLOAD_I2C_WRITE_ERROR;
36         vTaskDelay(READ_DELAY);
37         if (I2C_read(PAYLOAD_I2C_ADDRESS, buffer, size) == 0 && buffer[3] == 0)
return PAYLOAD_SUCCESS;
38     }
39     return PAYLOAD_TIMEOUT;
40 }
41
42 SoreqResult payloadSendCommand(char opcode, int size, unsigned char *buffer, int

```

```

delay) {
43  if (I2C_write(PAYLOAD_I2C_ADDRESS, (unsigned char *)&opcode, 1) != 0)
return PAYLOAD_I2C_WRITE_ERROR;
44  vTaskDelay(delay);
45  return payloadRead(size, buffer);
46 }
47
48
49  #define ADC_TO_VOLTAGE(R) ((2 * 4.096 * (R)) / (2 << 23))

50  #define VOLTAGE_TO_TEMPERATURE(V) (100 * ((V) * (5 / 2.0) - 2.73))

51
52 SoreqResult payloadReadEnvironment(PayloadEnvironmentData
*environment_data) {
53  unsigned char buffer[12];
54  SoreqResult res;
55
56  // Read RADFET voltages
57  res = payloadSendCommand(READ_RADFET_VOLTAGES, sizeof(buffer), buffer,
1250 / portTICK_RATE_MS);
58  if (res != PAYLOAD_SUCCESS) {
59      return res;
60  }
61  memcpy(&environment_data->fields.adc_conversion_radfet1, buffer + 4, 4);
62  memcpy(&environment_data->fields.adc_conversion_radfet2, buffer + 8, 4);
63  CHANGE_ENDIAN(environment_data->fields.adc_conversion_radfet1);
64  CHANGE_ENDIAN(environment_data->fields.adc_conversion_radfet2);
65
66  // Read temperature ADC value
67  int raw_temperature_adc;
68  res = payloadSendCommand(MEASURE_TEMPERATURE, sizeof(buffer), buffer,
845 / portTICK_RATE_MS);
69  if (res != PAYLOAD_SUCCESS) {
70      return res;
71  }
72  memcpy(&raw_temperature_adc, buffer + 4, 4);
73  CHANGE_ENDIAN(raw_temperature_adc);
74
75  // Extract and process ADC value
76  int remove_extra_bits = (raw_temperature_adc & ~(1 << 29)) >> 5; // Mask and
shift to remove redundant bits
77  double voltage = ADC_TO_VOLTAGE(remove_extra_bits); // Convert ADC value
to voltage
78  double temperature = VOLTAGE_TO_TEMPERATURE(voltage); // Convert
voltage to temperature

```

```

79  environment_data->fields.temperature = temperature;
80
81  return PAYLOAD_SUCCESS;
82 }
83
84
85 SoreqResult payloadReadEvents(PayloadEventData *event_data) {
86     unsigned char buffer[12];
87     SoreqResult res;
88
89     // Read SEL count
90     if ((res = payloadSendCommand(READ_PIC32_SEL, sizeof(buffer), buffer, 10 /
portTICK_RATE_MS)) != PAYLOAD_SUCCESS)
91         return res;
92     memcpy(&event_data->sel_count, buffer + 4, 4);
93     if (event_data->sel_count == 0) memcpy(&event_data->sel_count, buffer + 8, 4);
94     CHANGE_ENDIAN(event_data->sel_count);
95
96     // Read SEU count
97     if ((res = payloadSendCommand(READ_PIC32_SEU, sizeof(buffer), buffer, 100 /
portTICK_RATE_MS)) != PAYLOAD_SUCCESS)
98         return res;
99     memcpy(&event_data->seu_count, buffer + 4, 4);
100    CHANGE_ENDIAN(event_data->seu_count);
101
102    return PAYLOAD_SUCCESS;
103 }
104
105 SoreqResult payloadSoftReset() {
106     return payloadSendCommand(SOFT_RESET, 0, NULL, 0);
107 }
108
109 SoreqResult payloadTurnOff() {
110     isimepsv2_ivid7_piu__replyheader_t response;
111     return isimepsv2_ivid7_piu__outputbuschanneloff(EPS_INDEX,
PAYLOAD_BUS_CHANNEL, &response) ? EPS_ERROR : PAYLOAD_SUCCESS;
112 }
113
114 SoreqResult payloadTurnOn() {
115     Boolean flag;
116     FRAM_read((unsigned char*)&flag, PAYLOAD_IS_DEAD_ADDR,
PAYLOAD_IS_DEAD_SIZE);
117     if(flag) return FLAG_ON;
118     isimepsv2_ivid7_piu__replyheader_t response;
119     return isimepsv2_ivid7_piu__outputbuschannelon(EPS_INDEX,
PAYLOAD_BUS_CHANNEL, &response) ? EPS_ERROR : PAYLOAD_SUCCESS;

```

120 }

121

122

123

**SubSystemModules/
Housekeeping**


```

1
2 #ifndef TELEMETRYFILES_H_
3 #define TELEMETRYFILES_H_
4 // ---general
5
6 #define END_FILE_NAME_TX          "tx"
7 #define END_FILE_NAME_RX          "rx"
8 #define END_FILE_NAME_ANTENNA_SIDE_ZERO    "ata"
9 #define END_FILE_NAME_ANTENNA_SIDE_ONE     "atb"
10 #define END_FILENAME_WOD_TLM          "wod"
11 #define END_FILENAME_EPS_TLM          "eps"
12 #define END_FILENAME_SOLAR_PANELS_TLM    "slr"
13 #define END_FILENAME_LOGS             "log"
14 #define END_FILENAME_RADFET_TLM        "rad"
15 #define END_FILENAME_SEL_TLM          "sel"
16 #define END_FILENAME_SEU_TLM          "seu"
17
18
19 typedef enum {
20 // don't change the position of these
21 tlm_eps, // 0
22 tlm_tx,
23 tlm_ants0,
24 tlm_solar,
25 tlm_wod,
26 tlm_radfet,
27 tlm_sel,
28 tlm_seu,
29 // don't change the position of these
30
31 tlm_rx, // 8
32 tlm_ants1,
33 tlm_log
34 }tlm_type_t;
35 #endif /* TELEMETRYFILES_H_ */
36

```

```

1 #ifndef TELEMETRYCOLLECTOR_H_
2 #define TELEMETRYCOLLECTOR_H_
3
4 #include "GlobalStandards.h"
5 #include "TelemetryFiles.h"
6 #include "TLM_management.h"
7 #include <satellite-subsystems/IsisSolarPanelv2.h>
8
8 #include "SubSystemModules/PowerManagment/EPS.h"
9
9 #include "SubSystemModules/Communication/TRXVU.h"
10
10 #include <hal/supervisor.h>
11 #include <hcc/api_fat.h>
12 #include <hal/Storage/FRAM.h>
13 #include "TLM_management.h"
14
15 #include "utils.h"
16 #define NUM_OF_SUBSYSTEMS_SAVE_FUNCTIONS 9
17
18 typedef union __attribute__((packed)) _PeriodTimes
19 {
20     unsigned char raw[28];
21     struct __attribute__((packed))
22     {
23         unsigned int eps;
24         unsigned int trxvu;
25         unsigned int ants;
26         unsigned int solar_panels;
27         unsigned int wod;
28         unsigned int radfet;
29         unsigned int seu_sel;
30     } fields;
31 } PeriodTimes;
32
33 PeriodTimes periods;
34
35 typedef struct __attribute__((packed)) WOD_Telemetry_t
36 {
37     voltage_t voltBattery;          ///< the current voltage on the battery [mV]
38
38     voltage_t volt_5V;              ///< the current voltage on the 5V bus [mV]
39
39     voltage_t volt_3V3;             ///< the current voltage on the 3V3 bus [mV]

```

```

40  power_t charging_power;          ///< the current charging power [mW]
41  power_t consumed_power;          ///< the power consumed by the satellite [mW]
42  current_t electric_current;      ///< the up-to-date electric current of the battery
[mA]
43  current_t current_3V3;           ///< the up-to-date 3.3 Volt bus current of the
battery [mA]
44  current_t current_5V;           ///< the up-to-date 5 Volt bus current of the battery
[mA]
45  temp_t bat_temp;                /*!< 2 cell battery pack: not used 4 cell battery pack:
Battery pack temperature on the front of the battery pack. [degC] */

46  float solar_panels[NUMBER_OF_SOLAR_PANELS]; // temp of each solar panel
47  time_unix sat_time;             ///< current Unix time of the satellites clock   [sec]
48  unsigned int free_memory;        ///< number of bytes free in the satellites SD
[byte]
49  unsigned int corrupt_bytes;      ///< number of corrupted bytes in the memory
[bytes]
50  unsigned int total_memory;       ///< number of bytes in the memory   [bytes]
51  unsigned int used_bytes;         ///< number of used bytes in the memory   [bytes]
52  int lastFS_error;               ///< last error of the file system

53  unsigned int number_of_resets;   ///< counts the number of resets the satellite
has gone through [#]
54  time_unix sat_uptime;           ///< Sat uptime
55  unsigned int num_of_cmd_resets;  ///< counts the number of resets the satellite
has gone through due to ground station command [#]

56  int sel_counter;
57  int seu_counter;
58  int sensor_one_radfet;
59  int sensor_two_radfet;
60  double radfet_temp;
61  time_unix last_radfet_read_time;
62  power_t power_payload;
63  Boolean payload_flag;
64  EpsState_t eps_state;
65  int change_in_mode;
66  float temp_pa;
67  float temp_board;

```

```

68 float reflected_power; //in mW
69 float forward_power; //in mW
70 float doppler;
71 float rssi;
72 } WOD_Telemetry_t;
73
74
75     typedef struct solar_tlm { float values[ISIS_SOLAR_PANEL_COUNT]; }
solar_tlm_t;
76
77     typedef struct
78 {
79     int count;
80     int sat_resets_count;
81     int changes_in_mode;
82 } payloadSEL_data;
83
84     /**
85  * get all tlm save time periods from FRAM
86  */
87     void InitSavePeriodTimes();
88
89     /*!
90  * @brief saves all telemetries into the appropriate TLM files
91  */
92     void TelemetryCollectorLogic();
93
94     /*!
95  * @brief saves current EPS telemetry into file
96  */
97     void TelemetrySaveEPS();
98
99     /*!
100  * @brief saves current TRXVU telemetry into file
101  */
102     void TelemetrySaveTx();
103
104     /*!
105  * @brief saves current TRXVU telemetry into file
106  */
107     void TelemetrySaveRx();
108
109
110     /*!
111  * @brief saves current Antenna 0 telemetry into file
112  */

```

```

113     void TelemetrySaveAnt0();
114
115     /*!
116  * @brief saves current Antenna 1 telemetry into file
117  */
118     void TelemetrySaveAnt1();
119
120     /*!
121  * @brief saves current solar panel telemetry (temparture of each panel) into file
122  */
123     void TelemetrySaveSolarPanels();
124
125     /*!
126  * @brief saves current RADFET telemetry into file (one of the payloads)
127  */
128     void TelemetrySavePayloadRADFET();
129
130     /*!
131  * @brief saves current events telemetry into file (two of the payloads)
132  */
133     void TelemetrySavePayloadEvents();
134
135     /*!
136  * @brief saves current WOD telemetry into file
137  */
138     void TelemetrySaveWOD();
139
140     /*!
141  * @brief Gets all necessary telemetry and arranges it into a WOD structure
142  * @param[out] name=wod; type=WOD_Telemetry_t*; output WOD telemetry. If an
143  * error occurred while getting TLM the fields will be -1
144  * @return 0
145  */
146     int GetCurrentWODTelemetry(WOD_Telemetry_t *wod);
147
148     /*!
149  * @brief get current SEL telemetry (one of the payloads)
150  * @param[in] name=eventsData; type=PayloadEventData; the reading from the
151  * payload of the sel and seu.
152  * @param[out] name=selData; type=payloadSEL_data*; the struct we will save in
153  * file and we fill here.
154  */
155     void GetSEL_telemetry(PayloadEventData eventsData, payloadSEL_data
156     *selData);

```

```
155 * @brief saves current SEL telemetry into file (one of the payloads), and call the  
get function  
156 * @param[in] name=eventsData; type=PayloadEventData; the reading from the  
payload of the sel and seu.  
157 */  
158     void TelemetrySavePayloadSEL(PayloadEventData eventsData);  
159  
160     /*  
161 * Check according to the eps chanel if the payload is off  
162 * @return type=Boolean; TRUE is on, FALSE off or error  
163 * */  
164 Boolean IsThePayloadOn();  
165     #endif /* TELEMETRYCOLLECTOR_H_ */  
166
```

```

1 /*
2  * TelemetryCollector.c
3  *
4  * Created on: 31 7 2024
5  * Author: maayan
6  */
7
8  #include "TelemetryCollector.h"
9  #include <hal/supervisor.h>
10 #include <hcc/api_fat.h>
11 #include <satellite-subsystems/ismepsv2_ivid7_piu.h>
12
13 #include <satellite-subsystems/IsisSolarPanelv2.h>
14
15 #include "utils.h"
16
17 time_unix lastTimeSave[NUM_OF_SUBSYSTEMS_SAVE_FUNCTIONS] = {0};
18
19 /*
20  * get unix_time
21  */
22 time_unix GetTime()
23 {
24     time_unix time = 0;
25     logError(Time_getUnixEpoch((unsigned int*)&time), "GetTime -
Time_getUnixEpoch");
26     return time;
27 }
28
29 /**
30  * get all tlm save time periods from FRAM
31  */
32 void InitSavePeriodTimes()
33 {
34     logError(FRAM_read(periods.raw, TLM_SAVE_PERIOD_START_ADDR, sizeof
(periods.raw)), "InitSavePeriodTimes - FRAM_read");
35 }
36
37 /*
38  * @brief Gets all necessary telemetry and arranges it into a WOD structure
39  * @param[out] name=wod; type=WOD_Telemetry_t*; output WOD telemetry. If an
error occurred while getting TLM the fields will be -1
40  * @return 0
41  */
42 int GetCurrentWODTelemetry(WOD_Telemetry_t *wod)

```

```

41 {
42     supervisor_housekeeping_t mySupervisor_housekeeping_hk; //create a variable
that is the struct we need from supervisor
43     int error_supervisor =
logError(Supervisor_getHousekeeping(&mySupervisor_housekeeping_hk,
SUPERVISOR_SPI_INDEX), "GetCurrentWODTelemetry -
Supervisor_getHousekeeping"); //gets the variables to the struct and also check error.

44     F_SPACE space; //same just to SD
45     int ret = logError(f_getfreespace(f_getdrive(), &space),
"GetCurrentWODTelemetry - f_getfreespace"); //gets the variables to the struct

46     isis_vu_e__get_tx_telemetry__from_t telemetryTx;
47     int rvTx = isis_vu_e__get_tx_telemetry(0, &telemetryTx);
48     isis_vu_e__get_rx_telemetry__from_t telemetryRx;
49     int rvRx = isis_vu_e__get_rx_telemetry(0, &telemetryRx);
50     isismepsv2_ivid7_piu__gethousekeepingeng__from_t responseEPS; //Create a
variable that is the struct we need from EPS_isis

51     int error_eps = logError(ismepsv2_ivid7_piu__gethousekeepingeng(0
,&responseEPS), "GetCurrentWODTelemetry -
ismepsv2_ivid7_piu__gethousekeepingeng"); //Get struct and get kind of error

52     if(!error_eps)
53     {
54         wod->voltBattery = responseEPS.fields.batt_input.fields.volt;
55         wod->consumed_power = responseEPS.fields.dist_input.fields.power * 10;
56         wod->electric_current = responseEPS.fields.batt_input.fields.current;
57         wod->bat_temp = ((double)responseEPS.fields.temp2) * 0.01;
58         wod->current_3V3 = responseEPS.fields.vip_obc05.fields.current;
59         wod->current_5V = responseEPS.fields.vip_obc01.fields.current;
60         wod->volt_3V3 = responseEPS.fields.vip_obc05.fields.volt;
61         wod->volt_5V = responseEPS.fields.vip_obc01.fields.volt;
62         wod->charging_power = responseEPS.fields.batt_input.fields.power * 10;
63         wod->power_payload = responseEPS.fields.vip_obc04.fields.power * 10;
64     }
65     else // if have error in the eps put everything in that section to -1

66     {
67         wod->voltBattery = -1;
68         wod->consumed_power = -1;
69         wod->electric_current = -1;
70         wod->bat_temp = -1;
71         wod->volt_5V = -1;
72         wod->volt_3V3 = -1;
73         wod->charging_power = -1;

```



```

74     wod->current_5V = -1;
75     wod->current_3V3 = -1;
76     wod->power_payload = -1;
77 }
78
79 IsisSolarPanelv2_wakeup();
80 int error_sp;
81 uint8_t status = 0;
82 int32_t paneltemp = 0;
83 float conv_temp;
84 for(int panel = 0; panel < NUMBER_OF_SOLAR_PANELS; panel++) //Go for the
count of solar panels we have.
85 {
86     error_sp = IsisSolarPanelv2_getTemperature(panel, &paneltemp, &status); //
Gets the temperature of each panel and the error message.
87     if( error_sp ) //if there is error
88     {
89         wod->solar_panels[panel] = -1;
90         continue;
91     }
92     conv_temp = (float)(paneltemp) * ISIS_SOLAR_PANEL_CONV;
93     wod->solar_panels[panel] = conv_temp;
94 }
95 IsisSolarPanelv2_sleep(); //Puts the internal temperature sensor to sleep mode
96
97 if(!error_supervisor)
98     wod->sat_uptime = mySupervisor_housekeeping_hk.fields.iobcUptime /
portTICK_RATE_MS;
99 else // if have error in the supervisor put everything in that section to -1
100     wod->sat_uptime = -1;
101
102 if(!ret)
103 {
104     wod->free_memory = space.free;
105     wod->corrupt_bytes = space.bad;
106     wod->total_memory = space.total;
107     wod->used_bytes = space.used;
108 }
109 else // if have error in the file_system put everything in that section to -1
110 {
111     wod->free_memory = -1;
112     wod->corrupt_bytes = -1;

```

```

113     wod->total_memory = -1;
114     wod->used_bytes = -1;
115 }
116 wod->lastFS_error = f_getlasterror();
117
118 if(!rvTx)
119 {
120     wod->temp_pa = ((float) telemetryTx.fields.temp_pa) * -0.07669 + 195.6037;
121     wod->temp_board = ((float)telemetryRx.fields.temp_board) * -0.07669 +
195.6037;
122     short telemetryValue = telemetryTx.fields.reflected_power;
123     wod->reflected_power = ((float)(telemetryValue * telemetryValue)) * 5.887E-5;
124     telemetryValue = telemetryTx.fields.forward_power;
125     wod->forward_power = ((float)(telemetryValue * telemetryValue)) * 5.887E-5;
126 }
127 else
128 {
129     wod->temp_pa = -1;
130     wod->temp_board = -1;
131     wod->reflected_power = -1;
132     wod->forward_power = -1;
133 }
134
135 if(!rvRx)
136 {
137     wod->doppler = ((float)telemetryRx.fields.doppler) * (-38.1469726563);
138     wod->rssi = ((float)telemetryRx.fields.rssi) * (-0.5) - 22;
139 }
140 else
141 {
142     wod->doppler = -1;
143     wod->rssi = -1;
144 }
145
146 if(logError(Time_getUnixEpoch((unsigned int*)&wod->sat_time),
147 "GetCurrentWODTelemetry - Time_getUnixEpoch")) // if have error in getting the sat
148 time put the time to -1
149     wod->sat_time = -1;
150
151 unsigned int numberOfResets;
152 unsigned int numberOfCMDResets;
153 if(logError(FRAM_read((unsigned char*)&numberOfResets,
154 NUMBER_OF_RESETS_ADDR, NUMBER_OF_RESETS_SIZE),
155 "GetCurrentWODTelemetry - FRAM_read resets")) wod->number_of_resets = -1;
156 else wod->number_of_resets = numberOfResets;
157 if(logError(FRAM_read((unsigned char*)&numberOfCMDResets,

```

```

NUMBER_OF_CMD_RESETS_ADDR, NUMBER_OF_CMD_RESETS_SIZE),
"GetCurrentWODTelemetry - FRAM_read cmd resets")) wod->num_of_cmd_resets = -1;
154     else wod->num_of_cmd_resets = numberOfCMDResets;
155
156     if(IsThePayloadOn())
157     {
158         PayloadEventData eventsData;
159         if(!logError(payloadReadEvents(&eventsData), "GetCurrentWODTelemetry -
payloadReadEvents"))
160         {
161             wod->sel_counter = eventsData.sel_count;
162             wod->seu_counter = eventsData.seu_count;
163         }
164         else
165         {
166             wod->sel_counter = -1;
167             wod->seu_counter = -1;
168         }
169     }
170     else
171     {
172         wod->sel_counter = -1;
173         wod->seu_counter = -1;
174     }
175
176     PayloadEnvironmentData radfetData;
177     if(!logError(FRAM_read(radfetData.raw, LAST_RADFET_READ_START, sizeof
(radfetData.raw)), "GetCurrentWODTelemetry - FRAM_read"))
178     {
179         wod->sensor_one_radfet = radfetData.fields.adc_conversion_radfet1;
180         wod->sensor_two_radfet = radfetData.fields.adc_conversion_radfet2;
181         wod->radfet_temp = radfetData.fields.temperature;
182     }
183     else
184     {
185         wod->sensor_one_radfet = -1;
186         wod->sensor_two_radfet = -1;
187         wod->radfet_temp = -1;
188     }
189
190     time_unix lastTime;
191     if(!logError(FRAM_read((unsigned char*)&lastTime,
TIME_LAST_RADFET_READ_ADDR, TIME_LAST_RADFET_READ_SIZE),
"GetCurrentWODTelemetry - FRAM_read"))
192         wod->last_radfet_read_time = lastTime;
193     else

```

```

194     wod->last_radfet_read_time = -1;
195
196     Boolean flagPayload;
197     if(!logError(FRAM_read((unsigned char*)&flagPayload,
PAYLOAD_IS_DEAD_ADDR, PAYLOAD_IS_DEAD_SIZE), "GetCurrentWODTelemetry
- FRAM_read"))
198         wod->payload_flag = flagPayload;
199     else
200         wod->payload_flag = -1;
201
202     wod->eps_state = GetSystemState();
203
204     int countChange = 0;
205     if(!logError(FRAM_read((unsigned char*)&countChange,
NUM_OF_CHANGES_IN_MODE_ADDR, NUM_OF_CHANGES_IN_MODE_SIZE),
"GetCurrentWODTelemetry - FRAM_read"))
206         wod->change_in_mode = countChange;
207     else
208         wod->change_in_mode = -1;
209
210     return 0;
211 }
212
213     /*!
214     * @brief saves current WOD telemetry into file
215     */
216     void TelemetrySaveWOD()
217 {
218     time_unix time = GetTime();
219     if(time == 0) return;
220     WOD_Telemetry_t wod;
221     if(GetCurrentWODTelemetry(&wod)) return;
222     Write2File(&wod, tlm_wod);
223     lastTimeSave[tlm_wod] = time;
224 }
225
226
227     /*!
228     * @brief saves current EPS telemetry into file
229     */
230     void TelemetrySaveEPS()
231 {
232     isismepsv2_ivid7_piu__gethousekeepingeng__from_t responseEPS; //Create a
variable that is the struct we need from EPS_isis
233
234     time_unix time = GetTime();

```

```

234     if(time == 0) return;
235     lastTimeSave[tlm_eps] = time;
236     if(!logError(isismepsv2_ivid7_piu__gethousekeepingeng(0,&responseEPS),
"TelemetrySaveEPS - isismepsv2_ivid7_piu__gethousekeepingeng"))
237         Write2File(&responseEPS, tlm_eps); //Get struct and get kind of error

238
239 }
240
241     /*!
242  * @brief saves current TRXVU telemetry into file
243  */
244     void TelemetrySaveTx()
245 {
246     isis_vu_e__get_tx_telemetry__from_t txTelem;
247     time_unix time = GetTime();
248     if(time == 0) return;
249     lastTimeSave[tlm_tx] = time;
250     if(!logError(isis_vu_e__get_tx_telemetry(0, &txTelem), "TelemetrySaveTx -
isis_vu_e__get_tx_telemetry"))
251         Write2File(&txTelem, tlm_tx);
252 }
253
254     /*!
255  * @brief saves current TRXVU telemetry into file
256  */
257     void TelemetrySaveRx()
258 {
259     isis_vu_e__get_rx_telemetry__from_t rxTelem;
260     time_unix time = GetTime();
261     if(time == 0) return;
262     lastTimeSave[tlm_rx] = time;
263     if(!logError(isis_vu_e__get_rx_telemetry(0, &rxTelem), "TelemetrySaveRx -
isis_vu_e__get_rx_telemetry"))
264         Write2File(&rxTelem, tlm_rx);
265
266 }
267
268     /*!
269  * @brief saves current Antenna 0 telemetry into file
270  */
271     void TelemetrySaveAnt0()
272 {
273     isis_ants__get_all_telemetry__from_t antsTelem;
274     time_unix time = GetTime();
275     if(time == 0) return;

```

```

276 lastTimeSave[tlm_ants0] = time;
277 if(!logError(isis_ants__get_all_telemetry(0, &antsTelem), "TelemetrySaveAnt0 -
isis_ants__get_all_telemetry"))
278     Write2File(&antsTelem, tlm_ants0);
279 }
280
281  /*!
282  * @brief saves current Antenna 1 telemetry into file
283  */
284  void TelemetrySaveAnt1()
285 {
286     isis_ants__get_all_telemetry__from_t antsTelem;
287     time_unix time = GetTime();
288     if(time == 0) return;
289     lastTimeSave[tlm_ants1] = time;
290     if(!logError(isis_ants__get_all_telemetry(1, &antsTelem), "TelemetrySaveAnt1 -
isis_ants__get_all_telemetry"))
291         Write2File(&antsTelem, tlm_ants1);
292 }
293
294  /*!
295  * @brief saves current solar panel telemetry (temparture of each panel) into file
296  */
297  void TelemetrySaveSolarPanels()
298 {
299     time_unix time = GetTime();
300     if(time == 0) return;
301     IsisSolarPanelv2_wakeup();
302     int error_sp;
303     uint8_t status = 0;
304     int32_t paneltemp = 0;
305     float conv_temp;
306     solar_tlm_t tempSolar;
307     for(int panel = 0; panel < NUMBER_OF_SOLAR_PANELS; panel++ ) //Go for the
count of solar panels we have.
308     {
309         error_sp = IsisSolarPanelv2_getTemperature(panel, &paneltemp, &status); //
Gets the temperature of each panel and the error message.
310         if(error_sp) //if there is error
311         {
312             logError(error_sp, "TelemetrySaveSolarPanels -
IsisSolarPanelv2_getTemperature");
313             tempSolar.values[panel] = -1;
314             continue;
315         }

```

```

316     conv_temp = (float)(paneltemp) * ISIS_SOLAR_PANEL_CONV;
317     tempSolar.values[panel] = conv_temp;
318 }
319 IsisSolarPanelv2_sleep(); //Puts the internal temperature sensor to sleep mode

320 Write2File(&tempSolar, tlm_solar);
321 lastTimeSave[tlm_solar] = time;
322 }
323
324  /*!
325  * @brief saves current RADFET telemetry into file (one of the payloads)
326  */
327  void TelemetrySavePayloadRADFET()
328  {
329      time_unix time = GetTime();
330      if(time == 0) return;
331      PayloadEnvironmentData radfetData;
332      lastTimeSave[tlm_radfet] = time;
333      if(logError(payloadReadEnvironment(&radfetData),
334      "TelemetrySavePayloadRADFET - payloadReadEnvironment")) return;
335      Write2File(&radfetData, tlm_radfet);
336      logError(FRAM_writeAndVerify(radfetData.raw, LAST_RADFET_READ_START,
337      sizeof(radfetData.raw)), "TelemetrySavePayloadRADFET - FRAM_writeAndVerify");
338      logError(FRAM_writeAndVerify((unsigned char*)&time,
339      TIME_LAST_RADFET_READ_ADDR, TIME_LAST_RADFET_READ_SIZE),
340      "TelemetrySavePayloadRADFET - FRAM_writeAndVerify");
341  }
342
343  /*!
344  * @brief get current SEL telemetry (one of the payloads)
345  * @param[in] name=eventsData; type=PayloadEventData; the reading from the
346  * payload of the sel and seu.
347  * @param[out] name=seldata; type=payloadSEL_data*; the struct we will save in
348  * file and we fill here.
349  */
350  void GetSEL_telemetry(PayloadEventData eventsData, payloadSEL_data
351  *seldata)
352  {
353      seldata->count = eventsData.sel_count;
354      if(logError(FRAM_read((unsigned char*)&seldata->sats_resets_count,
355      NUMBER_OF_CMD_RESETS_ADDR, NUMBER_OF_CMD_RESETS_SIZE),
356      "GetSEL_telemetry - FRAM_read resets")) seldata->sats_resets_count = -1;
357      if(logError(FRAM_read((unsigned char*)&seldata->changes_in_mode,
358      NUM_OF_CHANGES_IN_MODE_ADDR, NUM_OF_CHANGES_IN_MODE_SIZE),
359      "GetSEL_telemetry - FRAM_read change in mode")) seldata->changes_in_mode = -1;
360  }

```

```

350
351     /*!
352  * @brief saves current SEL telemetry into file (one of the payloads), and call the
get function
353  * @param[in] name=eventsData; type=PayloadEventData; the reading from the
payload of the sel and seu.
354  */
355     void TelemetrySavePayloadSEL(PayloadEventData eventsData)

356 {
357     payloadSEL_data selData;
358     GetSEL_telemetry(eventsData, &selData);
359     Write2File(&selData, tlm_sel);
360 }
361
362     /*!
363  * @brief saves current events telemetry into file (two of the payloads)
364  */
365     void TelemetrySavePayloadEvents()
366 {
367     time_unix time = GetTime();
368     if(time == 0) return;
369     PayloadEventData eventsData;
370     lastTimeSave[tlm_sel] = time;
371     lastTimeSave[tlm_seu] = time;
372     if(logError(payloadReadEvents(&eventsData), "TelemetrySavePayloadEvents -
payloadReadEvents")) return;
373     Write2File(&eventsData.seu_count, tlm_seu);
374     TelemetrySavePayloadSEL(eventsData);
375 }
376
377     /*
378  * Check according to the eps chanel if the payload is off
379  * @return type=Boolean; TRUE is on, FALSE off or error
380  */
381 Boolean IsThePayloadOn()
382 {
383     isismepsv2_ivid7_piu__gethousekeepingeng__from_t response; //Create a
variable that is the struct we need from EPS_isis

384     int error_eps = logError(isismepsv2_ivid7_piu__gethousekeepingeng(0
,&response), "GetCurrentWODTelemetry -
isismepsv2_ivid7_piu__gethousekeepingeng"); //Get struct and get kind of error

385     if(error_eps) return FALSE;
386     if(response.fields.vip_obc04.fields.power == 0)

```



```

387     return FALSE;
388     return TRUE;
389 }
390
391     /*!
392     * @brief saves all telemetries into the appropriate TLM files
393     */
394     void TelemetryCollectorLogic()
395 {
396     if(CheckExecutionTime(lastTimeSave[tlm_eps], periods.fields.eps))
397         TelemetrySaveEPS();
398     if(CheckExecutionTime(lastTimeSave[tlm_tx], periods.fields.trxvu))
399         TelemetrySaveTx();
400     if(CheckExecutionTime(lastTimeSave[tlm_rx], periods.fields.trxvu))
401         TelemetrySaveRx();
402     if(CheckExecutionTime(lastTimeSave[tlm_ants0], periods.fields.ants))
403         TelemetrySaveAnt0();
404     if(CheckExecutionTime(lastTimeSave[tlm_ants1], periods.fields.ants))
405         TelemetrySaveAnt1();
406     if(CheckExecutionTime(lastTimeSave[tlm_wod], periods.fields.wod))
407         TelemetrySaveWOD();
408     if(CheckExecutionTime(lastTimeSave[tlm_solar], periods.fields.solar_panels))
409         TelemetrySaveSolarPanels();
410     if(IsThePayloadOn())
411     {
412         if(CheckExecutionTime(lastTimeSave[tlm_radfet], periods.fields.radfet))
413             TelemetrySavePayloadRADFET();
414
415         if(CheckExecutionTime(lastTimeSave[tlm_seu], periods.fields.seu_sel))
416             TelemetrySavePayloadEvents();
417     }
418
419
420 }
421
422

```

SubSystemModules/ Maintenance

```

1
2  #ifndef MAINTENANCE_H_
3  #define MAINTENANCE_H_
4
5  #include "GlobalStandards.h"
6  #include <hcc/api_fat.h>
7
8  #include "TLM_management.h"
9
10 #define MIN_FREE_SPACE_PERCENTAGE 20
11
12 #define MIN_FREE_SPACE_PERCENTAGE_TO_DELETE 25
13
14 #define DEPLOY_INTRAVAL 1800 //TBD: RBF every 1800 sec (30 minutes)
15
16 #define RESET_KEY 0xA6 // need to send this key to the reset command
17 // otherwise reset will not happen
18
19 /*
20  * Delete old files according to how much space we have left. (20% for start delete
21  * until we have 25% free)
22  * MIN_FREE_SPACE_PERCENTAGE - the minimum free space in bytes we want
23  * to keep in the SD in all times.
24  * If free space<minFreeSpace we start deleting old TLM files
25  */
26 void DeleteOldFiles();
27
28 /*
29  * @brief checks if the period time has passed
30  * @param[in] prev_exec_time last sample time (last execution time)
31  * @param[in] period period of execution time
32  * @return TRUE if difference between now and last execution time is longer than
33  * 'period'
34  * FALSE otherwise
35  */
36 Boolean CheckExecutionTime(time_unix prev_time, unsigned int period);
37
38 /*
39  * @brief checks if there is a memory corruption in the file system.
40  * @return TRUE if is corrupted.
41  * FALSE if no corruption (yay!)
42  */
43 Boolean IsFS_Corrupted();
44
45 /*

```

```

40 * @brief kick the ground station communication WDT because communication took
place.
41 */
42 void KickGroundCommWDT();
43
44 /*!
45 * @brief Checks if last GS communication time has exceeded its maximum
allowed time.
46 * @see NO_COMMUNICATION_WDT_KICK_TIME
47 * @return TRUE if a comm reset is needed- no communication for a long time
48 * FALSE no need for a reset. last communication is within range
49 */
50 Boolean IsGroundCommunicationWDTReset();
51
52
53
54 /*!
55 * @brief if a reset has been commanded and executed, SW reset or otherwise,
this function will be active.
56 * The function will lower the reset flag and send an ACK with the current
UNIX time.
57 * @return 0 on success
58 * Error code according to <hal/errors.h>
59 */
60 int WakeupFromResetCMD();
61
62 /*!
63 * @brief Calls the relevant functions in a serial order
64 */
65 void Maintenance();
66
67 /*
68 * Update most current sat time to FRAM for every time we have reset to have it.
69 */
70 void MostCurrentTimeToFRAM();
71
72 /*
73 * Check if we need to kill the payload (turn him off for good)
74 */
75 void NeedToKillPayload();
76
77 #endif /* MAINTENANCE_H_ */
78

```

```

1 /*
2  * Maintenance.c
3  *
4  * Created on: 12 8 2024
5  *   Author: maayan
6  */
7
8  #include "Maintenance.h"
9  #include <hal/Timing/Time.h>
10 #include <hal/Timing/WatchDogTimer.h>
11 #include <hcc/api_fat.h>
12 #include "SubSystemModules/Communication/AckHandler.h"
13
14 #include "utils.h"
15 #include <String.h>
16
17 Boolean flag = FALSE;
18
19 /*!
20  * @brief checks if the period time has passed
21  * @param[in] prev_exec_time last sample time (last execution time)
22  * @param[in] period period of execution time
23  * @return TRUE if difference between now and last execution time is longer than
24  'period'
25  * FALSE otherwise
26 */
27 Boolean CheckExecutionTime(time_unix prev_time, unsigned int period)
28 {
29     unsigned int timeNow;
30     int error = logError(Time_getUnixEpoch(&timeNow), "CheckExecutionTime -
31 Time_getUnixEpoch"); // Get time now
32     if(error) // if error return FALSE
33         return FALSE;
34     if(timeNow - prev_time >= period) // if the time needed pass then return TRUE.
35
36         return TRUE;
37     return FALSE;
38 }
39
40 /*!
41  * @brief checks if there is a memory corruption in the file system.
42  * @return TRUE if is corrupted.
43  * FALSE if no corruption (yay!)
44  * we do not use this function anywhere in the code.
45 */
46 Boolean IsFS_Corrupted()

```

```

43 {
44     F_SPACE space;
45     int ret = logError(f_getfreespace(f_getdrive(), &space), "IsFS_Corrupted -
f_getfreespace");
46     if(ret) return TRUE;
47     if(space.bad) return TRUE;
48     return FALSE;
49 }
50
51     /*!
52 * @brief kick the ground station communication WDT because communication took
place.
53 */
54     void KickGroundCommWDT()
55 {
56     time_unix timeNow;
57     if(logError(Time_getUnixEpoch((unsigned int*)&timeNow),
"KickGroundCommWDT - Time_getUnixEpoch")) return; // continue if there is no error
58     FRAM_writeAndVerify((unsigned char*)&timeNow, LAST_COMM_TIME_ADDR,
LAST_COMM_TIME_SIZE); // change last communication time.
59 }
60
61     /*!
62 * @brief    Checks if last GS communication time has exceeded its maximum
allowed time.
63 * @see      NO_COMMUNICATION_WDT_KICK_TIME
64 * @return    TRUE if a comm reset is needed- no communication for a long time
65 *           FALSE no need for a reset. last communication is within range
66 */
67 Boolean IsGroundCommunicationWDTReset()
68 {
69     time_unix lastComm = 0; // get time of last communicate.
70     if(logError(FRAM_read((unsigned char*)&lastComm,
LAST_COMM_TIME_ADDR, LAST_COMM_TIME_SIZE),
"IsGroundCommunicationWDTReset - FRAM_read")) return FALSE;
71     unsigned int WDTime = 0; // get period for WDT
72     if(logError(FRAM_read((unsigned char*)&WDTime,
NO_COMM_WDT_KICK_TIME_ADDR, NO_COMM_WDT_KICK_TIME_SIZE),
"IsGroundCommunicationWDTReset - FRAM_read")) return FALSE;
73     return CheckExecutionTime(lastComm, WDTime); // did the time pass? do we
need to reset?
74 }
75

```

```

76     /*!
77  * @brief   if a reset has been commanded and executed, SW reset or otherwise,
this function will be active.
78  *         The function will lower the reset flag and send an ACK with the current
UNIX time.
79  * @return   0 on success
80  *         Error code according to <hal/errors.h>
81  */
82     int WakeupFromResetCMD()
83 {
84     time_unix time = 0;
85     logError(Time_getUnixEpoch((unsigned int*)&time), "WakeupFromResetCMD -
Time_getUnixEpoch");
86
87     // general number of resets.
88     int reset = 0;
89     logError(FRAM_read((unsigned char*)&reset, NUMBER_OF_RESETS_ADDR,
NUMBER_OF_RESETS_SIZE), "WakeupFromResetCMD - FRAM_read");
90     reset += 1;
91     logError(FRAM_writeAndVerify((unsigned char*)&reset,
NUMBER_OF_RESETS_ADDR, NUMBER_OF_RESETS_SIZE),
"WakeupFromResetCMD - FRAM_writeAndVerify");
92
93     // check if we reset because of CMD reset.

94     Boolean flagCMDReset;
95     logError(FRAM_read((unsigned char*)&flagCMDReset,
RESET_CMD_FLAG_ADDR, RESET_CMD_FLAG_SIZE), "WakeupFromResetCMD -
FRAM_read");
96     if(!flagCMDReset) return SendAckPacket(ACK_RESET_WAKEUP , NULL, (
unsigned char*)&time, sizeof(time)); // if not CMD reset
97
98     // if CMD reset. (change flag back to false and add to counter.)

99     flagCMDReset = FALSE;
100     logError(FRAM_writeAndVerify((unsigned char*)&flagCMDReset,
RESET_CMD_FLAG_ADDR, RESET_CMD_FLAG_SIZE), "WakeupFromResetCMD -
FRAM_writeAndVerify");
101
102     int cmdReset = 0;
103     logError(FRAM_read((unsigned char*)&cmdReset,
NUMBER_OF_CMD_RESETS_ADDR, NUMBER_OF_CMD_RESETS_SIZE),
"WakeupFromResetCMD - FRAM_read");
104     cmdReset += 1;
105     logError(FRAM_writeAndVerify((unsigned char*)&cmdReset,
NUMBER_OF_CMD_RESETS_ADDR, NUMBER_OF_CMD_RESETS_SIZE),

```

```

"WakeupFromResetCMD - FRAM_writeAndVerify");
106
107 return SendAckPacket(ACK_RESET_WAKEUP , NULL, (unsigned char*)&time,
sizeof(time));
108
109 }
110
111 /*
112 * Update most current sat time to FRAM for every time we have reset to have it.
113 */
114 void MostCurrentTimeToFRAM()
115 {
116     time_unix timeNow = 0;
117     if(logError(Time_getUnixEpoch((unsigned int*)&timeNow),
"WMostCurrentTimeToFRAM - Time_getUnixEpoch")) return; // get time and if there is no
error continue.
118     logError(FRAM_writeAndVerify((unsigned char*)&timeNow,
MOST_UPDATED_SAT_TIME_ADDR, MOST_UPDATED_SAT_TIME_SIZE),
"WMostCurrentTimeToFRAM - FRAM_writeAndVerify"); // save in FRAM
119 }
120
121 /*
122 * Delete old files according to how much space we have left. (20% for start delete
until we have 25% free)
123 * MIN_FREE_SPACE_PERCENTAGE - the minimum free space in bytes we want
to keep in the SD in all times.
124 * If free_space < minFreeSpace we start deleting old TLM files
125 */
126 void DeleteOldFiles()
127 {
128     F_SPACE space;
129     int sd = f_getdrive();
130     if(sd != 0 && sd != 1)
131     {
132         logError(sd, "Maintenance - f_getdrive");
133         return;
134     }
135     if(logError(f_getfreespace(sd, &space), "Maintenance - f_getfreespace")) return;
//gets the variables to the struct
136     if(space.free >= ((space.total * MIN_FREE_SPACE_PERCENTAGE) / 100))
return; //if we only left 20% free
137     F_FIND find; // we delete until we have 25% free

138     f_findfirst("*.*", &find); // get the oldest file.
139     do
140     {

```



```

141     f_delete(find.filename); //delete
142     if(logError(f_getfreespace(sd, &space), "Maintenance - f_getfreespace"))
return; //gets the variables to the struct
143
144 }
145 while((space.free < (space.total *
MIN_FREE_SPACE_PERCENTAGE_TO_DELETE) / 100) && !f_findnext(&find));
146 // the while check if we still have more then 25% space used and we have more
files to delete we get the next one after the oldest.

147
148 }
149
150 /*
151 * Check if we need to kill the payload (turn him off for good)
152 */
153 void NeedToKillPayload()
154 {
155     if(flag == TRUE) return;
156     Boolean false = FALSE;
157     supervisor_housekeeping_t mySupervisor_housekeeping_hk; //create a variable
that is the struct we need from supervisor
158     int err =
logError(Supervisor_getHousekeeping(&mySupervisor_housekeeping_hk,
SUPERVISOR_SPI_INDEX), "NeedToKillPayload - Supervisor_getHousekeeping"); //
gets the variables to the struct and also check error.

159     if(err) return;
160     if(mySupervisor_housekeeping_hk.fields.iobcUptime / portTICK_RATE_MS > 60
) //check if we are past the time of reset
161     {
162         logError(FRAM_writeAndVerify((unsigned char*)&>false,
HAD_RESET_IN_A_MINUTE_ADDR, HAD_RESET_IN_A_MINUTE_SIZE),
"NeedToKillPayload - FRAM_writeAndVerify"); // say that we didn't reset twice in less
then a minute
163         flag = TRUE; // won't get here again. (in the first line we will leave function.

164     }
165 }
166
167
168 /*!
169 * @brief Calls the relevant functions in a serial order
170 */
171 void Maintenance()
172 {

```

```

173 MostCurrentTimeToFRAM();
174
175 DeleteOldFiles();
176
177 NeedToKillPayload();
178
179 if(IsGroundCommunicationWDTReset())
180 {
181     KickGroundCommWDT();
182     Boolean true = TRUE;
183     // In case of a Ground Communication WatchDog is active turn off payload
until turning him on from the ground.
184     logError(FRAM_writeAndVerify((unsigned char*)&true,
PAYLOAD_IS_DEAD_ADDR, PAYLOAD_IS_DEAD_SIZE), "Maintenance -
FRAM_writeAndVerify");
185     logError(payloadTurnOff(), "Maintenance - payloadTurnOff");
186     // Because it is happening to try to fix the TRXVU we reset the two parts (rx,
tx).
187     logError(isis_vu_e__reset_hw_rx(0), "Maintenance - isis_vu_e__reset_hw_rx"
);
188     logError(isis_vu_e__reset_hw_tx(0), "Maintenance - isis_vu_e__reset_hw_tx"
);
189     // Finally we are doing an hard reset to sat, in case of other problem.

190     isimepsv2_ivid7_piu__replyheader_t replyheader;
191     logError(isimepsv2_ivid7_piu__reset(0, &replyheader), "Maintenance -
isimepsv2_ivid7_piu__reset");
192     vTaskDelay(5000);
193 }
194 }
195

```

SubSystemModules/ Communication

```

1
2  #ifndef SPL_H_
3  #define SPL_H_
4
5  typedef enum __attribute__((__packed__)) spl_command_type_t {
6  trxvu_cmd_type,
7  eps_cmd_type,
8  filesystem_cmd_type, //2
9  managment_cmd_type,
10 payload_cmd_type,
11 ack_type, // 5
12 dump_type
13 }spl_command_type;
14
15
16  typedef enum __attribute__((__packed__)) ack_subtype_t
17 {
18  ACK_RECEIVE_COMM = 0x00,          // when receive any packet
19
20  ACK_COMD_EXEC    = 0x01,          // when executed any packet / SPL
command
21  ACK_RESET_WAKEUP = 0x7F,          // after waking up from reset
22
23  ACK_FS_RESET = 0x70,              // before going to filesystem reset
24
25  ACK_PAYLOAD_SOFT_RESET = 0x71,      // before going to payload reset
26
27  ACK_PAYLOAD_HARD_RESET = 0x72,
28  ACK_TX_HARD_RESET = 0x73,          // before going to trxvu reset
29
30  ACK_RX_HARD_RESET = 0x74,          // before going to trxvu reset
31
32  ACK_ANTs_RESET = 0x75,             // before reseting antennas
33
34  ACK_EPS_RESET = 0x80,              // before going to eps reset
35
36  ACK_SOFT_RESET = 0x81,             // before going to reset
37
38  ACK_HARD_RESET = 0x82,             // before starting hardware reset
39
40  ACK_FRAM_RESET = 0x83,
41
42  ACK_UPDATE_TIME = 0x12,            // after updating time

```

```
35
36 ACK_SET_NEW_TLM_PERIOD = 0x85,
37
38 ACK_UPDATE_BEACON_INTERVAL = 0x14,
39
40 ACK_UPDATE_RSSI_VALUE = 0x18,
41 ACK_UPDATE_EPS_VOLTAGES = 0x16,
42 ACK_UPDATE_EPS_ALPHA = 0x17,
43 ACK_EPS_RESET_WDT = 0x15,
44
45 ACK_MUTE = 0x8D,
46 ACK_UNMUTE = 0x8E,
47 ACK_ALLOW_TRANSPONDER = 0x8F,
48 ACK_TRANSPONDER_OFF = 0x9F,
49 ACK_IDLE_ON = 0xA1,
50 ACK_IDLE_OFF = 0xA2,
51
52
53 ACK_GET_ALPHA = 0x40,
54 ACK_GET_EPS_THRESHOLDS = 0x41,
55 ACK_GET_STATE = 0x42,
56 ACK_GET_HEATER_THRESHOLD = 0x43,
57 ACK_GET_FREE_SPACE = 0x44,
58 ACK_GET_TLM_SAVE_PERIODS = 0x45,
59 ACK_GET_SAT_TIME = 0x46,
60 ACK_GET_SAT_UPTIME = 0x47,
61 ACK_GET_GROUND_WDT = 0x48,
62 ACK_GET_RSSI_VALUE = 0x49,
63 ACK_GET_BEACON_INTERVAL = 0x4A,
64 ACK_GET_TX_UPTIME = 0x4B,
65 ACK_GET_RX_UPTIME = 0x4C,
66 ACK_GET_ANTTS_UPTIME = 0x4D,
67
68
69 ACK_DUMP_START = 0x90,
70 ACK_DUMP_FINISHED = 0x92,
71
72 ACK_FINISH_FIRST_ACTIVE = 0x35,
73
74 ACK_SWITCHING_SD_CARD = 0x30,
75
76 ACK_DELETE_TLM = 0xC0,           // after deleting TLM file(s)
77
78 ACK_FS_LAST_ERROR = 0xC1,
79 ACK_PING = 0xAA,
```

```

80  ACK_SET_GWDT = 0xA5,
81
82  ACK_KILL_PAYLOAD = 0x50,
83  ACK_TURN_ON_PAYLOAD = 0x51,
84
85  ACK_CHANGE_HEATER_THRESHOLDS = 0x55,
86
87  ACK_READY_FOR_TAKEOF = 0xFB,
88
89  ACK_UNKNOWN_SUBTYPE = 0xBB,           //when the given subtype is
unknown
90  ACK_UNKNOWN_TYPE = 0xCC,           //when the given type is unknown

91  ACK_ERROR_MSG = 0xFF           // send this ACK when error has
occurred
92 }ack_subtype_t;
93
94
95  typedef enum __attribute__((__packed__)) trxvu_subtypes_t
96 {
97  BEACON_SUBTYPE = 0x01, //0b00000001 =
98  MUTE_TRXVU = 0x02, //0b00010001 *
99  UNMUTE_TRXVU = 0x03, //0b10001000 *
100  ON_IDLE = 0x09, // *
101  OFF_IDLE = 0x0A,
102  GET_BEACON_INTERVAL = 0x06, //0b00100011 =
103  SET_BEACON_INTERVAL = 0x04, //0b00100100 *
104  SET_BEACON_INTERVAL_DEFAULT = 0x05, //0b00100100 *
105  GET_TX_UPTIME = 0x07, //0b01100110 = //long int

106  GET_RX_UPTIME = 0x08, //0b01101000 = same func as the TX
uptime //long int
107  SET_RSSI_TRANSPONDER = 0x0D, //0b10100101
108  GET_RSSI_TRANSPONDER = 0x14, //0b10100101
109  SET_RSSI_TRANSPONDER_DEFAULT = 0x0E, //0b10100101
110  SET_ON_TRANSPONDER = 0x0B, //0b10100110
111  SET_OFF_TRANSPONDER = 0x0C, //0b10100110
112  ANT_GET_UPTIME = 0x10, //0b10110011 = //long int

113  PING = 0x13,
114  OFF_ANTS_DEPLOY = 0x15
115
116 }trxvu_subtypes_t;
117
118
119  typedef enum __attribute__((__packed__)) eps_subtypes_t

```

```

120 {
121     RESET_EPS_WDT            = 0x01,
122     UPDATE_ALPHA            = 0x02,
123     UPDATE_ALPHA_DEFAULT    = 0x03,
124     GET_ALPHA               = 0x04,
125     UPDATE_THRESHOLD        = 0x05,
126     UPDATE_THRESHOLD_DEFAULT = 0x06,
127     GET_THRESHOLD           = 0x07,
128     GET_HEATER_VALUES       = 0x0C,
129     SET_HEATER_VALUES       = 0x0D,
130     GET_STATE               = 0x0E,
131
132 }eps_subtypes_t;
133
134
135     typedef enum __attribute__((__packed__)) filesystem_subtypes_t
136 {
137     DELETE_ALL_FILES        = 0x01,
138     DELETE_TLM              = 0xA1,
139     DELETE_TLM_BY_TYPE      = 0xB1,
140     GET_LAST_FS_ERROR       = 0xBB,
141     SET_TLM_PERIOD          = 0xCC,
142     SET_DEFAULT_TLM_PERIOD  = 0xCE,
143     GET_TLM_PERIOD          = 0xCD,
144     START_DUMP_BY_DAYS      = 0x0E,
145     GET_FREE_SPACE          = 0x0B,
146     SWITCH_SD_CARD          = 0x0A,
147     ABORT_DUMP              = 0x1E
148 }filesystem_subtypes_t;
149
150
151     typedef enum __attribute__((__packed__)) management_subtypes_t
152 {
153     GET_PERIOD_GROUND_WDT    = 0x01,
154     SET_PERIOD_GROUND_WDT    = 0x02,
155     UPDATE_SAT_TIME          = 0xDD,    //0b -11011101 ,
156     GENERIC_I2C_CMD          = 0xDF,    //0b -11011111 ,
157     RESET_COMPONENT          = 0xDE,    //0b -11011110
158     GET_SAT_UPTIME           = 0xE3,
159     GET_SAT_TIME             = 0xE4,
160     SET_FLAGS_TO_FIRST       = 0xFF
161 }management_subtypes_t;
162     //-----
163

```

```
164
165     typedef enum __attribute__((__packed__)) payload_subtypes_t
166 {
167     TURN_ON_KILL_PAYLOAD = 0x11,
168     TURN_OFF_KILL_PAYLOAD = 0x20
169 }payload_subtypes_t;
170 #endif /* SPL_H_ */
171
172
```



```

1
2  #ifndef SATCOMMANDS_H_
3  #define SATCOMMANDS_H_
4
5  #include "GlobalStandards.h"
6
7  #define MAX_COMMAND_DATA_LENGTH 200 ///< maximum AX25 data field
available for downlink
8  #define IMG_CHUNK_SIZE 50 //190 // leave room for chunk number and more
9
10 #define FRAM_MAX_ADDRESS 539216400
11
12  //!< how many command can be saved in the buffer
13  #define MAX_NUM_OF_DELAYED_CMD (100)
14  #define CUBE_SAT_ID    14
15  #define ALL_SAT_ID     10
16
17
18  typedef enum __attribute__((__packed__)) CMD_ERR{
19  command_success = 0,          ///< a successful operation. no errors
20  command_found = 0,           ///< a command was found
21  no_command_found ,           ///< no commands were found in command
buffers
22  index_out_of_bound,          ///< index out of bound error
23  null_pointer_error,          ///< input parameter pointer is null
24  execution_error,             ///< an execution error has occurred
25  invalid_sat_id               ///<
26 }CMD_ERR;
27
28  typedef struct __attribute__((__packed__)) sat_packet_t
29 {
30  unsigned int ID;
31  char cmd_type;
32  char cmd_subtype;
33  unsigned short length;
34  unsigned char data[MAX_COMMAND_DATA_LENGTH];
35

```

```

36 }sat_packet_t;
37
38  /*
39  * Change the packet we get to sat_packet_t struct
40  *
41  * @param[in] name= data; type= unsigned char *; The packet we get after reading
  from the frame (without the headers of the Ax25 protocol).
42  * @param[out] name= cmd; type= sat_packet_t *; The struct we put all the info we
  get from the packet.
43  * @return error according to CMD_ERR
44  *
45  */
46 CMD_ERR ParseDataToCommand(unsigned char * data, sat_packet_t *cmd);
47
48  /*
49  * Create a sat_packet_t struct that in the end the sat send to us
50  *
51  * @param[in] name= data; type= unsigned char *; The data part in the struct
52  * @param[in] name= data_length; type= unsigned short; The length of the data that
  going to in the struct. Also it will be in the length part in the struct
53  * @param[in] name= type; type= char; The cmd_type part in the struct. Says which
  subsystem its from.
54  * @param[in] name= subtype; type= char; The cmd_subtype part in the struct. Says
  which commend of the subsystem its from.
55  * @param[in] name= id; type= unsigned int; The ID part in the struct. Have the id of
  the sat and some other things.
56  * @param[out] name= cmd; type= sat_packet_t *; The struct we put all the info we
  get from the params.
57  * @return error according to CMD_ERR
58  *
59  */
60 CMD_ERR AssembleCommand(unsigned char *data, unsigned short data_length,
  char type, char subtype, unsigned int id, sat_packet_t *cmd);
61
62  /*
63  * According to the type in the cmd struct goes to the right type and there do the
  commend.
64  *
65  * @param[out] name= cmd; type= sat_packet_t *; The packet as sat_packet_t struct.
66  * @note if type not exist an unknown tpye ack is send.
67  * @return error according to <hal/errors.h>
68  *
69  */
70  int ActUponCommand(sat_packet_t *cmd);
71
72  #endif /* SATCOMMANDS_H_ */

```



```

1 /*
2  * SatCommandHandler.c
3  *
4  * Created on: 26 6 2024
5  * Author: Maayan
6  */
7
8  #include <hal/errors.h>
9
10  #include "SatCommandHandler.h"
11  #include "CommandDictionary.h"
12  #include "AckHandler.h"
13
14  #include "SPL.h"
15  #include "utils.h"
16
17  #include <string.h>
18
19  /*
20  * Change the packet we get to sat_packet_t struct
21  *
22  * @param[in] name= data; type= unsigned char *; The packet we get after reading
  from the frame (without the headers of the Ax25 protocol).
23  * @param[out] name= cmd; type= sat_packet_t *; The struct we put all the info we
  get from the packet.
24  * @return error according to CMD_ERR
25  *
26  */
27  CMD_ERR ParseDataToCommand(unsigned char * data, sat_packet_t *cmd)
28  {
29      if(data == NULL || cmd == NULL) // check none of the pointers is null
30
31          return null_pointer_error;
32
33      int plusPlace = 0; // get a counter for the start place of the data we read
34
35      if(memcpy(&cmd->ID, data, sizeof(cmd->ID)) == NULL) // check the memcpy
  worked and put in the cmd->ID the id of the sat and the other stuff there
36
37          return null_pointer_error;
38      plusPlace += sizeof(cmd->ID); // add the size of the ID to the variable
39
40      if(cmd->ID>>24 != CUBE_SAT_ID && cmd->ID>>24 != ALL_SAT_ID) // check we
  really are the sat it was send to.
41
42          return invalid_sat_id;
43  }

```

```

39
40 if(memcpy(&cmd->cmd_type, data + plusPlace, sizeof(cmd->cmd_type)) ==
NULL) // same just the cmd_type.
41     return null_pointer_error;
42 plusPlace += sizeof(cmd->cmd_type);
43
44 if(memcpy(&cmd->cmd_subtype, data + plusPlace, sizeof(cmd->cmd_subtype))
== NULL) // same just the cmd_subtype.
45     return null_pointer_error;
46 plusPlace += sizeof(cmd->cmd_subtype);
47
48 if(memcpy(&cmd->length, data + plusPlace, sizeof(cmd->length)) == NULL) //
same just the length.
49     return null_pointer_error;
50 plusPlace += sizeof(cmd->length);
51
52 if(cmd->length > MAX_COMMAND_DATA_LENGTH)
53     return index_out_of_bound;
54
55 if(cmd->length != 0){
56     if(memcpy(&cmd->data, data + plusPlace, cmd->length) == NULL) // same just
to the data which is the length that equal to the value of the variable length.
57         return null_pointer_error;
58 }
59
60 return command_success;
61 }
62
63 /*
64 * Create a sat_packet_t struct that in the end the sat send to us
65 *
66 * @param[in] name= data; type= unsigned char *; The data part in the struct
67 * @param[in] name= data_length; type= unsigned short; The length of the data that
going to in the struct. Also it will be in the length part in the struct
68 * @param[in] name= type; type= char; The cmd_type part in the struct. Says which
subsystem its from.
69 * @param[in] name= subtype; type= char; The cmd_subtype part in the struct.
Says which commend of the subsystem its from.
70 * @param[in] name= id; type= unsigned int; The ID part in the struct. Have the id
of the sat and some other things.
71 * @param[out] name= cmd; type= sat_packet_t *; The struct we put all the info we
get from the params.
72 * @return error according to CMD_ERR
73 *
74 */

```

```

75 CMD_ERR AssembleCommand(unsigned char *data, unsigned short data_length,
char type, char subtype, unsigned int id, sat_packet_t *cmd)
76 {
77     if(cmd == NULL)
78         return null_pointer_error;
79
80     cmd->ID = id;
81     cmd->cmd_type = type;
82     cmd->cmd_subtype = subtype;
83     cmd->length = data_length;
84     if(data_length != 0)
85     {
86         if(data == NULL)
87             return null_pointer_error;
88         else
89         {
90             if(data_length > MAX_COMMAND_DATA_LENGTH) // Check the
data_length is not larger then the max length available
91                 return index_out_of_bound;
92
93             memcpy(&cmd->data, data, data_length);
94         }
95     }
96
97     return command_success;
98 }
99
100 /*
101 * According to the type in the cmd struct goes to the right type and there do the
commend.
102 *
103 * @param[out] name= cmd; type= sat_packet_t *; The packet as sat_packet_t
struct.
104 * @note if type not exist an unknown tpye ack is send.
105 * @return error according to <hal/errors.h>
106 *
107 */
108 int ActUponCommand(sat_packet_t *cmd)
109 {
110     if(cmd == NULL)
111         return null_pointer_error;
112     // Go to each type known and according to spl_command_type struct and check
if it's equal to the type in the cmd. if equal go the right router
113     switch(cmd->cmd_type)

```

```

114 {
115     case trxvu_cmd_type:
116         return trxvu_command_router(cmd);
117     case eps_cmd_type:
118         return eps_command_router(cmd);
119     case filesystem_cmd_type:
120         return filesystem_command_router(cmd);
121     case managment_cmd_type:
122         return managment_command_router(cmd);
123     case payload_cmd_type:
124         return payload_command_router(cmd);
125     default:
126     {
127         return SendAckPacket(ACK_UNKNOWN_TYPE, cmd, NULL, 0); // Send
ack that says what written in unknownType_msg
128     }
129 }
130 }
131

```

```

1
2  #ifndef ACKHANDLER_H_
3  #define ACKHANDLER_H_
4  #include "SubSystemModules/Communication/AckErrors.h"

5  #include <hal/errors.h>
6  #include "SatCommandHandler.h"
7  #include "SPL.h"
8  #include "utils.h"
9  #include <freertos/FreeRTOS.h>
10 #include "AckErrors.h"
11
12
13  #define SendAnonymosAck(acksubtype)
SendAckPacket(acksubtype,NULL,NULL,0) //<! sends an ACK without command ID nor
data
14
15
16  /*!
17  * @brief transmits an ack packet as an SPL packet(using the SPL protocol)
18  * @param[in] name=acksubtype; type=ack_subtype_t; type of ack to be sent
according to ack_subtype_t enumeration
19  * @param[in] name=cmd; type=sat_packet_t*; the command for which the ACK is a
response to. CAN BE NULL!
20  * @param[in] name=data; type=unsigned char*; the data we want to send with the
ack. CAN BE NULL!
21  * @param[in] name=length; type=unsigned short; the length of the data we want to
send in bytes
22  * @return errors according t <hal/errors.h>
23  */
24  int SendAckPacket(ack_subtype_t acksubtype, sat_packet_t *cmd, unsigned char
*data, unsigned short length);
25
26
27  #endif /* ACKHANDLER_H_ */
28

```



```

1 /*
2  * AckHandler.c
3  *
4  * Created on: 23 7 2024
5  * Author: maayan
6  */
7
8 #include "AckHandler.h"
9 #include "TRXVU.h"
10
11 /*!
12  * @brief transmits an ack packet as an SPL packet(using the SPL protocol)
13  * @param[in] name=acksubtype; type=ack_subtype_t; type of ack to be sent
14  * according to ack_subtype_t enumeration
15  * @param[in] name=cmd; type=sat_packet_t*; the command for which the ACK is a
16  * response to. CAN BE NULL!
17  * @param[in] name=data; type=unsigned char*; the data we want to send with the
18  * ack. CAN BE NULL!
19  * @param[in] name=length; type=unsigned short; the length of the data we want to
20  * send in bytes
21  * @return errors according t <hal/errors.h>
22  */
23 int SendAckPacket(ack_subtype_t acksubtype, sat_packet_t *cmd, unsigned char
24 *data, unsigned short length)
25 {
26     sat_packet_t cmd1;
27     if(cmd != NULL)
28         logError(AssembleCommand(data, length, ack_type, acksubtype, cmd->ID,
29 &cmd1), "SendAckPacket - AssembleCommand"); // Assemble new cmd1 struct
30 according to data, length and cmd
31     else
32     {
33         unsigned int id = CUBE_SAT_ID;
34         id = id<<24;
35         logError(AssembleCommand(data, length, ack_type, acksubtype, id, &cmd1),
36 "SendAckPacket - AssembleCommand"); // Assemble new cmd1 struct according to
37 data, length. cmd is null.
38     }
39     int avalFrames;
40
41     return logError(TransmitSplPacket(&cmd1, &avalFrames), "SendAckPacket -
42 TransmitSplPacket"); // Send ack and return error if have else 0
43 }
44

```

```

1 /*
2  * AckErrors.h
3  *
4  * Created on: 17 9 2024
5  * Author: maayan
6  */
7
8 #ifndef ACKERRORS_H_
9 #define ACKERRORS_H_
10
11 #define ERROR_READ_FROM_FRAM 1
12
13 #define ERROR_WRITE_TO_FRAM 2
14
15 #define ERROR_WRITTEN_IN_FRAM_WRONG 3
16
17 #define ERROR_WRITE_TO_I2C 10
18
19 #define ERROR_CANT_GET_TIME 15
20
21 #define ERROR_WRONG_LENGTH_DATA 17 //the packet sent wrong
length of the data field in data
22
23 #define ERROR_CANT_SET_END_TIME 20
24
25 #define ERROR_NOT_VALID_SIDE 23
26
27 #define ERROR_GET_UPTIME 24
28
29 #define ERROR_CANT_RESET_WDT 26
30
31 #define ERROR_GET_FROM_STRUCT 27
32
33 #define ERROR_INVALID_ALPHA 28
34
35 #define ERROR_INVALID_TRESHOLD 29
36
37 #define ERROR_CANT_DO 30 //can't do what we want it to

```

```
35
36 #define ERROR_CANT_RESET          40

37
38 #define ERROR_COULDNT_UPDATE_SAT_TIME  50

39
40 #define ERROR_COULDNT_DELETE_ALL      60

41
42 #define ERROR_NOT_VALID_SD           69

43 #define ERROR_SAME_SD_WE_USE         70

44
45 #define UNKNOWN_ERROR                255

46 #endif /* ACKERRORS_H_ */
47
```

```

1
2  #ifndef COMMANDDICTIONARY_H_
3  #define COMMANDDICTIONARY_H_
4
5  #include "SPL.h"
6  #include "SatCommandHandler.h"
7
8
9  /*!
10 * @brief routes the data into the appropriate trxvu command according to the
    command sub type
11 * @param[in] name=cmd; type=sat_packet_t*: command pertaining to the TRXVU
    system, to be executed.
12 * @note if subtype not exist an unknown subtype ack is send.
13 * @return errors according to <hal/errors.h> and -1 on cmd NULL
14 */
15     int trxvu_command_router(sat_packet_t *cmd);
16
17     /*!
18 * @brief routes the data into the appropriate eps command according to the
    command sub type
19 * @param[in] cmd command pertaining to the EPS system, to be executed.
20 * @note the type and subtype of the command are already inside cmd
21 * @see sat_packet_t structure
22 * @return errors according to <hal/errors.h>
23 */
24     int eps_command_router(sat_packet_t *cmd);
25
26     /*!
27 * @brief routes the data into the appropriate file-system command according to the
    command sub type
28 * @param[in] cmd command pertaining to the FS system, to be executed.
29 * @note the type and subtype of the command are already inside cmd
30 * @see sat_packet_t structure
31 * @return errors according to <hal/errors.h>
32 */
33     int filesystem_command_router(sat_packet_t *cmd);
34
35     /*!
36 * @brief routes the data into the appropriate obc command according to the
    command sub type
37 * @param[in] cmd command pertaining to the MANAGMENT sub routine, to be
    executed.
38 * @note the type and subtype of the command are already inside cmd
39 * @see sat_packet_t structure
40 * @return errors according to <hal/errors.h>

```

```
41 */
42 int managment_command_router(sat_packet_t *cmd);
43
44 /*!
45 * @brief routes the data into the appropriate payload command according to the
46 * command sub type
47 * @param[in] cmd command pertaining to the payload sub routine, to be executed.
48 * @note the type and subtype of the command are already inside cmd
49 * @see sat_packet_t structure
50 * @return errors according to <hal/errors.h>
51 */
52 int payload_command_router(sat_packet_t *cmd);
53 #endif /* COMMANDDICTIONARY_H_ */
54
```

```

1 /*
2  * CommandDictionary.c
3  *
4  * Created on: 9 7 2024
5  * Author: Maayan
6  */
7
8  #include "SubsystemCommands/TRXVU_Commands.h"
9
10 #include "SubsystemCommands/EPs_Commands.h"
11
12 #include "SubsystemCommands/Maintenance_Commands.h"
13
14 #include "SubsystemCommands/FS_Commands.h"
15
16 #include "SubsystemCommands/Payload_Commands.h"
17
18 #include "CommandDictionary.h"
19 #include <stdio.h>
20
21 /*!
22  * @brief routes the data into the appropriate trxvu command according to the
23  * command sub type
24  * @param[in] name=cmd; type=sat_packet_t*: command pertaining to the TRXVU
25  * system, to be executed.
26  * @note if subtype not exist an unknown subtype ack is send.
27  * @return errors according to <hal/errors.h> and -1 on cmd NULL
28  */
29 int trxvu_command_router(sat_packet_t *cmd)
30 {
31     if(cmd == NULL)
32     {
33         printf("cmd_is_null\n");
34         return -1;
35     }
36     // Go to each subtype known and according to trxvu_subtypes_t struct and check
37     // if it's equal to the subtype in the cmd. if equal go the function of the command
38
39     switch(cmd->cmd_subtype)
40     {
41         case ON_IDLE:
42             return CMD_SetOn_IdleState(cmd);
43         case OFF_IDLE:
44             return CMD_SetOff_IdleState(cmd);
45         case SET_ON_TRANSPONDER:
46             return CMD_SetOn_Transponder(cmd);
47     }
48 }

```

```

38     case SET_OFF_TRANSPONDER:
39         return CMD_SetOff_Transponder(cmd);
40     case SET_RSSI_TRANSPONDER:
41         return CMD_SetRSSI_Transponder(cmd);
42     case SET_RSSI_TRANSPONDER_DEFAULT:
43         return CMD_SetRSSI_Transponder_DEFAULT(cmd);
44     case GET_RSSI_TRANSPONDER:
45         return CMD_GetRSSI_Transponder(cmd);
46     case MUTE_TRXVU:
47         return CMD_MuteTRXVU(cmd);
48     case UNMUTE_TRXVU:
49         return CMD_UnMuteTRXVU(cmd);
50     case SET_BEACON_INTERVAL:
51         return CMD_SetBeacon_Interval(cmd);
52     case SET_BEACON_INTERVAL_DEFAULT:
53         return CMD_SetBeacon_Interval_DEFAULT(cmd);
54     case GET_BEACON_INTERVAL:
55         return CMD_GetBeacon_Interval(cmd);
56     case GET_TX_UPTIME:
57         return CMD_GetTxUptime(cmd);
58     case GET_RX_UPTIME:
59         return CMD_GetRxUptime(cmd);
60     case ANT_GET_UPTIME:
61         return CMD_AntGetUptime(cmd);
62     case PING:
63         return CMD_Ping(cmd);
64     case OFF_ANTS_DEPLOY:
65         return CMD_OffAntsDeploy(cmd);
66     default:
67         return SendAckPacket(ACK_UNKNOWN_SUBTYPE, cmd, NULL, 0); //

```

Send ack that says what written in unknownSubtype_msg

```

68     }
69 }
70
71
72     /*!
73 * @brief routes the data into the appropriate eps command according to the
command sub type
74 * @param[in] cmd command pertaining to the EPS system, to be executed.
75 * @note the type and subtype of the command are already inside cmd
76 * @see sat_packet_t structure
77 * @return errors according to <hal/errors.h>
78 */
79     int eps_command_router(sat_packet_t *cmd)
80 {

```

```

81  if(cmd == NULL)
82  {
83      printf("cmd_is_null\r\n");
84      return -1;
85  }
86  // Go to each subtype known and according to trxvu_subtypes_t struct and check
  if it's equal to the subtype in the cmd. if equal go the function of the command

87  switch(cmd->cmd_subtype)
88  {
89      case UPDATE_ALPHA:
90          return CMD_UpdateAlpha(cmd);
91      case UPDATE_ALPHA_DEFAULT:
92          return CMD_RestoreDefaultAlpha(cmd);
93      case GET_ALPHA:
94          return CMD_GetAlpha(cmd);
95      case GET_THRESHOLD:
96          return CMD_GetThresholdVoltages(cmd);
97      case UPDATE_THRESHOLD:
98          return CMD_UpdateThresholdVoltages(cmd);
99      case UPDATE_THRESHOLD_DEFAULT:
100         return CMD_RestoreDefaultThresholdVoltages(cmd);
101     case GET_STATE:
102         return CMD_GetState(cmd);
103     case RESET_EPS_WDT:
104         return CMD_EPS_ResetWDT(cmd);
105     case GET_HEATER_VALUES:
106         return CMD_GetHeaterVal(cmd);
107     case SET_HEATER_VALUES:
108         return CMD_SetHeaterVal(cmd);
109     default:
110         return SendAckPacket(ACK_UNKNOWN_SUBTYPE, cmd, NULL, 0); //
  Send ack that says what written in unknownSubtype_msg

111  }
112 }
113
114
115  /*!
116  * @brief routes the data into the appropriate file-system command according to
  the command sub type
117  * @param[in] cmd command pertaining to the FS system, to be executed.
118  * @note the type and subtype of the command are already inside cmd
119  * @see sat_packet_t structure
120  * @return errors according to <hal/errors.h>
121  */

```



```

122     int filesystem_command_router(sat_packet_t *cmd)
123 {
124     if(cmd == NULL)
125     {
126         printf("cmd_is_null\r\n");
127         return -1;
128     }
129     switch(cmd->cmd_subtype)
130     {
131         case DELETE_ALL_FILES:
132             return CMD_DeleteAllFiles(cmd);
133         case START_DUMP_BY_DAYS:
134             return CMD_StartDump(cmd);
135         case DELETE_TLM:
136             return CMD_DeleteTLM(cmd);
137         case GET_LAST_FS_ERROR:
138             return CMD_GetLastFS_Error(cmd);
139         case SET_TLM_PERIOD:
140             return CMD_SetTLMPeriodTimes(cmd);
141         case GET_TLM_PERIOD:
142             return CMD_GetTLMPeriodTimes(cmd);
143         case GET_FREE_SPACE:
144             return CMD_FreeSpace(cmd);
145         case SWITCH_SD_CARD:
146             return CMD_SwitchSD_card(cmd);
147         case ABORT_DUMP:
148             return CMD_SendDumpAbortRequest();
149         case SET_DEFAULT_TLM_PERIOD:
150             return CMD_SetTLMPeriodTimes_default(cmd);
151         case DELETE_TLM_BY_TYPE:
152             return CMD_DeleteFilesOfType(cmd);
153         default:
154             return SendAckPacket(ACK_UNKNOWN_SUBTYPE, cmd, NULL, 0); //
Send ack that says what written in unknownSubtype_msg
155     }
156 }
157
158
159     /*!
160 * @brief routes the data into the appropriate obc command according to the
command sub type
161 * @param[in] cmd command pertaining to the MANAGMENT sub routine, to be
executed.
162 * @note the type and subtype of the command are already inside cmd
163 * @see sat_packet_t structure

```

```

164 * @return errors according to <hal/errors.h>
165 */
166 int managment_command_router(sat_packet_t *cmd)
167 {
168     if(cmd == NULL)
169     {
170         printf("cmd_is_null\r\n");
171         return -1;
172     }
173     switch(cmd->cmd_subtype)
174     {
175         case GET_PERIOD_GROUND_WDT:
176             return CMD_GetGsWdtKickTime(cmd);
177         case SET_PERIOD_GROUND_WDT:
178             return CMD_SetGsWdtKickTime(cmd);
179         case GET_SAT_TIME:
180             return CMD_GetSatTime(cmd);
181         case RESET_COMPONENT:
182             return CMD_ResetComponent(cmd);
183         case GET_SAT_UPTIME:
184             return CMD_GetSatUptime(cmd);
185         case UPDATE_SAT_TIME:
186             return CMD_UpdateSatTime(cmd);
187         case SET_FLAGS_TO_FIRST:
188             CMD_SetFirstActiveFlags();
189             return 0;
190         default:
191             return SendAckPacket(ACK_UNKNOWN_SUBTYPE, cmd, NULL, 0); //
Send ack that says what written in unknownSubtype_msg
192     }
193 }
194
195 /*!
196 * @brief routes the data into the appropriate payload command according to the
command sub type
197 * @param[in] cmd command pertaining to the payload sub routine, to be executed.
198 * @note the type and subtype of the command are already inside cmd
199 * @see sat_packet_t structure
200 * @return errors according to <hal/errors.h>
201 */
202 int payload_command_router(sat_packet_t *cmd)
203 {
204     if(cmd == NULL) return -1;
205     switch(cmd->cmd_subtype)
206     {

```

```
207     case TURN_ON_KILL_PAYLOAD:
208         return CMD_TurnOnKillPayloadFlag(cmd);
209     case TURN_OFF_KILL_PAYLOAD:
210         return CMD_TurnOffKillPayloadFlag(cmd);
211     default:
212         return SendAckPacket(ACK_UNKNOWN_SUBTYPE, cmd, NULL, 0); //
Send ack that says what written in unknownSubtype_msg

213     }
214 }
215
```

```

1 #ifndef TRXVU_H_
2     #define TRXVU_H_
3
4     #include <freertos/FreeRTOS.h>
5     #include <freertos/semphr.h>
6     #include <freertos/task.h>
7
8     #include <satellite-subsystems/isis_vu_e.h>
9     #include "satellite-subsystems/isis_ants.h"
10
11     #include "FRAM_FlightParameters.h"
12     #include "GlobalStandards.h"
13     #include "AckHandler.h"
14     #include "SatCommandHandler.h"
15     #include "utils.h"
16     #include <hal/Timing/Time.h>
17     #include "SubSystemModules/Maintenance/Maintenance.h"
18
19     /*#define WE_HAVE_ANTS 0*/
20
21     #define MAX_MUTE_TIME      (60*60*24*7)    ///< max mute duration will be
7 days = 60*60*24*3 [sec]
22     #define MAX_IDLE_TIME      (2400)    ///< max IDLE duration will be 40
minutes = 120 *20 [sec]
23     #define MAX_TRANS_TIME      (60*60*24*14)    // max time of transponder - 14
days
24
25     #define SIZE_RXFRAME      200
26     #define SIZE_TXFRAME      235
27     #define BEACON_SPL_ID      0x02FFFFFF
28
29     #define MIN_TRXVU_BUFF 5
30     #define DEFAULT_RSSI_VALUE 10
31
32 xSemaphoreHandle semaphorDump;
33 xQueueHandle queueAbortDump;
34
35     typedef struct __attribute__((__packed__))
36 {
37     sat_packet_t cmd;
38     unsigned char dump_type;
39     time_unix_t start;
40     time_unix_t end; // if passed 0 we use the readTLMFiles function. Otherwise, we
use the time range function
41     int resolution;

```

```

42 } dump_arguments_t;
43
44     typedef enum __attribute__((__packed__)) _ISIStrxvuTransponderMode
45 {
46     trxvu_transponder_off = 0x01,
47     trxvu_transponder_on = 0x02
48 } ISIStrxvutransponderMode;
49
50     /*
51     * Gets Idle end time value from FRAM
52     * @return type=time_unix; 0 on fail
53     *         idle end time on success
54     */
55     time_unix getIdleEndTime();
56
57     /*
58     * Sets Mute time end value in FRAM and check
59     * @return type=int; -1 on error
60     *         -2 on wrong set
61     *         0 on success
62     */
63     int setMuteEndTime(time_unix endTime);
64
65     /*
66     * Gets Mute time end value from FRAM
67     * @return type=time_unix; 0 on fail
68     *         mute end time on success
69     */
70     time_unix getMuteEndTime();
71
72     /*
73     * Sets the new beacon interval in period.
74     * return type=int; error according to <hal/errors.h>
75     */
76     int setNewBeaconIntervalToPeriod();
77
78     /*
79     * Gets transponder end time value from FRAM
80     * @return type=time_unix; 0 on fail
81     *         Transponder end time on success
82     */
83     time_unix getTransponderEndTime();
84
85     /**
86     * Sets transponder RSSI value in FRAM and check

```

```

87 * @return type=int;  -1 on error
88 *                    -2 on wrong set
89 *                    -3 on I2C_write error
90 *                    0 on success
91 *
92 */
93 int setTransponderRSSIinFRAM(short val);
94
95 /**
96 * Gets transponder RSSI value from FRAM
97 * @return type=short; RSSI value from FRAM, -1 on error
98 */
99 short getTransponderRSSIFromFRAM();
100
101 /*!
102 * @breif Initializes data filed for transmission - semaphores, parameters from the
FRAM
103 * @return
104 */
105 void InitTxModule();
106
107 /*
108 * set in FRAM that we need to stop try to deploy
109 */
110 void SetNeedToStopAntDeploy();
111
112 /*
113 * Initialize the TRXVU and ants.
114 *
115 * @return error according to <hal/errors.h>
116 */
117 int InitTrxvuAndAnts();
118
119 /*
120 * set in the TRXVU his config param. (freq, bitrate and more)
121 */
122 void SetTRXVU_config_param();
123
124 /*
125 * Have the TRXVU logic. (Beacon send, check if have packets, read packet etc.)
126 * @return type=int; return error if have and command_success if not
127 */
128 int TRX_Logic();
129
130 /*
131 * Check if we pass the time of the transponder and if so get out of this state.

```

```

132 * return type=int; -1 time end not smaller then time now
133 *           0 on success
134 *           error according to <hal/errors.h>
135 */
136 int turnOffTransponder();
137
138 /*
139 * Check if we pass the time of the idle and if so get out of this state.
140 * return type=int; -1 on error in getIdleEndTime
141 *           0 on success
142 *           error according to SetIdleState errors
143 */
144 int turnOffIdle();
145
146 /*
147 * set transponder on
148 * return type=int; according to I2C_write error list
149 */
150 int setTransponderOn();
151
152 /*
153 * set transponder off
154 * return type=int; according to I2C_write error list
155 */
156 int setTransponderOff();
157
158 /*
159 * Sets transponder end time value from FRAM
160 * @return type=time_unix; 0 on success
161 *           error according to <hal/errors.h>
162 */
163 int setTransponderEndTime(time_unix transponderEndTime);
164
165 /*
166 * check if we are needed to abort the dump
167 * @return type=Boolean; FALSE - not needed. TRUE - abort
168 */
169 Boolean CheckDumpAbort();
170
171 /*
172 * Check if we can transmit. (according to mute and EPS condition)
173 * @return type=Boolean; TRUE if we can transmit
174 *           FALSE if we can't
175 */
176 Boolean CheckTransmitionAllowed();
177

```

```

178     /*
179  * Send Ax25 packet
180  * @param name=cmd; type=sat_packet_t*; The packet the sat got and use to find
181  * all the required information (like the headers we add)
182  * @param name=avalFrames; type=int*; availed frames
183  * @return type=int; return -1 on cmd NULL
184  *          -2 if we can't transmmmit
185  *          Error code according to <hal/errors.h>
186  */
187     int TransmitSplPacket(sat_packet_t *packet, int *avalFrames);
188
189     /*
190  * @brief transmits beacon according to beacon logic
191  * @ return 0 if everything is fine
192  *          -1 if we not suppose to send beacon now
193  *          else it return error according to the stuff that didn't work
194  */
195     int BeaconLogic();
196
197     /*
198  * @brief set the idle state of the trxvu
199  * @param[in] state ON/OFF
200  * @param[in] duration for how long will the satellite be in idle state, if state is OFF
201  * than this value is ignored
202  * @return 0 in successful
203  *          -1 in failure
204  *          -2 FRAM read problem
205  *          -3 FRAM write problem
206  *          -4 wrong time set
207  *          -5 time problem
208  *          -6 not on or off
209  */
210     int SetIdleState(isis_vu_e__onoff_t state, time_unix duration);
211
212     /*
213  * Gets number of packets in waiting.
214  * @return type=int; -1 on error
215  *          number of packets on success
216  */
217     int GetNumberOfFramesInBuffer();
218
219     /*
220  * Get commend from the buffer and divide the info according to sat_packet_t
221  * headers
222  * @param[out] name=cmd; type=sat_packet_t*; Put here the info from packet

```


according to the sat_packet_t struct.

```
221 * @return type=CMD_ERR; return command_success on success
222 *           and error according to CMD_ERR
223 */
224 CMD_ERR GetOnlineCommand(sat_packet_t *cmd);
225
226
227 /*
228 * Send Ax25 packet with output data
229 * @param name=cmd; type=sat_packet_t*; The packet the sat got and use to find
all the required information (like the headers we add)
230 * @param name=data; type=unsigned char*; The data we want to send
231 * @param name=length; type=unsigned short; length of data
232 * @return type=int; return -1 on cmd NULL
233 *           -2 on fail in Assemble command
234 *           -3 if we can't transmit
235 *           Error code according to <hal/errors.h>
236 */
237 int TransmitDataAsSPL_Packet(sat_packet_t *cmd, unsigned char *data,
unsigned short length);
238
239 void setBeaconIntervalNOT_FROM_FRAM(unsigned int interval);
240
241 #endif
242
```

```

1 /*
2  * TRXVU.c
3  *
4  * Created on: 5 6 2024
5  * Author: maayan
6  */
7
8 #include <hal/errors.h>
9 #include <string.h>
10 #include <hal/Drivers/I2C.h>
11 #include "SubSystemModules/Housekeeping/TelemetryCollector.h"
12
13 #include "InitSystem.h"
14 #include "SatCommandHandler.h"
15 #include "TRXVU.h"
16 #include "SysI2CAddr.h"
17 #include "AckHandler.h"
18 #include "SPL.h"
19
20 Boolean continueDeploy;
21 time_unix lastTimeSendingBeacon;
22 unsigned int period;
23
24 #define TX_FREQUENCY 436400
25 #define RX_FREQUENCY 145970
26 #define NEED_TO_DEPLOY(uptime, add) ((uptime + add) % (30*60)) != 0
27
28 /*
29  * set in the TRXVU his config param. (freq, bitrate and more)
30  */
31 void SetTRXVU_config_param()
32 {
33     logError(isis_vu_e__set_bitrate(0, isis_vu_e__bitrate__9600bps),
34 "SetTRXVU_config_param - isis_vu_e__set_bitrate");
35
36     logError(isis_vu_e__set_tx_freq(0, TX_FREQUENCY),
37 "SetTRXVU_config_param - isis_vu_e__set_tx_freq");
38
39     logError(isis_vu_e__set_rx_freq(0, RX_FREQUENCY),
40 "SetTRXVU_config_param - isis_vu_e__set_rx_freq");
41
42     logError(isis_vu_e__set_transponder_in_freq(0, RX_FREQUENCY),
43 "SetTRXVU_config_param - isis_vu_e__set_transponder_in_freq");
44
45     logError(isis_vu_e__set_tx_pll_powerout(0, 0xCFEF), "SetTRXVU_config_param
46 - isis_vu_e__set_tx_pll_powerout");
47 }
48
49

```

```

40  /*
41  * Initialize the TRXVU and ants.
42  *
43  * @return error according to <hal/errors.h>
44  */
45  int InitTrxvuAndAnts(){
46  // Definition of I2C and TRXVU
47  ISIS_VU_E_t myTRXVU[1];
48  driver_error_t rv;
49
50  //I2C addresses defined
51  myTRXVU[0].rxAddr = I2C_TRXVU_RC_ADDR;
52  myTRXVU[0].txAddr = I2C_TRXVU_TC_ADDR;
53
54  //Buffer definition
55  myTRXVU[0].maxSendBufferLength = SIZE_TXFRAME;
56  myTRXVU[0].maxReceiveBufferLength = SIZE_RXFRAME;
57
58  //Initialize the trxvu subsystem
59  rv = ISIS_VU_E_Init(myTRXVU, 1);
60  SetTRXVU_config_param();
61
62  InitTxModule();
63
64  isimepsv2_ivid7_piu__replyheader_t response;
65  logError(isimepsv2_ivid7_piu__outputbuschannelon(0,
isimepsv2_ivid7_piu__imeps_channel__channel_5v_sw2, &response),
"InitTrxvuAndAnts - isimepsv2_ivid7_piu__outputbuschannelon");
66  ISIS_ANTS_t myAntennaAddress[2];
67  myAntennaAddress[0].i2cAddr = ANTS_SIDE_A_I2C_ADDR;
68  myAntennaAddress[1].i2cAddr = ANTS_SIDE_B_I2C_ADDR;
69  int errorAnts = ISIS_ANTS_Init(myAntennaAddress, 2);
70
71  logError(errorAnts, "Ants - ISIS_ANTS_Init");
72  logError(rv, "TRXVU - ISIS_VU_E_Init");
73
74  //Initialize the AntS system
75  if(rv == 6) return errorAnts;
76  return errorAnts + rv;
77
78 }
79
80 void IsNeededToContinueAntDeploy()
81 {
82  Boolean flagAnts = FALSE;
83  if(FRAM_read((unsigned char*)&flagAnts, TRY_TO_DEPLOY_ADDR,

```

```

TRY_TO_DEPLOY_SIZE)) return;
84  if(flagAnts == 0) return;
85  continueDeploy = flagAnts;
86  return;
87 }
88
89  /*
90  * set in FRAM that we need to stop try to deploy
91  */
92  void SetNeedToStopAntDeploy()
93 {
94  continueDeploy = FALSE;
95  if(FRAM_writeAndVerify((unsigned char*)&continueDeploy,
TRY_TO_DEPLOY_ADDR, TRY_TO_DEPLOY_SIZE)) return;
96 }
97
98
99  /*!
100 * @Brief Initializes data field for transmission - semaphores, parameters from the
FRAM
101 * @return
102 */
103  void InitTxModule()
104 {
105  //Get beacon interval from FRAM
106  setNewBeaconIntervalToPeriod();
107  IsNeededToContinueAntDeploy();
108  time_unix timeNow;
109  logError(Time_getUnixEpoch((unsigned int*)&timeNow), "InitTxModule -
Time_getUnixEpoch");
110  if(timeNow < getTransponderEndTime())
111      setTransponderOn();
112  vSemaphoreCreateBinary(semaphorDump);
113  queueAbortDump = xQueueCreate(1, sizeof(Booleen));
114 }
115
116  /*
117  * set transponder on
118  * return type=int; according to I2C_write error list
119  */
120  int setTransponderOn()
121 {
122  if(!CheckTransmitionAllowed()) return -1;
123  if(GetTxFlag()) return -1;
124  unsigned char data[] = {0x38, trxvu_transponder_on}; // 0x38 - number of
commend to change the transmitter mode.

```

```

125     return logError(I2C_write(I2C_TRXVU_TC_ADDR, data, 2), "setTransponderOn
- I2C_write"); // Set transponder on
126 }
127
128     /*
129  * set transponder off
130  * return type=int; according to I2C_write error list
131  * */
132     int setTransponderOff()
133 {
134     unsigned char data[] = {0x38, trxvu_transponder_off}; // 0x38 - number of
commend to change the transmitter mode.
135     return logError(I2C_write(I2C_TRXVU_TC_ADDR, data, 2),
"CMD_SetOff_Transponder - I2C_write"); // Set transponder off
136 }
137
138     /*
139  * Check if we pass the time of the transponder and if so get out of this state.
140  * return type=int; -1 on error in getTransponderEndTime
141  *         0 on success
142  *         error according to I2C_write errors
143  * */
144     int turnOffTransponder()
145 {
146     time_unix timeNow;
147     int error = logError(Time_getUnixEpoch((unsigned int*)&timeNow),
"turnOffTransponder - Time_getUnixEpoch");
148     if(error) return error;
149     time_unix timeEnd = getTransponderEndTime();
150     if(timeEnd == 0) return -1;
151     if(!CheckTransmitionAllowed())
152         error = 0;
153     else if(GetTxFlag()) error = 0;
154     else if(timeEnd > timeNow)
155         return 0;
156     error = setTransponderOff();
157     InitTrxvuAndAnts();
158     vTaskDelay(100);
159     if(error) return error;
160     time_unix new = 0;
161     return setTransponderEndTime(new);
162 }
163
164     /*
165  * Check if we pass the time of the idle and if so get out of this state.
166  * return type=int; -1 on error in getIdleEndTime

```

```

167 *          0 on success
168 *          error according to SetIdleState errors
169 */
170 int turnOffIdle()
171 {
172     if(!CheckTransmitionAllowed()) return SetIdleState(isis_vu_e__onoff__off, 0);
173     if(GetTxFlag()) return SetIdleState(isis_vu_e__onoff__off, 0);
174     time_unix timeNow;
175     int error = logError(Time_getUnixEpoch((unsigned int*)&timeNow),
"turnOffTransponder - Time_getUnixEpoch");
176     if(error) return error;
177     time_unix timeEnd = getIdleEndTime();
178     if(timeEnd == 0) return -1;
179     if(timeEnd > timeNow)
180         return 0;
181     return SetIdleState(isis_vu_e__onoff__off, 0);
182 }
183
184 /*
185 * Sets Mute time end value in FRAM and check
186 * @return type=int; -1 on error
187 *          -2 on wrong set
188 *          0 on success
189 */
190 int setMuteEndTime(time_unix endTime)
191 {
192     time_unix timeNow;
193     if(endTime > MAX_MUTE_TIME) // check we are in range and if not round it.
194         endTime = MAX_MUTE_TIME;
195     if(logError(Time_getUnixEpoch((unsigned int*)&timeNow), "setMuteEndTime -
Time_getUnixEpoch"))
196         return -1;
197     endTime += timeNow;
198     if(logError(FRAM_write((unsigned char*)&endTime, MUTE_END_TIME_ADDR,
MUTE_END_TIME_SIZE), "setMuteEndTime - FRAM_write"))
199         return -1;
200     time_unix check = getMuteEndTime();
201     if(check != endTime)
202         return logError(-2, "setMuteEndTime - Not written right");
203     return 0;
204 }
205
206 /*
207 * Gets Mute time end value from FRAM
208 * @return type=time_unix; 0 on fail

```

```

209 *          mute end time on success
210 */
211 time_unix getMuteEndTime()
212 {
213     time_unix muteEndTime = 0;
214     if(logError(FRAM_read((unsigned char*)&muteEndTime,
MUTE_END_TIME_ADDR, MUTE_END_TIME_SIZE), "getMuteEndTime -
FRAM_read"))
215         return 0;
216     return muteEndTime;
217 }
218
219 /*
220 * Sets transponder end time value from FRAM
221 * @param[int] name=transponderEndTime; type=time_unix; get the time we sat as
the transponder end time
222 * @return type=int; 0 on success
223 *          error according to <hal/errors.h>
224 */
225 int setTransponderEndTime(time_unix transponderEndTime)

226 {
227     if(transponderEndTime > MAX_TRANS_TIME)
228         transponderEndTime = MAX_TRANS_TIME;
229     return logError(FRAM_write((unsigned char*)&transponderEndTime,
TRANSPONDER_END_TIME_ADDR, TRANSPONDER_END_TIME_SIZE),
"setTransponderEndTime - FRAM_write");
230
231 }
232
233
234
235 /*
236 * Gets transponder end time value from FRAM
237 * @return type=time_unix; 0 on fail
238 *          Transponder end time on success
239 */
240 time_unix getTransponderEndTime()
241 {
242     time_unix transponderEndTime;
243     if(logError(FRAM_read((unsigned char*)&transponderEndTime,
TRANSPONDER_END_TIME_ADDR, TRANSPONDER_END_TIME_SIZE),
"getTransponderEndTime - FRAM_read"))
244         return 0;
245     return transponderEndTime;
246 }

```

```

247
248  /*
249  * Gets Idle end time value from FRAM
250  * @return type=time_unix; 0 on fail
251  *         idle end time on success
252  */
253 time_unix getIdleEndTime()
254 {
255     time_unix idleEndTime;
256     if(logError(FRAM_read((unsigned char*)&idleEndTime,
257         IDLE_END_TIME_ADDR, IDLE_END_TIME_SIZE), "getIdleEndTime - FRAM_read"))
258         return 0;
259     return idleEndTime;
260 }
261 /**
262  * Sets transponder RSSI value in FRAM and check
263  * @return type=int;  -1 on error
264  *                   -2 on wrong set
265  *                   -3 on I2C_write error
266  *                   0 on success
267  */
268 int setTransponderRSSIinFRAM(short val)
269 {
270     if(val < 0) //min according to TRXVU Transponder Mode Addendum (in drive)
271
272         val = 0;
273     else if(val > 4095) //max according to TRXVU Transponder Mode Addendum (in
274         drive)
275         val = 4095;
276     unsigned char data[] = {0x52, 0, 0};
277     memcpy(data + 1, (unsigned char*)&val, sizeof(short));
278     if(logError(I2C_write(I2C_TRXVU_TC_ADDR, data, 3),
279         "setTransponderRSSIinFRAM - I2C_write"))
280         return -3;
281     if(logError(FRAM_write((unsigned char*)&val, TRANSPONDER_RSSI_ADDR,
282         TRANSPONDER_RSSI_SIZE), "setTransponderRSSIinFRAM - FRAM_write"))
283         return -1;
284     short check = 0;
285     logError(FRAM_read((unsigned char*)&check, TRANSPONDER_RSSI_ADDR,
286         TRANSPONDER_RSSI_SIZE), "setTransponderRSSIinFRAM - FRAM_read");
287     if(check != val)
288         return logError(-2, "setTransponderRSSIinFRAM - Not written right");
289     return 0;
290 }

```



```

287
288     /**
289  * Gets transponder RSSI value from FRAM
290  * @return type=short; RSSI value from FRAM, -1 on error
291  */
292     short getTransponderRSSIFromFRAM()
293 {
294     short rssi_val;
295     if(logError(FRAM_read((unsigned char*)&rssi_val,
TRANSPONDER_RSSI_ADDR, TRANSPONDER_RSSI_SIZE),
"getTransponderRSSIFromFRAM - FRAM_read"))
296         return -1;
297     return rssi_val;
298 }
299
300     /**
301  * Sets the new beacon interval in period.
302  * return type=int; error according to <hal/errors.h>
303  */
304     int setNewBeaconIntervalToPeriod()
305 {
306     return logError(FRAM_read((unsigned char*)&period,
BEACON_INTERVAL_TIME_ADDR, BEACON_INTERVAL_TIME_SIZE),
"setNewBeaconIntervalToPeriod - FRAM_read");
307 }
308
309     /**
310  * @brief set the idle state of the trxvu
311  * @param[in] state ON/OFF
312  * @param[in] duration for how long will the satellite be in idle state, if state is OFF
than this value is ignored
313  * @return type=int;  0 in successful
314  *                   -1 in failure
315  *                   -2 FRAM read problem
316  *                   -3 FRAM write problem
317  *                   -4 wrong time set
318  *                   -5 time problem
319  *                   -6 not on or off
320  */
321     int SetIdleState(isis_vu_e__onoff_t state, time_unix duration)
322 {
323     if(!CheckTransmitionAllowed()) state = isis_vu_e__onoff__off;
324     else if(GetTxFlag()) state = isis_vu_e__onoff__off;
325     if(state == isis_vu_e__onoff__on)
326     {

```

```

327     if(logError(isis_vu_e__set_idle_state(ISIS_TRXVU_I2C_BUS_INDEX,
isis_vu_e__onoff__on), "SetIdleState - isis_vu_e__set_idle_state"))
328         return -1;
329     if(duration > MAX_IDLE_TIME)
330         duration = MAX_IDLE_TIME;
331     time_unix timeNow = 0;
332     if(logError(Time_getUnixEpoch((unsigned int*)&timeNow),
"CMD_UnMuteTRXVU - Time_getUnixEpoch"))
333         return -5;
334     duration += timeNow;
335     if(logError(FRAM_write((unsigned char*)&duration, IDLE_END_TIME_ADDR,
IDLE_END_TIME_SIZE), "SetIdleState - FRAM_write"))
336         return -3;
337     time_unix check = 0;
338     if(logError(FRAM_read((unsigned char*)&check, IDLE_END_TIME_ADDR,
IDLE_END_TIME_SIZE), "SetIdleState - FRAM_read"))
339         return -2;
340     if(check != duration)
341         return logError(-4, "SetIdleState - Not written right");
342 }
343 else if(state == isis_vu_e__onoff__off)
344 {
345     if(logError(isis_vu_e__set_idle_state(ISIS_TRXVU_I2C_BUS_INDEX,
isis_vu_e__onoff__off), "SetIdleState - isis_vu_e__set_idle_state"))
346         return -1;
347     duration = 0;
348     if(logError(FRAM_write((unsigned char*)&duration, IDLE_END_TIME_ADDR,
IDLE_END_TIME_SIZE), "SetIdleState - FRAM_write"))
349         return -3;
350     time_unix check = 0;
351     if(logError(FRAM_read((unsigned char*)&check, IDLE_END_TIME_ADDR,
IDLE_END_TIME_SIZE), "SetIdleState - FRAM_read"))
352         return -2;
353     if(check != duration)
354         return logError(-4, "SetIdleState - Not written right");
355 }
356 else return -6;
357 return 0;
358 }
359
360 /*
361 * Gets number of packets in waiting.
362 * @return type=int; -1 on error
363 *         number of packets on success
364 **/
365 int GetNumberOfFramesInBuffer()

```

```

366 {
367     unsigned short frameCount;
368     int err = logError(isis_vu_e__get_frame_count(0, &frameCount),
"GetNumberOfFramesInBuffer - isis_vu_e__get_frame_count"); // Get number of
packets in buffer
369     if(err != E_NO_SS_ERR)
370         return -1;
371     return frameCount;
372 }
373
374 /*
375 * Get commend from the buffer and divide the info according to sat_packet_t
headers
376 * @param[out] name=cmd; type=sat_packet_t*; Put here the info from packet
according to the sat_packet_t struct.
377 * @return type=CMD_ERR; return command_success on success
378 *         and error according to CMD_ERR
379 */
380 CMD_ERR GetOnlineCommand(sat_packet_t *cmd)
381 {
382     unsigned char rxframebuffer[SIZE_RXFRAME] = {0};
383     isis_vu_e__get_frame__from_t rx_frame = {0,0,0, rxframebuffer}; // Where the
packet saved after read
384     int error = logError(isis_vu_e__get_frame(0, &rx_frame), "GetOnlineCommand -
isis_vu_e__get_frame"); // Get packet
385     error += logError(isis_vu_e__remove_frame(0), "GetOnlineCommand -
isis_vu_e__remove_frame");
386     if(error != E_NO_SS_ERR)
387         return execution_error;
388     error = ParseDataToCommand(rx_frame.data, cmd); // Put the info from the
packet in the cmd parameter
389     if(error != invalid_sat_id) return logError(error, "GetOnlineCommand -
ParseDataToCommand");
390     return error;
391 }
392
393 /*
394 * Send Ax25 packet
395 * @param name=cmd; type=sat_packet_t*; The packet the sat got and use to find
all the required information (like the headers we add)
396 * @param name=avalFrames; type=int*; availed frames
397 * @return type=int; return -1 on cmd NULL
398 *         -2 if we can't transmmmit
399 *         Error code according to <hal/errors.h>
400 */
401 int TransmitSplPacket(sat_packet_t *packet, int *avalFrames)

```

```

402 {
403     if(!CheckTransmitionAllowed())
404         return -2;
405     unsigned char avail;
406     if(packet == NULL)
407         return -1;
408     size_t place = sizeof(packet->ID) + sizeof(packet->cmd_subtype) + sizeof
(packet->cmd_type) + sizeof(packet->length) + packet->length; // Get the length of the
data of the packet (including the headers we add)

409     int error = logError(isis_vu_e__send_frame(0, (unsigned char *)packet, place,
&avail), "TransmitSplPacket - isis_vu_e__send_frame"); // Transmit packet

410     *avalFrames = (int)avail; // Get avail Frames
411     return error;
412 }
413
414     /*
415     * Send Ax25 packet with output data
416     * @param name=cmd; type=sat_packet_t*; The packet the sat got and use to find
all the required information (like the headers we add)
417     * @param name=data; type=unsigned char*; The data we want to send
418     * @param name=length; type=unsigned short; length of data
419     * @return type=int; return -1 on cmd NULL
420     *             -2 on fail in Assemble command
421     *             -3 if we can't transmmmit
422     *             Error code according to <hal/errors.h>
423     */
424     int TransmitDataAsSPL_Packet(sat_packet_t *cmd, unsigned char *data,
unsigned short length)
425 {
426     if(!CheckTransmitionAllowed())
427         return -3;
428     else if(GetTxFlag()) return -3;
429     unsigned char avail;
430     if(cmd == NULL)
431         return -1;
432     if(AssembleCommand(data, length, cmd->cmd_type, cmd->cmd_subtype, cmd-
>ID, cmd)) return -2; // Change the packet for send with the needed info

433     size_t place = sizeof(cmd->ID) + sizeof(cmd->cmd_subtype) + sizeof(cmd-
>cmd_type) + sizeof(cmd->length) + cmd->length; // Get the length of the data of the
packet (including the headers we add)
434     return logError(isis_vu_e__send_frame(0, (unsigned char *)cmd, place, &avail),
"TransmitDataAsSPL_Packet - isis_vu_e__send_frame"); // Transmit packet

```

```

435 }
436
437     /*
438  * check if we are needed to abort the dump
439  * @return type=Boolean; FALSE - not needed. TRUE - abort
440  */
441 Boolean CheckDumpAbort()
442 {
443     Boolean recive = FALSE;
444     int err = xQueueReceive(queueAbortDump, &recive, (portTickType) 10);
445     if(!err)
446         return FALSE;
447     if(recive == TRUE)
448         return TRUE;
449     return FALSE;
450 }
451
452     /*
453  * Check if we can transmit. (according to mute and EPS condition)
454  * @return type=Boolean; TRUE if we can transmit
455  *             FALSE if we can't
456  */
457 Boolean CheckTransmitionAllowed()
458 {
459     time_unix timeNow;
460     logError(Time_getUnixEpoch((unsigned int*)&timeNow),
461 "CheckTransmitionAllowed - Time_getUnixEpoch");
462     if(timeNow < getMuteEndTime()) // check we are after the mute end time.
463
464         return FALSE;
465         return TRUE;
466 }
467
468     /*
469  * @brief transmits beacon according to beacon logic
470  * @ return 0 if everything is fine
471  *       -1 if we not suppose to send beacon now
472  *       else it return error according to the stuff that didn't work
473  */
474 int BeaconLogic()
475 {
476     if(!CheckExecutionTime(lastTimeSendingBeacon, period)) // Check if we can
477         transmit beacon and also if the period of time we need to wait pass
478
479         return -1;
480     if(!CheckTransmitionAllowed()) return -1;

```

```

477  sat_packet_t beacon;
478  short length = sizeof(WOD_Telemetry_t);
479  WOD_Telemetry_t data;
480  GetCurrentWODTelemetry(&data); // Gets the telemetry of the beacon and put it
in data.
481  unsigned int id = CUBE_SAT_ID;
482  id = id<<24;
483  logError(AssembleCommand((unsigned char *)&data, length, trxvu_cmd_type,
BEACON_SUBTYPE, id, &beacon), "BeaconLogic - AssembleCommand"); // Create
the beacon packet
484  int avalFrames;
485  SetTRXVU_config_param();
486  int error = logError(TransmitSpiPacket(&beacon, &avalFrames), "BeaconLogic -
TransmitSpiPacket"); // Send the beacon packet
487  if(error) // if error return it
488      return error;
489  return logError(Time_getUnixEpoch((unsigned int*)&lastTimeSendingBeacon),
"BeaconLogic - Time_getUnixEpoch"); // Check last time we send beacon to now.

490
491 }
492
493  void DeployAnts()
494 {
495  time_unix lastDeploy;
496  if(logError(FRAM_read((unsigned char*)&lastDeploy,
LAST_TRY_TO_DEPLOY_TIME_ADDR, LAST_TRY_TO_DEPLOY_TIME_SIZE),
"DeployAnts - FRAM_read")) return;
497  if(!CheckExecutionTime(lastDeploy, 30*60)) return;
498  AntArm(0);
499  AntDeployment(0);
500  AntArm(1);
501  AntDeployment(1);
502  time_unix timeUnix;
503  logError(Time_getUnixEpoch((unsigned int*)&timeUnix), "DeployAnts -
Time_getUnixEpoch");
504  if(logError(FRAM_writeAndVerify((unsigned char*)&timeUnix,
LAST_TRY_TO_DEPLOY_TIME_ADDR, LAST_TRY_TO_DEPLOY_TIME_SIZE),
"DeployAnts - FRAM_writeAndVerify")) return;
505
506 }
507
508  /*
509  * Have the TRXVU logic. (Beacon send, check if have packets, read packet etc.)
510  *@return type=int; return error if have and command_success if not
511  **/

```

```

512     int TRX_Logic()
513 {
514     sat_packet_t cmd;
515     int error = 0;
516     turnOffTransponder();
517     turnOffIdle();
518     BeaconLogic(); // do the beacon logic
519     if(continueDeploy) DeployAnts();
520     if(GetNumberOfFramesInBuffer() > 0) // Check if we have packets waiting

521     { // if so
522         error = GetOnlineCommand(&cmd); // Get the packet and put her in the
sat_packet_t struct (in the param cmd)
523         if(error != command_success) // Check if have error
524             return error;
525         KickGroundCommWDT();
526         SendAckPacket(ACK_RECEIVE_COMM, &cmd, NULL, 0); // Send ack of
receiving the packet
527         ActUponCommand(&cmd); // Go to do the command
528     }
529     return command_success;
530 }
531
532     void setBeaconIntervalNOT_FROM_FRAM(unsigned int interval)
533 {
534     period = interval;
535 }
536

```

**SubSystemModules/
Communication/
SubsystemCommands**


```

1
2  #ifndef EPS_COMMANDS_H_
3  #define EPS_COMMANDS_H_
4
5  #include "SubSystemModules/Communication/SatCommandHandler.h"
6
7  #include "SubSystemModules/Communication/AckHandler.h"
8
9  #include "SubSystemModules/Communication/AckErrors.h"
10
11 #include "SubSystemModules/Communication/TRXVU.h"
12
13 #include "SubSystemModules/PowerManagment/EPS.h"
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37

```

```

/*
 * Set update threshold voltages.
 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
 * use to find all the required information (the headers we add and the new thresholds)
 * @return type=int; return type of error
 * -1 on cmd NULL
 * errors according to "AckErrors.h" and send ack
 */
int CMD_UpdateThresholdVoltages(sat_packet_t *cmd);

```

```

/*
 * Get threshold voltages.
 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
 * use to find all the required information (the headers we add)
 * @return type=int; return type of error
 * 1 on FRAM_read error, and TransmitDataAsSPL_Packet
 * errors
 */
int CMD_GetThresholdVoltages(sat_packet_t *cmd);

```

```

/*
 * restore default threshold voltages.
 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
 * use to find all the required information (the headers we add)
 * @return type=int; return type of error
 * -1 on cmd NULL
 * errors according to "AckErrors.h" and send ack
 */
int CMD_RestoreDefaultThresholdVoltages(sat_packet_t *cmd);

```

```

38 * Set new alpha value.
39 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the headers we add and the new alpha val)
40 * @return type=int; return type of error
41 *                                     -1 on cmd NULL
42 *                                     errors according to "AckErrors.h"
43 **/
44 int CMD_UpdateAlpha(sat_packet_t *cmd);
45
46 /*
47 * Set default alpha value.
48 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the headers we add)
49 * @return type=int; return type of error
50 *                                     -1 on cmd NULL
51 *                                     errors according to "AckErrors.h"
52 **/
53 int CMD_RestoreDefaultAlpha(sat_packet_t *cmd);
54
55 /*
56 * Get alpha value.
57 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the headers we add)
58 * @return type=int; return type of error
59 *                                     1 on FRAM_read error, and TransmitDataAsSPL_Packet
errors
60 **/
61 int CMD_GetAlpha(sat_packet_t *cmd);
62
63 /*
64 * Get state of EPS.
65 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the headers we add)
66 * @return type=int; return type of error TransmitDataAsSPL_Packet
67 **/
68 int CMD_GetState(sat_packet_t *cmd);
69
70 /*
71 * Reset EPS WDT
72 * @param[in] name=cmd; type=sat_packet_t*; The packet the sat got and use to
find all the required information (the headers we add)
73 * @return type=int; 26 can't reset, error according to SendAckPacket
74 **/
75 int CMD_EPS_ResetWDT(sat_packet_t *cmd);
76
77 /**

```

```

78 * get heater values from the config parameters.
79 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
    use to find all the required information (the headers we add)
80 * @return type=int; return error according to
    isismepsv2_ivid7_piu__getconfigurationparameter or TransmitDataAsSPL_Packet
81 */
82     int CMD_GetHeaterVal(sat_packet_t *cmd);
83
84     /** DO NOT USE!!!
85 * set heater values from the config parameters.
86 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
    use to find all the required information (the headers we add and the threshold)
87 * @return type=int; return error according to
    isismepsv2_ivid7_piu__getconfigurationparameter or TransmitDataAsSPL_Packet
88 */
89     int CMD_SetHeaterVal(sat_packet_t* cmd);
90
91     #endif /* EPS_COMMANDS_H_ */
92

```

```

1 /*
2  * EPS_Commands.c
3  *
4  * Created on: 21 Nov 2024
5  * Author: maayan
6  */
7
8  #include "EPS_Commands.h"
9  #include <string.h>
10
11  /*
12  * Set new alpha value.
13  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
14  * use to find all the required information (the headers we add and the new alpha val)
15  * @return type=int; return type of error
16  * -1 on cmd NULL
17  * errors according to "AckErrors.h"
18  */
19 int CMD_UpdateAlpha(sat_packet_t *cmd)
20 {
21     if(cmd == NULL) return -1;
22     unsigned char error_ack = 0;
23     if(cmd->length != 4)
24     {
25         error_ack = ERROR_WRONG_LENGTH_DATA;
26         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
27         Send ack error according to "AckErrors.h"
28         return error_ack;
29     }
30     float alpha;
31     memcpy(&alpha, cmd->data, cmd->length);
32     int error = UpdateAlpha(alpha);
33     switch(error)
34     {
35     case -1:
36     {
37         error_ack = ERROR_WRITE_TO_FRAM;
38         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
39         Send ack error according to "AckErrors.h"
40         return error_ack;
41     }
42     case -2:
43     {
44         error_ack = ERROR_INVALID_ALPHA;
45         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
46         Send ack error according to "AckErrors.h"

```

```

43     return error_ack;
44 }
45 case -4:
46 {
47     error_ack = ERROR_READ_FROM_FRAM;
48     SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
49     return error_ack;
50 }
51 case -5:
52 {
53     error_ack = ERROR_WRITTEN_IN_FRAM_WRONG;
54     SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
55     return error_ack;
56 }
57 default:
58     break;
59 }
60 return SendAckPacket(ACK_UPDATE_EPS_ALPHA , cmd, NULL, 0);
61 }
62
63 /*
64 * Set default alpha value.
65 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the headers we add)
66 * @return type=int; return type of error
67 *
68 * -1 on cmd NULL
69 * errors according to "AckErrors.h"
70 */
71 int CMD_RestoreDefaultAlpha(sat_packet_t *cmd)
72 {
73     if(cmd == NULL) return -1;
74     cmd->length = 4;
75     float alpha = DEFAULT_ALPHA_VALUE;
76     memcpy(cmd->data, (unsigned char*)&alpha, cmd->length);
77     return CMD_UpdateAlpha(cmd);
78 }
79
80 /*
81 * Get alpha value.
82 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the headers we add)
83 * @return type=int; return type of error
84 *
85 * 1 on FRAM_read error, and TransmitDataAsSPL_Packet
errors

```

```

84 */
85     int CMD_GetAlpha(sat_packet_t *cmd)
86 {
87     float alpha;
88     int error = GetAlpha(&alpha);
89     if(error == -2)
90     {
91         unsigned char error_ack = ERROR_READ_FROM_FRAM;
92         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
93         return error_ack;
94     }
95     return SendAckPacket(ACK_GET_ALPHA, cmd, (unsigned char*)&alpha,
EPS_ALPHA_FILTER_VALUE_SIZE); // Send back the alpha value
96 }
97
98 /*
99 * Get threshold voltages.
100 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the headers we add)
101 * @return type=int; return type of error
102 *
103 * 1 on FRAM_read error, and TransmitDataAsSPL_Packet
errors
104 */
105     int CMD_GetThresholdVoltages(sat_packet_t *cmd)
106 {
107     EpsThreshVolt_t threshold;
108     int error = GetThresholdVoltages(&threshold);
109     if(error == -2)
110     {
111         unsigned char error_ack = ERROR_READ_FROM_FRAM;
112         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
113         return error_ack;
114     }
115     return SendAckPacket(ACK_GET_EPS_THRESHOLDS, cmd, (unsigned char
*)&threshold.raw, EPS_THRESH_VOLTAGES_SIZE); // Send back the threshold voltages
116 }
117
118 /*
119 * Set update threshold voltages.
120 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the headers we add and the new thresholds)

```

```

121 * @return type=int; return type of error
122 *           -1 on cmd NULL
123 *           errors according to "AckErrors.h" and send ack
124 */
125 int CMD_UpdateThresholdVoltages(sat_packet_t *cmd)
126 {
127     if(cmd == NULL) return -1;
128     unsigned char error_ack = 0;
129     if(cmd->length != 8)
130     {
131         error_ack = ERROR_WRONG_LENGTH_DATA;
132         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
133         // Send ack error according to "AckErrors.h"
134         return error_ack;
135     }
136     EpsThreshVolt_t threshold;
137     memcpy(threshold.raw, &(cmd->data), cmd->length);
138     int error = UpdateThresholdVoltages(threshold);
139     switch(error)
140     {
141     case -1:
142     {
143         error_ack = ERROR_WRITE_TO_FRAM;
144         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack));
145         // Send ack error according to "AckErrors.h"
146         return error_ack;
147     }
148     case -2:
149     {
150         error_ack = ERROR_INVALID_TRESHOLD;
151         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack));
152         // Send ack error according to "AckErrors.h"
153         return error_ack;
154     }
155     case -3:
156     {
157         error_ack = ERROR_READ_FROM_FRAM;
158         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack));
159         // Send ack error according to "AckErrors.h"
160         return error_ack;
161     }
162     case -4:
163     {
164         error_ack = ERROR_WRITTEN_IN_FRAM_WRONG;
165         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack));
166         // Send ack error according to "AckErrors.h"

```

```

162     return error_ack;
163 }
164 default:
165     break;
166 }
167 return SendAckPacket(ACK_UPDATE_EPS_VOLTAGES , cmd, NULL, 0);
168 }
169
170 /*
171  * restore default threshold voltages.
172  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
173  * use to find all the required information (the headers we add)
174  * @return type=int; return type of error
175  * -1 on cmd NULL
176  * errors according to "AckErrors.h" and send ack
177  */
178 int CMD_RestoreDefaultThresholdVoltages(sat_packet_t *cmd)
179 {
180     if(cmd == NULL) return -1;
181     cmd->length = 8;
182     voltage_t defaultThershold[NUMBER_OF_THRESHOLD_VOLTAGES] =
183     DEFAULT_EPS_THRESHOLD_VOLTAGES;
184     EpsThreshVolt_t threshold;
185     for(int i = 0; i < NUMBER_OF_THRESHOLD_VOLTAGES; i++)
186         threshold.raw[i] = defaultThershold[i];
187     memcpy(cmd->data, (unsigned char*)&threshold, cmd->length);
188     return CMD_UpdateThresholdVoltages(cmd);
189 }
190
191 /*
192  * Get state of EPS.
193  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
194  * use to find all the required information (the headers we add)
195  * @return type=int; return type of error TransmitDataAsSPL_Packet
196  */
197 int CMD_GetState(sat_packet_t *cmd)
198 {
199     EpsState_t state = GetSystemState();
200     return SendAckPacket(ACK_GET_STATE, cmd, (unsigned char*)&state, sizeof
201     (state)); // Send back the state of the eps
202 }
203
204 /*
205  * Reset EPS WDT
206  * @param[in] name=cmd; type=sat_packet_t*; The packet the sat got and use to

```



```

find all the required information (the headers we add)
204 * @return type=int; 26 can't reset, error according to SendAckPacket
205 */
206 int CMD_EPS_ResetWDT(sat_packet_t *cmd)
207 {
208     isimepsv2_ivid7_piu__replyheader_t response;
209
210
211     int error = isimepsv2_ivid7_piu__resetwatchdog(0, &response);
212     if(error)
213     {
214         unsigned char error_ack = ERROR_CANT_RESET_WDT;
215         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
216         return error_ack;
217     }
218     return SendAckPacket(ACK_EPS_RESET_WDT , cmd, NULL, 0);
219 }
220
221 /**
222 * get heater values from the config parameters.
223 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the headers we add)
224 * @return type=int; return error according to
isimepsv2_ivid7_piu__getconfigurationparameter or TransmitDataAsSPL_Packet
225 */
226 int CMD_GetHeaterVal(sat_packet_t* cmd)
227 {
228     isimepsv2_ivid7_piu__getconfigurationparameter__from_t response;
229     //threshold low (when to start)
230     int err = logError(isimepsv2_ivid7_piu__getconfigurationparameter(0, 0x3000,
&response), "CMD_GetHeaterVal - isimepsv2_ivid7_piu__getconfigurationparameter"
); //gets the data structure of the parameter to response according to the id of the
parameter
231     if(err)
232     {
233         unsigned char ackErr = ERROR_CANT_DO;
234         SendAckPacket(ACK_ERROR_MSG , cmd, &ackErr, sizeof(ackErr)); // Send
ack error according to "AckErrors.h"
235         return err;
236     }
237     //threshold high (when to end)
238     isimepsv2_ivid7_piu__getconfigurationparameter__from_t response2;
239     err = logError(isimepsv2_ivid7_piu__getconfigurationparameter(0, 0x3003,
&response2), "CMD_GetHeaterVal - isimepsv2_ivid7_piu__getconfigurationparameter"
); //gets the data structure of the parameter to response according to the id of the

```

```

parameter
240     if(err)
241     {
242         unsigned char ackErr = ERROR_CANT_DO;
243         SendAckPacket(ACK_ERROR_MSG , cmd, &ackErr, sizeof(ackErr)); // Send
ack error according to "AckErrors.h"
244         return err;
245     }
246
247     short responses[2] = {0, 0};
248     memcpy((unsigned char*)&responses[0], response.fields.par_val, 2);
249     memcpy((unsigned char*)&responses[1], response2.fields.par_val, 2);
250     float responses_float[2] = {(responses[0] * 0.01), (responses[1] * 0.01)};
251     return SendAckPacket(ACK_GET_HEATER_THRESHOLD, cmd, (unsigned char
*)responses_float, sizeof(responses_float)); // Send back the alpha value
252 }
253
254 /**
255  * set heater values from the config parameters. DO NOT USE!!!
256  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the headers we add and the threshold)
257  * @return type=int; return error according to
isimepsv2_ivid7_piu__getconfigurationparameter or TransmitDataAsSPL_Packet
258  */
259     int CMD_SetHeaterVal(sat_packet_t* cmd)
260 {
261     if(cmd == NULL) return -1;
262     unsigned char error_ack = 0;
263     if(cmd->length != 8)
264     {
265         error_ack = ERROR_WRONG_LENGTH_DATA;
266         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
267         return error_ack;
268     }
269     float forUsToSwitchToShort;
270     memcpy((unsigned char*)&forUsToSwitchToShort, cmd->data, sizeof(float));
271     int16_t temp = (short)(forUsToSwitchToShort / 0.01);
272     isimepsv2_ivid7_piu__setconfigurationparameter__from_t response;
273     isimepsv2_ivid7_piu__setconfigurationparameter__to_t send;
274     //threshold low
275     send.fields.par_id = 0x3000;
276     memcpy(send.fields.par_val, (unsigned char*)&temp, sizeof(temp));
277     int err = logError(isimepsv2_ivid7_piu__setconfigurationparameter(0, &send,
&response), "CMD_SetHeaterVal - isimepsv2_ivid7_piu__setconfigurationparameter");

```

```

///gets the data structure of the parameter to response according to the id of the
parameter
278  if(err)
279  {
280      unsigned char ackErr = ERROR_CANT_DO;
281      SendAckPacket(ACK_ERROR_MSG , cmd, &ackErr, sizeof(ackErr)); // Send
ack error according to "AckErrors.h"
282      return err;
283  }
284  //threshold high
285  send.fields.par_id = 0x3003;
286  memcpy((unsigned char*)&forUsToSwitchToShort, cmd->data + sizeof(float),
sizeof(float));
287  temp = (short)(forUsToSwitchToShort / 0.01);
288  memcpy(send.fields.par_val, (unsigned char*)&temp, sizeof(temp));
289  err = logError(isismepsv2_ivid7_piu__setconfigurationparameter(0, &send,
&response), "CMD_SetHeaterVal - isismepsv2_ivid7_piu__setconfigurationparameter");
///gets the data structure of the parameter to response according to the id of the
parameter
290  if(err)
291  {
292      unsigned char ackErr = ERROR_CANT_DO;
293      SendAckPacket(ACK_ERROR_MSG , cmd, &ackErr, sizeof(ackErr)); // Send
ack error according to "AckErrors.h"
294      return err;
295  }
296  return SendAckPacket(ACK_CHANGE_HEATER_THRESHOLDS, cmd, NULL, 0
);
297 }
298
299

```

```

1
2  #ifndef FS_COMMANDS_H_
3  #define FS_COMMANDS_H_
4  #define WAIT_TIME_SEM_DUMP 3
5  #include "SubSystemModules/Communication/SatCommandHandler.h"

6  #include "TLM_management.h"
7  #include "SubSystemModules/Communication/AckHandler.h"

8  #include "SubSystemModules/Housekeeping/TelemetryCollector.h"

9  #include "Maintenance_Commands.h"
10
11
12  /*
13  * Delete tlm files from sd according to tlmType, start date, end date
14  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
15  * use to find all the required information (the headers we add and tlmType, start date,
16  * end date)
17  * @return type=int; -1 cmd null
18  *          17 wrong data length
19  *          60 didn't delete all files
20  *          according to SendAckPacket errors.
21  *          0 on success
22  */
23
24  int CMD_DeleteTLM(sat_packet_t *cmd);
25
26  /*
27  * Delete tlm files from sd according to tlmType
28  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
29  * use to find all the required information (the headers we add and tlmType)
30  * @return type=int; -1 cmd null
31  *          17 wrong data length
32  *          30 can't do
33  *          according to SendAckPacket errors.
34  *          0 on success
35  */
36
37  int CMD_DeleteFilesOfType(sat_packet_t *cmd);
38
39  /*
40  * formatting the SD card.
41  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
42  * use to find all the required information (the headers we add)
43  * @return type=int; return type of error according to SendAckPacket and

```

Hard_ComponenetReset errors

```
40 */
41 int CMD_DeleteAllFiles(sat_packet_t *cmd);
42
43 /*
44 * Get last FS error
45 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the headers we add)
46 * @return type=int; according to SendAckPacket
47 */
48 int CMD_GetLastFS_Error(sat_packet_t *cmd);
49
50 /*
51 * Get free space on the sd in bytes <- it's not precise but it's the best we got
52 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the headers we add)
53 * @return type=int; 27 on error get from struct
54 * according to TransmitDataAsSPL_Packet
55 */
56 int CMD_FreeSpace(sat_packet_t *cmd);
57
58 /*
59 * start dump according to days and create the task of dump
60 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the headers we add and tlmType, start date,
end date)
61 * @return type=int; -1 cmd null
62 * 17 wrong data length
63 * 30 Already have a dump active (can't do two at the same time)
64 * 0 on success
65 *
66 */
67 int CMD_StartDump(sat_packet_t *cmd);
68
69 /*
70 * abort dump. (stops the dump)
71 * @return type=int; according to xQueueSend errors
72 */
73 int CMD_SendDumpAbortRequest();
74
75 /*
76 * the function start dump call for the task. have the dump final logic.
77 * @param[in] name=dump; type=void*; have the parameters we need for the dump.
78 */
79 void TackDump(void *dump);
80
```

```

81     /*
82  * Switch between the sd cards.
83  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
84  * use to find all the required information (the headers we add and sd to switch to)
85  * @return type=int;  -1 cmd null
86  *                    27 wrong data length
87  *                    -4 invalid sd
88  *                    -2 read from FRAM
89  *                    -3 write to FRAM
90  *                    according to Hard_ComponenetReset
91  */
92     int CMD_SwitchSD_card(sat_packet_t *cmd);
93
94     /*
95  * Set new period of saving telemetry according to tlmPeriod and new period
96  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
97  * use to find all the required information (the headers we add and tlmPeriod, new period)
98  * @return type=int; -1 cmd null
99  *                    17 wrong data length
100  *                    error according to SaveAndCheck
101  *                    30 invalid tlmPeriod
102  */
103     int CMD_SetTLMPeriodTimes(sat_packet_t *cmd);
104
105     /*
106  * Set default period of saving telemetry
107  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
108  * use to find all the required information (the headers we add)
109  * @return type=int;  according to FRAM_writeAndVerify
110  */
111     int CMD_SetTLMPeriodTimes_default(sat_packet_t *cmd);
112
113     /*
114  * Get telemetry period times from FRAM
115  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
116  * use to find all the required information (the headers we add)
117  * @return type=int;  according to FRAM_read and TransmitDataAsSPL_Packet
118  */
119     int CMD_GetTLMPeriodTimes(sat_packet_t *cmd);

```

```

1 /*
2  * FS_Commands.c
3  *
4  * Created on: 26 Dec 2024
5  * Author: maayan
6  */
7
8  #include "FS_Commands.h"
9  #include "String.h"
10     dump_arguments_t arg;
11
12     /*
13  * formatting the SD card.
14  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
15  * use to find all the required information (the headers we add)
16  * @return type=int; return type of error according to SendAckPacket and
17  * Hard_ComponenetReset errors
18  */
19     int CMD_DeleteAllFiles(sat_packet_t *cmd)
20     {
21         if(Delete_allTMFilesFromSD())
22         {
23             unsigned char ackError = ERROR_CANT_DO;
24             return SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
25         }
26         SendAckPacket(ACK_DELETE_TLM, cmd, NULL, 0);
27         return Hard_ComponenetReset(cmd);
28     }
29
30     /*
31  * start dump according to days and create the task of dump
32  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
33  * use to find all the required information (the headers we add and tlmType, start date,
34  * end date)
35  * @return type=int; -1 cmd null
36  * 17 wrong data length
37  * 30 Already have a dump active (can't do two at the same time)
38  * 0 on success
39  */
40     int CMD_StartDump(sat_packet_t *cmd)
41     {
42         if(cmd == NULL) return -1;
43         unsigned char ackError = 0;
44         if(cmd->length != 13)
45         {

```

```

43     ackError = ERROR_WRONG_LENGTH_DATA;
44     SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
45     return ackError;
46 }
47 //get the param to give to dump task
48 arg.cmd = *cmd;
49 memcpy((unsigned char*)&arg.dump_type, cmd->data, 1);
50 memcpy((unsigned char*)&arg.t_start, cmd->data + 1, sizeof(time_unix));
51 memcpy((unsigned char*)&arg.t_end, cmd->data + 5, sizeof(time_unix));
52 memcpy((unsigned char*)&arg.resolution, cmd->data + 9, sizeof(int)); //in seconds

53 if(arg.dump_type > tlm_log)
54 {
55     ackError = ERROR_CANT_DO;
56     SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
57     return ackError;
58 }
59 //check we not already in dump
60 if(xSemaphoreTake(semaphorDump,
SECONDS_TO_TICKS(WAIT_TIME_SEM_DUMP)) == pdFALSE)
61 {
62     ackError = ERROR_CANT_DO;
63     SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
64     return ackError;
65 }
66 xTaskHandle taskHandle;
67 xTaskGenericCreate(TackDump, (const signed char*) "CMD_StartDump", 4096,
&arg, configMAX_PRIORITIES - 2, &taskHandle, NULL, NULL); //create dump task

68 return 0;
69 }
70
71 /*
72 * abort dump. (stops the dump)
73 * @return type=int; according to xQueueSend errors
74 */
75 int CMD_SendDumpAbortRequest()
76 {
77     Boolean true = TRUE;
78     return xQueueSend(queueAbortDump, &true, (portTickType) 10); //enter to the
queue true for the dump to stop
79 }
80
81 /*
82 * the function start dump call for the task. have the dump final logic.
83 * @param[in] name=dump; type=void*; have the parameters we need for the dump.

```



```

84 */
85 void TackDump(void *dump)
86 {
87     if(dump == NULL) return;
88     dump_arguments_t *dump_arg = (dump_arguments_t*)dump;
89
90     f_enterFS();
91     int numOfDay = (dump_arg->t_end - dump_arg->t_start) / 24 / 3600; //get num
of days
92     Time start;
93     timeU2time(dump_arg->t_start, &start); //change from time_unix to Time

94     SendAckPacket(ACK_DUMP_START, &dump_arg->cmd, NULL, 0);
95     ReadTLMFiles(dump_arg->dump_type, start, numOfDay, dump_arg->cmd.ID,
dump_arg->resolution); //sends the dump packets
96     SendAckPacket(ACK_DUMP_FINISHED, &dump_arg->cmd, NULL, 0);
97     f_releaseFS();
98     xSemaphoreGive(semaphorDump); //release the dump semaphor

99     vTaskDelete(NULL); //Delete the task
100 }
101
102 /*
103 * Delete tlm files from sd according to tlmType, start date, end date
104 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the headers we add and tlmType, start date,
end date)
105 * @return type=int; -1 cmd null
106 *             17 wrong data length
107 *             60 didn't delete all files
108 *             according to SendAckPacket errors.
109 *             0 on success
110 *
111 */
112 int CMD_DeleteTLM(sat_packet_t *cmd)
113 {
114     if(cmd == NULL) return -1;
115     unsigned char ackError = 0;
116     if(cmd->length != 9)
117     {
118         ackError = ERROR_WRONG_LENGTH_DATA;
119         SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
120         return ackError;
121     }
122     tlm_type_t type;
123     time_unix start;

```

```

124     time_unix end;
125     memcpy((unsigned char*)&type, cmd->data, 1);
126     memcpy((unsigned char*)&start, cmd->data + 1, sizeof(time_unix));
127     memcpy((unsigned char*)&end, cmd->data + 5, sizeof(time_unix));
128     if(type > tlm_log)
129     {
130         ackError = ERROR_CANT_DO;
131         SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
132         return ackError;
133     }
134     Time start_t;
135     timeU2time(start, &start_t);
136     int numOfDay = (end - start) / 24 / 3600;
137     int error = logError(DeleteTLMFiles(type, start_t, numOfDay),
138 "CMD_DeleteFilesOfType - DeleteTLMFiles");
139     if(error != numOfDay + 1)
140     {
141         ackError = ERROR_COULDNT_DELETE_ALL;
142         SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
143         return ackError;
144     }
145     return SendAckPacket(ACK_DELETE_TLM, cmd, NULL, 0);
146 }
147
148 /*
149 * Get last FS error
150 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
151 * use to find all the required information (the headers we add)
152 * @return type=int; according to SendAckPacket
153 */
154 int CMD_GetLastFS_Error(sat_packet_t *cmd)
155 {
156     int FS_error = f_getlasterror();
157     return SendAckPacket(ACK_FS_LAST_ERROR, cmd, (unsigned char
158 *)&FS_error, sizeof(FS_error));
159 }
160
161 /*
162 * Get free space on the sd in bytes <- it's not precise but it's the best we got
163 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
164 * use to find all the required information (the headers we add)
165 * @return type=int; 27 on error get from struct
166 * according to TransmitDataAsSPL_Packet
167 */
168 int CMD_FreeSpace(sat_packet_t *cmd)
169 {

```

```

166  F_SPACE space; //FS struct
167  int error = logError(f_getfreespace(f_getdrive(), &space), "CMD_FreeSpace -
f_getfreespace"); //gets the variables to the struct
168  if(error)
169  {
170      unsigned char ackError = ERROR_GET_FROM_STRUCT;
171      SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
172      return ackError;
173  }
174  return SendAckPacket(ACK_GET_FREE_SPACE, cmd, (unsigned char*)&space.
free, sizeof(space.free));
175 }
176
177  /*
178  * Help function to CMD_SetTLMPeriodTimes. sets the new one according to the
params we got
179  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the headers we add)
180  * @param[in] name=addr; type=unsigned int; get the place in FRAM we need to
write to.
181  * @param[in] name=saveTime; type=unsigned int; the new period time we want to
save.
182  * @return type=int; 1 on read from FRAM
183  *                  2 on write to FRAM
184  *                  3 on written wrong in FRAM
185  *                  0 on success
186  */
187  int SaveAndCheck(unsigned int addr, unsigned int saveTime, sat_packet_t*
cmd)
188  {
189      unsigned char ackError = 0;
190      int error = logError(FRAM_writeAndVerify((unsigned char*)&saveTime, addr,
sizeof(saveTime)), "CMD_SetTLMPeriodTimes - FRAM_writeAndVerify");
191      if(error)
192      {
193          ackError = ERROR_WRITE_TO_FRAM;
194          SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
195          return ackError;
196      }
197      unsigned int check = 0;
198      error = logError(FRAM_read((unsigned char*)&check, addr, sizeof(saveTime)),
"SaveAndCheck - FRAM_read");
199      if(error)
200      {
201          ackError = ERROR_READ_FROM_FRAM;
202          SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));

```

```

203     return ackError;
204 }
205 if(check != saveTime)
206 {
207     ackError = ERROR_WRITTEN_IN_FRAM_WRONG;
208     SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
209     return ackError;
210 }
211 InitSavePeriodTimes();
212 return 0;
213 }
214
215 /*
216 * Set new period of saving telemetry according to tlmPeriod and new period
217 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
218 * use to find all the required information (the headers we add and tlmPeriod, new period)
219 * @return type=int; -1 cmd null
220 *          17 wrong data length
221 *          error according to SaveAndCheck
222 *          30 invalid tlmPeriod
223 */
224 int CMD_SetTLMPeriodTimes(sat_packet_t *cmd)
225 {
226     if(cmd == NULL) return -1;
227     unsigned char ackError = 0;
228     if(cmd->length != 5)
229     {
230         ackError = ERROR_WRONG_LENGTH_DATA;
231         SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
232         return ackError;
233     }
234     char type;
235     unsigned int saveTime;
236     memcpy(&type, cmd->data, 1);
237     memcpy(&saveTime, cmd->data + 1, sizeof(int));
238     int error;
239     switch(type)
240     {
241     case 0: //eps
242         error = SaveAndCheck(EPS_SAVE_TLM_PERIOD_ADDR, saveTime,
243                             cmd);
244         if(error)
245             return error;
246         break;
247     case 1: //trx
248         error = SaveAndCheck(TRXVU_SAVE_TLM_PERIOD_ADDR, saveTime,

```

```

cmd);
247     if(error)
248         return error;
249     break;
250 case 2: //ants
251     error = SaveAndCheck(ANT_SAVE_TLM_PERIOD_ADDR, saveTime,
cmd);
252     if(error)
253         return error;
254     break;
255 case 3: //sp
256     error = SaveAndCheck(SOLAR_SAVE_TLM_PERIOD_ADDR, saveTime,
cmd);
257     if(error)
258         return error;
259     break;
260 case 4: //wod
261     error = SaveAndCheck(WOD_SAVE_TLM_PERIOD_ADDR, saveTime,
cmd);
262     if(error)
263         return error;
264     break;
265 case 5: //rad
266     error = SaveAndCheck(RADFET_SAVE_TLM_PERIOD_ADDR, saveTime,
cmd);
267     if(error)
268         return error;
269     break;
270 case 6: //seu_sel
271     error = SaveAndCheck(SEU_SEL_SAVE_TLM_PERIOD_ADDR,
saveTime, cmd);
272     if(error)
273         return error;
274     break;
275 default:
276     ackError = ERROR_CANT_DO;
277     SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
278     return ackError;
279 }
280 SendAckPacket(ACK_SET_NEW_TLM_PERIOD, cmd, NULL, 0);
281 return 0;
282
283 }
284
285 /*
286  * Set default period of saving telemetry

```

```

287 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the headers we add)
288 * @return type=int; according to FRAM_writeAndVerify
289 */
290 int CMD_SetTLMPeriodTimes_default(sat_packet_t *cmd)
291 {
292     int arrPeriod[7] = {DEFAULT_EPS_SAVE_TLM_TIME,
DEFAULT_TRXVU_SAVE_TLM_TIME, DEFAULT_ANT_SAVE_TLM_TIME,
DEFAULT_SOLAR_SAVE_TLM_TIME, DEFAULT_WOD_SAVE_TLM_TIME,
DEFAULT_RADFET_SAVE_TLM_TIME, DEFAULT_SEU_SEL_SAVE_TLM_TIME};
293     int error = logError(FRAM_writeAndVerify((unsigned char*)arrPeriod,
TLM_SAVE_PERIOD_START_ADDR, sizeof(arrPeriod)),
"CMD_SetTLMPeriodTimes_default - FRAM_writeAndVerify");
294     if(error)
295     {
296         unsigned char ackError = ERROR_WRITE_TO_FRAM;
297         SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
298         return error;
299     }
300     InitSavePeriodTimes();
301     SendAckPacket(ACK_SET_NEW_TLM_PERIOD, cmd, NULL, 0);
302     return 0;
303 }
304
305 /*
306 * Get telemetry period times from FRAM
307 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the headers we add)
308 * @return type=int; according to FRAM_read and TransmitDataAsSPL_Packet
309 */
310 int CMD_GetTLMPeriodTimes(sat_packet_t *cmd)
311 {
312     int error = 0;
313     int arrPeriod[7] = {0};
314     error = logError(FRAM_read((unsigned char*)arrPeriod,
TLM_SAVE_PERIOD_START_ADDR, sizeof(arrPeriod)), "CMD_GetTLMPeriodTimes -
FRAM_read");
315     if(error)
316     {
317         unsigned char ackError = ERROR_READ_FROM_FRAM;
318         SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
319         return error;
320     }
321     return SendAckPacket(ACK_GET_TLM_SAVE_PERIODS, cmd, (unsigned char
*)arrPeriod, sizeof(arrPeriod));
322 }

```

```

323
324  /*
325  * Switch between the sd cards.
326  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
327  * use to find all the required information (the headers we add and sd to switch to)
328  * @return type=int;  -1 cmd null
329  *                    27 wrong data length
330  *                    -4 invalid sd
331  *                    -2 read from FRAM
332  *                    -3 write to FRAM
333  *                    according to Hard_ComponenetReset
334  */
335  int CMD_SwitchSD_card(sat_packet_t *cmd)
336  {
337      if(cmd == NULL) return -1;
338      unsigned char ackError = 0;
339      if(cmd->length != 1)
340      {
341          ackError = ERROR_WRONG_LENGTH_DATA;
342          SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
343          return ackError;
344      }
345      uint8_t newSD = cmd->data[0];
346      uint8_t oldSD;
347      if(newSD != 1 && newSD != 0)
348      {
349          unsigned char ackError = ERROR_NOT_VALID_SD;
350          SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
351          return -4;
352      }
353      if(logError(FRAM_read((unsigned char*)&oldSD, SD_CARD_USED_ADDR,
354                          SD_CARD_USED_SIZE), "CMD_SwitchSD_card - FRAM_read"))
355      {
356          unsigned char ackError = ERROR_READ_FROM_FRAM;
357          SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
358          return -2;
359      }
360      if(oldSD == newSD)
361      {
362          unsigned char ackError = ERROR_SAME_SD_WE_USE;
363          SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
364          return 0;
365      }
366      if(logError(FRAM_writeAndVerify((unsigned char*)&newSD,
367                                      SD_CARD_USED_ADDR, SD_CARD_USED_SIZE), "CMD_SwitchSD_card -
368      FRAM_writeAndVerify"))

```

```

365 {
366     unsigned char ackError = ERROR_WRITE_TO_FRAM;
367     SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
368     return -3;
369 }
370 SendAckPacket(ACK_SWITCHING_SD_CARD, cmd, NULL, 0);
371 return Hard_ComponenetReset(cmd);
372 }
373
374 /*
375 * Delete tlm files from sd according to tlmType
376 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
377 * use to find all the required information (the headers we add and tlmType)
378 * @return type=int; -1 cmd null
379 *          17 wrong data length
380 *          30 can't do
381 *          according to SendAckPacket errors.
382 *          0 on success
383 */
384 int CMD_DeleteFilesOfType(sat_packet_t *cmd)
385 {
386     if(cmd == NULL) return -1;
387     unsigned char ackError = 0;
388     if(cmd->length != 1)
389     {
390         ackError = ERROR_WRONG_LENGTH_DATA;
391         SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
392         return ackError;
393     }
394     uint8_t type;
395     memcpy(&type, cmd->data, 1);
396     if(type > tlm_log)
397     {
398         ackError = ERROR_CANT_DO;
399         SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
400         return ackError;
401     }
402     char endFile[3] = {0};
403     int structNotNeeded;
404     GetTlmTypeInfo(type, endFile, &structNotNeeded);
405     F_FIND find;
406     char fileFind[5] = {0};
407     sprintf(fileFind, "%.5s", endFile);
408     f_findfirst(fileFind, &find);
409     do

```



```
410 {  
411     f_delete(find.filename);  
412 }  
413 while(!f_findnext(&find));  
414 return SendAckPacket(ACK_DELETE_TLM, cmd, NULL, 0);  
415 }  
416
```

```

1
2  #ifndef MAINTENANCE_COMMANDS_H_
3  #define MAINTENANCE_COMMANDS_H_
4
5  #include "SubSystemModules/Communication/SatCommandHandler.h"
6
7  #include "SubSystemModules/Communication/AckHandler.h"
8
9  #include "SubSystemModules/Communication/TRXVU.h"
10
11 #include "SubSystemModules/PowerManagment/EPS.h"
12
13 #include "SubSystemModules/Maintenance/Maintenance.h"
14
15 #include <hal/Timing/Time.h>
16 #include <hal/supervisor.h>
17
18 #define MIN_GROUND_WDT (3*24*3600) //3 days
19
20 typedef enum __attribute__((__packed__)) reset_type_t
21 {
22     reset_software, //from eps
23     reset_hardware, //from eps
24     reset_tx_hard,
25     reset_rx_hard,
26     reset_ants,
27     reset_payload,
28     reset_payload_hard,
29     reset_fram,
30     reset_filesystem
31 }reset_type_t;
32
33 /*int CMD_GenericI2C(sat_packet_t *cmd);*/
34
35 /*
36  * Set the time on the OBC
37  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
38  * use to find all the required information (like the headers we add and the new time)
39  * @return type=int;    -1 on cmd Null
40  *                      -3 on wrong length
41  *                      1 if cant set new time
42  *                      type of error according to <hal/error.h>
43  *                      0 on success.
44  */

```

```

40     int CMD_UpdateSatTime(sat_packet_t *cmd);
41
42     /*
43  * Get the time save on the sat at the sat.
44  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (like the headers we add)
45  * @return type=int; return type of error 0 on success.
46  */
47     int CMD_GetSatTime(sat_packet_t *cmd);
48
49     /*
50  * Get the time the OBC was active since the last reset
51  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (like the headers we add)
52  * @return type=int; return type of error according to <hal/error.h> or
TransmitDataAsSPL_Packet errors
53  *         0 on success.
54  */
55     int CMD_GetSatUptime(sat_packet_t *cmd);
56
57
58     /*help functions of CMD_ResetComponent*/
59
60     int HardPayload_ComponenetReset(sat_packet_t *cmd);
61     int Payload_ComponenetReset(sat_packet_t *cmd);
62
63     int HardTX_ComponenetReset(sat_packet_t *cmd);
64
65     int HardRX_ComponenetReset(sat_packet_t *cmd);
66
67     int Soft_ComponenetReset(sat_packet_t *cmd);
68
69     int Hard_ComponenetReset(sat_packet_t *cmd);
70
71     int Ants_ComponenetReset(sat_packet_t *cmd);
72
73     int FRAM_ComponenetReset(sat_packet_t *cmd);
74
75     int FS_ComponenetReset(sat_packet_t *cmd);
76
77     /*!
78  * @brief     starts a reset according to 'reset_type_t'
79  *           The function will raise the reset flag and send an ACK before starting the
reset.
80  * @return    0 on success

```

```

81 *          Error code according to <hal/errors.h>
82 */
83     int CMD_ResetComponent(sat_packet_t *cmd);
84
85
86     /*
87  * Set new ground WDT time
88  * @param[in] name=cmd; type=sat_packet_t*; The packet the sat got and use to
find all the required information (like the headers we add, and the new WDT)
89  * @return type=int;  -1 on cmd NULL;
90  *                   -2 on wrong length data
91  *                   -3 cant do
92  *                   -4 write to fram
93  *                   error according to SendAckPacket
94  */
95     int CMD_SetGsWdtKickTime(sat_packet_t* cmd);
96
97     /*
98  * Get the ground WDT duration;
99  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (like the headers we add)
100 * @return type=int; return -1 on read from fram, error according to SendAckPacket
101 */
102     int CMD_GetGsWdtKickTime(sat_packet_t* cmd);
103
104     /*
105  * Set first active flags
106  * @return type=int; sum of errors in minus
107  */
108     int CMD_SetFirstActiveFlags();
109
110     #endif /* MAINTENANCE_COMMANDS_H_ */
111

```

```

1 /*
2  * Maintenance_Commands.c
3  *
4  * Created on: 3 Dec 2024
5  *   Author: maayan
6  */
7
8  #include "Maintenance_Commands.h"
9  #include "SubSystemModules/Maintenance/Maintenance.h"
10
11  #include <string.h>
12
13  /*
14   * Get the time save on the sat at the sat.
15   * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
16   * use to find all the required information (like the headers we add)
17   * @return type=int; return type of error 0 on success.
18   */
19  int CMD_GetSatTime(sat_packet_t *cmd)
20  {
21      time_unix timeNow;
22      int error = logError(Time_getUnixEpoch((unsigned int*)&timeNow),
23                          "CMD_GetSatTime - Time_getUnixEpoch");
24      if(error)
25      {
26          unsigned char ackError = ERROR_CANT_GET_TIME;
27          SendAckPacket(ACK_ERROR_MSG , cmd, &ackError, sizeof(ackError)); //
28          Send ack error according to "AckErrors.h"
29          return error;
30      }
31      return SendAckPacket(ACK_GET_SAT_TIME, cmd, (unsigned char*)&timeNow,
32                          sizeof(time_unix)); //send back the sat time
33  }
34
35  /*help functions of CMD_ResetComponent*/
36
37  int Payload_ComponenetReset(sat_packet_t *cmd)
38  {
39      SendAckPacket(ACK_PAYLOAD_SOFT_RESET, cmd, NULL, 0);
40      return logError(payloadSoftReset(), "Payload_ComponenetReset -
41      payloadSoftReset");
42  }
43
44  int HardPayload_ComponenetReset(sat_packet_t *cmd)
45  {
46      SendAckPacket(ACK_PAYLOAD_HARD_RESET, cmd, NULL, 0);
47  }

```

```

40  payloadTurnOff();
41  return payloadTurnOn();
42 }
43
44  int HardTX_ComponenetReset(sat_packet_t *cmd)
45 {
46  SendAckPacket(ACK_TX_HARD_RESET, cmd, NULL, 0);
47  logError(isis_vu_e__reset_hw_tx(0), "HardTX_ComponenetReset -
isis_vu_e__reset_hw_tx");
48  return InitTrxvuAndAnts();
49 }
50
51  int HardRX_ComponenetReset(sat_packet_t *cmd)
52 {
53  SendAckPacket(ACK_RX_HARD_RESET, cmd, NULL, 0);
54  logError(isis_vu_e__reset_hw_rx(0), "HardRX_ComponenetReset -
isis_vu_e__reset_hw_rx");
55  return InitTrxvuAndAnts();
56 }
57
58  int Soft_ComponenetReset(sat_packet_t *cmd)
59 {
60  Boolean true = TRUE; Boolean false = FALSE;
61  SendAckPacket(ACK_SOFT_RESET, cmd, NULL, 0);
62  logError(FRAM_writeAndVerify((unsigned char*)&true,
RESET_CMD_FLAG_ADDR, RESET_CMD_FLAG_SIZE), "Soft_ComponenetReset -
cmd reset flag");
63  logError(FRAM_writeAndVerify((unsigned char*)&>false,
HAD_RESET_IN_A_MINUTE_ADDR, HAD_RESET_IN_A_MINUTE_SIZE),
"Soft_ComponenetReset - FRAM_writeAndVerify");
64  restart();
65  vTaskDelay(50);
66  return 0;
67 }
68
69  int Ants_ComponenetReset(sat_packet_t *cmd)
70 {
71  SendAckPacket(ACK_ANTS_RESET, cmd, NULL, 0);
72  int err = logError(isis_ants__reset(0), "Ants_ComponenetReset -
isis_ants__reset - 0");
73  err += logError(isis_ants__reset(1), "Ants_ComponenetReset - isis_ants__reset -
1");
74  return err;
75 }
76
77  int Hard_ComponenetReset(sat_packet_t *cmd)

```

```

78 {
79  isismepsv2_ivid7_piu__replyheader_t replyheader;
80  Boolean true = TRUE; Boolean false = FALSE;
81  logError(FRAM_writeAndVerify((unsigned char*)&true,
RESET_CMD_FLAG_ADDR, RESET_CMD_FLAG_SIZE), "Hard_ComponenetReset -
cmd reset flag");
82  logError(FRAM_writeAndVerify((unsigned char*)&>false,
HAD_RESET_IN_A_MINUTE_ADDR, HAD_RESET_IN_A_MINUTE_SIZE),
"Hard_ComponenetReset - FRAM_writeAndVerify");
83  SendAckPacket(ACK_HARD_RESET, cmd, NULL, 0);
84  return logError(isismepsv2_ivid7_piu__reset(0, &replyheader),
"Hard_ComponenetReset - isismepsv2_ivid7_piu__reset");
85 }
86
87  int FS_ComponenetReset(sat_packet_t *cmd)
88 {
89  SendAckPacket(ACK_FS_RESET, cmd, NULL, 0);
90  DeInitializeFS();
91  return InitializeFS();
92 }
93
94  int FRAM_ComponenetReset(sat_packet_t *cmd)
95 {
96  SendAckPacket(ACK_FRAM_RESET, cmd, NULL, 0);
97  FRAM_stop();
98  return logError(FRAM_start(), "FRAM_ComponenetReset - FRAM_start");
99 }
100
101  int SendErrorCantReset(sat_packet_t *cmd)
102 {
103  unsigned char ackError = ERROR_CANT_RESET;
104  SendAckPacket(ACK_ERROR_MSG , cmd, &ackError, sizeof(ackError)); // Send
ack error according to "AckErrors.h"
105  return ackError;
106 }
107
108
109  /*!
110 * @brief   starts a reset according to 'reset_type_t'
111 *         The function will raise the reset flag and send an ACK before starting the
reset.
112 * @return  0 on success
113 *         Error code according to <hal/errors.h>
114 */
115  int CMD_ResetComponent(sat_packet_t *cmd)
116 {

```

```

117 unsigned char ackError = 0;
118 if(cmd->length != 1)
119 {
120     ackError = ERROR_WRONG_LENGTH_DATA;
121     SendAckPacket(ACK_ERROR_MSG , cmd, &ackError, sizeof(ackError)); //
Send ack error according to "AckErrors.h"
122     return ackError;
123 }
124 uint8_t type;
125 memcpy(&type, cmd->data, cmd->length);
126 MostCurrentTimeToFRAM();
127 switch(type)
128 {
129     case reset_software:
130         return Soft_ComponenetReset(cmd);
131     case reset_hardware:
132     {
133         if(Hard_ComponenetReset(cmd))
134             return SendErrorCantReset(cmd);
135         else
136             vTaskDelay(50000);
137         return 0;
138     }
139     case reset_tx_hard:
140     {
141         if(HardTX_ComponenetReset(cmd))
142             return SendErrorCantReset(cmd);
143         return 0;
144     }
145     case reset_rx_hard:
146     {
147         if(HardRX_ComponenetReset(cmd))
148             return SendErrorCantReset(cmd);
149         return 0;
150     }
151     case reset_ants:
152     {
153         if(Ants_ComponenetReset(cmd))
154             return SendErrorCantReset(cmd);
155         return 0;
156     }
157     case reset_payload:
158     {
159         if(Payload_ComponenetReset(cmd))
160             return SendErrorCantReset(cmd);
161         return 0;

```



```

162     }
163     case reset_payload_hard:
164     {
165         if(HardPayload_ComponenetReset(cmd))
166             return SendErrorCantReset(cmd);
167         return 0;
168     }
169     case reset_frame:
170     {
171         if(FRAM_ComponenetReset(cmd))
172             return SendErrorCantReset(cmd);
173         return 0;
174     }
175     case reset_filesystem:
176     {
177         if(FS_ComponenetReset(cmd))
178             return SendErrorCantReset(cmd);
179         return 0;
180     }
181     default:
182     {
183         ackError = ERROR_CANT_DO;
184         SendAckPacket(ACK_ERROR_MSG , cmd, (unsigned char*)&ackError,
sizeof(ackError)); // Send ack error according to "AckErrors.h"
185         return ackError;
186     }
187 }
188 }
189
190 /*
191  * Get the time the OBC was active since the last reset
192  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (like the headers we add)
193  * @return type=int; return type of error according to <hal/error.h> or
TransmitDataAsSPL_Packet errors
194  *         0 on success.
195  */
196 int CMD_GetSatUptime(sat_packet_t *cmd)
197 {
198     supervisor_housekeeping_t supervisorHK; //create a variable that is the struct
we need from supervisor
199     int error = logError(Supervisor_getHousekeeping(&supervisorHK,
SUPERVISOR_SPI_INDEX), "CMD_GetSatUptime - Supervisor_getHousekeeping"); //
gets the variables to the struct and also check error.

```

```

200     if(error)
201     {
202         unsigned char ack_error = ERROR_GET_FROM_STRUCT;
203         SendAckPacket(ACK_ERROR_MSG , cmd, (unsigned char*)&ack_error,
sizeof(ack_error)); // Send ack error according to "AckErrors.h"

204         return error;
205     }
206     return SendAckPacket(ACK_GET_SAT_UPTIME, cmd, (unsigned char
*)&supervisorHK.fields.iobcUptime, sizeof(supervisorHK.fields.iobcUptime)); //send
back the sat time
207 }
208
209
210     /*
211     * Set the time on the OBC
212     * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (like the headers we add and the new time)
213     * @return type=int;    -1 on cmd Null
214     *                      -3 on wrong length
215     *                      1 if cant set new time
216     *                      type of error according to <hal/error.h>
217     *                      0 on success.
218     */
219     int CMD_UpdateSatTime(sat_packet_t *cmd)
220 {
221     //can sat at min on 1.1.2000 00:00:00 start at 1.1.1970 so the min time in
seconds is 946,728,000 or in hex 386D EC40 and we need it as opposite so 40 EC 6D
38
222     if(cmd == NULL) return -1;
223     unsigned char error_ack = 0;
224     if(cmd->length != 4)
225     {
226         error_ack = ERROR_WRONG_LENGTH_DATA;
227         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
228         return -3;
229     }
230     time_unix passSatTime;
231     logError(Time_getUnixEpoch((unsigned int*)&passSatTime),
"CMD_UpdateSatTime - Time_getUnixEpoch");
232     time_unix newSatTime;
233     memcpy((unsigned char*)&newSatTime, cmd->data, cmd->length);
234     int error = logError(Time_setUnixEpoch(newSatTime), "CMD_UpdateSatTime -
Time_setUnixEpoch");
235     if(error)

```

```

236 {
237     error_ack = ERROR_COULDNT_UPDATE_SAT_TIME;
238     SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
239     return 1;
240 }
241 MostCurrentTimeToFRAM();
242 KickGroundCommWDT();
243 if(newSatTime < passSatTime)
244     Delete_allTMFilesFromSD();
245 return SendAckPacket(ACK_UPDATE_TIME , cmd, (unsigned char
*)&newSatTime, sizeof(newSatTime));
246
247 }
248
249 /*
250 * Set new ground WDT time
251 * @param[in] name=cmd; type=sat_packet_t*; The packet the sat got and use to
find all the required information (like the headers we add, and the new WDT)
252 * @return type=int; -1 on cmd NULL;
253 * -2 on wrong length data
254 * -3 cant do
255 * -4 write to fram
256 * error according to SendAckPacket
257 */
258 int CMD_SetGsWdtKickTime(sat_packet_t* cmd)
259 {
260     if(cmd == NULL) return -1;
261     unsigned char error_ack = 0;
262     if(cmd->length != 4)
263     {
264         error_ack = ERROR_WRONG_LENGTH_DATA;
265         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
266         return -2;
267     }
268     unsigned int newGroundWDT;
269     memcpy((unsigned char*)&newGroundWDT, cmd->data, cmd->length);
270     if(newGroundWDT < MIN_GROUND_WDT)
271     {
272         error_ack = ERROR_CANT_DO;
273         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
274         return -3;
275     }
276     if(logError(FRAM_writeAndVerify((unsigned char*)&newGroundWDT,

```

```

NO_COMM_WDT_KICK_TIME_ADDR, NO_COMM_WDT_KICK_TIME_SIZE),
"CMD_SetGsWdtKickTime - FRAM_writeAndVerify"))
277 {
278     error_ack = ERROR_WRITE_TO_FRAM;
279     SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
280     return -4;
281 }
282 return SendAckPacket(ACK_SET_GWDT, cmd, NULL, 0);
283 }
284
285 /*
286 * Get the ground WDT duration;
287 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (like the headers we add)
288 * @return type=int; return -1 on read from fram, error according to SendAckPacket
289 */
290 int CMD_GetGsWdtKickTime(sat_packet_t* cmd)
291 {
292     unsigned int groundWDT = 0;
293     if(logError(FRAM_read((unsigned char*)&groundWDT,
NO_COMM_WDT_KICK_TIME_ADDR, NO_COMM_WDT_KICK_TIME_SIZE),
"CMD_GetGsWdtKickTime - FRAM_read"))
294     {
295         unsigned char ackError = ERROR_READ_FROM_FRAM;
296         SendAckPacket(ACK_ERROR_MSG , cmd, &ackError, sizeof(ackError)); //
Send ack error according to "AckErrors.h"
297         return -1;
298     }
299     return SendAckPacket(ACK_GET_GROUND_WDT, cmd, (unsigned char
*)&groundWDT, sizeof(groundWDT));
300
301 }
302
303 /*
304 * Set first active flags
305 * @return type=int; sum of errors in minus
306 */
307 int CMD_SetFirstActiveFlags()
308 {
309     int error = 0;
310     Boolean false = FALSE;
311     logError(FRAM_writeAndVerify((unsigned char*)&>false,
PAYLOAD_IS_DEAD_ADDR, PAYLOAD_IS_DEAD_SIZE), "CMD_SetFirstActiveFlags -
FRAM_writeAndVerify");
312     Boolean true = TRUE;

```

```
313  logError(FRAM_writeAndVerify((unsigned char*)&true,
FIRST_ACTIVATION_FLAG_ADDR, FIRST_ACTIVATION_FLAG_SIZE),
"CMD_SetFirstActiveFlags - first activation flag = 1");
314  int zero = 0;
315  logError(FRAM_writeAndVerify((unsigned char*)&zero,
SECONDS_SINCE_DEPLOY_ADDR, SECONDS_SINCE_DEPLOY_SIZE),
"CMD_SetFirstActiveFlags - FRAM_writeAndVerify"); //TODO: need to be written before
not here (like firstActiveFlag)
316  logError(FRAM_writeAndVerify((unsigned char*)&>false,
HAD_RESET_IN_A_MINUTE_ADDR, HAD_RESET_IN_A_MINUTE_SIZE),
"CMD_SetFirstActiveFlags - FRAM_writeAndVerify");
317
318  int minusOne = -1;
319  logError(FRAM_writeAndVerify((unsigned char*)&minusOne,
NUMBER_OF_RESETS_ADDR, NUMBER_OF_RESETS_SIZE),
"CMD_SetFirstActiveFlags - FRAM_writeAndVerify");
320
321  logError(FRAM_writeAndVerify((unsigned char*)&zero,
NUMBER_OF_CMD_RESETS_ADDR, NUMBER_OF_CMD_RESETS_ADDR),
"CMD_SetFirstActiveFlags - FRAM_writeAndVerify");
322
323  SendAckPacket(ACK_READY_FOR_TAKEOF, NULL, NULL, 0);
324  return error;
325 }
326
```

```

1 /*
2  * Payload_Commands.h
3  *
4  * Created on: 2 Jan 2025
5  * Author: maayan
6  */
7
8 #ifndef PAYLOAD_COMMANDS_H_
9 #define PAYLOAD_COMMANDS_H_
10
11 #include "SubSystemModules/Payload/payload_drivers.h"
12
13 #include "SubSystemModules/Communication/AckHandler.h"
14
15 /*
16  * turn on the flag of kill payload and turn payload off
17  * @param[in] name=cmd; type=sat_packet_t*; The packet the sat got and use to
18  * find all the required information (the headers we add)
19  * @return type=int; 2 on write_to_frame problem
20  * error according to SendAckPacket
21  */
22 int CMD_TurnOnKillPayloadFlag(sat_packet_t *cmd);
23
24 /*
25  * turn off the flag of kill payload and turn payload on
26  * @param[in] name=cmd; type=sat_packet_t*; The packet the sat got and use to
27  * find all the required information (the headers we add)
28  * @return type=int; 2 on write_to_frame problem
29  * error according to SendAckPacket
30  */
31 int CMD_TurnOffKillPayloadFlag(sat_packet_t *cmd);
32
33 #endif /* PAYLOAD_COMMANDS_H_ */

```

```

1 /*
2  * Payload_Commands.c
3  *
4  * Created on: 2 Jan 2025
5  * Author: maayan
6  */
7
8 #include "Payload_Commands.h"
9 #include <hal/supervisor.h>
10
11 /*
12  * turn on the flag of kill payload and turn payload off
13  * @param[in] name=cmd; type=sat_packet_t*; The packet the sat got and use to
14  * find all the required information (the headers we add)
15  * @return type=int; 2 on write_to_frame problem
16  * error according to SendAckPacket
17  */
18 int CMD_TurnOnKillPayloadFlag(sat_packet_t *cmd)
19 {
20     Boolean true = TRUE;
21     if(logError(FRAM_writeAndVerify((unsigned char*)&true,
22     PAYLOAD_IS_DEAD_ADDR, PAYLOAD_IS_DEAD_SIZE),
23     "CMD_TurnOnKillPayloadFlag - FRAM_writeAndVerify"))
24     {
25         unsigned char ackError = ERROR_WRITE_TO_FRAME;
26         SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
27         return ackError;
28     }
29     logError(payloadTurnOff(), "CMD_TurnOnKillPayloadFlag - payloadTurnOff");
30     return SendAckPacket(ACK_KILL_PAYLOAD, cmd, NULL, 0);
31 }
32
33 /*
34  * turn off the flag of kill payload and turn payload on
35  * @param[in] name=cmd; type=sat_packet_t*; The packet the sat got and use to
36  * find all the required information (the headers we add)
37  * @return type=int; 2 on write_to_frame problem
38  * error according to SendAckPacket
39  */
40 int CMD_TurnOffKillPayloadFlag(sat_packet_t *cmd)
41 {
42     Boolean false = FALSE;
43     if(logError(FRAM_writeAndVerify((unsigned char*)&>false,
44     PAYLOAD_IS_DEAD_ADDR, PAYLOAD_IS_DEAD_SIZE),
45     "CMD_TurnOffKillPayloadFlag - FRAM_writeAndVerify"))

```

```
41 {
42     unsigned char ackError = ERROR_WRITE_TO_FRAM;
43     SendAckPacket(ACK_ERROR_MSG, cmd, &ackError, sizeof(ackError));
44     return ackError;
45 }
46 payloadTurnOn();
47 return SendAckPacket(ACK_TURN_ON_PAYLOAD, cmd, NULL, 0);
48 }
49
```



```

1
2  #ifndef TRXVU_COMMANDS_H_
3  #define TRXVU_COMMANDS_H_
4
5  #include "SubSystemModules/Communication/TRXVU.h"
6
7  #include "SubSystemModules/Communication/AckHandler.h"
8
9  #include "SubSystemModules/Communication/SatCommandHandler.h"
10 #include "SubSystemModules/Communication/AckErrors.h"
11
12 #include <hal/Drivers/I2C.h>
13 #include <string.h>
14
15 // wait time for taking semaphores
16 #define WAIT_TIME_SEM_DUMP 3
17 #define WAIT_TIME_SEM_TX 3
18
19 #define ANTENNA_DEPLOYMENT_TIMEOUT 10 //<! in seconds
20
21 /**
22  * set trxvu idle on with time end
23  * @param[in] name=cmd; type=sat_packet_t*; The packet the sat got and use to
24  find all the required information (like the idle duration and the headers we add)
25  * @return type=int; -6 on cmd null
26  *
27  * other according to SetIdleState error list.
28  * or SendAckPacket
29  */
30 int CMD_SetOn_IdleState(sat_packet_t *cmd);
31
32 /* set trxvu idle off
33 * @param[in] name=cmd; type=sat_packet_t*; The packet the sat got and use to
34 find all the required information (like the headers we add)
35 * @return type=int; according to SetIdleState error list.
36 *
37 * or SendAckPacket if we don't have error in the SetIdleState
38 */
39 int CMD_SetOff_IdleState(sat_packet_t *cmd);
40
41 /*
42 * Set transmitter to transponder state for the time written in the data in cmd
43 * @param[in] name=cmd; type=sat_packet_t*; The packet the sat got and use to
44 find all the required information (like the mute duration and the headers we add)
45 * @return type=int; -1 on cmd NULL
46 *
47 * -2 on written wrong number to FRAM

```

```

40 *          -3 on incorrect length
41 *          -4 can't read from FRAM
42 *          -5 can't write to FRAM
43 *          errors according to I2C_write
44 */
45     int CMD_SetOn_Transponder(sat_packet_t *cmd);
46
47     /*
48 * The command sets the transponder off.
49 * @param[in and out] name=cmd; type=sat_packet_t; The packet the sat got and
50 * use to find all the required information (like the headers we add)
51 * @return type=int; return type of error and if the parameter is NULL return -1.
52 */
52     int CMD_SetOff_Transponder(sat_packet_t *cmd);
53
54     /*
55 * The command change the rssi value in the FRAM and check it change correctly.
56 * also check the new rssi value is between the 0 and 4095
57 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
58 * use to find all the required information (like the new rssi val and the headers we add)
59 * @return type=int; return type of error and if the parameter is NULL return -1 and
60 * on error in setTransponderRSSIinFRAM, -2 if length isn't in size.
61 */
61     int CMD_SetRSSI_Transponder(sat_packet_t *cmd);
62
63     /*
64 * The command change the rssi value in the FRAM to default and check it change
65 * correctly.
66 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
67 * use to find all the required information and set the data to default rssi val (the headers
68 * we add)
69 * @return type=int; return type of error and if the parameter is NULL return -1 and
70 * on error in setTransponderRSSIinFRAM, -2 if length isn't in size.
71 */
71     int CMD_SetRSSI_Transponder_DEFAULT(sat_packet_t *cmd);
72
73     /*
74 * The command get the rssi value from the FRAM and send it back.
75 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
76 * use to find all the required information (like the headers we add)
77 * @return type=int; return type of error 0 on success and if the parameter (cmd) is
78 * NULL return -2, if cant read from fram return -1.
79 */
79     int CMD_GetRSSI_Transponder(sat_packet_t *cmd);
80
81     /*

```

```

76 * The command change the end time of mute to unmute the transmitter.
77 * @param[in] name=cmd; type=sat_packet_t*; The packet the sat got and use to
find all the required information (like the headers we add)
78 * @return type=int; return type of error.
79 */
80 int CMD_UnMuteTRXVU(sat_packet_t *cmd);
81
82 /*
83 * Set transmitter to mute for the time written in the data in cmd
84 * @param[in] name=cmd; type=sat_packet_t*; The packet the sat got and use to
find all the required information (like the mute duration and the headers we add)
85 * @return type=int; -1 on cmd NULL
86 * -3 on incorrect length
87 * errors according to setMuteEndTime
88 */
89 int CMD_MuteTRXVU(sat_packet_t *cmd);
90
91 /*
92 * The command change the beacon interval in the FRAM to default interval
93 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the headers we add)
94 * @return type=int; return type of error.
95 */
96 int CMD_SetBeacon_Interval_DEFAULT(sat_packet_t *cmd);
97
98 /*
99 * The command change the beacon interval in the FRAM and check it change
correctly. also check the new interval is between the max and min interval
100 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (like the new_interval and the headers we add)
101 * @return type=int; return type of error and if the parameter is NULL return -1.
102 */
103 int CMD_SetBeacon_Interval(sat_packet_t *cmd);
104
105 /*
106 * The command get the beacon interval from the FRAM and send it back.
107 * @param[in and out] name=cmd; type=sat_packet_t; The packet the sat got and
use to find all the required information (like the headers we add)
108 * @return type=int; return type of error and if the parameter (cmd) is NULL return
-1.
109 */
110 int CMD_GetBeacon_Interval(sat_packet_t *cmd);
111
112 /*
113 * Get transmitter uptime and send to ground
114 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and

```

```

use to find all the required information (like the headers we add)
115 * @return type=int; return type of error according to <hal/errors.h>
116 */
117     int CMD_GetTxUptime(sat_packet_t *cmd);
118
119     /*
120 * Get receiver uptime and send to ground
121 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (like the headers we add)
122 * @return type=int; return type of error according to <hal/errors.h>
123 */
124     int CMD_GetRxUptime(sat_packet_t *cmd);
125
126     //int CMD_DeleteAllDelayedBuffer(sat_packet_t *cmd);

127
128     /*int CMD_AntGetArmStatus(sat_packet_t *cmd);*/

129
130     /*
131 * Helper function to get side of ants.
132 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the ant side and the headers we add)
133 * @param[out] name=side; type=char*; here we left the side for further use.
134 */
135     int GetAntSide(sat_packet_t *cmd, char *side);
136
137     /*
138 * Gets Ant uptime according to side.
139 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the ant side and the headers we add)
140 * @return type=int; return type of error according to this
141 *                                     -1 cmd NULL
142 *                                     -2 if length isn't in size.
143 *                                     -3 got wrong side (not exist one)
144 *                                     else from <hal/errors.h>
145 */
146     int CMD_AntGetUptime(sat_packet_t *cmd);
147
148     /*
149 * Send ack ping
150 * @param[in] name=cmd; type=sat_packet_t*; Not needed can be NULL
151 * @return type=int; according to errors <hal/errors.h>
152 */
153     int CMD_Ping(sat_packet_t *cmd);
154

```

```
155     /*
156  * turn up the flag of stop deploy ants.
157  * @return type=int; return error according to SendAckpacket;
158  */
159     int CMD_OffAntsDeploy(sat_packet_t *cmd);
160
161     #endif
162     /*COMMANDS_H_*/
163
```

```

1 /*
2  * TRXVU_Commands.c
3  *
4  * Created on: 3 9 2024
5  * Author: maayan
6  */
7
8  #include "TRXVU_Commands.h"
9
10 /**
11  * set trxvu idle on with time end
12  * @param[in] name=cmd; type=sat_packet_t*; The packet the sat got and use to
find all the required information (like the idle duration and the headers we add)
13  * @return type=int; -6 on cmd null
14  *          other according to SetIdleState error list.
15  *          or SendAckPacket
16  */
17  int CMD_SetOn_IdleState(sat_packet_t *cmd)
18 {
19     unsigned char error_ack;
20     if(cmd == NULL)
21         return -6;
22     if(cmd->length != 4)
23     {
24         error_ack = ERROR_WRONG_LENGTH_DATA;
25         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
26         return -3;
27     }
28     int duration;
29     memcpy(&duration, cmd->data, cmd->length);
30     if(duration > MAX_IDLE_TIME) duration = MAX_IDLE_TIME;
31     int error = SetIdleState(isis_vu_e__onoff__on, duration);
32     switch(error)
33     {
34     case 0:
35         break;
36     case -1:
37     {
38         error_ack = ERROR_CANT_DO;
39         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack));
// Send ack error according to "AckErrors.h"
40         return error;
41     }
42     case -2:
43     {

```

```

44     error_ack = ERROR_READ_FROM_FRAM;
45     SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack));
// Send ack error according to "AckErrors.h"
46     return error;
47 }
48 case -3:
49 {
50     error_ack = ERROR_WRITE_TO_FRAM;
51     SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack));
// Send ack error according to "AckErrors.h"
52     return error;
53 }
54 case -4:
55 {
56     error_ack = ERROR_WRITTEN_IN_FRAM_WRONG;
57     SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack));
// Send ack error according to "AckErrors.h"
58     return error;
59 }
60 case -5:
61 {
62     error_ack = ERROR_CANT_GET_TIME;
63     SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack));
// Send ack error according to "AckErrors.h"
64     return error;
65 }
66 default:
67 {
68     error_ack = UNKNOWN_ERROR;
69     SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack));
70     return error;
71 }
72 }
73 return SendAckPacket(ACK_IDLE_ON , cmd, (unsigned char*)&duration, sizeof
(duration));
74 }
75
76 /* set trxvu idle off
77 * @param[in] name=cmd; type=sat_packet_t*; The packet the sat got and use to
find all the required information (like the headers we add)
78 * @return type=int; according to SetIdleState error list.
79 *         or SendAckPacket if we don't have error in the SetIdleState
80 */
81 int CMD_SetOff_IdleState(sat_packet_t *cmd)
82 {
83     unsigned char error_ack;

```

```

84  int error = SetIdleState(isis_vu_e__onoff__off, 0);
85  switch(error)
86  {
87      case 0:
88          break;
89      case -1:
90          {
91              error_ack = ERROR_CANT_DO;
92              SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack));
93              // Send ack error according to "AckErrors.h"
94              return error;
95          }
96      case -2:
97          {
98              error_ack = ERROR_READ_FROM_FRAM;
99              SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack));
100             // Send ack error according to "AckErrors.h"
101             return error;
102         }
103     case -3:
104         {
105             error_ack = ERROR_WRITE_TO_FRAM;
106             SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack));
107             // Send ack error according to "AckErrors.h"
108             return error;
109         }
110     case -4:
111         {
112             error_ack = ERROR_WRITTEN_IN_FRAM_WRONG;
113             SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack));
114             // Send ack error according to "AckErrors.h"
115             return error;
116         }
117     default:
118         {
119             error_ack = ERROR_CANT_GET_TIME;
120             SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack));
121             // Send ack error according to "AckErrors.h"
122             return error;
123         }
124 }
125 return SendAckPacket(ACK_IDLE_OFF , cmd, NULL, 0);
126 }
127
128 /*
129 * Set transmitter to transponder state for the time written in the data in cmd

```



```

125 * @param[in] name=cmd; type=sat_packet_t*; The packet the sat got and use to
find all the required information (like the mute duration and the headers we add)
126 * @return type=int; -1 on cmd NULL
127 *          -2 on written wrong number to FRAM
128 *          -3 on incorrect length
129 *          -4 can't read from FRAM
130 *          -5 can't write to FRAM
131 *          errors according to I2C_write
132 */
133 int CMD_SetOn_Transponder(sat_packet_t *cmd)
134 {
135     unsigned char error_ack;
136     if(cmd == NULL)
137         return -1;
138     if(cmd->length != 4)
139     {
140         error_ack = ERROR_WRONG_LENGTH_DATA;
141         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
142         return -3;
143     }
144     time_unix duration;
145     memcpy(&duration, cmd->data, cmd->length);
146     if(duration > MAX_TRANS_TIME)
147         duration = MAX_TRANS_TIME;
148     int timeNow;
149     logError(Time_getUnixEpoch((unsigned int*)&timeNow),
"CMD_SetOn_Transponder - Time_getUnixEpoch");
150     duration += timeNow;
151     if(logError(FRAM_write((unsigned char*)&duration,
TRANSPONDER_END_TIME_ADDR, TRANSPONDER_END_TIME_SIZE),
"CMD_SetOn_Transponder - FRAM_write"))
152     {
153         error_ack = ERROR_WRITE_TO_FRAM;
154         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
155         return -5;
156     }
157     time_unix check = getTransponderEndTime();
158     if(!check)
159     {
160         error_ack = ERROR_READ_FROM_FRAM;
161         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
162         return -4;
163     }

```

```

164  if(check != duration)
165  {
166      error_ack = ERROR_WRITTEN_IN_FRAM_WRONG;
167      SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
168      return -2;
169  }
170  SendAckPacket(ACK_ALLOW_TRANSPONDER , cmd, (unsigned char
*)&duration, sizeof(duration)); // Send ack of success in turn on transponder and to how
much time
171  SetIdleState(isis_vu_e__onoff__off, 0);
172  vTaskDelay(10);
173  short rssi = getTransponderRSSIFromFRAM();
174  if(rssi != -1)
175      setTransponderRSSIinFRAM(rssi);
176  int error = setTransponderOn(); // Set transponder on
177  if(error == -1) return 0;
178  if(error)
179  {
180      error_ack = ERROR_WRITE_TO_I2C;
181      SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
182      return error;
183  }
184  vTaskDelay(30);
185  return 0;
186
187 }
188
189  /*
190  * The command sets the transponder off.
191  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (like the headers we add)
192  * @return type=int; return type of error.
193  */
194  int CMD_SetOff_Transponder(sat_packet_t *cmd)
195  {
196      unsigned char error_ack;
197      int error = setTransponderOff();
198      vTaskDelay(30);
199      if(error)
200      {
201          error_ack = ERROR_WRITE_TO_I2C;
202          SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
203          return error;

```

```

204     }
205     time_unix timeNow = 1;
206     error = logError(FRAM_write((unsigned char*)&timeNow,
TRANSPONDER_END_TIME_ADDR, TRANSPONDER_END_TIME_SIZE),
"CMD_SetOff_Transponder - FRAM_write");
207     if(error)
208     {
209         error_ack = ERROR_WRITE_TO_FRAM;
210         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
211         return error;
212     }
213     InitTrxvuAndAnts();
214     time_unix check = getTransponderEndTime();
215     if(!check)
216     {
217         error_ack = ERROR_READ_FROM_FRAM;
218         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
219         return -4;
220     }
221     if(check != timeNow)
222     {
223         error_ack = ERROR_WRITTEN_IN_FRAM_WRONG;
224         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
225         return -2;
226     }
227     return SendAckPacket(ACK_TRANSPONDER_OFF, cmd, NULL, 0); // Send ack
of success in turn off transponder
228 }
229
230 /*
231 * The command change the rssi value in the FRAM and check it change correctly.
also check the new rssi value is between the 0 and 4095
232 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (like the new rssi val and the headers we add)
233 * @return type=int; return type of error and if the parameter is NULL return -1 and
on error in setTransponderRSSIinFRAM, -2 if length isn't in size.
234 */
235 int CMD_SetRSSI_Transponder(sat_packet_t *cmd)
236 {
237     unsigned char error_ack;
238     if(cmd == NULL)
239         return -1;
240     short new_rssi_val = -1;

```

```

241  if(cmd->length != 2)
242  {
243      error_ack = ERROR_WRONG_LENGTH_DATA;
244      SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
245      return -2;
246  }
247  memcpy(&new_rssi_val, &cmd->data, cmd->length);
248  int error = setTransponderRSSIinFRAM(new_rssi_val);
249  if(error == -2)
250  {
251      error_ack = ERROR_WRITTEN_IN_FRAM_WRONG;
252      SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
253      return error;
254  }
255  else if(error)
256  {
257      error_ack = ERROR_CANT_DO;
258      SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
259      return error;
260  }
261  return SendAckPacket(ACK_UPDATE_RSSI_VALUE , cmd, (unsigned char
*)&new_rssi_val, sizeof(new_rssi_val)); // Send ack of success in change rssi and to
what
262 }
263
264  /*
265  * The command change the rssi value in the FRAM to default and check it change
correctly.
266  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information and set the data to default rssi val (the headers
we add)
267  * @return type=int; return type of error and if the parameter is NULL return -1 and
on error in setTransponderRSSIinFRAM, -2 if length isn't in size.
268  */
269  int CMD_SetRSSI_Transponder_DEFAULT(sat_packet_t *cmd)
270 {
271  cmd->length = 2;
272  short new_rssi = DEFAULT_RSSI_VALUE;
273  memcpy(cmd->data, (unsigned char*)&new_rssi, cmd->length);
274  return CMD_SetRSSI_Transponder(cmd);
275 }
276
277  /*

```

```

278 * The command get the rssi value from the FRAM and send it back.
279 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (like the headers we add)
280 * @return type=int; return type of error 0 on success and if the parameter (cmd) is
NULL return -2, if cant read from fram return -1.
281 */
282 int CMD_GetRSSI_Transponder(sat_packet_t *cmd)
283 {
284     if(cmd == NULL)
285         return -2;
286     short rssi_val = getTransponderRSSIFromFRAM();
287     if(rssi_val == -1)
288     {
289         unsigned char error_ack = ERROR_READ_FROM_FRAM;
290         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
291         return -1;
292     }
293     return SendAckPacket(ACK_GET_RSSI_VALUE, cmd, (unsigned char
*)&rssi_val, TRANSPONDER_RSSI_SIZE); // Send back the beacon interval
294 }
295
296 /*
297 * Set transmitter to mute for the time written in the data in cmd
298 * @param[in] name=cmd; type=sat_packet_t*; The packet the sat got and use to
find all the required information (like the mute duration and the headers we add)
299 * @return type=int; -1 on cmd NULL
300 *             -3 on incorrect length
301 *             errors according to setMuteEndTime
302 */
303 int CMD_MuteTRXVU(sat_packet_t *cmd)
304 {
305     unsigned char error_ack;
306     if(cmd == NULL)
307         return -1;
308     if(cmd->length != 4)
309     {
310         error_ack = ERROR_WRONG_LENGTH_DATA;
311         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
312         return -3;
313     }
314     int muteEndTime = 0;
315     memcpy(&muteEndTime, cmd->data, cmd->length);
316     if(muteEndTime > MAX_MUTE_TIME)

```

```

317     muteEndTime = MAX_MUTE_TIME;
318     SendAckPacket(ACK_MUTE , cmd, (unsigned char*)&muteEndTime, sizeof
(muteEndTime)); // Send ack of success at mute
319     SetIdleState(isis_vu_e__onoff__off, 0);
320     int error = setMuteEndTime(muteEndTime);
321     setTransponderOff();
322     if(error == -2)
323     {
324         error_ack = ERROR_WRITTEN_IN_FRAM_WRONG;
325         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
326         return error;
327     }
328     else if(error)
329     {
330         error_ack = ERROR_CANT_SET_END_TIME;
331         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
332         return error;
333     }
334     return 0;
335 }
336
337 /*
338 * Change the end time of mute to unmute the transmitter.
339 * @param[in] name=cmd; type=sat_packet_t*; The packet the sat got and use to
find all the required information (like the headers we add)
340 * @return type=int; return type of error.
341 */
342 int CMD_UnMuteTRXVU(sat_packet_t *cmd)
343 {
344     unsigned char error_ack;
345     int error = setMuteEndTime(0); // set new end time to time now

346     if(error == -2)
347     {
348         error_ack = ERROR_WRITTEN_IN_FRAM_WRONG;
349         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
350         return error;
351     }
352     else if(error)
353     {
354         error_ack = ERROR_CANT_SET_END_TIME;
355         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"

```

```

356     return error;
357 }
358 return SendAckPacket(ACK_UNMUTE , cmd, NULL, 0); // Send ack of success
in unmuting the transmitter
359 }
360
361 /*
362 * The command change the beacon interval in the FRAM and check it change
correctly. also check the new interval is between the max and min interval
363 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (like the new_interval and the headers we add)
364 * @return type=int; return type of error and if the parameter is NULL return -1, -2 if
length isn't in size.
365 */
366 int CMD_SetBeacon_Interval(sat_packet_t *cmd)
367 {
368     unsigned char error_ack;
369     if(cmd == NULL)
370         return -1;
371     if(cmd->length != 4)
372     {
373         error_ack = ERROR_WRONG_LENGTH_DATA;
374         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
375         return -2;
376     }
377     unsigned int new_interval; //You are right it's not needed

378     int error;
379     memcpy(&new_interval, &cmd->data, cmd->length);
380     if(new_interval >= MAX_BEACON_INTERVAL) // Check if new_interval is bigger
then max and if it is compares him to max interval

381         new_interval = MAX_BEACON_INTERVAL;
382     else if(new_interval <= MIN_BEACON_INTERVAL) // Same only to min interval

383         new_interval = MIN_BEACON_INTERVAL;
384     error = logError(FRAM_write((unsigned char*)&new_interval,
BEACON_INTERVAL_TIME_ADDR, BEACON_INTERVAL_TIME_SIZE),
"CMD_SetBeacon_Interval - FRAM_write"); // Write the new interval to FRAM

385     if(error)
386     {
387         error_ack = ERROR_WRITE_TO_FRAM;
388         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"

```

```

389     return error;
390 }
391 unsigned int check;
392 error = logError(FRAM_read((unsigned char*)&check,
BEACON_INTERVAL_TIME_ADDR, BEACON_INTERVAL_TIME_SIZE),
"CMD_SetBeacon_Interval - FRAM_read"); // Read from FRAM in the place we wrote
to for check
393 if(error)
394 {
395     error_ack = ERROR_READ_FROM_FRAM;
396     SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
397     return error;
398 }
399 if(check != new_interval) // Check if what we wrote and what have been written is
the same
400 {
401     error_ack = ERROR_WRITTEN_IN_FRAM_WRONG;
402     SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
403     return error;
404 }
405 setNewBeaconIntervalToPeriod();
406
407 return SendAckPacket(ACK_UPDATE_BEACON_INTERVAL , cmd, (unsigned
char*)&new_interval, sizeof(new_interval)); // // Send ack with the new_interval with
subtype of ACK_UPDATE_BEACON_INTERVAL

408 }
409
410 /*
411 * The command change the beacon interval in the FRAM to default interval
412 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (the headers we add)
413 * @return type=int; return type of error.
414 */
415 int CMD_SetBeacon_Interval_DEFAULT(sat_packet_t *cmd)
416 {
417     cmd->length = 4;
418     int beacon_interval = DEFAULT_BEACON_INTERVAL_TIME;
419     memcpy(cmd->data, (unsigned char*)&beacon_interval, cmd->length);
420     return CMD_SetBeacon_Interval(cmd);
421 }
422
423 /*
424 * The command get the beacon interval from the FRAM and send it back.

```



```

425 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (like the headers we add)
426 * @return type=int; return type of error and if the parameter (cmd) is NULL return
-1.
427 */
428     int CMD_GetBeacon_Interval(sat_packet_t *cmd)
429 {
430     unsigned int period;
431     int error = logError(FRAM_read((unsigned char*)&period,
BEACON_INTERVAL_TIME_ADDR, BEACON_INTERVAL_TIME_SIZE),
"CMD_GetBeacon_Interval - FRAM_read"); // Read the beacon interval from FRAM

432     if(error)
433     {
434         unsigned char error_ack = ERROR_READ_FROM_FRAM;
435         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
436         return error;
437     }
438     if(cmd == NULL)
439         return -1;
440     return SendAckPacket(ACK_GET_BEACON_INTERVAL, cmd, (unsigned char
*)&period, BEACON_INTERVAL_TIME_SIZE); // Send back the beacon interval

441 }
442
443     /*
444 * Get transmitter uptime and send to ground
445 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
use to find all the required information (like the headers we add)
446 * @return type=int; return type of error according to <hal/errors.h>
447 */
448     int CMD_GetTxUptime(sat_packet_t *cmd)
449 {
450     uint32_t uptime;
451     int error = logError(isis_vu_e__tx_uptime(ISIS_TRXVU_I2C_BUS_INDEX,
&uptime), "CMD_GetTxUptime - isis_vu_e__tx_uptime");
452     if(error)
453     {
454         unsigned char error_ack = ERROR_GET_UPTIME;
455         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
Send ack error according to "AckErrors.h"
456         return error;
457     }
458     return SendAckPacket(ACK_GET_TX_UPTIME, cmd, (unsigned char*)&uptime,
sizeof(uptime));

```

```

459 }
460
461  /*
462  * Get receiver uptime and send to ground
463  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
464  * use to find all the required information (like the headers we add)
465  * @return type=int; return type of error according to <hal/errors.h>
466  */
467 int CMD_GetRxUptime(sat_packet_t *cmd)
468 {
469     uint32_t uptime;
470     int error = logError(isis_vu_e__rx_uptime(ISIS_TRXVU_I2C_BUS_INDEX,
471     &uptime), "CMD_GetRxUptime - isis_vu_e__rx_uptime");
472     if(error)
473     {
474         unsigned char error_ack = ERROR_GET_UPTIME;
475         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
476         Send ack error according to "AckErrors.h"
477         return error;
478     }
479     return SendAckPacket(ACK_GET_RX_UPTIME, cmd, (unsigned char*)&uptime,
480     sizeof(uptime));
481 }
482
483  /*
484  * Helper function to get side of ants.
485  * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
486  * use to find all the required information (the ant side and the headers we add)
487  * @param[out] name=side; type=char*; here we left the side for further use.
488  */
489 int GetAntSide(sat_packet_t *cmd, char *side)
490 {
491     unsigned char error_ack;
492     if(cmd == NULL) return -1;
493     if(cmd->length != 1)
494     {
495         error_ack = ERROR_WRONG_LENGTH_DATA;
496         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
497         Send ack error according to "AckErrors.h"
498         return -2;
499     }
500     *side = cmd->data[0];
501     return 0;
502 }
503
504  /*

```

```

499 * Gets Ant uptime according to side.
500 * @param[in and out] name=cmd; type=sat_packet_t*; The packet the sat got and
    use to find all the required information (the ant side and the headers we add)
501 * @return type=int; return type of error according to this
502 *                                     -1 cmd NULL
503 *                                     -2 if length isn't in size.
504 *                                     -3 got wrong side (not exist one)
505 *                                     else from <hal/errors.h>
506 */
507 int CMD_AntGetUptime(sat_packet_t *cmd)
508 {
509     unsigned char error_ack;
510     if(cmd == NULL) return -1;
511     int error = 0;
512     char side;
513     error = GetAntSide(cmd, &side);
514     if(error) return error;
515     uint32_t uptime;
516     if(side == 0)
517         error = logError(isis_ants__get_uptime(0, &uptime), "CMD_AntGetUptime -
    isis_ants__get_uptime - 0");
518     else if(side == 1)
519         error = logError(isis_ants__get_uptime(1, &uptime), "CMD_AntGetUptime -
    isis_ants__get_uptime - 1");
520     else
521     {
522         error_ack = ERROR_NOT_VALID_SIDE;
523         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
    Send ack error according to "AckErrors.h"
524         return -3;
525     }
526     if(error)
527     {
528         error_ack = ERROR_GET_UPTIME;
529         SendAckPacket(ACK_ERROR_MSG , cmd, &error_ack, sizeof(error_ack)); //
    Send ack error according to "AckErrors.h"
530         return error;
531     }
532     return SendAckPacket(ACK_GET_ANTS_UPTIME, cmd, (unsigned char
    *)&uptime, sizeof(uptime));
533 }
534
535 /*
536 * Send ack ping
537 * @param[in] name=cmd; type=sat_packet_t*; Not needed can be NULL
538 * @return type=int; according to errors <hal/errors.h>

```

```
539 */
540     int CMD_Ping(sat_packet_t *cmd)
541 {
542     unsigned char pong_msg[] = "pong";
543     return SendAckPacket(ACK_PING , cmd, pong_msg, sizeof(pong_msg)); // send
ack ping (says pong)
544 }
545
546     /*
547  * turn up the flag of stop deploy ants.
548  * @return type=int; return error according to SendAckpacket;
549  */
550     int CMD_OffAntsDeploy(sat_packet_t *cmd)
551 {
552     SetNeedToStopAntDeploy();
553     return SendAckPacket(ACK_COMD_EXEC, cmd, NULL, 0);
554 }
555
```

drivers

מהספק (לא דברים

שאפשר למצוא באינטרנט)

```

1 /*
2  * isis_ants.h
3  *
4  * AUTOGENERATED CODE
5  * Please do not perform manual edits
6  * Generated using autogen v1.0.3
7  *
8  * Generated from: isis_ants.yaml
9  */
10
11     #ifndef ISIS_ANTS_H_
12     #define ISIS_ANTS_H_
13
14     #include "isis_ants_types.h"
15     #include <satellite-subsystems/common_types.h>
16
17     #ifdef __cplusplus
18     extern "C"
19 {
20     #endif
21
22     /*!
23  *   Initialize driver for a number of ISIS_ANTS instances. The first instance can
24  *   then be referenced by using index 0, the second by using index 1, etc.
25  *   @param[in] isis_ants Pointer to array of ISIS_ANTS instances
26  *   @param[in] isis_antsCount Number of ISIS_ANTS instances pointed to by
27  *   isis_ants input parameter
28  *   @return Error code as specified in common_types.h
29  */
30     driver_error_t ISIS_ANTS_Init(const ISIS_ANTS_t* isis_ants, uint8_t
31     isis_antsCount);
32
33     /*!
34  *   Arm the antenna system
35  *   @param[in] index Index of ISIS_ANTS in list provided during driver initialization
36  *   @return Error code as specified in common_types.h
37  */
38     driver_error_t isis_ants__arm(uint8_t index);
39
40     /*!
41  *   Cancel deployment activation
42  *   @param[in] index Index of ISIS_ANTS in list provided during driver initialization

```

```

43 * @return Error code as specified in common_types.h
44 */
45     driver_error_t isis_ants__cancel_deploy(uint8_t index);
46
47     /*!
48 * Cancel independent burn mode
49 *
50 * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
51 * @return Error code as specified in common_types.h
52 */
53     driver_error_t isis_ants__cancel_id(uint8_t index);
54
55     /*!
56 * Performs a reset of the I2C watchdog without performing any other operation
57 *
58 * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
59 * @return Error code as specified in common_types.h
60 */
61     driver_error_t isis_ants__clear_wdg(uint8_t index);
62
63     /*!
64 * Deploy antenna 1
65 *
66 * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
67 * @param[in] max_time_in maximum antenna deployment activation time
68 * @return Error code as specified in common_types.h
69 */
70     driver_error_t isis_ants__deploy1(uint8_t index, uint8_t max_time_in);
71
72     /*!
73 * Deploy antenna 1 with override
74 *
75 * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
76 * @param[in] max_time_in maximum antenna deployment activation time
77 * @return Error code as specified in common_types.h
78 */
79     driver_error_t isis_ants__deploy1_override(uint8_t index, uint8_t max_time_in);
80
81     /*!
82 * Deploy antenna 2
83 *
84 * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
85 * @param[in] max_time_in maximum antenna deployment activation time
86 * @return Error code as specified in common_types.h
87 */
88     driver_error_t isis_ants__deploy2(uint8_t index, uint8_t max_time_in);

```

```

89
90     /*!
91  * Deploy antenna 2 with override
92  *
93  * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
94  * @param[in] max_time_in maximum antenna deployment activation time
95  * @return Error code as specified in common_types.h
96  */
97     driver_error_t isis_ants__deploy2_override(uint8_t index, uint8_t max_time_in);
98
99     /*!
100  * Deploy antenna 3
101  *
102  * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
103  * @param[in] max_time_in maximum antenna deployment activation time
104  * @return Error code as specified in common_types.h
105  */
106     driver_error_t isis_ants__deploy3(uint8_t index, uint8_t max_time_in);
107
108     /*!
109  * Deploy antenna 3 with override
110  *
111  * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
112  * @param[in] max_time_in maximum antenna deployment activation time
113  * @return Error code as specified in common_types.h
114  */
115     driver_error_t isis_ants__deploy3_override(uint8_t index, uint8_t max_time_in);
116
117     /*!
118  * Deploy antenna 4
119  *
120  * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
121  * @param[in] max_time_in maximum antenna deployment activation time
122  * @return Error code as specified in common_types.h
123  */
124     driver_error_t isis_ants__deploy4(uint8_t index, uint8_t max_time_in);
125
126     /*!
127  * Deploy antenna 4 with override
128  *
129  * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
130  * @param[in] max_time_in maximum antenna deployment activation time
131  * @return Error code as specified in common_types.h
132  */
133     driver_error_t isis_ants__deploy4_override(uint8_t index, uint8_t max_time_in);
134

```



```

135     /*!
136  * Disarm the antenna system
137  *
138  * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
139  * @return Error code as specified in common_types.h
140  */
141     driver_error_t isis_ants__disarm(uint8_t index);
142
143     /*!
144  * Obtain ISIS Antenna Systems all telemetry structure.
145  *
146  * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
147  * @param[out] response Struct with response from subsystem
148  * @return Error code as specified in common_types.h
149  */
150     driver_error_t isis_ants__get_all_telemetry(uint8_t index,
151     isis_ants__get_all_telemetry__from_t *response);
152
153     /*!
154  * Report antenna 1 deployment system activation count
155  *
156  * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
157  * @param[out] deploy_count_out antenna deployment activation count
158  * @return Error code as specified in common_types.h
159  */
160     driver_error_t isis_ants__get_deploy1_count(uint8_t index, uint8_t
161     *deploy_count_out);
162
163     /*!
164  * Report antenna 1 deployment activation time
165  *
166  * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
167  * @param[out] deploy_time_out
168  * @parblock
169  * antenna activation time
170  *
171  * conversion: eng. value = 0.05 * raw
172  * @endparblock
173  * @return Error code as specified in common_types.h
174  */
175     driver_error_t isis_ants__get_deploy1_time(uint8_t index, uint16_t
176     *deploy_time_out);
177
178     /*!
179  * Report antenna 2 deployment system activation count
180  *
181  * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
182  * @param[out] deploy_count_out antenna deployment activation count
183  * @return Error code as specified in common_types.h
184  */
185     driver_error_t isis_ants__get_deploy2_count(uint8_t index, uint8_t
186     *deploy_count_out);
187
188     /*!
189  * Report antenna 2 deployment activation time
190  *
191  * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
192  * @param[out] deploy_time_out
193  * @parblock
194  * antenna activation time
195  *
196  * conversion: eng. value = 0.05 * raw
197  * @endparblock
198  * @return Error code as specified in common_types.h
199  */
200     driver_error_t isis_ants__get_deploy2_time(uint8_t index, uint16_t
201     *deploy_time_out);

```

```

178 * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
179 * @param[out] deploy_count_out antenna deployment activation count
180 * @return Error code as specified in common_types.h
181 */
182     driver_error_t isis_ants__get_deploy2_count(uint8_t index, uint8_t
*deploy_count_out);
183
184     /*!
185 * Report antenna 2 deployment activation time
186 *
187 * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
188 * @param[out] deploy_time_out
189 * @parblock
190 * antenna activation time
191 *
192 * conversion: eng. value = 0.05 * raw
193 * @endparblock
194 * @return Error code as specified in common_types.h
195 */
196     driver_error_t isis_ants__get_deploy2_time(uint8_t index, uint16_t
*deploy_time_out);
197
198     /*!
199 * Report antenna 3 deployment system activation count
200 *
201 * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
202 * @param[out] deploy_count_out antenna deployment activation count
203 * @return Error code as specified in common_types.h
204 */
205     driver_error_t isis_ants__get_deploy3_count(uint8_t index, uint8_t
*deploy_count_out);
206
207     /*!
208 * Report antenna 3 deployment activation time
209 *
210 * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
211 * @param[out] deploy_time_out
212 * @parblock
213 * antenna activation time
214 *
215 * conversion: eng. value = 0.05 * raw
216 * @endparblock
217 * @return Error code as specified in common_types.h
218 */
219     driver_error_t isis_ants__get_deploy3_time(uint8_t index, uint16_t
*deploy_time_out);

```

```

220
221     /*!
222  * Report antenna 4 deployment system activation count
223  *
224  * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
225  * @param[out] deploy_count_out antenna deployment activation count
226  * @return Error code as specified in common_types.h
227  */
228     driver_error_t isis_ants__get_deploy4_count(uint8_t index, uint8_t
*deploy_count_out);
229
230     /*!
231  * Report antenna 4 deployment activation time
232  *
233  * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
234  * @param[out] deploy_time_out
235  * @parblock
236  * antenna activation time
237  *
238  * conversion: eng. value = 0.05 * raw
239  * @endparblock
240  * @return Error code as specified in common_types.h
241  */
242     driver_error_t isis_ants__get_deploy4_time(uint8_t index, uint16_t
*deploy_time_out);
243
244     /*!
245  * Get independent burn time left
246  *
247  * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
248  * @param[out] uptime_out Independent burn time left
249  * @return Error code as specified in common_types.h
250  */
251     driver_error_t isis_ants__get_idtimeleft(uint8_t index, uint16_t *uptime_out);
252
253     /*!
254  * Report deployment status
255  *
256  * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
257  * @param[out] response Struct with response from subsystem
258  * @return Error code as specified in common_types.h
259  */
260     driver_error_t isis_ants__get_status(uint8_t index,
isis_ants__get_status__from_t *response);
261
262     /*!

```

```

263 * Measure antenna system temperature
264 *
265 * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
266 * @param[out] response Struct with response from subsystem
267 * @return Error code as specified in common_types.h
268 */
269     driver_error_t isis_ants__get_temperature(uint8_t index,
isis_ants__get_temperature__from_t *response);
270
271     /*!
272 * Obtain ISIS Antenna Systems uptime value.
273 *
274 * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
275 * @param[out] uptime_out ISIS Antenna Systems uptime value in seconds
276 * @return Error code as specified in common_types.h
277 */
278     driver_error_t isis_ants__get_uptime(uint8_t index, uint32_t *uptime_out);
279
280     /*!
281 * Reset the board
282 *
283 * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
284 * @return Error code as specified in common_types.h
285 */
286     driver_error_t isis_ants__reset(uint8_t index);
287
288     /*!
289 * Start automated sequential antenna deployment
290 *
291 * @param[in] index Index of ISIS_ANTS in list provided during driver initialization
292 * @param[in] max_time_in maximum antenna deployment activation time
293 * @return Error code as specified in common_types.h
294 */
295     driver_error_t isis_ants__start_auto_deploy(uint8_t index, uint8_t max_time_in)
;
296
297     #ifdef __cplusplus
298 }
299     #endif
300
301     #endif /* ISIS_ANTS_H_ */
302
303

```

```

1 /*
2  * isis_vu_e.h
3  *
4  * AUTOGENERATED CODE
5  * Please do not perform manual edits
6  * Generated using autogen v1.0.3
7  *
8  * Generated ICD version: 1.0
9  * Generated from:
10 * - isis_vu_rx_e.yaml
11 * - isis_vu_tx_e.yaml
12 */
13
14 #ifndef ISIS_VU_E_H_
15 #define ISIS_VU_E_H_
16
17 #include "isis_vu_e_types.h"
18 #include <satellite-subsystems/common_types.h>
19
20 #ifdef __cplusplus
21 extern "C"
22 {
23     #endif
24
25     /*!
26  * Initialize driver for a number of ISIS_VU_E instances. The first instance can
27  * then be referenced by using index 0, the second by using index 1, etc.
28  * @param[in] isis_vu_e Pointer to array of ISIS_VU_E instances
29  * @param[in] isis_vu_eCount Number of ISIS_VU_E instances pointed to by
30  * isis_vu_e input parameter
31  * @return Error code as specified in common_types.h
32  */
33     driver_error_t ISIS_VU_E_Init(const ISIS_VU_E_t* isis_vu_e, uint8_t
34     isis_vu_eCount);
35
36     /*!
37  * Clears any beacon that is currently set in the transceiver.
38  * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
39  * @return Error code as specified in common_types.h
40  */
41     driver_error_t isis_vu_e__clear_beacon(uint8_t index);
42
43     /*!

```

```

43 * Gets the AX.25 FROM callsign in the transmitter.
44 *
45 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
46 * @param[out] response Struct with response from subsystem
47 * @return Error code as specified in common_types.h
48 */
49     driver_error_t isis_vu_e__get_callsign_from(uint8_t index,
isis_vu_e__get_callsign_from__from_t *response);
50
51     /*!
52 * Gets the AX.25 TO callsign in the transmitter.
53 *
54 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
55 * @param[out] response Struct with response from subsystem
56 * @return Error code as specified in common_types.h
57 */
58     driver_error_t isis_vu_e__get_callsign_to(uint8_t index,
isis_vu_e__get_callsign_to__from_t *response);
59
60     /*!
61 * Get frame from receive buffer
62 *
63 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
64 * @param[out] data_out Pointer to buffer for the response from subsystem
65 * @return Error code as specified in common_types.h
66 */
67     driver_error_t isis_vu_e__get_frame(uint8_t index,
isis_vu_e__get_frame__from_t *response);
68
69     /*!
70 * Get frame from receive buffer
71 *
72 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
73 * @param[out] data_out Pointer to buffer for the response from subsystem
74 * @param[in] data_length Number of elements to read for output struct member
'data' in bytes
75 * @return Error code as specified in common_types.h
76 */
77     driver_error_t isis_vu_e__get_frame_variable(uint8_t index,
isis_vu_e__get_frame__from_t *response, size_t data_length);
78
79     /*!
80 * Retrieves the number of frames that are currently stored in the receiver buffer.
81 *
82 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
83 * @param[out] frame_count_out Number of frames in receive buffer

```

```

84 * @return Error code as specified in common_types.h
85 */
86     driver_error_t isis_vu_e__get_frame_count(uint8_t index, uint16_t
*frame_count_out);
87
88     /*!
89 * Retrieves the length of frames that are currently stored in the receiver buffer.
90 *
91 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
92 * @param[out] p_frame_length_out Pointer to buffer for the response from
subsystem
93 * @return Error code as specified in common_types.h
94 */
95     driver_error_t isis_vu_e__get_frame_count_length(uint8_t index,
isis_vu_e__get_frame_count_length__from_t *response);
96
97     /*!
98 * Retrieves the length of frames that are currently stored in the receiver buffer.
99 *
100 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
101 * @param[out] p_frame_length_out Pointer to buffer for the response from
subsystem
102 * @param[in] p_frame_length_length Number of elements to read for output struct
member 'p_frame_length' in bytes
103 * @return Error code as specified in common_types.h
104 */
105     driver_error_t isis_vu_e__get_frame_count_length_variable(uint8_t index,
isis_vu_e__get_frame_count_length__from_t *response, size_t p_frame_length_length)
;
106
107     /*!
108 * Get full frame from receive buffer
109 *
110 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
111 * @param[out] data_out Pointer to buffer for the response from subsystem
112 * @return Error code as specified in common_types.h
113 */
114     driver_error_t isis_vu_e__get_full_frame(uint8_t index,
isis_vu_e__get_full_frame__from_t *response);
115
116     /*!
117 * Get full frame from receive buffer
118 *
119 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
120 * @param[out] data_out Pointer to buffer for the response from subsystem
121 * @param[in] data_length Number of elements to read for output struct member

```

'data' in bytes

```
122 * @return Error code as specified in common_types.h
123 */
124     driver_error_t isis_vu_e__get_full_frame_variable(uint8_t index,
isis_vu_e__get_full_frame__from_t *response, size_t data_length);
125
126     /*!
127 * Get receiver frequency
128 *
129 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
130 * @param[out] response Struct with response from subsystem
131 * @return Error code as specified in common_types.h
132 */
133     driver_error_t isis_vu_e__get_rx_freq(uint8_t index,
isis_vu_e__get_rx_freq__from_t *response);
134
135     /*!
136 * Get last reset cause
137 *
138 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
139 * @param[out] last_reset_cause_out Last reset cause
140 * @return Error code as specified in common_types.h
141 */
142     driver_error_t isis_vu_e__get_rx_last_reset_cause(uint8_t index, uint8_t
*last_reset_cause_out);
143
144     /*!
145 * Get PLL error
146 *
147 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
148 * @param[out] response Struct with response from subsystem
149 * @return Error code as specified in common_types.h
150 */
151     driver_error_t isis_vu_e__get_rx_pll_error(uint8_t index,
isis_vu_e__get_rx_pll_error__from_t *response);
152
153     /*!
154 * Get firmware information
155 *
156 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
157 * @param[out] response Struct with response from subsystem
158 * @return Error code as specified in common_types.h
159 */
160     driver_error_t isis_vu_e__get_rx_swinfom(uint8_t index,
isis_vu_e__get_rx_swinfom__from_t *response);
161
```



```

162     /*!
163  * Measure all telemetry channels
164  *
165  * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
166  * @param[out] response Struct with response from subsystem
167  * @return Error code as specified in common_types.h
168  */
169     driver_error_t isis_vu_e__get_rx_telemetry(uint8_t index,
170 isis_vu_e__get_rx_telemetry__from_t *response);
171     /*!
172  * Get transponder input frequency
173  *
174  * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
175  * @param[out] response Struct with response from subsystem
176  * @return Error code as specified in common_types.h
177  */
178     driver_error_t isis_vu_e__get_transponder_in_freq(uint8_t index,
179 isis_vu_e__get_transponder_in_freq__from_t *response);
180     /*!
181  * Get transmitter frequency
182  *
183  * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
184  * @param[out] freq_out Current frequency of transmitter in kHz
185  * @return Error code as specified in common_types.h
186  */
187     driver_error_t isis_vu_e__get_tx_freq(uint8_t index, uint32_t *freq_out);
188
189     /*!
190  * Get last reset cause
191  *
192  * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
193  * @param[out] last_reset_cause_out Last reset cause
194  * @return Error code as specified in common_types.h
195  */
196     driver_error_t isis_vu_e__get_tx_last_reset_cause(uint8_t index, uint8_t
197 *last_reset_cause_out);
198     /*!
199  * Get PLL error
200  *
201  * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
202  * @param[out] response Struct with response from subsystem
203  * @return Error code as specified in common_types.h
204  */

```

```

205     driver_error_t isis_vu_e__get_tx_pll_error(uint8_t index,
isis_vu_e__get_tx_pll_error__from_t *response);
206
207     /*!
208 * Get TX RSSI
209 *
210 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
211 * @param[out] rssi_out TX RSSI
212 * @return Error code as specified in common_types.h
213 */
214     driver_error_t isis_vu_e__get_tx_rssi(uint8_t index, uint16_t *rssi_out);
215
216     /*!
217 * Get firmware information
218 *
219 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
220 * @param[out] response Struct with response from subsystem
221 * @return Error code as specified in common_types.h
222 */
223     driver_error_t isis_vu_e__get_tx_swinf(uint8_t index,
isis_vu_e__get_tx_swinf__from_t *response);
224
225     /*!
226 * Measure all the telemetry channels
227 *
228 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
229 * @param[out] response Struct with response from subsystem
230 * @return Error code as specified in common_types.h
231 */
232     driver_error_t isis_vu_e__get_tx_telemetry(uint8_t index,
isis_vu_e__get_tx_telemetry__from_t *response);
233
234     /*!
235 * Get the stored telemetry channels during the last transmission
236 *
237 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
238 * @param[out] response Struct with response from subsystem
239 * @return Error code as specified in common_types.h
240 */
241     driver_error_t isis_vu_e__get_tx_telemetry_last(uint8_t index,
isis_vu_e__get_tx_telemetry_last__from_t *response);
242
243     /*!
244 * Removes the oldest frame from the receive buffer. This is the same frame that
can be retrieved from the receiver buffer command. If there are 0 frames in the receive
buffer this command has no effect.

```

```

245 *
246 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
247 * @return Error code as specified in common_types.h
248 */
249     driver_error_t isis_vu_e__remove_frame(uint8_t index);
250
251     /*!
252 * Removes all frames from the receive buffer. If there are 0 frames in the receive
253 * buffer this command has no effect.
254 *
255 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
256 * @return Error code as specified in common_types.h
257 */
258     driver_error_t isis_vu_e__remove_frame_all(uint8_t index);
259
260     /*!
261 * Power cycles the full board (transmitter and receiver will be both reset).
262 *
263 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
264 * @return Error code as specified in common_types.h
265 */
266     driver_error_t isis_vu_e__reset_hw_rx(uint8_t index);
267
268     /*!
269 * Power cycles the full board (transmitter and receiver will be both reset).
270 *
271 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
272 * @return Error code as specified in common_types.h
273 */
274     driver_error_t isis_vu_e__reset_hw_tx(uint8_t index);
275
276     /*!
277 * Performs a reset of the I2C watchdog without performing any other operation.
278 *
279 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
280 * @return Error code as specified in common_types.h
281 */
282     driver_error_t isis_vu_e__reset_wdg_rx(uint8_t index);
283
284     /*!
285 * Performs a reset of the I2C watchdog without performing any other operation.
286 *
287 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
288 * @return Error code as specified in common_types.h
289 */
290     driver_error_t isis_vu_e__reset_wdg_tx(uint8_t index);

```

```

290
291     /*!
292  * Report receiver uptime
293  *
294  * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
295  * @param[out] uptime_out Reports the amount of time the transmitter MCU has
296  * @return Error code as specified in common_types.h
297  */
298     driver_error_t isis_vu_e__rx_uptime(uint8_t index, uint32_t *uptime_out);
299
300     /*!
301  * Transmit an AX.25 frame with default callsigns or HDLC frame.
302  *
303  * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
304  * @param[in] data_in Data to put in the frame
305  * @param[in] data_length Length of data_in parameter in number of elements,
306  * @param[out] slots_available_out Number of the available slots in the
307  * @return Error code as specified in common_types.h
308  */
309     driver_error_t isis_vu_e__send_frame(uint8_t index, const uint8_t* data_in,
310     size_t data_length, uint8_t *slots_available_out);
311     /*!
312  * Transmit an AX.25 frame with override callsigns and specified content.
313  *
314  * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
315  * @param[in] params Struct with parameters for subsystem
316  * @param[in] data_length Length of input struct member 'data' in number of
317  * @param[out] slots_available_out Number of the available slots in the
318  * @return Error code as specified in common_types.h
319  */
320     driver_error_t isis_vu_e__send_frame_with_callsigns(uint8_t index, const
321     isis_vu_e__send_frame_with_callsigns__to_t *params, size_t data_length, uint8_t
322     *slots_available_out);
323     /*!
324  * Set the parameters for the AX25 Beacon with default callsigns or HDLC Beacon.
325  * @param[in] index Index of ISIS_VU_E in list provided during driver initialization

```

```

326 * @param[in] params Struct with parameters for subsystem
327 * @param[in] data_length Length of input struct member 'data' in number of
elements, maximum value: 235
328 * @return Error code as specified in common_types.h
329 */
330     driver_error_t isis_vu_e__set_beacon(uint8_t index, const
isis_vu_e__set_beacon__to_t *params, size_t data_length);
331
332     /*!
333 * Set the parameters for the AX25 Beacon with override callsigns.
334 *
335 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
336 * @param[in] params Struct with parameters for subsystem
337 * @param[in] data_length Length of input struct member 'data' in number of
elements, maximum value: 235
338 * @return Error code as specified in common_types.h
339 */
340     driver_error_t isis_vu_e__set_beacon_with_callsigns(uint8_t index, const
isis_vu_e__set_beacon_with_callsigns__to_t *params, size_t data_length);
341
342     /*!
343 * Set AX.25 transmission bitrate
344 *
345 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
346 * @param[in] bitrate_in Transmission bitrate.
347 * @return Error code as specified in common_types.h
348 */
349     driver_error_t isis_vu_e__set_bitrate(uint8_t index, isis_vu_e__bitrate_t
bitrate_in);
350
351     /*!
352 * Sets the default AX.25 FROM callsign in the transmitter.
353 *
354 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
355 * @param[in] params Struct with parameters for subsystem
356 * @return Error code as specified in common_types.h
357 */
358     driver_error_t isis_vu_e__set_callsign_from(uint8_t index, const
isis_vu_e__set_callsign_from__to_t *params);
359
360     /*!
361 * Sets the default AX.25 TO callsign in the transmitter.
362 *
363 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
364 * @param[in] params Struct with parameters for subsystem
365 * @return Error code as specified in common_types.h

```

```

366 */
367     driver_error_t isis_vu_e__set_callsign_to(uint8_t index, const
isis_vu_e__set_callsign_to__to_t *params);
368
369     /*!
370 * Sets the state of the transmitter when it is idle.
371 *
372 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
373 * @param[in] idle_state_in The idle state of the transmitter to be set.
374 * @return Error code as specified in common_types.h
375 */
376     driver_error_t isis_vu_e__set_idle_state(uint8_t index, isis_vu_e__onoff_t
idle_state_in);
377
378     /*!
379 * Set receiver frequency
380 *
381 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
382 * @param[in] freq_in Desired frequency for receiver in kHz
383 * @return Error code as specified in common_types.h
384 */
385     driver_error_t isis_vu_e__set_rx_freq(uint8_t index, uint32_t freq_in);
386
387     /*!
388 * Set transponder input frequency
389 *
390 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
391 * @param[in] freq_in Desired transponder input frequency in kHz
392 * @return Error code as specified in common_types.h
393 */
394     driver_error_t isis_vu_e__set_transponder_in_freq(uint8_t index, uint32_t
freq_in);
395
396     /*!
397 * Set transmitter frequency
398 *
399 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
400 * @param[in] freq_in Desired frequency for transmitter in kHz
401 * @return Error code as specified in common_types.h
402 */
403     driver_error_t isis_vu_e__set_tx_freq(uint8_t index, uint32_t freq_in);
404
405     /*!
406 * Set transmitter mode
407 *
408 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization

```

```

409 * @param[in] mode_in Transmitter mode (1: nominal, 2: transponder)
410 * @return Error code as specified in common_types.h
411 */
412     driver_error_t isis_vu_e__set_tx_mode(uint8_t index, uint8_t mode_in);
413
414     /*!
415     * PLL power out
416     *
417     * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
418     * @param[in] reg_value_in Configuration register value for power output of the
419     synthesizer
419     * @return Error code as specified in common_types.h
420     */
421     driver_error_t isis_vu_e__set_tx_pll_powerout(uint8_t index, uint16_t
422 reg_value_in);
423     /*!
424     * Set THR RSSI
425     *
426     * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
427     * @param[in] mode_in THR RSSI
428     * @return Error code as specified in common_types.h
429     */
430     driver_error_t isis_vu_e__set_tx_thr_rssi(uint8_t index, uint16_t mode_in);
431
432     /*!
433     * Reports general information about the state the transmitter is currently in.
434     *
435     * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
436     * @param[out] response Struct with response from subsystem
437     * @return Error code as specified in common_types.h
438     */
439     driver_error_t isis_vu_e__state(uint8_t index, isis_vu_e__state__from_t
440 *response);
441     /*!
442     * Clear PA overtemperature flag, set to zero
443     *
444     * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
445     * @return Error code as specified in common_types.h
446     */
447     driver_error_t isis_vu_e__tx_clear_pa_overtemp_flag(uint8_t index);
448
449     /*!
450     * Get PA overtemperature flag
451     *

```

```

452 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
453 * @param[out] state_out PA overtemperature flag (0: no issue, 1: overtemperature
454 * @return Error code as specified in common_types.h
455 */
456     driver_error_t isis_vu_e__tx_get_pa_overtemp_flag(uint8_t index, uint8_t
457 *state_out);
458     /*
459 * Reports the amount of time the transmitter MCU has been active since the last
460 * reset, also known as up- time.
461 * @param[in] index Index of ISIS_VU_E in list provided during driver initialization
462 * @param[out] uptime_out Seconds uptime value.
463 * @return Error code as specified in common_types.h
464 */
465     driver_error_t isis_vu_e__tx_uptime(uint8_t index, uint32_t *uptime_out);
466
467     #ifdef __cplusplus
468 }
469     #endif
470
471     #endif /* ISIS_VU_E_H_ */
472
473

```



```
1 /*
2  * isis_vu_e_types.h
3  *
4  * AUTOGENERATED CODE
5  * Please do not perform manual edits
6  * Generated using autogen v1.0.3
7  *
8  * Generated ICD version: 1.0
9  * Generated from:
10 * - isis_vu_rx_e.yaml
11 * - isis_vu_tx_e.yaml
12 */
13
14 #ifndef ISIS_VU_E_TYPES_H_
15 #define ISIS_VU_E_TYPES_H_
16
17 #include <stddef.h>
18 #include <stdint.h>
19
20 #ifdef __cplusplus
21 extern "C"
22 {
23 #endif
24
25 /* DEFINES */
26
27 #define ISIS_VU_E_SEND_FRAME_ID {0x10}
28
29 #define ISIS_VU_E_SEND_FRAME_WITH_CALLSIGNS_ID {0x11}
30
31 #define ISIS_VU_E_SET_BEACON_ID {0x14}
32
33 #define ISIS_VU_E_SET_BEACON_WITH_CALLSIGNS_ID {0x15}
34
35 #define ISIS_VU_E_GET_RX_TELEMETRY_ID {0x1A}
36
37 #define ISIS_VU_E_CLEAR_BEACON_ID {0x1F}
38
39 #define ISIS_VU_E_GET_CALLSIGN_TO_ID {0x20}
40
41 #define ISIS_VU_E_GET_CALLSIGN_FROM_ID {0x21}
42
43 #define ISIS_VU_E_GET_FRAME_COUNT_ID {0x21}
44
45 #define ISIS_VU_E_GET_FRAME_ID {0x22}
```

```
37  #define ISIS_VU_E_SET_CALLSIGN_TO_ID {0x22}
38  #define ISIS_VU_E_GET_FULL_FRAME_ID {0x23}
39  #define ISIS_VU_E_SET_CALLSIGN_FROM_ID {0x23}
40  #define ISIS_VU_E_REMOVE_FRAME_ID {0x24}
41  #define ISIS_VU_E_SET_IDLE_STATE_ID {0x24}
42  #define ISIS_VU_E_GET_FRAME_COUNT_LENGTH_ID {0x25}
43  #define ISIS_VU_E_GET_TX_TELEMETRY_ID {0x25}
44  #define ISIS_VU_E_GET_TX_TELEMETRY_LAST_ID {0x26}
45  #define ISIS_VU_E_REMOVE_FRAME_ALL_ID {0x26}
46  #define ISIS_VU_E_SET_BITRATE_ID {0x28}
47  #define ISIS_VU_E_SET_RX_FREQ_ID {0x32}
48  #define ISIS_VU_E_SET_TX_FREQ_ID {0x32}
49  #define ISIS_VU_E_GET_RX_FREQ_ID {0x33}
50  #define ISIS_VU_E_GET_TX_FREQ_ID {0x33}
51  #define ISIS_VU_E_GET_RX_PLL_ERROR_ID {0x34}
52  #define ISIS_VU_E_GET_TX_PLL_ERROR_ID {0x34}
53  #define ISIS_VU_E_SET_TX_PLL_POWEROUT_ID {0x35}
54  #define ISIS_VU_E_SET_TX_MODE_ID {0x38}
55  #define ISIS_VU_E_RX_UPTIME_ID {0x40}
56  #define ISIS_VU_E_TX_UPTIME_ID {0x40}
57  #define ISIS_VU_E_STATE_ID {0x41}
58  #define ISIS_VU_E_GET_RX_SWINFO_ID {0x42}
59  #define ISIS_VU_E_GET_TX_SWINFO_ID {0x42}
60  #define ISIS_VU_E_GET_RX_LAST_RESET_CAUSE_ID {0x50}
```

```

61  #define ISIS_VU_E_GET_TX_LAST_RESET_CAUSE_ID {0x50}
62  #define ISIS_VU_E_GET_TX_RSSI_ID {0x51}
63  #define ISIS_VU_E_SET_TX_THR_RSSI_ID {0x52}
64  #define ISIS_VU_E_TX_GET_PA_OVERTEMP_FLAG_ID {0x60}
65  #define ISIS_VU_E_TX_CLEAR_PA_OVERTEMP_FLAG_ID {0x61}
66  #define ISIS_VU_E_SET_TRANSPONDER_IN_FREQ_ID {0x80}
67  #define ISIS_VU_E_GET_TRANSPONDER_IN_FREQ_ID {0x81}
68  #define ISIS_VU_E_RESET_HW_RX_ID {0xAB}
69  #define ISIS_VU_E_RESET_HW_TX_ID {0xAB}
70  #define ISIS_VU_E_RESET_WDG_RX_ID {0xCC}
71  #define ISIS_VU_E_RESET_WDG_TX_ID {0xCC}
72
73  #define ISIS_VU_E_SEND_FRAME_MAX_DATA_LENGTH 235
74  #define
ISIS_VU_E_SEND_FRAME_WITH_CALLSIGNS_MAX_DATA_LENGTH 235
75  #define ISIS_VU_E_SET_BEACON_MAX_DATA_LENGTH 235
76  #define
ISIS_VU_E_SET_BEACON_WITH_CALLSIGNS_MAX_DATA_LENGTH 235
77  #define ISIS_VU_E_GET_FRAME_MAX_DATA_LENGTH 200
78  #define ISIS_VU_E_GET_FULL_FRAME_MAX_DATA_LENGTH 218
79  #define
ISIS_VU_E_GET_FRAME_COUNT_LENGTH_MAX_P_FRAME_LENGTH_LENGTH
80
81  /* ENUMS */
82
83  /*!
84  *
85  */

```

```

86     typedef enum __attribute__((__packed__)) isis_vu_e__bitrate_t

87 {
88     isis_vu_e__bitrate__1200bps = 1u,
89     isis_vu_e__bitrate__2400bps = 2u,
90     isis_vu_e__bitrate__4800bps = 4u,
91     isis_vu_e__bitrate__9600bps = 8u,
92 } isis_vu_e__bitrate_t;
93
94     /*!
95     * Enum for telemetry baudrate
96     */
97     typedef enum __attribute__((__packed__)) isis_vu_e__bitrate_tlm_t

98 {
99     isis_vu_e__bitrate_tlm__1200bps = 0u,
100    isis_vu_e__bitrate_tlm__2400bps = 1u,
101    isis_vu_e__bitrate_tlm__4800bps = 2u,
102    isis_vu_e__bitrate_tlm__9600bps = 3u,
103 } isis_vu_e__bitrate_tlm_t;
104
105     /*!
106     * Enum for on/off state
107     */
108     typedef enum __attribute__((__packed__)) isis_vu_e__onoff_t

109 {
110     isis_vu_e__onoff__off = 0u,
111     isis_vu_e__onoff__on = 1u,
112 } isis_vu_e__onoff_t;
113
114     /* STRUCTS */
115
116     /*!
117     * ISIS_VU_E instance structure
118     */
119     typedef struct
120 {
121     uint8_t rxAddr; /*!< I2C address used for rx part of the subsystem */

122     uint8_t txAddr; /*!< I2C address used for tx part of the subsystem */

123     size_t maxReceiveBufferLength;
124     size_t maxSendBufferLength;
125 } ISIS_VU_E_t;
126

```

```

127     /*!
128  * Union for storing the parameters received by get_callsign_from.
129  */
130     typedef union __attribute__((__packed__)) _
131     isis_vu_e__get_callsign_from__from_t
132     {
133         unsigned char raw[7];
134         struct __attribute__((__packed__))
135         {
136             char ascii[6]; /*!< AX.25 FROM callsign to be used for AX.25 frames */
137
138             uint8_t number; /*!< */
139         } fields;
140     } isis_vu_e__get_callsign_from__from_t;
141
142     /*!
143  * Union for storing the parameters received by get_callsign_to.
144  */
145     typedef union __attribute__((__packed__)) _
146     isis_vu_e__get_callsign_to__from_t
147     {
148         unsigned char raw[7];
149         struct __attribute__((__packed__))
150         {
151             char ascii[6]; /*!< AX.25 TO callsign to be used for AX.25 frames */
152
153             uint8_t number; /*!< */
154         } fields;
155     } isis_vu_e__get_callsign_to__from_t;
156
157     /*!
158  * Structure the parameters received by get_frame.
159  */
160     typedef struct __attribute__((__packed__))
161     isis_vu_e__get_frame__from_t
162     {
163         uint16_t length; /*!< Frame contents size */
164         int16_t doppler; /*!< Raw measurement value of the instantaneous Doppler
offset of the signal at the receiver port. \note conversion: eng. value = -38.1469726563 *
raw + 0*/
165         int16_t rssi; /*!< This field contains the measured Received Signal Strength
Indicator (RSSI) at the reception time \note conversion: eng. value = -0.5 * raw + -22*/
166
167         uint8_t* data; /*!< Frame contents (maximum length: 200 elements) */
168     } isis_vu_e__get_frame__from_t;

```

```

164
165     /*!
166  * Structure the parameters received by get_frame_count_length.
167  */
168     typedef struct __attribute__((__packed__))
169     _isis_vu_e__get_frame_count_length__from_t

170 {
171     uint16_t frames_nb; /*!< Number of frames in receive buffer */

172     uint16_t* p_frame_length; /*!< Frame contents size (maximum length: 80
elements) */
173 } _isis_vu_e__get_frame_count_length__from_t;
174
175     /*!
176  * Structure the parameters received by get_full_frame.
177  */
178     typedef struct __attribute__((__packed__))
179     _isis_vu_e__get_full_frame__from_t
180 {
181     uint16_t length; /*!< Full frame contents size */
182     int16_t doppler; /*!< Raw measurement value of the instantaneous Doppler
offset of the signal at the receiver port. \note conversion: eng. value = -38.1469726563 *
raw + 0*/
183     int16_t rssi; /*!< This field contains the measured Received Signal Strength
Indicator (RSSI) at the reception time \note conversion: eng. value = -0.5 * raw + -22*/

184     uint8_t* data; /*!< Frame contents (maximum length: 218 elements) */

185 } _isis_vu_e__get_full_frame__from_t;
186
187     /*!
188  * Union for storing the parameters received by get_rx_freq.
189  */
190     typedef union __attribute__((__packed__)) _isis_vu_e__get_rx_freq__from_t

191 {
192     unsigned char raw[5];
193     struct __attribute__((__packed__))
194     {
195         uint32_t freq; /*!< Current frequency of receiver in kHz */

196         uint8_t status; /*!< Operation status (0: OK, 252: SX1278 is not ready, 253:
SX1278 is busy, 254: SX1278 read error, 255: internal error) */

197     } fields;

```

```

198 } isis_vu_e__get_rx_freq__from_t;
199
200     /*!
201  * Union for storing the parameters received by get_rx_pll_error.
202  */
203     typedef union __attribute__((__packed__)) _
isis_vu_e__get_rx_pll_error__from_t
204 {
205     unsigned char raw[4];
206     struct __attribute__((__packed__))
207     {
208         uint16_t locked_errcount; /*!< Lock error count */
209         uint16_t freq_errcount; /*!< Frequency error count */
210     } fields;
211 } isis_vu_e__get_rx_pll_error__from_t;
212
213     /*!
214  * Union for storing the parameters received by get_rx_swinfo.
215  */
216     typedef union __attribute__((__packed__)) _isis_vu_e__get_rx_swinfo__from_t
217 {
218     unsigned char raw[80];
219     struct __attribute__((__packed__))
220     {
221         char swinfo[80]; /*!< Firmware information */
222     } fields;
223 } isis_vu_e__get_rx_swinfo__from_t;
224
225     /*!
226  * Union for storing the parameters received by get_rx_telemetry.
227  */
228     typedef union __attribute__((__packed__)) _
isis_vu_e__get_rx_telemetry__from_t
229 {
230     unsigned char raw[22];
231     struct __attribute__((__packed__))
232     {
233         int16_t doppler; /*!< Raw measurement value of the instantaneous Doppler
offset of the signal at the receiver port. \note conversion: eng. value = -38.1469726563 *
raw + 0*/
234         int16_t rssi; /*!< This field contains the measured Received Signal Strength
Indicator (RSSI) at the reception time \note conversion: eng. value = -0.5 * raw + -22*/
235         uint16_t voltage; /*!< Raw measurement value of the power bus voltage. \note

```

```

conversion: eng. value = 0.00488 * raw */
236     uint16_t current_total; /*!< Raw measurement value of the total supply current.
\note conversion: eng. value = 0.3052 * raw */

237     uint16_t current_tx; /*!< Raw measurement value of the transmitter current.
\note conversion: eng. value = 0.3052 * raw */

238     uint16_t current_rx; /*!< Raw measurement value of receiver current. \note
conversion: eng. value = 0.3052 * raw */
239     uint16_t current_pa; /*!< Raw measurement power amplifier current \note
conversion: eng. value = 0.3052 * raw */
240     uint16_t temp_pa; /*!< Raw measurement value of the power amplifier
temperature. \note conversion: eng. value = -0.07669 * raw + 195.6037*/

241     uint16_t temp_board; /*!< Raw measurement of board temperature \note
conversion: eng. value = -0.07669 * raw + 195.6037*/

242     int16_t last_doppler; /*!< Last packed received Doppler offset at the receiver
port. \note conversion: eng. value = -38.1469726563 * raw + 0*/

243     int16_t last_rssi; /*!< Last packed received Received Signal Strength Indicator
(RSSI) at the receiver port \note conversion: eng. value = -0.5 * raw + -22*/

244 } fields;
245 } isis_vu_e__get_rx_telemetry__from_t;
246
247     /*!
248 * Union for storing the parameters received by get_transponder_in_freq.
249 */
250     typedef union __attribute__((__packed__)) _
isis_vu_e__get_transponder_in_freq__from_t

251 {
252     unsigned char raw[5];
253     struct __attribute__((__packed__))
254     {
255         uint32_t freq; /*!< Current transponder input frequency in kHz */

256         uint8_t status; /*!< Operation status (0: OK, 252: AX5043 is not ready, 253:
AX5043 is busy, 254: AX5043 read error, 255: internal error) */

257     } fields;
258 } isis_vu_e__get_transponder_in_freq__from_t;
259
260     /*!
261 * Union for storing the parameters received by get_tx_pll_error.

```



```

262 */
263     typedef union __attribute__((__packed__)) _
isis_vu_e__get_tx_pll_error__from_t
264 {
265     unsigned char raw[4];
266     struct __attribute__((__packed__))
267     {
268         uint16_t locked_errcount; /*!< Locked error count */
269         uint16_t freq_errcount; /*!< Frequency error count */

270     } fields;
271 } isis_vu_e__get_tx_pll_error__from_t;
272
273     /*!
274  * Union for storing the parameters received by get_tx_swinfo.
275  */
276     typedef union __attribute__((__packed__)) _isis_vu_e__get_tx_swinfo__from_t

277 {
278     unsigned char raw[80];
279     struct __attribute__((__packed__))
280     {
281         char swinfo[80]; /*!< Firmware information */
282     } fields;
283 } isis_vu_e__get_tx_swinfo__from_t;
284
285     /*!
286  * Union for storing the parameters received by get_tx_telemetry.
287  */
288     typedef union __attribute__((__packed__)) _
isis_vu_e__get_tx_telemetry__from_t
289 {
290     unsigned char raw[18];
291     struct __attribute__((__packed__))
292     {
293         uint16_t reflected_power; /*!< Raw measurement value of the instantaneous
RF reflected power at the transmitter port. \note conversion: eng. value = 0.00005887 *
raw<sup>2</sup> */
294         uint16_t forward_power; /*!< Raw measurement value of the instantaneous
RF forward power at the transmitter port. \note conversion: eng. value = 0.00005887 *
raw<sup>2</sup> */
295         uint16_t voltage; /*!< Raw measurement value of the power bus voltage. \note
conversion: eng. value = 0.00488 * raw */
296         uint16_t current_total; /*!< Raw measurement value of the total supply current.
\note conversion: eng. value = 0.3052 * raw */

```

```

297     uint16_t current_tx; /*!< Raw measurement value of the transmitter current.
\note conversion: eng. value = 0.3052 * raw */

298     uint16_t current_rx; /*!< Raw measurement value of receiver current. \note
conversion: eng. value = 0.3052 * raw */
299     uint16_t current_pa; /*!< Raw measurement power amplifier current \note
conversion: eng. value = 0.3052 * raw */
300     uint16_t temp_pa; /*!< Raw measurement value of the power amplifier
temperature. \note conversion: eng. value = -0.07669 * raw + 195.6037*/

301     uint16_t temp_board; /*!< Raw measurement of board temperature \note
conversion: eng. value = -0.07669 * raw + 195.6037*/

302 } fields;
303 } isis_vu_e__get_tx_telemetry__from_t;
304
305 /*!
306 * Union for storing the parameters received by get_tx_telemetry_last.
307 */
308 typedef union __attribute__((__packed__)) _
isis_vu_e__get_tx_telemetry_last__from_t
309 {
310     unsigned char raw[18];
311     struct __attribute__((__packed__))
312     {
313         uint16_t reflected_power; /*!< Raw measurement value of the instantaneous
RF reflected power at the transmitter port. \note conversion: eng. value = 0.00005887 *
raw<sup>2</sup> */
314         uint16_t forward_power; /*!< Raw measurement value of the instantaneous
RF forward power at the transmitter port. \note conversion: eng. value = 0.00005887 *
raw<sup>2</sup> */
315         uint16_t voltage; /*!< Raw measurement value of the power bus voltage. \note
conversion: eng. value = 0.00488 * raw */
316         uint16_t current_total; /*!< Raw measurement value of the total supply current.
\note conversion: eng. value = 0.3052 * raw */

317         uint16_t current_tx; /*!< Raw measurement value of the transmitter current.
\note conversion: eng. value = 0.3052 * raw */

318         uint16_t current_rx; /*!< Raw measurement value of receiver current. \note
conversion: eng. value = 0.3052 * raw */
319         uint16_t current_pa; /*!< Raw measurement power amplifier current \note
conversion: eng. value = 0.3052 * raw */
320         uint16_t temp_pa; /*!< Raw measurement value of the power amplifier
temperature. \note conversion: eng. value = -0.07669 * raw + 195.6037*/

```

```

321     uint16_t temp_board; /*!< Raw measurement of board temperature \note
conversion: eng. value = -0.07669 * raw + 195.6037*/

322 } fields;
323 } isis_vu_e__get_tx_telemetry_last__from_t;
324
325     /*!
326 * Structure the parameters sent by send_frame_with_callsigns.
327 */
328     typedef struct __attribute__((__packed__))
329 _isis_vu_e__send_frame_with_callsigns__to_t

330 {
331     uint8_t from_callsign[7]; /*!< The new 7 characters from callsign. */

332     uint8_t to_callsign[7]; /*!< The new 7 characters to callsign. */

333     uint8_t* data; /*!< Data to put in the AX.25 message (maximum length: 235
elements) */
334 } isis_vu_e__send_frame_with_callsigns__to_t;
335
336     /*!
337 * Structure the parameters sent by set_beacon.
338 */
339     typedef struct __attribute__((__packed__))
340 _isis_vu_e__set_beacon__to_t
341 {
342     uint16_t interval; /*!< Interval of beacon transmission */

343     uint8_t* data; /*!< Data to put in the frame (maximum length: 235 elements) */

344 } isis_vu_e__set_beacon__to_t;
345
346     /*!
347 * Structure the parameters sent by set_beacon_with_callsigns.
348 */
349     typedef struct __attribute__((__packed__))
350 _isis_vu_e__set_beacon_with_callsigns__to_t

351 {
352     uint16_t interval; /*!< Interval of beacon transmission */

353     uint8_t from_callsign[7]; /*!< The new 7 characters from callsign. */

354     uint8_t to_callsign[7]; /*!< The new 7 characters to callsign. */

```

```

355     uint8_t* data; /*!< Data to put in the AX.25 message (maximum length: 235
elements) */
356 } isis_vu_e__set_beacon_with_callsigns__to_t;
357
358     /*!
359  * Union for storing the parameters sent by set_callsign_from.
360  */
361     typedef union __attribute__((__packed__)) _
isis_vu_e__set_callsign_from__to_t
362 {
363     unsigned char raw[7];
364     struct __attribute__((__packed__))
365     {
366         char ascii[6]; /*!< AX.25 TO callsign to be used for AX.25 frames */
367
368         uint8_t number; /*!< */
369     } fields;
370 } isis_vu_e__set_callsign_from__to_t;
371
372     /*!
373  * Union for storing the parameters sent by set_callsign_to.
374  */
375     typedef union __attribute__((__packed__)) _isis_vu_e__set_callsign_to__to_t
376 {
377     unsigned char raw[7];
378     struct __attribute__((__packed__))
379     {
380         char ascii[6]; /*!< AX.25 TO callsign to be used for AX.25 frames */
381
382         uint8_t number; /*!< */
383     } fields;
384 } isis_vu_e__set_callsign_to__to_t;
385
386     /*!
387  * Union for storing the parameters received by state.
388  */
389     typedef union __attribute__((__packed__)) _isis_vu_e__state__from_t
390 {
391     unsigned char raw[1];
392     struct __attribute__((__packed__))
393     {
394         isis_vu_e__onoff_t idle : 1; /*!< Transmitter idle state */
395
396         isis_vu_e__onoff_t beacon : 1; /*!< Beacon active */

```

```
394     isis_vu_e__bitrate_tlm_t bitrate : 2; /*!< Transmission bitrate. */
395 } fields;
396 } isis_vu_e__state__from_t;
397
398     #ifdef __cplusplus
399 }
400     #endif
401
402     #endif /* ISIS_VU_E_TYPES_H_ */
403
404
```

```

1 /*
2  * isismepsv2_ivid7_piu_types.h
3  *
4  * AUTOGENERATED CODE
5  * Please do not perform manual edits
6  * Generated using autogen v1.0.3
7  *
8  * Generated from:
9  * - imepsv2_structs.yaml
10 * - imepsv2_ivid7_piu.yaml
11 */
12
13     #ifndef ISISMEPSV2_IVID7_PIU_TYPES_H_
14
15
16     #include <stddef.h>
17     #include <stdint.h>
18
19     #ifdef __cplusplus
20     extern "C"
21 {
22     #endif
23
24     /* DEFINES */
25
26     #define ISISMEPSV2_IVID7_PIU_NOP_ID {0x1A, 0x07, 0x02, 0x00}
27
28     #define ISISMEPSV2_IVID7_PIU_CANCEL_ID {0x1A, 0x07, 0x04, 0x00}
29
30     #define ISISMEPSV2_IVID7_PIU_RESETWATCHDOG_ID {0x1A, 0x07, 0x06,
0x00}
31     #define ISISMEPSV2_IVID7_PIU_OUTPUTBUSGROUPON_ID {0x1A, 0x07,
0x10, 0x00}
32     #define ISISMEPSV2_IVID7_PIU_OUTPUTBUSGROUPOFF_ID {0x1A, 0x07,
0x12, 0x00}
33     #define ISISMEPSV2_IVID7_PIU_OUTPUTBUSGROUPSTATE_ID {0x1A,
0x07, 0x14, 0x00}
34     #define ISISMEPSV2_IVID7_PIU_OUTPUTBUSCHANNELON_ID {0x1A, 0x07,
0x16, 0x00}
35     #define ISISMEPSV2_IVID7_PIU_OUTPUTBUSCHANNELOFF_ID {0x1A,
0x07, 0x18, 0x00}
36     #define ISISMEPSV2_IVID7_PIU_SWITCHTONOMINAL_ID {0x1A, 0x07, 0x30,
0x00}
37     #define ISISMEPSV2_IVID7_PIU_SWITCHTOSAFETY_ID {0x1A, 0x07, 0x32,
0x00}

```

```

0x00}
36  #define ISISMEPSV2_IVID7_PIU_GETSYSTEMSTATUS_ID {0x1A, 0x07,
0x40, 0x00}
37  #define ISISMEPSV2_IVID7_PIU_GETOVERCURRENTFAULTSTATE_ID
{0x1A, 0x07, 0x42, 0x00}
38  #define ISISMEPSV2_IVID7_PIU_GETCONFIGURATIONPARAMETER_ID
{0x1A, 0x07, 0x82, 0x00}
39  #define ISISMEPSV2_IVID7_PIU_SETCONFIGURATIONPARAMETER_ID
{0x1A, 0x07, 0x84, 0x00}
40  #define ISISMEPSV2_IVID7_PIU_RESETCONFIGURATIONPARAMETER_ID
{0x1A, 0x07, 0x86, 0x00}
41  #define ISISMEPSV2_IVID7_PIU_RESETCONFIGURATION_ID {0x1A, 0x07,
0x90, 0x00}
42  #define ISISMEPSV2_IVID7_PIU_LOADCONFIGURATION_ID {0x1A, 0x07,
0x92, 0x00}
43  #define ISISMEPSV2_IVID7_PIU_SAVECONFIGURATION_ID {0x1A, 0x07,
0x94, 0x00}
44  #define ISISMEPSV2_IVID7_PIU_GETHOUSEKEEPINGRAW_ID {0x1A, 0x07,
0xA0, 0x00}
45  #define ISISMEPSV2_IVID7_PIU_GETHOUSEKEEPINGRAWINCDB_ID
{0x1A, 0x07, 0xA0, 0x00}
46  #define ISISMEPSV2_IVID7_PIU_GETHOUSEKEEPINGENG_ID {0x1A, 0x07,
0xA2, 0x00}
47  #define ISISMEPSV2_IVID7_PIU_GETHOUSEKEEPINGENGINECDB_ID
{0x1A, 0x07, 0xA2, 0x00}
48  #define
ISISMEPSV2_IVID7_PIU_GETHOUSEKEEPINGENGRUNNINGAVGINCDB_ID {0x1A,
0x07, 0xA4, 0x00}
49  #define ISISMEPSV2_IVID7_PIU_GETHOUSEKEEPINGRUNNINGAVG_ID
{0x1A, 0x07, 0xA4, 0x00}
50  #define ISISMEPSV2_IVID7_PIU_RESET_ID {0x1A, 0x07, 0xAA, 0x00}

51  #define ISISMEPSV2_IVID7_PIU_CORRECTTIME_ID {0x1A, 0x07, 0xC4,
0x00}
52  #define ISISMEPSV2_IVID7_PIU_ZERORESETCAUSECOUNTERS_ID {0x1A,
0x07, 0xC6, 0x00}
53
54
55  /* ENUMS */
56
57  /*!
58  * Enumeration of all iMEPS channels
59  */
60  typedef enum __attribute__((__packed__))
isimepsv2_ivid7_piu__imeps_channel_t
61 {

```

```

62  isismepsv2_ivid7_piu__imeps_channel__channel_16v_perm = 0u,
63  isismepsv2_ivid7_piu__imeps_channel__channel_5v_perm = 1u,
64  isismepsv2_ivid7_piu__imeps_channel__channel_5v_sw1 = 2u,
65  isismepsv2_ivid7_piu__imeps_channel__channel_5v_sw2 = 3u,
66  isismepsv2_ivid7_piu__imeps_channel__channel_5v_sw3 = 4u,
67  isismepsv2_ivid7_piu__imeps_channel__channel_3v3_perm = 5u,
68  isismepsv2_ivid7_piu__imeps_channel__channel_3v3_sw1 = 6u,
69  isismepsv2_ivid7_piu__imeps_channel__channel_3v3_sw2 = 7u,
70  isismepsv2_ivid7_piu__imeps_channel__channel_3v3_sw3 = 8u,
71 } isismepsv2_ivid7_piu__imeps_channel_t;
72
73  /* STRUCTS */
74
75  /*!
76  * ISISMEPSV2_IVID7_PIU instance structure
77  */
78  typedef struct
79  {
80      uint8_t i2cAddr; /*!< I2C address used for this instance */
81
82  } ISISMEPSV2_IVID7_PIU_t;
83
84  /*!
85  * Union for storing the parameters for struct ReplyHeader.
86  */
87  typedef union __attribute__((__packed__)) _
ismepsv2_ivid7_piu__replyheader_t
88  {
89      unsigned char raw[5];
90      struct __attribute__((__packed__))
91      {
92          uint8_t stid; /*!< See "System Type Identifier (STID)" (page 22) */
93
94          uint8_t ivid; /*!< See "Interface Version Identifier (IVID)" (page 23) */
95
96          uint8_t rc; /*!< See "Board Identifier (BID)" (page 24) */
97
98          uint8_t bid; /*!< Response code as stated in the header (inside parenthesis). */
99
100         uint8_t cmderr : 4; /*!< See "Response Status Information (STAT)" (page 25) */
101
102         uint8_t reserved : 3; /*!< See "Response Status Information (STAT)" (page 25) */
103
104         uint8_t new_flag : 1; /*!< See "Response Status Information (STAT)" (page 25) */
105     } fields;

```



```

99 } isismepsv2_ivid7_piu__replyheader_t;
100
101  /*!
102  * Union for storing the parameters for struct VIPDch.
103  */
104  typedef union __attribute__((__packed__)) _isimepsv2_ivid7_piu__vipdch_t
105  {
106      unsigned char raw[6];
107      struct __attribute__((__packed__))
108      {
109          int16_t volt; /*!< \note conversion: eng. value [in mV] = 0.9765625 * raw + 0*/
110
111          int16_t current; /*!< \note conversion: eng. value [in mA] = 0.1525879 * raw +
0*/
112          int16_t power; /*!< \note conversion: eng. value [in mW] = 4.8828125 * raw +
0*/
113      } fields;
114  } isimepsv2_ivid7_piu__vipdch_t;
115
116  /*!
117  * Union for storing the parameters for struct VIPDvd.
118  */
119  typedef union __attribute__((__packed__)) _isimepsv2_ivid7_piu__vipdvd_t
120  {
121      unsigned char raw[6];
122      struct __attribute__((__packed__))
123      {
124          int16_t volt; /*!< \note conversion: eng. value [in mV] = 0.9765625 * raw + 0*/
125
126          int16_t current; /*!< \note conversion: eng. value [in mA] = 0.30517578 * raw +
0*/
127          int16_t power; /*!< \note conversion: eng. value [in mW] = 9.765625 * raw + 0*/
128      } fields;
129  } isimepsv2_ivid7_piu__vipdvd_t;
130
131  /*!
132  * Union for storing the parameters for struct VIPDeng.
133  */
134  typedef union __attribute__((__packed__)) _isimepsv2_ivid7_piu__vipdeng_t
135  {
136      unsigned char raw[6];
137      struct __attribute__((__packed__))

```

```

136  {
137      int16_t volt; /*!< */
138      int16_t current; /*!< */
139      int16_t power; /*!< \note conversion: eng. value [in mW] = 10 * raw */

140  } fields;
141 } isismepsv2_ivid7_piu__vipdeng_t;
142
143  /*!
144  * Union for storing the parameters for struct BatteryPackStatus.
145  */
146  typedef union __attribute__((__packed__)) _
isismepsv2_ivid7_piu__batterypackstatus_t
147 {
148     unsigned char raw[2];
149     struct __attribute__((__packed__))
150     {
151         uint16_t under_voltage_cell_1 : 1; /*!< Battery cell 1 under voltage */

152         uint16_t under_voltage_cell_2 : 1; /*!< Battery cell 2 under voltage */

153         uint16_t under_voltage_cell_3 : 1; /*!< Battery cell 3 under voltage */

154         uint16_t under_voltage_cell_4 : 1; /*!< Battery cell 4 under voltage */

155         uint16_t over_voltage_cell_1 : 1; /*!< Battery cell 1 over voltage */

156         uint16_t over_voltage_cell_2 : 1; /*!< Battery cell 2 over voltage */

157         uint16_t over_voltage_cell_3 : 1; /*!< Battery cell 3 over voltage */

158         uint16_t over_voltage_cell_4 : 1; /*!< Battery cell 4 over voltage */

159         uint16_t balancing_cell_1 : 1; /*!< Battery cell 1 balancing */

160         uint16_t balancing_cell_2 : 1; /*!< Battery cell 2 balancing */

161         uint16_t balancing_cell_3 : 1; /*!< Battery cell 3 balancing */

162         uint16_t balancing_cell_4 : 1; /*!< Battery cell 4 balancing */

163         uint16_t heater_active : 1; /*!< Battery pack heater status */

164         uint16_t reserved : 2; /*!< Battery pack heater status */

165         uint16_t pack_enabled : 1; /*!< Battery pack enabled. Bitflag is set when the

```

```

battery is connected to the output bus. */
166     } fields;
167 } isimepsv2_ivid7_piu__batterypackstatus_t;
168
169     /*!
170  * Union for storing the parameters for struct CCSDraw.
171  */
172     typedef union __attribute__((__packed__)) _isimepsv2_ivid7_piu__ccsdraw_t

173 {
174     unsigned char raw[8];
175     struct __attribute__((__packed__))
176     {
177         uint16_t volt_in_mppt; /*!< \note conversion: eng. value [in mV] = 20.5078125
* raw + 0*/
178         uint16_t curr_in_mppt; /*!< \note conversion: eng. value [in mA] =
3.255208333 * raw + -41*/
179         uint16_t volt_out_mppt; /*!< \note conversion: eng. value [in mV] =
20.5078125 * raw + 0*/
180         uint16_t curr_out_mppt; /*!< \note conversion: eng. value [in mA] =
3.255208333 * raw + -41*/
181     } fields;
182 } isimepsv2_ivid7_piu__ccsdraw_t;
183
184     /*!
185  * Union for storing the parameters for struct CCSDeng.
186  */
187     typedef union __attribute__((__packed__)) _isimepsv2_ivid7_piu__ccsdeng_t

188 {
189     unsigned char raw[8];
190     struct __attribute__((__packed__))
191     {
192         int16_t volt_in_mppt; /*!< */
193         int16_t curr_in_mppt; /*!< */
194         int16_t volt_out_mppt; /*!< */
195         int16_t curr_out_mppt; /*!< */
196     } fields;
197 } isimepsv2_ivid7_piu__ccsdeng_t;
198
199     /*!
200  * Union for storing the parameters received by getconfigurationparameter.
201  */
202     typedef union __attribute__((__packed__)) _
isimepsv2_ivid7_piu__getconfigurationparameter__from_t

```

```

203 {
204     unsigned char raw[16];
205     struct __attribute__((packed))
206     {
207         isimepsv2_ivid7_piu__replyheader_t reply_header; /*!< */
208         uint8_t reserved; /*!< */
209         uint16_t par_id; /*!< parameter id of the parameter to get */

210         uint8_t par_val[8]; /*!< parameter value encompassing n bytes. The length
depends on the parameter type. */
211     } fields;
212 } isimepsv2_ivid7_piu__getconfigurationparameter__from_t;
213
214     /*!
215 * Union for storing the parameters received by gethousekeepingeng.
216 */
217     typedef union __attribute__((packed)) _
isimepsv2_ivid7_piu__gethousekeepingeng__from_t

218 {
219     unsigned char raw[116];
220     struct __attribute__((packed))
221     {
222         isimepsv2_ivid7_piu__replyheader_t reply_header; /*!< "See \\"System Type
Identifier (STID)\\" (page 22)" */
223         uint8_t reserved; /*!< reserved */
224         int16_t volt_brdsup; /*!< Voltage of internal board supply in raw form */

225         int16_t temp; /*!< Measured temperature provided by a sensor internal to the
MCU in raw form \note conversion: eng. value [in °C] = 0.01 * raw */

226         isimepsv2_ivid7_piu__vipdeng_t dist_input; /*!< Input V, I and P data taken
at the input of the distribution part of the unit in raw form. Negative values indicate
output flow. */
227         isimepsv2_ivid7_piu__vipdeng_t batt_input; /*!< Input V, I and P data taken
at the input of the battery part of the unit in raw form. Negative values indicate output
flow. */
228         uint16_t stat_obc_on; /*!< Bitflag field indicating channel-on status for the
output bus channels. */
229         uint16_t stat_obc_ocf; /*!< Bitflag field indicating overcurrent latch-off fault
status for the output bus channels. */
230         isimepsv2_ivid7_piu__batterypackstatus_t bat_stat; /*!< Bitflag field
indicating BP board */
231         int16_t temp2; /*!< 2 and 4 cell battery pack: Battery pack temperature in
between the center battery cells. \note conversion: eng. value [in °C] = 0.01 * raw */

```

```

232     int16_t temp3; /*!< 2 cell battery pack: not used 4 cell battery pack: Battery
pack temperature on the front of the battery pack. \note conversion: eng. value [in °C] =
0.01 * raw */
233     int16_t volt_vd0; /*!< Voltage of voltage domain 0 */

234     int16_t volt_vd1; /*!< Voltage of voltage domain 0 */

235     int16_t volt_vd2; /*!< Voltage of voltage domain 0 */

236     isimepsv2_ivid7_piu__vipdeng_t vip_obc00; /*!< Output V, I and P of output
bus channel */
237     isimepsv2_ivid7_piu__vipdeng_t vip_obc01; /*!< Output V, I and P of output
bus channel */
238     isimepsv2_ivid7_piu__vipdeng_t vip_obc02; /*!< Output V, I and P of output
bus channel */
239     isimepsv2_ivid7_piu__vipdeng_t vip_obc03; /*!< Output V, I and P of output
bus channel */
240     isimepsv2_ivid7_piu__vipdeng_t vip_obc04; /*!< Output V, I and P of output
bus channel */
241     isimepsv2_ivid7_piu__vipdeng_t vip_obc05; /*!< Output V, I and P of output
bus channel */
242     isimepsv2_ivid7_piu__vipdeng_t vip_obc06; /*!< Output V, I and P of output
bus channel */
243     isimepsv2_ivid7_piu__vipdeng_t vip_obc07; /*!< Output V, I and P of output
bus channel */
244     isimepsv2_ivid7_piu__vipdeng_t vip_obc08; /*!< Output V, I and P of output
bus channel */
245     isimepsv2_ivid7_piu__ccsdeng_t cc1; /*!< Data on conditioning chain */

246     isimepsv2_ivid7_piu__ccsdeng_t cc2; /*!< Data on conditioning chain */

247     isimepsv2_ivid7_piu__ccsdeng_t cc3; /*!< Data on conditioning chain */

248 } fields;
249 } isimepsv2_ivid7_piu__gethousekeepingeng__from_t;
250
251 /*!
252 * Union for storing the parameters received by gethousekeepingengincdb.
253 */
254 typedef union __attribute__((packed)) _
isimepsv2_ivid7_piu__gethousekeepingengincdb__from_t

255 {
256     unsigned char raw[274];
257     struct __attribute__((packed))
258     {

```

```

259     isismepsv2_ivid7_piu__replyheader_t reply_header; /*!< "See \""System Type
Identifier (STID)\"" (page 22)" */
260     uint8_t reserved; /*!< reserved */
261     int16_t volt_brdsup; /*!< Voltage of internal board supply in raw form */

262     int16_t temp; /*!< Measured temperature provided by a sensor internal to the
MCU in raw form \note conversion: eng. value [in °C] = 0.01 * raw */

263     isismepsv2_ivid7_piu__vipdeng_t dist_input; /*!< Input V, I and P data taken
at the input of the distribution part of the unit in raw form. Negative values indicate
output flow. */
264     isismepsv2_ivid7_piu__vipdeng_t batt_input; /*!< Input V, I and P data taken
at the input of the battery part of the unit in raw form. Negative values indicate output
flow. */
265     uint16_t stat_obc_on; /*!< Bitflag field indicating channel-on status for the
output bus channels. */
266     uint16_t stat_obc_ocf; /*!< Bitflag field indicating overcurrent latch-off fault
status for the output bus channels. */
267     isismepsv2_ivid7_piu__batterypackstatus_t bat_stat; /*!< Bitflag field
indicating BP board */
268     int16_t temp2; /*!< 2 and 4 cell battery pack: Battery pack temperature in
between the center battery cells. */
269     int16_t temp3; /*!< 2 cell battery pack: not used 4 cell battery pack: Battery
pack temperature on the front of the battery pack. */

270     int16_t volt_vd0; /*!< Voltage of voltage domain 0 */

271     int16_t volt_vd1; /*!< Voltage of voltage domain 0 */

272     int16_t volt_vd2; /*!< Voltage of voltage domain 0 */

273     isismepsv2_ivid7_piu__vipdeng_t vip_obc00; /*!< Output V, I and P of output
bus channel */
274     isismepsv2_ivid7_piu__vipdeng_t vip_obc01; /*!< Output V, I and P of output
bus channel */
275     isismepsv2_ivid7_piu__vipdeng_t vip_obc02; /*!< Output V, I and P of output
bus channel */
276     isismepsv2_ivid7_piu__vipdeng_t vip_obc03; /*!< Output V, I and P of output
bus channel */
277     isismepsv2_ivid7_piu__vipdeng_t vip_obc04; /*!< Output V, I and P of output
bus channel */
278     isismepsv2_ivid7_piu__vipdeng_t vip_obc05; /*!< Output V, I and P of output
bus channel */
279     isismepsv2_ivid7_piu__vipdeng_t vip_obc06; /*!< Output V, I and P of output
bus channel */
280     isismepsv2_ivid7_piu__vipdeng_t vip_obc07; /*!< Output V, I and P of output

```

```

bus channel */
281     isismepsv2_ivid7_piu__vipdeng_t vip_obc08; /*!< Output V, I and P of output
bus channel */
282     isismepsv2_ivid7_piu__ccsdeng_t cc1; /*!< Data on conditioning chain */

283     isismepsv2_ivid7_piu__ccsdeng_t cc2; /*!< Data on conditioning chain */

284     isismepsv2_ivid7_piu__ccsdeng_t cc3; /*!< Data on conditioning chain */

285     isismepsv2_ivid7_piu__vipdeng_t vip_obc09; /*!< Output V, I and P of output
bus channel */
286     isismepsv2_ivid7_piu__vipdeng_t vip_obc10; /*!< Output V, I and P of output
bus channel */
287     isismepsv2_ivid7_piu__vipdeng_t vip_obc11; /*!< Output V, I and P of output
bus channel */
288     isismepsv2_ivid7_piu__vipdeng_t vip_obc12; /*!< Output V, I and P of output
bus channel */
289     isismepsv2_ivid7_piu__vipdeng_t vip_obc13; /*!< Output V, I and P of output
bus channel */
290     isismepsv2_ivid7_piu__vipdeng_t vip_obc14; /*!< Output V, I and P of output
bus channel */
291     isismepsv2_ivid7_piu__vipdeng_t vip_obc15; /*!< Output V, I and P of output
bus channel */
292     isismepsv2_ivid7_piu__ccsdeng_t cc4; /*!< Data on conditioning chain */

293     isismepsv2_ivid7_piu__ccsdeng_t cc5; /*!< Data on conditioning chain */

294     uint16_t stat_ob_ex_on; /*!< Bitflag field indicating channel-on status for
theextended output bus channels 16 through 31. */

295     uint16_t stat_ob_ex_ocf; /*!< Bitflag field indicating overcurrent latch-off fault
status for the extended output bus channels 16 through 31. */

296     isismepsv2_ivid7_piu__vipdeng_t vip_obc16; /*!< Output V, I and P of output
bus channel */
297     isismepsv2_ivid7_piu__vipdeng_t vip_obc17; /*!< Output V, I and P of output
bus channel */
298     isismepsv2_ivid7_piu__vipdeng_t vip_obc18; /*!< Output V, I and P of output
bus channel */
299     isismepsv2_ivid7_piu__vipdeng_t vip_obc19; /*!< Output V, I and P of output
bus channel */
300     isismepsv2_ivid7_piu__vipdeng_t vip_obc20; /*!< Output V, I and P of output
bus channel */
301     isismepsv2_ivid7_piu__vipdeng_t vip_obc21; /*!< Output V, I and P of output
bus channel */
302     isismepsv2_ivid7_piu__vipdeng_t vip_obc22; /*!< Output V, I and P of output

```

```

bus channel */
303     isismepsv2_ivid7_piu__vipdeng_t vip_obc23; /*!< Output V, I and P of output
bus channel */
304     isismepsv2_ivid7_piu__vipdeng_t vip_obc24; /*!< Output V, I and P of output
bus channel */
305     isismepsv2_ivid7_piu__vipdeng_t vip_obc25; /*!< Output V, I and P of output
bus channel */
306     isismepsv2_ivid7_piu__vipdeng_t vip_obc26; /*!< Output V, I and P of output
bus channel */
307     isismepsv2_ivid7_piu__vipdeng_t vip_obc27; /*!< Output V, I and P of output
bus channel */
308     isismepsv2_ivid7_piu__vipdeng_t vip_obc28; /*!< Output V, I and P of output
bus channel */
309     isismepsv2_ivid7_piu__vipdeng_t vip_obc29; /*!< Output V, I and P of output
bus channel */
310     isismepsv2_ivid7_piu__vipdeng_t vip_obc30; /*!< Output V, I and P of output
bus channel */
311     isismepsv2_ivid7_piu__vipdeng_t vip_obc31; /*!< Output V, I and P of output
bus channel */
312 } fields;
313 } isismepsv2_ivid7_piu__gethousekeepingengincdb__from_t;
314
315     /*!
316 * Union for storing the parameters received by
gethousekeepingengrunningavgincdb.
317 */
318     typedef union __attribute__((__packed__)) _
isismepsv2_ivid7_piu__gethousekeepingengrunningavgincdb__from_t

319 {
320     unsigned char raw[274];
321     struct __attribute__((__packed__))
322     {
323         isismepsv2_ivid7_piu__replyheader_t reply_header; /*!< "See \""System Type
Identifier (STID)\"" (page 22)" */
324         uint8_t reserved; /*!< reserved */
325         int16_t volt_brdsup; /*!< Voltage of internal board supply in raw form */

326         int16_t temp; /*!< Measured temperature provided by a sensor internal to the
MCU in raw form \note conversion: eng. value = 0.01 * raw */

327         isismepsv2_ivid7_piu__vipdeng_t dist_input; /*!< Input V, I and P data taken
at the input of the distribution part of the unit in raw form. Negative values indicate
output flow. */
328         isismepsv2_ivid7_piu__vipdeng_t batt_input; /*!< Input V, I and P data taken
at the input of the battery part of the unit in raw form. Negative values indicate output

```


flow. */

329 uint16_t stat_obc_on; /*!< Bitflag field indicating channel-on status for the output bus channels. */

330 uint16_t stat_obc_ocf; /*!< Bitflag field indicating overcurrent latch-off fault status for the output bus channels. */

331 isimepsv2_ivid7_piu__batterypackstatus_t bat_stat; /*!< Bitflag field indicating BP board */

332 int16_t temp2; /*!< 2 and 4 cell battery pack: Battery pack temperature in between the center battery cells. */

333 int16_t temp3; /*!< 2 cell battery pack: not used 4 cell battery pack: Battery pack temperature on the front of the battery pack. */

334 int16_t volt_vd0; /*!< Voltage of voltage domain 0 */

335 int16_t volt_vd1; /*!< Voltage of voltage domain 0 */

336 int16_t volt_vd2; /*!< Voltage of voltage domain 0 */

337 isimepsv2_ivid7_piu__vipdeng_t vip_obc00; /*!< Output V, I and P of output bus channel */

338 isimepsv2_ivid7_piu__vipdeng_t vip_obc01; /*!< Output V, I and P of output bus channel */

339 isimepsv2_ivid7_piu__vipdeng_t vip_obc02; /*!< Output V, I and P of output bus channel */

340 isimepsv2_ivid7_piu__vipdeng_t vip_obc03; /*!< Output V, I and P of output bus channel */

341 isimepsv2_ivid7_piu__vipdeng_t vip_obc04; /*!< Output V, I and P of output bus channel */

342 isimepsv2_ivid7_piu__vipdeng_t vip_obc05; /*!< Output V, I and P of output bus channel */

343 isimepsv2_ivid7_piu__vipdeng_t vip_obc06; /*!< Output V, I and P of output bus channel */

344 isimepsv2_ivid7_piu__vipdeng_t vip_obc07; /*!< Output V, I and P of output bus channel */

345 isimepsv2_ivid7_piu__vipdeng_t vip_obc08; /*!< Output V, I and P of output bus channel */

346 isimepsv2_ivid7_piu__ccsdeng_t cc1; /*!< Data on conditioning chain */

347 isimepsv2_ivid7_piu__ccsdeng_t cc2; /*!< Data on conditioning chain */

348 isimepsv2_ivid7_piu__ccsdeng_t cc3; /*!< Data on conditioning chain */

349 isimepsv2_ivid7_piu__vipdeng_t vip_obc09; /*!< Output V, I and P of output bus channel */

350 isimepsv2_ivid7_piu__vipdeng_t vip_obc10; /*!< Output V, I and P of output bus channel */

351 isimepsv2_ivid7_piu__vipdeng_t vip_obc11; /*!< Output V, I and P of output
bus channel */
352 isimepsv2_ivid7_piu__vipdeng_t vip_obc12; /*!< Output V, I and P of output
bus channel */
353 isimepsv2_ivid7_piu__vipdeng_t vip_obc13; /*!< Output V, I and P of output
bus channel */
354 isimepsv2_ivid7_piu__vipdeng_t vip_obc14; /*!< Output V, I and P of output
bus channel */
355 isimepsv2_ivid7_piu__vipdeng_t vip_obc15; /*!< Output V, I and P of output
bus channel */
356 isimepsv2_ivid7_piu__ccsdeng_t cc4; /*!< Data on conditioning chain */

357 isimepsv2_ivid7_piu__ccsdeng_t cc5; /*!< Data on conditioning chain */

358 uint16_t stat_ob_ex_on; /*!< Bitflag field indicating channel-on status for
theextended output bus channels 16 through 31. */

359 uint16_t stat_ob_ex_ocf; /*!< Bitflag field indicating overcurrent latch-off fault
status for the extended output bus channels 16 through 31. */

360 isimepsv2_ivid7_piu__vipdeng_t vip_obc16; /*!< Output V, I and P of output
bus channel */
361 isimepsv2_ivid7_piu__vipdeng_t vip_obc17; /*!< Output V, I and P of output
bus channel */
362 isimepsv2_ivid7_piu__vipdeng_t vip_obc18; /*!< Output V, I and P of output
bus channel */
363 isimepsv2_ivid7_piu__vipdeng_t vip_obc19; /*!< Output V, I and P of output
bus channel */
364 isimepsv2_ivid7_piu__vipdeng_t vip_obc20; /*!< Output V, I and P of output
bus channel */
365 isimepsv2_ivid7_piu__vipdeng_t vip_obc21; /*!< Output V, I and P of output
bus channel */
366 isimepsv2_ivid7_piu__vipdeng_t vip_obc22; /*!< Output V, I and P of output
bus channel */
367 isimepsv2_ivid7_piu__vipdeng_t vip_obc23; /*!< Output V, I and P of output
bus channel */
368 isimepsv2_ivid7_piu__vipdeng_t vip_obc24; /*!< Output V, I and P of output
bus channel */
369 isimepsv2_ivid7_piu__vipdeng_t vip_obc25; /*!< Output V, I and P of output
bus channel */
370 isimepsv2_ivid7_piu__vipdeng_t vip_obc26; /*!< Output V, I and P of output
bus channel */
371 isimepsv2_ivid7_piu__vipdeng_t vip_obc27; /*!< Output V, I and P of output
bus channel */
372 isimepsv2_ivid7_piu__vipdeng_t vip_obc28; /*!< Output V, I and P of output
bus channel */

```

373     isimepsv2_ivid7_piu__vipdeng_t vip_obc29; /*!< Output V, I and P of output
bus channel */
374     isimepsv2_ivid7_piu__vipdeng_t vip_obc30; /*!< Output V, I and P of output
bus channel */
375     isimepsv2_ivid7_piu__vipdeng_t vip_obc31; /*!< Output V, I and P of output
bus channel */
376 } fields;
377 } isimepsv2_ivid7_piu__gethousekeepingengrunningavgincdb__from_t;
378
379 /*!
380 * Union for storing the parameters received by gethousekeepingraw.
381 */
382 typedef union __attribute__((__packed__)) _
isimepsv2_ivid7_piu__gethousekeepingraw__from_t

383 {
384     unsigned char raw[174];
385     struct __attribute__((__packed__))
386     {
387         isimepsv2_ivid7_piu__replyheader_t reply_header; /*!< "See \""System Type
Identifier (STID)\"" (page 22)" */
388         uint8_t reserved; /*!< reserved */
389         uint16_t volt_brdsup; /*!< Voltage of internal board supply in raw form */

390         uint16_t temp; /*!< Measured temperature provided by a sensor internal to the
MCU in raw form \note conversion: eng. value [in C] = 0.32234 * raw + -280*/

391         isimepsv2_ivid7_piu__vipdvd_t dist_input; /*!< Input V, I and P data taken at
the input of the distribution part of the unit in raw form. Negative values indicate output
flow. */
392         isimepsv2_ivid7_piu__vipdvd_t batt_input; /*!< Input V, I and P data taken at
the input of the battery part of the unit in raw form. Negative values indicate output flow.
*/
393         uint16_t stat_ob_on; /*!< Bitflag field indicating channel-on status for output
bus channels 0 through 15. */
394         uint16_t stat_ob_ocf; /*!< Bitflag field indicating overcurrent latch-off fault
status for output bus channels 0 through 15. */

395         isimepsv2_ivid7_piu__batterypackstatus_t bat_stat; /*!< Bitflag field
indicating BP board */
396         uint16_t temp2; /*!< 2 and 4 cell battery pack: Battery pack temperature in
between the center battery cells. \note conversion: eng. value = -0.00000000334856 *
raw<sup>3</sup> + 0.0000221414 * raw<sup>2</sup> + -0.066728 * raw + 93.3244*/

397         uint16_t temp3; /*!< 2 cell battery pack: not used 4 cell battery pack: Battery
pack temperature on the front of the battery pack. \note conversion: eng. value =

```

```

-0.00000000334856 * raw<sup>3</sup> + 0.0000221414 * raw<sup>2</sup> +
-0.066728 * raw + 93.3244*/
398     uint16_t volt_vd0; /*!< Voltage of voltage domain 0 in raw form */

399     uint16_t volt_vd1; /*!< Voltage of voltage domain 1 in raw form */

400     uint16_t volt_vd2; /*!< Voltage of voltage domain 2 in raw form */

401     isimepsv2_ivid7_piu__vipdch_t vip_obc00; /*!< Output V, I and P of output
bus channel in raw form */
402     isimepsv2_ivid7_piu__vipdch_t vip_obc01; /*!< Output V, I and P of output
bus channel in raw form */
403     isimepsv2_ivid7_piu__vipdch_t vip_obc02; /*!< Output V, I and P of output
bus channel in raw form */
404     isimepsv2_ivid7_piu__vipdch_t vip_obc03; /*!< Output V, I and P of output
bus channel in raw form */
405     isimepsv2_ivid7_piu__vipdch_t vip_obc04; /*!< Output V, I and P of output
bus channel in raw form */
406     isimepsv2_ivid7_piu__vipdch_t vip_obc05; /*!< Output V, I and P of output
bus channel in raw form */
407     isimepsv2_ivid7_piu__vipdch_t vip_obc06; /*!< Output V, I and P of output
bus channel in raw form */
408     isimepsv2_ivid7_piu__vipdch_t vip_obc07; /*!< Output V, I and P of output
bus channel in raw form */
409     isimepsv2_ivid7_piu__vipdch_t vip_obc08; /*!< Output V, I and P of output
bus channel in raw form */
410     isimepsv2_ivid7_piu__ccsdraw_t cc1; /*!< Data on conditioning chain in raw
form. */
411     isimepsv2_ivid7_piu__ccsdraw_t cc2; /*!< Data on conditioning chain in raw
form. */
412     isimepsv2_ivid7_piu__ccsdraw_t cc3; /*!< Data on conditioning chain in raw
form. */
413     isimepsv2_ivid7_piu__vipdch_t vip_obc09; /*!< Output V, I and P of output
bus channel in raw form */
414     isimepsv2_ivid7_piu__vipdch_t vip_obc10; /*!< Output V, I and P of output
bus channel in raw form */
415     isimepsv2_ivid7_piu__vipdch_t vip_obc11; /*!< Output V, I and P of output
bus channel in raw form */
416     isimepsv2_ivid7_piu__vipdch_t vip_obc12; /*!< Output V, I and P of output
bus channel in raw form */
417     isimepsv2_ivid7_piu__vipdch_t vip_obc13; /*!< Output V, I and P of output
bus channel in raw form */
418     isimepsv2_ivid7_piu__vipdch_t vip_obc14; /*!< Output V, I and P of output
bus channel in raw form */
419     isimepsv2_ivid7_piu__vipdch_t vip_obc15; /*!< Output V, I and P of output
bus channel in raw form */

```

```

420     isimepsv2_ivid7_piu__ccsdraw_t cc4; /*!< Data on conditioning chain in raw
form. */
421     isimepsv2_ivid7_piu__ccsdraw_t cc5; /*!< Data on conditioning chain in raw
form. */
422 } fields;
423 } isimepsv2_ivid7_piu__gethousekeepingraw__from_t;
424
425     /*!
426 * Union for storing the parameters received by gethousekeepingrawincdb.
427 */
428     typedef union __attribute__((__packed__)) _
isimepsv2_ivid7_piu__gethousekeepingrawincdb__from_t

429 {
430     unsigned char raw[274];
431     struct __attribute__((__packed__))
432     {
433         isimepsv2_ivid7_piu__replyheader_t reply_header; /*!< "See \""System Type
Identifier (STID)\"" (page 22)" */
434         uint8_t reserved; /*!< reserved */
435         uint16_t volt_brdsup; /*!< Voltage of internal board supply in raw form */

436         uint16_t temp; /*!< Measured temperature provided by a sensor internal to the
MCU in raw form \note conversion: eng. value [in C] = 0.244444 * raw + -1168*/

437         isimepsv2_ivid7_piu__vipdvd_t dist_input; /*!< Input V, I and P data taken at
the input of the distribution part of the unit in raw form. Negative values indicate output
flow. */
438         isimepsv2_ivid7_piu__vipdvd_t batt_input; /*!< Input V, I and P data taken at
the input of the battery part of the unit in raw form. Negative values indicate output flow.
*/
439         uint16_t stat_obc_on; /*!< Bitflag field indicating channel-on status for the
output bus channels. */
440         uint16_t stat_obc_ocf; /*!< Bitflag field indicating overcurrent latch-off fault
status for the output bus channels. */
441         isimepsv2_ivid7_piu__batterypackstatus_t bat_stat; /*!< Bitflag field
indicating BP board */
442         uint16_t temp2; /*!< 2 and 4 cell battery pack: Battery pack temperature in
between the center battery cells. */
443         uint16_t temp3; /*!< 2 cell battery pack: not used 4 cell battery pack: Battery
pack temperature on the front of the battery pack. */

444         uint16_t volt_vd0; /*!< Voltage of voltage domain 0 in raw form */

445         uint16_t volt_vd1; /*!< Voltage of voltage domain 1 in raw form */

```

```

446      uint16_t volt_vd2; /*!< Voltage of voltage domain 2 in raw form */

447      isimepsv2_ivid7_piu__vipdch_t vip_obc00; /*!< Output V, I and P of output
bus channel in raw form */
448      isimepsv2_ivid7_piu__vipdch_t vip_obc01; /*!< Output V, I and P of output
bus channel in raw form */
449      isimepsv2_ivid7_piu__vipdch_t vip_obc02; /*!< Output V, I and P of output
bus channel in raw form */
450      isimepsv2_ivid7_piu__vipdch_t vip_obc03; /*!< Output V, I and P of output
bus channel in raw form */
451      isimepsv2_ivid7_piu__vipdch_t vip_obc04; /*!< Output V, I and P of output
bus channel in raw form */
452      isimepsv2_ivid7_piu__vipdch_t vip_obc05; /*!< Output V, I and P of output
bus channel in raw form */
453      isimepsv2_ivid7_piu__vipdch_t vip_obc06; /*!< Output V, I and P of output
bus channel in raw form */
454      isimepsv2_ivid7_piu__vipdch_t vip_obc07; /*!< Output V, I and P of output
bus channel in raw form */
455      isimepsv2_ivid7_piu__vipdch_t vip_obc08; /*!< Output V, I and P of output
bus channel in raw form */
456      isimepsv2_ivid7_piu__ccsdraw_t cc1; /*!< Data on conditioning chain in raw
form. */
457      isimepsv2_ivid7_piu__ccsdraw_t cc2; /*!< Data on conditioning chain in raw
form. */
458      isimepsv2_ivid7_piu__ccsdraw_t cc3; /*!< Data on conditioning chain in raw
form. */
459      isimepsv2_ivid7_piu__vipdch_t vip_obc09; /*!< Output V, I and P of output
bus channel in raw form */
460      isimepsv2_ivid7_piu__vipdch_t vip_obc10; /*!< Output V, I and P of output
bus channel in raw form */
461      isimepsv2_ivid7_piu__vipdch_t vip_obc11; /*!< Output V, I and P of output
bus channel in raw form */
462      isimepsv2_ivid7_piu__vipdch_t vip_obc12; /*!< Output V, I and P of output
bus channel in raw form */
463      isimepsv2_ivid7_piu__vipdch_t vip_obc13; /*!< Output V, I and P of output
bus channel in raw form */
464      isimepsv2_ivid7_piu__vipdch_t vip_obc14; /*!< Output V, I and P of output
bus channel in raw form */
465      isimepsv2_ivid7_piu__vipdch_t vip_obc15; /*!< Output V, I and P of output
bus channel in raw form */
466      isimepsv2_ivid7_piu__ccsdraw_t cc4; /*!< Data on conditioning chain in raw
form. */
467      isimepsv2_ivid7_piu__ccsdraw_t cc5; /*!< Data on conditioning chain in raw
form. */
468      uint16_t stat_ob_ex_on; /*!< Bitflag field indicating channel-on status for
theextended output bus channels 16 through 31. */

```



```
469     uint16_t stat_ob_ex_ocf; /*!< Bitflag field indicating overcurrent latch-off fault
status for the extended output bus channels 16 through 31. */
```

```
470     isimepsv2_ivid7_piu__vipdch_t vip_obc16; /*!< Output V, I and P of output
bus channel in raw form */
```

```
471     isimepsv2_ivid7_piu__vipdch_t vip_obc17; /*!< Output V, I and P of output
bus channel in raw form */
```

```
472     isimepsv2_ivid7_piu__vipdch_t vip_obc18; /*!< Output V, I and P of output
bus channel in raw form */
```

```
473     isimepsv2_ivid7_piu__vipdch_t vip_obc19; /*!< Output V, I and P of output
bus channel in raw form */
```

```
474     isimepsv2_ivid7_piu__vipdch_t vip_obc20; /*!< Output V, I and P of output
bus channel in raw form */
```

```
475     isimepsv2_ivid7_piu__vipdch_t vip_obc21; /*!< Output V, I and P of output
bus channel in raw form */
```

```
476     isimepsv2_ivid7_piu__vipdch_t vip_obc22; /*!< Output V, I and P of output
bus channel in raw form */
```

```
477     isimepsv2_ivid7_piu__vipdch_t vip_obc23; /*!< Output V, I and P of output
bus channel in raw form */
```

```
478     isimepsv2_ivid7_piu__vipdch_t vip_obc24; /*!< Output V, I and P of output
bus channel in raw form */
```

```
479     isimepsv2_ivid7_piu__vipdch_t vip_obc25; /*!< Output V, I and P of output
bus channel in raw form */
```

```
480     isimepsv2_ivid7_piu__vipdch_t vip_obc26; /*!< Output V, I and P of output
bus channel in raw form */
```

```
481     isimepsv2_ivid7_piu__vipdch_t vip_obc27; /*!< Output V, I and P of output
bus channel in raw form */
```

```
482     isimepsv2_ivid7_piu__vipdch_t vip_obc28; /*!< Output V, I and P of output
bus channel in raw form */
```

```
483     isimepsv2_ivid7_piu__vipdch_t vip_obc29; /*!< Output V, I and P of output
bus channel in raw form */
```

```
484     isimepsv2_ivid7_piu__vipdch_t vip_obc30; /*!< Output V, I and P of output
bus channel in raw form */
```

```
485     isimepsv2_ivid7_piu__vipdch_t vip_obc31; /*!< Output V, I and P of output
bus channel in raw form */
```

```
486 } fields;
```

```
487 } isimepsv2_ivid7_piu__gethousekeepingrawincdb__from_t;
```

```
488
```

```
489     /*!
```

```
490 * Union for storing the parameters received by gethousekeepingrunningavg.
```

```
491 */
```

```
492     typedef union __attribute__((__packed__)) _
isimepsv2_ivid7_piu__gethousekeepingrunningavg__from_t
```

```
493 {
```

```

494 unsigned char raw[116];
495 struct __attribute__((__packed__))
496 {
497     isimepsv2_ivid7_piu__replyheader_t reply_header; /*!< "See \""System Type
Identifier (STID)\"" (page 22)" */
498     uint8_t reserved; /*!< reserved */
499     int16_t volt_brdsup; /*!< Voltage of internal board supply in raw form */

500     uint16_t temp; /*!< Measured temperature provided by a sensor internal to the
MCU in raw form \note conversion: eng. value [in C] = 0.244444 * raw + -1168*/

501     isimepsv2_ivid7_piu__vipdeng_t dist_input; /*!< Input V, I and P data taken
at the input of the distribution part of the unit in raw form. Negative values indicate
output flow. */
502     isimepsv2_ivid7_piu__vipdeng_t batt_input; /*!< Input V, I and P data taken
at the input of the battery part of the unit in raw form. Negative values indicate output
flow. */
503     uint16_t stat_obc_on; /*!< Bitflag field indicating channel-on status for the
output bus channels. */
504     uint16_t stat_obc_ocf; /*!< Bitflag field indicating overcurrent latch-off fault
status for the output bus channels. */
505     isimepsv2_ivid7_piu__batterypackstatus_t bat_stat; /*!< Bitflag field
indicating BP board */
506     int16_t temp2; /*!< 2 and 4 cell battery pack: Battery pack temperature in
between the center battery cells. */
507     int16_t temp3; /*!< 2 cell battery pack: not used 4 cell battery pack: Battery
pack temperature on the front of the battery pack. */

508     int16_t volt_vd0; /*!< Voltage of voltage domain 0 */

509     int16_t volt_vd1; /*!< Voltage of voltage domain 0 */

510     int16_t volt_vd2; /*!< Voltage of voltage domain 0 */

511     isimepsv2_ivid7_piu__vipdeng_t vip_obc00; /*!< Output V, I and P of output
bus channel */
512     isimepsv2_ivid7_piu__vipdeng_t vip_obc01; /*!< Output V, I and P of output
bus channel */
513     isimepsv2_ivid7_piu__vipdeng_t vip_obc02; /*!< Output V, I and P of output
bus channel */
514     isimepsv2_ivid7_piu__vipdeng_t vip_obc03; /*!< Output V, I and P of output
bus channel */
515     isimepsv2_ivid7_piu__vipdeng_t vip_obc04; /*!< Output V, I and P of output
bus channel */
516     isimepsv2_ivid7_piu__vipdeng_t vip_obc05; /*!< Output V, I and P of output
bus channel */

```



```

517     isimepsv2_ivid7_piu__vipdeng_t vip_obc06; /*!< Output V, I and P of output
bus channel */
518     isimepsv2_ivid7_piu__vipdeng_t vip_obc07; /*!< Output V, I and P of output
bus channel */
519     isimepsv2_ivid7_piu__vipdeng_t vip_obc08; /*!< Output V, I and P of output
bus channel */
520     isimepsv2_ivid7_piu__ccsdeng_t cc1; /*!< Data on conditioning chain */

521     isimepsv2_ivid7_piu__ccsdeng_t cc2; /*!< Data on conditioning chain */

522     isimepsv2_ivid7_piu__ccsdeng_t cc3; /*!< Data on conditioning chain */

523 } fields;
524 } isimepsv2_ivid7_piu__gethousekeepingrunningavg__from_t;
525
526 /*
527 * Union for storing the parameters received by getovercurrentfaultstate.
528 */
529 typedef union __attribute__((__packed__)) _
isimepsv2_ivid7_piu__getovercurrentfaultstate__from_t

530 {
531     unsigned char raw[46];
532     struct __attribute__((__packed__))
533     {
534         isimepsv2_ivid7_piu__replyheader_t reply_header; /*!< */
535         uint8_t reserved; /*!< */
536         uint16_t stat_ob_on; /*!< */
537         uint16_t stat_ob_ext_on; /*!< */
538         uint16_t stat_ob_ocf; /*!< */
539         uint16_t stat_ob_ext_ocf; /*!< */
540         uint16_t ocf_cnt_obc00; /*!< */
541         uint16_t ocf_cnt_obc01; /*!< */
542         uint16_t ocf_cnt_obc02; /*!< */
543         uint16_t ocf_cnt_obc03; /*!< */
544         uint16_t ocf_cnt_obc04; /*!< */
545         uint16_t ocf_cnt_obc05; /*!< */
546         uint16_t ocf_cnt_obc06; /*!< */
547         uint16_t ocf_cnt_obc07; /*!< */
548         uint16_t ocf_cnt_obc08; /*!< */
549         uint16_t ocf_cnt_obc09; /*!< */
550         uint16_t ocf_cnt_obc10; /*!< */
551         uint16_t ocf_cnt_obc11; /*!< */
552         uint16_t ocf_cnt_obc12; /*!< */
553         uint16_t ocf_cnt_obc13; /*!< */
554         uint16_t ocf_cnt_obc14; /*!< */

```

```

555     uint16_t ocf_cnt_obc15; /*!< */
556 } fields;
557 } isimepsv2_ivid7_piu__getovercurrentfaultstate__from_t;
558
559 /*!
560 * Union for storing the parameters received by getsystemstatus.
561 */
562 typedef union __attribute__((__packed__)) _
isimepsv2_ivid7_piu__getsystemstatus__from_t

563 {
564     unsigned char raw[36];
565     struct __attribute__((__packed__))
566     {
567         isimepsv2_ivid7_piu__replyheader_t reply_header; /*!< */
568         uint8_t mode; /*!< Indicates the current mode of the system. */

569         uint8_t conf; /*!< Read/write configuration parameters have been changed by
the EPS master since the last parameters load/save operation */

570         uint8_t reset_cause; /*!< Cause of last reset. */
571         uint32_t uptime; /*!< */
572         uint16_t error; /*!< */
573         uint16_t rc_cnt_pwron; /*!< */
574         uint16_t rc_cnt_wdg; /*!< */
575         uint16_t rc_cnt_cmd; /*!< */
576         uint16_t rc_cnt_pweron_mcu; /*!< */
577         uint16_t rc_cnt_emlopo; /*!< */
578         uint16_t prevcmd_elapsed; /*!< */
579         uint32_t unix_time; /*!< */
580         uint8_t unix_year; /*!< \note conversion: eng. value = raw+ 2000*/

581         uint8_t unix_month; /*!< */
582         uint8_t unix_day; /*!< */
583         uint8_t unix_hour; /*!< */
584         uint8_t unix_minute; /*!< */
585         uint8_t unix_second; /*!< */
586     } fields;
587 } isimepsv2_ivid7_piu__getsystemstatus__from_t;
588
589 /*!
590 * Union for storing the parameters sent by outputbusgroupoff.
591 */
592 typedef union __attribute__((__packed__)) _
isimepsv2_ivid7_piu__outputbusgroupoff__to_t

```

```

593 {
594     unsigned char raw[4];
595     struct __attribute__((__packed__))
596     {
597         uint16_t obc_bf; /*!< Bitflag field indicating which output bus channels to turn-
off. */
598         uint16_t obc_ext_bf; /*!< Optional. Bitflag field indicating which extended
output bus channels to turn-on. These are channels 16 through 31. Similar to CH_BF in
operation. */
599     } fields;
600 } isimepsv2_ivid7_piu__outputbusgroupoff__to_t;
601
602     /*!
603  * Union for storing the parameters sent by outputbusgroupon.
604  */
605     typedef union __attribute__((__packed__)) _
isimepsv2_ivid7_piu__outputbusgroupon__to_t

606 {
607     unsigned char raw[4];
608     struct __attribute__((__packed__))
609     {
610         uint16_t obc_bf; /*!< Bitflag field indicating which output bus channels to turn-
on. */
611         uint16_t obc_ext_bf; /*!< Optional. Bitflag field indicating which extended
output bus channels to turn-on. These are channels 16 through 31. Similar to CH_BF in
operation. */
612     } fields;
613 } isimepsv2_ivid7_piu__outputbusgroupon__to_t;
614
615     /*!
616  * Union for storing the parameters sent by outputbusgroupstate.
617  */
618     typedef union __attribute__((__packed__)) _
isimepsv2_ivid7_piu__outputbusgroupstate__to_t

619 {
620     unsigned char raw[4];
621     struct __attribute__((__packed__))
622     {
623         uint16_t obc_bf; /*!< Bitflag field indicating the required output bus channel on-
state. */
624         uint16_t obc_ext_bf; /*!< Optional. Bitflag field indicating which extended
output bus channels to turn-on. These are channels 16 through 31. Similar to CH_BF in
operation. */
625     } fields;

```

```

626 } isimepsv2_ivid7_piu__outputbusgroupstate__to_t;
627
628     /*!
629  * Union for storing the parameters received by resetconfigurationparameter.
630  */
631     typedef union __attribute__((__packed__)) _
isimepsv2_ivid7_piu__resetconfigurationparameter__from_t

632 {
633     unsigned char raw[16];
634     struct __attribute__((__packed__))
635     {
636         isimepsv2_ivid7_piu__replyheader_t reply_header; /*!< */
637         uint8_t reserved; /*!< */
638         uint16_t par_id; /*!< parameter id of the parameter to get */

639         uint8_t par_val[8]; /*!< parameter value encompassing n bytes. The length
depends on the parameter type. */
640     } fields;
641 } isimepsv2_ivid7_piu__resetconfigurationparameter__from_t;
642
643     /*!
644  * Union for storing the parameters sent by setconfigurationparameter.
645  */
646     typedef union __attribute__((__packed__)) _
isimepsv2_ivid7_piu__setconfigurationparameter__to_t

647 {
648     unsigned char raw[10];
649     struct __attribute__((__packed__))
650     {
651         uint16_t par_id; /*!< parameter id of the parameter to get */

652         uint8_t par_val[8]; /*!< new parameter value encompassing n bytes. The
length depends on the parameter type. */
653     } fields;
654 } isimepsv2_ivid7_piu__setconfigurationparameter__to_t;
655
656     /*!
657  * Union for storing the parameters received by setconfigurationparameter.
658  */
659     typedef union __attribute__((__packed__)) _
isimepsv2_ivid7_piu__setconfigurationparameter__from_t

660 {
661     unsigned char raw[16];

```

```

662 struct __attribute__((__packed__))
663 {
664     isismepsv2_ivid7_piu__replyheader_t reply_header; /*!< */
665     uint8_t reserved; /*!< */
666     uint16_t par_id; /*!< parameter id of the parameter to get */

667     uint8_t par_val[8]; /*!< parameter value encompassing n bytes. The length
depends on the parameter type. */
668 } fields;
669 } isismepsv2_ivid7_piu__setconfigurationparameter__from_t;
670
671 #ifdef __cplusplus
672 }
673 #endif
674
675 #endif /* ISISMEPSV2_IVID7_PIU_TYPES_H_ */

676
677

```

```

1 /**
2  * @file IsisSolarPanelv2.h
3  * @brief ISIS Solar Panel temperature sensors, version 2 (using an LTC ADC driver)
4  */
5
6  #ifndef SRC_ISISSOLARPANELV2_H_
7  #define SRC_ISISSOLARPANELV2_H_
8
9  #include <at91/peripherals/pio/pio.h>
10 #include <hal/Drivers/SPI.h>
11 #include <freertos/FreeRTOS.h>
12
13 #include <stdint.h>
14
15 #ifdef __cplusplus
16 extern "C"
17 {
18     #endif
19
20     /** Macro used for converting temperature to floating point value */
21
22     #define ISIS_SOLAR_PANEL_CONV ( 1.0 / 1024.0 )
23
24     /**
25      * Return values for ISIS solar panel function calls.
26      */
27     typedef enum _IsisSolarPanelv2_Error_t
28     {
29         ISIS_SOLAR_PANEL_ERR_INPUT    = -6, /*!< One of the input parameters was
incorrect */
30         ISIS_SOLAR_PANEL_ERR_TIMEOUT = -5, /*!< There was a timeout waiting for
the LTC */
31         ISIS_SOLAR_PANEL_ERR_ADC      = -4, /*!< The LTC ADC reported an error */
32         ISIS_SOLAR_PANEL_ERR_PIN      = -3, /*!< Error configuring the GPIO pins */
33         ISIS_SOLAR_PANEL_ERR_SPI      = -2, /*!< Error communicating on the SPI bus
*/
34         ISIS_SOLAR_PANEL_ERR_STATE    = -1, /*!< The internal temperature sensor
is not in the appropriate state for this action */
35     } IsisSolarPanelv2_Error_t;
36
37     /**

```

```

38 * State of the internal temperature sensor.
39 */
40     typedef enum _IsisSolarPanelv2_State_t
41 {
42     ISIS_SOLAR_PANEL_STATE_NOINIT, /*!< Unitialized */
43     ISIS_SOLAR_PANEL_STATE_SLEEP, /*!< Sleeping */
44     ISIS_SOLAR_PANEL_STATE_AWAKE, /*!< Active */
45 }
46 IsisSolarPanelv2_State_t;
47
48     /**
49 * Channels available for temperature measurements.
50 * Note: Some channels might not be in use
51 */
52     typedef enum _IsisSolarPanel_panel_t
53 {
54     ISIS_SOLAR_PANEL_0    = 0,
55     ISIS_SOLAR_PANEL_1    = 1,
56     ISIS_SOLAR_PANEL_2    = 2,
57     ISIS_SOLAR_PANEL_3    = 3,
58     ISIS_SOLAR_PANEL_4    = 4,
59     ISIS_SOLAR_PANEL_5    = 5,
60     ISIS_SOLAR_PANEL_6    = 6,
61     ISIS_SOLAR_PANEL_7    = 7,
62     ISIS_SOLAR_PANEL_8    = 8,
63     ISIS_SOLAR_PANEL_COUNT = 9
64 }
65 IsisSolarPanelv2_Panel_t;
66
67     typedef struct {
68     portTickType delay_ms; /*!< Delay between polling of status of the LTC chip */
69
69     portTickType timeout_ms; /*!< Timeout to wait for an action on the LTC chip to
complete */
70 }
71 IsisSolarPanelv2_timeouts_s;
72
73     /**
74 * Initializes the internal hardware and software required for measuring the
75 * temperatures of the ISIS solar panels. This function assumes the SPI driver
76 * has already been initialized properly.
77 *
78 * @param[in] spi_slave Contains the SPI slave on SPI bus 1 to which LTC ADC
driver is connected
79 * @param[in] reset_pin GPIO pin used to reset the LTC chip, will be configured as
output

```

```

80 * @param[in] int_pin GPIO pin used for interrupts from the LTC chip, will be
configured as input
81 * @return A value defined by IsisSolarPanelv2_Error_t
82 */
83 IsisSolarPanelv2_Error_t IsisSolarPanelv2_initialize( SPISlave spi_slave, Pin*
reset_pin, Pin* int_pin );
84
85 /**
86 * Sets the ADC sampling delay and timeout.
87 * If the sampling is unsuccessful, *delay* is applied and a new sampling is
performed until
88 * either the sampling is successful or *timeout* is met. By default this driver uses 5
ms
89 * for the delay and 500 ms for the timeout.
90 *
91 * @param[in] p_timeout New timing parameters to be used by the driver
92 * @return A value defined by IsisSolarPanelv2_Error_t
93 */
94 IsisSolarPanelv2_Error_t IsisSolarPanelv2_set_interface_timeout( const
IsisSolarPanelv2_timeouts_s *p_timeout );
95
96 /**
97 * Puts the internal temperature sensor to sleep mode. Reducing its power-
98 * consumption.
99 *
100 * @note When in sleep mode, the temperature sensor will need to be woken by a
call to
101 * IsisSolarPanelv2_wakeup() before attempting to acquire any temperature
102 * readings. Waking the sensor could take up to 500ms to complete.
103 *
104 * @return A value defined by IsisSolarPanelv2_Error_t
105 */
106 IsisSolarPanelv2_Error_t IsisSolarPanelv2_sleep( void );
107
108 /**
109 * Wakes the internal temperature sensor from sleep mode. Allowing it to acquire
110 * temperature readings from the solar panels.
111 *
112 * @note Waking the sensor could take up to 500ms to complete.
113 *
114 * @return A value defined by IsisSolarPanelv2_Error_t
115 */
116 IsisSolarPanelv2_Error_t IsisSolarPanelv2_wakeup(void);
117
118 /**
119 * Acquires the temperature from the specified panel as well as a status byte

```



```

120 * used for fault reporting. The temperature is reported from -273.16 to
121 * 8192 degrees Celsius with a resolution of 1/1024.
122 *
123 * @param[in] panel The specified panel for which the temperature should be
124 * acquired
125 * @param[out] panelTemp The acquired temperature for the specified panel
126 * @param[out] status The status of the acquired temperature. Used for fault
127 * reporting.
128 * @return A value defined by IsisSolarPanelv2_Error_t
129 */
130 IsisSolarPanelv2_Error_t IsisSolarPanelv2_getTemperature
( IsisSolarPanelv2_Panel_t panel, int32_t* panelTemp, uint8_t *status );
131
132 /**
133 * Returns the current state of the internal temperature sensor.
134 *
135 * @return The state as defined by IsisSolarPanelv2_State_t
136 */
137 IsisSolarPanelv2_State_t IsisSolarPanelv2_getState( void );
138
139 #ifdef __cplusplus
140 }
141 #endif
142
143 #endif /* SRC_ISISSOLARPANELV2_H_ */
144

```

```

1 /*
2  * common_types.h
3  *
4  * Created on: 13 Dec 2019
5  *   Author: asau
6  */
7
8  #ifndef INCLUDE_SATELLITE_SUBSYSTEMS_COMMON_TYPES_H_
9
10 #define INCLUDE_SATELLITE_SUBSYSTEMS_COMMON_TYPES_H_
11
12 typedef enum _driver_error_t
13 {
14     driver_error_none = 0, /*!< No error occurred */
15     driver_error_i2c = 1, /*!< An error occurred while attempting to communicate over
16     I2C */
17     driver_error_spi = 2, /*!< An error occurred while attempting to communicate over
18     SPI */
19     driver_error_uart = 3, /*!< An error occurred while attempting to communicate
20     over UART */
21     driver_error_param = 4, /*!< One of the provided input parameters was invalid */
22     driver_error_malloc = 5, /*!< An error occurred while trying to allocate memory */
23     driver_error_reinit = 6, /*!< The driver has already been initialized */
24     driver_error_param_length = 7, /*!< One of the provided input parameters was too
25     big or too small */
26     driver_error_reply_crc = 8, /*!< The CRC in the subsystem's reply did not match
27     the data */
28     driver_error_mutex = 9, /*!< Acquisition of mutex failed */
29     driver_error_remote_processing = 10, /*!< Subsystem processing of the
30     command failed */
31     driver_error_uninit = 11, /*!< Driver has not been initialized */
32     driver_error_busy = 12, /*!< Driver is unable to process action at this moment */
33     driver_error_other = 255, /*!< An unknown error occurred */
34 }
35 driver_error_t;
36
37 #endif /* INCLUDE_SATELLITE_SUBSYSTEMS_COMMON_TYPES_H_ */

```



```

1 /*
2  * isismepsv2_ivid7_piu.h
3  *
4  * AUTOGENERATED CODE
5  * Please do not perform manual edits
6  * Generated using autogen v1.0.3
7  *
8  * Generated from:
9  * - imepsv2_structs.yaml
10 * - imepsv2_ivid7_piu.yaml
11 */
12
13 #ifndef ISISMEPSV2_IVID7_PIU_H_
14 #define ISISMEPSV2_IVID7_PIU_H_
15
16 #include "isismepsv2_ivid7_piu_types.h"
17 #include <satellite-subsystems/common_types.h>
18
19 #ifdef __cplusplus
20 extern "C"
21 {
22 #endif
23
24 /*!
25  * Initialize driver for a number of ISISMEPSV2_IVID7_PIU instances. The first
instance can then be referenced by using index 0, the second by using index 1, etc.
26  *
27  * @param[in] isismepsv2_ivid7_piu Pointer to array of ISISMEPSV2_IVID7_PIU
instances
28  * @param[in] isismepsv2_ivid7_piuCount Number of ISISMEPSV2_IVID7_PIU
instances pointed to by isismepsv2_ivid7_piu input parameter
29  * @return Error code as specified in common_types.h
30 */
31 driver_error_t ISISMEPSV2_IVID7_PIU_Init(const ISISMEPSV2_IVID7_PIU_t*
isismepsv2_ivid7_piu, uint8_t isismepsv2_ivid7_piuCount);
32
33 /*!
34  * Switches off any command enable output bus channels that have been switched
on after the system powered up. Only output bus channels that can be commanded
off are affected. All force enable channels will remain enabled.
35  *
36  * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
37  * @param[out] reply_header_out
38  * @return Error code as specified in common_types.h

```

```

39 */
40     driver_error_t isimepsv2_ivid7_piu__cancel(uint8_t index,
isimepsv2_ivid7_piu__replyheader_t *reply_header_out);
41
42     /*!
43 * Correct the unit's unix time with the specified amount of seconds
44 *
45 * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
46 * @param[in] correction_in Amount of seconds to correct the unix time with.
47 * @param[out] reply_header_out
48 * @return Error code as specified in common_types.h
49 */
50     driver_error_t isimepsv2_ivid7_piu__correcttime(uint8_t index, int32_t
correction_in, isimepsv2_ivid7_piu__replyheader_t *reply_header_out);
51
52     /*!
53 * Get the value of a configuration parameter.
54 *
55 * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
56 * @param[in] par_id_in parameter id of the parameter to get
57 * @param[out] response Struct with response from subsystem
58 * @return Error code as specified in common_types.h
59 */
60     driver_error_t isimepsv2_ivid7_piu__getconfigurationparameter(uint8_t index,
uint16_t par_id_in, isimepsv2_ivid7_piu__getconfigurationparameter__from_t
*response);
61
62     /*!
63 * Prepare the response buffer with housekeeping data. The housekeeping data is
returned in engineering values.
64 *
65 * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
66 * @param[out] response Struct with response from subsystem
67 * @return Error code as specified in common_types.h
68 */
69     driver_error_t isimepsv2_ivid7_piu__gethousekeepingeng(uint8_t index,
isimepsv2_ivid7_piu__gethousekeepingeng__from_t *response);
70
71     /*!
72 * Prepare the response buffer with housekeeping data. The housekeeping data is
returned in raw form, as received from the hardware, unaltered by the main controller.
73 *
74 * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver

```

initialization

```
75 * @param[out] response Struct with response from subsystem
76 * @return Error code as specified in common_types.h
77 */
78     driver_error_t isismepsv2_ivid7_piu__gethousekeepingengincdb(uint8_t index,
isismepsv2_ivid7_piu__gethousekeepingengincdb__from_t *response);
79
80     /*!
81 * Prepare the response buffer with running average housekeeping data. The
housekeeping data is returned in engineering values.
82 *
83 * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
84 * @param[out] response Struct with response from subsystem
85 * @return Error code as specified in common_types.h
86 */
87     driver_error_t isismepsv2_ivid7_piu__gethousekeepingengrunningavgincdb(
uint8_t index, isismepsv2_ivid7_piu__gethousekeepingengrunningavgincdb__from_t
*response);
88
89     /*!
90 * Prepare the response buffer with housekeeping data. The housekeeping data is
returned in raw form, as received from the hardware, unaltered by the main controller.
91 *
92 * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
93 * @param[out] response Struct with response from subsystem
94 * @return Error code as specified in common_types.h
95 */
96     driver_error_t isismepsv2_ivid7_piu__gethousekeepingraw(uint8_t index,
isismepsv2_ivid7_piu__gethousekeepingraw__from_t *response);
97
98     /*!
99 * Prepare the response buffer with housekeeping data. The housekeeping data is
returned in raw form, as received from the hardware, unaltered by the main controller.
100 *
101 * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
102 * @param[out] response Struct with response from subsystem
103 * @return Error code as specified in common_types.h
104 */
105     driver_error_t isismepsv2_ivid7_piu__gethousekeepingrawincdb(uint8_t index,
isismepsv2_ivid7_piu__gethousekeepingrawincdb__from_t *response);
106
107     /*!
108 * Prepare the response buffer with running average housekeeping data. The
```

housekeeping data is returned in engineering values.

```
109 *
110 * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
111 * @param[out] response Struct with response from subsystem
112 * @return Error code as specified in common_types.h
113 */
114     driver_error_t isismepsv2_ivid7_piu__gethousekeepingrunningavg(uint8_t
index, isismepsv2_ivid7_piu__gethousekeepingrunningavg__from_t *response);
115
116     /*!
117 * Prepare the response buffer with output bus over current events. Over current
fault counters are incremented each time a bus is latched off due to an overcurrent
event.
118 *
119 * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
120 * @param[out] response Struct with response from subsystem
121 * @return Error code as specified in common_types.h
122 */
123     driver_error_t isismepsv2_ivid7_piu__getovercurrentfaultstate(uint8_t index,
isismepsv2_ivid7_piu__getovercurrentfaultstate__from_t *response);
124
125     /*!
126 * Return system status information
127 *
128 * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
129 * @param[out] response Struct with response from subsystem
130 * @return Error code as specified in common_types.h
131 */
132     driver_error_t isismepsv2_ivid7_piu__getsystemstatus(uint8_t index,
isismepsv2_ivid7_piu__getsystemstatus__from_t *response);
133
134     /*!
135 * Load all configuration parameters from non-volatile memory, discarding any
changes made in volatile memory. This is performed automatically at system startup if a
valid load configuration is encountered in non-volatile memory. If no (valid) configuration
is found, the system will initialize using hard coded defaults.
136 *
137 * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
138 * @param[in] conf_key_in Configuration key: 0xA8. Any other value causes the
reset command to be rejected with a parameter error. The reset will not be performed in
that case.
139 * @param[out] reply_header_out
```

```

140 * @return Error code as specified in common_types.h
141 */
142     driver_error_t isimepsv2_ivid7_piu__loadconfiguration(uint8_t index, uint8_t
conf_key_in, isimepsv2_ivid7_piu__replyheader_t *reply_header_out);
143
144     /*!
145 * Performs a no-operation. This is useful to check the availability of the system,
without changing anything about the current configuration or operation.
146 *
147 * @param[in] index Index of ISIMEPSV2_IVID7_PIU in list provided during driver
initialization
148 * @param[out] reply_header_out
149 * @return Error code as specified in common_types.h
150 */
151     driver_error_t isimepsv2_ivid7_piu__nop(uint8_t index,
isimepsv2_ivid7_piu__replyheader_t *reply_header_out);
152
153     /*!
154 * Turn a single output bus channel off using the bus channel index. Index 0
represents channel 0 (OBC0), index 1 represents channel 1 (OBC1), etc.
155 *
156 * @param[in] index Index of ISIMEPSV2_IVID7_PIU in list provided during driver
initialization
157 * @param[in] obc_idx_in Bus channel index of the channel to turn off. Trying to
switch a permanent bus channels will be ignored. An index corresponding to a
nonexistent bus channel will be rejected.
158 * @param[out] reply_header_out
159 * @return Error code as specified in common_types.h
160 */
161     driver_error_t isimepsv2_ivid7_piu__outputbuschanneloff(uint8_t index,
isimepsv2_ivid7_piu__imeps_channel_t obc_idx_in,
isimepsv2_ivid7_piu__replyheader_t *reply_header_out);
162
163     /*!
164 * Turn a single output bus channel on using the bus channel index. Index 0
represents channel 0 (OBC0), index 1 represents channel 1 (OBC1), etc.
165 *
166 * @param[in] index Index of ISIMEPSV2_IVID7_PIU in list provided during driver
initialization
167 * @param[in] obc_idx_in Bus channel index of the channel to turn on Trying to
switch a permanent bus channels will be ignored. An index corresponding to a
nonexistent bus channel will be rejected.
168 * @param[out] reply_header_out
169 * @return Error code as specified in common_types.h
170 */
171     driver_error_t isimepsv2_ivid7_piu__outputbuschannelon(uint8_t index,

```



```

isimepsv2_ivid7_piu__imeps_channel_t obc_idx_in,
isimepsv2_ivid7_piu__replyheader_t *reply_header_out);
172
173     /*!
174  * Turn-off output bus channels that are marked with a 1-bit, leave bus channels
that are not marked unaltered. The least-significant bit corresponds to bus channel 0
(OBC0), the next bit corresponds to channel 1 (OBC1), etc.
175  *
176  * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
177  * @param[in] params Struct with parameters for subsystem
178  * @param[out] reply_header_out
179  * @return Error code as specified in common_types.h
180  */
181     driver_error_t isimepsv2_ivid7_piu__outputbusgroupoff(uint8_t index, const
isimepsv2_ivid7_piu__outputbusgroupoff__to_t *params,
isimepsv2_ivid7_piu__replyheader_t *reply_header_out);
182
183     /*!
184  * Turn-on output bus channels that are marked with a 1-bit, leave bus channels
that are not marked unaltered. The least-significant bit corresponds to bus channel 0
(OBC0), the next bit corresponds to channel 1 (OBC1), etc.
185  *
186  * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
187  * @param[in] params Struct with parameters for subsystem
188  * @param[out] reply_header_out
189  * @return Error code as specified in common_types.h
190  */
191     driver_error_t isimepsv2_ivid7_piu__outputbusgroupon(uint8_t index, const
isimepsv2_ivid7_piu__outputbusgroupon__to_t *params,
isimepsv2_ivid7_piu__replyheader_t *reply_header_out);
192
193     /*!
194  * Turn-on bus channels that are marked with a 1-bit, turn-off bus channels that are
not marked (i.e. 0-bit). The least-significant bit corresponds to bus channel 0 (OBC00),
the next bit corresponds to channel 1 (OBC01), etc.
195  *
196  * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
197  * @param[in] params Struct with parameters for subsystem
198  * @param[out] reply_header_out
199  * @return Error code as specified in common_types.h
200  */
201     driver_error_t isimepsv2_ivid7_piu__outputbusgroupstate(uint8_t index, const
isimepsv2_ivid7_piu__outputbusgroupstate__to_t *params,

```

```

isismepsv2_ivid7_piu__replyheader_t *reply_header_out);
202
203     /*!
204  * Perform a software induced reset of the MCU.
205  *
206  * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
207  * @param[out] reply_header_out
208  * @return Error code as specified in common_types.h
209  */
210     driver_error_t isismepsv2_ivid7_piu__reset(uint8_t index,
isismepsv2_ivid7_piu__replyheader_t *reply_header_out);
211
212     /*!
213  * Reset all configuration parameters to hard-coded defaults, discarding any
changes made, in volatile memory (only!). This is performed automatically at system
startup before an attempt to load a configuration is performed. If no (valid) configuration
is found that can be loaded, the system will use hard coded defaults.
214  *
215  * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
216  * @param[in] conf_key_in Configuration key: 0xA8. Any other value causes the
reset command to be rejected with a parameter error. The reset will not be performed in
that case.
217  * @param[out] reply_header_out
218  * @return Error code as specified in common_types.h
219  */
220     driver_error_t isismepsv2_ivid7_piu__resetconfiguration(uint8_t index, uint8_t
conf_key_in, isismepsv2_ivid7_piu__replyheader_t *reply_header_out);
221
222     /*!
223  * Reset a parameter to its default hard-coded value. All parameters have this value
at system power-up or after the software reset command.
224  *
225  * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
226  * @param[in] par_id_in parameter id of the parameter to get
227  * @param[out] response Struct with response from subsystem
228  * @return Error code as specified in common_types.h
229  */
230     driver_error_t isismepsv2_ivid7_piu__resetconfigurationparameter(uint8_t
index, uint16_t par_id_in, isismepsv2_ivid7_piu__resetconfigurationparameter__from_t
*response);
231
232     /*!
233  * Resets the watchdog timer keeping the system from performing a reset.

```

```

234 *
235 * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
236 * @param[out] reply_header_out
237 * @return Error code as specified in common_types.h
238 */
239     driver_error_t isismepsv2_ivid7_piu__resetwatchdog(uint8_t index,
isismepsv2_ivid7_piu__replyheader_t *reply_header_out);
240
241     /*!
242 * Commit all read/write configuration parameters kept in volatile memory to non-
volatile memory.
243 *
244 * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
245 * @param[in] checksum_in To force save this value can be set to 0. The save will
then proceed without performing CRC verification.
246 * @param[out] reply_header_out
247 * @return Error code as specified in common_types.h
248 */
249     driver_error_t isismepsv2_ivid7_piu__saveconfiguration(uint8_t index, uint16_t
checksum_in, isismepsv2_ivid7_piu__replyheader_t *reply_header_out);
250
251     /*!
252 * Change a configuration parameter. The change will take effect immediately and
any function using the parameter will use the new value
253 *
254 * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
255 * @param[in] params Struct with parameters for subsystem
256 * @param[out] response Struct with response from subsystem
257 * @return Error code as specified in common_types.h
258 */
259     driver_error_t isismepsv2_ivid7_piu__setconfigurationparameter(uint8_t index,
const isismepsv2_ivid7_piu__setconfigurationparameter__to_t *params,
isismepsv2_ivid7_piu__setconfigurationparameter__from_t *response);
260
261     /*!
262 *
263 *
264 * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
265 * @param[out] reply_header_out
266 * @return Error code as specified in common_types.h
267 */
268     driver_error_t isismepsv2_ivid7_piu__switchtonominal(uint8_t index,

```

```

isismepsv2_ivid7_piu__replyheader_t *reply_header_out);
269
270     /*!
271  *
272  *
273  * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
274  * @param[out] reply_header_out
275  * @return Error code as specified in common_types.h
276  */
277     driver_error_t isismepsv2_ivid7_piu__switchtosafety(uint8_t index,
isismepsv2_ivid7_piu__replyheader_t *reply_header_out);
278
279     /*!
280  * Write all reset cause counters to zero in persistent memory.
281  *
282  * @param[in] index Index of ISISMEPSV2_IVID7_PIU in list provided during driver
initialization
283  * @param[out] reply_header_out
284  * @return Error code as specified in common_types.h
285  */
286     driver_error_t isismepsv2_ivid7_piu__zeroresetcausecounters(uint8_t index,
isismepsv2_ivid7_piu__replyheader_t *reply_header_out);
287
288     #ifdef __cplusplus
289 }
290     #endif
291
292     #endif /* ISISMEPSV2_IVID7_PIU_H_ */
293
294

```

נספחים

Satellite

datasheet OBC

ISIS-OBC Datasheet

ISIS.ISIS-OBC.DS.1.1



Issue 1.3

Release information

Written by:	H. Péter-Contesse, A. Piplani, E. Timmer	2018-12-21
Checked by:	L. Rotthier, A. Palun	2018-12-21
Approved by:	M. den Ouden	2018-12-21
Distribution List:	ISIS internal, ISIS-OBC customers	

Change log



ISIS-OBC Datasheet

Doc ID: ISIS.ISIS-OBC.DS.1.1

Issue: 1.3

Date: 2018-12-21

Page: ii of vii

Date	Issue	Modified by	Section / Pages Affected	Reason for Change
2014-04-17	0.1_Draft0	H. Péter-Contesse	All	First Release
2014-04-29	0.1_Draft1	H. Péter-Contesse	All	Completed, Updated after comments from LROT, JHEN and JROT.
2014-05-01	1.0	H. Péter-Contesse	Table 6-1	Updated table, document approved by JROT.
2014-05-28	1.1	H. Péter-Contesse	Table 1-1 and Table 6-3	Updated power consumption values.
2016-02-10	1.2	R. Fernandez	Table 1-1	Mass figure updated (APAL input).
2018-21-12	1.3	E.Timmer	§4.2, §5.3	Updated FM Daughterboard RevB

Applicable Documents

AD01			
AD02			
AD03			
AD04			

Reference Documents

RD01	ISIS-OBC QuickStart Guide v2.1	iOBC QuickStart Guide	V2.1
RD02	UM10204	NXP, I2C-bus specification and user manual, http://www.nxp.com/documents/user_manual/UM10204.pdf	Rev. 6, April 2014
RD03			
RD04			
RD05			

TBD/TBC/TBW

TBD/TBC/TBW	Responsible	Action	Page



Disclaimer

The contents of this document are subject to the relevant provisions in the contract concluded between the parties. ISIS – Innovative Solutions In Space B.V. ("ISIS") shall not be liable, in full or in part, for any damage arising out from the application or use of any product or circuit described herein, in case such application or use are carried out in a manner not in line with the instructions and warranties provided in the User Manual, Safety Manual, product information sheets or any other document provided by ISIS upon the delivery of the product ("Documents"). Further, ISIS shall not be liable for any damage caused by any use which exceeds the function(s) of the product, or does not conform to such function(s) as described in the Documents. ISIS shall not be liable for any damage arising from a use which is not carried out in a manner conforming to acceptable practices in the aerospace industry.

ISIS warrants that the product is supplied after relevant tests had shown the product is in good order and functioning, as far as these tests may indicate and predict product functionality.



List of Acronyms

1/2/3U	1-Unit, 2-Unit, 3-Unit; commonly referring to the singles and multiples of the commercially available CubeSat sizes
AIV	Assembly, integration, verification
COTS	Commercial off the shelf
COM	Centre of Mass
EMC	Electro-Magnetic Compatibility
EMI	Electro-Magnetic Interference
ESD	Electrostatic discharge
I ² C	Inter integrated circuit communication bus.
iOBC	ISIS on-board computer
ISIS	Innovative Solutions In Space BV.
RH	Relative Humidity
N/A	Not Applicable
PWM	Pulse Width Modulation
TBC	To Be Confirmed
TBD	To Be Determined
TBW	To Be Written
UM	User Manual



Table of Contents

1	Overview	1
2	Functional Description	2
3	Development Environment.....	5
4	Daughterboard option.....	8
5	Connectors location and pin attribution	12
6	Electrical specifications	21
7	Mechanical Characteristics	26
8	Storage and Handling Information.....	29

Table of Figures

Figure 2-1: Block diagram.	2
Figure 3-1: Flight software layers diagram.	6
Figure 4-1: EM daughterboard assembly (top view).	9
Figure 4-2: EM daughterboard assembly (X+ side view).	9
Figure 4-3: EM daughterboard assembly (Y- side view).	9
Figure 4-4: EM daughterboard assembly (X- side view).	9
Figure 4-5: FM daughterboard assembly (with iOBC, top view).	10
Figure 4-6: FM daughterboard (bottom view).	10
Figure 4-7: FM daughterboard assembly (X+ side view).	10
Figure 4-8: FM daughterboard assembly (Y- side view).	10
Figure 4-9: FM daughterboard assembly (X- side view).	10
Figure 5-1: Connectors location (top).	13
Figure 5-2: Connectors location (bottom).	13
Figure 5-3: Stress relief soldered connection.	14
Figure 5-4: EM daughterboard connector location (top).	16
Figure 5-5: FM daughterboard connector location (bottom, through view from top).	19
Figure 7-1: Motherboard outline (top view).	26
Figure 7-2: Drills, mounting points, J2 centre and H1 pin1 location (top view).	27
Figure 7-3: Drill sizes.	27
Figure 7-4: Recommended daughterboard outline [mm] (top view).	28
Figure 7-5: Height of components on motherboard [mm] (top view).	28

Table of Tables

Table 1-1: Overall specifications.	1
Table 4-1: Daughterboard connector options (mating with J2).	11
Table 5-1: CSKB pin attribution.	14
Table 5-2: Breakout wires.	14



Table 5-3: J1 pin attribution.....	14
Table 5-4: J2 pin attribution.....	15
Table 5-5: J3 pin attribution.....	16
Table 5-6: J5 pin attribution.....	17
Table 5-7: J6 pin attribution.....	17
Table 5-8: J7 pin attribution.....	18
Table 5-9: J8 pin attribution.....	18
Table 5-10: J9 pin attribution.....	18
Table 5-11: J5 pin attribution.....	19
Table 5-12: J6 pin attribution.....	20
Table 5-13: J7 pin attribution.....	20
Table 5-14: J8 pin attribution.....	20
Table 5-15: J9 pin attribution.....	20
Table 6-1: Detailed electrical specifications for each net.	21
Table 6-2: Power on/off requirements.	25
Table 6-3: Power consumption.....	25

1 Overview

The iOBC is a CubeSat standard compatible On-Board Computer designed specifically for use in Nano-Satellites. It provides a large variety of interfacing options as well as processing capability while still being very power efficient. At the same time, the ability to plug in a daughterboard allows for a great degree of flexibility. Table 1-1 show the general characteristics of the OBC.

Table 1-1: Overall specifications.

Processor	400MHz, 32-bit ARM9 (AT91SAM9G20)
RAM	32MB SDRAM
Non-volatile data storage	2x2GB SD-Cards with FAT32 file system 256kB FRAM (high endurance and fast read/writes)
Code storage	1MB NOR-Flash
Timing	2 redundant real-time Clocks
Watchdog	External on-board watchdog and power supervisor
On-board sensing	Temperature, current and voltage measurements with over-current protection
Interfaces	1x I ² C (master or slave, Fast-mode, ≤400kbit/s) 1x SPI: Up to 8 slaves (≤10Mbit/s) 2x UARTs (≤10Mbit/s, depending configuration): - 1x LVCMOS or RS232 levels (hardware configuration) - 1x RS232 or RS422/485 levels (software configuration) 1x ADC: 8 input channels, 8 or 10-bit modes PWM: 6 output channels GPIO: 27 pins USB: 1x Host and 1x Device (≤12Mbit/s) 1x Image Sensor Interface for directly interfacing with CMOS image sensors (shared with GPIOs)
Programming and debug capabilities	JTAG for programming and debugging, Additional debug UART for console user-interface, 4xLEDs
Average power consumption	380mW, typical usage @ 3.3V supply
Qualified operating temperature range	-25°C to +65°C
Storage temperature range	-40°C to +80°C (RH < 60%)
Dimensions	96 x 90 x 12.4mm (including FM daughterboard)
Mass	106g (including daughterboard)

2 Functional Description

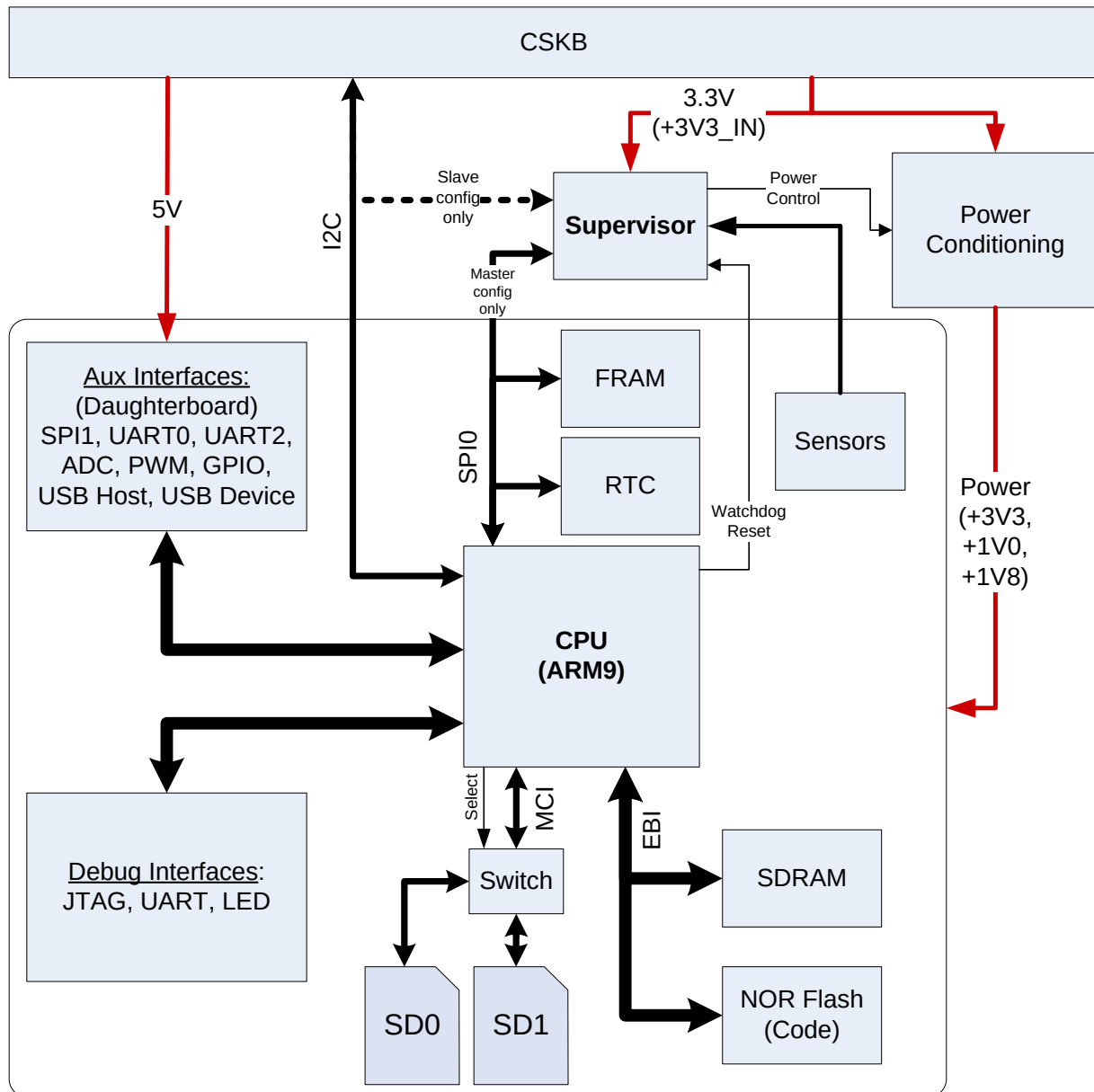


Figure 2-1: Block diagram.

CPU: The iOBC uses a 32-bit ARM9 microprocessor from Atmel (AT91SAM9G20) as its main CPU. Although quite powerful at 400MHz core speed, the processor is quite low-power as it uses 1.0V as its core voltage. The CPU has a number of interfaces to the outside world that are described later.

Supervisor: The supervisor is a PIC microcontroller that controls the power distribution across the board. It can individually switch on/off the CPU along with the other components of the board but separately controls the power of the RTC in order to keep the time on the satellite while the rest of the OBC is power-cycled. The power conditioning circuit includes overcurrent protection.

It also serves the crucial role of an external watchdog for the CPU. This is a window



(frequency) based watchdog: If the CPU toggles the Watchdog-Reset line too frequently or not frequently enough, both cases will lead to the Supervisor power-cycling the OBC.

The Supervisor also has several sensors for Temperature, Voltage and Current measurement. These measurements can be read out by the CPU using the SPI bus. When the iOBC is in a slave configuration, the master OBC can also read this data over the I2C satellite bus.

RAM: The CPU executes its code from this 32MB, low-power SDRAM. Volatile data is also stored here.

Code Storage: The 1MB parallel NOR Flash is used for storing code. When the OBC boots up, the code is copied to the RAM for faster execution.

Critical Data Storage: The OBC has a 256kB FRAM connected to the CPU over an SPI bus. FRAM is a Non-Volatile storage medium that is more robust than Flash or EEPROM memories. It can sustain Trillions of writes (10^{14}) and retain data for >151 years at 65°C (~20 years for typical Flash). It has fast accesses (no delay writes) and it does not require page-erase cycles like a Flash. At the same time, the memory cells are not susceptible to Single Event Upsets (SEU) due to radiations, compared to SRAM cells (note that the memory controller is still susceptible to SEU). Therefore, it should be used for storing critical and changing data such as flight parameters, flight plans etc.

Mass Data Storage: The OBC can use 2x 2GB SD-Cards for mass non-volatile storage of data. ISIS uses high quality industrial SD-Cards with Single Level Cell (SLC) Flash memory to improve reliability of the SD-Cards. Nevertheless, there are 2 cards for redundancy. While one card is in use, the other card is kept off to keep it safe from SEUs and for power-efficiency. Both cards are kept off until they are needed. The OBC uses the FAT32 file system for data storage. FAT32 is chosen for its widespread use and simplicity. However, it is not a journaling file system, making it susceptible to data corruption on power-loss. Therefore, FRAM is provided for storing critical data or data that is updated very often.

Programming and Debug Capabilities: The OBC contains a JTAG interface for programming and debugging of code. 4 LEDs present on the board can be controlled by the CPU for quick debug indication. In addition, a dedicated UART interface is present for user-interface console.

Timing: The OBC contains two Real Time Clocks. The external RTC is an advanced Real Time Clock that is accurate as it compensates for clock-drift due to temperature variations (± 3.5 ppm over temperature range and an additional ± 5 ppm of aging over 10 years). In addition, the CPU contains its own Real Time Timer as well. The timing driver from ISIS uses both these clocks for redundancy.

Interfaces: The iOBC has a very large variety of interfacing options as it was designed to be useable with a variety of payloads and subsystems.

I²C: The I²C bus is the de-facto standard for inter-subsystem communication in CubeSats. The iOBC can be used in Master or Slave configuration and drivers are provided for both. The maximum transfer speed is 400kbit/s (fast-mode), depending on the line capacitance and pull-up resistors. See RD02 for more details.

SPI: Although the CPU has 2 SPI buses, SPI0 is used solely for communication

within the board and SPI1 is used solely for external communication. This is done to keep traffic on one bus from affecting the other, and prevents electrical disturbance from outside factors affecting the board. Both Master and Slave configurations are supported on SPI1. Up to 8 chip select lines are available.

UARTs: The OBC has 2 UART interfaces with a variety of signalling options available. See Table 6-1 for the speed limitations for all configurations.

UART0 can be used as 3.3V TTL signalling or RS232 signalling depending on the configuration requested in the Option Sheet.

UART2 uses a dynamically reconfigurable chip (via software) to support either RS232 (with RTS/CTS flow control) or full-duplex RS422/485 differential bus.

Please note that the UART1 bus of the CPU is not available as those pins are already in use by other functions.

USB: The OBC simultaneously supports both USB host and device as two separate buses at 12Mbits/s. A demo driver for the device mode of USB is provided with the OBC. USB host drivers are not provided with the OBC but can be purchased separately.

ADC: The OBC has an 8-channel, 10-bit ADC which can sample at the maximum rates of 250ksamples/s (8-bit mode) or 75ksamples/s (10-bit mode). The ADC driver supports continuous periodic sampling and uses DMA for efficient operation. The input voltage range is 0V to 2.5V due to the use of an accurate voltage reference (0.1%, $\leq 50\text{ppm}/^{\circ}\text{C}$).

PWM: The OBC supports 6 PWM output channels whose duty-cycles can be controlled individually.

GPIO: The OBC supports 27 GPIO pins. Some of the GPIO pins are also exposed on the CSKB connector for harness free interfacing to other subsystems on the satellite bus. In addition, some of the GPIO pins can also be used alternatively as an Image Sensor Interface which can connect directly to CMOS camera sensors.

Drivers are provided by ISIS for all the interfaces available on the OBC. The drivers for bus interfaces (I2C, SPI, UART) use DMA for efficient transfer and employ FreeRTOS queues for flexibility and maintaining atomicity of transactions while servicing transfer requests from any number of tasks operating in parallel.

Note that freeRTOS is configured to not perform pre-emptive task switching to remove requirements for data contention management. Task switching by freeRTOS is performed once a task yields control through calling the `vTaskDelay` function.

3 Development Environment

The iOBC comes bundled with a Software Development Kit. The SDK can be set up using a convenient installation process.

The SDK consists of the following parts:

1. Eclipse Integrated Development Environment which can be used for developing, compiling and debugging code.
2. ARM GCC compiler which is used directly from Eclipse.
3. FreeRTOS for simple multi-tasking of software on the OBC.
4. Atmel SAM-BA for flashing code to the OBC.
5. PuTTY console for interfacing to the OBC.
6. Libraries from Atmel for basic interfacing to the CPU.
7. FAT32 file system for SD-Cards.
8. ISIS Hardware Abstraction Layer along with code examples to help users getting started quickly.

The Hardware Abstraction Layer (HAL) consists of the following drivers provided by ISIS in addition to the libraries from Atmel and FAT32 file system: I²C, SPI, UART, ADC, PWM, GPIO, LED, FRAM, Timing, Watchdog and Reset, Supervisor interface.

Please see the ISIS OBC QuickStart Guide RD01 for instructions on how to set up your development environment.

3.1 Additional Software

The Hardware Abstraction Layer of the OBC allows for easy, efficient and robust interfacing. However, software for an entire mission involves some additional effort.

Figure 3-1 depicts the several layers of design needed for the OBC Flight Software. ISIS has developed additional libraries that can help in completing this task. These libraries are called the Satellite Subsystem Interface and Mission Support Package.

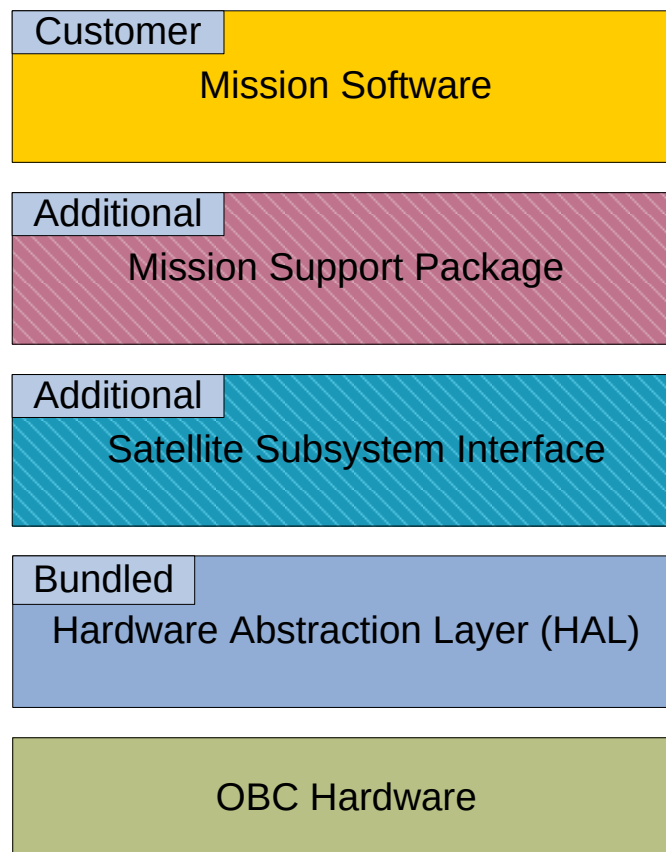


Figure 3-1: Flight software layers diagram.

Satellite Subsystem Interface

This library implements interfaces to a variety of subsystems that can be used on a nanosatellite bus. These interfaces include: ISIS Antenna System, ISIS TRXUV Comm System, ISIS Solar Panels and GomSpace Electrical Power Supply.

ISIS will continue to grow its portfolio of interfaces supported in this library. If you purchase any ISIS subsystem not listed above along with this library, ISIS can support inclusion of that subsystem into this library. Support for specific 3rd party subsystems can be added upon request by customers. Contact support@isispace.nl for additional information.

Mission Support Package

This library provides additional building blocks needed for robust and efficient OBC flight software. The following libraries are currently available as a part of this package:

Parameters Storage: Flight parameters are the key variables used to control the behaviour of the satellite. The parameters storage system keeps these variables in FRAM (which is inherently very robust). In addition, it uses data protection and duplication schemes to provide additional reliability.

Robust Hierarchical Logging: Much of the data acquired by the satellite is stored in



non-volatile memory for later retrieval when the satellite passes over the ground station. Housekeeping data, error logs are some common examples. For simple missions this can also be used to store periodical payload data. This logging mechanism will store the latest data in reliable FRAM and older data in SD-Cards that have more capacity.

ISIS will continue to add several more capabilities into the Mission Support Package. Contact support@isispace.nl for the latest information.

4 Daughterboard option

The ISIS OBC uses a unique combination of a Motherboard and Daughterboard in its design. This approach has several advantages:

1. The daughterboard design is quite flexible and can be built to match different applications while keeping the motherboard the same. This allows customers to add additional interfacing capabilities as required for their application.
2. The daughterboard provides additional fan-out area for connectors and harness for the large variety interfaces supported by the OBC.

When the iOBC is fitted with a daughterboard, it does not use more volume than a standard CSKB board. Therefore, the flexibility of design is achieved without occupying any additional volume on the satellite.

Different types of daughterboards are available from ISIS. Their specifications are described in section 4.1, 4.2, 5.2 and 5.3.

In addition customers can design their own daughterboards or request ISIS for a custom design. Note that specific requirements on the design of the daughterboard have to be followed. This is described in section 4.3.

The daughterboard is mounted with 4x M2 screws to the motherboard. Dedicated spacers are provided as well.

CAUTION: The daughterboard must be mated and de-mated with care in order not to damage any board or connectors. Keep both boards parallel during the process. When fastening the mounting screws, a special attention should be taken in order not to generate any metallic particles that could short traces or exposed pads on the motherboard. Note that it is particularly difficult to see and remove any particle that would go below the BGA package of the CPU or the memory.

4.1 EM Daughterboard

The EM daughterboard is designed for easy use within the laboratory environment. It exposes all the interfaces above the board as standard 2.54mm pin headers which can be probed easily. In addition, standard A and B USB connectors are placed for the host and device USB interface.

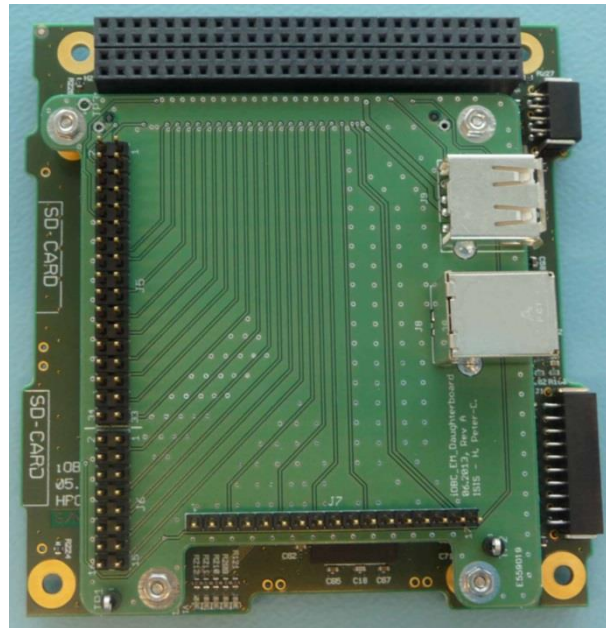


Figure 4-1: EM daughterboard assembly (top view).

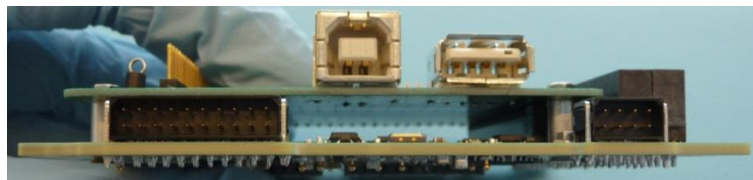


Figure 4-2: EM daughterboard assembly (X+ side view).

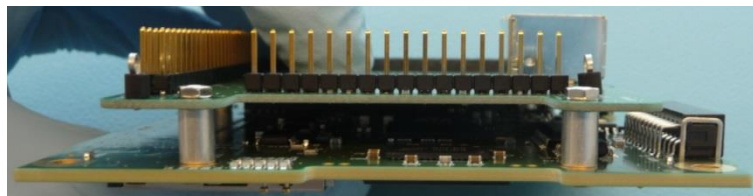


Figure 4-3: EM daughterboard assembly (Y- side view).

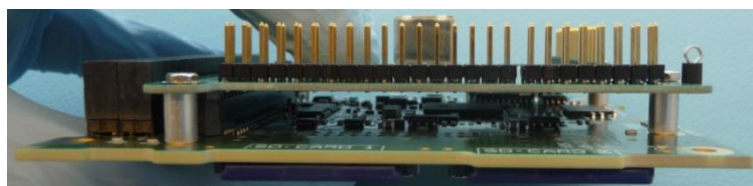


Figure 4-4: EM daughterboard assembly (X- side view).

4.2 FM Daughterboard

In contrast to the EM daughterboard, the FM daughterboard exposes all the signals below the board. This means all the connectors are placed in-between the motherboard and daughterboard which allows for high space efficiency within the satellite. All the connectors can be mated and de-mated from the sides. This makes it easy to alter the connections in an integrated stack. The connectors also have a higher reliability compared to the EM daughterboard.

In order to have the smallest mated height possible, the FM daughterboard uses a

SAMTEC ERM8-060-02.0-S-DV-TR connector to mate with J2. All components are placed on the bottom side of the daughterboard. The PCB thickness is 1mm. The mounting holes have a countersunk and the mounting screws have a flat countersunk head in order to fit within the height of the PCB.



Figure 4-5: FM daughterboard assembly (with iOBC, top view).

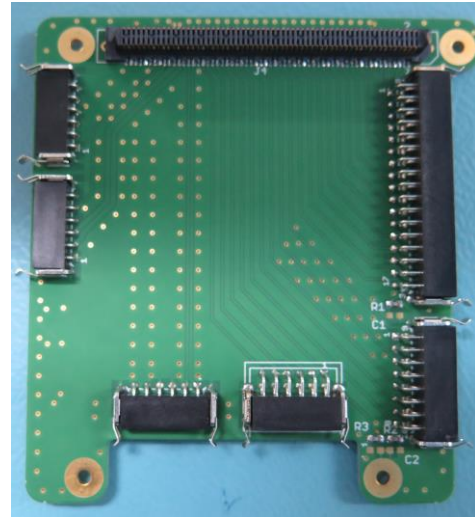


Figure 4-6: FM daughterboard (bottom view).

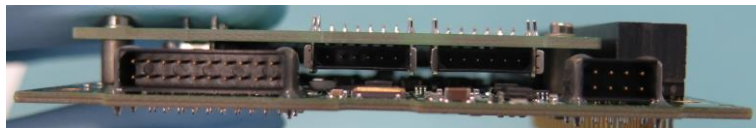


Figure 4-7: FM daughterboard assembly (X+ side view).



Figure 4-8: FM daughterboard assembly (Y- side view).



Figure 4-9: FM daughterboard assembly (X- side view).

4.3 Custom Daughterboard

A custom daughterboard can be built in order to take advantage of the full processing power and functionalities offered by the iOBC.

J2 connects the motherboard to the daughterboard. Its pin attribution is described in Table 5-4 while the electrical specifications of the different signals are described in Table 6-1.

There are several options for the mating connector on the daughterboard providing



a choice of distances in between the top of the motherboard and the bottom of the daughterboard. The options are described in Table 4-1.

Table 4-1: Daughterboard connector options (mating with J2).

Daughterboard connector	Mated height [mm]	Comments
SAMTEC ERM8-060-02.0-S-DV-TR	7	Used on EM and FM daughterboards
SAMTEC ERM8-060-05.0-S-DV-TR	10	
SAMTEC ERM8-060-08.0-S-DV-TR	13	Require additional height in stack
SAMTEC ERM8-060-09.0-S-DV-TR	14	Require additional height in stack

There are firm limitations in order not to interfere with any component on the motherboard. Detailed mechanical drawings are presented in section 7 for this purpose.

5 Connectors location and pin attribution

5.1 Motherboard

Figure 5-1 and Figure 5-2 show the location of the different connectors on the iOBC motherboard. The 2 SD-cards are placed on the bottom of the board.

Table 5-1 to Table 5-5 show the part number and pin attribution for each connector. The CSKB breakout connections described in Table 5-2 provide pads required to solder wires in order to connect to a different stack (via an in-line connector for example). The wire has to pass through the stress relief hole as shown in Figure 5-3 and then soldered. Epoxy must then be applied to secure the wire in place.

Note that the pins having the same net name on the CSKB (Table 5-1) are **always** connected together electrically, even when they are not used by the iOBC for any internal connection (for example, when the check box corresponding to the respective pin has been left un-ticked in the option sheet).

Note that the Debug LEDs are not described in the pin attribution because they use dedicated output pins on the CPU and have no other connection. Their interface is provided in the HAL library.

A programming and debug adapter connecting to J1 (Table 5-3) is provided by ISIS: it converts to a standard JTAG 20-pin connector and provides a UART over USB for debug purposes. **This adapter disables the supervisor watchdog feature when it is plugged into J1 AND when the SAM-ICE is connected to the 20-pin JTAG connector.**

Note that J3 (Table 5-5) is an optional connector. Normally, it cannot be placed together with a daughterboard since the nets are shared with J2 and because of its height preventing components on the daughterboard on this area. The purpose of this connector is to provide a simple and reliable connector that can be used for flight, in case only a subset of the peripherals is required (SPI1, UART0 and some GPIO). It is possible to use a wired connection in this case without the use of a daughterboard.

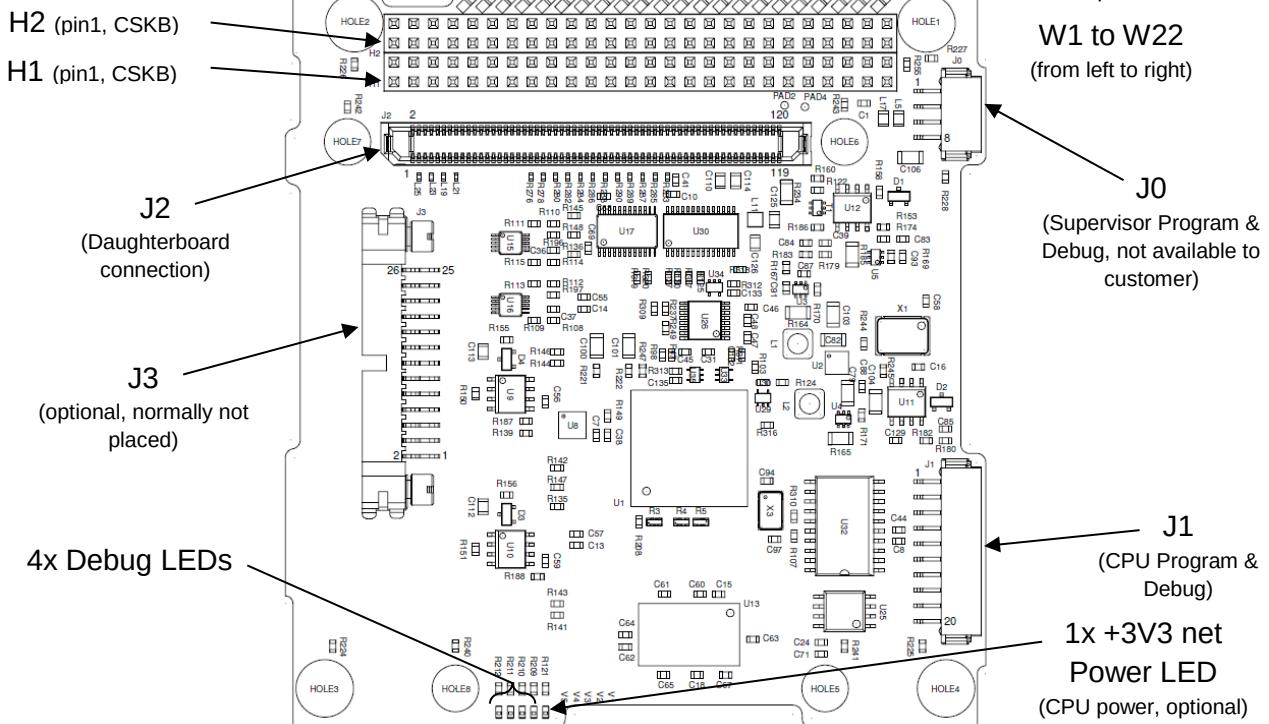


Figure 5-1: Connectors location (top).

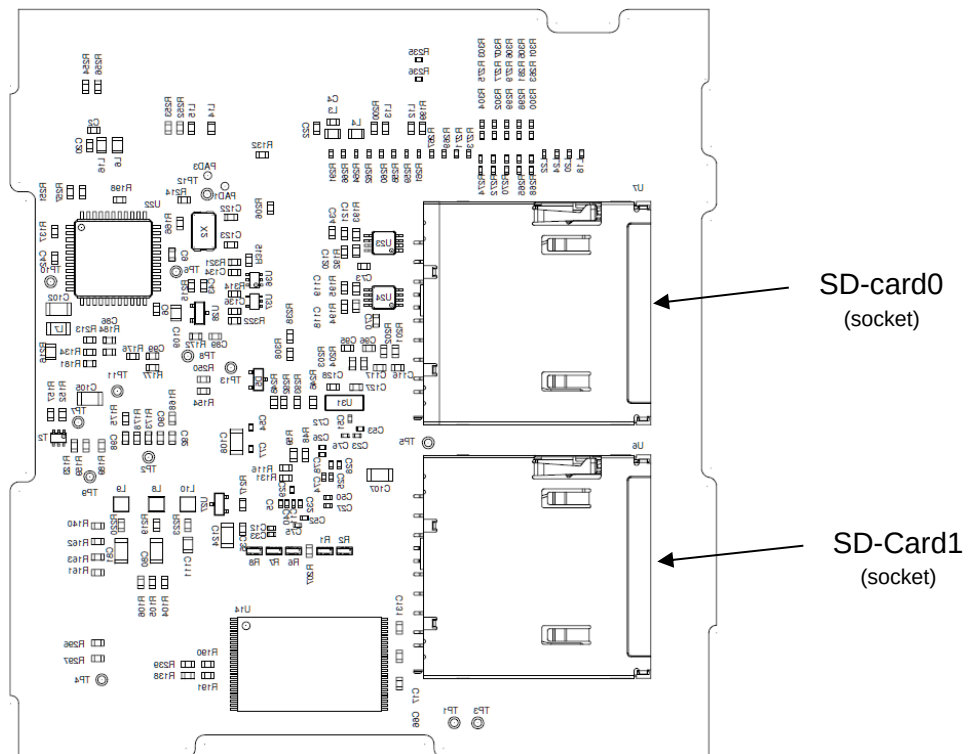


Figure 5-2: Connectors location (bottom).



ISIS-OBC Datasheet

Doc ID: ISIS.ISIS-OBC.DS.1.1
Issue: 1.3
Date: 2018-12-21
Page: 14 of 30

Table 5-1: CSKB pin attribution.

Cubesat Kit connector (CSKB)																											
ESQ-126-39-G-D or ESQ-126-38-G-D or SSQ-126-21-G-D or TSW-126-07-G-D (Female part of the connector is on top of the board)																											
H2											DRXD		+5V	+3V3_H2	GND	GND							VBAT		GPIO 24	GPIO 26	
	2	4	6	8	10	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52	
											DTXD		+5V	+3V3_H2	GND								VBAT			GPIO 25	
	1	3	5	7	9	11	13	15	17	19	21	23	25	27	29	31	33	35	37	39	41	43	45	47	49	51	
H1																							GPIO 23	+3V3_SW1 (al)	+3V3_SW2 (al)		
	2	4	6	8	10	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52	
											SCL (al)	SDA (al)										SDA	SCL	GPIO 22	+5V_SW1	+5V_SW2	
	1	3	5	7	9	11	13	15	17	19	21	23	25	27	29	31	33	35	37	39	41	43	45	47	49	51	

Table 5-2: Breakout wires.

Breakout wires		
#	Net name	Comment
W1	DTXD	
W2	DRXD	
W3	GND	
W4	SCL (al)	
W5	SDA (al)	
W6	GND	
W7	+5V	
W8	+5V	
W9	+3V3_H2	
W10	+3V3_H2	
W11	GND	
W12	SDA	
W13	SDA	
W14	GND	
W15	SCL	
W16	SCL	
W17	VBAT	Breakout wire only
W18	VBAT	Breakout wire only
W19	+5V_SW1	Breakout wire only
W20	+3V3_SW1 (al)	
W21	+5V_SW2	Breakout wire only
W22	+3V3_SW2 (al)	

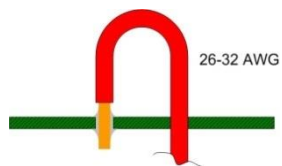


Figure 5-3: Stress relief soldered connection.

Table 5-3: J1 pin attribution.

J1					
CPU (ARM) programming / Debug					
Harwin M80-8412042					
#	Net name	#	Net name	Comments	
1	1 +3V3	2	11 +3V3	This is an OUTPUT! Do NOT connect a 3.3V supply here (or it would damage the board). Can source little current. For voltage sensing only. Provides the power to the I/O of the JTAG programmer.	
3	2 NTRST	4	12 GND		
5	3 TDI	6	13 GND		
7	4 TMS	8	14 GND		
9	5 TCK	10	15 GND		
11	6 RTCK	12	16 GND		
13	7 TDO	14	17 GND		
15	8 NRST	16	18 GND		
17	9 DRXD	18	19 GND		
19	10 DTXD	20	20 EN_WATCHDOG	Connect to GND to disable supervisor watchdog. Leave open to enable supervisor watchdog.	



ISIS-OBC Datasheet

Doc ID: ISIS.ISIS-OBC.DS.1.1

Issue: 1.3

Date: 2018-12-21

Page: 15 of 30

Table 5-4: J2 pin attribution.

J2, Daughterboard I/O, SAMTEC ERF8-060-05.0-S-DV-TR			
#	Net name	#	Net name
1	GND	2	GND
3	AIN0	4	AIN1
5	GND	6	GND
7	AIN2	8	AIN3
9	GND	10	GND
11	AIN4	12	AIN5
13	GND	14	GND
15	AIN6	16	AIN7
17	GND	18	GND
19	SPI1_NPCS0	20	SPI1_NPCS1
21	GND	22	GND
23	SPI1_NPCS2	24	SPI1_SPCK
25	GND	26	GND
27	SPI1_MOSI	28	SPI1_MISO
29	GND	30	GND
31	GPIO0/D8	32	GPIO1/D9
33	GND	34	GND
35	GPIO2/D10	36	GPIO3/D11
37	GND	38	GND
39	GPIO4/D0	40	GPIO5/D1
41	GND	42	GND
43	GPIO6/D2	44	GPIO7/D3
45	GND	46	GND
47	GPIO8/D4	48	GPIO9/D5
49	GND	50	GND
51	GPIO10/D6	52	GPIO11/D7
53	GND	54	GND
55	GPIO12/PCK	56	GPIO13/VSYNC
57	GND	58	GND
59	GPIO14/HSYNC	60	GPIO15/MCK
61	GND	62	GND
63	GPIO16	64	GPIO17
65	GND	66	GND
67	GPIO18	68	GPIO19
69	GND	70	GND
71	GPIO20	72	GPIO21
73	GND	74	GND
75	PWM0/TC0	76	PWM1
77	GND	78	GND
79	PWM2/TC1	80	PWM3
81	GND	82	GND
83	PWM4/TC2	84	PWM5
85	GND	86	GND
87	RX0	88	TX0
89	GND	90	GND
91	RX2/RX+	92	TX2/TX+/TRX+
93	CTS2/RX-	94	RTS2/TX-/TRX-
95	GND	96	GND
97	USBD_DP	98	USBH_DP
99	USBD_DM	100	USBH_DM
101	GND	102	GND
103	USBD_VBUS	104	USBH_VBUS
105	GND	106	GND
107	+5V	108	+5V
109	+3V3_IN	110	+3V3_IN
111	+3V3	112	+3V3
113	GND	114	GND
115	+5V_SW1	116	+5V_SW1
117	test pad 1	118	test pad 2
119	test pad 3	120	test pad 4

Table 5-5: J3 pin attribution.

J3			
Optional, normally not placed. Cannot be placed together with a daughterboard.			
Harwin M80-5S22605MQ			
#	Net name	#	Net name
25	13 GND	26	26 GND
23	12 SPI1_NPCS0	24	25 SPI1_NPCS1
21	11 GND	22	24 GND
19	10 SPI1_NPCS2	20	23 SPI1_SPCK
17	9 GND	18	22 GND
15	8 SPI1_MOSI	16	21 SPI1_MISO
13	7 GND	14	20 GND
11	6 GPIO0/D8	12	19 GPIO1/D9
9	5 GND	10	18 GND
7	4 GPIO2/D10	8	17 GPIO3/D11
5	3 GND	6	16 GND
3	2 RX0	4	15 TX0
1	1 GND	2	14 GND

Mechanical SMD pads are connected to GND.

5.2 EM Daughterboard

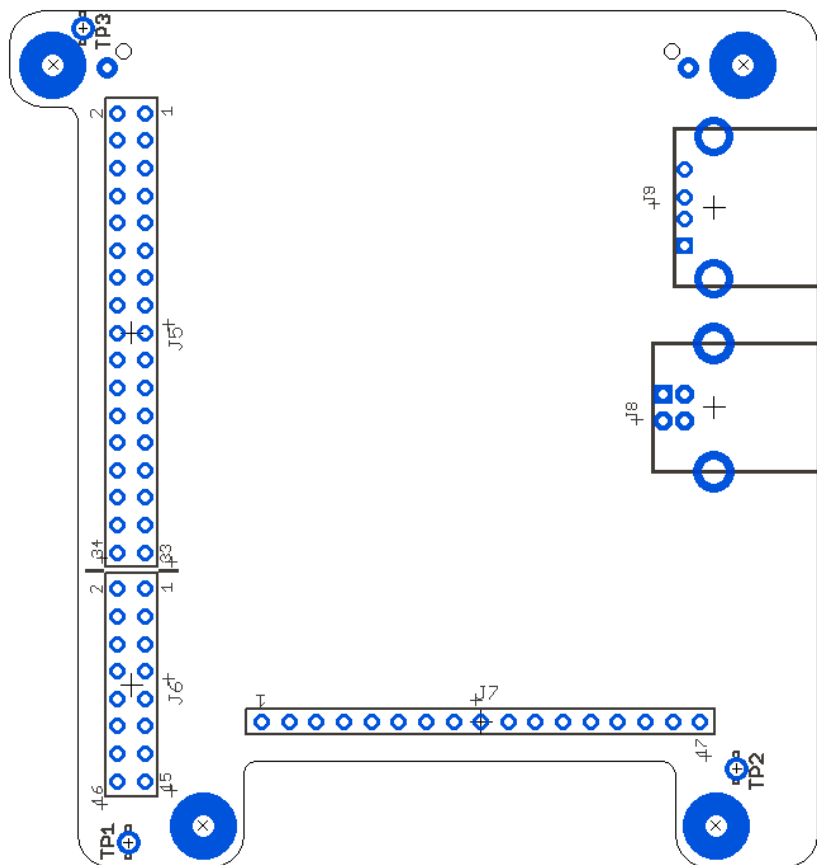


Figure 5-4: EM daughterboard connector location (top).

Figure 5-4 provides the location of the different connectors on the EM daughterboard. Table 5-6 to Table 5-10 provide the part number and pin attribution of the different connectors.

Note that TP1, TP2 and TP3 are connected to GND.

Note that in Table 5-10, USBH_VBUS is 3.3V and not 5V as in a typical USB. Moreover it is not intended to provide power to a device, but should only be used as a power sensing pin, since a 1kohm resistor is placed in serie (see end of Table 6-1 for more details.)

Table 5-6: J5 pin attribution.

J5			
2.54mm male pin header			
SAMTEC TSW-117-07-F-D			
#	Net name	#	Net name
2	18 AIN0	1	1 GND
4	19 AIN2	3	2 AIN1
6	20 GND	5	3 AIN3
8	21 AIN5	7	4 AIN4
10	22 AIN7	9	5 AIN6
12	23 SPI1_NPCS0	11	6 GND
14	24 SPI1_NPCS2	13	7 SPI1_NPCS1
16	25 SPI1_SPCK	15	8 GND
18	26 SPI1_MISO	17	9 SPI1_MOSI
20	27 GPIO0/D8	19	10 GND
22	28 GPIO2/D10	21	11 GPIO1/D9
24	29 GND	23	12 GPIO3/D11
26	30 GPIO5/D1	25	13 GPIO4/D0
28	31 GPIO7/D3	27	14 GPIO6/D2
30	32 GPIO8/D4	29	15 GND
32	33 GPIO10/D6	31	16 GPIO9/D5
34	34 +3V3_SENSE_J5	33	17 GPIO11/D7

Table 5-7: J6 pin attribution.

J6			
2.54mm male pin header			
SAMTEC TSW-108-07-F-D			
#	Net name	#	Net name
2	9 GPIO12/PCK	1	1 GND
4	10 GPIO13/VSYNC	3	2 GND
6	11 GND	5	3 GPIO14/HSYNC
8	12 GND	7	4 GPIO15/MCK
10	13 GPIO17	9	5 GPIO16
12	14 GND	11	6 GPIO18
14	15 GPIO20	13	7 GPIO19
16	16 +3V3_SENSE_J6	15	8 GPIO21

Table 5-8: J7 pin attribution.

J7	
2.54mm male pin header	
SAMTEC TSW-117-07-F-S	
#	Net name
1	+3V3_SENSE_J7
2	PWM0/TC0
3	PWM1
4	PWM2/TC1
5	GND
6	PWM3
7	PWM4/TC2
8	PWM5
9	GND
10	RX0
11	TX0
12	GND
13	RX2/RX+
14	CTS2/RX-
15	GND
16	TX2/TX+/TRX+
17	RTS2/TX-/TRX-

Table 5-9: J8 pin attribution.

J8	
USB-B Connector	
FCI 61729-0010BLF	
#	Net name
1	USBD_VBUS
2	USBD_DM
3	USBD_DP
4	GND

Table 5-10: J9 pin attribution.

J9	
USB-A Connector	
FCI 87520-0010BLF	
#	Net name
1	USBH_VBUS
2	USBH_DM
3	USBH_DP
4	GND

5.3 FM Daughterboard

The location of the connectors on the FM daughterboard is shown in Figure 5-5. The part number and pin attribution for all the respective connectors are described in Table 5-11 to Table 5-15.

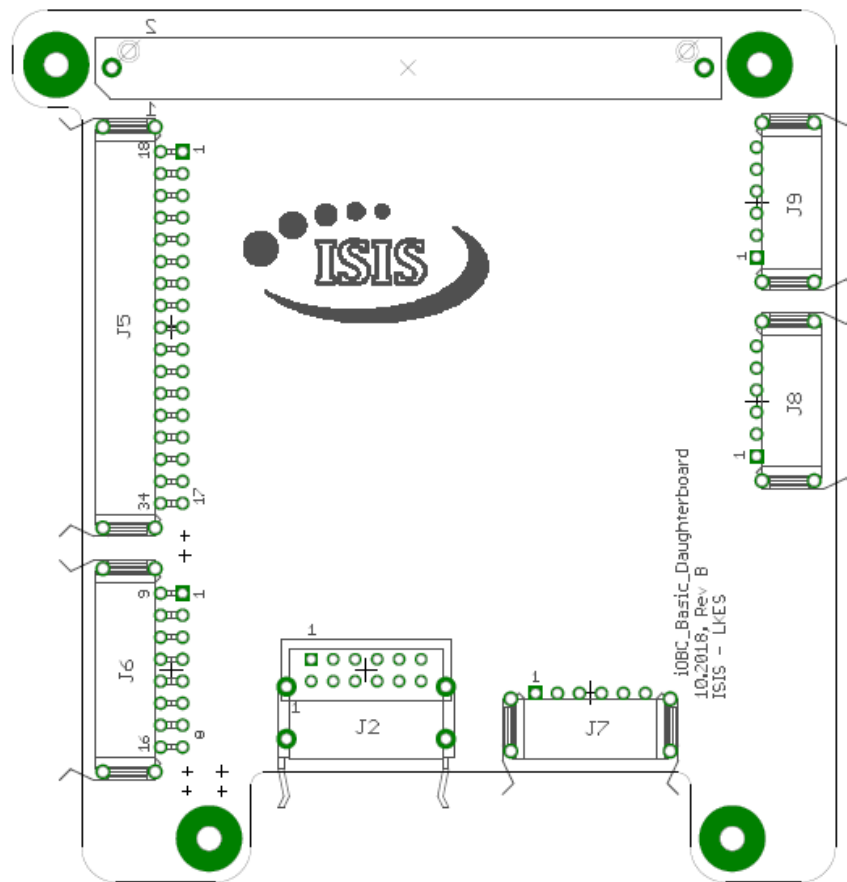


Figure 5-5: FM daughterboard connector location (bottom, through view from top).

Table 5-11: J5 pin attribution.

J5			
Harwin M80-8513442			
Mate with Harwin M80-8883405			
#	Net name	#	Net name
1	1 GND	2	18 AIN0
3	2 AIN1	4	19 AIN2
5	3 AIN3	6	20 GND
7	4 AIN4	8	21 AIN5
9	5 AIN6	10	22 AIN7
11	6 GND	12	23 SPI1_NPCS0
13	7 SPI1_NPCS1	14	24 SPI1_NPCS2
15	8 GND	16	25 SPI1_SPCK
17	9 SPI1_MOSI	18	26 SPI1_MISO
19	10 GND	20	27 GPIO0/D8
21	11 GPIO1/D9	22	28 GPIO2/D10
23	12 GPIO3/D11	24	29 GND
25	13 GPIO4/D0	26	30 GPIO5/D1
27	14 GPIO6/D2	28	31 GPIO7/D3
29	15 GND	30	32 GPIO8/D4
31	16 GPIO9/D5	32	33 GPIO10/D6
33	17 GPIO11/D7	34	34 +3V3_SENSE_J5

Table 5-12: J6 pin attribution.

J6			
Harwin M80-8511642			
Mate with Harwin M80-8881605			
#	Net name	#	Net name
1	1 GND	2	9 GPIO12/PCK
3	2 GND	4	10 GPIO13/VSYN
5	3 GPIO14/HSYN	6	11 GND
7	4 GPIO15/MCK	8	12 GND
9	5 GPIO16	10	13 GPIO17
11	6 GPIO18	12	14 GND
13	7 GPIO19	14	15 GPIO20
15	8 GPIO21	16	16 +3V3_SENSE_J6

Table 5-13: J2 pin attribution.

J2			
Harwin M80-8511242			
Mate with Harwin M80-8881205			
#	Net name	#	Net name
1	1 PWM0/TC0	2	7 +3V3
3	2 PMW1	4	8 PWM2/TC1
5	3 PWM3	6	9 PWM3/TC2
7	4 GND	8	10 PWM5
9	5 GND	10	11 GND
11	6 RX0	12	12 TX0

Table 5-14: J7 pin attribution.

J7	
Harwin M80-8420642	
Mate with Harwin M80-8980605.	
#	Net name
1	RX2/Rx+
2	CTS2/Rx-
3	GND
4	GND
5	TX2/TX+/TRX+
6	RTS2/TX-/TRX-

Table 5-16: J8 pin attribution.

J8	
Harwin M80-8420642	
Mate with Harwin M80-8990605 (large bore). Use AWG22 (Harwin M80-9220099) for ~97Ω differential impedance with twisted cables.	
#	Net name
1	GND
2	USBD_DP
3	USBD_DM
4	GND
5	USBD_VBUS
6	GND

Table 5-15: J9 pin attribution.

J9	
Harwin M80-8420642	
Mate with Harwin M80-8990605	
#	Net name
1	GND
2	USBH_DP
3	USBH_DM
4	GND
5	USBH_VBUS
6	GND



6 Electrical specifications

The electrical specifications for all the nets exposed on the connectors are presented in Table 6-1. Table 6-2 and Table 6-3 show additional power supply requirements.

Table 6-1: Detailed electrical specifications for each net.

Connector name	Net name	Function	Dire- ction	VIL [V]		VIH min [V]		VOL [V]		VOH [V]		Total serie resistance [Ω] ^{*1}	Pull up/down resistance [Ω] ^{*2}	Speed	Configu- ration option	Comments
				Min	Max	Min	Max	Min	Max	Min	Max		Typ			
W3/W6/W11/ W14/H2	GND	Ground	I	0	0	-	-	-	-	-	-	-	-	-	-	Power supply return, ground
W9/W10/H2	+3V3_H2	3.3V power input	I	-	-	3.1	3.5	-	-	-	-	-	-	-	HW	Default power input, becomes +3V3_IN if used.
W20/H1	+3V3_SW1 (al)	3.3V power input	I												HW	Optional power input, becomes +3V3_IN if used.
W22/H1	+3V3_SW2 (al)	3.3V power input	I												HW	Optional power input, becomes +3V3_IN if used.
	+3V3_IN	Main power input	I													Main power input for CPU and supervisor.
J2	+3V3	CPU power supply	O	-	-	-	-	0	-	-	+3V3_IN	-	-	-	-	This is an OUTPUT . Derived internally from +3V3_IN via a current limiting switch. It is the CPU power supply (I/O and DCDC converters for core and memory voltages, and daughterboard). Will go to 0V if the supervisor switches of the CPU (i.e. because of the watchdog trigger). On the daughterboard, supports only typically ~20mA to 30mA (Higher current is possible depending on which peripherals of the CPU are used). Use or sense this power supply on the daughterboard in order to avoid any I/O leakage to the CPU.
W7/W8/H2	+5V	5.0V input	I/O	-	-	4.5	5.5	-	-	-	-	-	-	-	-	Not used by iOBC, but availalbe as breakout. Provided to daughterboard.
W19/H1/J2	+5V_SW1	5.0V input	I/O	-	-	-	-	-	-	-	-	-	-	-	HW	Switched line on GomSpace EPS, not used by iOBC, but available as breakout and on daughterboard.
W21/H1	+5V_SW2	5.0V input	I/O	-	-	-	-	-	-	-	-	-	-	-	HW	Switched line on GomSpace EPS, not used by iOBC, but available as breakout.
W17/W18/H2	VBAT	Battery bus	I/O	-	-	-	-	-	-	-	-	-	-	-	-	Not used by iOBC, but availalbe as breakout.
W15/W16/H1	SCL	I2C clock	I/O	-0.5	0.9	2.52	5.5	0	0.2	-	5.5	-	-	≤ 400kbit/s	HW	5V tolerant. Buffered (with full I2C levels compliance for 3.3V or 5V); a voltage can be applied when board is off. ESD protection 5.5kV HBM.
W4/H1	SCL (al)														HW	
W12/W13/H1	SDA	HW														
W5/H1	SDA (al)	HW														
		I2C data														

*1 : Includes the maximum guaranteed serie resistance of the output buffer and any serie resistor on the line

*2 : "SW, Up" mean that an optional internal pull-up can be configured in software. The pull-up resistance values is $40k\Omega \leq R \leq 190k\Omega$. No weak pull-down can be configured. IMPORTANT: the pull-up resistor is ALWAYS enabled during boot and can be disabled afterwards. If it is necessary that the level remains low at during boot, a $\leq 4.7k\Omega$ external pull-down resistor must be used.



ISIS-OBC Datasheet

Doc ID: ISIS.ISIS-OBC.DS.1.1
Issue: 1.3
Date: 2018-12-21
Page: 22 of 30

Connector name	Net name	Function	Dire- ction	VIL [V]		VIH min [V]		VOL [V]		VOH [V]		Total serie resistance [Ω] *1	Pull up/down resistance [Ω] *2	Speed	Configu- ration option	Comments
				Min	Max	Min	Max	Min	Max	Min	Max		Typ			
W1/H2/J1	DTXD	Debug UART transmit	O	-	-	-	-	0	-	-	+3V3	380	SW, Up	2Mbit/s typ	HW	Optional on H2, available as breakout. Schmitt trigger buffer, 5V tolerant. No leakage in +3V3 when iOBC is off or VIH > +3V3.
W2/H2/J1	DRXD	Debug UART receive	I	-0.5	0.89	2.25	5.5	-	-	-	-	330	10k, Up		HW	Optional on H2, available as breakout. Schmitt trigger buffer, 5V tolerant. No leakage in +3V3 when iOBC is off or VIH > +3V3.
J1	+3V3	JTAG target voltage	O	-	-	-	-	0	-	-	+3V3	-	-	-	-	Used as power for the I/O of the JTAG emulator here. Do not use that pin to power anything else.
	NTRST	JTAG reset	I/O	-0.3	0.8	2	+3V3 + 0.3	0	-	-	+3V3	-	1k, Up	-	-	
	TDI	JTAG data input	I	-0.3	0.8	2	+3V3 + 0.3	-	-	-	-		100k, Up	-	-	
	TMS	JTAG mode set	I/O	-0.3	0.8	2	+3V3 + 0.3	0	-	-	+3V3		100k, Up	-	-	
	TCK	JTAG clock	I	-0.3	0.8	2	+3V3 + 0.3	-	-	-	-		10k, Up	-	-	
	RTCK	JTAG return test clock	O	-	-	-	-	0	-	-	+3V3	-	-	-	-	
	TDO	JTAG data output	O	-	-	-	-	0	-	-	+3V3	-	-	-	-	
	NRST	Target CPU reset signal	I/O	-0.3	0.8	2	+3V3 + 0.3	0	-	-	+3V3	-	3k3, Up	-	-	
	EN_WATCHDOG	Enable watchdog	I	-0.3	0.8	2	+3V3 + 0.3	-	-	-	-	-	3k3, Up	-	-	Connect to GND to disable watchdog. Leave open to enable watchdog.
H1	GPIO22	General purpose I/O	I/O	-0.3	0.8	2	+3V3 + 0.3	0	-	-	+3V3	330 - 380	SW, Up	-	HW	
	GPIO23														HW	
H2	GPIO24														HW	
	GPIO25														HW	
	GPIO26														HW	
J2	AIN0	Analog input	I	0	-	-	2.5	-	-	-	-	-	-	≤125kHz signal bandwidth (1 analog channel only, 8 bit mode)	-	Inputs have 1uA leakage max, 40pF typ input capacitance. Note that the ADC uses a 2.5V reference with 0.43% overall accuracy.
J2	AIN1															
J2	AIN2															
J2	AIN3															
J2	AIN4															
J2	AIN5															
J2	AIN6															
J2	AIN7															
J2	SPI1_NPCS0	SPI chip select	I/O	-0.3	0.8	2	+3V3 + 0.3	0	-	-	+3V3	150	SW, Up	-	SW	Via level shifter; an external pull-down will not guarantee a low level at boot. The level will set high during boot. Output only: used for SPI master only.
J2	SPI1_NPCS1		O									125	-		SW	
J2	SPI1_NPCS2											125	-		SW	
J2	SPI1_SPCK	SPI clock	I/O									100 - 150	SW, Up	≤10Mbit/s	SW	
J2	SPI1_MOSI	SPI data master out slave in													SW	
J2	SPI1_MISO	SPI data master in slave out													SW	



ISIS-OBC Datasheet

Doc ID: ISIS.ISIS-OBC.DS.1.1
Issue: 1.3
Date: 2018-12-21
Page: 23 of 30

Connector name	Net name	Function	Direction	VIL [V]		VIH min [V]		VOL [V]		VOH [V]		Total series resistance [Ω] ^{*1}	Pull up/down resistance [Ω] ^{*2}	Speed	Configuration option	Comments
				Min	Max	Min	Max	Min	Max	Min	Max		Typ			
J2	GPIO0/D8	Image sensor data	I	-0.3	0.8	2	+3V3 + 0.3	-	-	-	-	100	SW, Up	-	SW	
J2	GPIO1/D9														SW	
J2	GPIO2/D10														SW	
J2	GPIO3/D11														SW	
J2	GPIO4/D0														SW	
J2	GPIO5/D1														SW	
J2	GPIO6/D2														SW	
J2	GPIO7/D3														SW	
J2	GPIO8/D4														SW	
J2	GPIO9/D5														SW	
J2	GPIO10/D6														SW	
J2	GPIO11/D7														SW	
J2	GPIO12/PCK	Image sensor pixel clock	I	-0.3	0.8	2	+3V3 + 0.3	-	-	-	-	100	SW, Up	-	SW	
J2	GPIO13/VSYN	Image sensor vertical sync													SW	
J2	GPIO14/HSYN	Image sensor pixel horizontal sync													SW	
J2	GPIO15/MCK	Image sensor clock to sensor	O	-	-	-	-	0	-	-	+3V3	150	SW, Up	-	SW	
J2	GPIO0/D8	General purpose I/O	I/O	-0.3	0.8	2	+3V3 + 0.3	0	-	-	+3V3	100 - 150	SW, Up	-	SW	Optional SW chip select for SPI1: SPI1_NPCS3_SW
J2	GPIO1/D9														SW	Optional SW chip select for SPI1: SPI1_NPCS4_SW
J2	GPIO2/D10														SW	Optional SW chip select for SPI1: SPI1_NPCS5_SW
J2	GPIO3/D11														SW	Optional SW chip select for SPI1: SPI1_NPCS6_SW
J2	GPIO4/D0														SW	Optional SW chip select for SPI1: SPI1_NPCS7_SW
J2	GPIO5/D1														SW	
J2	GPIO6/D2														SW	
J2	GPIO7/D3														SW	
J2	GPIO8/D4														SW	
J2	GPIO9/D5														SW	
J2	GPIO10/D6														SW	
J2	GPIO11/D7														SW	
J2	GPIO12/PCK														SW	
J2	GPIO13/VSYN														SW	
J2	GPIO14/HSYN														SW	
J2	GPIO15/MCK														SW	
J2	GPIO16														SW	
J2	GPIO17														SW	
J2	GPIO18														SW	
J2	GPIO19														SW	
J2	GPIO20														SW	
J2	GPIO21														SW	



ISIS-OBC Datasheet

Doc ID: ISIS.ISIS-OBC.DS.1.1
Issue: 1.3
Date: 2018-12-21
Page: 24 of 30

Connector name	Net name	Function	Dire- ction	VIL [V]		VIH min [V]		VOL [V]		VOH [V]		Total serie resistance [Ω] ^{*1}	Pull up/down resistance [Ω] ^{*2}	Speed	Configu- ration option	Comments	
				Min	Max	Min	Max	Min	Max	Min	Max		Typ				
J2	PWM0/TC0	PWM output	O	-	-	-	-	0	-	-	+3V3	150	SW, Up	≤33MHz	SW	Via level shifter; an external pull-down will not garantee a low level at boot. The level will set high during boot. Output only.	
J2	PWM1											125	-		-		
J2	PWM2/TC1											150	SW, Up		SW		
J2	PWM3											125	-		-		
J2	PWM4/TC2											150	SW, Up		SW		
J2	PWM5											125	-		-		
J2	PWM0/TC0	Timer counter input	I	-0.3	0.8	2	+3V3 + 0.3	-	-	-	-	100	SW, Up	TBD	SW	Timer counter / capture functionality not implemented in HAL yet.	
J2	PWM2/TC1														SW		
J2	PWM4/TC2														SW		
J2	RX0	UART receive	I	-0.5	0.89	2.25	5.5	-	-	-	-	100	100k, Up	≤500kbit/s TBC (SW limited)	HW	Schmitt trigger buffer, 5V tolerant. Small leakage via 100kohm into +3V3 when iOBC is off or VIH > +3V3.	
J2	TX0	UART transmit	O	-	-	-	-	0	-	-	+3V3	150	SW, Up		HW		
J2	RX0	RS232 receive	I	-25	0.6	2.4	25	-	-	-	-	-	5k, Down	≤500kbit/s (SW limited), ≥250kbit/s	HW	ESD protection +-15kV HBM.	
J2	TX0	RS232 transmit	O	-	-	-	-	-13.2	-5	5	13.2	-	-		HW		
J2	RX2/RX+	RS232 receive	I	-15	0.5	2.5	15	-	-	-	-	-	5k, Down	typ @ 1000pF //3kΩ load	SW	ESD protection +-26kV HBM.	
J2	CTS2/RX-	RS232 clear to send													SW		
J2	TX2/TX+/TRX+	RS232 transmit	O	-	-	-	-	-7.5	-5	5	7.5	-	-		SW		
J2	RTS2/TX-/TRX-	RS232 ready to send													SW		
J2	RX2/RX+	RS485 full duplex receive	I	+15V absolute voltage max, +6V differential max, +0.2V threshold max				+6V differential max, + 0.2V differential min, +3V common mode voltage max				-	125k, Down	SW	ESD protection +-26kV HBM. Software enabled 120ohm differential termination. 120ohm differential impedance.		
J2	CTS2/RX-		SW														
J2	TX2/TX+/TRX+	RS485 full duplex transmit	O											SW			
J2	RTS2/TX-/TRX-		SW														
J2	TX2/TX+/ TRX+	RS485 single duplex transmit receive	I/O											TBD	SW	Functionality not implemented in HAL yet.	
J2	RTS2/TX-/ TRX-													SW			
J2	USBD_DP	USB data	I/O	-0.3	0.8	2	+3V3 + 0.3	0	0.3	2.8	3.6	-	-	≤12Mbit/s	-	90ohm differential impedance.	
J2	USBD_DM													-			
J2	USBD_VBUS	USB connection sense	I	-0.5	0.89	2.25	5.5	-	-	-	-	-	27k, Down	-	-	Schmitt trigger buffer, 5V tolerant. No leakage into +3V3 when iOBC is off or VIH > +3V3.	
J2	USBH_DP	USB data	I/O	-0.3	0.8	2	+3V3 + 0.3	0	0.3	2.8	3.6	-	-	≤12Mbit/s	-	90ohm differential impedance.	
J2	USBH_DM													-			
J2	USBH_VBUS	USB connection sense	O	-	-	-	-	0	-	-	+3V3	1k	-	-	-	Connected to +3V3 via 1kohm.	
J5, J6, J7	+3V3_SENSE_Jx	Sense power of DMU															

Table 6-2: Power on/off requirements.

+3V3_IN Input voltage rate [V/ms]			
	Min	Max	Comments
Rise rate (at power-on)	0.2	5	Higher slew rate than the maximum specified might damage EMI filters on the iOBC or create brownout issues due to the large amount of capacitance present on the board.
Fall rate (at power-off)	-	-	No requirement

Table 6-3: Power consumption.

Configu- ration	State description ⁽¹⁾	Parameter	Unit	Value for +3V3_IN net			Comments
				Min ⁽²⁾	Typ ⁽³⁾	Max ⁽⁴⁾	
-	All	+3V3_IN voltage	V	3.1	3.3	3.5	
Master and slave	CPU ON, typical usage ⁽⁵⁾	Average current	mA	64.5	116	158	Depending on peripherals used and processing load. Without any I/O sourcing current.
		Average power	mW	200	383	551	
	CPU ON, using SD- Card	Average current	mA	-	95	-	
	At CPU power on	Peak current	mA	-	530	-	In-rush current for about 0.4ms
Master	IDLE (CPU IDLE)	Average current	mA	61	64.5	75.3	
		Average power	mW	188	213	263	
Slave	IDLE (CPU off)	Average current	mA	7.2	8.4	10.9	
		Average power	mW	22	28	38	

⁽¹⁾ Note that the supervisor is powered on in all configurations and all cases. It cannot be switched off.

⁽²⁾ Over full temperature and input voltage range.

⁽³⁾ At 20°C and 3.30V input voltage.

⁽⁴⁾ Over full temperature and input voltage range and including 5% margin from measured value.

⁽⁵⁾ Typical situation using SDRAM, SD-Card, SPI, PWM, ...

7 Mechanical Characteristics

Figure 7-2 and Figure 7-3 provides the position of the different mounting holes on the motherboard. The centre of J2 are pin 1 of H1 are also indicated.

The mounting pads are plated through holes (they cannot be threaded). They are normally connected to GND but can be disconnected on request. The mounting pads are:

- Motherboard: 3.2mm drill, 7.4mm diameter pad (for typical 6mm diameter spacer).
- Daughterboard: 2.2mm drill, 6mm diameter pad (for typical 5mm diameter spacer).

Figure 7-4 shows the recommended daughterboard outline for custom daughterboards. Figure 7-5 shows the height of the different components on the motherboard per area. Table 4-1 shows the different mating height options for the daughterboard. These values constrain the design of the daughterboard, in order not to clash with any component on the motherboard.

Note: it is highly recommended to keep at least 1.5mm of clearance in between any component on the motherboard and the custom daughterboard. This is necessary since the boards bend due to the vibrations in the launch environment.

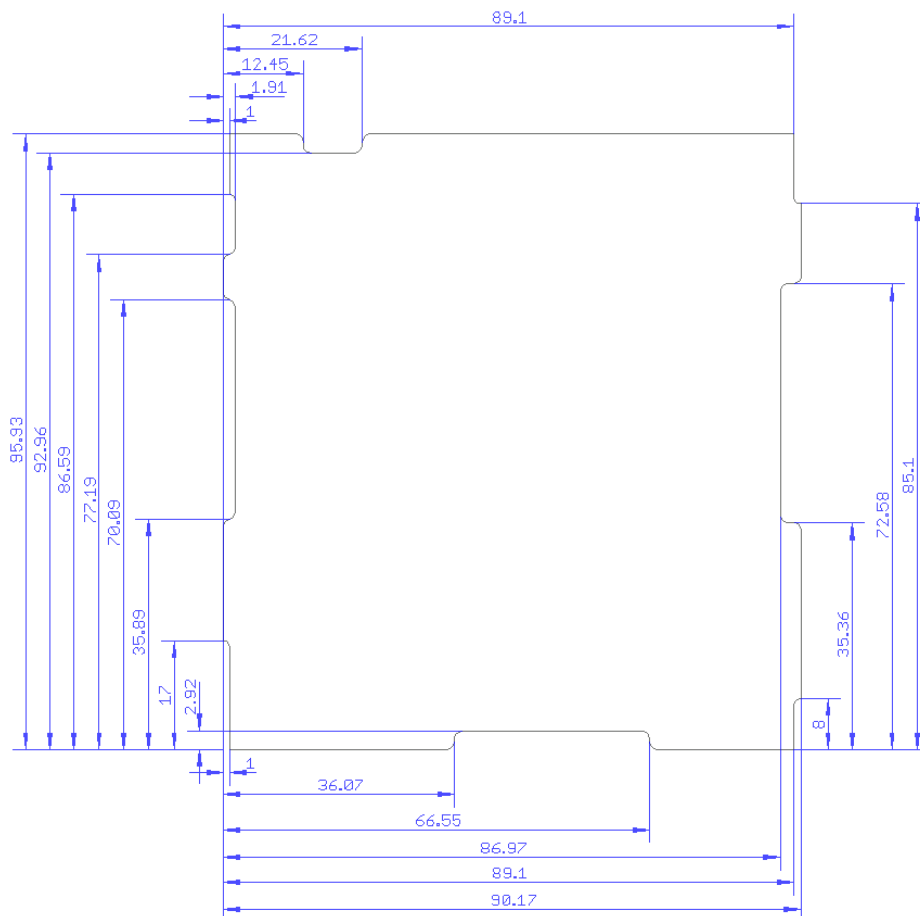


Figure 7-1: Motherboard outline (top view).

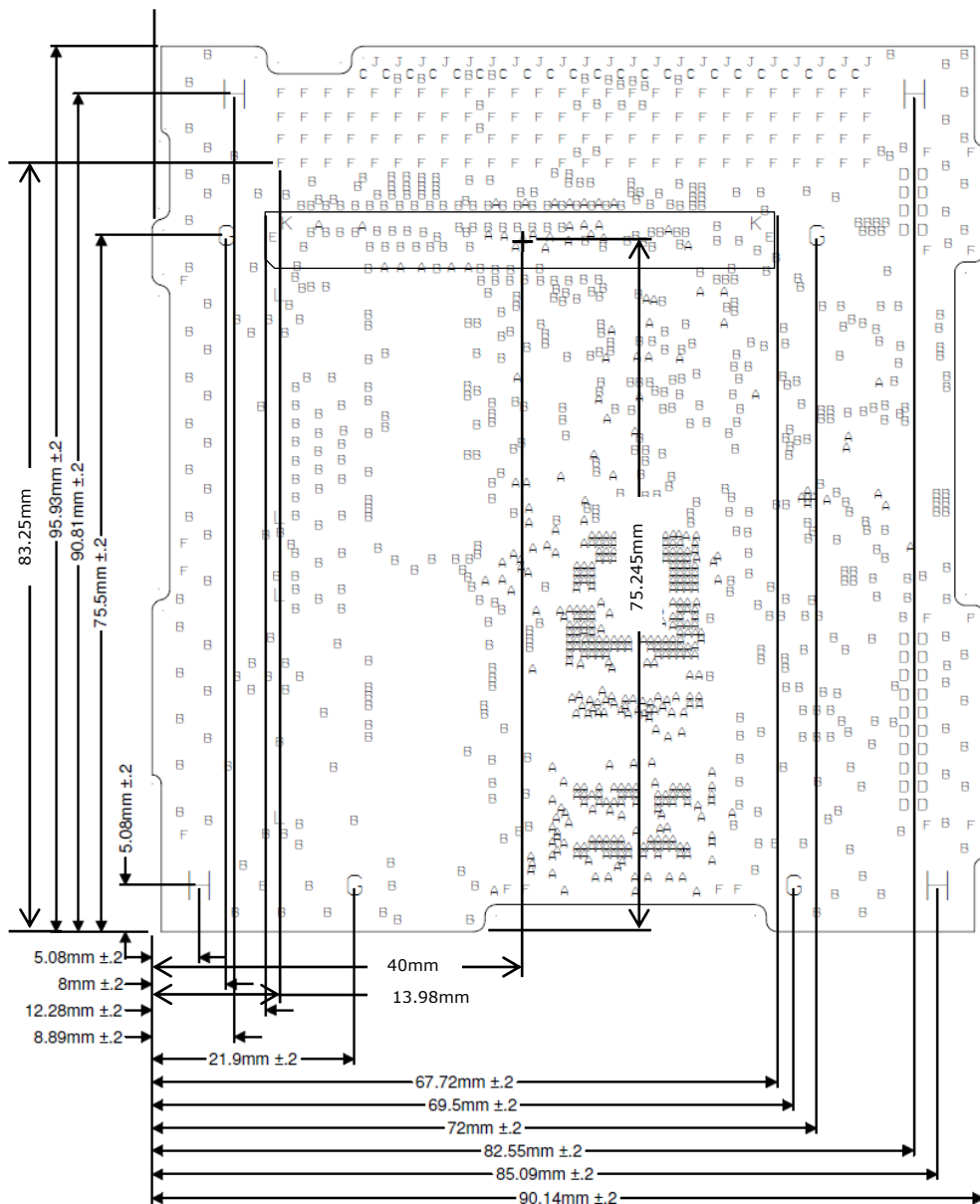


Figure 7-2: Drills, mounting points, J2 centre and H1 pin1 location (top view).

Through Holes					
Symbol	Diameter (mm)	Tolerance (mm)	Plated	Punched	Quantity
A	0.15	+/- 0.05	Yes	No	418
B	0.30	+/- 0.10	Yes	No	611
C	0.70	+/- 0.10	Yes	No	22
D	0.80	+/- 0.10	Yes	No	28
E	0.84	+/- 0.10	Yes	No	2
F	1.00	+/- 0.10	Yes	No	120
G	2.20	+/- 0.10	Yes	No	4
H	3.20	+/- 0.10	Yes	No	4
J	1.00	+/- 0.05	No	No	22
K	1.45	+/- 0.05	No	No	2
L	1.50	+/- 0.05	No	No	4

Daughterboard mounting holes
Motherboard mounting holes

Figure 7-3: Drill sizes.

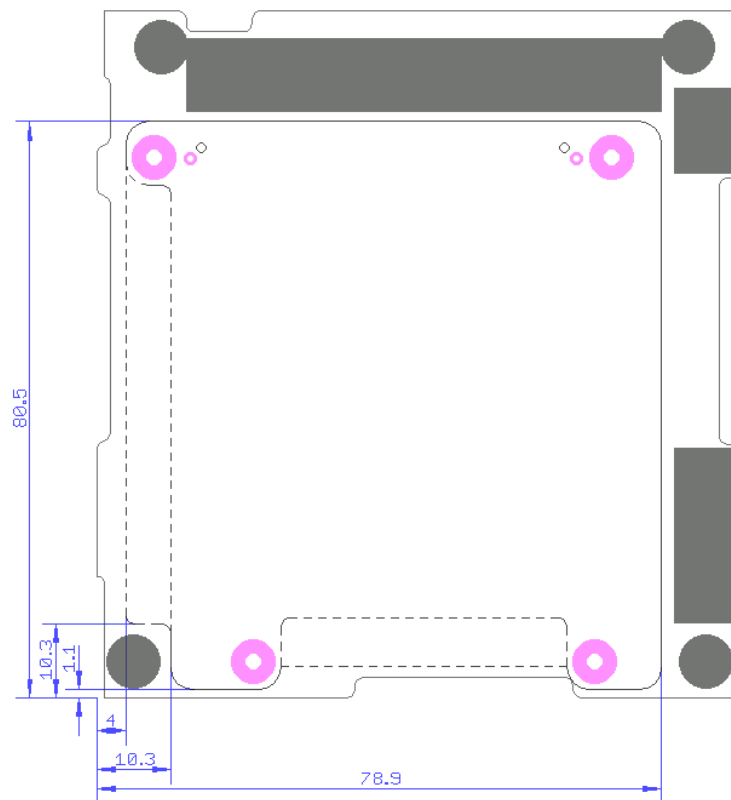


Figure 7-4: Recommended daughterboard outline [mm] (top view).

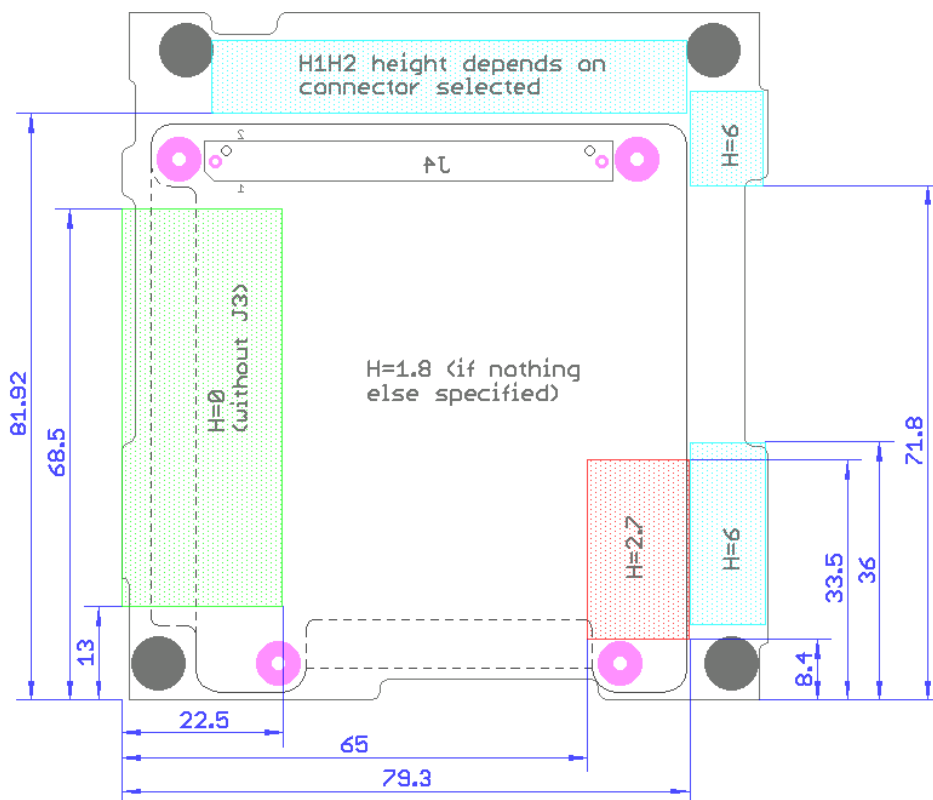


Figure 7-5: Height of components on motherboard [mm] (top view).

8 Storage and Handling Information

8.1 Handling



Note that the iOBC is sensitive to Electro Static Discharge (ESD).

8.1.1 Electrostatic discharge ESD

CAUTION



The printed circuit board can be damaged by electrostatic discharge. Do not touch any of the boards unless it is absolutely necessary. If you must handle them, wear a grounded wrist strap and take other antistatic precautions. Wear a grounded wrist strap any time you must handle the board.

8.1.2 Exposed Voltages

WARNING



Handling the board with an active power supply connection is not recommended. The board itself could be damaged and there is a possibility of electric shock hazard

In the event of Emergency, disconnect the power supply and proceed, if required, with first aid activities.

8.1.3 Current Limit protection

CAUTION



Ensure that over-current protection to a level of 1A or less is present when connecting to external power supplies.

8.1.4 Operation Conditions

CAUTION



Limit the number of connector mating cycles to less than 50 (about 10 cycles are used during functional testing)

CAUTION

The IOBC is supplied with final functional test results. Operating it outside its prescribed operating conditions may impede functionality.

Ensure that the system is always operated within its qualification temperature range.

CAUTION

This system does not have a protective housing and is therefore not intended for outdoor use as the board electronics might be damaged.

8.2 Storage

CAUTION

Store the IOBC inside a sealed ESD bag and in an environment controlled area.

The absolute maximum ratings for storage temperature are from -40 to +80°C with a Relative Humidity <60%

8.3 Disposal

WARNING

This product contains materials that can be harmful for the Environment and as such it should not be disposed of with conventional waste but treated according to WEEE regulations (UE Directives 2002/96/EC and further amendments) and brought to an appropriate recycling facility.

נספחים

HoopoeWorld

```

1 __title__ = "ServerSetup.py"
2 __author__ = 'Maayan'
3 import os
4 import subprocess
5
6 os.chdir(os.path.dirname(os.path.abspath(__file__)) + "\\Certificate")
7 CA = {"key": r"rootCA.key", "cert": r"rootCA.crt"}
8
9 def main():
10     print(f"current working dir: {os.getcwd()}")
11     if not os.path.exists(CA["key"]) or not os.path.exists(CA["cert"]):
12         subprocess.run(args=f"openssl genrsa -out {CA["key"]} 4096".split(" "), check=
True)
13         subprocess.run(args=f"openssl req -x509 -new -nodes -key {CA["key"]} -
sha256 -days 3650 -out {CA["cert"]} -subj \"/C=IL/O=HSL/CN=HoopoeWorld\"".split(" "),
check=True)
14
15     subprocess.run(args="openssl genrsa -out server.key 4096".split(" "), check=True)
16     subprocess.run(args="openssl req -new -key server.key -out server.csr -config
server.cnf".split(" "), check=True)
17     subprocess.run(args=f"openssl x509 -req -in server.csr -CA {CA["cert"]} -CAkey
{CA["key"]} -CAcreateserial -out server.crt -days 3650 -sha256 -extfile server.cnf -
extensions v3_ca".split(" "), check=True)
18
19     try:
20         print("where pip? " + subprocess.run(args="where pip".split(" "), capture_output
= True, text = True, check=True).stdout)
21     except subprocess.CalledProcessError:
22         print("need to install pip")
23         return
24     subprocess.run(args="pip install requests".split(" "), check=True)
25     subprocess.run(args="pip install pandas".split(" "), check=True)
26     subprocess.run(args="pip install openpyxl".split(" "), check=True)
27
28     if __name__ == "__main__":
29         main()
30

```

```

1 __title__ = "createSQLTable.py"
2 __author__ = 'Maayan'
3 import json
4 import sqlite3
5 connection_sql = sqlite3.connect("../data/SatDatabase.db")
6 cursor_obj = connection_sql.cursor()
7
8 #right now I will make it for the format we created in our program. If I have more
time, I will maybe change it to be more general.

9
10 types = {
11     "uint": "UNSIGNED INT", # in sql you don't have unsigned. and because I don't
do stuff with it but store it we make it text.
12     "int": "INT",
13     "ushort": "UNSIGNED SHORT",
14     "short": "SHORT",
15     "unixtime": "TIMESTAMP", #will give integer.
16     "float": "FLOAT",
17     "byte": ["CHAR(1)", "TEXT"], #enum is TEXT, not enum is char(1)

18     "string": "TEXT",
19     "double": "DOUBLE"
20 }
21 #raw is in Hex.
22 def type_of_param(param):
23     """
24     in case we have an enum we want to save in the server the text and not the byte
val.
25     :param param: the dict from the json.
26     :return: type of param that will be written int the SQL.
27     """
28     try:
29         if param["type"] == "byte":
30             # enum need text because we will do the decoding but normal will need 1
byte
31             if "enum" in param.keys(): return types["byte"][1]
32             return types["byte"][0]
33         return types[param["type"]]
34     except KeyError or Exception:
35         print("not have type field or an unknown type. can see in documents the
correct types.")
36         raise TypeError("CantGetParams")
37
38
39 def make_params_for_table(format_file):

```

```

40  """
41  take from the file the json and because we have only on type of subtype in this
program
42  we take all the params and make them fit to add to the creation of the json file.
43  :param format_file: the json file we get from the config
44  :param prime_key: normally the timestamp of the sat, the name of the param
from config
45  :return: string of the params for the creation of the SQL.
46  """
47  params = []
48  try:
49      with open(format_file, 'r') as file:
50          data = json.load(file)
51  except FileNotFoundError or OSError:
52      print("an unknown file. (maybe not in the correct folder. need to be in jsons.")
53      raise TypeError("CantGetParams")
54  try:
55      prime_key = data["settings"]["prime_key"]
56      data = data["subType"]["params"]
57  except KeyError:
58      print("don't have the needed main tags (settings, prime_key, subtype or
params)")
59      raise TypeError("CantGetParams")
60  for param in data: #maybe check if there is a way we will write in little endian

61      try:
62          if param["type"] == "byte" and "enum" in param.keys(): pass #need to add
here something on the enum.
63      except KeyError:
64          print("don't have the main things in the define of the parameter. \n(not
knowing which. need to check were you don't have name or type)")
65          raise TypeError("CantGetParams")
66          #add parameter to the string according to how it will look in the SQL creation.
including primary key.
67          params.append(f'{param["name"]} {type_of_param(param) + " PRIMARY KEY"
if param["name"] == prime_key else type_of_param(param)}')
68          if param["name"] == prime_key and param["type"] != "unixtime": print("know that
the primary key you chose is not time so it may cause problems later")
69  return " NOT NULL,\n".join(params + ["raw TEXT NOT NULL"])
70
71  def create_tables():
72  """
73  going over each sat in config and create the table for who is not already there.
74  """
75  try:
76      with open(r'..\jsons\config.json', 'r') as file:

```

```

77         data = json.load(file)
78     except OSError: print("you don't have the config file or it's not in the right folder.")
79     for x in data:
80         #for each sat in data
81         try: table_name = data[x]["tableName"]
82         except KeyError:
83             print("you don't have table name tag in config.")
84             break
85         try:
86             params = make_params_for_table(rf"..\\jsons\\{data[x]['beacon_json']}")
87         except KeyError or Exception:
88             print("you have a problem in params or don't have a necessary in config.")
89             break
90         sql_query = f"CREATE TABLE IF NOT EXISTS {table_name} ({params});"
91         try:
92             cursor_obj.execute(sql_query)
93             print(f"Table, {table_name}, is Ready")
94         except Exception as e: print(f"had an exception '{e}' in creating table.")
95     connection_sql.commit()
96     connection_sql.close()
97
98     def main():
99         create_tables()
100
101     if __name__ == "__main__":
102         main()
103

```

```

1 __title__ = "DecodeDataInPacket.py"
2 __author__ = 'Maayan'
3 import json, threading, sqlite3, requests, math, os
4 import time, csv, io
5 from struct import unpack
6 from datetime import datetime
7 import pandas as pd
8 from createSQLTable import create_tables
9
10 # I think I make enough checks to make sure it won't be at the same time in
both function
11 connection_sql = sqlite3.connect("../data/SatDatabase.db", check_same_thread=
False)
12 connection_sql.row_factory = sqlite3.Row
13
14 #make the start dictionaries, stuff I want to have at all time, so I won't need to
read the json all the time.
15 def check_make_formula(formula):
16     """
17     take the given format (format is the formula) check it's fine and return the clean
one if possible.
18     :param formula: format of a number (all types: short, int, float etc.)
19     :return: error on a problem
20             the clean formula on success.
21     """
22     formula = formula.replace(" ", "").lower()
23     if formula.count("x") == 0:
24         raise TypeError("Incorrect format, you need to have at lest one x")
25     count_open = 0
26     f = 0
27     while f < len(formula):
28         if not formula[f].isdigit() and formula[f] not in "x()-+%/": #check we don't have
an unknown char
29             raise TypeError("Incorrect format, have invalid chars.")
30         if formula[f] == "(": count_open += 1 #so in the end we can check that all the
parentheses are closed
31         elif formula[f] == ")" and count_open == 0: #make sure we have closer only
when needed
32             raise TypeError("Incorrect format, you have more ) then (")
33         elif formula[f] == ")": count_open -= 1 #same as with the add one when (
34         if f != 0: #because we check before (on f-1)
35             if formula[f] == "(" and formula[f-1] == ".": #not ".("
36                 raise TypeError("Incorrect format, you have dot in before parentheses")
37             elif formula[f] == ")" and formula[f-1] in "*/-+%" : #not "+)" or "* )" etc.

```



```

38         raise TypeError("Incorrect format, the format is not finished")
39     elif formula[f] == "(" and formula[f-1] not in "*/-+%(": #to add * if it possible

40         try: formula = formula[:f] + "*" + formula[f+1:]
41     except IndexError: raise TypeError("Incorrect format, the format is not
finished.") #if we don't have next
42         f += 1
43     f += 1
44     if count_open != 0: #no have open parentheses
45         raise TypeError("Incorrect format, you don't close all the parentheses")
46     if formula[-1] in "*/%(".: #format finished
47         raise TypeError("Incorrect format, the format is not finished.")
48
49     return formula
50     def get_norad_id(satellites_name):
51     """
52     get from the SatNogs website the norad IDs that I will need to get to the satellites.
53     :param satellites_name: a list that have all the satNogs names of the satellites
(that are in the config).
54     :return: list of dicts of satName and his noradID.
55     """
56     norad_id = []
57     url = f"https://db.satnogs.org/api/satellites/?search&format=json"

58     response = requests.get(url) #get all the norad IDs in satNogs

59     data = response.json()
60     for sat in data:
61         if sat['name'] in satellites_name: #find the relevant ones
62             norad_id.append({"satName": sat['name'], "noradId": sat['norad_cat_id']})
63     return norad_id
64
65     def make_dicts_according_to_config():
66     """
67     get all the jsons (config and the ones of format) and put them in a dict, check
them and other.
68     :return: sats, jsons, satIds.
69     sats - dict of dicts, name of sat and a few params, some are from json and
some are from the code.
70     jsons - also dict of dicts, have the format jsons, so I won't have it more than
ones to each json.
71     satIds - list of dicts, what we get from get_norad_id
72     """
73     try:
74         with open(r'..\jsons\config.json', 'r') as file: #open config
75             data = json.load(file)

```

```

76     except OSError:
77         raise TypeError("Can't make it work, You don't have the config file or it's not in
the right folder.")
78     sats = {}
79     jsons = {}
80     sat_ids = []
81     for x in data:
82         #for each sat in data (x is the sat name)
83         try:
84             table_name = data[x]["tableName"] #name of SQL table.
85             callsign = data[x]["callsign"] #callsign of sat
86             format_json = data[x]['beacon_json'] #which file is the format

87             sat_ids.append(data[x]["satnogs_name"]) #here need to put from the chat
the ids.
88             sats[x] = {"table_name": table_name, "callsign": callsign,
89                       "json": format_json, "satnogs": sat_ids[-1], "threadLock":
threading.Lock(),
90                       "cursor": connection_sql.cursor()} #make the dict, also have a
threadLock and cursor for each sat.
91             if format_json not in jsons.keys(): #if we don't already have it

92                 with open(rf'..\jsons\{format_json}', 'r') as file: #try open if error have an
exception.
93                     jsons[format_json] = json.load(file)
94                     try:
95                         setting = jsons[format_json]["settings"] #check we have an opcode.

96                         setting["opcode"] = setting["opcode"]
97                         setting["sizeof_header"] = setting["sizeof_header"]
98                         setting["place_start_opcode"] = setting["place_start_opcode"]
99                     except KeyError: raise TypeError(f"Can't make it work, Don't have
opcode or all the headers in json file ({format_json})")
100                     except Exception as e: raise TypeError(f"Can't make it work, need to
be int here: {format_json}")
101                     try: #make sure we have subType and params, also if you have format
make it to work.
102                         new_params = []
103                         for param in jsons[format_json]["subType"]["params"]:
104                             if "format" in param.keys():
105                                 param["format"] = check_make_formula(param["format"])
106                                 new_params.append(param)
107                         jsons[format_json]["subType"]["params"] = new_params
108                     except KeyError:
109                         raise TypeError(f"Can't make it work, Don't have params or subType
in json file ({format_json})")

```

```

110     except KeyError:
111         raise TypeError("Can't make it work, Don't have all the needed tags in
config.")
112     except OSError:
113         raise TypeError("Can't make it work, You don't have this file in the jsons
folder")
114     try:
115         sat_ids = get_norad_id(sat_ids)
116     except Exception as e:
117         raise TypeError(f"Can't make it work, had a problem: {e}")
118     return sats, jsons, sat_ids
119
120     def jsons_for_html(json_format):
121         """
122         take from the json the needed params and make from them the subSystem it's
from.
123         when we are really entering stuff we check 2 things. 1. if raw, 2. where is it here.
124         :param json_format: all the formats as jsons and needed to change them.
125         :return: jsons but for the html
126         """
127         jsons = {
128         }
129         counter = {} #make sure we don't have more than 10 in subsystem
130
131         for x in json_format: #doing it to each format.
132             jsons[x] = {"primaryKey": json_format[x]["settings"]["prime_key"]} #keep the
primary key
133             parameters = json_format[x]["subType"]["params"] #get all the params to sort.
134
135             are_unix = [] #save all the params that are time. (because I will need it after)
136
137             for param in parameters: # going through the params
138
139                 if param["type"] == "unixtime": are_unix.append(param["name"]) #check if
we are time param and if so add.
140                 if "subSystem" in param.keys(): #check if there is this tag.
141
142                     if param["subSystem"] not in jsons[x]: #add subSystem if not exist
143
144                         jsons[x][param["subSystem"]] = [param["name"]]
145                     elif counter[param["subSystem"]] < 10: #if we have less than 10.
146
147                         jsons[x][param["subSystem"]].append(param["name"])
148                     else: #if more...
149                         #it will do something like that: if you have more than 10 in "EPS" it will
be "EPS 1" etc.

```

```

143         if counter[param["subSystem"]] % 10 == 0: jsons[x][param["subSystem"
] + f" {str(counter[param["subSystem"]] // 10)}"] = [param["name"]]
144         else: jsons[x][param["subSystem"] + f" {str(counter[param["subSystem"
]] // 10)}"].append(param["name"])
145         counter[param["subSystem"]] = counter[param["subSystem"]] + 1 if
param["subSystem"] in counter.keys() else 1
146         else: #if not put in general. those the same but in general

147         if "General" not in jsons[x]:
148             jsons[x]["General"] = [param["name"]]
149         elif counter["General"] < 10:
150             jsons[x]["General"].append(param["name"])
151         else:
152             if counter["General"] % 10 == 0:
153                 jsons[x][f"General {str(counter["General"]) // 10}"] = [param["name"]]
154             else:
155                 jsons[x][f"General {str(counter["General"]) // 10}"].append(param[
"name"])
156             counter["General"] = counter["General"] + 1 if "General" in counter.keys()
else 1
157         jsons[x]["time_param"] = are_unix #because I will need to change it to normal
after.
158     return jsons
159 SATELLITES, JSONS, SatIds = make_dicts_according_to_config()
160     try:
161         JSONS_FOR_HTML = jsons_for_html(JSONS)
162     except KeyError or Exception:
163         print("you have a problem in one of the normal jsons, fix it (it was fine while the
table created first)")
164         raise TypeError("problem in json")
165
166     #used in server
167     def create_options():
168         """
169         create the options for the current html file
170         :return: list of the options.
171         """
172         ret = ""
173         for sat in SATELLITES:
174             ret += f"<option>{sat}</option>\n\t\t\t"
175         return ret
176
177     def get_raw(raw, count):
178         """make the raw data look more like hexDump."""

179     return " ".join([raw[i - 2: i] if i % 16 != 0 else ""].join(f"{raw[i - 2: i]}&nbsp&nbsp&nbsp" if i

```

```

% (16*count) != 0 else f'{raw[i - 2: i]}<br>') for i in range(2, len(raw), 2)] + [raw[len(raw)
- 2:]]
180 # return raw
181
182     def make_for_html(sat_name, last_date, top, limit=0, width=1500):
183         """
184         take from SQL and put it in the format for the html. it's added to what I already
185         have.
186         :param width: the size of the windows when opened
187         :param sat_name: name of sat as written in SATELLITES
188         :param last_date: in time unix. say from where I need to take. (as time come) -
189         type(str) but have int.
190         :param top: is for the top of the page or for the bottom. according to that I know if
191         I gave the min I have or the max
192         :param limit: how many packet I take. if 0 take all.
193         :return: build for html, min_date, max_date <- know what to take according to top
194         or bottom.
195         """
196         #how many in a row.
197         count_raw = math.ceil(width / 300) if width / 300 - math.floor(width) >= 0.8 else
198         math.floor(width / 300) #get how to separate the raw
199         limit = f"LIMIT {limit}" if limit != 0 else ""
200         sat: dict = SATELLITES[sat_name] #get dict of the sat.
201         sql_query = f"SELECT * FROM {sat['table_name']} WHERE
202         {JSONS_FOR_HTML[sat['json']]['primaryKey']} {'>' if top else '<'} {last_date} ORDER
203         BY {JSONS_FOR_HTML[sat['json']]['primaryKey']} DESC {limit}"
204         with sat["threadLock"]: # when I can execute the query.
205
206             sat["cursor"].execute(sql_query)
207             data = sat["cursor"].fetchall()
208             html_code = ""
209             json_sorted = dict(sorted(JSONS_FOR_HTML[sat['json']].items())) #sort
210             according to dictionary place
211             time_params = json_sorted["time_param"] #what are the ones who need change
212             to date.
213             primary = json_sorted["primaryKey"] #who is the primaryKey (need only at the
214             end)
215             del json_sorted["primaryKey"], json_sorted["time_param"] #remove them so I will
216             not look at that.
217             count = count_raw if count_raw < len(json_sorted.keys()) else len
218             (json_sorted.keys()) #get how to separate the subtypes
219
220             width = 100 // count - 3 #count of % of subtype size
221             for row in data: #go over each row that I got from the SQL (have 25 each time)
222
223                 #the start of the full part and the raw container

```

```

209     html_code += f'<div class="containerPacket">\n<div class="divRaw">\n<div
style="text-align: center"><u><b>Hex row:</b></u></div>\n<div class="divHexRaw">
{get_raw(row["raw"].upper(), count_raw)}</div></div>\n'
210     already_did = 0
211     for i in range(1, len(json_sorted) // count + 1): #going the len divided by count

212         html_code += '<div class="container">\n'
213         for k in range(count): #making the count small parts that are in a row (more
will go down)
214             html_code += f'<div class="containerSub" style="width:{width}%">\n<div
class="titleSub">{list(json_sorted.keys())[already_did]}</div>\n<hr />\n'
215             for param_name in json_sorted[list(json_sorted.keys())[already_did]]:
216                 html_code += f'<u>{param_name}</u> {row[param_name] if
param_name not in time_params else
datetime.fromtimestamp(row[param_name]).strftime("%d.%m.%Y %H:%M:%S")}<br /
>\n'
217             html_code += '</div></div>\n'
218             already_did += 1
219             html_code += '</div>\n'
220             # if len is not divided by count, going at the end for the part that is left

221         if len(json_sorted) % count != 0:
222             html_code += '<div class="container">\n'
223             for k in range(len(json_sorted) % count): #how much is left
224                 html_code += f'<div class="containerSub" style="width:{width}%">\n<div
class="titleSub">{list(json_sorted.keys())[k + already_did]}</div>\n<hr />\n'

225             for param_name in json_sorted[list(json_sorted.keys())[k + already_did]]:
226                 html_code += f'<u>{param_name}</u> {row[param_name] if
param_name not in time_params else
datetime.fromtimestamp(row[param_name]).strftime("%d.%m.%Y %H:%M:%S")}<br /
>\n'
227             html_code += '</div></div>\n'
228             html_code += '</div>\n'
229             html_code += '</div>\n'
230         if not data: return "", last_date, last_date #if got nothing
231         return html_code, data[-1][primary], data[0][primary] #else return
232
233     def make_excel(params):
234         """
235         make according to params and SQL an Excel table to send to client.
236         :param params: dict that 100% have type and satName. all the other are
according to type.
237         we can understand how to parse the request according to type.
238         :return: excelTable, file_name

```

```

239     """
240     sql_filter = "" #because it doesn't have to be a where filter and there can be no
filter at all.
241     #get static for the function
242     download_type = params["type"]
243     json_format = SATELLITES[params["satName"]]["json"]
244     primary_key = JSONS[json_format]["settings"]["prime_key"]
245     if download_type == "StartToEndTime": #check between
246         try: int(params["start"]) + int(params["end"])
247         except Exception: raise TypeError("got something that is not int")
248         sql_filter = f"WHERE {params["start"]} < {primary_key} AND {params["end"]} >
{primary_key}"
249     elif download_type == "StartTime": #get from startTime up to current

250         try: int(params["start"])
251         except Exception: raise TypeError("got something that is not int")
252         sql_filter = f"WHERE {params["start"]} < {primary_key}"
253     sql_query = f"SELECT * FROM {SATELLITES[params["satName"]]["table_name"
]} {sql_filter} ORDER BY {primary_key} DESC " #make query
254     if download_type == "Limit": #add limit if have
255         try: int(params["limit"])
256         except Exception: raise TypeError("got something that is not int")
257         sql_query += f"LIMIT {params["limit"]}"
258     with SATELLITES[params["satName"]]["threadLock"]:
259         df = pd.read_sql(sql_query, connection_sql) #read the sql according to pandas

260     for col in JSONS_FOR_HTML[json_format]["time_param"]: #chage format of
unix_time to date-time format
261         if col in df.columns:
262             df[col] = pd.to_datetime(df[col], unit='s')
263             df[col] = df[col].dt.strftime('%d.%m.%Y %H:%M:%S')
264         df["raw"] = df["raw"].str.upper() #big letters
265         output = io.BytesIO() #temporary place
266         file_name = f'{params["satName"]}_{datetime.now().strftime("%d-%m-%Y")}'
#create file name
267         with pd.ExcelWriter(output, engine='openpyxl') as writer: #make the writer with
the .xlsx engine
268             df.to_excel(writer, index=False, sheet_name=file_name[:31]) #incase the
file_name is bigger than possible, create excel

269     return output, file_name
270
271     #satNogs, save to SQL
272 SIZE_IN_BYTES = {
273     "uint": 4,
274     "int": 4,

```



```

275 "ushort": 2,
276 "short": 2,
277 "unixtime": 4, #will give integer.
278 "float": 4,
279 "byte": 1, #enum is TEXT, not enum is char(1)

280 "string": 230, #need to check when we have the first 00, and according to that
we know. that's the max written here.
281 "double": 8
282 }
283
284 def doing_format(data, data_type, format_data):
285     """
286     change the params according to format or enum
287     :param data: what are we doing the format on
288     :param data_type: which type is this data
289     :param format_data: the dict or string that we have there the format.
290     :return: data after format.
291     """
292     # enum but I need string so that's how to make num a char of the num

293     if data_type == "byte": return format_data[chr(ord(data) + 0x30)]
294     # format we perfected before now only using it.

295     if data_type in ["short", "ushort", "uint", "float", "double", "int"]:
296         return eval(format_data, {"x": data})
297     return data
298
299     def decoded_param(data_bytes, data_type, is_big):
300         """
301         with unpack take the hex and make it normal (readable).
302         :param data_bytes: the data in hex
303         :param data_type: which type
304         :param is_big: is in big endian
305         :return: data after changing from hex to data.
306         """
307         if data_type == "string": return data_bytes.decode("utf-8")
308         if data_type == "byte": return unpack(('>' if is_big else '<') + "c", data_bytes)[0]
309         if data_type == "short": return unpack(('>' if is_big else '<') + "h", data_bytes)[0]
310         if data_type == "uint": return unpack(('>' if is_big else '<') + "I", data_bytes)[0]
311         if data_type == "ushort": return unpack(('>' if is_big else '<') + "H", data_bytes)[0]
312         if data_type == "float": return unpack(('>' if is_big else '<') + "f", data_bytes)[0]
313         if data_type == "double": return unpack(('>' if is_big else '<') + "d", data_bytes)[0]
314         if data_type in ["int", "unixtime"]: return unpack(('>' if is_big else '<') + "i",
data_bytes)[0]
315

```



```

316     def decode_data_for_sql(data, json_params, setting):
317         """
318         getting parameters and by them making the params that needed to create the
319         INSERT to the SQL.
320         :param data: Hex that is only from the data section of Ax25
321         :param json_params: according to what we will learn getting the params and the
322         type of each.
323         :param setting: have the setting of the json
324         :return: values for SQL INSERT
325         """
326         i = setting["sizeof_header"] #where in the packet we start. (where the data is)
327
328         decode_to_sql = []
329         for param in json_params:
330             if param["type"] != "string": #if not string, I know the size os take the data there
331
332                 bytes_from_data = data[i: i + SIZE_IN_BYTES[param["type"]]]
333                 i += SIZE_IN_BYTES[param["type"]]
334             else: #try to find the first "\0" after to know where it ends.
335
336                 bytes_from_data = data[i: i + data[i:].index(0x00)]
337                 i += data[i:].index(0x00)
338             if "isBigEndian" in param.keys():
339                 to_append = decoded_param(bytes_from_data, param["type"], param[
340 "isBigEndian"]) #if big send True or False
341             else:
342                 to_append = decoded_param(bytes_from_data, param["type"], setting[
343 "isDefaultBigEndian"]) #send default
344             if param["type"] == "byte" and "enum" in param.keys():
345                 to_append = doing_format(to_append, "byte", param["enum"]) #make the
346                 format of enum in byte, if we have
347             elif "format" in param.keys():
348                 to_append = doing_format(to_append, param["type"], param["format"])
349                 #format to all the other
350             decode_to_sql.append(to_append) #add to the list of variables
351
352         decode_to_sql.append(data.hex()) #also add raw
353         return decode_to_sql
354
355     def check_respond(response):
356         """
357         get the response from the server and needed to check we don't have 429 error
358         or invalid token.
359         :param response: the response from the satNogs.
360         :return: False or response.

```

```

351     """
352     if response.status_code == 404: raise TypeError("Can't read from satNogs.
353     Maybe they change url.")
354     elif response.status_code == 401:
355         if response.json()["detail"] == "Invalid token.":
356             raise TypeError("Invalid Token to satNogs. Please change (explanation in
357             doc)")
358         else:
359             raise TypeError("Non existing Token to satNogs. Please add (explanation in
360             doc)")
361     elif response.status_code == 429:
362         print("read too much, wait a day and try again")
363         time.sleep(24 * 60 * 60) #one day
364         return False
365     return response
366
367     class SatNogsToSQL:
368         # IMPORTANT: satNogs doesn't like when we get too much in a day, so if it's not
369         # a new sat first get the csv and decrypt it.
370         # have how below. (that mean also change the newestDates)
371
372         def __init__(self, newest_dates=None):
373             if not newest_dates: self.newest_dates = {SATELLITES[sat]["satnogs"]}:
374             "2000-01-01T00:00:00+00:00" for sat in SATELLITES} #if didn't get put start.
375
376         else:
377             self.newest_dates = newest_dates
378             for sat in SATELLITES:
379                 if SATELLITES[sat]["satnogs"] not in newest_dates:
380                     self.newest_dates[SATELLITES[sat]["satnogs"]] =
381                     "2000-01-01T00:00:00+00:00"
382             token = '935188971e64257d0736b4f89f575791312226fb'
383
384             self.__headers = {'Authorization': f'Token {token}'}
385             self.run = True
386             self.__write_cursor = connection_sql.cursor()
387
388         def check_packet(self, packet, timestamp):
389             """
390             get packet and check if it's time is bigger, have this callsign and the correct
391             opcode.
392             :param packet: one result of data (a full frame)
393             :param timestamp: current timestamp of this packet
394             :return: true, [sat name and frame (in bytes)] or False ["" or "time", ""]
395             """
396             frame = bytes.fromhex(packet)

```

```

387     src = frame[:6] #place of callsign
388     frame = frame[16:] #all the other packet (including our made headers)

389     callsign = ".join(chr(b >> 1) for b in src[:6]) # according to Ax25 protocol get
callsign
390     if callsign not in [SATELLITES[sat]["callsign"] for sat in SATELLITES]: return
False, ["", ""] #if not in satellites
391     #else get name, name in satnogs
392     sat_name, satnog_name = [(sat, SATELLITES[sat]["satnogs"]) for sat in
SATELLITES if callsign == SATELLITES[sat]["callsign"]][0]
393     #check the time is fine, and we really need this packet, and it's not just
because
394     if datetime.fromisoformat(timestamp.replace("Z", "+00:00")) <
datetime.fromisoformat(self.newest_dates[satnog_name]): return False, ["time", ""]
395     #make sure we have the correct opcode

396     opcode = bytes.fromhex(JSONS[SATELLITES[sat_name]["json"]]["settings"][
"opcode"])
397     place_opcode = JSONS[SATELLITES[sat_name]["json"]]["settings"][
"place_start_opcode"] - 1
398     if frame[place_opcode:place_opcode + len(opcode)] != opcode: return False, [
"", ""]
399     return True, [sat_name, frame]
400
401 def enter_packets(self, packets):
402     """
403     enter packets (a full page) to sql
404     :param packets: data in json from satNogs.
405     :return: maxTime, if we had a problem.
406     """
407     results = packets["results"]
408     ret_val = "fine"
409     for res in results: #going over the packets
410         ret = self.check_packet(res["frame"], res["timestamp"])
411         if not ret[0]: #if error
412             if ret[1][0] != "time": continue
413             else: # and time I need to stop.
414                 ret_val = "time"
415                 break
416         sat_name, packet = ret[1] #get both satName and the frame (that is in
bytes)
417         json_file = JSONS[SATELLITES[sat_name]["json"]] #the dict of the json

418         #get values to enter to SQL
419         values = decode_data_for_sql(packet, json_file["subType"]["params"],
json_file["settings"])

```

```

420         #change to SQL insert values format.
421         sql_query = f"INSERT OR IGNORE INTO {SATELLITES[sat_name][
'table_name']} VALUES ({' '.join([str(x) if type(x) != str else f'"{x}"' for x in values])});"
422         self.__write_cursor.execute(sql_query)
423         return str(datetime.fromisoformat(results[0]["timestamp"].replace("Z", "+00:00"
))), ret_val # need to add the check for the time. when I get there I should stop the loop
and only do what's before. (by if == then)
424
425     def infinite_loop(self):
426         """
427         loop of the reading for satNogs
428         :return: nothing
429         """
430         try:
431             while self.run:
432                 for norad_id in SatIds: #go through each sat that we have
433
434                     url = f"https://db.satnogs.org/api/telemetry/?satellite={norad_id[
'noradId']}"
435                     try:
436                         response = check_respond(requests.get(url, headers=self
.__headers)) #ask for telemetry
437                         while not response: response = check_respond(requests.get(url,
headers=self.__headers))
438                         except ConnectionError: raise TypeError("You aren't connect to the
internet or something like that.")
439
440                         data = response.json()
441                         times, val = self.enter_packets(data) #get time and add to SQL
442
443                         print("hi") #debug
444                         if val == "time": #if time I want to exit
445                             print("time") #debug
446                             self.newest_dates[norad_id["satName"]] = times
447                             connection_sql.commit() # save it in SQL
448                             time.sleep(120) #so the website we read from won't band as. that is
true for all the sleep.
449                             continue
450                             time.sleep(20)
451                             while data["next"]: #if we have another page
452                                 try:
453                                     response = check_respond(requests.get(data["next"], headers=
self.__headers)) #get this one also
454                                     while not response: response = check_respond(requests.get(data[
"next"], headers=self.__headers))
455                                     except ConnectionError: raise TypeError("You aren't connect to the

```

```

internet or something like that.")
454         data = response.json() #do the same as before

455         val = self.enter_packets(data)
456         if val[1] == "time":
457             print("time")
458             break
459         time.sleep(20)
460         self.newest_dates[norad_id["satName"]] = times
461         connection_sql.commit() # save it in SQL
462         print("bye") #debug
463         time.sleep(120)
464         time.sleep(7200) # 2 hours wait.
465     finally:
466         #at end write the most current.
467         connection_sql.commit()
468         with open("../jsons/newestTime.json", "w") as file: file.write(json.dumps(self
.newest_dates))
469
470     def setup(self):
471         try:
472             path = input("Enter path of csv to add (full path or relative): ")
473             while path != "exit":
474                 date_start = "2000-01-01T00:00:00+00:00"
475                 sat_name = input("Enter satellite name according to satNOGS: ")
476                 with open(path, newline="", encoding="utf-8") as f:
477                     reader = csv.reader(f)
478                     count = 0
479                     data = {"results": []}
480                     for row in reader:
481                         row = row[0]
482                         if len(data["results"]) >= 25:
483                             self.enter_packets(data)
484                             data["results"] = []
485                             data_mini = row.split("|")[:2]
486                             data["results"].append(
487                                 {"timestamp": f"{data_mini[0].replace(" ", "T")}}Z", "frame":
data_mini[1]})
488                             if not count:
489                                 date_start = f"{data_mini[0].replace(" ", "T")+00:00"
490                                 count += 1
491                             self.newest_dates[sat_name] = date_start
492                             connection_sql.commit()
493                             path = input("Enter path of csv to add (full path or relative). \nexit to stop: ")
494         finally:

```

```
495         connection_sql.commit()
496         with open("../jsons/newestTime.json", "w") as file:
497             file.write(json.dumps(self.newest_dates))
498
499
500     def main():
501         create_tables()
502         with open("../jsons/newestTime.json", "r") as file:
503             packets_to_sql = SatNogsToSQL(json.load(file))
504         packets_to_sql.setup()
505
506     if __name__ == "__main__":
507         try:
508             main()
509         finally: connection_sql.close()
510
```

```

1 __title__ = "server.py"
2 __author__ = 'Maayan'
3 import http.server, socketserver, ssl, json, threading
4 import DecodeDataInPacket as DDIP
5
6 website_folder = "/webpage"
7
8 class ThreadedHTTPServer(socketserver.ThreadingMixIn,
http.server.HTTPServer):
9     """make the Handle requests in a separate thread."""

10     daemon_threads = True
11
12     class Handler(http.server.SimpleHTTPRequestHandler):
13     def create_send(self, top):
14         """
15         create send for both top and bottom. create the params for the html.
16         :param top: a True or False if it's a request for top or bottom.
17         :return: the html to add, what the oldest got, what the newest got.
18         """
19         #if the code has an error the except id outside.

20         params = self.path.split("?")[-1].split("&") #get the params from the request.

21         params = {param.split("=")[0]: param.split("=")[1].replace("%20", " ") for param
in params} #split according to the protocol
22         # also get and in case of a space add it.

23         sat_name, most_resent = params["satName"], params["mostResent"] if top
else params["leastResent"]
24         limit = 25 if not top else 0
25         html, oldest, newest = DDIP.make_for_html(sat_name, most_resent, top, limit,
int(params["width"])) #according to the function in decode.

26         #make headers.
27         self.send_response(200)
28         self.send_header("Content-type", "text/html")
29         self.end_headers()
30         return html, oldest, newest
31
32     def do_GET(self):
33         """
34         do all the GET requests, it overwrites the function but at the end also used it.
35         :return: nothing
36         """
37         print(f"Received request for: {self.path}")

```

```

38     try:
39         # check request according to protocol
40         if self.path.startswith("/chooseSatellite/"):
41             # think of the create_send just for choose sat (without the start time
because we never got a thing)
42             params = self.path.split("?")[-1].split("&") # get the params from the
request.
43             params = {param.split("=")[0]: param.split("=")[1].replace("%20", " ") for
param in params}
44             sat_name = params["satName"]
45             html, oldest, newest = DDIP.make_for_html(sat_name, 0, True, 25, int
(params["width"]))
46             self.send_response(200)
47             self.send_header("Content-type", "text/html")
48             self.end_headers()
49             # send to json of the data back.
50             self.wfile.write(json.dumps({"mostResent":newest, "leastResent":oldest,
"data":html}).encode("utf-8"))
51             return
52             # protocol chooseSatellite/satName
53         elif self.path.startswith("/addTop/"):
54             html, oldest, newest = self.create_send(True) #is top
55             # send to json of the data back.
56             self.wfile.write(json.dumps({"mostResent":newest, "data":html}).encode(
"utf-8"))
57             return
58             # protocol addTop/?satName={}&mostResent={}

59         elif self.path.startswith("/addBottom/"):
60             html, oldest, newest = self.create_send(False) #isn't top
61             # send to json of the data back.
62             self.wfile.write(json.dumps({"leastResent":oldest, "data":html}).encode(
"utf-8"))
63             return
64             # protocol addBottom/?satName={}&leastResent={}

65         elif self.path.startswith("/downloadData/"):
66             params = self.path.split("?")[-1].split("&")
67             params = {param.split("=")[0]: param.split("=")[1].replace("%20", " ") for
param in params} #get params
68             excel, file_name = DDIP.make_excel(params) #make excel
69             #make the needed headers that with them the client will know he needs
to download it to the computer.
70             self.send_response(200)
71             self.send_header('Content-Type', 'application/vnd.openxmlformats-
officedocument.spreadsheetml.sheet')

```



```

72         self.send_header('Content-Disposition', f'attachment; filename="{
{file_name}.xlsx"}')
73         self.end_headers()
74         self.wfile.write(excel.getvalue()) #send Excel file.
75         excel.close()
76         return
77         #protocol: downloadData/?type={typeOfDownload}&satName={} &limit={}
or &start={} &end={} or &start={} or none.
78     except Exception as e: #in case of an error send error

79         print(f"error {e}")
80         self.send_response(400)
81         self.end_headers()
82         self.wfile.write("had an error".encode())
83
84     # get normal static files. default if it's not other stuff.

85     if self.path == "/" or self.path == "":
86         self.path = f"{website_folder}/hoopoeWorld.html" #have the current sats at
selection and the main page.
87     else: self.path = f"{website_folder}{self.path}"
88     print(f"Changed request for: {self.path}")
89     return super().do_GET()
90
91
92     def https_server():
93         """
94         https server thread. here I create the class and the start of the https
95         :return: None
96         """
97         server_address = ('0.0.0.0', 443)
98         httpd = ThreadedHTTPServer(server_address, Handler) #make it with the class
above.
99         # SSL Setup
100         context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
101         context.load_cert_chain(certfile="Certificate/server.crt", keyfile="Certificate/
server.key") #get the Certificate
102         # Wrap the socket (to be with SSL)
103         httpd.socket = context.wrap_socket(httpd.socket, server_side=True)
104         httpd.serve_forever() #run the http until we can't
105
106     def main():
107     try:
108         DDIP.create_tables() #make sure all the tables are created

109         #Create the selection from the json and template. Used while the server is

```

active reset each run.

```
110     with open(r"webpage\templateWebsite.html", 'r') as file:
111         website = file.read()
112     website = website.replace("<!--Server-->", DDIP.create_options())
113     with open(r"webpage\hoopoeWorld.html", 'w') as file:
114         file.write(website)
115
116     try: #create class with the pass times or without.
117
118         with open("../jsons/newestTime.json", "r") as file:
119             packets_to_sql = DDIP.SatNogsToSQL(json.load(file))
120         except OSError or Exception: packets_to_sql = DDIP.SatNogsToSQL()
121     https_thread = threading.Thread(target=https_server)
122     https_thread.start() #thread of https.
123     packets_to_sql.infinite_loop() #call the loop.
124
125     finally:
126         packets_to_sql.run = False
127         https_thread.join() #close it.
128         DDIP.connection_sql.close()
129         # It doesn't really matter if we have an error here. because the only way to
130         have it's because of other error.
131
132     if __name__ == "__main__":
133         main()
134
```

```

1 __title__ = "serverBackup.py"
2 __author__ = 'Maayan'
3 import ssl, json, threading, socket, os
4 import DecodeDataInPacket as DDIP
5 #without http librarys.
6 threads = []
7 website_folder = "webpage"
8
9 def set_header_type(file_name):
10     """
11     get the Content-Type for files
12     :param file_name: what is the full name (with the .)
13     :return: file type for http
14     """
15     file_ending = file_name[file_name.rindex(".") + 1:]
16     if file_ending == "html":
17         return f"text/html; charset=utf-8"
18     elif file_ending == "txt":
19         return "text/plain"
20     elif file_ending == "js":
21         return "text/javascript; charset=utf-8"
22     elif file_ending == "css":
23         return "text/css"
24     elif file_ending == "xlsx":
25         return "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
26
27     elif file_ending in ["ico", "jpeg", "jpg", "png", "gif"]:
28         return f"image/{file_ending}"
29     return "unknown"
30
31 def get_http_get_msg(sock) -> tuple:
32     """
33     get the request from the client (mostly the url)
34     :param sock: the client socket.
35     :return: (if it is a get request, path/url)
36     """
37     data = b""
38     while b"\r\n\r\n" not in data:
39         data_to_add = sock.recv(1024)
40         if data_to_add != b"":
41             data += data_to_add
42     data = data.decode()
43     if "GET" in data:
44         return True, data.split(" ")[1]
45     elif "POST" in data:

```

```

46     return False, ""
47 return False, ""
48
49 class Handler(threading.Thread):
50 def __init__(self, path, sock, is_get):
51     """
52     get the super from thread and also params about the request.
53     :param path: what was the path (the url)
54     :param sock: client socket already wrap with ssl.
55     :param is_get: check if there is get (if not return bad format because I don't
have post)
56     """
57     super().__init__()
58     self.path = path
59     self.__sock = sock
60     self.__header = {
61         "Content-Type": "text" #default
62     }
63     self.code = 200
64     self.__is_get = is_get
65
66 def set_headers(self, header, data):
67     """
68     add header with info to the dict. also no length (added on send)
69     :param header: name of the header (ex. Content-Type)
70     :param data: type, info. (ex. text/html)
71     :return: none, change param from init
72     """
73     if header == "Content-Length": return
74     self.__header[header] = data
75
76 def headers_send(self) -> str:
77     """
78     make the headers that were added in the format of http
79     :return: str, headerName: info\r\n ...
80     """
81     return "\r\n".join(f"{key}: {self.__header[key]}" for key in self.__header.keys())
82
83 def create_scroll(self, top:bool) -> tuple:
84     """
85     create send for both top and bottom. create the params for the html.
86     :param top: a True or False if it's a request for top or bottom.
87     :return: the html to add, what the oldest got, what the newest got.
88     """
89     # if the code has an error the except is outside.

```

```

90     params = self.path.split("?")[-1].split("&") # get the params from the request.

91     params = {param.split("=")[0]: param.split("=")[1].replace("%20", " ") for param
in params} # split according to the protocol
92     # also get and in case of a space add it.

93     sat_name, most_resent = params["satName"], params["mostResent"] if top
else params["leastResent"]
94     limit = 25 if not top else 0
95     html, oldest, newest = DDIP.make_for_html(sat_name, most_resent, top, limit,
int(params["width"])) # according to the function in decode.

96     # make headers.
97     self.set_headers("Content-Type", "application/json")
98     self.code = 200
99     return html, oldest, newest
100
101 def send(self, data=b''):
102     """
103     sending the response for the request from the client.
104     header and code takes from init. Also deal with errors.
105     :param data: the data that is going to be in the body
106     :return: none
107     """
108     data_to_send = b''
109     if self.code == 200:
110         data_to_send += f"HTTP/1.1 200 OK\r\n{self.headers_send()}\r\nContent-
Length: {len(data)}\r\n\r\n".encode() + data
111     elif self.code == 404:
112         data_to_send += b"HTTP/1.1 404 Not Found"
113     elif self.code == 400:
114         data_to_send += b"HTTP/1.1 400 Bad Request"

115     print(f"sent >> {data_to_send[:150]}")
116     self.__sock.send(data_to_send)
117
118 def run(self):
119     """
120     override the run of thread, but have the full client logic
121     :return: none
122     """
123     print(f"Received request for: {self.path}")
124     if not self.__is_get:
125         self.code = 400
126         self.send()
127     try:

```

```

128         # check request according to protocol
129         if self.path.startswith("/chooseSatellite/"):
130             # think of the create_send just for choose sat (without the start time
because we never got a thing)
131             params = self.path.split("?")[-1].split("&") # get the params from the
request.
132             params = {param.split("=")[0]: param.split("=")[1].replace("%20", " ") for
param in params}
133             sat_name = params["satName"]
134             html, oldest, newest = DDIP.make_for_html(sat_name, 0, True, 25, int
(params["width"]))
135
136             self.code = 200
137             self.set_headers("Content-Type", "application/json")
138             # send to json of the data back.
139             self.send(json.dumps({"mostResent":newest, "leastResent":oldest, "data"
:html}).encode("utf-8"))
140             return
141             # protocol chooseSatellite/satName
142         elif self.path.startswith("/addTop/"):
143             html, oldest, newest = self.create_scroll(True) #is top
144             # send to json of the data back.
145             self.send(json.dumps({"mostResent":newest, "data":html}).encode("utf-8"
))
146             return
147             # protocol addTop/?satName={}&mostResent={}

148         elif self.path.startswith("/addBottom/"):
149             html, oldest, newest = self.create_scroll(False) #isn't top
150             # send to json of the data back.
151             self.send(json.dumps({"leastResent":oldest, "data":html}).encode("utf-8"))
152             return
153             # protocol addBottom/?satName={}&leastResent={}

154         elif self.path.startswith("/downloadData/"):
155             params = self.path.split("?")[-1].split("&")
156             params = {param.split("=")[0]: param.split("=")[1].replace("%20", " ") for
param in params} #get params
157             excel, file_name = DDIP.make_excel(params) #make excel
158             #make the needed headers that with them the client will know he needs
to download it to the computer.
159             self.code = 200
160             self.set_headers('Content-Type', 'application/vnd.openxmlformats-
officedocument.spreadsheetml.sheet')
161             self.set_headers('Content-Disposition', f'attachment; filename="
{file_name}.xlsx"')

```

```

162         self.send(excel.getvalue()) #send Excel file.
163         excel.close()
164         return
165         #protocol: downloadData/?type={typeOfDownload}&satName={}&limit={}
or &start={}&end={} or &start={} or none.
166     else:
167         if self.path == "/" or self.path == "":
168             self.path = f"{website_folder}/hoopoeWorld.html" #have the current
sats at selection and the main page.
169         else: self.path = f"{website_folder}{self.path}"
170         print(f"Changed request for: {self.path}")
171         if os.path.exists(self.path):
172             self.set_headers("Content-Type", set_header_type(self.path))
173             with open(self.path, 'rb') as f:
174                 self.send(f.read())
175         else:
176             self.code = 404
177             self.send()
178     except Exception as e: #in case of an error send error

179         print(f"error {e}")
180         self.code = 400
181         self.send()
182     finally:
183         self.__sock.close()
184
185     def https_server():
186         """
187         https server thread. here I create the class and the start of the https
188         :return: None
189         """
190         server_address = ('0.0.0.0', 443)
191         # SSL Setup
192         context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
193         context.load_cert_chain(certfile="Certificate/server.crt", keyfile="Certificate/
server.key") #get the Certificate
194
195         server_socket = socket.socket()
196         server_socket.bind(server_address) # Using 4433 for testing

197         server_socket.listen(20)
198         global threads
199         try:
200             while True:
201                 cli_sock, addr = server_socket.accept()
202                 connstream = context.wrap_socket(cli_sock, server_side=True)

```

```

203
204     is_get, path = get_http_get_msg(connstream)
205     t = Handler(path=path, sock=connstream, is_get=is_get)
206     t.start()
207     threads.append(t)
208
209     # removes finished threads from the list
210
211     for k in threads:
212         if not k.is_alive(): k.join()
213     threads = [t for t in threads if t.is_alive()]
214 finally:
215     server_socket.close()
216
217 def main():
218     try:
219         DDIP.create_tables() #make sure all the tables are created
220
221         #Create the selection from the json and template. Used while the server is
222         active reset each run.
223         with open(r"webpage\templateWebsite.html", 'r') as file:
224             website = file.read()
225             website = website.replace("<!--Server-->", DDIP.create_options())
226         with open(r"webpage\hoopoeWorld.html", 'w') as file:
227             file.write(website)
228
229         try: #create class with the pass times or without.
230
231             with open("../jsons/newestTime.json", "r") as file:
232                 packets_to_sql = DDIP.SatNogsToSQL(json.load(file))
233             except OSError or Exception: packets_to_sql = DDIP.SatNogsToSQL()
234             https_thread = threading.Thread(target=https_server)
235             https_thread.start() #thread of https.
236             packets_to_sql.infinite_loop() #call the loop.
237
238         finally:
239             packets_to_sql.run = False
240             https_thread.join() #close it.
241             DDIP.connection_sql.close()
242             # It doesn't really matter if we have an error here. because the only way to
243             have it's because of other error.
244
245         if __name__ == "__main__":

```



```
244  main()
245
```

config_format.json

```
1 {
2   "{satName}": {
3     "callsign": "{sat_callsign}",
4     "id": "{dec number, id from protocol}",
5     "tableName": "{what the sql table will be called}",
6     "satnogs_name": "{how the sat is called in satnogs}",
7     "beacon_json": "{the json of the data/beacon format}"
8   },
9   "{satName2} can add as much as you will like": {
10     "callsign": "{sat_callsign}",
11     "id": "{dec number, id from protocol}",
12     "tableName": "{what the sql table will be called}",
13     "satnogs_name": "{how the sat is called in satnogs}",
14     "beacon_json": "{the json of the data/beacon format}"
15   }
16 }
17
```

config.json

```
1 {
2   "tevel 11": {
3     "callsign": "TLV2-1",
4     "satnogs_name": "TEVEL2-1",
5     "id": "11",
6     "tableName": "Tevel11Data",
7     "beacon_json": "tevels_beacon.json"
8   },
9   "tevel 14": {
10    "callsign": "TVL2-4",
11    "satnogs_name": "TEVEL2-4",
12    "id": "14",
13    "tableName": "Tevel14Data",
14    "beacon_json": "tevels_beacon.json"
15  },
16  "tevel 16": {
17    "callsign": "TVL2-6",
18    "satnogs_name": "TEVEL2-6",
19    "id": "16",
20    "tableName": "Tevel16Data",
21    "beacon_json": "tevels_beacon.json"
22  },
23  "tevel 19": {
24    "callsign": "TVL2-9",
25    "satnogs_name": "TEVEL2-9",
26    "id": "19",
27    "tableName": "Tevel19Data",
28    "beacon_json": "tevels_beacon.json"
29  }
30 }
31
```

data_format.json

```

1 {
2   "settings": {
3     "isDefaultBigEndian": false,
4     "opcode": "{opcode in hex (type + subtype)",
5     "sizeof_header": "{full size of header. ITS an int (not a string like now})",
6     "place_start_opcode": "{counter start at 1. ITS an int (not a string like now})",
7     "prime_key": "which param is the primary key"
8   },
9   "subType": {
10    "name": "{name of function}",
11    "params": [
12      {
13        "name": "{name of param}",
14        "type": "{type of param}",
15        "subSystem": "{if we have a system it is about} - from here included not a must",
16
17        "enum": "{if the type is byte and want to display enum}",
18        "format": "{in case we have changes for the param (x). can do +, -, *, /, ** (for
19        power})",
20        "isBigEndian": false
21      },
22      {
23        "name": "{second param, going according to the order in packet} - and can do
24        more params",
25        "type": "{param type}"
26      }
27    ]
28  }
29 }

```

tevels_beacon.json


```
1 {
2   "settings": {
3     "isDefaultBigEndian": false,
4     "opcode": "0001",
5     "sizeof_header": 8,
6     "place_start_opcode": 5,
7     "prime_key": "sat_time"
8   },
9   "subType": {
10    "name": "beacon",
11    "params": [
12      {
13        "name": "voltBattery",
14        "type": "short",
15        "subSystem": "EPS"
16      },
17      {
18        "name": "volt_5V",
19        "type": "short",
20        "subSystem": "EPS"
21      },
22      {
23        "name": "volt_3V3",
24        "type": "short",
25        "subSystem": "EPS"
26      },
27      {
28        "name": "charging_power",
29        "type": "short",
30        "subSystem": "EPS"
31      },
32      {
33        "name": "consumed_power",
34        "type": "short",
35        "subSystem": "EPS"
36      },
37      {
38        "name": "electric_current",
39        "type": "short",
40        "subSystem": "EPS"
41      },
42      {
43        "name": "current_3V3",
44        "type": "short",
45        "subSystem": "EPS"
46      },

```

```
47 {
48   "name": "current_5V",
49   "type": "short",
50   "subSystem": "EPS"
51 },
52 {
53   "name": "bat_temp",
54   "type": "short",
55   "subSystem": "EPS"
56 },
57 {
58   "name": "solar_panels0",
59   "type": "float",
60   "subSystem": "Solar Panels"
61 },
62 {
63   "name": "solar_panels1",
64   "type": "float",
65   "subSystem": "Solar Panels"
66 },
67 {
68   "name": "solar_panels2",
69   "type": "float",
70   "subSystem": "Solar Panels"
71 },
72 {
73   "name": "solar_panels3",
74   "type": "float",
75   "subSystem": "Solar Panels"
76 },
77 {
78   "name": "solar_panels4",
79   "type": "float",
80   "subSystem": "Solar Panels"
81 },
82 {
83   "name": "solar_panels5",
84   "type": "float",
85   "subSystem": "Solar Panels"
86 },
87 {
88   "name": "sat_time",
89   "type": "unixtime",
90   "subSystem": "OBC"
91 },
92 {
```

```
93     "name": "free_memory",
94     "type": "uint",
95     "subSystem": "OBC"
96 },
97 {
98     "name": "corrupt_bytes",
99     "type": "uint",
100     "subSystem": "OBC"
101 },
102 {
103     "name": "total_memory",
104     "type": "uint",
105     "subSystem": "OBC"
106 },
107 {
108     "name": "used_bytes",
109     "type": "uint",
110     "subSystem": "OBC"
111 },
112 {
113     "name": "lastFS_error",
114     "type": "int",
115     "subSystem": "OBC"
116 },
117 {
118     "name": "number_of_resets",
119     "type": "uint",
120     "subSystem": "OBC"
121 },
122 {
123     "name": "sat_uptime",
124     "type": "uint",
125     "subSystem": "OBC"
126 },
127 {
128     "name": "num_of_cmd_resets",
129     "type": "uint",
130     "subSystem": "OBC"
131 },
132 {
133     "name": "sel_counter",
134     "type": "int",
135     "subSystem": "Payload"
136 },
137 {
138     "name": "seu_counter",
```

```
139     "type": "int",
140     "subSystem": "Payload"
141 },
142 {
143     "name": "sensor_one_radfet",
144     "type": "int",
145     "subSystem": "Payload"
146 },
147 {
148     "name": "sensor_two_radfet",
149     "type": "int",
150     "subSystem": "Payload"
151 },
152 {
153     "name": "radfet_temp",
154     "type": "double",
155     "subSystem": "Payload"
156 },
157 {
158     "name": "last_radfet_read_time",
159     "type": "unixtime",
160     "subSystem": "Payload"
161 },
162 {
163     "name": "power_payload",
164     "type": "short",
165     "subSystem": "Payload"
166 },
167 {
168     "name": "payload_kill_flag",
169     "type": "uint",
170     "subSystem": "Payload"
171 },
172 {
173     "name": "eps_state",
174     "type": "byte",
175     "enum": {
176         "0": "operational",
177         "1": "cruise",
178         "2": "powerSafeMode"
179     },
180     "subSystem": "EPS"
181 },
182 {
183     "name": "change_in_mode",
184     "type": "int",
```

```
185     "subSystem": "Payload"
186   },
187   {
188     "name": "temp_pa",
189     "type": "float",
190     "subSystem": "TRXVU"
191   },
192   {
193     "name": "temp_board",
194     "type": "float",
195     "subSystem": "TRXVU"
196   },
197   {
198     "name": "reflected_power",
199     "type": "float",
200     "subSystem": "TRXVU"
201   },
202   {
203     "name": "forward_power",
204     "type": "float",
205     "subSystem": "TRXVU"
206   },
207   {
208     "name": "doppler",
209     "type": "float",
210     "subSystem": "TRXVU"
211   },
212   {
213     "name": "rssi",
214     "type": "float",
215     "subSystem": "TRXVU"
216   }
217 ]
218 }
219 }
220
```

```

1 // name of file: clock.js
2 setInterval(myTime, 1000);
3
4 function myTime() {
5   var d = new Date();
6
7   // Use toUTCString() for the date and time
8   var date =
9     d.toUTCString().split(" ")[0] +
10    " " +
11    d.toUTCString().split(" ")[1] +
12    " " +
13    d.toUTCString().split(" ")[2] +
14    " " +
15    d.toUTCString().split(" ")[3];
16   var time = d.toUTCString().split(" ")[4]; // Just the time part
17
18   var finalTime = date + "</br>" + time + " UTC";
19   let clock = document.getElementById("clock");
20   clock.innerHTML = finalTime;
21   let width = window.innerWidth;
22   if (width < 1300) {
23     clock.style.fontSize = `${Math.round(width / 60)}px`;
24   } else {
25     clock.style.fontSize = `25px`;
26   }
27 }
28

```

```

1 //name of file: dynamicFunctions.js
2 let satName = "Choose satellite";
3 let newestTime = 0;
4 let oldestTime = 0;
5 let isLoading = false;
6 const TWO_HOURS = 2 * 60 * 60 * 1000; //in millisecond.
7 const WINDOW_WIDTH = window.innerWidth;
8
9 /**
10 * onchange function that when we change a satellite it will ask for the most current
data of this sat.
11 * also change the global variables accordingly, and open and close the option to
download as excel.
12 * @param {string} choice - which sat or default
13 * @returns null
14 */
15 async function ChooseSat(choice) {
16 if (isLoading) return;
17 isLoading = true; //so we will not go down or up and at the same time change. (will
be bad)
18 choice = choice.value;
19 let placeholder = document.getElementById("packets"); //where at the end the
packets will go to
20 let download = document.getElementById("openDownload"); //for the option to
download
21 if (choice == "Choose satellite") {
22 //if none of the satellites are chosen
23 placeholder.innerHTML = "";
24 download.innerHTML = "";
25 return 0;
26 }
27 const url = `/chooseSatellite/?satName=${choice}&width=${WINDOW_WIDTH}`;
28 try {
29 const response = await fetch(url); //try to get data from server
30 if (!response.ok) {
31 throw new Error("HTTP error! status: " + response.status);
32 }
33 const data = await response.json(); //get json from that.
34 //put global params
35 satName = choice;
36 newestTime = data.mostResent;
37 oldestTime = data.leastResent;
38 placeholder.innerHTML = data.data; //place the packets in place
39 //add the option for download

```

```

40 download.innerHTML = `<button class="popupButton" onclick="OpenPopup()"></
button><div class="popup" id="popup"><div class="closeBtn"
onclick="ClosePopup()">&times;</div><div><div id="title" class="title">Download data
for ${satName}</div><p>Choose type of download:<select id="chooseDownload"
class="item selectDownloadType" onchange="DownloadType(this)"><option
value="StartToEndTime">Start to End Time</option><option value="StartTime">Start
Time</option><option value="Limit">Limit</option><option value="All"> All </option></
select></p><div id="types"><p>Enter start date: <input type="datetime-local" id="start-
date" name="start-date" step="1" /></p><p>Enter end date: <input type="datetime-
local" id="end-date" name="end-date" step="1" /><br /></p></div><button
type="submit" onclick="SendDownloadRequest()">download</button><div
class="error" id="error"></div></div></div>`;
41 } catch (error) {
42     console.error("Error fetching data: ", error, message);
43 }
44 isLoading = false;
45 }
46
47 /**
48  * according to choice add the correct filter.
49  * @param {string} choice - choice of which download filter to use.
50  */
51 function DownloadType(choice) {
52     choice = choice.value;
53     let placeholder = document.getElementById("types");
54     switch (choice) {
55         case "StartToEndTime":
56             placeholder.innerHTML =
57                 '<p>Enter start date: <input type="datetime-local" id="start-date" name="start-
date" step="1" /></p><p>Enter end date: <input type="datetime-local" id="end-date"
name="end-date" step="1" /></p>';
58             break;
59         case "StartTime":
60             placeholder.innerHTML =
61                 '<p>Enter start date: <input type="datetime-local" id="start-date" name="start-
date" step="1" /></p>';
62             break;
63         case "Limit":
64             placeholder.innerHTML =
65                 '<p>Amount of packets: <input type="number" id="limit" name="limit"
placeholder="amount of packets" min="1"/></p>';
66             break;
67         case "All":
68             placeholder.innerHTML = "";
69             break;
70     }

```



```

71 }
72
73 /**
74 * check if we didn't enter stuff.
75 * @param {string} element
76 * @returns true on empty false on full.
77 */
78 function CheckNull(element) {
79 if (element == "") {
80 return true;
81 }
82 return false;
83 }
84
85 /**
86 * both open and close but add to the button. (close just in case they don't want to
go up in the website).
87 */
88 function OpenPopup() {
89 let popup = document.getElementById("popup");
90 popup.classList.toggle("open-popup"); //if have remove else add.
91 }
92 /**
93 * close the popup. (for the x button)
94 */
95 function ClosePopup() {
96 let popup = document.getElementById("popup");
97 popup.classList.remove("open-popup");
98 }
99
100 /**
101 * check if the form is valid and if not write an error msg.
102 * @returns false on error, true otherwise.
103 */
104 function CheckForm() {
105 const choice = document.getElementById("chooseDownload").value;
106 let errorPlace = document.getElementById("error");
107 errorPlace.innerHTML = "";
108 switch (choice) {
109 case "StartToEndTime":
110 var startTime = document.getElementById("start-date").value;
111 var endTime = document.getElementById("end-date").value;
112 if (CheckNull(startTime) || CheckNull(endTime)) {
113 errorPlace.innerHTML =
114 "Error: don't have at least one time. Need to add";

```

```

115     return false;
116 }
117 if (startTime > endTime) {
118     errorPlace.innerHTML = "Error: start time is bigger then end time";
119     return false;
120 }
121 if (startTime < "1970-01-01T00:00" || endTime < "1970-01-01T00:00") {
122     //because of time_unix
123     errorPlace.innerHTML = "Error: time need to be after 1.1.1970 00:00";
124     return false;
125 }
126 break;
127 case "StartTime":
128     var startTime = document.getElementById("start-date").value;
129     if (CheckNull(startTime)) {
130         errorPlace.innerHTML =
131             "Error: don't have at least start time. Need to add";
132         return false;
133     }
134     if (startTime < "1970-01-01T00:00") {
135         //because of time_unix
136         errorPlace.innerHTML = "Error: time need to be after 1.1.1970 00:00";
137         return false;
138     }
139     break;
140 case "Limit":
141     var limit = document.getElementById("limit").value;
142     if (CheckNull(limit)) {
143         errorPlace.innerHTML = "Error: don't have limit. Need to add";
144         return false;
145     }
146     break;
147 case "All":
148     break;
149 default:
150     return false; //a not known place.
151 }
152 return true;
153 }
154
155 /**
156 * send and get the request for download .xlsx
157 * @returns null
158 */
159 async function SendDownloadRequest() {
160     if (!CheckForm()) return; //if not valid

```

```

161 const choice = document.getElementById("chooseDownload").value;
162 let url = `/downloadData/?type=${choice}&satName=${satName}`; //the start of
the url that is right to all types.
163 switch (choice) {
164     case "StartToEndTime": {
165         const start = Math.floor(
166             document.getElementById("start-date").valueAsNumber / 1000,
167         ); //I know for a fact it's not null, and time
168         const end = Math.floor(
169             document.getElementById("end-date").valueAsNumber / 1000,
170         );
171         url += `&start=${start}&end=${end}`; //add the other params according to
protocol.
172         break;
173     }
174     case "StartTime": {
175         const start = Math.floor(
176             document.getElementById("start-date").valueAsNumber / 1000,
177         );
178         url += `&start=${start}`;
179         break;
180     }
181     case "Limit": {
182         const limit = document.getElementById("limit").valueAsNumber; //get limit as a
number
183         url += `&limit=${limit}`; ///add the param according to protocol.

184         break;
185     }
186     case "All":
187         break;
188 }
189 window.location.href = url; //get from server the file by the url.

190 }
191
192 /**
193  * make the website look like it has an infinite scrolling down.
194  * this is what it send.
195  * @returns null
196  */
197 async function ScrollDown() {
198     if (satName == "Choose satellite") return;
199     if (isLoading) return; //same check as chooseSat
200     isLoading = true;
201     const container = document.getElementById("packets");

```

```

202 const url = `/addBottom/?satName=${satName}&leastResent=${oldestTime}
&width=${WINDOW_WIDTH}`; //to get the 25 packets that are just after the oldest I
have.
203 try {
204   const response = await fetch(url);
205   if (!response.ok) {
206     //check for an error
207     throw new Error("HTTP error! status: " + response.status);
208   }
209   const data = await response.json();
210   oldestTime = data.leastResent; //change the oldest presented time

211   container.insertAdjacentHTML("beforeend", data.data); //add to the end of the div

212 } catch (error) {
213   console.error("Error fetching data: ", error.message);
214 }
215 isLoading = false;
216 }
217
218 /**
219 * every two hours this function is called to make sure we have current data.
220 * this is what it send.
221 * @returns null
222 */
223 async function ScrollUp() {
224   const container = document.getElementById("packets");
225   if (satName == "Choose satellite") return;
226   const url = `/addTop/?satName=${satName}&mostResent=${newestTime}&width=
${WINDOW_WIDTH}`; //to get the packets that are just before what I have.

227   try {
228     const response = await fetch(url);
229     if (!response.ok) {
230       throw new Error("HTTP error! status: " + response.status);
231     }
232     const data = await response.json();
233     if (data.data == "") return; //in case there is none
234     newestTime = data.mostResent; //change the newest presented time

235     container.insertAdjacentHTML("afterbegin", data.data); //add to the start of the
div
236   } catch (error) {
237     console.error("Error fetching data: ", error.message);
238   }
239 }

```

```
240     /**
241     * do the infinite scrolling.
242     */
243     window.addEventListener("scroll", () => {
244     const { scrollTop, scrollHeight, clientHeight } = document.documentElement;
245     if (scrollTop + clientHeight >= scrollHeight - 20) {
246     ScrollDown();
247     }
248 });
249     setInterval(ScrollUp, TWO_HOURS); //up every two hours
250
```

```
1  /*name of file: HooppoeWorldStyle.css*/
2
3  body {
4  background-image: url("Images/5.png");
5  display: flow-root;
6  }
7
8  /*a flex container*/
9  .container {
10 width: 100%;
11 display: flex;
12 }
13
14 .item {
15 width: 20%;
16 height: 60px;
17 text-align: center;
18 }
19
20 .header {
21 background-color: midnightblue;
22 color: azure;
23 position: fixed;
24 height: 70px;
25 top: 0;
26 left: 0;
27 }
28
29 /*the select satellite options*/
30 .selectOption {
31 font-family: "Times New Roman";
32 color: black;
33 background-color: aliceblue;
34 height: 50px;
35 border-radius: 8px;
36 align-self: center;
37 margin: 10px;
38 padding: 5px;
39 font-size: x-large;
40 box-shadow: 0px 8px 16px 0px rgba(0, 0, 0, 0.2);
41 }
42
43 /*for the imgs*/
44 .icon {
45 align-self: center;
46 }
```

```
47
48     .title {
49     flex-grow: 1;
50     font-size: 40px;
51     align-self: center;
52     color: aliceblue;
53     text-align: center;
54     font-family: "Levenim MT";
55     font-style: normal;
56 }
57
58     /*display clock*/
59     .clock {
60     align-content: center;
61     color: white;
62     font-family: "Times New Roman";
63     height: 50px;
64     align-self: center;
65     margin: 10px;
66     padding: 5px;
67     font-size: 25px;
68 }
69
70     /*the container of each packet (have raw and subs inside)*/
71
72     .containerPacket {
73     font-family: "Times New Roman";
74     color: black;
75     background-color: aliceblue;
76     width: 90%;
77     border-radius: 8px;
78     margin-top: 1%;
79     margin-left: 5%;
80     padding: 5px;
81     font-size: x-large;
82     box-shadow: 0px 8px 16px 0px rgba(0, 0, 0, 0.2);
83 }
84     /*the box that have the raw values.*/
85     .divRaw {
86     margin: 1%;
87     background-color: white;
88     font-family: "Times New Roman";
89     color: black;
90     align-content: center;
91     align-self: center;
```

```
92 font-size: x-large;
93 overflow-wrap: break-word;
94 box-shadow: 0px 8px 16px 0px rgba(0, 0, 0, 0.2);
95 }
96
97 .divHexRaw {
98 margin-left: 2%;
99 font-family: "consolas";
100 font-size: large;
101 }
102
103 /*the small white box for each subsystem*/
104
105 .containerSub {
106 font-family: "Times New Roman";
107 color: black;
108 background-color: white;
109 margin: 1%;
110 padding: 5px;
111 font-size: large;
112 box-shadow: 0px 8px 16px 0px rgba(0, 0, 0, 0.2);
113 }
114
115 .titleSub {
116 text-align: center;
117 font-weight: 600;
118 }
119
120 /*popup button*/
121 .popupButton {
122 position: fixed;
123 bottom: 2%;
124 right: 1%;
125 border-radius: 50%;
126 box-shadow: 0px 8px 16px 0px rgb(72, 142, 252, 0.4);
127 height: 50px;
128 width: 50px;
129 align-items: center;
130 background-image: url("Images/downloadImg.svg");
131 background-size: contain;
132 cursor: pointer;
133 border-color: azure;
134 transition:
135   transform 0.2s ease,
136   box-shadow 0.2s ease;
```



```
137     .popupButton:hover {
138   transform: translateY(-10px);
139   box-shadow: 0px 12px 20px 5px rgb(72, 142, 252, 0.4);
140 }
141     .popupButton:active {
142   transform: scale(0.95);
143 }
144
145   /*popup window*/
146   .popup {
147   position: fixed;
148   top: 0;
149   left: 50%;
150   transform: translate(-50%, -50%) scale(0.1);
151   justify-content: center;
152   background: rgba(52, 66, 105, 0.6);
153   border-radius: 10px;
154   padding: 10px;
155   padding-top: 5px;
156   width: 40%;
157   text-align: center;
158   backdrop-filter: blur(10px);
159   box-shadow:
160     11px 8px 17px 7px rgba(0, 0, 0, 0.4),
161     inset 0px -2px 17px 7px rgb(83 83 83 / 27%);
162   visibility: hidden;
163   transition:
164     transform 0.4s,
165     top 0.4s;
166   color: #d1faff;
167   font-size: x-large;
168   font-style: oblique;
169 }
170   .open-popup {
171   visibility: visible;
172   top: 50%;
173   transform: translate(-50%, -50%) scale(1);
174 }
175
176   .closeBtn {
177   color: aliceblue;
178   font-size: x-large;
179   text-align: right;
180   cursor: pointer;
181 }
182
```

```
183     /*submit*/
184     button[type="submit"] {
185     background-color: #99bcf9;
186     color: #001850;
187     padding: 7px 10px;
188     border: none;
189     border-radius: 6px;
190     text-align: center;
191     font-family: cursive;
192     font-size: 15px;
193     cursor: pointer;
194     box-shadow: 4px 5px 16px 0px rgb(0 0 0 / 73%);
195 }
196
197     button[type="submit"]:active {
198     background-color: #99c3ff;
199     color: #b8ffcf;
200 }
201
202     button[type="submit"]:hover {
203     background-color: #001850;
204     color: #99bcf9;
205 }
206
207     input[type="datetime-local"] {
208     cursor: text;
209 }
210
211     .selectDownloadType {
212     font-family: "Times New Roman";
213     color: black;
214     background-color: aliceblue;
215     height: 30px;
216     border-radius: 8px;
217     align-self: center;
218     margin: 10px;
219     padding: 5px;
220     font-size: medium;
221     box-shadow: 0px 8px 16px 0px rgba(0, 0, 0, 0.2);
222     width: fit-content;
223 }
224
225     .error {
226     color: #ff8181;
227     font-weight: 700;
228     font-size: large;
```

```
229 margin-top: 10px;  
230 }  
231
```

```

1 py--name of file: templateWebsite.html-->
2 <!DOCTYPE html>
3 <html lang="en" xmlns="http://www.w3.org/1999/xhtml">
4 <head>
5 <meta charset="utf-8" />
6 <title>HoopoeWorld</title>
7 <link rel="icon" type="image/x-icon" href="Images\favicon.png?v=2" />
8 <link rel="stylesheet" href="HoopoeWorldStyle.css" />
9 </head>
10 <body style="background-color: black">
11 <script src="clock.js"></script>
12 <script src="dynamicFunctions.js"></script>
13 <div class="container header">
14 <select
15     id="chooseSat"
16     class="item selectOption"
17     onchange="ChooseSat(this)"
18 >
19 <option>Choose satellite</option>
20 <!--Server-->
21 </select>
22 <div class="container title">
23 
29 <div class="icon title">HoopoeWorld</div>
30 
35 <!--making mirror effect-->
36 </div>
37 <div class="item title clock" id="clock"></div>
38 </div>
39 <div style="height: 70px"></div>
40 <div id="packets">
41 <!--data from server-->
42 </div>
43 <div id="openDownload">
44 <!--only when we have a sat.-->
45 </div>

```

```
46 </body>
47 </html>
48
```